MBToolkit v3.5 Technical Code Documentation

Architecture, runtime flows, every callable, regression tests, build automation, and complete source-code screenshots

Documented package

`MBToolkit_v3.5_Package.zip`

Source snapshot SHA-256

`fd7be65c7b05cee311a3b5e8f09b5a6626f4e4b72da1e1af831ec891d7d94e1c`

Documentation build date

6 November 2026

Verification

132/132 regression tests passed before documentation generation.

This document describes the exact source snapshot above. It does not document proposed future features that are not present in this package.

The final appendix reproduces every Python and Windows batch source line as code screenshots. The explanatory chapters refer to the same line numbers.

Contents

1. Documented source snapshot 3
2. Version history and changes in this version 7
3. Architecture 8
4. Passive capture pipeline 10
5. Frame model and parser behavior 11
6. Session metrics and Bus Health 13
7. Active Bus Slave Finder 15
8. Master (poll) tab 18
9. Slave (simulator) tab 22
10. Modbus TCP: Client and Server tabs 25
11. UI, language and theme design 28
12. Persistence, logs and export 29
13. Module-by-module reference 30
14. Regression-test suite 146
15. Build and launch automation 158
16. Known boundaries and extension points 159
17. Appendix A - Complete source screenshots 160

1. Documented source snapshot

This reference was generated from the current MBToolkit v3.5 package, without modifying the application source.

Python source lines	Documented Python + BAT lines
19,433	19,839

Package structure

```
MBToolkit_v3.5_Package.zip
├── tools/  (documentation and package builders)
├── MBToolkit_v3.5_Technical_Code_Documentation.pdf  (this document)
├── MBToolkit.bat
├── README.md
├── README.en.md
├── README.txt
└── MBToolkit v3.5/
    ├── MBToolkit.py
    ├── mb_config.py
    ├── mb_protocol.py
    ├── mb_capture.py
    ├── mb_diagnostics.py
    ├── mb_view.py
    ├── mb_slave_finder.py
    ├── mb_master.py
    ├── mb_master_ui.py
    ├── mb_slave.py
    ├── mb_slave_ui.py
    ├── mb_table_ui.py
    ├── mb_tcp.py
    ├── mb_tcp_ui.py
    ├── mb_i18n.py
    ├── mb_theme.py
    ├── mb_widgets.py
    ├── mb_gui.py
    ├── tests/test_regression.py
    ├── tests/test_master.py
    ├── tests/test_slave.py
    ├── tests/test_tcp.py
    ├── run_tests.bat
    ├── build_exe.bat
    ├── MBToolkit.ico
    ├── DEVELOPMENT_NOTES.md
    └── CHANGELOG.md
```

Source inventory

Python module	Lines	Primary responsibility
`MBToolkit.py`	29	Application launcher and top-level entry point.
`mb_config.py`	637	Central configuration, constants, user settings validation and filesystem paths.
`mb_protocol.py`	1007	Protocol engine for passive Modbus RTU parsing and session metrics.
`mb_capture.py`	913	Serial-port management and passive acquisition pipeline.
`mb_diagnostics.py`	286	Presentation-oriented diagnostics derived from captured records.
`mb_view.py`	1881	Traffic view, filtering, sorting, Frame Inspector, Bus Health updates, CSV export and TXT logging.
`mb_slave_finder.py`	1625	Active Modbus RTU Slave discovery and Device Identification.
`mb_master.py`	695	Modbus RTU master engine, frames and data formats.
`mb_master_ui.py`	1479	Master (poll) tab user interface.
`mb_slave.py`	629	Modbus RTU slave (simulator) engine.
`mb_slave_ui.py`	1668	Slave (simulator) tab user interface.
`mb_table_ui.py`	792	Window-grid code shared by the Master and Slave tabs.
`mb_tcp.py`	500	Modbus TCP transport.
`mb_tcp_ui.py`	509	Modbus TCP Client and Server tabs.
`mb_i18n.py`	729	Runtime Portuguese/English translation layer.
`mb_theme.py`	692	Fixed-geometry Light/Dark theme implementation.
`mb_widgets.py`	418	Custom reusable Tkinter widgets.
`mb_gui.py`	2748	Main application composition and global UI behavior.
`test_regression.py`	787	Automated regression suite.
`test_master.py`	602	Master regression tests.
`test_slave.py`	537	Slave regression tests.
`test_tcp.py`	270	Modbus TCP regression tests.

Integrity hashes

Source file	Lines	SHA-256
MBToolkit v3.5/MBToolkit.py	29	fe17c0be094d1cdd767f0d3a6502b4ea544e6d12f935bc4f641f9f8c589d4546
MBToolkit v3.5/mb_config.py	637	a3c9cc0389a00f648a913dadbcd1b4ab27c40c5fbbcbc2db9453238a83e45658
MBToolkit v3.5/mb_protocol.py	1007	8c4c41051e41f0c998692206ed0204d91f057cce5fce726b15ad2cce503c1ef2
MBToolkit v3.5/mb_capture.py	913	d05166ac79d8b546d7527aea97be87a516d17b81291d278da36c928933406f33
MBToolkit v3.5/mb_diagnostics.py	286	c67ed1df18f1d79b0feb4011a036f92cbeef338929a529a4f218acd1f78e1552
MBToolkit v3.5/mb_view.py	1881	fac08668363fb06679c8d9166b95709debc078b34c9acc3a6790a11b18e5d27d
MBToolkit v3.5/mb_slave_finder.py	1625	f192a6305486ea73411d920d9b6b9de19091bc612afef60ea3f5871fcd66938b
MBToolkit v3.5/mb_master.py	695	e4e956940449cfcf8caa059ca550184af0298d32f72729c7f5376c0ec431604f
MBToolkit v3.5/mb_master_ui.py	1479	2d1cf37767f636f6f31ea8d39dddf1364992c180513bdd09ebb1e797f82b6922
MBToolkit v3.5/mb_slave.py	629	42500c79110ebe2f27dfd06bad81999320db72c0625d0280067d78e55c5a8cee
MBToolkit v3.5/mb_slave_ui.py	1668	847fab008254fcb0cf989f66dc9d3dd2a1f1c1dd6e58f3da89ef869ffde6fa38
MBToolkit v3.5/mb_table_ui.py	792	6ac1533958761ed7180fc03670d019f0d605c2e62dfc8edfc7e05371b8b08500
MBToolkit v3.5/mb_tcp.py	500	53c21c5603cf6a3664c2b5e5bbb0f55abe0f9440341b1f3ff568a200b5fff4e6
MBToolkit v3.5/mb_tcp_ui.py	509	1ad3419cca195d5339496327086e44eaa7aed452d556bb86a41a25ff6c56a34a
MBToolkit v3.5/mb_i18n.py	729	75061bc71561f3895deb33bf3d789d3bcfa8401f6b4350bb8a09d8ed65d67f82
MBToolkit v3.5/mb_theme.py	692	7b03e5a4f4cc646a96cd1839d2a92794ea923d232ae7e50a93b5908ac4ab8d9e
MBToolkit v3.5/mb_widgets.py	418	3a732639b6813ae827290b8ca5af567158b2ee736e941c89ae3d77ea7a184d2f
MBToolkit v3.5/mb_gui.py	2748	56a2d86cdc6fb7d298e149b733ab84c3ad473bd7b6dcd3033a6a477cd51fd28f

Source file	Lines	SHA-256
`MBToolkit v3.5/tests/test_regression.py`	787	`b5a3f7e75c0b385f8cb874fdf3b16c8e867108c59406078d966f333db4dba70c`
`MBToolkit v3.5/tests/test_master.py`	602	`a01432c6e8951bf1deff66832ffb92e4187fbaf2923d8a0d1e32442a787fda9d`
`MBToolkit v3.5/tests/test_slave.py`	537	`4139e1e5cba1c5d97a51bb1bd8f4a8a026218392d387ac8de265283c7bbcbe05`
`MBToolkit v3.5/tests/test_tcp.py`	270	`08101c93368627aa7ca8da028ff16852f4a4498ec371352c4cd48b3d2fd8c50b`
`MBToolkit.bat`	210	`4dcfe23fa45a1b97e136be2db75b4fb4763cb0db3cfae46edb4464cf19b4f52e`
`MBToolkit v3.5/build_exe.bat`	151	`5d1b64d6be1e91673938109b0d8495caac3e256065f8d891160f423302b2679c`
`MBToolkit v3.5/run_tests.bat`	45	`98c20bdbf89f4f478c4b38b7c8f5a55fae834f328edaf7d2979c8823f2236eae`

> **Regression status.** The supplied suite was executed directly from this source tree. All 132 tests passed.

2. Version history and changes in this version

Release history of the documented code base. The detailed list below applies to v3.5, the version described by every other chapter of this document.

Version	Date	Summary
v3.0	21 September 2026	Bilingual PT-PT / English interface, FC43/14 Device Identification, regression suite and first Technical Code Documentation.
v3.1	23 October 2026	Broadcast/echo-write pairing fixes, USB split-frame reassembly, in-place timeout marking, Finder attempts and idle-bus check, address-mapped read data, table-driven CRC and centralised translations.
v3.2	23 October 2026	Master (poll) tab with one window per Slave / data area, cyclic reads, writes, 16/32-bit data formats and per-window counters; the release ZIP now also carries this PDF and the documentation tools.
v3.3	23 October 2026	Renamed from MBSniffer to MBToolkit, with a new high-resolution icon and automatic migration of preferences and workspaces.
v3.4	23 October 2026	Slave (simulator) tab: several simulated Slave IDs and data tables answering a real master, with fault simulation per Slave ID; window grid code shared with the Master.
v3.5	6 November 2026	Modbus TCP Client and Server tabs: the Master and Slave windows over the network, running alongside the serial tabs.

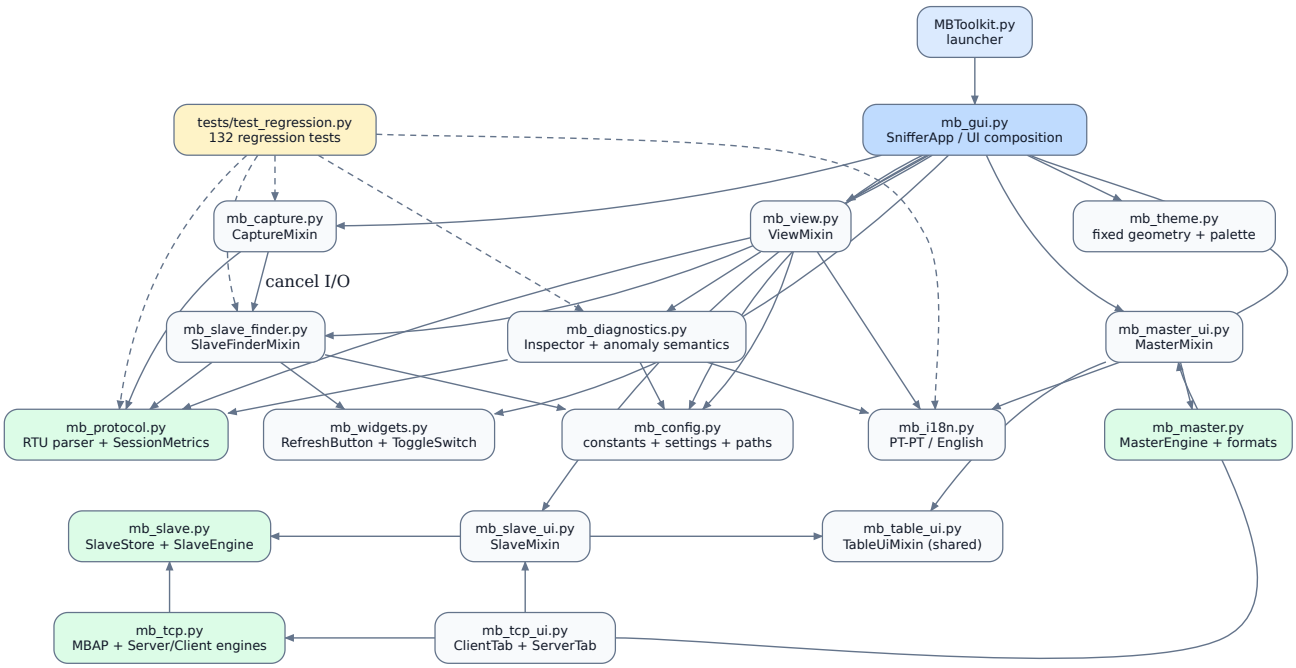
Changes in v3.5

v3.5 adds Modbus TCP in two tabs of their own, **Client** and **Server**. See chapter "Modbus TCP: Client and Server tabs".

- **New modules:** `mb_tcp.py` (MBAP framing, `ServerEngine`, `ClientEngine`, `list_ipv4_interfaces()`; no widgets) and `mb_tcp_ui.py` (`ClientTab`, `ServerTab`).
- **No duplicated Modbus logic:** a TCP request becomes the equivalent RTU frame and is served by `handle_request()`; the Client builds requests with `mb_master` and matches responses with `find_master_response()`.
- **No duplicated tab code:** the Client and Server are second instances of `MasterMixin` and `SlaveMixin` on `TabInstance`; the mixins gained class attributes and connection methods for the TCP tabs to override. `MasterEngine` gained `event_prefix` and `transact_for()`.
- **Independent of the serial lock:** Client and Server run alongside the Sniffer, the Finder, the Master and the Slave.
- **Server listening address:** this PC only by default, one network interface (with IP and adapter name) or all interfaces, with a confirmation outside 127.0.0.1.
- Settings `client` / `server`, workspaces in `Workspaces\Client` and `Workspaces\Server`.
- The regression suite grew to 132 tests (`tests/test_tcp.py`: Client and Server over 127.0.0.1).

3. Architecture

MBToolkit uses one Tkinter root object with five behavior mixins, plus two tab objects for Modbus TCP. Protocol parsing and metrics are separated from the GUI, while the active Finder, Master, Slave, Client and Server have their own worker logic. This is a pragmatic modular architecture rather than a strict MVC/MVVM framework.



Module dependency and responsibility overview.

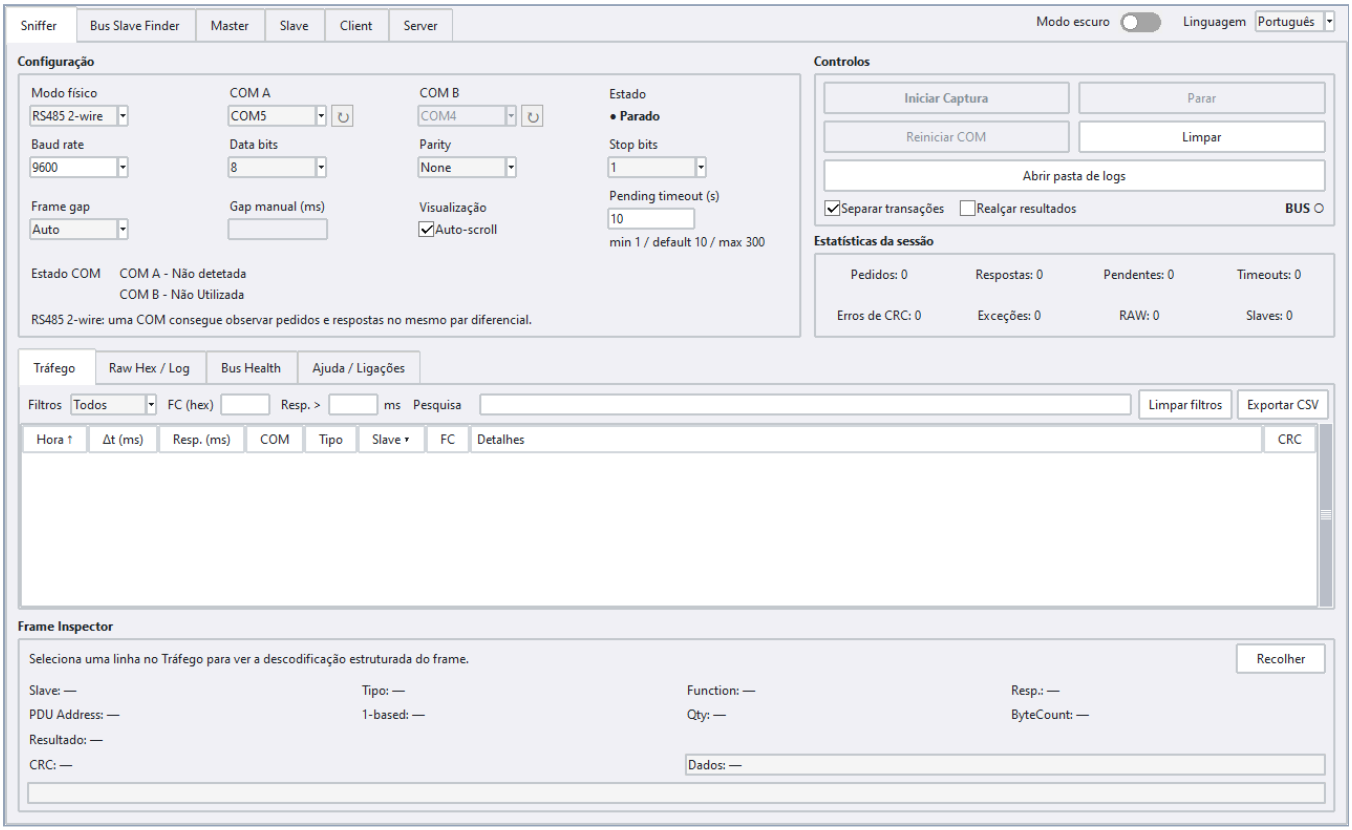
Composition of SnifferApp

`SnifferApp(MasterMixin, SlaveMixin, SlaveFinderMixin, CaptureMixin, ViewMixin, tk.Tk)` composes the application. `mb_gui.py` constructs and coordinates the UI; the five mixins supply the Master tab, the Slave tab, active discovery, passive capture, and presentation/logging behavior. `MasterMixin` and `SlaveMixin` both inherit `TableUiMixin` from `mb_table_ui.py`, which holds their common window-grid widgets. The Modbus TCP tabs are `app.client_tab` and `app.server_tab`: instances of `ClientTab(MasterMixin, TabInstance)` and `ServerTab(SlaveMixin, TabInstance)` that keep their own state and look up everything else on the application.

Important architectural boundaries

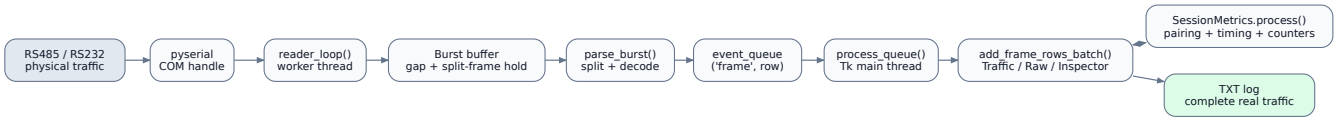
- **Protocol vs UI:** CRC, frame recognition, parsing and SessionMetrics are in `mb_protocol.py` and have no Tkinter dependency.
- **Workers vs Tk:** serial reader/discovery threads communicate through `event_queue`; Tk widgets are updated from the main thread.
- **Source history vs presentation:** filters and sorting rebuild the view from `_frame_history`; they do not modify the raw session record or statistics.

- **Passive vs active:** the Sniffer only reads. Bus Slave Finder intentionally transmits requests and is protected by a safety confirmation/global busy state. The Master and the Slave transmit too and share the same busy state: only one of Sniffer, Finder, Master and Slave runs at a time.
- **Appearance vs geometry:** Light/Dark changes colors; language changes text; both are designed not to rebuild widget geometry.



Current Sniffer UI generated from the documented source snapshot.

4. Passive capture pipeline



From physical bytes to UI records and disk logging.

1. **Port selection.** `selected_port_plan()` maps RS485/RS232 mode to one or two receive channels.
2. **Serial configuration.** `serial_config()` translates Baud/Data bits/Parity/Stop bits into pyserial constants.
3. **Gap calculation.** Automatic mode uses the shared Modbus t3.5 helper; manual mode validates the user-entered millisecond value.
4. **Reader workers.** One daemon thread per active receive channel reads bytes, records the arrival time of every read and closes a burst after the configured silence. If the burst ends with the start of an unfinished known frame, the reader waits up to `SNIFFER_SPLIT_FRAME_HOLD_MS` for the rest (USB latency-timer splits).
5. **Parsing.** Each burst is split/resynchronised and decoded by `parse_burst()`; each frame keeps the arrival time of the read holding its first byte.
6. **Thread boundary.** Parsed rows are placed in `event_queue`.
7. **Main-thread processing.** `process_queue()` drains events in batches and passes frame batches to `add_frame_rows_batch()`.
8. **Metrics/view/log.** The view runs SessionMetrics, updates Traffic/Raw/Inspector/Bus Health and writes complete real traffic to the lazy TXT log.

> **Threading rule.** The serial threads do not call Tkinter methods. The queue is the explicit synchronization boundary between workers and the GUI thread.

5. Frame model and parser behavior

RTU CRC and frame timing

`crc16_modbus()` implements the Modbus CRC-16 algorithm with a 256-entry lookup table (`_CRC16_TABLE`). `append_crc()` appends low byte then high byte. `modbus_rtu_interframe_gap_seconds()` provides the common t3.5 timing used by Sniffer and Finder.

Recognised frame shapes

`_candidate_prefixes_at()` recognises structural lengths for FC01/02/03/04, FC05/06/08, FC07, FC15/16, FC22, FC23 and standard Exception frames. Unknown Function Codes are deliberately handled conservatively.

Resynchronisation

The parser first tries a recognised, CRC-valid frame at offset zero. If that fails, it searches later offsets but only using recognised frame shapes. This is the performance and false-positive safeguard introduced after the earlier brute-force CRC scan.

- Known-shape prefix before a recovered valid frame + bad CRC → `CRC ERROR`.
- Unstructured bytes before a recovered frame → `RAW/UNSYNC`.
- Known-shape complete remainder + bad CRC → `CRC ERROR`.
- Unsupported-FC complete remainder with valid CRC → `UNKNOWN / OK`.
- Otherwise → `RAW/UNPARSED`.

Bursts split by USB adapters

`burst_may_continue()` re-runs the splitter on the burst and inspects only the final unvalidated segment: if it starts with a plausible Slave ID and a known Function Code but is shorter than every candidate length, more bytes are expected. The reader re-checks only when the buffer length changes, so a long noise burst is not re-parsed on every poll.

Read-response bits

FC01/FC02 responses are decoded least-significant bit first by `read_bits_from_response()`. `read_bits_details()` groups the bits per data byte, shows at most 32 of them in Traffic and trims the padding of the last byte once the matching request quantity is known.

Parser row schema

Field	Meaning
`time`	Display timestamp string.
`channel`	BUS, A→B, B→A or SIM source.
`type`	REQUEST, RESPONSE, EXCEPTION, RAW/UNSYNC, RAW/UNPARSED, UNKNOWN or REQ/RESP (FC05/06/08/22, resolved to REQUEST/RESPONSE by SessionMetrics).
`slave`, `fc`	Decoded address and hexadecimal Function Code when available.
`details`	Human-readable protocol details.
`crc`	OK, ERROR or ?.
`raw`	Original frame bytes rendered as hexadecimal.
`pdu_address`, `one_based`, `qty`	Addressing/quantity metadata when decoded.
`_frame_len`	Number of observed bytes represented by the row.
`_epoch`	Floating timestamp used for timing/pairing: arrival time of the read holding the frame's first byte.

6. Session metrics and Bus Health

`SessionMetrics` is stateful per capture/session. It records counters, pending requests, response samples and diagnostic aggregates before GUI filters are applied.

Request/response pairing

A request key is `(Slave ID, base Function Code)`. Valid REQUEST rows are queued FIFO per key. RESPONSE/EXCEPTION rows remove the oldest matching pending request and compute response time from the two epoch timestamps.

> **Pairing limitation.** Modbus RTU has no transaction identifier. The implementation therefore uses FIFO matching by Slave and Function Code. If a non-standard master pipelines multiple same-Slave/same-FC requests without waiting for responses, pairing cannot be proven uniquely from RTU headers alone.

Echo-shaped functions

FC05, FC06, FC08 and FC22 requests and their normal responses have the same length, so the parser emits `REQ/RESP`. `_resolve_echo_direction()` classifies the frame before accounting: RESPONSE when an older pending request with the same key exists (identical bytes for the exact-echo FCs 05/06/22; opposite channel in RS232 dual RX), otherwise REQUEST.

> **Echo limitation.** On a single RS485 channel, if a master repeats the same write to a Slave that never answers, the repetition is indistinguishable from an echo response and is paired as one.

Broadcast

Requests to Slave 0 are counted as requests and in `broadcast_requests`, but are never pending: by definition they receive no response and must not become timeouts.

Timeouts

Pending requests older than the configured Pending timeout are expired. The timeout is attached to the original REQUEST record through its sequence ID; MBToolkit does not invent a fake timeout frame. The view retags only the affected rows instead of rebuilding the table.

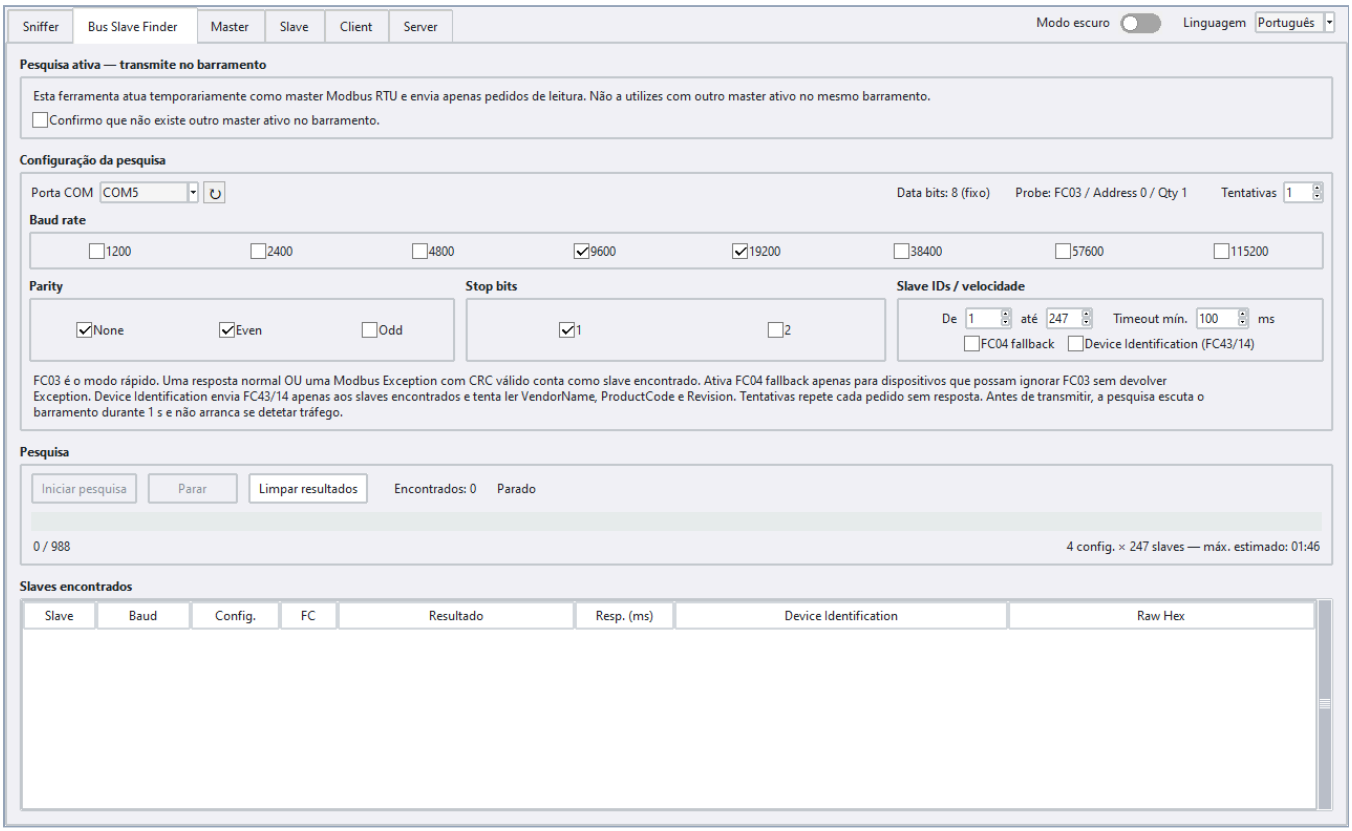
Bus Health calculations

- **Request rate:** requests divided by observed elapsed session time.
- **Response time:** average, minimum, maximum and nearest-rank P95 from valid paired responses.

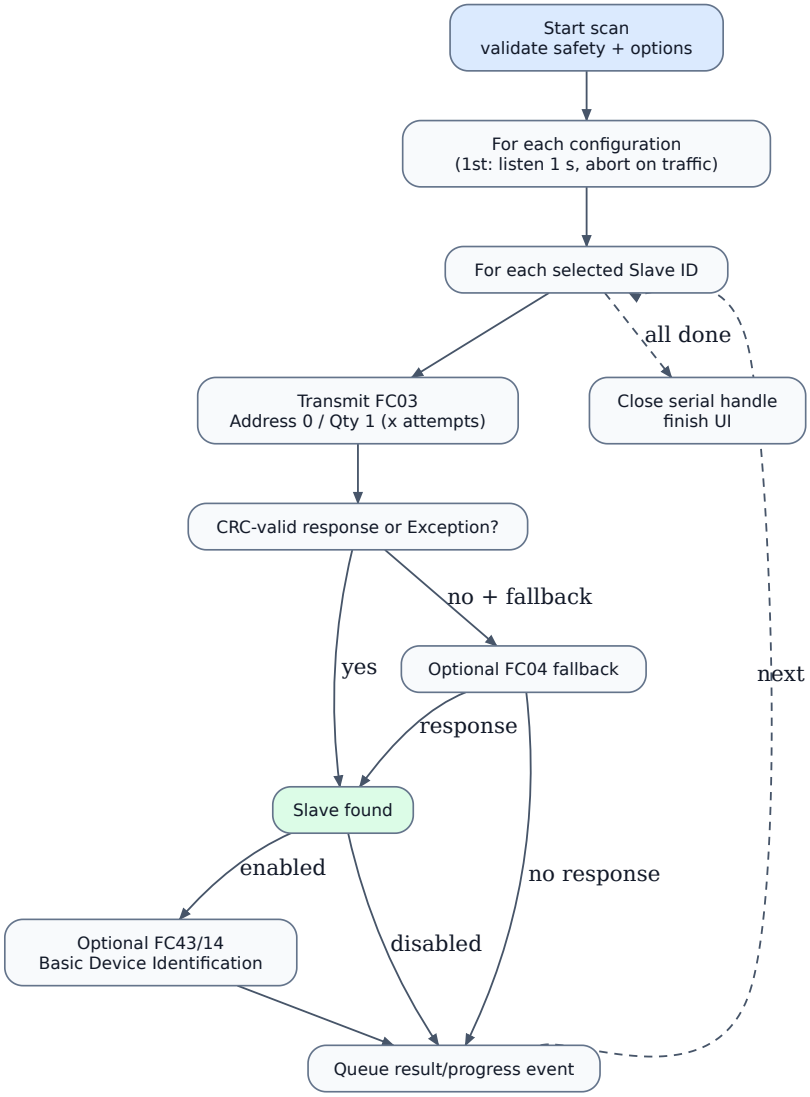
- **Slowest Slave:** highest average response time among sampled Slaves.
- **Unvalidated bytes:** bytes in explicit CRC-error frames plus RAW blocks divided by all observed bytes.
- **Timeout rate:** expired requests / observed non-broadcast requests.
- **Approximate bus utilisation:** observed bytes × configured serial bits per character / (baud × elapsed time).
- **Exceptions / Slave:** counts of Exception responses grouped by Slave ID.

> **Physical-layer boundary.** Bus utilisation and unvalidated-byte percentage are software/capture indicators. The program does not measure analog RS485 voltage, bias, reflections, ringing, common-mode level or rise/fall times.

7. Active Bus Slave Finder



Current active Finder UI generated from the documented source.



Discovery flow for each selected configuration and Slave ID.

Discovery semantics

The standard probe is FC03, Address 0, Quantity 1. A CRC-valid normal response proves the Slave exists. A CRC-valid Modbus Exception also proves the Slave exists, because the addressed device had to receive and interpret the request to generate that Exception.

FC04 fallback is optional and is attempted only after FC03 produces no accepted response. **Attempts** (1–3) repeats each unanswered probe before moving on. Device Identification is also optional and is performed only after a Slave has already been found.

Idle-bus check

Before the first transmission of a scan, `bus_activity_detected()` listens for `DISCOVERY_IDLE_CHECK_SECONDS` on the first configuration. Any byte, even one framed at another baud rate, means something else is transmitting: the worker raises an error naming the port and the first bytes, and nothing is sent. This complements the safety checkbox.

FC43/14 Device Identification

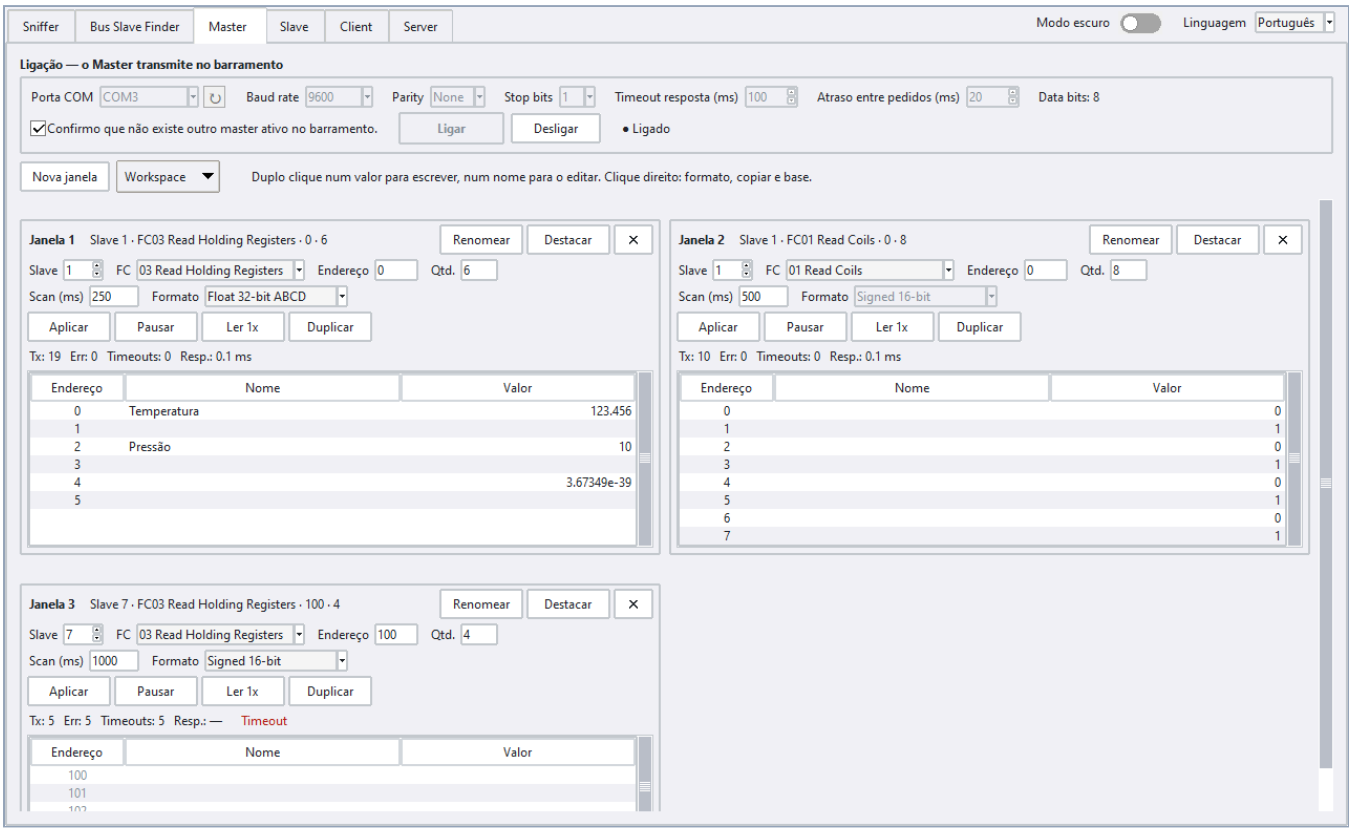
The request uses Function Code 0x2B, MEI type 0x0E and Basic Device Identification read code 0x01. The parser reads variable-length objects, validates CRC and follows `More Follows` pages up to the configured page cap. Display output focuses on object 0x00 VendorName, 0x01 ProductCode and 0x02 MajorMinorRevision.

Driver/error handling

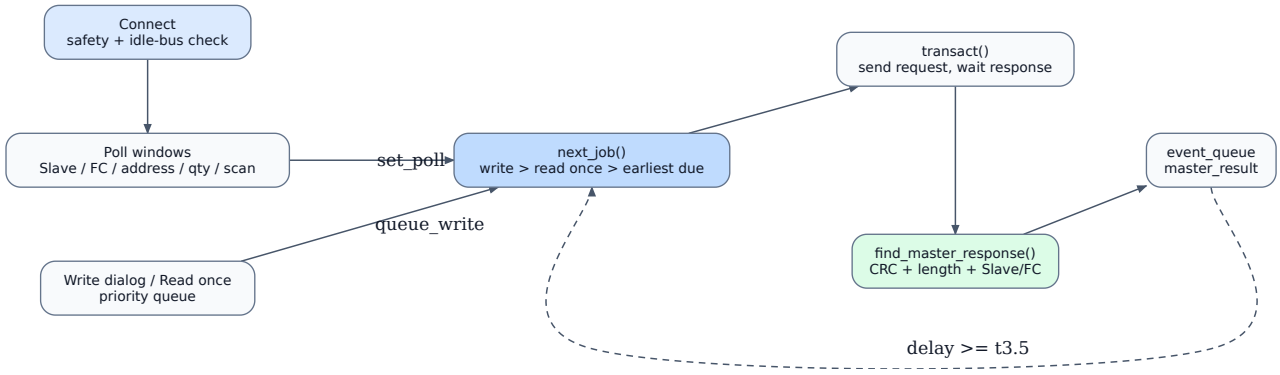
A serial write/read failure is treated as a communication/driver error, not as an ordinary "Slave did not respond" result. Write timeout recovery can cancel/clear driver buffers and retry once. Stop requests use `threading.Event` and best-effort cancel_read/cancel_write support.

> **Operational safety.** This tab actively transmits Modbus RTU requests. It must not be run on a bus that already has another active master unless the system is explicitly designed for multi-master behavior.

8. Master (poll) tab



Master tab connected to the in-memory test slave of tests/test_master.py.



Master engine: one request at a time over one serial port.

The Master tab reads and writes values as a Modbus RTU master, in the style of Modbus Poll. Each **poll window** describes one data area of one Slave; the state of a window lives in a `MasterWindow` object, independent of where its pane is shown.

Scheduling on one bus

Modbus RTU allows a single request on the bus at a time, so one `MasterEngine` thread serves every window over one serial port. Its `next_job()` returns, in this order: a queued write, a queued "Read once", or the enabled window whose next due time is earliest. A window becomes due again `scan_ms` after its previous request started (0 polls continuously). After every transaction the engine leaves the configured

delay between polls on the bus, never less than Modbus t3.5. Response time and the response timeout count from the end of the transmitted request.

Frames and responses

`build_read_request()` and `build_write_request()` build FC01–04 and FC05/06/15/16 frames within the Modbus limits (2000 bits / 125 registers per read, 1968 coils / 123 registers per write). `find_master_response()` scans the received bytes for the CRC-valid frame of the expected length from the addressed Slave, or a 5-byte exception. Read requests echoed by an RS485 adapter are skipped because their length and CRC do not fit. Write responses must echo the request (FC05/06) or its address/quantity (FC15/16). Broadcast writes (Slave 0) are not answered and are reported as sent.

Data formats

Format	Meaning
Signed / Unsigned 16-bit	One register as a two's-complement or unsigned integer.
Hex / Binary 16-bit	One register as `0xHHHH` or two binary bytes.
Signed / Unsigned / Float 32-bit	Two registers. Register n carries bytes A B and register n+1 bytes C D; the order names the value's bytes: ABCD big-endian, CDAB word swap, BADC byte swap, DCBA little-endian. The value is shown on the first register's row.
Signed / Unsigned 64-bit, Double	Four registers (bytes A B, C D, E F, G H): ABCDEFGH big-endian, GHEFCDAB registers in reverse order, BADCFEHG byte swap, HGFEDCBA little-endian. Doubles show up to 15 significant digits.

Per-address formats

The window's **Format** is the default. `MasterWindow.formats` maps the PDU address where a value starts to its own format, chosen in the right-click menu for the selected rows (drag with the mouse, or Ctrl/Shift, for several; ttk has no rubber-band selection, so `TableUiMixin._table_drag_start/move/stop` implement it, with auto-scroll past the table edges). `apply_format_to_rows()` applies 16-bit formats to every selected row and 32/64-bit formats to consecutive rows in multiples of 2/4, removing overlapping per-address formats; "Reset to window format" removes them. `register_groups()` splits the window into values; a default multi-register value cut by a per-address format, or running past the window, shows "—". The same menu copies the selected rows (also Ctrl+C) as tab-separated text and sets the address base (0 = PDU Address, 1 = manual numbering).

Scaling

`MasterWindow.scales` maps the PDU address where a value starts to a scale built by `validate_scale()`: two raw/engineering points, 0–6 decimals and a unit. `format_window_values()` shows scaled values as `e1 + (raw − r1) × (e2 − e1) / (r2 − r1)` with the unit; Hex and Binary stay raw. When writing, the typed engineering value is converted back with `raw_text_for_scaled()` (rounded for integer formats).

Open in Master

`master_definition_from_record()` takes Slave, function, address and quantity from a Traffic REQUEST or from the request matched to a RESPONSE/EXCEPTION (FC01–FC04 only). `master_open_definition()` creates the window, shows the Master tab and, when the Master is not connected, copies the port and serial parameters. The Bus Slave Finder opens a found Slave the same way from its result row (FC03/FC04, address 0, 10 registers, the baud/parity/stop it answered on).

Reconnection

The engine receives a `reopen` callable. A serial or driver failure during the session closes the port, posts `master_reconnecting` and retries every `MASTER_RECONNECT_INTERVAL_SECONDS` until the port opens again (`master_reconnected`) or the user disconnects; the values turn grey in the meantime.

Writing

Double-clicking a value in an FC01 or FC03 window opens the write dialog. `master_build_write()` validates the Slave (0–247), the address in the window's base and the values, encodes them with the format of the clicked value and builds an FC05, FC06, FC15 or FC16 request; 32/64-bit formats require FC16. Several values separated by ";" go to consecutive addresses. An accepted write queues a "Read once" of the window so the table shows the new value.

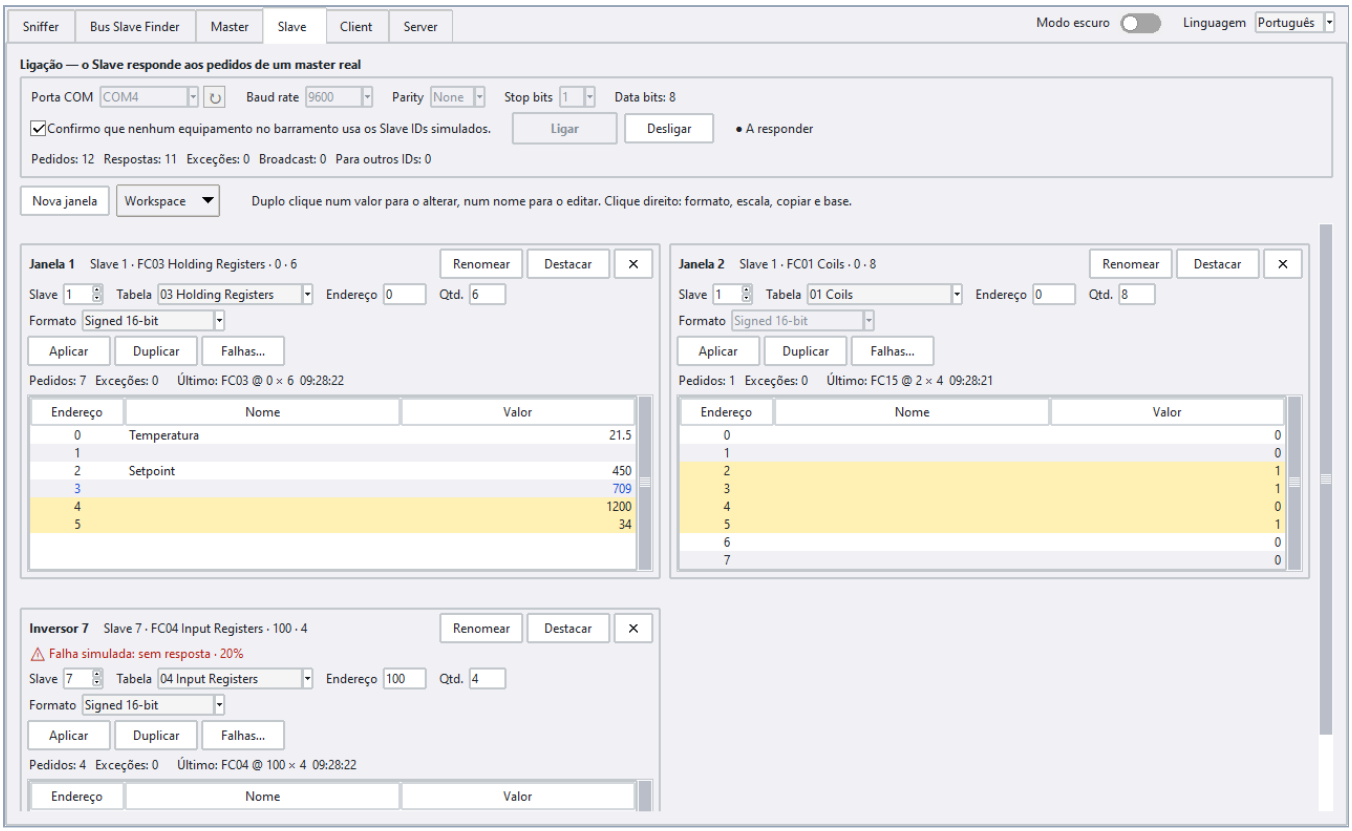
Window life cycle

- **Apply** validates the fields and updates the engine; a changed data area resets the counters. Results of a request sent with the previous definition are ignored.
- Each pane has a header with the window name (**Rename** or double-click), a summary with the Slave, the function code and name (e.g. FC03 Read Holding Registers), the address and the quantity, **Pop out** and a × close button, so closing never depends on the pane width.
- **Pop out** rebuilds the pane inside a `tk.Toplevel`; closing it or **Dock** returns it to the grid. The grid uses as many equal columns as the tab width allows, measured from the panes' requested width (which grows with Windows display scaling).
- Panes created after start-up receive the application's mouse-wheel guard tag: the wheel scrolls the window grid and never changes combobox/spinbox values.
- The **Workspace** menu saves named `.mbsw` files (JSON) with the connection parameters, every window, its name and address names in `%APPDATA%\MBToolkit\Workspaces\Master` (`get_master_workspaces_folder()`, which moves workspaces left directly in `Workspaces` by earlier versions into it once), beside `settings.json`; it lists them to open or delete. Invalid entries are skipped with a warning.
- Changes to the open workspace mark it "(modified)". `settings.json` keeps `master.last_workspace`, reopened at start-up without connecting; on exit, `master_confirm_close()` asks Yes/No/Cancel only for a saved workspace with unsaved changes.

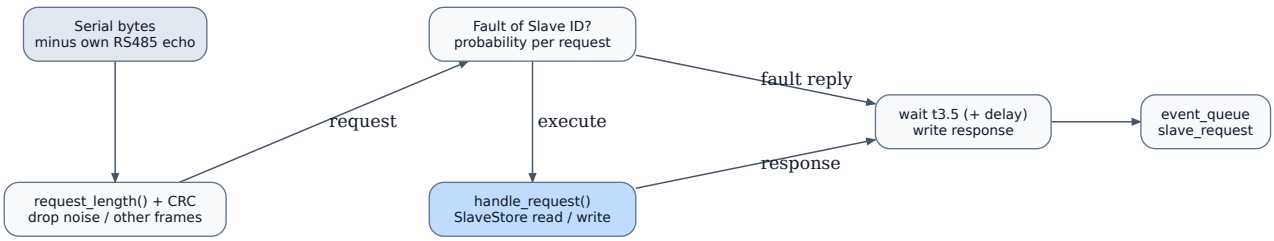
Operational safety. Like Bus Slave Finder, the Master transmits. Connecting requires the safety confirmation and an idle-bus check; while it is connected the global busy flag locks Sniffer, Simulation and Finder.

Phase boundary. Logging, conditional colours, charts, Test Center, FC22/23 and Modbus TCP are planned later phases and are not part of v3.5.

9. Slave (simulator) tab



Slave tab answering a Master engine over the in-memory link of tests/test_slave.py.



Slave engine: assemble a request, answer it from the store, apply faults.

The Slave tab is the inverse of the Master: the application behaves as one or more Modbus RTU devices, listens on a COM port and answers the requests of a real master. Each window is one data table (Coils, Discrete Inputs, Holding Registers or Input Registers) of one simulated Slave ID, defined by a start address and a quantity (up to 2000). Its state lives in a `SlaveWindow`.

Data store

`SlaveStore` keeps the values per (Slave ID, table) and address, starting at 0, plus the set of addresses defined by the windows (`set_areas()`, called whenever a window is added, applied or closed) and the active fault of each Slave ID. It is guarded by a lock: the engine thread reads and writes while the GUI edits. Windows over the same addresses therefore show the same values.

Answering one request

`handle_request(store, frame)` executes one CRC-valid request like a real device:

- a Slave ID that is not simulated is `ignored` (no reply);
- an unsupported function gets Exception 01; a wrong length, a quantity outside the Modbus limits, an FC05 value other than FF00/0000 or a byte count that does not match gets Exception 03;
- addresses not entirely inside the defined areas get Exception 02;
- reads return the stored bits or registers; writes store the values and answer with the echo (FC05/06) or the address/quantity (FC15/16), listing the written ranges in `SlaveReply.written`;
- broadcast (Slave 0) writes are executed on every simulated Slave ID whose areas cover the addresses and are never answered.

Serial loop

`SlaveEngine` reads the port continuously. `request_length()` knows the size of FC01–06 (8 bytes) and FC15/16 (9 + byte count) requests, so a request is answered as soon as it is complete and CRC-valid. Bytes that do not form a request (other devices' responses, noise) are dropped one at a time until a request lines up; after `SLAVE_RESYNC_SILENCE_MS` of silence the rest of the buffer is resolved, and a CRC-valid frame of an unknown function gets Exception 01. Before replying the engine waits the Modbus t3.5 gap; after replying it expects the RS485 adapter echo of its own response and removes it from the received bytes. Each handled request is posted as `slave_request` with its Slave ID, table, address, quantity, status and written ranges. Serial failures reopen the port like the Master.

Fault simulation

Faults are set per Slave ID (a real device fails on all its tables) and validated by `validate_fault()`: delay (0–10000 ms), no response, a chosen Exception (01, 02, 03, 04 or 06), wrong CRC, and a probability (1–100 %) drawn for every request. When the fault triggers, no response wins over the Exception, which wins over executing the request; the wrong CRC and the delay apply to whatever is sent. Only active faults are stored; their description is shown in red in the header of every window of that Slave ID.

Windows and values

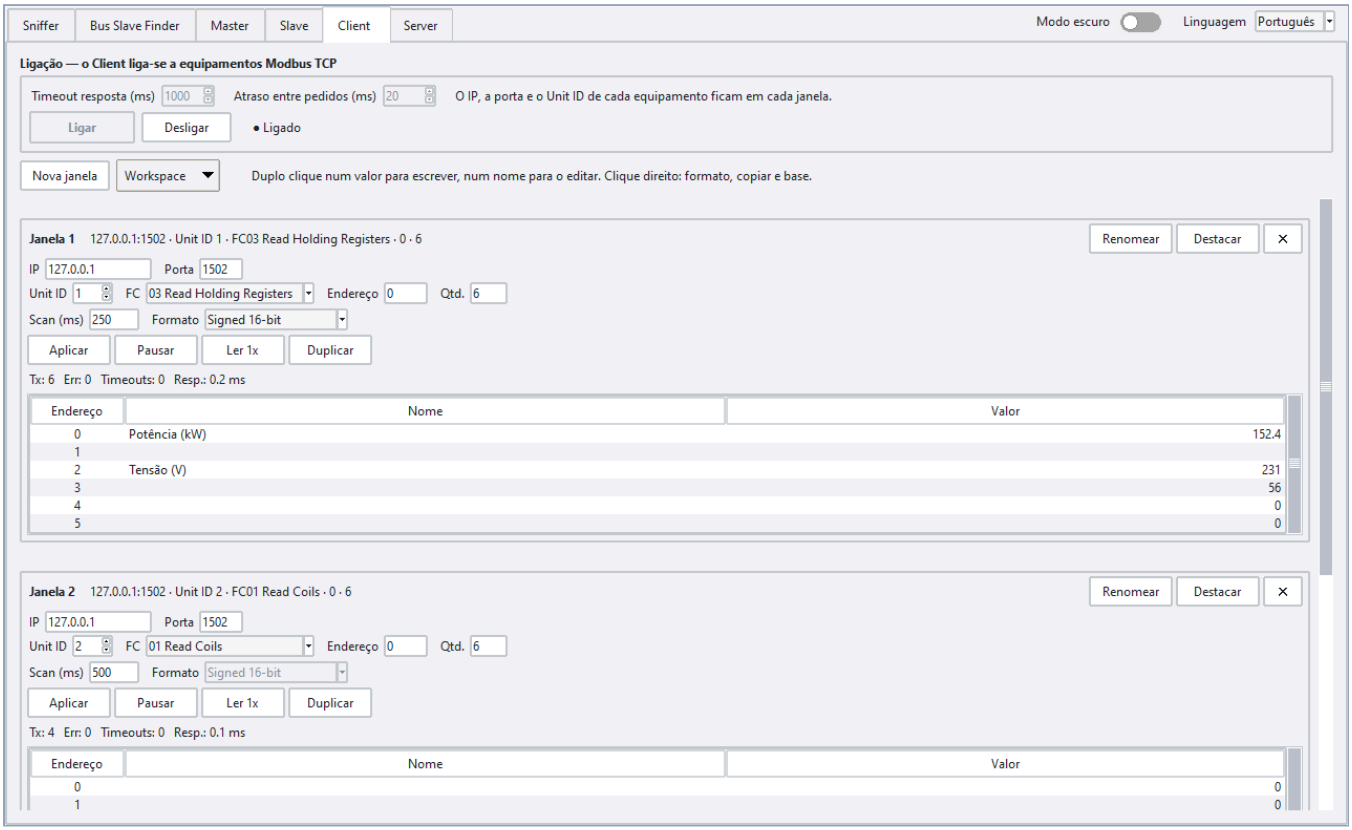
- Double-clicking a value edits it locally: bits toggle; registers use the value's format and scale (`slave_set_value()`). Nothing is sent on the bus.
- Master writes highlight the written rows for `SLAVE_HIGHLIGHT_SECONDS` (the `written` tag replaces the zebra tag so it is always visible); a 250 ms timer removes expired highlights.
- Each window counts the requests and exceptions that touched its addresses and shows the last request; the connection bar counts requests, responses, exceptions, broadcasts and requests for other IDs.
- **Automatic values:** `SlaveWindow.generators` maps the PDU address where a value starts to a `validate_generator()` setting: counter, ramp (sawtooth), sine or random for registers, with minimum and maximum in the units shown (format and scale) and a period of 0.1–3600 s; toggle or random for bits. `ValueGenerator.value(t)` is a function of the elapsed time (random draws one value per period). Every `SLAVE_GENERATOR_TICK_MS` the tab converts the values to registers

(`slave_generator_registers()`, inverse scale, rounded for integer formats) and writes only what changed into the store; those rows carry the `generated` tag (blue text). They are saved in the workspace and win over master writes on the next tick.

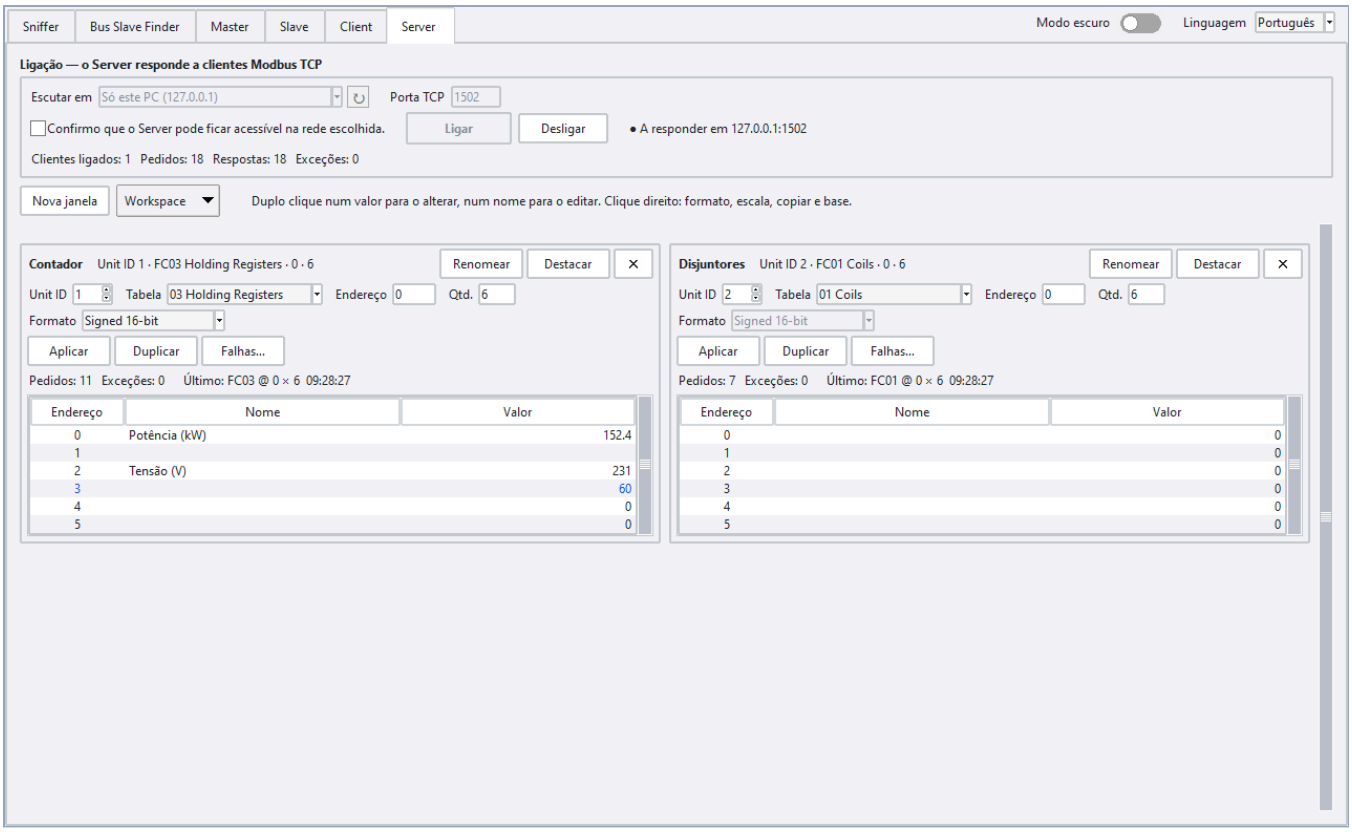
- **Open in Slave:** `slave_definition_from_record()` maps a Traffic record to a table and area (FC01–04 read tables, FC05/15 coils, FC06/16 holding registers); `slave_open_from_master()` copies a Master window's area, names, formats and scales.
- The Table field and the window header show the read function code with the table name (`table_choice_label()`: "03 Holding Registers"), like the FC field of the Master.
- Formats, scales, names, Copy, address base, drag selection, Rename, Pop out and close come from `TableUiMixin`, exactly as in the Master.
- The Slave Workspace menu saves configuration only (connection, windows, names, formats, scales and faults) in `%APPDATA%\MBToolkit\Workspaces\Slave`, format `MBToolkit Slave workspace`; opening one resets the values to 0. `slave.last_workspace` is reopened at start-up and `slave_confirm_close()` asks to save a changed one on exit.

> **Operational safety.** A simulated Slave ID that also exists on the bus makes two devices answer the same request. Connecting requires a confirmation that no real device uses the simulated IDs; while listening, the global busy flag locks Sniffer, Simulation, Finder and Master. Windows and USB adapters usually add a few milliseconds to every response.

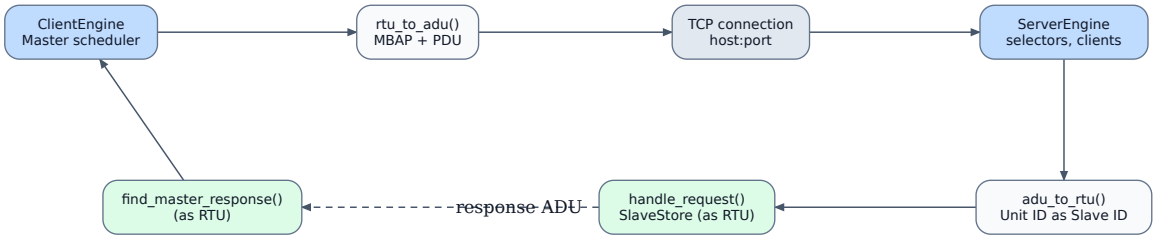
10. Modbus TCP: Client and Server tabs



Client tab polling the Server tab of the same application on 127.0.0.1:1502; the third window points at an unreachable device.



Server tab answering the Client tab (Unit IDs 1 and 2, an automatic value).



Modbus TCP: the Client and Server reuse the RTU Modbus core.

Modbus TCP carries the same PDU as Modbus RTU (function code and data) inside a 7-byte MBAP header instead of the Slave ID and CRC: Transaction ID, Protocol ID (0), Length and Unit ID. The Client and Server tabs are the Master and Slave tabs over that transport, in tabs of their own so they can run at the same time as the serial tabs.

One Modbus core, two envelopes

`rtu_to_adu()` and `adu_to_rtu()` convert between an RTU frame and an MBAP ADU (the Unit ID becomes the Slave byte, the CRC is computed). The Server turns each request into the equivalent RTU frame and answers it with `handle_request()`, the same function as the serial Slave; the Client builds its requests with `mb_master` and checks each response with `find_master_response()`. Function codes, exceptions, data formats and limits are therefore exactly those of the RTU tabs.

Server engine

- One thread serves the listening socket and up to `SERVER_MAX_CLIENTS` connections with `selectors`; each connection has its own receive buffer. On Windows the port is bound with `SO_EXCLUSIVEADDRUSE`, so a port in use is reported instead of shared.
- `adu_length()` reads the Length field; an impossible length closes the connection (the stream is out of step), a Protocol ID other than 0 is ignored.
- A Unit ID that is not simulated gets Exception 0B (Gateway Target Device Failed to Respond). Faults per Unit ID apply as on RTU, except that the corrupted response carries Protocol ID 1 (there is no CRC) and a delay is scheduled, so other clients are still served meanwhile.
- Events: `server_ready`, `server_client` (connected count), `server_request` (same shape as `slave_request`, plus the peer) and `server_stopped`.

Client engine

- `ClientEngine` subclasses `MasterEngine`: the same scheduler (writes, read once, earliest due window) with `event_prefix="client"`; only `transact_for(window_id, request)` is replaced.
- Each window names its device (`ClientPollDefinition.host` / `port`). One TCP connection per device is opened on first use, kept open and reopened after a failure, at most once per `CLIENT_RETRY_SECONDS`; an unreachable device gives `no_connection`, shown as "Sem ligação".
- Responses are matched by Transaction ID; a late answer to an earlier request is skipped. The Unit ID may be 0–255 and is never a broadcast in Modbus TCP.

Tab objects

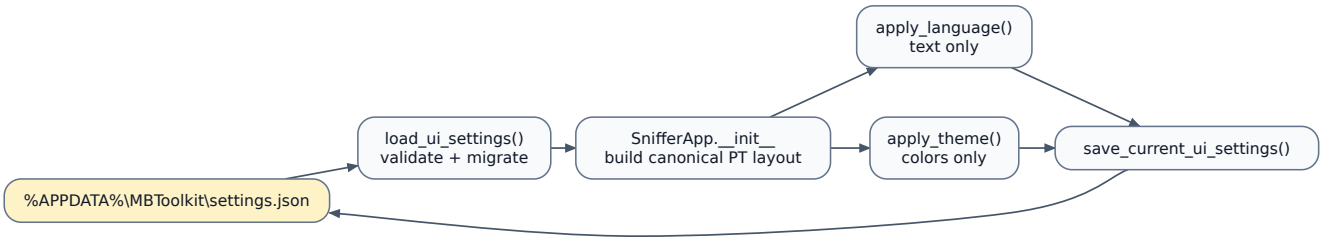
`TabInstance` stores the attributes a tab sets (windows, engine, widgets, `busy`) on the instance and forwards every other lookup (`tr`, `after`, `event_queue`, settings, theme) to the application. Class attributes such as `_master_tab_name`, `_master_settings_key`, `_master_ws_folder` and `_master_id_label` (and their `_slave_*` counterparts) describe each tab; the TCP tabs override them and the connection methods (`_master_build_connection`, `_master_build_target`, `master_connect`, `_slave_build_connection`, `slave_connect`, ...). Top-level windows and menus use `_table_root()` as parent. `process_queue()` routes `client_*` and `server_*` events to the two tab objects.

Listening address

"Escutar em" offers `127.0.0.1` (default), each IPv4 interface of the PC from `list_ipv4_interfaces()` (a hidden PowerShell `Get-NetIPAddress` with UTF-8 output, run in a thread; link-local addresses last) and `0.0.0.0`. Listening anywhere other than 127.0.0.1 requires the confirmation that the Server may be reachable on that network.

> **Network safety.** A Server on a network interface answers, and accepts writes from, any device of that network. The default keeps it on this PC; the choice of interface and the confirmation are explicit.

11. UI, language and theme design



Settings, language and theme lifecycle.

Canonical-language strategy

Portuguese (Portugal) is the canonical static UI language. `canonical_ui_text()` can map known English captions back to PT-PT, then `translate_text()` renders either canonical Portuguese or an explicit English translation. Technical English terms without an explicit mapping are left unchanged.

Messages with live values are stored as PT-PT templates in `PT_TO_EN` and rendered with `SnifferApp.trf(template, **values)`. Code never branches on `current_language`; a regression test enforces this.

Runtime language changes

`apply_language()` updates widget text, table headings, context menus, Help content and live status strings. Protocol bytes and parser data are not translated; only presentation fragments such as `Registos` / `Registers` are changed.

Language-independent geometry

The UI is built using the canonical Portuguese captions, then `_freeze_language_layout_geometry()` measures key dimensions and freezes them. This prevents shorter/longer English captions from moving the surrounding rectangles or controls.

Light/Dark invariant

The application uses one ttk `clam` construction. `mb_theme.py` configures geometry once and palette functions only change colors/state maps. Custom Canvas widgets receive palette updates separately.

Mouse-wheel behavior

An application bindtag lets Treeviews/Text/Listboxes/scrollbars consume wheel events normally while stopping wheel propagation elsewhere. This preserves table scrolling without allowing the Notebook to change tabs with the wheel.

12. Persistence, logs and export

User settings

On Windows, preferences are stored in `%APPDATA%\MBToolkit\settings.json`. Loading is defensive and every persisted choice/range is validated before use. Saves use a temporary file and replace operation.

Persisted data includes language, Light/Dark, last main tab, serial format, frame-gap mode/value, Pending timeout, auto-scroll, transaction separators, highlighting and Finder options (including attempts and the minimum timeout, migrated from the old 40 ms default to 100 ms) and the Master and Slave connection parameters. Master and Slave windows are not part of the settings: they are saved explicitly in workspace files. COM names, window geometry, traffic/results and the Finder safety confirmation are intentionally not persisted.

TXT capture log

Real-capture logs are lazy: the file is created only when traffic actually arrives. The disk log is complete and independent of table filters, sorting and the bounded GUI history.

CSV export

CSV exports the records currently retained in `_frame_history`, not necessarily an arbitrarily long complete session. It includes timing, channel, type, address metadata, CRC, raw bytes, timeout status and anomaly classification.

Bounded GUI history

The UI retains up to approximately `MAX_UI_FRAMES = 20000` records and prunes in chunks after a hysteresis threshold. Raw Text has a similar bounded-line policy. This prevents the GUI from growing indefinitely while keeping the capture log complete.

13. Module-by-module reference

The following chapters provide a complete callable inventory for every Python module. Each function/method is tied to its exact source-line range. The complete source itself appears as screenshots in Appendix A.

MBToolkit.py

Application launcher and top-level entry point. It sets the development simulation switch before importing the GUI, applies the Windows application identity, creates SnifferApp and enters Tkinter's main event loop.

Source lines	SHA-256
29	`fe17c0be094d1cdd767f0d3a6502b4ea544e6d12f935bc4f641f9f8c589d4546`

Imports: `mb_config, mb_config.set_windows_app_user_model_id, mb_gui.SnifferApp`

Module constants / configuration data

Name	Definition
`debug_sim`	`0`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`main` main()	23-25	Main.	`set_windows_app_user_model_id, SnifferApp().mainloop, SnifferApp`

mb_config.py

Central configuration, constants, user settings validation and filesystem paths. It defines versioned application constants, discovery limits, GUI history limits, Modbus exception metadata, persistent settings behavior, application/log directories and PyInstaller resource lookup.

Source lines	SHA-256
637	`a3c9cc0389a00f648a913dadbcd1b4ab27c40c5fbbcbc2db9453238a83e45658`

Imports: `ctypes, ipaddress, json, os, shutil, sys, pathlib.Path`

Module constants / configuration data

Name	Definition
`APP_NAME`	`'MBToolkit'`
`LEGACY_APP_NAME`	`'MBSniffer'`
`APP_VERSION`	`'3.5'`
`debug_sim`	`0`
`DISCOVERY_BAUD_RATES`	`(1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200)`
`DISCOVERY_PARITIES`	`('None', 'Even', 'Odd')`
`DISCOVERY_STOP_BITS`	`('1', '2')`
`DISCOVERY_DEFAULT_MIN_TIMEOUT_MS`	`100`
`DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS`	`40`
`DISCOVERY_MIN_TIMEOUT_MS`	`10`
`DISCOVERY_MAX_TIMEOUT_MS`	`2000`
`DISCOVERY_PROBE_ADDRESS`	`0`
`DISCOVERY_PROBE_QTY`	`1`
`DISCOVERY_WRITE_TIMEOUT_SECONDS`	`1.5`
`DISCOVERY_WRITE_RETRIES`	`1`
`DISCOVERY_RX_BUFFER_LIMIT`	`1024`
`DISCOVERY_MIN_ATTEMPTS`	`1`
`DISCOVERY_MAX_ATTEMPTS`	`3`
`DISCOVERY_DEFAULT_ATTEMPTS`	`1`

Name	Definition
DISCOVERY_IDLE_CHECK_SECONDS	1.0
UI_QUEUE_BATCH_SIZE	200
UI_QUEUE_IDLE_INTERVAL_MS	50
UI_QUEUE_BUSY_INTERVAL_MS	10
MIN_PENDING_REQUEST_TIMEOUT_SECONDS	1.0
DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS	10.0
MAX_PENDING_REQUEST_TIMEOUT_SECONDS	300.0
PENDING_REQUEST_TIMEOUT_SECONDS	DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS
MASTER_MAX_READ_BITS	2000
MASTER_MAX_READ_REGISTERS	125
MASTER_MAX_WRITE_BITS	1968
MASTER_MAX_WRITE_REGISTERS	123
MASTER_DEFAULT_RESPONSE_TIMEOUT_MS	1000
MASTER_MIN_RESPONSE_TIMEOUT_MS	10
MASTER_MAX_RESPONSE_TIMEOUT_MS	10000
MASTER_DEFAULT_DELAY_MS	20
MASTER_MIN_DELAY_MS	0
MASTER_MAX_DELAY_MS	10000
MASTER_DEFAULT_SCAN_MS	1000
MASTER_MAX_SCAN_MS	3600000
MASTER_MAX_WINDOWS	32
MASTER_RECONNECT_INTERVAL_SECONDS	2.0
MASTER_WORKSPACE_FORMAT	'MBToolkit Master workspace'
MASTER_WORKSPACE_FORMATS	(MASTER_WORKSPACE_FORMAT, 'MBSniffer Master workspace')
MASTER_WORKSPACE_VERSION	1
SLAVE_MAX_WINDOWS	32
SLAVE_MAX_QTY	2000
SLAVE_HIGHLIGHT_SECONDS	1.5

Name	Definition
`SLAVE_FAULT_MAX_DELAY_MS`	`10000`
`SLAVE_GENERATOR_TICK_MS`	`100`
`SLAVE_RESYNC_SILENCE_MS`	`40.0`
`SLAVE_WORKSPACE_FORMAT`	`'MBToolkit Slave workspace'`
`SLAVE_WORKSPACE_VERSION`	`1`
`CLIENT_WORKSPACE_FORMAT`	`'MBToolkit Client workspace'`
`SERVER_WORKSPACE_FORMAT`	`'MBToolkit Server workspace'`
`TCP_DEFAULT_PORT`	`502`
`SERVER_DEFAULT_BIND`	`'127.0.0.1'`
`SNIFFER_SPLIT_FRAME_HOLD_MS`	`40.0`
`SLOW_RESPONSE_THRESHOLD_MS`	`500.0`
`MODBUS_EXCEPTION_CODES`	`{1: ('Illegal Function', 'A função Modbus pedida não é suportada/permitida pelo dispositivo.'), 2: ('Illegal Data Address', 'O endereço ou a gama d…`
`MAX_UI_FRAMES`	`20000`
`MAX_RAW_TEXT_LINES`	`20000`
`UI_PRUNE_CHUNK_FRAMES`	`2000`
`RAW_TEXT_PRUNE_CHUNK_LINES`	`2000`
`RAW_TRANSACTION_SEPARATOR`	`'—' * 72 + '\n'`
`SIMULATION_FUNCTION_CODES`	`(1, 2, 3, 4)`
`SIMULATION_EXCEPTION_CODES`	`(1, 2, 3, 4, 6)`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`resource_path` resource_path(filename)	159-165	Return a bundled/source resource path for normal Python or PyInstaller.	`Path, Path(__file__).resolve`

Callable	Lines	Responsibility	Representative direct calls
`set_windows_app_user_model_id` set_windows_app_user_model_id()	167-179	Give Windows a stable application identity so the taskbar can associate the Tk window with MBToolkit instead of a generic Python process.	`ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID`
`get_settings_file` get_settings_file()	183-204	Return the persistent per-user settings file.	`_settings_folder, folder.mkdir, folder.exists, _migrate_legacy_settings, Path.home`
`_settings_folder` _settings_folder(app_name)	207-213	Settings folder.	`os.environ.get, Path.home, Path, app_name.lower`
`_migrate_legacy_settings` _migrate_legacy_settings(folder)	216-228	Copy MBSniffer's settings and workspaces into the new MBToolkit folder.	`_settings_folder, legacy.is_dir, shutil.copytree`
`_validated_choice` _validated_choice(value, allowed, default)	231-232	Validated choice.	—
`_validated_bool` _validated_bool(value, default)	235-236	Validated bool.	`isinstance`
`_validated_number_string` _validated_number_string(value, minimum, maximum, default)	239-246	Validated number string.	`str, float`
`_validated_int_string` _validated_int_string(value, minimum, maximum, default)	249-256	Validated int string.	`str, int`
`load_ui_settings` load_ui_settings()	259-500	Reads settings.json defensively and validates every persisted choice/range.	`get_settings_file, isinstance, data.get, json.loads, sniffer_in.get`
`_validated_ipv4` _validated_ipv4(value, default)	503-508	IPv4 address text remembered in settings.json, or *default*.	`str, ipaddress.IPv4Address, str(value).strip`
`_validated_workspace_name` _validated_workspace_name(value)	511-518	Workspace name remembered in settings.json, or "" when unusable.	`value.strip, isinstance, any, len`
`save_ui_settings` save_ui_settings(settings=None, **updates)	521-545	Merges updates with validated current settings and uses a temporary file plus replace operation for atomic persistence; settings failures never prevent normal application use.	`get_settings_file, load_ui_settings, isinstance, current.update, path.parent.mkdir`

Callable	Lines	Responsibility	Representative direct calls
`get_application_directory` get_application_directory()	548-569	Resolves a user-facing application directory consistently for PyInstaller EXE, packaged BAT/source mode and development mode.	`getattr, (launcher_dir / 'MBToolkit.bat').is_file, Path(__file__).resolve, Path.cwd, Path(sys.executable).resolve`
`get_workspaces_folder` get_workspaces_folder()	571-579	Return/create the Master workspace folder beside settings.json:.	`folder.mkdir, get_settings_file`
`get_master_workspaces_folder` get_master_workspaces_folder()	582-601	Return/create the Master workspace folder inside the workspace folder:.	`get_workspaces_folder, folder.exists, folder.mkdir, root.glob, old.is_file`
`get_client_workspaces_folder` get_client_workspaces_folder()	604-608	Return/create %APPDATA%\MBToolkit\Workspaces\Client (Modbus TCP Client).	`folder.mkdir, get_workspaces_folder`
`get_server_workspaces_folder` get_server_workspaces_folder()	611-615	Return/create %APPDATA%\MBToolkit\Workspaces\Server (Modbus TCP Server).	`folder.mkdir, get_workspaces_folder`
`get_slave_workspaces_folder` get_slave_workspaces_folder()	618-626	Return/create the Slave workspace folder inside the workspace folder:.	`folder.mkdir, get_workspaces_folder`
`get_logs_folder` get_logs_folder()	629-637	Creates and returns the MBToolkit Logs directory beside the launcher/EXE.	`folder.mkdir, get_application_directory`

Detailed behavior of the key paths

`load_ui_settings` lines 259-500

Reads settings.json defensively and validates every persisted choice/range. Invalid/corrupt data falls back safely. It also migrates the older highlight_anomalies key to highlight_results and the old 40 ms Finder timeout default to 100 ms.

`save_ui_settings` lines 521-545

Merges updates with validated current settings and uses a temporary file plus replace operation for atomic persistence; settings failures never prevent normal application use.

`get_application_directory` lines 548-569

Resolves a user-facing application directory consistently for PyInstaller EXE, packaged BAT/source mode and development mode.

`get_logs_folder` **lines 629-637**

Creates and returns the MBToolkit Logs directory beside the launcher/EXE.

mb_protocol.py

Protocol engine for passive Modbus RTU parsing and session metrics. It implements CRC16, RTU t3.5 timing, frame-shape recognition, resynchronisation, decoding, request/response pairing, timeout accounting, echo-write and broadcast classification, split-frame detection, Bus Health metrics and debug traffic generation.

Source lines	SHA-256
1007	`8c4c41051e41f0c998692206ed0204d91f057cce5fce726b15ad2cce503c1ef2`

Imports: `math, random, time, bisect.bisect_right, collections.defaultdict, deque, datetime.datetime, mb_config.MODBUS_EXCEPTION_CODES, PENDING_REQUEST_TIMEOUT_SECONDS, SIMULATION_FUNCTION_CODES, SIMULATION_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS`

Module constants / configuration data

Name	Definition
`_CRC16_TABLE`	`_build_crc16_table()`
`KNOWN_FRAME_FUNCTION_CODES`	`(1, 2, 3, 4, 5, 6, 7, 8, 15, 16, 22, 23)`
`EXACT_ECHO_FUNCTION_CODES`	`(5, 6, 22)`
`BROADCAST_UNIT_ID`	`0`
`READ_BITS_DETAILS_LIMIT`	`32`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`_build_crc16_table` _build_crc16_table()	19-29	Build crc16 table.	`range, tuple, table.append`
`crc16_modbus` crc16_modbus(data)	36-41	Computes the standard Modbus CRC-16 value (initial 0xFFFF, reflected polynomial 0xA001) with one lookup in the precomputed 256-entry _CRC16_TABLE per byte. append_crc() stores the low CRC byte first, matching Modbus RTU wire order.	—
`crc_ok` crc_ok(frame)	43-48	Crc ok.	`crc16_modbus, len`

Callable	Lines	Responsibility	Representative direct calls
`append_crc` append_crc(payload)	50-52	Append crc.	`crc16_modbus, bytes`
`hex_bytes` hex_bytes(data)	54-55	Hex bytes.	`' '.join`
`modbus_rtu_interframe_gap_seconds` modbus_rtu_interframe_gap_seconds(baud, bits_per_char)	57-72	Implements the automatic t3.5 rule used by capture and discovery: 3.5 character times at or below 19200 bit/s, fixed 1.750 ms above 19200 bit/s.	`int, float, ValueError`
`plausible_request_qty` plausible_request_qty(fc, qty)	74-79	Plausible request qty.	—
`_plausible_unit_id` _plausible_unit_id(value)	89-94	Return True for Modbus RTU unit addresses that can appear on the wire.	`int`
`_candidate_prefixes_at` _candidate_prefixes_at(buf, start=0)	97-168	Infers only structurally plausible lengths for supported Modbus frame shapes.	`set, len, int, _plausible_unit_id, out.append`
`candidate_prefixes` candidate_prefixes(buf)	171-173	Compatibility wrapper for callers/tests that inspect offset zero.	`_candidate_prefixes_at`
`_crc_ok_at` _crc_ok_at(buf, start, length)	176-183	CRC check without copying the whole remaining capture buffer.	`crc16_modbus, len, memoryview`
`find_valid_prefix` find_valid_prefix(buf, start=0)	186-191	Find a CRC-valid, structurally recognised Modbus frame at *start*.	`_candidate_prefixes_at, _crc_ok_at, len`
`guess_invalid_whole_frame` guess_invalid_whole_frame(buf)	194-201	Classify a complete known-shape frame whose CRC is invalid.	`_candidate_prefixes_at, len, _plausible_unit_id`
`_unknown_whole_frame_is_valid` _unknown_whole_frame_is_valid(buf)	204-217	Conservative fallback for one unsupported FC occupying the whole burst.	`_candidate_prefixes_at, crc_ok, _plausible_unit_id, len`
`split_capture_buffer` split_capture_buffer(data)	220-269	Walks a burst from left to right.	`bytearray, find_valid_prefix, max, range, guess_invalid_whole_frame`
`burst_may_continue` burst_may_continue(data)	271-302	Reports whether a burst ends with the start of a known frame that has not fully arrived yet, so the reader can wait for bytes delayed by a USB adapter latency timer instead of emitting two RAW rows.	`split_capture_buffer, any, _plausible_unit_id, len, _candidate_prefixes_at`

Callable	Lines	Responsibility	Representative direct calls
`read_bits_from_response` read_bits_from_response(frame, qty=None)	305-322	Return FC01/FC02 coil/input states, least-significant bit first.	`len, range, int`
`read_bits_details` read_bits_details(frame, qty=None)	330-338	Formats FC01/FC02 response bits in address order, grouped per data byte, trimmed to the request quantity when known and capped at 32 visible bits.	`read_bits_from_response, ''.join, ' '.join, len, str`
`decode_frame` decode_frame(frame, kind_hint)	341-449	Extracts Slave, FC, PDU address, one-based address, quantity and selected payload details from a split frame.	`str(kind_hint).startswith, str, len, MODBUS_EXCEPTION_CODES.get, read_bits_details`
`format_clock_time` format_clock_time(epoch)	451-452	Format clock time.	`datetime.fromtimestamp(float(epoch)).strftime, datetime.fromtimestamp, float`
`parse_burst` parse_burst(data, timestamp, channel='', event_time=None, chunk_times=None)	455-498	Transforms split frames into the row dictionary consumed by the rest of the program.	`split_capture_buffer, time.time, float, len, decode_frame`
`class SessionMetrics`	**500-872**	**Tracks pairing, counters and bus-health diagnostics for one session.**	**Class container**
`SessionMetrics.__init__` __init__(self, pending_timeout_seconds=PENDING_REQUEST_TIMEOUT_SECONDS)	503-505	Init.	`float, self.reset`
`SessionMetrics.reset` reset(self)	507-531	Reset.	`set, defaultdict, deque`
`SessionMetrics._key` _key(self, row)	533-543	Key.	`int, row.get`
`SessionMetrics.pending_count` pending_count(self)	545-546	Pending count.	`sum, len, self.pending.values`
`SessionMetrics.pop_expired_requests` pop_expired_requests(self)	548-551	Pop expired requests.	—
`SessionMetrics.expire_pending` expire_pending(self, now=None)	553-591	Expire stale requests and retain their sequence IDs for the GUI.	`float, list, time.time, self.pending.keys, self.pending.pop`
`SessionMetrics.force_timeout` force_timeout(self, slave, fc)	593-617	Debug/simulation helper: expire the oldest pending request for one key.	`self.pending.get, pending_list.pop, dict, self.pending.pop, int`

Callable	Lines	Responsibility	Representative direct calls
`SessionMetrics._resolve_echo_direction` _resolve_echo_direction(self, row)	619-651	Classifies FC05/06/08/22 frames as REQUEST or RESPONSE from the pending queue: same key, identical bytes for exact echoes and the opposite channel in RS232 dual RX.	`self._key, self.pending.get, row.get, candidate.get`
`SessionMetrics._record_response_time` _record_response_time(self, row, response_ms)	653-664	Record response time.	`float, self.response_times_ms.append, self.response_times_by_slave[slave].append, int, row.get`
`SessionMetrics.process` process(self, row)	666-765	This is the session accounting hot path.	`float, self.expire_pending, self.pop_expired_requests, row.get, self._record_response_time`
`SessionMetrics._p95` _p95(values)	768-773	P95.	`sorted, max, min, float, len`
`SessionMetrics.health_summary` health_summary(self, *, baud=9600, data_bits=8, parity='None', stop_bits=1, now=None)	775-872	Builds the Bus Health model: request rate, response average/min/max/P95, slowest Slave, unvalidated-byte rate, timeout rate, active Slave count, approximate bus utilisation and exception distribution.	`list, self._p95, self.response_times_by_slave.items, time.time, max`
`u16_bytes` u16_bytes(value)	874-876	U16 bytes.	`bytes, int`
`corrupt_crc` corrupt_crc(frame)	878-882	Corrupt crc.	`bytearray, bytes, len`
`build_simulation_transaction` build_simulation_transaction(slave, fc, rng)	884-928	Build one syntactically valid Modbus request/response pair.	`int, rng.randint, append_crc, bytes, b''.join`
`build_random_simulation_plan` build_random_simulation_plan(rng=None)	930-952	Build exactly four simulation transactions: FC01, FC02, FC03 and FC04, one each.	`rng.sample, zip, random.Random, range, build_simulation_transaction`
`build_anomaly_simulation_plan` build_anomaly_simulation_plan(rng=None)	956-1006	Creates the deterministic categories required to preview diagnostic highlighting: normal response, slow response, Exception, bad CRC, timeout and RAW bytes.	`rng.sample, build_simulation_transaction, append_crc, corrupt_crc, random.Random`

Detailed behavior of the key paths

`crc16_modbus` **lines 36-41**

Computes the standard Modbus CRC-16 value (initial 0xFFFF, reflected polynomial 0xA001) with one lookup in the precomputed 256-entry _CRC16_TABLE per byte. append_crc() stores the low CRC byte first, matching Modbus RTU wire order.

`modbus_rtu_interframe_gap_seconds` **lines 57-72**

Implements the automatic t3.5 rule used by capture and discovery: 3.5 character times at or below 19200 bit/s, fixed 1.750 ms above 19200 bit/s.

`_candidate_prefixes_at` **lines 97-168**

Infers only structurally plausible lengths for supported Modbus frame shapes. This constrains resynchronisation and avoids the former brute-force CRC scan across arbitrary lengths.

`split_capture_buffer` **lines 220-269**

Walks a burst from left to right. A valid recognised frame is emitted immediately; otherwise the parser searches later offsets for a recognised CRC-valid frame. Bytes before the recovered frame become CRC ERROR if they exactly match a known frame shape, otherwise RAW/UNSYNC. If nothing can be recovered, the remainder becomes CRC ERROR, UNKNOWN/OK, or RAW/UNPARSED depending on its structure and CRC.

`burst_may_continue` **lines 271-302**

Reports whether a burst ends with the start of a known frame that has not fully arrived yet, so the reader can wait for bytes delayed by a USB adapter latency timer instead of emitting two RAW rows.

`read_bits_details` **lines 330-338**

Formats FC01/FC02 response bits in address order, grouped per data byte, trimmed to the request quantity when known and capped at 32 visible bits.

`decode_frame` **lines 341-449**

Extracts Slave, FC, PDU address, one-based address, quantity and selected payload details from a split frame. It handles Exceptions and common read/write Function Codes without modifying the raw bytes.

`parse_burst` **lines 455-498**

Transforms split frames into the row dictionary consumed by the rest of the program. It attaches display time, channel, decoded fields, raw hex, frame length and a floating epoch timestamp; with chunk_times each frame gets the arrival time of its own serial read.

`SessionMetrics.process` lines 666-765

This is the session accounting hot path. It expires old pending requests first, resolves REQ/RESP frames, updates byte/counter statistics, stores valid non-broadcast requests by (Slave, FC), FIFO-pairs responses/exceptions to pending requests, calculates response time and returns timing/pairing information to the view.

`SessionMetrics._resolve_echo_direction` lines 619-651

Classifies FC05/06/08/22 frames as REQUEST or RESPONSE from the pending queue: same key, identical bytes for exact echoes and the opposite channel in RS232 dual RX. Broadcast frames are always requests.

`SessionMetrics.health_summary` lines 775-872

Builds the Bus Health model: request rate, response average/min/max/P95, slowest Slave, unvalidated-byte rate, timeout rate, active Slave count, approximate bus utilisation and exception distribution.

`build_anomaly_simulation_plan` lines 956-1006

Creates the deterministic categories required to preview diagnostic highlighting: normal response, slow response, Exception, bad CRC, timeout and RAW bytes.

> **Design note.** Important design boundary: this module contains no Tkinter code and can be regression-tested independently. That separation is valuable if the project is later rebuilt for a thesis or extended with additional front ends.

mb_capture.py

Serial-port management and passive acquisition pipeline. It opens selected COM ports, starts reader threads, converts inter-frame silence into capture bursts, parses those bursts, queues GUI events, controls simulation, handles COM restart and performs orderly shutdown.

Source lines	SHA-256
913	`d05166ac79d8b546d7527aea97be87a516d17b81291d278da36c928933406f33`

Imports: `queue, random, threading, time, traceback, datetime.datetime, tkinter, tkinter.messagebox, mb_config.APP_NAME, APP_VERSION, MIN_PENDING_REQUEST_TIMEOUT_SECONDS, MAX_PENDING_REQUEST_TIMEOUT_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS, UI_QUEUE_BATCH_SIZE, UI_QUEUE_IDLE_INTERVAL_MS, UI_QUEUE_BUSY_INTERVAL_MS, debug_sim, mb_protocol.parse_burst, burst_may_continue, build_anomaly_simulation_plan, modbus_rtu_interframe_gap_seconds, mb_slave_finder.cancel_serial_io`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class CaptureMixin`	**33-912**	**Methods for serial capture, COM management and simulation.**	**Class container**
`CaptureMixin.set_sniffer_refresh_state` set_sniffer_refresh_state(self, state)	36-41	Apply the same state to both Sniffer COM refresh buttons.	`getattr, button.configure`
`CaptureMixin.update_mode_ui` update_mode_ui(self)	43-63	Update mode ui.	`self.mode_var.get, self.port_b_combo.configure, self.update_com_status, self.mode_help_var.set, self.tr`
`CaptureMixin.update_gap_state` update_gap_state(self)	65-68	Update gap state.	`self.gap_entry.configure, self.gap_mode_var.get`
`CaptureMixin.get_serial_modules` get_serial_modules(self)	70-76	Get serial modules.	—
`CaptureMixin.refresh_ports` refresh_ports(self)	78-123	Refresh ports.	`self.get_serial_modules, hasattr, self.update_com_status, self.port_a_var.set, self.port_b_var.set`

Callable	Lines	Responsibility	Representative direct calls
`CaptureMixin.detected_ports` detected_ports(self)	125-133	Return the set of COM device names currently reported by Windows/pyserial.	`self.get_serial_modules, set, list_ports.comports`
`CaptureMixin.port_status_text` port_status_text(self, label, port, enabled=True)	135-151	Port status text.	`self.serial_by_port.get, self.detected_ports, self.tr`
`CaptureMixin.update_com_status` update_com_status(self)	153-172	Update COM status without opening/probing a port.	`self.port_a_status_var.set, self.port_b_status_var.set, hasattr, self.mode_var.get, self.port_status_text`
`CaptureMixin.poll_com_status` poll_com_status(self)	174-186	Poll com status.	`self.update_com_status, hasattr, self.update_stats_labels, self.after`
`CaptureMixin.selected_port_plan` selected_port_plan(self)	188-206	Translates physical mode into one or two receive channels.	`self.mode_var.get, self.port_a_var.get().strip, self.port_b_var.get().strip, ValueError, plan.append`
`CaptureMixin.open_selected_ports` open_selected_ports(self, serial, config)	208-221	Open the selected capture ports and return [(channel, serial_obj), ...].	`self.selected_port_plan, opened.append, self.open_serial_port, ser_obj.close`
`CaptureMixin.install_opened_ports` install_opened_ports(self, opened)	223-228	Install opened ports.	`self.update_com_status, getattr`
`CaptureMixin.start_reader_threads` start_reader_threads(self, opened, gap_ms)	230-240	Start reader threads.	`threading.Thread, self.reader_threads.append, t.start`
`CaptureMixin.restart_com_port` restart_com_port(self)	242-338	Close and reopen the selected COM port(s).	`self.get_serial_modules, self.restart_btn.configure, self.status_var.set, self.stop_event.set, list`
`CaptureMixin.set_simulation_button_state` set_simulation_button_state(self, state)	340-343	Change Simulation button state only when the optional button exists.	`self.sim_btn.configure`
`CaptureMixin.selected_pending_timeout_seconds` selected_pending_timeout_seconds(self)	345-364	Selected pending timeout seconds.	`float, ValueError, self.pending_timeout_var.get().strip().replace, self.trf, self.tr`
`CaptureMixin.calculated_gap_ms` calculated_gap_ms(self)	366-378	Returns the manual gap after validation, or computes the automatic RTU t3.5 value from baud/data/parity/stop settings using the shared protocol timing helper.	`int, float, self.gap_mode_var.get, self.baud_var.get, self.data_var.get`

Callable	Lines	Responsibility	Representative direct calls
`CaptureMixin.serial_config` serial_config(self, serial)	380-402	Serial config.	`int, self.baud_var.get, self.parity_var.get, self.stop_var.get, self.data_var.get`
`CaptureMixin.open_serial_port` open_serial_port(self, serial, port, config)	404-436	Open serial port.	`serial.Serial, ser.open, ser.reset_input_buffer`
`CaptureMixin.start_capture` start_capture(self)	438-496	Validates state and serial settings, opens the selected port(s), resets the current session, prepares lazy logging, starts reader workers and updates the GUI to capturing state.	`self.get_serial_modules, self.mode_var.get, self.port_a_var.get().strip, self.port_b_var.get().strip, self.install_opened_ports`
`CaptureMixin.reader_loop` reader_loop(self, ser, gap_ms, channel, generation)	498-575	Runs in a daemon thread per capture channel.	`max, min, bytearray, parse_burst, self.event_queue.put`
`CaptureMixin.stop_capture` stop_capture(self)	577-580	Stop capture.	`self.stop_event.set, self.status_var.set, self.tr`
`CaptureMixin.run_simulation` run_simulation(self)	582-683	Runs the internal diagnostic scenario without opening serial hardware.	`self.reset_session_metrics, self.close_log, self.status_var.set, self.start_btn.configure, self.restart_btn.configure`
`CaptureMixin.process_queue` process_queue(self)	685-840	Main-thread bridge between workers and Tkinter.	`range, flush_frames, self.metrics.expire_pending, time.time, self.metrics.pop_expired_requests`
`CaptureMixin.on_close` on_close(self)	842-912	Performs defensive shutdown: saves settings, signals all workers, cancels serial I/O, joins threads with timeouts, closes serial handles/logs and destroys the Tk root.	`self.stop_event.set, self.discovery_stop_event.set, cancel_serial_io, list, self.close_log`

Detailed behavior of the key paths

`selected_port_plan` lines 188-206

Translates physical mode into one or two receive channels. RS485 and single-RX RS232 use COM A; dual-RX RS232 requires distinct COM A and COM B and labels their directions A→B/B→A.

`calculated_gap_ms` lines 366-378

Returns the manual gap after validation, or computes the automatic RTU t3.5 value from baud/data/parity/stop settings using the shared protocol timing helper.

`start_capture` lines 438-496

Validates state and serial settings, opens the selected port(s), resets the current session, prepares lazy logging, starts reader workers and updates the GUI to capturing state.

`reader_loop` lines 498-575

Runs in a daemon thread per capture channel. It accumulates serial bytes and their read times into a burst, considers the burst complete after the configured inter-frame silence (or after the split-frame hold when the burst ends mid-frame), then calls parse_burst() and posts frame events to the GUI queue.

`run_simulation` lines 582-683

Runs the internal diagnostic scenario without opening serial hardware. It feeds the same parser/event path used by real capture, including forced timeout handling.

`process_queue` lines 685-840

Main-thread bridge between workers and Tkinter. It drains events in bounded batches, batches frame insertions for performance, handles capture/discovery status events, expires requests and reschedules itself with a faster cadence when backlog exists.

`on_close` lines 842-912

Performs defensive shutdown: saves settings, signals all workers, cancels serial I/O, joins threads with timeouts, closes serial handles/logs and destroys the Tk root.

> **Design note.** Tkinter widgets are never updated directly by reader threads. Workers communicate through event_queue; process_queue() runs on the Tk main thread.

mb_diagnostics.py

Presentation-oriented diagnostics derived from captured records. It maps Function Codes to names, recalculates/display CRC fields, extracts payloads, classifies anomalies/results and builds the structured Frame Inspector text.

Source lines	SHA-256
286	`c67ed1df18f1d79b0feb4011a036f92cbeef338929a529a4f218acd1f78e1552`

Imports: `mb_config.MODBUS_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS, mb_protocol.crc16_modbus, read_bits_from_response, mb_i18n.LANGUAGE_PORTUGUESE, translate_details, translate_text`

Module constants / configuration data

Name	Definition
`FUNCTION_NAMES`	`{1: 'Read Coils', 2: 'Read Discrete Inputs', 3: 'Read Holding Registers', 4: 'Read Input Registers', 5: 'Write Single Coil', 6: 'Write Single Regis…`
`RESULT_LABELS`	`{'success': ('✓ Sucesso', 'Verde'), 'slow': ('⚠ Resposta lenta', 'Amarelo'), 'exception': ('Exception', 'Vermelho'), 'crc': ('× CRC Error', 'Vermel…`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`raw_hex_to_bytes` raw_hex_to_bytes(raw)	26-30	Raw hex to bytes.	`bytes.fromhex, str`
`fc_number` fc_number(record)	33-41	Fc number.	`str(record.get('fc', '') or '').strip, int, record.get, str`
`function_label` function_label(record, language=LANGUAGE_PORTUGUESE)	44-49	Function label.	`fc_number, FUNCTION_NAMES.get, translate_text`
`crc_fields` crc_fields(record)	52-62	Crc fields.	`str(record.get('type', '')).startswith, raw_hex_to_bytes, crc16_modbus, record.get, len`

Callable	Lines	Responsibility	Representative direct calls
`response_payload` response_payload(record, language=LANGUAGE_PORTUGUESE)	65-127	Return (byte_count, data_hex, decoded_data) where available.	`raw_hex_to_bytes, str, record.get, len, MODBUS_EXCEPTION_CODES.get`
`anomaly_kind` anomaly_kind(record, threshold_ms=SLOW_RESPONSE_THRESHOLD_MS)	130-153	Classifies a record with explicit priority: bad CRC, timeout, Exception, RAW, then slow response.	`bool, str(record.get('type', '')).startswith, str, record.get, float`
`result_info` result_info(record, successful_request=False, language=LANGUAGE_PORTUGUESE)	168-207	Turns the anomaly/pairing state into a semantic result key, user-facing label and color name.	`anomaly_kind, str, record.get, translate_text`
`inspector_values` inspector_values(record, successful_request=False, language=LANGUAGE_PORTUGUESE)	210-264	Builds every Frame Inspector field, including recalculated CRC received/calculated values, decoded payload, timeout suffix and translated result text.	`crc_fields, response_payload, str, result_info, record.get`
`decoded_record_text` decoded_record_text(record, successful_request=False, language=LANGUAGE_PORTUGUESE)	267-286	Serialises the same Frame Inspector information into newline-separated text for the copy-decoded-frame context command.	`inspector_values, '\n'.join`

Detailed behavior of the key paths

`anomaly_kind` lines 130-153

Classifies a record with explicit priority: bad CRC, timeout, Exception, RAW, then slow response. The priority prevents conflicting colors/results.

`result_info` lines 168-207

Turns the anomaly/pairing state into a semantic result key, user-facing label and color name. Successful completed pairs are distinguished from pending requests.

`inspector_values` lines 210-264

Builds every Frame Inspector field, including recalculated CRC received/calculated values, decoded payload, timeout suffix and translated result text.

`decoded_record_text` lines 267-286

Serialises the same Frame Inspector information into newline-separated text for the copy-decoded-frame context command.

mb_view.py

Traffic view, filtering, sorting, Frame Inspector, Bus Health updates, CSV export and TXT logging. It owns the bounded in-memory frame history, converts parser rows into richer view records, pairs display state with SessionMetrics, maintains table/raw views and keeps the real capture log independent from GUI filtering.

Source lines	SHA-256
1881	`fac08668363fb06679c8d9166b95709debc078b34c9acc3a6790a11b18e5d27d`

Imports: `csv, math, os, subprocess, sys, time, datetime.datetime, tkinter, tkinter.filedialog, messagebox, mb_config.MAX_UI_FRAMES, MAX_RAW_TEXT_LINES, UI_PRUNE_CHUNK_FRAMES, RAW_TEXT_PRUNE_CHUNK_LINES, SLOW_RESPONSE_THRESHOLD_MS, get_application_directory, get_logs_folder, mb_diagnostics.anomaly_kind, decoded_record_text, inspector_values, mb_i18n.canonical_ui_text, translate_details, mb_protocol.read_bits_details`

Module constants / configuration data

Name	Definition
`TRAFFIC_MENU_OPEN_IN_MASTER`	`8`
`TRAFFIC_MENU_OPEN_IN_SLAVE`	`9`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class ViewMixin`	**37-1880**	**Methods for rendering, filtering, statistics and logging.**	**Class container**
`ViewMixin.reset_session_metrics` reset_session_metrics(self)	40-42	Reset session metrics.	`self.metrics.reset, self.update_stats_labels`
`ViewMixin.update_stats_labels` update_stats_labels(self)	44-109	Converts SessionMetrics into both compact session counters and Bus Health values.	`self.stat_requests_var.set, self.stat_responses_var.set, self.stat_pending_var.set, self.stat_timeouts_var.set, self.stat_crc_var.set`
`ViewMixin.pulse_bus_activity` pulse_bus_activity(self)	114-133	Pulse bus activity.	`getattr, var.set, self.after, self.after_cancel`
`ViewMixin._clear_bus_activity_pulse` _clear_bus_activity_pulse(self)	135-139	Clear bus activity pulse.	`getattr, var.set`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.schedule_advanced_filter_refresh` schedule_advanced_filter_refresh(self, *_args)	141-154	Schedules advanced filter refresh.	`getattr, self.after, self.after_cancel`
`ViewMixin.clear_advanced_filters` clear_advanced_filters(self)	156-176	Clear advanced filters.	`getattr, self.advanced_filter_type_var.set, self.advanced_filter_fc_var.set, self.advanced_filter_response_ms_var.set, self.advanced_filter…`
`ViewMixin.normalized_fc_filter` normalized_fc_filter(value)	179-191	Normalized fc filter.	`str(value or '').strip().upper, text.startswith, int, str(value or '').strip, str`
`ViewMixin.record_passes_advanced_filters` record_passes_advanced_filters(self, record)	193-254	Implements Type, FC, response-time and free-text filtering.	`getattr, canonical_ui_text, str, self.normalized_fc_filter, self.advanced_filter_response_ms_var.get().strip`
`ViewMixin.configure_anomaly_tags` configure_anomaly_tags(self, dark=None)	256-287	Configure anomaly tags.	`palette.items, hasattr, bool, self.tree.tag_configure, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get`
`ViewMixin.anomaly_tag_for_record` anomaly_tag_for_record(self, record)	289-326	Maps the semantic result of a record to the Treeview tag used for colour highlighting, including completed normal REQUEST/RESPONSE pairs.	`anomaly_kind, str, getattr, self.highlight_anomalies_var.get, record.get`
`ViewMixin.on_highlight_anomalies_changed` on_highlight_anomalies_changed(self)	328-335	Event handler for highlight anomalies changed.	`self.configure_anomaly_tags, self.configure_inspector_result_styles, self.render_filtered_view, self.update_frame_inspector, getattr`
`ViewMixin.mark_timed_out_records` mark_timed_out_records(self, expired, rerender=True)	337-363	Finds original REQUEST records by their sequence IDs and marks them timed_out rather than inventing artificial timeout frames.	`len, item.get, changed_records.append, canonical_ui_text, self.render_filtered_view`
`ViewMixin.refresh_record_tags` refresh_record_tags(self, records)	365-377	Re-applies the result tag to the Treeview rows of the given records and refreshes the Frame Inspector when one of them is selected.	`self.selected_record, self.anomaly_tag_for_record, self._tree_iids_by_seq.get, any, self.update_frame_inspector`
`ViewMixin.selected_record` selected_record(self)	379-383	Selected record.	`self.tree.selection, self._record_by_iid.get`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.update_frame_inspector` update_frame_inspector(self, record=None)	385-414	Builds the structured inspector values for the selected traffic record and applies result styling while preserving the raw frame.	`bool, inspector_values, self.inspector_vars.items, self.selected_raw_var.set, getattr`
`ViewMixin.toggle_frame_inspector` toggle_frame_inspector(self)	416-428	Toggle frame inspector.	`getattr, callable, self.inspector_body.grid, self.inspector_toggle_btn.configure, self.inspector_body.grid_remove`
`ViewMixin.copy_to_clipboard` copy_to_clipboard(self, text)	430-436	Copy to clipboard.	`self.clipboard_clear, self.clipboard_append, self.update_idletasks, str`
`ViewMixin.copy_selected_raw` copy_selected_raw(self)	438-441	Copy selected raw.	`self.selected_record, self.copy_to_clipboard, record.get`
`ViewMixin.copy_selected_decoded` copy_selected_decoded(self)	443-456	Copy selected decoded.	`self.selected_record, bool, self.copy_to_clipboard, decoded_record_text, record.get`
`ViewMixin.context_filter_selected_slave` context_filter_selected_slave(self)	458-471	Context filter selected slave.	`self.selected_record, str, self.slave_filter_vars.items, self.update_slave_heading, self.render_filtered_view`
`ViewMixin.context_filter_selected_fc` context_filter_selected_fc(self)	473-487	Context filter selected fc.	`self.selected_record, str, self.advanced_filter_fc_var.set, self.render_filtered_view, record.get`
`ViewMixin.context_show_selected_transaction` context_show_selected_transaction(self)	489-518	Context show selected transaction.	`self.selected_record, record.get, self.slave_filter_vars.values, self.update_slave_heading, self.advanced_filter_type_var.set`
`ViewMixin.clear_transaction_filter` clear_transaction_filter(self)	520-522	Clear transaction filter.	`self.render_filtered_view`
`ViewMixin.context_open_selected_in_master` context_open_selected_in_master(self)	524-527	Context open selected in master.	`self.selected_record, self.master_open_from_record`
`ViewMixin.context_open_selected_in_slave` context_open_selected_in_slave(self)	529-532	Context open selected in slave.	`self.selected_record, self.slave_open_from_record`
`ViewMixin.show_traffic_context_menu` show_traffic_context_menu(self, event)	534-562	Show traffic context menu.	`self.tree.identify_row, self.tree.selection_set, self.tree.focus, self.on_row_select, callable`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.export_session_csv` export_session_csv(self, path=None)	564-666	Exports the frames retained by the GUI history, including timing, decoded addressing, raw data and anomaly/timeout fields.	`list, datetime.now().strftime, filedialog.asksaveasfilename, messagebox.showinfo, open`
`ViewMixin.tree_record_column_values` tree_record_column_values(self, record)	668-679	Tree record column values.	`record.get, translate_details`
`ViewMixin.measure_tree_column_width` measure_tree_column_width(self, column, records, heading_text=None)	681-699	Measure the pixel width required by a non-wrapped Treeview column.	`self.tree_heading_font.measure, self.tree_headings.get, str, self.tree_record_column_values(record).get, max`
`ViewMixin.available_details_width` available_details_width(self)	701-719	Return the width that can be assigned to Details without intentionally forcing horizontal scrolling in the current Treeview viewport.	`int, max, self.tree.winfo_width, self.tree.column`
`ViewMixin.wrap_text_to_pixels` wrap_text_to_pixels(self, value, max_px)	721-771	Word-wrap text using the actual Tk font width.	`str, max, '\n'.join, int, text.splitlines`
`ViewMixin.details_horizontal_rule` details_horizontal_rule(self)	773-785	Return a rule that fills the Details column as far as the Treeview cell permits.	`max, int, self.tree_cell_font.measure, self.tree.column, math.ceil`
`ViewMixin.completed_pairs_for_records` completed_pairs_for_records(self, records)	787-827	Return complete visible REQUEST/RESPONSE pairs in current display order.	`pairs.sort, record.get, response.get, seq_to_index.get, min`
`ViewMixin.separator_after_sequences` separator_after_sequences(self, records)	829-846	Return logical frame seq ids that need a separator AFTER them.	`self.completed_pairs_for_records, self.separate_transactions_var.get, set, len`
`ViewMixin.tree_separator_values` tree_separator_values(self)	848-853	Tree separator values.	`self.details_horizontal_rule`
`ViewMixin.insert_tree_separator_after_seq` insert_tree_separator_after_seq(self, seq)	855-873	Insert one Details-width separator after the visual rows of seq.	`self._tree_iids_by_seq.get, self.tree.insert, self.tree.index, self.tree_separator_values`
`ViewMixin.sync_tree_transaction_separators` sync_tree_transaction_separators(self, records=None)	875-895	Synchronize standalone transaction separator rows without rebuilding normal traffic rows.	`self.separator_after_sequences, set, self.records_for_display, self._tree_separator_iid_by_seq.pop, self._raw_by_iid.pop`
`ViewMixin.detail_lines_for_record` detail_lines_for_record(self, record)	897-919	Return wrapped Details as separate visual Treeview rows.	`max, self.wrap_text_to_pixels, wrapped.splitlines, translate_details, int`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.set_tree_rowheight_for_records` set_tree_rowheight_for_records(self, records)	921-926	Treeview row height stays fixed; wrapping uses continuation rows.	`self.tree_style.configure`
`ViewMixin.update_tree_rowheight_for_record` update_tree_rowheight_for_record(self, record)	928-930	No-op by design: one logical frame may use several compact rows.	—
`ViewMixin.autofit_tree_columns` autofit_tree_columns(self, records=None)	932-975	Non-Details columns fit the longest visible value/title.	`self.measure_tree_column_width, max, self.tree.column, self.set_tree_rowheight_for_records, self.records_for_display`
`ViewMixin.expand_tree_columns_for_record` expand_tree_columns_for_record(self, record)	977-1027	Incrementally enlarge only the compact non-Details columns.	`self.tree_record_column_values, self.tree_heading_font.measure, self.tree_cell_font.measure, int, self.measure_tree_column_width`
`ViewMixin.refresh_tree_heading_width` refresh_tree_heading_width(self, column)	1029-1045	Dynamic Hora/Slave headings remain compact.	`self.records_for_display, self.measure_tree_column_width, self.tree.column`
`ViewMixin.schedule_tree_reflow` schedule_tree_reflow(self, _event=None)	1047-1061	Debounce window/Treeview resizing and then re-wrap Details to the new available width.	`self.after, self.after_cancel`
`ViewMixin.reflow_tree_after_resize` reflow_tree_after_resize(self)	1063-1067	Reflow tree after resize.	`self.render_filtered_view, hasattr`
`ViewMixin._slave_sort_key` _slave_sort_key(self, slave)	1069-1073	Slave sort key.	`int, str`
`ViewMixin.rebuild_slave_filter_menu` rebuild_slave_filter_menu(self)	1075-1102	Rebuild slave filter menu.	`self.slave_filter_menu.delete, self.slave_filter_menu.add_command, self.update_slave_heading, self.slave_filter_menu.add_separator, sorted`
`ViewMixin.update_slave_heading` update_slave_heading(self)	1104-1121	Update slave heading.	`self.tree.heading, self.refresh_tree_heading_width, self.tr, self.slave_filter_vars.items, var.get`
`ViewMixin.show_slave_filter_menu` show_slave_filter_menu(self)	1123-1136	Open the slave filter menu at the position where the Slave heading was clicked.	`self.winfo_pointerx, self.slave_filter_menu.tk_popup, self.winfo_pointery, self.slave_filter_menu.grab_release`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.register_slave_for_filter` register_slave_for_filter(self, slave)	1138-1149	Register slave for filter.	`str(slave or '').strip, tk.BooleanVar, self.rebuild_slave_filter_menu, str`
`ViewMixin.select_all_slaves` select_all_slaves(self)	1151-1156	Select all slaves.	`self.slave_filter_vars.values, self.update_slave_heading, self.render_filtered_view, var.set`
`ViewMixin.select_no_slaves` select_no_slaves(self)	1158-1163	Select no slaves.	`self.slave_filter_vars.values, self.update_slave_heading, self.render_filtered_view, var.set`
`ViewMixin.on_slave_filter_changed` on_slave_filter_changed(self)	1165-1175	Event handler for slave filter changed.	`self.update_slave_heading, self.render_filtered_view, var.get, bool, all`
`ViewMixin.selected_slave_ids` selected_slave_ids(self)	1177-1181	Selected slave ids.	`self.slave_filter_vars.items, var.get`
`ViewMixin.record_passes_slave_filter` record_passes_slave_filter(self, record)	1183-1193	Record passes slave filter.	`self.selected_slave_ids, str, record.get`
`ViewMixin.time_order_descending` time_order_descending(self)	1195-1196	Time order descending.	`canonical_ui_text(self.time_order_var.get()).startswith, canonical_ui_text, self.time_order_var.get`
`ViewMixin.on_time_order_changed` on_time_order_changed(self)	1198-1207	Event handler for time order changed.	`self.tree.heading, self.render_filtered_view, self.time_order_descending, self.tr`
`ViewMixin.toggle_time_order` toggle_time_order(self)	1209-1214	Toggle time order.	`self.time_order_descending, self.on_time_order_changed, self.time_order_var.set, self.tr`
`ViewMixin.records_for_display` records_for_display(self)	1216-1226	Applies current filtering and time-order state to the bounded source-of-truth frame history.	`self.time_order_descending, records.reverse, self.record_passes_slave_filter, self.record_passes_advanced_filters`
`ViewMixin.raw_line_for_record` raw_line_for_record(self, record)	1228-1251	Raw line for record.	`record.get, translate_details`
`ViewMixin.raw_separator_width_chars` raw_separator_width_chars(self, raw_lines=None)	1253-1273	Width of one Raw Hex separator in visible character cells.	`max, self.raw_text_font.measure, int, self.raw_text.winfo_width`
`ViewMixin.raw_transaction_separator` raw_transaction_separator(self, width_chars=None)	1275-1278	Raw transaction separator.	`max, int`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.schedule_raw_separator_reflow` schedule_raw_separator_reflow(self, _event=None)	1280-1290	Schedules raw separator reflow.	`self.after, self.after_cancel`
`ViewMixin.reflow_raw_separator_after_resize` reflow_raw_separator_after_resize(self)	1292-1295	Reflow raw separator after resize.	`self.separate_transactions_var.get, self.render_raw_text_only`
`ViewMixin.insert_tree_record` insert_tree_record(self, record, index='end', allow_separator=True)	1297-1365	Insert tree record.	`self.detail_lines_for_record, visual_rows.append, self.anomaly_tag_for_record, list, self.tree.insert`
`ViewMixin.render_raw_text_only` render_raw_text_only(self)	1367-1420	Render raw text only.	`self.records_for_display, self.separator_after_sequences, list, self.raw_separator_width_chars, self.raw_transaction_separator`
`ViewMixin.render_filtered_view` render_filtered_view(self)	1422-1482	Render filtered view.	`self.selected_record, self.tree.get_children, self._tree_history.clear, self._tree_iids_by_seq.clear, self._tree_separator_iid_by_seq.clear`
`ViewMixin.add_frame_row` add_frame_row(self, row)	1484-1486	Add frame row.	`self.add_frame_rows_batch`
`ViewMixin.add_frame_rows_batch` add_frame_rows_batch(self, rows)	1488-1686	Central view ingestion path.	`self.pulse_bus_activity, ''.join, self.update_stats_labels, self.prune_ui_history, self.metrics.process`
`ViewMixin.prune_ui_history` prune_ui_history(self)	1688-1711	Keeps memory/UI work bounded by removing old records in chunks after a hysteresis threshold.	`self._successful_request_seqs.intersection_update, self.render_filtered_view, len, self._frame_history.popleft, record.get`
`ViewMixin.on_row_select` on_row_select(self, _event=None)	1713-1715	Event handler for row select.	`self.selected_record, self.update_frame_inspector`
`ViewMixin.finish_capture_ui` finish_capture_ui(self)	1717-1730	Finish capture ui.	`self.status_var.set, self.start_btn.configure, self.stop_btn.configure, self.restart_btn.configure, self.set_simulation_button_state`
`ViewMixin.app_directory` app_directory(self)	1732-1733	App directory.	`get_application_directory`
`ViewMixin.prepare_log_session` prepare_log_session(self, prefix, started_at, header)	1735-1748	Prepare a real-capture log session without creating a file.	`self.close_log, started_at.strftime`

Callable	Lines	Responsibility	Representative direct calls
`ViewMixin.ensure_log_started` ensure_log_started(self)	1750-1789	Creates the TXT capture log lazily only when real traffic actually arrives, preventing empty logs from start/stop operations with no data.	`target.open, get_logs_folder, self.log_file.write, messagebox.showerror, ''.join`
`ViewMixin.queue_log_note` queue_log_note(self, text)	1791-1799	Write a capture note to an already-open log, or keep it in memory until the first traffic frame creates the log.	`self.log_line, self.pending_log_notes.append`
`ViewMixin.log_line` log_line(self, text)	1801-1806	Log line.	`self.log_file.write`
`ViewMixin.close_log` close_log(self)	1808-1823	Close log.	`self.log_file.write, self.log_file.close, datetime.now().isoformat, datetime.now`
`ViewMixin.open_log_folder` open_log_folder(self)	1825-1835	Open log folder.	`get_logs_folder, os.startfile, messagebox.showerror, str, subprocess.Popen`
`ViewMixin.clear_view` clear_view(self)	1837-1880	Clear view.	`self.tree.get_children, self.raw_text.delete, self.selected_raw_var.set, self._tree_history.clear, self._tree_iids_by_seq.clear`

Detailed behavior of the key paths

`update_stats_labels` lines 44-109

Converts SessionMetrics into both compact session counters and Bus Health values. It recalculates health using the currently configured serial parameters so approximate bus utilisation reflects the selected format.

`record_passes_advanced_filters` lines 193-254

Implements Type, FC, response-time and free-text filtering. Filtering is display-only; it does not alter SessionMetrics or the on-disk capture log.

`anomaly_tag_for_record` lines 289-326

Maps the semantic result of a record to the Treeview tag used for colour highlighting, including completed normal REQUEST/RESPONSE pairs.

`mark_timed_out_records` **lines 337-363**

Finds original REQUEST records by their sequence IDs and marks them timed_out rather than inventing artificial timeout frames. Visible rows are retagged in place; the table is rebuilt only while the Timeouts filter is active.

`refresh_record_tags` **lines 365-377**

Re-applies the result tag to the Treeview rows of the given records and refreshes the Frame Inspector when one of them is selected.

`update_frame_inspector` **lines 385-414**

Builds the structured inspector values for the selected traffic record and applies result styling while preserving the raw frame.

`export_session_csv` **lines 564-666**

Exports the frames retained by the GUI history, including timing, decoded addressing, raw data and anomaly/timeout fields. This differs intentionally from the complete TXT capture log.

`records_for_display` **lines 1216-1226**

Applies current filtering and time-order state to the bounded source-of-truth frame history.

`add_frame_rows_batch` **lines 1488-1686**

Central view ingestion path. It assigns stable sequence IDs, runs SessionMetrics, enriches parser rows into view records (including the matched request address/quantity for read responses), tracks successful pairs, writes complete real traffic to disk, updates visible table/raw views, prunes history and auto-scrolls.

`prune_ui_history` **lines 1688-1711**

Keeps memory/UI work bounded by removing old records in chunks after a hysteresis threshold. The disk log is independent and remains complete.

`ensure_log_started` **lines 1750-1789**

Creates the TXT capture log lazily only when real traffic actually arrives, preventing empty logs from start/stop operations with no data.

> **Design note.** There are deliberately two histories: the bounded GUI history and the complete TXT log. Sorting/filtering only changes presentation; statistics are derived before filtering.

mb_slave_finder.py

Active Modbus RTU Slave discovery and Device Identification. It checks that the bus is idle, builds safe read-only FC03/FC04 probes with configurable attempts, calculates adaptive timeouts, performs robust serial writes/reads, accepts normal or CRC-valid Exception responses as proof of a Slave, optionally reads FC43/14 Basic Device Identification and drives the Finder UI through queued events.

Source lines	SHA-256
1625	`f192a6305486ea73411d920d9b6b9de19091bc612afef60ea3f5871fcd66938b`

Imports: `threading, time, tkinter, tkinter.ttk, messagebox, mb_config.DISCOVERY_BAUD_RATES, DISCOVERY_PARITIES, DISCOVERY_STOP_BITS, DISCOVERY_DEFAULT_MIN_TIMEOUT_MS, DISCOVERY_MIN_TIMEOUT_MS, DISCOVERY_MAX_TIMEOUT_MS, DISCOVERY_PROBE_ADDRESS, DISCOVERY_PROBE_QTY, DISCOVERY_WRITE_TIMEOUT_SECONDS, DISCOVERY_WRITE_RETRIES, DISCOVERY_RX_BUFFER_LIMIT, DISCOVERY_MIN_ATTEMPTS, DISCOVERY_MAX_ATTEMPTS, DISCOVERY_DEFAULT_ATTEMPTS, DISCOVERY_IDLE_CHECK_SECONDS, MODBUS_EXCEPTION_CODES, mb_protocol.append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds, mb_widgets.RefreshButton`

Module constants / configuration data

Name	Definition
`DEVICE_ID_FC`	`43`
`DEVICE_ID_MEI_TYPE`	`14`
`DEVICE_ID_READ_BASIC`	`1`
`DEVICE_ID_MORE_FOLLOWS`	`255`
`DEVICE_ID_BASIC_OBJECTS`	`(0, 1, 2)`
`DEVICE_ID_MAX_PAGES`	`8`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`build_discovery_probe` build_discovery_probe(slave, fc)	35-48	Constructs a read-only FC03/FC04 request for Address 0, Quantity 1 and appends the RTU CRC.	`bytes, append_crc, ValueError, int`

Callable	Lines	Responsibility	Representative direct calls
`build_device_identification_probe` build_device_identification_probe(slave, object_id=0)	57-71	Builds FC43 / MEI type 14 / Basic Device Identification requests, beginning at a selectable Object ID.	`int, append_crc, ValueError, bytes`
`_decode_device_id_value` _decode_device_id_value(value)	73-75	Decode a Modbus Device Identification ASCII object for display.	`bytes(value).decode('ascii', errors='replace').strip, bytes(value).decode, bytes`
`find_device_identification_response` find_device_identification_response(buffer, slave)	77-158	Parses FC43/14 responses and Exceptions from a receive buffer, validates CRC, decodes variable-length ASCII objects and exposes More Follows / Next Object ID for paged identification.	`bytes, int, range, max, len`
`read_device_identification_response` read_device_identification_response(ser, slave, timeout_s, stop_event)	160-182	Read until a valid FC43/14 response/exception is received or timed out.	`bytearray, find_device_identification_response, time.perf_counter, max, stop_event.is_set`
`device_identification_timeout_seconds` device_identification_timeout_seconds(baud, parity, stopbits, minimum_timeout_ms=DISCOVERY_DEFAULT_MIN_TIMEOUT_MS)	184-201	Conservative timeout for one FC43/14 response, up to one RTU frame.	`max, discovery_bits_per_char, int, float, modbus_rtu_interframe_gap_seconds`
`device_identification_request_transmit_seconds` device_identification_request_transmit_seconds(baud, parity, stopbits)	203-207	Estimated on-wire duration of the 7-byte FC43/14 request.	`float, discovery_bits_per_char, max, int`
`format_device_identification` format_device_identification(result)	209-224	Return compact Device Identification text for the Finder result table.	`any, result.get, int, objects.get, ' | '.join`
`read_basic_device_identification` read_basic_device_identification(ser, slave, baud, parity, stopbits, minimum_timeout_ms, stop_event, serial_module)	226-301	Repeats FC43/14 requests while More Follows is set, merging objects until VendorName, ProductCode and MajorMinorRevision are available or the paging limit/response terminates.	`range, stop_event.is_set, build_device_identification_probe, write_discovery_request, read_device_identification_response`
`discovery_bits_per_char` discovery_bits_per_char(parity, stopbits)	303-307	Return serial bits/character: start + 8 data + parity + stop.	`float`
`discovery_response_timeout_seconds` discovery_response_timeout_seconds(baud, parity, stopbits, minimum_timeout_ms=DISCOVERY_DEFAULT_MIN_TIMEOUT_MS)	309-330	Calculates a response deadline from serial character time, the expected 7-byte FC03/FC04 response and an inter-frame/processing margin, never below the user-configured minimum.	`max, discovery_bits_per_char, int, float, modbus_rtu_interframe_gap_seconds`

Callable	Lines	Responsibility	Representative direct calls
`discovery_request_transmit_seconds` discovery_request_transmit_seconds(baud, parity, stopbits)	332-334	Estimated on-wire duration of the fixed 8-byte discovery request.	`float, discovery_bits_per_char, max, int`
`find_discovery_response` find_discovery_response(buffer, slave, fc)	336-379	Scans received bytes for the requested Slave and Function Code.	`bytes, int, range, max, len`
`read_discovery_response` read_discovery_response(ser, slave, fc, timeout_s, stop_event)	381-407	Read non-blocking serial bytes until a valid probe response or timeout.	`bytearray, find_discovery_response, time.perf_counter, max, stop_event.is_set`
`bus_activity_detected` bus_activity_detected(ser, listen_s, stop_event)	409-427	Listens passively for the idle-check period and returns any bytes received; a non-empty result means another device is transmitting and the scan must not start.	`bytearray, bytes, time.perf_counter, max, stop_event.is_set`
`cancel_serial_io` cancel_serial_io(ser)	430-440	Best-effort cancellation of pending serial I/O.	`getattr, callable, method`
`recover_discovery_write` recover_discovery_write(ser)	442-457	Best-effort cleanup after a discovery write timeout.	`getattr, callable, ser.reset_output_buffer, ser.reset_input_buffer, cancel`
`write_discovery_request` write_discovery_request(ser, request, baud, parity, stopbits, stop_event, serial_module)	459-523	Transmits one probe without using ser.flush(), retries one write timeout after driver/buffer recovery, waits for estimated on-wire transmission time and raises actual driver errors instead of disguising them as no-response Slaves.	`bytes, getattr, discovery_bits_per_char, max, range`
`format_duration` format_duration(seconds)	525-531	Format duration.	`max, divmod, int, round, float`
`class SlaveFinderMixin`	**534-1624**	**Methods that implement the active Bus Slave Finder tab.**	**Class container**
`SlaveFinderMixin.build_discovery_tab` build_discovery_tab(self, page)	537-921	Build the full-window active Modbus RTU slave-discovery workspace.	`getattr(self, 'ui_settings', {}).get, set, page.columnconfigure, page.rowconfigure, ttk.LabelFrame`
`SlaveFinderMixin._show_discovery_context_menu` _show_discovery_context_menu(self, event)	923-932	Show discovery context menu.	`self.discovery_tree.identify_row, self.discovery_tree.selection_set, self.discovery_context_menu.entryconfigure, self.discovery_context_men…`

Callable	Lines	Responsibility	Representative direct calls
`SlaveFinderMixin.discovery_open_selected_in_master` discovery_open_selected_in_master(self)	934-950	Open the selected found Slave in a new Master window.	`self.discovery_tree.selection, self.discovery_tree.item, {'N': 'None', 'E': 'Even', 'O': 'Odd'}.get, self.master_open_definition, int`
`SlaveFinderMixin.fit_discovery_columns_to_viewport` fit_discovery_columns_to_viewport(self, _event=None)	952-981	Fit Slave Finder columns to the visible Treeview width.	`int, max, sum, len, enumerate`
`SlaveFinderMixin.on_discovery_options_changed` on_discovery_options_changed(self)	983-986	Event handler for discovery options changed.	`hasattr, self.update_discovery_estimate, self.update_discovery_start_state`
`SlaveFinderMixin.selected_discovery_configs` selected_discovery_configs(self)	988-997	Selected discovery configs.	`self.discovery_baud_vars[baud].get, self.discovery_parity_vars[p].get, self.discovery_stop_vars[s].get`
`SlaveFinderMixin.selected_discovery_slaves` selected_discovery_slaves(self)	999-1006	Selected discovery slaves.	`int, list, self.discovery_slave_start_var.get, self.discovery_slave_end_var.get, ValueError`
`SlaveFinderMixin.selected_discovery_min_timeout_ms` selected_discovery_min_timeout_ms(self)	1008-1018	Selected discovery min timeout ms.	`float, self.discovery_min_timeout_var.get, ValueError, self.trf`
`SlaveFinderMixin.selected_discovery_attempts` selected_discovery_attempts(self)	1020-1030	Selected discovery attempts.	`int, self.discovery_attempts_var.get, ValueError, self.trf`
`SlaveFinderMixin.discovery_config_label` discovery_config_label(self, parity, stopbits)	1032-1034	Discovery config label.	—
`SlaveFinderMixin.update_discovery_estimate` update_discovery_estimate(self)	1036-1093	Update discovery estimate.	`bool, self.discovery_progress_text_var.set, self.discovery_estimate_var.set, self.selected_discovery_configs, self.selected_discovery_slaves`
`SlaveFinderMixin.update_discovery_start_state` update_discovery_start_state(self)	1095-1111	Update discovery start state.	`self.discovery_start_btn.configure, hasattr, bool, self.selected_discovery_slaves, self.selected_discovery_min_timeout_ms`

Callable	Lines	Responsibility	Representative direct calls
`SlaveFinderMixin.clear_discovery_results` clear_discovery_results(self)	1113-1122	Clear discovery results.	`self.discovery_tree.get_children, self.discovery_found_var.set, self.discovery_progress_var.set, self.discovery_status_var.set, self.update…`
`SlaveFinderMixin.open_discovery_serial_port` open_discovery_serial_port(self, serial, port, baud, parity, stopbits)	1124-1162	Open discovery serial port.	`serial.Serial, int, ser.open, ser.reset_input_buffer, ser.reset_output_buffer`
`SlaveFinderMixin.start_discovery_scan` start_discovery_scan(self)	1164-1255	Validates the safety confirmation, COM selection, Slave range, attempts, selected serial configurations and global busy state.	`self.get_serial_modules, bool, self.clear_discovery_results, self.discovery_stop_event.clear, self.discovery_progress_var.set`
`SlaveFinderMixin.stop_discovery_scan` stop_discovery_scan(self)	1257-1264	Stop discovery scan.	`self.discovery_status_var.set, self.discovery_stop_event.set, self.discovery_stop_btn.configure, cancel_serial_io, self.tr`
`SlaveFinderMixin.discovery_worker` discovery_worker(self, serial, port, configs, slaves, minimum_timeout_ms, fallback_fc04, identify_devices, attempts=DISCOVERY_DEFAULT_ATTEMPTS)	1266-1545	Iterates selected serial configurations and Slave IDs.	`time.perf_counter, str, self.event_queue.put, len, self.discovery_stop_event.is_set`
`SlaveFinderMixin.apply_discovery_progress` apply_discovery_progress(self, payload)	1547-1562	Apply discovery progress.	`int, max, self.discovery_progress_var.set, self.discovery_progress_text_var.set, self.discovery_found_var.set`
`SlaveFinderMixin.add_discovery_result` add_discovery_result(self, payload)	1564-1581	Add discovery result.	`self.discovery_found_var.set, self.discovery_tree.insert, self.trf, payload.get`
`SlaveFinderMixin.finish_discovery_ui` finish_discovery_ui(self, payload)	1583-1624	Returns both Finder and Sniffer controls to idle state and presents completed/stopped/error status without leaving the application locked in a busy state.	`self.discovery_stop_event.clear, self.discovery_stop_btn.configure, self.discovery_clear_btn.configure, self.discovery_refresh_btn.configur…`

Detailed behavior of the key paths

`build_discovery_probe` lines 35-48

Constructs a read-only FC03/FC04 request for Address 0, Quantity 1 and appends the RTU CRC. Input validation restricts Slave IDs to 1..247 and FC to 03/04.

`find_discovery_response` lines 336-379

Scans received bytes for the requested Slave and Function Code. A CRC-valid normal register response or CRC-valid Modbus Exception is accepted; echoes and unrelated traffic are ignored.

`discovery_response_timeout_seconds` lines 309-330

Calculates a response deadline from serial character time, the expected 7-byte FC03/FC04 response and an inter-frame/processing margin, never below the user-configured minimum.

`bus_activity_detected` lines 409-427

Listens passively for the idle-check period and returns any bytes received; a non-empty result means another device is transmitting and the scan must not start.

`write_discovery_request` lines 459-523

Transmits one probe without using ser.flush(), retries one write timeout after driver/buffer recovery, waits for estimated on-wire transmission time and raises actual driver errors instead of disguising them as no-response Slaves.

`build_device_identification_probe` lines 57-71

Builds FC43 / MEI type 14 / Basic Device Identification requests, beginning at a selectable Object ID.

`find_device_identification_response` lines 77-158

Parses FC43/14 responses and Exceptions from a receive buffer, validates CRC, decodes variable-length ASCII objects and exposes More Follows / Next Object ID for paged identification.

`read_basic_device_identification` lines 226-301

Repeats FC43/14 requests while More Follows is set, merging objects until VendorName, ProductCode and MajorMinorRevision are available or the paging limit/response terminates.

`start_discovery_scan` lines 1164-1255

Validates the safety confirmation, COM selection, Slave range, attempts, selected serial configurations and global busy state. It disables conflicting controls and launches the discovery worker.

`discovery_worker` **lines 1266-1545**

Iterates selected serial configurations and Slave IDs. It opens one configuration at a time, checks once that the bus is idle, probes FC03 then optional FC04 with the selected attempts, optionally performs Device Identification for found Slaves, publishes progress/results, and always closes the active serial handle.

`finish_discovery_ui` **lines 1583-1624**

Returns both Finder and Sniffer controls to idle state and presents completed/stopped/error status without leaving the application locked in a busy state.

Design note. The Finder is active traffic generation, unlike the passive Sniffer. The UI requires an explicit confirmation that another master is not active and the global busy lock prevents simultaneous capture/scan inside this application.

mb_master.py

Modbus RTU master engine, frames and data formats. It builds FC01-04 read and FC05/06/15/16 write requests, matches responses and exceptions, converts registers to and from 16/32-bit formats in four byte orders and runs MasterEngine, which serves every poll window over one serial port one request at a time. It contains no widgets.

Source lines	SHA-256
695	`e4e956940449cfcf8caa059ca550184af0298d32f72729c7f5376c0ec431604f`

Imports: `math, struct, threading, time, collections.deque, dataclasses.dataclass, mb_config.MASTER_RECONNECT_INTERVAL_SECONDS, MASTER_MAX_READ_BITS, MASTER_MAX_READ_REGISTERS, MASTER_MAX_WRITE_BITS, MASTER_MAX_WRITE_REGISTERS, mb_protocol.append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds, mb_slave_finder.bus_activity_detected, cancel_serial_io, discovery_bits_per_char, write_discovery_request`

Module constants / configuration data

Name	Definition
`READ_FUNCTION_CODES`	`(1, 2, 3, 4)`
`BIT_FUNCTION_CODES`	`(1, 2)`
`WRITABLE_READ_FCS`	`{1: (5, 15), 3: (6, 16)}`
`BYTE_ORDERS`	`('ABCD', 'CDAB', 'BADC', 'DCBA')`
`BYTE_ORDERS_64`	`('ABCDEFGH', 'GHEFCDAB', 'BADCFEHG', 'HGFEDCBA')`
`WIRE_LETTERS`	`'ABCDEFGH'`
`REGISTER_FORMATS`	`(('int16', 'Signed 16-bit', 1), ('uint16', 'Unsigned 16-bit', 1), ('hex16', 'Hex 16-bit', 1), ('bin16', 'Binary 16-bit', 1)) + tuple(((f'{kind}_{or…`
`FORMAT_LABELS`	`{key: label for key, label, _size in REGISTER_FORMATS}`
`FORMAT_KEYS_BY_LABEL`	`{label: key for key, label, _size in REGISTER_FORMATS}`
`FORMAT_SIZES`	`{key: size for key, _label, size in REGISTER_FORMATS}`
`DEFAULT_FORMAT`	`'int16'`
`_STRUCT_CODES`	`{'int32': '>i', 'uint32': '>I', 'float32': '>f', 'int64': '>q', 'uint64': '>Q', 'float64': '>d'}`

Name	Definition
`SCALE_MAX_DECIMALS`	`6`
`SCALE_MAX_UNIT_LENGTH`	`12`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`_value_bytes` _value_bytes(registers, order)	81-84	Reorder the wire bytes of 2 or 4 registers into big-endian value bytes.	`b''.join, bytes, WIRE_LETTERS.index`
`_registers_from_value_bytes` _registers_from_value_bytes(value, order)	87-91	Registers from value bytes.	`bytearray, enumerate, len, WIRE_LETTERS.index, range`
`format_float` format_float(value, digits=7)	94-99	Format float.	`math.isnan, math.isinf`
`decode_value` decode_value(registers, fmt)	102-110	Numeric value of one value (None for the Hex/Binary display formats).	`fmt.split, struct.unpack, _value_bytes`
`validate_scale` validate_scale(raw_min, raw_max, eng_min, eng_max, decimals=1, unit='')	117-133	Return a scale dict from user text/numbers; raise ValueError (PT text).	`any, int, ValueError, str(unit or '').strip, float`
`scale_value` scale_value(number, scale)	136-139	Linear map from raw units to engineering units through two points.	—
`unscale_value` unscale_value(value, scale)	142-147	Inverse of scale_value(): engineering units back to raw units.	`ValueError`
`format_scaled` format_scaled(number, scale)	150-152	Format scaled.	`scale.get, scale_value`
`format_value` format_value(registers, fmt)	155-173	Display text of one value occupying ``FORMAT_SIZES[fmt]`` registers.	`fmt.split, str, struct.unpack, format_float, _value_bytes`
`register_groups` register_groups(qty, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None)	176-205	Split a window's registers into values: [(offset, size, fmt or None)].	`overrides.get, groups.append, next, range, min`
`format_window_values` format_window_values(registers, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None, scales=None)	208-233	Return one display string per register, honouring per-address formats.	`register_groups, len, scales.get, out.extend, decode_value`

Callable	Lines	Responsibility	Representative direct calls
`raw_text_for_scaled` raw_text_for_scaled(value_text, scale, fmt)	236-245	Convert an engineering value typed by the user into raw text for *fmt*.	`unscale_value, fmt.split('_')[0].startswith, str, float, repr`
`format_register_values` format_register_values(registers, fmt)	248-250	Turns raw registers into display strings for the selected format; 32-bit values are shown on the first register of each pair and an unpaired last register shows a dash.	`format_window_values`
`_parse_int` _parse_int(text)	253-260	Parse int.	`str(text).strip().replace(' ', '').replace, text.lower, lowered.startswith, int, str(text).strip().replace`
`encode_register_value` encode_register_value(text, fmt)	263-307	Converts one user-entered value into the register(s) it occupies in the selected format and byte order, rejecting values that do not fit.	`str(text).strip, ValueError, fmt.split, kind.startswith, _registers_from_value_bytes`
`encode_coil_value` encode_coil_value(text)	310-316	Encode coil value.	`str(text).strip().lower, ValueError, str(text).strip, str`
`_u16` _u16(value)	323-324	U16.	`bytes`
`read_limit` read_limit(fc)	327-328	Read limit.	—
`validate_read_definition` validate_read_definition(slave, fc, address, qty)	331-342	Raise ValueError (PT canonical message) for an invalid poll definition.	`ValueError, int, read_limit`
`build_read_request` build_read_request(slave, fc, address, qty)	345-347	Build read request.	`validate_read_definition, append_crc, _u16, bytes, int`
`build_write_request` build_write_request(slave, fc, address, values)	350-381	Builds FC05/06/15/16 requests within the Modbus write limits; Slave 0 is accepted for broadcast.	`append_crc, int, ValueError, len, _u16`
`expected_response_length` expected_response_length(request)	384-392	Expected response length.	—
`find_master_response` find_master_response(buffer, request)	399-447	Finds the CRC-valid response of the expected length from the addressed Slave, or a Modbus exception, and decodes read values.	`bytes, expected_response_length, range, len, crc_ok`
`class PollDefinition`	**455-465**	**PollDefinition.**	**Class container**

Callable	Lines	Responsibility	Representative direct calls
`PollDefinition.request` request(self)	464-465	Request.	`build_read_request`
`class MasterEngine`	**468-695**	**Serve all poll windows over one serial port, one transaction at a time.**	**Class container**
`MasterEngine.__init__` __init__(self, ser, event_queue, *, baud, parity, stopbits, response_timeout_ms, delay_ms, serial_module=None, idle_check_seconds=0.0, reopen=None, reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS, event_prefix='master')	486-513	Init.	`float, int, max, modbus_rtu_interframe_gap_seconds, threading.Lock`
`MasterEngine.set_poll` set_poll(self, definition)	516-519	Set poll.	`self._wake.set`
`MasterEngine.remove_poll` remove_poll(self, window_id)	521-524	Remove poll.	`self._polls.pop, deque`
`MasterEngine.read_once` read_once(self, window_id)	526-530	Read once.	`self._wake.set, self._once.append`
`MasterEngine.queue_write` queue_write(self, window_id, request, description='')	532-535	Queue write.	`self._wake.set, self._writes.append, bytes`
`MasterEngine.start` start(self)	537-539	Start.	`threading.Thread, self.thread.start`
`MasterEngine.stop` stop(self, join_timeout=1.0)	541-547	Stop.	`self.stop_event.set, self._wake.set, cancel_serial_io, thread.is_alive, thread.join`
`MasterEngine.next_job` next_job(self, now=None)	550-573	Chooses the next transaction: queued writes first, then "Read once" requests, then the enabled window whose next due time is earliest.	`time.perf_counter, min, self._once.popleft, self._polls.get, self._writes.popleft`
`MasterEngine.run` run(self)	576-604	Worker loop: optional idle-bus check, then one transaction at a time with the configured delay between polls, posting master_ready, master_result and master_stopped events.	`self.event_queue.put, bus_activity_detected, self.stop_event.is_set, self._serve_jobs, hex_bytes`
`MasterEngine._serve_jobs` _serve_jobs(self)	606-628	Serve writes / reads until stopped; serial errors propagate.	`self.stop_event.is_set, self.next_job, self.event_queue.put, self.stop_event.wait, self._wake.wait`

Callable	Lines	Responsibility	Representative direct calls
`MasterEngine._reconnect` _reconnect(self, detail)	630-645	Close the failed port and reopen it; False when stopped meanwhile.	`self.event_queue.put, self.ser.close, self.stop_event.wait, self.reopen`
`MasterEngine.transact_for` transact_for(self, window_id, request)	647-649	Transact *request* for a window (the TCP Client picks its device here).	`self.transact`
`MasterEngine.transact` transact(self, request)	651-695	Sends one request and waits for its response until the response timeout, classifying the result as ok, exception, invalid, timeout or broadcast.	`write_discovery_request, time.perf_counter, bytearray, self.stop_event.is_set, hex_bytes`

Detailed behavior of the key paths

`format_register_values` lines 248-250

Turns raw registers into display strings for the selected format; 32-bit values are shown on the first register of each pair and an unpaired last register shows a dash.

`encode_register_value` lines 263-307

Converts one user-entered value into the register(s) it occupies in the selected format and byte order, rejecting values that do not fit.

`build_write_request` lines 350-381

Builds FC05/06/15/16 requests within the Modbus write limits; Slave 0 is accepted for broadcast.

`find_master_response` lines 399-447

Finds the CRC-valid response of the expected length from the addressed Slave, or a Modbus exception, and decodes read values. Adapter echoes of read requests are skipped.

`MasterEngine.next_job` lines 550-573

Chooses the next transaction: queued writes first, then "Read once" requests, then the enabled window whose next due time is earliest.

`MasterEngine.run` lines 576-604

Worker loop: optional idle-bus check, then one transaction at a time with the configured delay between polls, posting master_ready, master_result and master_stopped events.

`MasterEngine.transact` **lines 651-695**

Sends one request and waits for its response until the response timeout, classifying the result as ok, exception, invalid, timeout or broadcast.

> **Design note.** The engine owns the serial port while the Master is connected. Widgets only change definitions through set_poll(), read_once() and queue_write(); results come back through event_queue.

mb_master_ui.py

Master (poll) tab user interface. MasterMixin builds the connection bar and the grid of poll windows, validates window definitions, renders values and counters, opens the write and name dialogs, pops windows out to their own top-level windows and saves or opens workspace files.

Source lines	SHA-256
1479	`2d1cf37767f636f6f31ea8d39dddf1364992c180513bdd09ebb1e797f82b6922`

Imports: `time, tkinter, tkinter.font, tkinter.messagebox, ttk, mb_config.DISCOVERY_BAUD_RATES, DISCOVERY_IDLE_CHECK_SECONDS, MASTER_DEFAULT_DELAY_MS, MASTER_DEFAULT_RESPONSE_TIMEOUT_MS, MASTER_DEFAULT_SCAN_MS, MASTER_MAX_DELAY_MS, MASTER_MAX_RESPONSE_TIMEOUT_MS, MASTER_MAX_SCAN_MS, MASTER_MAX_WINDOWS, MASTER_MIN_DELAY_MS, MASTER_MIN_RESPONSE_TIMEOUT_MS, MASTER_WORKSPACE_FORMATS, MASTER_WORKSPACE_VERSION, MODBUS_EXCEPTION_CODES, get_master_workspaces_folder, mb_diagnostics.FUNCTION_NAMES, mb_master.BIT_FUNCTION_CODES, DEFAULT_FORMAT, FORMAT_KEYS_BY_LABEL, FORMAT_LABELS, FORMAT_SIZES, READ_FUNCTION_CODES, WRITABLE_READ_FCS, MasterEngine, PollDefinition, build_write_request, encode_coil_value, encode_register_value, format_window_values, raw_text_for_scaled, register_groups, validate_read_definition, mb_table_ui.SERIAL_PARITIES, SERIAL_STOP_BITS, TABLE_MAX_TITLE_LENGTH, TABLE_PANE_MAX_ROWS, TABLE_PANE_MIN_ROWS, TABLE_PANE_MIN_WIDTH, TableUiMixin, TableWorkspace, apply_format_to_rows, parse_int_field, validated_formats, validated_names, validated_scales, mb_table_ui.list_workspaces, mb_table_ui.workspace_path, mb_theme.theme_colors, mb_widgets.RefreshButton`

Module constants / configuration data

Name	Definition
`MASTER_PARITIES`	`SERIAL_PARITIES`
`MASTER_STOP_BITS`	`SERIAL_STOP_BITS`
`MASTER_PANE_MIN_WIDTH`	`TABLE_PANE_MIN_WIDTH`
`MASTER_PANE_MIN_ROWS`	`TABLE_PANE_MIN_ROWS`
`MASTER_PANE_MAX_ROWS`	`TABLE_PANE_MAX_ROWS`
`MASTER_OK_MESSAGE_SECONDS`	`3.0`
`MASTER_MAX_TITLE_LENGTH`	`TABLE_MAX_TITLE_LENGTH`
`MASTER_WHEEL_TAG`	`'MBToolkit.MasterWheel'`
`FC_CHOICES`	`tuple((fc_choice_label(fc) for fc in READ_FUNCTION_CODES))`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`workspace_path` workspace_path(name)	80-82	File of a Master workspace (in Workspaces\Master).	`_workspace_path, get_master_workspaces_folder`
`list_workspaces` list_workspaces()	85-87	Names of the saved Master workspaces, sorted alphabetically.	`_list_workspaces, get_master_workspaces_folder`
`fc_choice_label` fc_choice_label(fc)	90-91	Fc choice label.	—
`class MasterWindow`	**97-185**	**State of one poll window (independent of where its pane is shown).**	**Class container**
`MasterWindow.__init__` __init__(self, window_id, number, slave=1, fc=3, address=0, qty=10, scan_ms=MASTER_DEFAULT_SCAN_MS, base=0, fmt=DEFAULT_FORMAT, names=None, enabled=True, title='', formats=None, scales=None)	100-123	Init.	`dict, self.reset_counters, str(title or '').strip, int, (formats or {}).items`
`MasterWindow.reset_counters` reset_counters(self)	125-135	Reset counters.	—
`MasterWindow.definition` definition(self)	137-139	Definition.	`PollDefinition`
`MasterWindow.to_json` to_json(self)	141-150	To json.	`str, sorted, self.names.items, self.formats.items, self.scales.items`
`MasterWindow.groups` groups(self)	152-154	[(row offset, size, fmt or None)] of the values shown in this window.	`register_groups`
`MasterWindow.group_of_row` group_of_row(self, row)	156-160	Group of row.	`self.groups`
`MasterWindow.validated_kwargs` validated_kwargs(data)	163-185	Validate one workspace window entry; raise ValueError if invalid.	`validate_read_definition, int, data.get, isinstance, ValueError`
`class MasterMixin`	**188-1475**	**Master (poll) tab GUI.**	**Class container**
`MasterMixin.build_master_tab` build_master_tab(self, page)	217-286	Builds the connection bar, toolbar and scrollable grid of poll windows, restoring the saved connection parameters.	`getattr(self, 'ui_settings', {}).get, page.columnconfigure, page.rowconfigure, self._master_build_connection, ttk.Frame`
`MasterMixin._master_build_connection` _master_build_connection(self, page, saved)	288-366	Connection bar of the serial Master (COM port and serial format).	`ttk.LabelFrame, connection.grid, ttk.Frame, row.grid, ttk.Label(row, text='Porta COM').grid`
`MasterMixin._master_restore_workspace` _master_restore_workspace(self, name)	368-370	Master restore workspace.	`self.master_open_workspace, self.master_add_window`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin._master_install_wheel_tags` _master_install_wheel_tags(self, root_widget)	373-374	Master install wheel tags.	`self._table_install_wheel_tags`
`MasterMixin._master_wheel` _master_wheel(self, event)	376-377	Master wheel.	`self._table_wheel, getattr`
`MasterMixin._master_window_name` _master_window_name(self, window)	379-380	Master window name.	`self.trf`
`MasterMixin._master_window_summary` _master_window_summary(self, window)	382-385	Master window summary.	—
`MasterMixin._master_window_title` _master_window_title(self, window)	387-388	Master window title.	`self._master_window_name, self._master_window_summary`
`MasterMixin._master_counters_text` _master_counters_text(self, window)	390-393	Master counters text.	—
`MasterMixin._master_set_message` _master_set_message(self, window, template, style='normal', **values)	395-398	Master set message.	`time.monotonic, self._master_render_status`
`MasterMixin._master_render_status` _master_render_status(self, window)	400-422	Master render status.	`self._master_panes.get, self.trf, pane['counters_var'].set, pane['message_var'].set, pane['message_label'].configure`
`MasterMixin._master_build_pane` _master_build_pane(self, window, parent)	425-530	Master build pane.	`ttk.LabelFrame, frame.columnconfigure, self._table_build_header, self._master_build_target, frame.rowconfigure`
`MasterMixin._master_build_target` _master_build_target(self, frame, pane, window)	532-534	Row with the device of the window (TCP Client only); None here.	—
`MasterMixin._master_parse_target` _master_parse_target(self, pane)	536-538	Device fields of a window as a dict (TCP Client only; ValueError if invalid).	—
`MasterMixin._master_validate_definition` _master_validate_definition(self, slave, fc, address, qty)	540-541	Master validate definition.	`validate_read_definition`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin._master_update_format_state` _master_update_format_state(self, window)	543-547	Master update format state.	`self._master_panes.get, pane['format_combo'].configure`
`MasterMixin._master_fill_rows` _master_fill_rows(self, window)	549-560	Master fill rows.	`self._master_panes.get, tree.delete, tree.configure, range, self._master_render_values`
`MasterMixin._master_render_values` _master_render_values(self, window)	562-580	Master render values.	`self._master_panes.get, enumerate, format_window_values, tree.item, tree.set`
`MasterMixin._master_schedule_reflow` _master_schedule_reflow(self, _event=None)	582-588	Master schedule reflow.	`self.after, self.after_cancel`
`MasterMixin._master_reflow` _master_reflow(self)	590-598	Arrange docked panes in as many equal columns as the width allows.	`self._table_reflow, sorted, self.master_windows.values`
`MasterMixin.master_add_window` master_add_window(self, kwargs=None)	601-619	Master add window.	`next, self._master_window_class, self._master_build_pane, self._master_reflow, self._master_mark_changed`
`MasterMixin.master_duplicate_window` master_duplicate_window(self, window_id)	621-625	Master duplicate window.	`self.master_windows.get, self._master_window_class.validated_kwargs, self.master_add_window, window.to_json`
`MasterMixin.master_close_window` master_close_window(self, window_id)	627-645	Master close window.	`self.master_windows.pop, self._master_panes.pop, self._master_reflow, self.master_engine.remove_poll, pane.get`
`MasterMixin.master_apply_window` master_apply_window(self, window_id)	647-688	Validates the fields of one window, updates its model, rebuilds its rows and hands the new definition to the running engine.	`self.master_windows.get, self._master_panes.get, FORMAT_KEYS_BY_LABEL.get, target.items, self._master_fill_rows`
`MasterMixin.master_toggle_pause` master_toggle_pause(self, window_id)	690-700	Master toggle pause.	`self.master_windows.get, self._master_render_status, self._master_mark_changed, self.master_engine.set_poll, window.definition`
`MasterMixin.master_read_once` master_read_once(self, window_id)	702-706	Master read once.	`self.master_engine.read_once, messagebox.showinfo, self.tr`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin.master_toggle_popout` master_toggle_popout(self, window_id)	708-736	Moves a window between the grid and its own top-level window by rebuilding its pane from the model.	`self.master_windows.get, self._master_panes.pop, pane.get, self._master_render_status, self._master_reflow`
`MasterMixin._master_on_double_click` _master_on_double_click(self, window_id, event)	739-752	Master on double click.	`self._master_panes.get, tree.identify_row, tree.selection_set, tree.identify_column, self._master_edit_name`
`MasterMixin._master_edit_name` _master_edit_name(self, window_id, row)	754-781	Master edit name.	`self.master_windows.get, self._table_dialog, ttk.Label(body, text=self.trf('Nome do endereço {address}', address=pdu + window.base)).grid, …`
`MasterMixin._master_edit_value` _master_edit_value(self, window_id)	783-885	Master edit value.	`self.master_windows.get, self._master_panes.get, pane['tree'].selection, int, WRITABLE_READ_FCS.get`
`MasterMixin.master_build_write` master_build_write(self, window, slave_text, address_text, values_text, fc, fmt=None, scale=None)	887-915	Validates the write dialog and encodes the values with the window format into an FC05/06/15/16 request.	`parse_int_field, build_write_request, ValueError, str(values_text).split, p.strip`
`MasterMixin._master_connection_settings` _master_connection_settings(self)	918-933	Master connection settings.	`parse_int_field, self.master_timeout_var.get, self.master_delay_var.get, self.master_port_var.get().strip, int`
`MasterMixin.master_connect` master_connect(self)	935-997	Checks the busy flag, safety confirmation and settings, opens the port, starts MasterEngine with the idle-bus check and locks the other active features.	`self.get_serial_modules, list, MasterEngine, self.master_windows.values, self._master_lock_other_features`
`MasterMixin.master_disconnect` master_disconnect(self)	999-1004	Master disconnect.	`self.master_status_var.set, self.master_disconnect_btn.configure, self.master_engine.stop, self.tr`
`MasterMixin._master_lock_other_features` _master_lock_other_features(self, locked)	1006-1024	Master lock other features.	`self.master_connect_btn.configure, self.master_disconnect_btn.configure, self.master_refresh_btn.configure, getattr, self.start_btn.configu…`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin._master_finish_session` _master_finish_session(self, payload)	1026-1061	Master finish session.	`self._master_lock_other_features, self.master_status_var.set, self.master_windows.values, (payload or {}).get, engine.stop`
`MasterMixin.handle_master_event` handle_master_event(self, event, payload)	1064-1084	Handle master event.	`self.master_status_var.set, self.tr, self.master_windows.values, self.trf, self._master_render_values`
`MasterMixin._master_apply_result` _master_apply_result(self, result)	1086-1138	Updates counters, values and the status line of the window that issued a transaction, ignoring results of an outdated definition.	`self.master_windows.get, result.get, self._master_render_values, self._master_set_message, self.master_engine.read_once`
`MasterMixin.master_workspace_data` master_workspace_data(self)	1141-1148	Master workspace data.	`self._master_connection_json, w.to_json, sorted, self.master_windows.values`
`MasterMixin._master_connection_json` _master_connection_json(self)	1150-1158	Master connection json.	`self.master_port_var.get, self.master_baud_var.get, self.master_parity_var.get, self.master_stop_var.get, self.master_timeout_var.get`
`MasterMixin._master_apply_connection` _master_apply_connection(self, connection)	1160-1173	Connection settings of an opened workspace (only while disconnected).	`str, self.master_baud_var.set, connection.get, self.master_parity_var.set, self.master_stop_var.set`
`MasterMixin._master_mark_changed` _master_mark_changed(self)	1175-1176	Master mark changed.	`self.master_workspace.mark_changed`
`MasterMixin._master_build_workspace_menu` _master_build_workspace_menu(self)	1178-1181	Rebuilds the Workspace menu each time it opens, listing the saved workspaces under Open and Delete.	`self.master_workspace.fill_menu, self._master_apply_menu_palette`
`MasterMixin.master_save_workspace` master_save_workspace(self, name=None)	1183-1184	Master save workspace.	`self.master_workspace.save`
`MasterMixin.master_delete_workspace` master_delete_workspace(self, name, confirm=True)	1186-1187	Master delete workspace.	`self.master_workspace.delete`
`MasterMixin.master_open_workspace` master_open_workspace(self, name, quiet=False)	1189-1190	Loads a named workspace from the workspace folder, validating every window and skipping invalid entries with a warning.	`self.master_workspace.open`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin._master_load_workspace_data` _master_load_workspace_data(self, data)	1192-1208	Replace the windows with those of *data*; returns the invalid count.	`list, data.get, self.master_close_window, isinstance, self._master_apply_connection`
`MasterMixin.master_rename_window` master_rename_window(self, window_id, name=None)	1211-1228	Gives a window its own name, shown in its header and saved in the workspace; an empty name restores the default.	`self.master_windows.get, self._master_render_status, self._master_mark_changed, self._table_ask_text, name.strip`
`MasterMixin._master_fields_changed` _master_fields_changed(self, window, pane)	1231-1243	True when the definition fields differ from what was last applied.	`pane['slave_var'].get().strip, str, pane['fc_var'].get, fc_choice_label, pane['address_var'].get().strip`
`MasterMixin._master_update_apply_state` _master_update_apply_state(self, window_id)	1245-1255	Master update apply state.	`self.master_windows.get, self._master_panes.get, self._master_fields_changed, pane['apply_btn'].configure`
`MasterMixin._master_build_row_menu` _master_build_row_menu(self)	1258-1259	Master build row menu.	`tk.Menu, self._table_root`
`MasterMixin._master_selected_rows` _master_selected_rows(self, window_id)	1261-1262	Master selected rows.	`self._table_selected_rows, self._master_panes.get`
`MasterMixin._master_show_row_menu` _master_show_row_menu(self, window_id, event)	1264-1284	Master show row menu.	`self.master_windows.get, self._master_panes.get, tree.identify_row, self._table_fill_row_menu, self._master_apply_menu_palette`
`MasterMixin._master_row_menu_extra` _master_row_menu_extra(self, window)	1286-1289	Master row menu extra.	`getattr, callable, opener`
`MasterMixin.master_set_row_format` master_set_row_format(self, window_id, fmt)	1291-1304	Applies a format (or the window default) to the selected rows through apply_format_to_rows() and redraws the values.	`self.master_windows.get, self._master_render_values, self._master_mark_changed, apply_format_to_rows, self._master_selected_rows`
`MasterMixin.master_definition_from_record` master_definition_from_record(self, record)	1307-1330	Read definition (slave, fc, address, qty) behind a Traffic record, or None.	`int, record.get, validate_read_definition, str`
`MasterMixin.master_open_from_record` master_open_from_record(self, record)	1332-1347	Master open from record.	`self.master_definition_from_record, self.master_open_definition, messagebox.showinfo, getattr, self.port_a_var.get().strip`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin.master_open_definition` master_open_definition(self, definition, port='', baud='', parity='', stopbits='')	1349-1369	Creates a Master window from a definition coming from the Sniffer or the Finder and copies the serial parameters when disconnected.	`self.master_add_window, self.main_notebook.select, str, self.master_baud_var.set, self.master_parity_var.set`
`MasterMixin._master_selected_group_starts` _master_selected_group_starts(self, window)	1371-1373	PDU addresses where the values of the selected rows start.	`self._table_selected_group_starts, self._master_panes.get`
`MasterMixin.master_edit_scale` master_edit_scale(self, window_id)	1375-1389	Asks for a linear scale and applies it to the values of the selected rows.	`self.master_windows.get, self._master_selected_group_starts, self._master_render_values, self._master_mark_changed, messagebox.showwarning`
`MasterMixin.master_remove_scale` master_remove_scale(self, window_id)	1391-1401	Master remove scale.	`self.master_windows.get, self._master_selected_group_starts, self._master_render_values, self._master_mark_changed, window.scales.pop`
`MasterMixin.master_copy_rows` master_copy_rows(self, window_id)	1403-1405	Copy the selected rows (all rows if none) as tab-separated text.	`self._table_copy_rows, self._master_panes.get`
`MasterMixin.master_set_base` master_set_base(self, window_id, base)	1407-1417	Master set base.	`self.master_windows.get, self._master_panes.get, pane['address_var'].set, self._master_fill_rows, self._master_render_status`
`MasterMixin.master_confirm_close` master_confirm_close(self)	1420-1423	Asks whether to save a saved workspace that has unsaved changes before the application closes; Cancel keeps it open.	`getattr, workspace.confirm_close`
`MasterMixin.master_apply_language` master_apply_language(self)	1426-1439	Master apply language.	`self.master_workspace.update_label, self.master_windows.values, hasattr, self._master_panes.get, self._table_translate_headings`
`MasterMixin._master_apply_menu_palette` _master_apply_menu_palette(self)	1441-1443	Master apply menu palette.	`self._table_apply_menus_palette, getattr`
`MasterMixin.master_apply_palette` master_apply_palette(self, dark)	1445-1461	Master apply palette.	`self._master_apply_menu_palette, theme_colors, hasattr, getattr(self, '_master_panes', {}).values, getattr`

Callable	Lines	Responsibility	Representative direct calls
`MasterMixin.master_shutdown` master_shutdown(self)	1463-1475	Master shutdown.	`getattr, engine.stop, ser.close`
`fc_choice_label_write` fc_choice_label_write(fc)	1478-1479	Fc choice label write.	—

Detailed behavior of the key paths

`build_master_tab` lines 217-286

Builds the connection bar, toolbar and scrollable grid of poll windows, restoring the saved connection parameters.

`master_apply_window` lines 647-688

Validates the fields of one window, updates its model, rebuilds its rows and hands the new definition to the running engine.

`master_build_write` lines 887-915

Validates the write dialog and encodes the values with the window format into an FC05/06/15/16 request.

`master_connect` lines 935-997

Checks the busy flag, safety confirmation and settings, opens the port, starts MasterEngine with the idle-bus check and locks the other active features.

`_master_apply_result` lines 1086-1138

Updates counters, values and the status line of the window that issued a transaction, ignoring results of an outdated definition.

`master_toggle_popout` lines 708-736

Moves a window between the grid and its own top-level window by rebuilding its pane from the model.

`master_open_workspace` lines 1189-1190

Loads a named workspace from the workspace folder, validating every window and skipping invalid entries with a warning.

`_master_build_workspace_menu` lines 1178-1181

Rebuilds the Workspace menu each time it opens, listing the saved workspaces under Open and Delete.

`master_rename_window` **lines 1211-1228**

Gives a window its own name, shown in its header and saved in the workspace; an empty name restores the default.

`master_set_row_format` **lines 1291-1304**

Applies a format (or the window default) to the selected rows through apply_format_to_rows() and redraws the values.

`master_edit_scale` **lines 1375-1389**

Asks for a linear scale and applies it to the values of the selected rows.

`master_open_definition` **lines 1349-1369**

Creates a Master window from a definition coming from the Sniffer or the Finder and copies the serial parameters when disconnected.

`master_confirm_close` **lines 1420-1423**

Asks whether to save a saved workspace that has unsaved changes before the application closes; Cancel keeps it open.

> **Design note.** Window state lives in MasterWindow objects, so a pane can be destroyed and rebuilt (pop out / dock, language switch) without losing data.

mb_slave.py

Modbus RTU slave (simulator) engine. SlaveStore keeps the data tables, defined areas and faults of every simulated Slave ID; handle_request() executes one request like a real device (reads, writes, Exceptions 01/02/03, broadcast); SlaveEngine assembles requests from the serial bytes, applies the fault simulation, answers after t3.5 and removes the adapter echo. It contains no widgets.

Source lines
629

SHA-256
`42500c79110ebe2f27dfd06bad81999320db72c0625d0280067d78e55c5a8cee`

Imports: `math, random, threading, time, dataclasses.dataclass, field, mb_config.MASTER_MAX_READ_BITS, MASTER_MAX_READ_REGISTERS, MASTER_MAX_WRITE_BITS, MASTER_MAX_WRITE_REGISTERS, MASTER_RECONNECT_INTERVAL_SECONDS, SLAVE_FAULT_MAX_DELAY_MS, SLAVE_RESYNC_SILENCE_MS, mb_protocol.append_crc, crc_ok, modbus_rtu_interframe_gap_seconds, mb_slave_finder.cancel_serial_io, discovery_bits_per_char, write_discovery_request`

Module constants / configuration data

Name	Definition
`TABLES`	`('coils', 'discrete', 'holding', 'input')`
`TABLE_LABELS`	`{'coils': 'Coils', 'discrete': 'Discrete Inputs', 'holding': 'Holding Registers', 'input': 'Input Registers'}`
`BIT_TABLES`	`('coils', 'discrete')`
`FC_TABLES`	`{1: 'coils', 2: 'discrete', 3: 'holding', 4: 'input', 5: 'coils', 15: 'coils', 6: 'holding', 16: 'holding'}`
`SUPPORTED_FUNCTION_CODES`	`(1, 2, 3, 4, 5, 6, 15, 16)`
`WRITE_FUNCTION_CODES`	`(5, 6, 15, 16)`
`FAULT_EXCEPTION_CODES`	`(1, 2, 3, 4, 6)`
`EXCEPTION_ILLEGAL_FUNCTION`	`1`
`EXCEPTION_ILLEGAL_ADDRESS`	`2`
`EXCEPTION_ILLEGAL_VALUE`	`3`
`TABLE_READ_FC`	`{'coils': 1, 'discrete': 2, 'holding': 3, 'input': 4}`
`REGISTER_GENERATORS`	`('counter', 'ramp', 'sine', 'random')`
`BIT_GENERATORS`	`('toggle', 'random')`

Name	Definition
`GENERATOR_MIN_PERIOD_S`	`0.1`
`GENERATOR_MAX_PERIOD_S`	`3600.0`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`table_label` table_label(table)	56-57	Table label.	`TABLE_LABELS.get`
`table_choice_label` table_choice_label(table)	60-62	Table field text: read function code + name, e.g. "03 Holding Registers".	—
`default_fault` default_fault()	69-71	Default fault.	—
`validate_fault` validate_fault(data)	74-92	Return a clean fault dict from user/workspace data; raise ValueError (PT).	`isinstance, ValueError, int, bool, data.get`
`fault_active` fault_active(fault)	95-98	True when the fault settings change how the slave answers.	`bool, fault.get`
`_number` _number(value, message)	113-120	Number.	`float, math.isnan, math.isinf, ValueError, str(value).strip().replace`
`validate_generator` validate_generator(data, bits=False)	123-151	Return a clean automatic-value dict; raise ValueError (PT canonical text).	`data.get, _number, result.update, isinstance, ValueError`
`class ValueGenerator`	**154-193**	**Value of one automatic address as a function of the elapsed time.**	**Class container**
`ValueGenerator.__init__` __init__(self, config, rng=None)	163-167	Init.	`dict, random.Random`
`ValueGenerator.value` value(self, elapsed_s)	169-193	Value.	`max, int, math.floor, self.rng.uniform, self.rng.randint`
`class SlaveStore`	**200-267**	**Data tables, defined areas and faults of every simulated Slave ID.**	**Class container**
`SlaveStore.__init__` __init__(self)	209-213	Init.	`threading.RLock`

Callable	Lines	Responsibility	Representative direct calls
`SlaveStore.set_areas` set_areas(self, areas)	215-222	Replace the defined areas: iterable of (slave, table, address, qty).	`defined.setdefault((int(slave), table), set()).update, range, defined.setdefault, int, set`
`SlaveStore.slave_ids` slave_ids(self)	224-226	Slave ids.	—
`SlaveStore.covers` covers(self, slave, table, address, qty)	228-231	Covers.	`self._defined.get, bool, all, range`
`SlaveStore.read` read(self, slave, table, address, qty)	233-236	Read.	`self._values.get, values.get, range`
`SlaveStore.write` write(self, slave, table, address, values)	238-243	Write.	`self._values.setdefault, enumerate, int`
`SlaveStore.clear_values` clear_values(self)	245-247	Clear values.	—
`SlaveStore.set_fault` set_fault(self, slave, fault)	249-254	Set fault.	`fault_active, dict, self._faults.pop, int`
`SlaveStore.fault` fault(self, slave)	256-259	Fault.	`self._faults.get, dict`
`SlaveStore.faults` faults(self)	261-263	Faults.	`dict, self._faults.items`
`SlaveStore.clear_faults` clear_faults(self)	265-267	Clear faults.	—
`request_length` request_length(buffer)	274-287	Length of the request frame starting at buffer[0]; None if not known yet.	`len`
`exception_frame` exception_frame(slave, fc, code)	290-291	Exception frame.	`append_crc, bytes`
`class SlaveReply`	**295-305**	**Outcome of one request: what to send back and what it touched.**	**Class container**
`_u16` _u16(data, index)	308-309	U16.	—
`_parse_request` _parse_request(frame)	312-347	Return (table, address, qty, values or None) or raise _ExceptionReply.	`_u16, _ExceptionReply, len, range`
`class _ExceptionReply`	**350-353**	**_ExceptionReply.**	**Class container**
`_ExceptionReply.__init__` __init__(self, code)	351-353	Init.	`super().__init__, super`

Callable	Lines	Responsibility	Representative direct calls
`handle_request` handle_request(store, frame)	356-415	Execute one CRC-valid request frame and return a SlaveReply.	`SlaveReply, store.read, append_crc, _parse_request, store.covers`
`corrupt_crc` corrupt_crc(frame)	418-422	Same frame with a wrong CRC (fault simulation).	`bytearray, bytes`
`class SlaveEngine`	**429-629**	**Answer the requests of a real master on one serial port.**	**Class container**
`SlaveEngine.__init__` __init__(self, ser, event_queue, store, *, baud, parity, stopbits, serial_module=None, reopen=None, reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS, silence_ms=SLAVE_RESYNC_SILENCE_MS, rng=None)	447-469	Init.	`int, float, discovery_bits_per_char, modbus_rtu_interframe_gap_seconds, max`
`SlaveEngine.start` start(self)	471-473	Start.	`threading.Thread, self.thread.start`
`SlaveEngine.stop` stop(self, join_timeout=1.0)	475-480	Stop.	`self.stop_event.set, cancel_serial_io, thread.is_alive, thread.join, threading.current_thread`
`SlaveEngine.run` run(self)	483-500	Run.	`self.event_queue.put, self.stop_event.is_set, self._listen, self._reconnect, type`
`SlaveEngine._reconnect` _reconnect(self, detail)	502-517	Reconnect.	`self._buffer.clear, self.event_queue.put, self.ser.close, self.stop_event.wait, self.reopen`
`SlaveEngine._listen` _listen(self)	519-534	Read and answer until stopped; serial errors propagate.	`time.perf_counter, self.stop_event.is_set, int, self.stop_event.wait, self._strip_echo`
`SlaveEngine.process` process(self, final)	536-561	Answer every complete request in the buffer.	`request_length, bytes, crc_ok, self.stop_event.is_set, self._answer`
`SlaveEngine._strip_echo` _strip_echo(self, data)	563-578	Remove the RS485 adapter echo of our last response from *data*.	`time.perf_counter, len`
`SlaveEngine._answer` _answer(self, frame)	580-629	Answer.	`self.event_queue.put, self.store.fault, SlaveReply, corrupt_crc, list`

Design note. handle_request() is a pure function of the store and the frame, so every reply is testable without a serial port; the engine only adds framing, timing, faults and echo removal around it.

mb_slave_ui.py

Slave (simulator) tab user interface. SlaveMixin builds the connection bar and the grid of simulated tables, edits values locally, highlights master writes, shows per-window and global counters, edits the faults of each Slave ID and saves or opens Slave workspaces.

Source lines	SHA-256
1668	`847fab008254fcb0cf989f66dc9d3dd2a1f1c1dd6e58f3da89ef869ffde6fa38`

Imports: `time, tkinter, tkinter.messagebox, ttk, mb_config.DISCOVERY_BAUD_RATES, MODBUS_EXCEPTION_CODES, SLAVE_FAULT_MAX_DELAY_MS, SLAVE_GENERATOR_TICK_MS, SLAVE_HIGHLIGHT_SECONDS, SLAVE_MAX_QTY, SLAVE_MAX_WINDOWS, SLAVE_WORKSPACE_FORMAT, SLAVE_WORKSPACE_VERSION, get_slave_workspaces_folder, mb_master.DEFAULT_FORMAT, FORMAT_KEYS_BY_LABEL, FORMAT_LABELS, FORMAT_SIZES, encode_coil_value, encode_register_value, format_window_values, READ_FUNCTION_CODES, raw_text_for_scaled, register_groups, unscale_value, mb_slave.BIT_GENERATORS, BIT_TABLES, FAULT_EXCEPTION_CODES, FC_TABLES, REGISTER_GENERATORS, TABLE_READ_FC, TABLES, SlaveEngine, SlaveStore, default_fault, fault_active, table_choice_label, table_label, validate_fault, validate_generator, ValueGenerator, mb_table_ui.SERIAL_PARITIES, SERIAL_STOP_BITS, TABLE_MAX_TITLE_LENGTH, TABLE_PANE_MAX_ROWS, TABLE_PANE_MIN_ROWS, TABLE_PANE_MIN_WIDTH, TableUiMixin, TableWorkspace, apply_format_to_rows, parse_int_field, validated_formats, validated_names, validated_scales, mb_theme.theme_colors, mb_widgets.RefreshButton`

Module constants / configuration data

Name	Definition
`SLAVE_WHEEL_TAG`	`'MBToolkit.SlaveWheel'`
`GENERATOR_LABELS`	`{'counter': 'Contador', 'ramp': 'Rampa', 'sine': 'Senoide', 'random': 'Aleatório', 'toggle': 'Alternar 0/1'}`
`TABLE_CHOICES`	`tuple((table_choice_label(t) for t in TABLES))`
`TABLE_KEYS_BY_LABEL`	`{table_choice_label(t): t for t in TABLES}`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`fault_parts` fault_parts(fault, bad_reply='CRC errado')	86-101	(PT template, values) pieces describing an active fault.	`fault.get, parts.append`

Callable	Lines	Responsibility	Representative direct calls
`class SlaveWindow`	**104-198**	**State of one simulated data table (independent of where it is shown).**	**Class container**
`SlaveWindow.__init__` __init__(self, window_id, number, slave=1, table='holding', address=0, qty=10, base=0, fmt=DEFAULT_FORMAT, names=None, title='', formats=None, scales=None, generators=None)	107-129	Init.	`dict, self.reset_counters, str(title or '').strip, int, (formats or {}).items`
`SlaveWindow.reset_counters` reset_counters(self)	131-134	Reset counters.	—
`SlaveWindow.bits` bits(self)	137-138	Bits.	—
`SlaveWindow.area` area(self)	140-141	Area.	—
`SlaveWindow.overlap` overlap(self, address, qty)	143-147	Row range of this window covered by [address, address + qty), or None.	`max, min`
`SlaveWindow.groups` groups(self)	149-150	Groups.	`register_groups`
`SlaveWindow.group_of_row` group_of_row(self, row)	152-156	Group of row.	`self.groups`
`SlaveWindow.to_json` to_json(self)	158-167	To json.	`str, sorted, self.names.items, self.formats.items, self.scales.items`
`SlaveWindow.drop_invalid_generators` drop_invalid_generators(self)	169-178	Keep only automatic values that still fit the window's area and table.	`self.generators.items, validate_generator`
`SlaveWindow.validated_kwargs` validated_kwargs(data)	181-198	Validate one workspace window entry; raise ValueError if invalid.	`validate_area, data.get, isinstance, ValueError, int`
`validated_generators` validated_generators(generators, bits)	201-210	Automatic values of a workspace window entry; invalid ones are dropped.	`isinstance, generators.items, str(key).isdigit, validate_generator, int`
`slave_definition_from_record` slave_definition_from_record(record)	213-240	Slave window (slave, table, address, qty) behind a Traffic record, or None.	`int, validate_area, record.get, str`
`validate_area` validate_area(slave, table, address, qty)	243-254	Raise ValueError (PT canonical message) for an invalid window area.	`ValueError, int`
`class SlaveMixin`	**257-1668**	**Slave (simulator) tab GUI.**	**Class container**

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.build_slave_tab` build_slave_tab(self, page)	282-355	Build slave tab.	`getattr(self, 'ui_settings', {}).get, SlaveStore, self._slave_reset_stats, page.columnconfigure, page.rowconfigure`
`SlaveMixin._slave_build_connection` _slave_build_connection(self, page, saved)	357-425	Connection bar of the serial Slave (COM port, serial format, counters).	`ttk.LabelFrame, connection.grid, ttk.Frame, row.grid, ttk.Label(row, text='Porta COM').grid`
`SlaveMixin._slave_restore_workspace` _slave_restore_workspace(self, name)	427-429	Slave restore workspace.	`self.slave_open_workspace, self.slave_add_window`
`SlaveMixin._slave_reset_stats` _slave_reset_stats(self)	432-434	Slave reset stats.	—
`SlaveMixin._slave_render_stats` _slave_render_stats(self)	436-440	Slave render stats.	`self.slave_stats_var.set, self.trf`
`SlaveMixin._slave_window_name` _slave_window_name(self, window)	442-443	Slave window name.	`self.trf`
`SlaveMixin._slave_window_summary` _slave_window_summary(self, window)	445-448	Slave window summary.	`table_label`
`SlaveMixin._slave_fault_text` _slave_fault_text(self, slave)	450-458	Slave fault text.	`self.slave_faults.get, ', '.join, self.trf, fault_active, fault_parts`
`SlaveMixin._slave_render_status` _slave_render_status(self, window)	460-484	Slave render status.	`self._slave_panes.get, pane['counters_var'].set, pane['last_var'].set, pane['name_var'].set, pane['summary_var'].set`
`SlaveMixin._slave_build_pane` _slave_build_pane(self, window, parent)	487-576	Slave build pane.	`ttk.LabelFrame, frame.columnconfigure, frame.rowconfigure, self._table_build_header, tk.StringVar`
`SlaveMixin._slave_update_format_state` _slave_update_format_state(self, window)	578-581	Slave update format state.	`self._slave_panes.get, pane['format_combo'].configure`
`SlaveMixin._slave_fill_rows` _slave_fill_rows(self, window)	583-594	Slave fill rows.	`self._slave_panes.get, tree.delete, tree.configure, range, self._slave_render_values`
`SlaveMixin._slave_render_values` _slave_render_values(self, window)	596-622	Slave render values.	`self._slave_panes.get, self.slave_store.read, time.monotonic, set, enumerate`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin._slave_render_table` _slave_render_table(self, slave, table)	624-627	Slave render table.	`self.slave_windows.values, self._slave_render_values`
`SlaveMixin._slave_schedule_reflow` _slave_schedule_reflow(self, _event=None)	629-635	Slave schedule reflow.	`self.after, self.after_cancel`
`SlaveMixin._slave_reflow` _slave_reflow(self)	637-644	Slave reflow.	`self._table_reflow, sorted, self.slave_windows.values`
`SlaveMixin._slave_sync_areas` _slave_sync_areas(self)	647-649	Tell the store which addresses exist (what the windows show).	`self.slave_store.set_areas, w.area, self.slave_windows.values`
`SlaveMixin.slave_add_window` slave_add_window(self, kwargs=None)	651-670	Slave add window.	`next, SlaveWindow, window.drop_invalid_generators, self._slave_sync_areas, self._slave_sync_generators`
`SlaveMixin.slave_duplicate_window` slave_duplicate_window(self, window_id)	672-675	Slave duplicate window.	`self.slave_windows.get, self.slave_add_window, SlaveWindow.validated_kwargs, window.to_json`
`SlaveMixin.slave_close_window` slave_close_window(self, window_id)	677-694	Slave close window.	`self.slave_windows.pop, self._slave_sync_areas, self._slave_sync_generators, self._slave_panes.pop, self._slave_reflow`
`SlaveMixin._slave_fields_changed` _slave_fields_changed(self, window, pane)	696-706	Slave fields changed.	`pane['slave_var'].get().strip, str, pane['table_var'].get, table_choice_label, pane['address_var'].get().strip`
`SlaveMixin._slave_update_apply_state` _slave_update_apply_state(self, window_id)	708-717	Slave update apply state.	`self.slave_windows.get, self._slave_panes.get, self._slave_fields_changed, pane['apply_btn'].configure`
`SlaveMixin.slave_apply_window` slave_apply_window(self, window_id)	719-754	Slave apply window.	`self.slave_windows.get, self._slave_panes.get, FORMAT_KEYS_BY_LABEL.get, self._slave_fill_rows, self._slave_update_format_state`
`SlaveMixin.slave_toggle_popout` slave_toggle_popout(self, window_id)	756-783	Slave toggle popout.	`self.slave_windows.get, self._slave_panes.pop, pane.get, self._slave_render_status, self._slave_reflow`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.slave_rename_window` slave_rename_window(self, window_id, name=None)	785-802	Slave rename window.	`self.slave_windows.get, self._slave_render_status, self._slave_mark_changed, self._table_ask_text, name.strip`
`SlaveMixin._slave_on_double_click` _slave_on_double_click(self, window_id, event)	805-818	Slave on double click.	`self._slave_panes.get, tree.identify_row, tree.selection_set, tree.identify_column, self._slave_edit_name`
`SlaveMixin._slave_edit_name` _slave_edit_name(self, window_id, row)	820-839	Slave edit name.	`self.slave_windows.get, self._table_ask_text, name.strip, self._slave_panes.get, self._slave_mark_changed`
`SlaveMixin.slave_set_value` slave_set_value(self, window, row, text)	841-861	Store the value typed for *row* (first register of its value).	`self.slave_store.write, self._slave_render_table, window.group_of_row, window.scales.get, encode_register_value`
`SlaveMixin._slave_edit_value` _slave_edit_value(self, window_id)	863-921	Slave edit value.	`self.slave_windows.get, self._slave_panes.get, pane['tree'].selection, int, window.group_of_row`
`SlaveMixin._slave_show_row_menu` _slave_show_row_menu(self, window_id, event)	924-945	Slave show row menu.	`self.slave_windows.get, self._slave_panes.get, tree.identify_row, self._table_fill_row_menu, self._slave_apply_menu_palette`
`SlaveMixin._slave_selected_rows` _slave_selected_rows(self, window_id)	947-948	Slave selected rows.	`self._table_selected_rows, self._slave_panes.get`
`SlaveMixin.slave_set_row_format` slave_set_row_format(self, window_id, fmt)	950-962	Slave set row format.	`self.slave_windows.get, self._slave_render_values, self._slave_mark_changed, apply_format_to_rows, self._slave_selected_rows`
`SlaveMixin.slave_edit_scale` slave_edit_scale(self, window_id)	964-976	Slave edit scale.	`self.slave_windows.get, self._table_selected_group_starts, self._slave_render_values, self._slave_mark_changed, self._slave_panes.get`
`SlaveMixin.slave_remove_scale` slave_remove_scale(self, window_id)	978-988	Slave remove scale.	`self.slave_windows.get, self._table_selected_group_starts, self._slave_panes.get, self._slave_render_values, self._slave_mark_changed`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.slave_set_base` slave_set_base(self, window_id, base)	990-1000	Slave set base.	`self.slave_windows.get, self._slave_panes.get, pane['address_var'].set, self._slave_fill_rows, self._slave_render_status`
`SlaveMixin._slave_sync_generators` _slave_sync_generators(self)	1003-1018	Rebuild the running automatic values from the windows' settings.	`time.monotonic, self.slave_windows.values, window.generators.items, self.after, self._slave_generators.get`
`SlaveMixin.slave_generator_registers` slave_generator_registers(self, window, address, value)	1020-1041	Registers (or bit) for an automatic *value* at *address*; None if it does not fit.	`window.group_of_row, window.scales.get, fmt.split('_')[0].startswith, str, encode_register_value`
`SlaveMixin._slave_generate` _slave_generate(self)	1043-1068	Write the current automatic values into the store and show them.	`time.monotonic, set, list, getattr, self._slave_generators.items`
`SlaveMixin.slave_set_generator` slave_set_generator(self, window_id, starts, config)	1070-1082	Set (config) or remove (None) the automatic value of the given addresses.	`self.slave_windows.get, self._slave_sync_generators, self._slave_render_values, self._slave_mark_changed, window.generators.pop`
`SlaveMixin.slave_remove_generator` slave_remove_generator(self, window_id)	1084-1092	Slave remove generator.	`self.slave_windows.get, bool, self.slave_set_generator, self._table_selected_group_starts, self._slave_panes.get`
`SlaveMixin.slave_edit_generator` slave_edit_generator(self, window_id)	1094-1182	Dialog: automatic value for the selected values.	`self.slave_windows.get, self._table_selected_group_starts, self._table_dialog, ttk.Label(body, text=self.trf('Valor automático de {count} v…`
`SlaveMixin.slave_open_definition` slave_open_definition(self, definition, port='', baud='', parity='', stopbits='')	1185-1199	Add a Slave window and show the Slave tab.	`self.slave_add_window, self._slave_apply_connection, self.main_notebook.select, getattr`
`SlaveMixin.slave_open_from_record` slave_open_from_record(self, record)	1201-1214	Slave open from record.	`slave_definition_from_record, self.slave_open_definition, messagebox.showinfo, getattr, self.port_a_var.get().strip`
`SlaveMixin.slave_open_from_master` slave_open_from_master(self, master_window)	1216-1233	Simulate the data area of a Master window (same names, formats and scales).	`self.slave_open_definition, dict, getattr, master_window.scales.items, self.master_baud_var.get`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.slave_set_fault` slave_set_fault(self, slave, fault)	1236-1245	Set (or clear, with None / an inactive fault) the fault of a Slave ID.	`fault_active, self.slave_store.set_fault, self.slave_windows.values, dict, self.slave_faults.pop`
`SlaveMixin.slave_edit_faults` slave_edit_faults(self, window_id)	1247-1322	Slave edit faults.	`self.slave_windows.get, self._table_dialog, ttk.Label(body, text=self.trf(self._slave_fault_intro, slave=slave)).grid, tk.StringVar, tk.Boo…`
`SlaveMixin.slave_connect` slave_connect(self)	1325-1384	Slave connect.	`self.get_serial_modules, self.slave_port_var.get().strip, list, SlaveEngine, self._slave_reset_stats`
`SlaveMixin.slave_disconnect` slave_disconnect(self)	1386-1391	Slave disconnect.	`self.slave_status_var.set, self.slave_disconnect_btn.configure, self.slave_engine.stop, self.tr`
`SlaveMixin._slave_lock_other_features` _slave_lock_other_features(self, locked)	1393-1410	While the Slave listens, nothing else may use a serial port.	`self.slave_connect_btn.configure, self.slave_disconnect_btn.configure, self.slave_refresh_btn.configure, self.set_sniffer_refresh_state, se…`
`SlaveMixin._slave_finish_session` _slave_finish_session(self, payload)	1412-1433	Slave finish session.	`self._slave_lock_other_features, self.slave_status_var.set, (payload or {}).get, engine.stop, self.tr`
`SlaveMixin.handle_slave_event` handle_slave_event(self, event, payload)	1436-1452	Handle slave event.	`self.slave_status_var.set, self.tr, self.trf, self._slave_finish_session, self.slave_port_var.get`
`SlaveMixin.slave_request_text` slave_request_text(self, info)	1454-1471	Short description of a request: "FC03 @ 0 × 10 → Exception 02".	`info.get, self.tr`
`SlaveMixin._slave_apply_request` _slave_apply_request(self, info)	1473-1519	Slave apply request.	`info.get, self._slave_render_stats, time.strftime, self.slave_windows.values, time.localtime`
`SlaveMixin._slave_schedule_tick` _slave_schedule_tick(self)	1521-1523	Slave schedule tick.	`self.after`
`SlaveMixin._slave_tick` _slave_tick(self)	1525-1538	Remove expired write highlights.	`time.monotonic, self.slave_windows.values, self._slave_schedule_tick, self._slave_render_values, bool`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.slave_workspace_data` slave_workspace_data(self)	1541-1552	Configuration only: the register values are not saved.	`self._slave_connection_json, self.slave_windows.values, w.to_json, str, sorted`
`SlaveMixin._slave_connection_json` _slave_connection_json(self)	1554-1560	Slave connection json.	`self.slave_port_var.get, self.slave_baud_var.get, self.slave_parity_var.get, self.slave_stop_var.get`
`SlaveMixin._slave_apply_connection` _slave_apply_connection(self, connection)	1562-1571	Serial settings from a workspace or another tab (valid ones only).	`str, self.slave_baud_var.set, connection.get, self.slave_parity_var.set, self.slave_stop_var.set`
`SlaveMixin._slave_mark_changed` _slave_mark_changed(self)	1573-1574	Slave mark changed.	`self.slave_workspace.mark_changed`
`SlaveMixin._slave_build_workspace_menu` _slave_build_workspace_menu(self)	1576-1578	Slave build workspace menu.	`self.slave_workspace.fill_menu, self._slave_apply_menu_palette`
`SlaveMixin.slave_save_workspace` slave_save_workspace(self, name=None)	1580-1581	Slave save workspace.	`self.slave_workspace.save`
`SlaveMixin.slave_delete_workspace` slave_delete_workspace(self, name, confirm=True)	1583-1584	Slave delete workspace.	`self.slave_workspace.delete`
`SlaveMixin.slave_open_workspace` slave_open_workspace(self, name, quiet=False)	1586-1588	Replace the windows and faults with a saved workspace; values restart at 0.	`self.slave_workspace.open`
`SlaveMixin._slave_load_workspace_data` _slave_load_workspace_data(self, data)	1590-1615	Replace windows and faults with those of *data*; returns the invalid count.	`list, self.slave_store.clear_values, self.slave_store.clear_faults, data.get, self.slave_close_window`
`SlaveMixin.slave_confirm_close` slave_confirm_close(self)	1617-1620	Ask to save a changed Slave workspace; False cancels closing.	`getattr, workspace.confirm_close`
`SlaveMixin.slave_apply_language` slave_apply_language(self)	1623-1636	Slave apply language.	`self.slave_workspace.update_label, self._slave_render_stats, self.slave_windows.values, hasattr, self._slave_panes.get`
`SlaveMixin._slave_apply_menu_palette` _slave_apply_menu_palette(self)	1638-1640	Slave apply menu palette.	`self._table_apply_menus_palette, getattr`

Callable	Lines	Responsibility	Representative direct calls
`SlaveMixin.slave_apply_palette` slave_apply_palette(self, dark)	1642-1653	Slave apply palette.	`self._slave_apply_menu_palette, getattr, getattr(self, '_slave_panes', {}).values, canvas.configure, self._table_configure_tree_tags`
`SlaveMixin.slave_shutdown` slave_shutdown(self)	1655-1668	Slave shutdown.	`getattr, engine.stop, ser.close`

mb_table_ui.py

Window-grid code shared by the Master and Slave tabs. TableUiMixin builds the window header, the value table with drag selection, zebra rows and frame, the right-click menu, the scale and text dialogs, the grid reflow and the Workspace menu; TableWorkspace saves, opens and deletes the workspace files of one tab and tracks its open / modified state; module helpers validate workspace names and list workspace files.

Source lines	SHA-256
792	`6ac1533958761ed7180fc03670d019f0d605c2e62dfc8edfc7e05371b8b08500`

Imports: `json, os, subprocess, sys, tkinter, tkinter.messagebox, ttk, mb_master.FORMAT_SIZES, REGISTER_FORMATS, validate_scale, mb_theme.theme_colors`

Module constants / configuration data

Name	Definition
`TABLE_PANE_MIN_WIDTH`	`600`
`TABLE_PANE_MIN_ROWS`	`3`
`TABLE_PANE_MAX_ROWS`	`15`
`TABLE_STRIPE_LIGHT`	`'#f2f4f6'`
`TABLE_STRIPE_DARK`	`'#262d34'`
`TABLE_WRITTEN_LIGHT`	`'#fff1b8'`
`TABLE_WRITTEN_DARK`	`'#5a4b12'`
`TABLE_GENERATED_LIGHT`	`'#1d4ed8'`
`TABLE_GENERATED_DARK`	`'#7cc4ff'`
`TABLE_MAX_TITLE_LENGTH`	`40`
`SERIAL_PARITIES`	`('None', 'Even', 'Odd')`
`SERIAL_STOP_BITS`	`('1', '2')`
`WORKSPACE_SUFFIX`	`'.mbsw'`
`INVALID_FILENAME_CHARS`	`'<>:"/\\|?*'`
`RESERVED_FILENAMES`	`{'CON', 'PRN', 'AUX', 'NUL'} | {f'COM{i}' for i in range(1, 10)} | {f'LPT{i}' for i in range(1, 10)}`
`WHEEL_GUARD_TAG`	`'MBToolkit.MouseWheelGuard'`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`parse_int_field` parse_int_field(text, minimum, maximum, message)	48-55	Parse int field.	`int, ValueError, str(text).strip, str`
`validate_workspace_name` validate_workspace_name(name)	58-68	Return a cleaned workspace name usable as a Windows file name.	`str(name or '').strip().rstrip, ValueError, any, name.split('.')[0].upper, str(name or '').strip`
`workspace_path` workspace_path(name, folder)	71-72	Workspace path.	`validate_workspace_name`
`list_workspaces` list_workspaces(folder)	75-81	Names of the saved workspaces in *folder*, sorted alphabetically.	`folder.glob, sorted, f.is_file`
`apply_format_to_rows` apply_format_to_rows(window, rows, fmt)	84-115	Set the per-address format of the selected rows of *window*.	`sorted, list, set, ValueError, window.formats.items`
`validated_names` validated_names(names)	118-120	Validated names.	`int, str, isinstance, names.items, str(k).isdigit`
`validated_formats` validated_formats(formats)	123-125	Validated formats.	`int, isinstance, formats.items, str(k).isdigit, str`
`validated_scales` validated_scales(scales)	128-140	Scales of a workspace window entry; invalid ones are dropped.	`isinstance, scales.items, validate_scale, item.get, int`
`class TableWorkspace`	**143-294**	**Named workspace files of one tab (Master or Slave) and their state.**	**Class container**
`TableWorkspace.__init__` __init__(self, app, *, folder, formats, data, load, label_var, close_question)	154-165	Init.	`tuple`
`TableWorkspace.names` names(self)	167-168	Names.	`list_workspaces, self.folder`
`TableWorkspace.path` path(self, name)	170-171	Path.	`workspace_path, self.folder`
`TableWorkspace.update_label` update_label(self)	173-181	Update label.	`self.label_var.set, app.trf`
`TableWorkspace._set` _set(self, name, save_settings=True)	183-188	Set.	`self.update_label, self.app._schedule_ui_settings_save`
`TableWorkspace.mark_changed` mark_changed(self)	190-196	Record a change to the open workspace (ignored while loading one).	`self.update_label`

Callable	Lines	Responsibility	Representative direct calls
`TableWorkspace.forget` forget(self)	198-201	No workspace open any more (e.g. every window was closed).	`self._set`
`TableWorkspace.fill_menu` fill_menu(self, menu)	203-207	Fill menu.	`self.app._table_fill_workspace_menu, self.names, self.app._table_open_folder, self.folder`
`TableWorkspace.save` save(self, name=None)	209-236	Save the tab under *name* (asked when None); True when saved.	`self.path, self._set, app._table_ask_text, validate_workspace_name, path.exists`
`TableWorkspace.delete` delete(self, name, confirm=True)	238-251	Delete.	`self.path(name).unlink, self._set, messagebox.askyesno, messagebox.showerror, app.tr`
`TableWorkspace.open` open(self, name, quiet=False)	253-280	Replace the tab's windows with a saved workspace (by name).	`self._set, self.load, messagebox.showwarning, open, json.load`
`TableWorkspace.confirm_close` confirm_close(self)	282-294	Ask to save a changed workspace; False cancels closing the program.	`messagebox.askyesnocancel, app.tr, app.trf, self.save`
`class TabInstance`	**297-314**	**Second copy of a tab mixin with its own state, sharing the application.**	**Class container**
`TabInstance.__init__` __init__(self, app)	308-309	Init.	—
`TabInstance.__getattr__` __getattr__(self, name)	311-314	Getattr.	`getattr, AttributeError`
`class TableUiMixin`	**317-792**	**Widget code shared by the Master and Slave window grids.**	**Class container**
`TableUiMixin._table_root` _table_root(self)	320-322	Tk parent for new top-level windows and menus of this tab.	`getattr`
`TableUiMixin._table_dialog` _table_dialog(self, title)	325-333	Table dialog.	`tk.Toplevel, dialog.title, dialog.transient, dialog.resizable, self._table_apply_toplevel_palette`
`TableUiMixin._table_ask_text` _table_ask_text(self, title, caption, initial)	335-358	Small modal text prompt; returns the text or None when cancelled.	`self._table_dialog, ttk.Label(body, text=caption).grid, tk.StringVar, ttk.Entry, entry.grid`

Callable	Lines	Responsibility	Representative direct calls
`TableUiMixin._table_ask_scale` _table_ask_scale(self, window, starts)	360-428	Ask for a linear scale and set it on the values starting at *starts*.	`self._table_dialog, ttk.Label(body, text=self.trf('Escala de {count} valor(es) selecionado(s):', count=len(starts))).grid, ttk.Label(body, …`
`TableUiMixin._table_build_header` _table_build_header(self, frame, pane, rename, popout, close)	431-456	Window name + summary on the left; Rename / Pop out / ✕ on the right.	`ttk.Frame, header.grid, header.columnconfigure, tk.StringVar, ttk.Label`
`TableUiMixin._table_build_tree` _table_build_tree(self, frame, pane, row, on_double, on_return, on_menu)	458-489	Value table (address / name / value) with a thin 1 px frame.	`tk.Frame, table.grid, table.columnconfigure, table.rowconfigure, ttk.Treeview`
`TableUiMixin._table_translate_captions` _table_translate_captions(self, *containers)	491-499	Captions are written in canonical PT; show them in the current language.	`widget.winfo_children, child.cget, child.configure, child.keys, self.tr`
`TableUiMixin._table_translate_headings` _table_translate_headings(self, pane)	501-506	Table translate headings.	`pane['tree'].heading, self.tr`
`TableUiMixin._table_row_tags` _table_row_tags(index)	509-511	Alternate rows get a slightly darker background (zebra striping).	—
`TableUiMixin._table_configure_tree_tags` _table_configure_tree_tags(self, tree)	513-531	Table configure tree tags.	`bool, theme_colors, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, tree.tag_configure, isinstance`
`TableUiMixin._table_reflow` _table_reflow(self, canvas, grid, grid_item, frames, empty_label, uniform)	534-553	Arrange docked pane *frames* in as many equal columns as the width allows.	`max, canvas.itemconfigure, range, enumerate, canvas.winfo_width`
`TableUiMixin._table_build_scroll_grid` _table_build_scroll_grid(self, page, row)	555-570	Scrollable area that holds the window grid; returns (canvas, grid, item).	`ttk.Frame, holder.grid, holder.columnconfigure, holder.rowconfigure, tk.Canvas`
`TableUiMixin._table_install_wheel_tags` _table_install_wheel_tags(self, root_widget, wheel_tag)	572-586	Give new widgets the global wheel guard; plain panels scroll the grid.	`stack.pop, stack.extend, tuple, isinstance, widget.winfo_children`
`TableUiMixin._table_wheel` _table_wheel(canvas, event)	589-605	Table wheel.	`canvas.yview, canvas.yview_scroll, getattr`
`TableUiMixin._table_drag_start` _table_drag_start(self, pane, event)	612-621	Table drag start.	`tree.identify_row, bool, tuple, tree.selection, tree.identify_region`

Callable	Lines	Responsibility	Representative direct calls
`TableUiMixin._table_drag_move` _table_drag_move(self, pane, event)	623-629	Table drag move.	`pane.get, self._table_drag_update`
`TableUiMixin._table_drag_update` _table_drag_update(self, pane)	631-650	Table drag update.	`pane.get, tree.winfo_height, tree.identify_row, tree.get_children, sorted`
`TableUiMixin._table_drag_stop` _table_drag_stop(self, pane)	652-659	Table drag stop.	`pane.get, drag.get, self.after_cancel`
`TableUiMixin._table_selected_rows` _table_selected_rows(pane)	663-666	Table selected rows.	`sorted, int, pane['tree'].selection, iid.startswith`
`TableUiMixin._table_selected_group_starts` _table_selected_group_starts(self, window, pane)	668-676	PDU addresses where the values of the selected rows start.	`self._table_selected_rows, window.group_of_row, starts.append`
`TableUiMixin._table_copy_rows` _table_copy_rows(self, pane)	678-687	Copy the selected rows (all rows if none) as tab-separated text.	`list, '\t'.join, self.clipboard_clear, self.clipboard_append, tree.selection`
`TableUiMixin._table_fill_row_menu` _table_fill_row_menu(self, menu, pane, window, registers, *, set_format, edit_scale, remove_scale, copy, set_base, extra=())	689-728	Right-click menu: Formato ▸, Repor, Escala..., Remover escala, Copiar, Base ▸.	`menu.delete, menu.winfo_children, tk.Menu, format_menu.add_separator, format_menu.add_cascade`
`TableUiMixin._table_popup` _table_popup(menu, event)	731-735	Table popup.	`menu.tk_popup, menu.grab_release`
`TableUiMixin._table_fill_workspace_menu` _table_fill_workspace_menu(self, menu, names, *, save, open_, delete, folder)	738-756	Workspace menu with the saved workspaces listed directly in it.	`menu.delete, menu.winfo_children, menu.add_command, tk.Menu, menu.add_cascade`
`TableUiMixin._table_open_folder` _table_open_folder(self, folder)	758-767	Table open folder.	`os.startfile, messagebox.showerror, str, subprocess.Popen, self.tr`
`TableUiMixin._table_apply_toplevel_palette` _table_apply_toplevel_palette(self, top)	770-775	Table apply toplevel palette.	`bool, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, top.configure, getattr, tk.BooleanVar`
`TableUiMixin._table_apply_menus_palette` _table_apply_menus_palette(self, *menus)	777-792	Colour menus and every submenu below them (classic Tk widgets).	`bool, theme_colors, getattr(self, 'dark_mode_var', tk.BooleanVar(value=False)).get, stack.pop, stack.extend`

> **Design note.** A pane is a plain dict of widgets and a window a model object, so the same widget code serves MasterWindow and SlaveWindow.

mb_tcp.py

Modbus TCP transport. MBAP framing and conversion to and from RTU frames, ServerEngine (several clients with selectors, Unit IDs, faults, Exception 0B), ClientEngine (the Master scheduler over one TCP connection per device) and the list of the PC's IPv4 interfaces. It contains no widgets.

Source lines	SHA-256
500	`53c21c5603cf6a3664c2b5e5bbb0f55abe0f9440341b1f3ff568a200b5fff4e6`

Imports: `ipaddress, json, os, random, selectors, socket, subprocess, threading, time, dataclasses.dataclass, mb_master.MasterEngine, PollDefinition, build_read_request, find_master_response, mb_protocol.append_crc, hex_bytes, mb_slave.FC_TABLES, handle_request`

Module constants / configuration data

Name	Definition
`MBAP_LENGTH`	`7`
`MODBUS_TCP_PORT`	`502`
`MBAP_MAX_LENGTH`	`254`
`EXCEPTION_GATEWAY_TARGET_FAILED`	`11`
`SERVER_MAX_CLIENTS`	`16`
`CLIENT_RETRY_SECONDS`	`1.0`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`build_adu` build_adu(transaction, unit, pdu, protocol=0)	46-50	MBAP header + PDU.	`bytes, (len(pdu) + 1).to_bytes, transaction.to_bytes, protocol.to_bytes, len`
`adu_length` adu_length(buffer)	53-64	Total length of the ADU starting at buffer[0]; None if the header is incomplete.	`len, ValueError`
`parse_adu` parse_adu(adu)	67-69	(transaction, protocol, unit, pdu) of one complete ADU.	`bytes`
`rtu_to_adu` rtu_to_adu(rtu_frame, transaction)	72-74	ADU for an RTU frame: Slave ID becomes Unit ID, the CRC is dropped.	`build_adu`

Callable	Lines	Responsibility	Representative direct calls
`adu_to_rtu` adu_to_rtu(unit, pdu)	77-79	Equivalent RTU frame (Unit ID as Slave ID, valid CRC) of a TCP PDU.	`append_crc, bytes`
`validate_host` validate_host(text)	82-87	Return a cleaned IP address or host name; raise ValueError (PT).	`str(text or '').strip, any, ValueError, str, len`
`validate_tcp_port` validate_tcp_port(text)	90-97	Validate tcp port.	`int, ValueError, str(text).strip, str`
`list_ipv4_interfaces` list_ipv4_interfaces()	100-140	[(ip, interface name)] of this PC's IPv4 addresses, loopback excluded.	`sorted, set, seen.add, result.append, subprocess.run(['powershell', '-NoProfile', '-NonInteractive', '-Command', command], capture_output=T…`
`_request_area` _request_area(fc, pdu)	147-153	(table, address, qty) named by a request PDU, for the window counters.	`len`
`class ServerEngine`	**156-353**	**Modbus TCP server: answers several clients from one SlaveStore.**	**Class container**
`ServerEngine.__init__` __init__(self, event_queue, store, *, host, port, rng=None, max_clients=SERVER_MAX_CLIENTS)	169-181	Init.	`int, threading.Event, random.Random`
`ServerEngine.start` start(self)	183-185	Start.	`threading.Thread, self.thread.start`
`ServerEngine.stop` stop(self, join_timeout=1.0)	187-191	Stop.	`self.stop_event.set, thread.is_alive, thread.join, threading.current_thread`
`ServerEngine.run` run(self)	193-232	Run.	`selectors.DefaultSelector, selector.register, self.event_queue.put, list, selector.close`
`ServerEngine._accept` _accept(self, listener, selector)	235-248	Accept.	`sock.setblocking, selector.register, self.event_queue.put, listener.accept, len`
`ServerEngine._drop` _drop(self, sock, selector, notify=True)	250-263	Drop.	`self._clients.pop, selector.unregister, sock.close, self.event_queue.put, len`
`ServerEngine._receive` _receive(self, sock, selector)	265-290	Receive.	`self._clients.get, buffer.extend, sock.recv, self._drop, bytes`
`ServerEngine._send` _send(self, sock, data)	292-296	Send.	`sock.sendall`

Callable	Lines	Responsibility	Representative direct calls
`ServerEngine._send_due` _send_due(self, selector)	298-306	Send due.	`time.perf_counter, self._send`
`ServerEngine._answer` _answer(self, sock, peer, adu)	309-353	Answer.	`parse_adu, _request_area, self.event_queue.put, time.time, self.store.slave_ids`
`class ClientPollDefinition`	**361-370**	**Poll window of the TCP Client: the device's address and Unit ID 0–255.**	**Class container**
`ClientPollDefinition.request` request(self)	366-370	Request.	`build_read_request, append_crc, bytes`
`class ClientEngine`	**373-500**	**Modbus TCP client: the Master scheduler over one connection per device.**	**Class container**
`ClientEngine.__init__` __init__(self, event_queue, *, response_timeout_ms, delay_ms)	384-391	Init.	`super().__init__, max, super, float`
`ClientEngine.run` run(self)	393-402	Run.	`super().run, self._sockets.values, super, sock.close`
`ClientEngine._connection` _connection(self, target)	404-420	Connection.	`self._sockets.get, self._failed.get, self._failed.pop, ConnectionError, socket.create_connection`
`ClientEngine._close` _close(self, target)	422-428	Close.	`self._sockets.pop, sock.close`
`ClientEngine.transact_for` transact_for(self, window_id, request)	430-500	Transact for.	`append_crc, rtu_to_adu, hex_bytes, time.perf_counter, bytearray`

> **Design note.** Only the envelope and the sockets are TCP-specific: every request and response goes through the same Modbus core as the RTU tabs.

mb_tcp_ui.py

Modbus TCP Client and Server tabs. ClientTab and ServerTab are second instances of MasterMixin and SlaveMixin on TabInstance; they replace the connection bar, add the device row of each Client window, choose the Server listening address and start the TCP engines.

Source lines	SHA-256
509	`1ad3419cca195d5339496327086e44eaa7aed452d556bb86a41a25ff6c56a34a`

Imports: `ipaddress, threading, tkinter, tkinter.messagebox, ttk, mb_config.CLIENT_WORKSPACE_FORMAT, MASTER_DEFAULT_DELAY_MS, MASTER_DEFAULT_RESPONSE_TIMEOUT_MS, MASTER_MAX_DELAY_MS, MASTER_MAX_RESPONSE_TIMEOUT_MS, MASTER_MIN_DELAY_MS, MASTER_MIN_RESPONSE_TIMEOUT_MS, SERVER_DEFAULT_BIND, SERVER_WORKSPACE_FORMAT, TCP_DEFAULT_PORT, get_client_workspaces_folder, get_server_workspaces_folder, mb_master.validate_read_definition, mb_master_ui.MasterMixin, MasterWindow, mb_slave_ui.SlaveMixin, mb_table_ui.TabInstance, parse_int_field, mb_tcp.ClientEngine, ClientPollDefinition, ServerEngine, list_ipv4_interfaces, validate_host, validate_tcp_port, mb_widgets.RefreshButton`

Module constants / configuration data

Name	Definition
`LOCALHOST`	`'127.0.0.1'`
`ALL_INTERFACES`	`'0.0.0.0'`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class ClientWindow`	**55-85**	**Poll window of the Client: a Master window plus the device (host:port).**	**Class container**
`ClientWindow.__init__` __init__(self, window_id, number, host=LOCALHOST, port=TCP_DEFAULT_PORT, **kwargs)	61-64	Init.	`super().__init__, super`
`ClientWindow.definition` definition(self)	66-68	Definition.	`ClientPollDefinition`
`ClientWindow.to_json` to_json(self)	70-73	To json.	`super().to_json, data.update, super`

Callable	Lines	Responsibility	Representative direct calls
`ClientWindow.validated_kwargs` validated_kwargs(data)	76-85	Validated kwargs.	`int, MasterWindow.validated_kwargs, kwargs.update, isinstance, ValueError`
`class ClientTab`	**88-256**	**Modbus TCP Client tab: the Master tab with one TCP device per window.**	**Class container**
`ClientTab.__init__` __init__(self, app)	107-110	Init.	`super().__init__, super`
`ClientTab._master_build_connection` _master_build_connection(self, page, saved)	113-151	Master build connection.	`ttk.LabelFrame, connection.grid, ttk.Frame, row.grid, tk.StringVar`
`ClientTab._master_connection_json` _master_connection_json(self)	153-155	Master connection json.	`self.master_timeout_var.get, self.master_delay_var.get`
`ClientTab._master_apply_connection` _master_apply_connection(self, connection)	157-161	Master apply connection.	`str(connection.get(key, '')).isdigit, var.set, str, connection.get`
`ClientTab.settings_json` settings_json(self)	163-166	Settings json.	`self._master_connection_json`
`ClientTab._master_build_target` _master_build_target(self, frame, pane, window)	169-181	Master build target.	`ttk.Frame, target.grid, tk.StringVar, enumerate, ttk.Label(target, text=caption).grid`
`ClientTab._master_parse_target` _master_parse_target(self, pane)	183-185	Master parse target.	`validate_host, validate_tcp_port, pane['host_var'].get, pane['port_var'].get`
`ClientTab._master_validate_definition` _master_validate_definition(self, slave, fc, address, qty)	187-189	Master validate definition.	`validate_read_definition`
`ClientTab._master_fields_changed` _master_fields_changed(self, window, pane)	191-198	Master fields changed.	`super()._master_fields_changed, super, pane['host_var'].get().strip, pane['port_var'].get().strip, str`
`ClientTab._master_window_summary` _master_window_summary(self, window)	200-201	Master window summary.	`super()._master_window_summary, super`
`ClientTab.master_build_write` master_build_write(self, window, slave_text, address_text, values_text, fc, fmt=None, scale=None)	203-209	Write request for the Unit ID (0–255); never a broadcast in Modbus TCP.	`parse_int_field, super().master_build_write, super, bytes`

Callable	Lines	Responsibility	Representative direct calls
`ClientTab.open_in_server` open_in_server(self, window)	211-212	Open in server.	`self.app.server_tab.slave_open_from_master`
`ClientTab._master_connection_settings` _master_connection_settings(self)	215-224	Master connection settings.	`parse_int_field, self.master_timeout_var.get, self.master_delay_var.get`
`ClientTab.master_connect` master_connect(self)	226-246	Master connect.	`list, ClientEngine, self.master_windows.values, self._master_lock_other_features, self.master_status_var.set`
`ClientTab._master_lock_other_features` _master_lock_other_features(self, locked)	248-256	Only this tab's own controls: TCP does not lock the serial features.	`self.master_connect_btn.configure, self.master_disconnect_btn.configure, widget.configure`
`class ServerTab`	**263-509**	**Modbus TCP Server tab: the Slave tab listening on an IP and TCP port.**	**Class container**
`ServerTab.__init__` __init__(self, app)	280-285	Init.	`super().__init__, super`
`ServerTab._slave_build_connection` _slave_build_connection(self, page, saved)	288-340	Slave build connection.	`ttk.LabelFrame, connection.grid, ttk.Frame, row.grid, saved.get`
`ServerTab._server_choices` _server_choices(self)	343-354	[(ip, caption)] offered in "Escutar em", this PC first, all last.	`choices.append, self.tr, self.trf`
`ServerTab._server_fill_interfaces` _server_fill_interfaces(self)	356-360	Server fill interfaces.	`self._server_choices, next, self.server_bind_var.set`
`ServerTab._server_bind_selected` _server_bind_selected(self, _event=None)	362-367	Server bind selected.	`self.server_bind_var.get, self._server_choices, self._schedule_ui_settings_save`
`ServerTab.server_refresh_interfaces` server_refresh_interfaces(self)	369-376	List this PC's IPv4 interfaces in the background (PowerShell can take 1 s).	`threading.Thread(target=worker, daemon=True).start, event_queue.put, threading.Thread, list_ipv4_interfaces`
`ServerTab._slave_connection_json` _slave_connection_json(self)	378-379	Slave connection json.	`self.server_port_var.get`
`ServerTab._slave_apply_connection` _slave_apply_connection(self, connection)	381-390	Slave apply connection.	`connection.get, str(connection.get('port', '')).isdigit, self.server_port_var.set, str, self._server_fill_interfaces`

Callable	Lines	Responsibility	Representative direct calls
`ServerTab.settings_json` settings_json(self)	392-395	Settings json.	`self._slave_connection_json`
`ServerTab._slave_render_stats` _slave_render_stats(self)	398-403	Slave render stats.	`self.slave_stats_var.set, self.trf, len, getattr`
`ServerTab._server_status_text` _server_status_text(self)	405-406	Server status text.	`self.trf`
`ServerTab.slave_connect` slave_connect(self)	409-443	Slave connect.	`list, ServerEngine, self._slave_reset_stats, self._slave_render_stats, self.slave_windows.values`
`ServerTab._slave_lock_other_features` _slave_lock_other_features(self, locked)	445-455	Only this tab's own controls: TCP does not lock the serial features.	`self.slave_connect_btn.configure, self.slave_disconnect_btn.configure, self.slave_refresh_btn.configure, widget.configure, isinstance`
`ServerTab._slave_finish_session` _slave_finish_session(self, payload)	457-476	Slave finish session.	`self._slave_lock_other_features, self.slave_status_var.set, self._slave_render_stats, (payload or {}).get, engine.stop`
`ServerTab.handle_server_event` handle_server_event(self, event, payload)	478-495	Handle server event.	`self.slave_status_var.set, self._server_status_text, payload.get, self._slave_render_stats, self._server_clients.pop`
`ServerTab.slave_apply_language` slave_apply_language(self)	497-502	Slave apply language.	`super().slave_apply_language, hasattr, self._server_fill_interfaces, self.slave_status_var.set, super`
`ServerTab.slave_shutdown` slave_shutdown(self)	504-509	Slave shutdown.	`getattr, engine.stop`

> **Design note.** The tab classes contain only what differs from the serial tabs; everything else is inherited from MasterMixin and SlaveMixin.

mb_i18n.py

Runtime Portuguese/English translation layer. Portuguese (Portugal) is the canonical UI text. Static translations, runtime-status patterns, protocol-detail fragments and the complete English Help content are kept separate from protocol logic so switching language affects presentation only.

Source lines	SHA-256
729	`75061bc71561f3895deb33bf3d789d3bcfa8401f6b4350bb8a09d8ed65d67f82`

Imports: `mb_config.DISCOVERY_IDLE_CHECK_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS`

Module constants / configuration data

Name	Definition
`LANGUAGE_PORTUGUESE`	`'Português'`
`LANGUAGE_ENGLISH`	`'English'`
`LANGUAGE_CHOICES`	`(LANGUAGE_PORTUGUESE, LANGUAGE_ENGLISH)`
`PT_TO_EN`	`{'Linguagem': 'Language', 'Modo escuro': 'Dark mode', 'Configuração': 'Configuration', 'Modo físico': 'Physical mode', 'RS232 dual RX: COM A escuta…`
`EN_TO_PT`	`{english: portuguese for portuguese, english in PT_TO_EN.items()}`
`EXCEPTION_DESCRIPTION_EN`	`{1: 'The requested Modbus function is not supported or permitted by the device.', 2: 'The requested address or address range is not valid for this …`
`DETAIL_FRAGMENTS_PT_TO_EN`	`(('Registos (PDU Address=valor):', 'Registers (PDU Address=value):'), ('Bits (PDU Address=valor):', 'Bits (PDU Address=value):'), ('Registos=', 'Re…`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`normalize_language` normalize_language(value)	425-426	Normalize language.	—

Callable	Lines	Responsibility	Representative direct calls
`canonical_ui_text` canonical_ui_text(text)	429-432	Normalises a known English caption back to its canonical PT-PT form.	`str, EN_TO_PT.get`
`translate_text` translate_text(text, language)	435-440	Returns the English translation for known canonical PT-PT UI strings when English is selected; unknown/technical terms are returned unchanged.	`canonical_ui_text, normalize_language, PT_TO_EN.get`
`translate_details` translate_details(text, language)	451-460	Translates selected human-readable fragments embedded inside protocol detail strings (DETAIL_FRAGMENTS_PT_TO_EN) while preserving numbers, addresses, register values and other protocol data.	`str, normalize_language, value.replace`
`exception_description` exception_description(code, portuguese_text, language)	463-466	Exception description.	`normalize_language, EXCEPTION_DESCRIPTION_EN.get`
`english_help_blocks` english_help_blocks(max_ui_frames_text, max_raw_lines_text, min_pending, max_pending, debug_sim=False)	470-700	Returns the complete English Help / Connections document as structured blocks so Help switches language together with the rest of the UI.	`blocks.extend`
`translate_runtime_text` translate_runtime_text(text, language)	703-729	Uses regular expressions to translate live status strings that contain counters, elapsed time, selected configuration or estimated duration.	`str, translate_text, normalize_language, re.match, re.sub`

Detailed behavior of the key paths

`canonical_ui_text` lines 429-432

Normalises a known English caption back to its canonical PT-PT form. This makes language switching reversible instead of translating an already translated string repeatedly.

`translate_text` lines 435-440

Returns the English translation for known canonical PT-PT UI strings when English is selected; unknown/technical terms are returned unchanged.

`translate_details` **lines 451-460**

Translates selected human-readable fragments embedded inside protocol detail strings (DETAIL_FRAGMENTS_PT_TO_EN) while preserving numbers, addresses, register values and other protocol data.

`english_help_blocks` **lines 470-700**

Returns the complete English Help / Connections document as structured blocks so Help switches language together with the rest of the UI.

`translate_runtime_text` **lines 703-729**

Uses regular expressions to translate live status strings that contain counters, elapsed time, selected configuration or estimated duration.

> **Design note.** The translation design treats PT-PT as canonical and leaves technical English terms unchanged unless a translation is explicitly defined.

mb_theme.py

Fixed-geometry Light/Dark theme implementation. It configures a single ttk clam layout once, installs stable checkbox geometry and changes only palette values at runtime. This is deliberately designed so theme switching does not move or resize controls.

Source lines	SHA-256
692	`7b03e5a4f4cc646a96cd1839d2a92794ea923d232ae7e50a93b5908ac4ab8d9e`

Imports: `tkinter, tkinter.ttk`

Module constants / configuration data

Name	Definition
`BOOTSTRAP_ACTIVE`	`False`
`THEME_ENGINE`	`'clam'`
`LIGHT`	`{'root': '#f3f4f6', 'panel': '#f3f4f6', 'panel_alt': '#ffffff', 'input': '#ffffff', 'input_readonly': '#f5f6f7', 'text': '#1f2328', 'muted': '#5963…`
`DARK`	`{'root': '#171a1f', 'panel': '#171a1f', 'panel_alt': '#1f242a', 'input': '#20262c', 'input_readonly': '#252c33', 'text': '#e8eaed', 'muted': '#aab2…`
`_CHECK_IMAGES`	`None`
`_CHECK_ELEMENT`	`'MBS.Check.indicator'`
`_FIXED_GEOMETRY_READY`	`False`
`_INTERP_ID`	`None`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`theme_colors` theme_colors(root=None, dark=False)	70-71	Theme colors.	`dict`
`_ensure_engine` _ensure_engine(style)	74-95	Select/configure the fixed engine once per Tk interpreter.	`id, _configure_fixed_geometry, style.theme_use, style.theme_names`

Callable	Lines	Responsibility	Representative direct calls
`_configure_fixed_geometry` _configure_fixed_geometry(style)	98-296	Defines ttk element layouts, paddings, borders, row heights and widget metrics once.	`style.configure, style.map, _install_fixed_checkbutton_element, style.layout`
`_paint_checkbox_image` _paint_checkbox_image(image, *, fill, border, check=None)	299-314	Repaint one fixed-size checkbox image without changing its dimensions.	`image.put`
`_install_fixed_checkbutton_element` _install_fixed_checkbutton_element(style)	317-367	Creates the custom checkbox indicator element/images used to keep checkbox dimensions stable across palette changes.	`style.layout, repr, tk.PhotoImage, style.element_names, style.element_create`
`_repaint_checkboxes` _repaint_checkboxes(colors, dark)	370-408	Repaint checkboxes.	`_paint_checkbox_image`
`_apply_palette` _apply_palette(root, style, colors, dark)	411-666	Changes ttk colors/state maps and classic Tk colors without changing layout definitions, supporting the invariant that theme switching changes appearance but not geometry.	`style.configure, style.map, _repaint_checkboxes, root.configure`
`apply_theme` apply_theme(root, dark=False, style=None)	669-677	Change palette only.	`_ensure_engine, _apply_palette, ttk.Style, bool`
`apply_light_theme` apply_light_theme(root, style=None)	680-681	Apply light theme.	`apply_theme`
`apply_dark_theme` apply_dark_theme(root, style=None)	684-685	Apply dark theme.	`apply_theme`
`apply_modern_theme` apply_modern_theme(root, dark=False)	688-692	Public startup helper that selects the fixed clam engine and applies either the Light or Dark palette.	`ttk.Style, _ensure_engine, _apply_palette, bool`

Detailed behavior of the key paths

`_configure_fixed_geometry` lines 98-296

Defines ttk element layouts, paddings, borders, row heights and widget metrics once. These values are intentionally independent of Light/Dark selection.

`_install_fixed_checkbutton_element` lines 317-367

Creates the custom checkbox indicator element/images used to keep checkbox dimensions stable across palette changes.

`_apply_palette` **lines 411-666**

Changes ttk colors/state maps and classic Tk colors without changing layout definitions, supporting the invariant that theme switching changes appearance but not geometry.

`apply_modern_theme` **lines 688-692**

Public startup helper that selects the fixed clam engine and applies either the Light or Dark palette.

Design note. The theme module is geometry-sensitive: changing colors must not rebuild ttk layouts. This is why theme engine selection and fixed geometry are separated from palette application.

mb_widgets.py

Custom reusable Tkinter widgets. RefreshButton draws a stable square refresh control on a Canvas; ToggleSwitch draws the Light/Dark switch. Both implement their own state, hover/press behavior and palette updates.

Source lines	SHA-256
418	`3a732639b6813ae827290b8ca5af567158b2ee736e941c89ae3d77ea7a184d2f`

Imports: `tkinter`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class RefreshButton`	**7-250**	**Canvas-based square button that draws the ↻ glyph and recentres it from its actual rendered bounding box.**	**Class container**
`RefreshButton.__init__` __init__(self, master, command, size=26, cursor='hand2', **kwargs)	20-53	Init.	`super().__init__, self.bind, self._redraw, self._schedule_final_redraw, super`
`RefreshButton._system_color` _system_color(widget, name, fallback)	56-61	System color.	`widget.winfo_rgb`
`RefreshButton._schedule_final_redraw` _schedule_final_redraw(self)	63-77	Redraw after Tk has settled requested widget dimensions.	`self.after, self.after_cancel`
`RefreshButton._final_redraw` _final_redraw(self)	79-84	Final redraw.	`self._redraw`
`RefreshButton.configure` configure(self, cnf=None, **kwargs)	86-97	Configure.	`kwargs.pop, super().configure, str, self._redraw, self._schedule_final_redraw`
`RefreshButton.cget` cget(self, key)	101-104	Cget.	`super().cget, super`
`RefreshButton.invoke` invoke(self)	106-109	Invoke.	`callable, self._command`
`RefreshButton.set_dark_mode` set_dark_mode(self, enabled)	111-114	Set dark mode.	`bool, self._redraw, self._schedule_final_redraw`
`RefreshButton._palette` _palette(self)	116-146	Palette.	`self._system_color`

Callable	Lines	Responsibility	Representative direct calls
`RefreshButton._center_item_bbox` _center_item_bbox(self, item, width, height)	148-181	Center the visible Tk bbox of one Canvas item on the square.	`range, self.bbox, self.move, round, abs`
`RefreshButton._redraw` _redraw(self, _event=None)	183-220	Redraw.	`self._palette, self.delete, self.create_rectangle, self.create_text, self._center_item_bbox`
`RefreshButton._on_enter` _on_enter(self, _event)	222-225	Event handler for enter.	`self._redraw`
`RefreshButton._on_leave` _on_leave(self, _event)	227-230	Event handler for leave.	`self._redraw`
`RefreshButton._on_press` _on_press(self, _event)	232-235	Event handler for press.	`self._redraw`
`RefreshButton._on_release` _on_release(self, event)	237-250	Event handler for release.	`self._redraw, callable, self._command, self.winfo_width, self.winfo_height`
`class ToggleSwitch`	**254-418**	**Canvas-based boolean switch bound to a Tk variable.**	**Class container**
`ToggleSwitch.__init__` __init__(self, master, variable, command=None, width=42, height=22, cursor='hand2', **kwargs)	257-292	Init.	`super().__init__, self.bind, self.variable.trace_add, self._redraw, self._schedule_final_redraw`
`ToggleSwitch._schedule_final_redraw` _schedule_final_redraw(self)	294-300	Schedules final redraw.	`self.after, self.after_cancel`
`ToggleSwitch._final_redraw` _final_redraw(self)	302-307	Final redraw.	`self._redraw`
`ToggleSwitch.set_dark_mode` set_dark_mode(self, enabled)	309-312	Set dark mode.	`bool, self._redraw, self._schedule_final_redraw`
`ToggleSwitch._parent_bg` _parent_bg(self)	314-321	Parent bg.	`self.winfo_rgb`
`ToggleSwitch._press` _press(self, _event=None)	323-326	Press.	—
`ToggleSwitch._toggle` _toggle(self, _event=None)	328-332	Toggle.	`self.variable.set, callable, self.command, bool, self.variable.get`
`ToggleSwitch._enter` _enter(self, _event=None)	334-336	Enter.	`self._redraw`
`ToggleSwitch._leave` _leave(self, _event=None)	338-340	Leave.	`self._redraw`

Callable	Lines	Responsibility	Representative direct calls
`ToggleSwitch._redraw` _redraw(self, _event=None)	342-418	Redraw.	`bool, self.configure, self.delete, self.create_rectangle, self.create_oval`

Detailed behavior of the key paths

`RefreshButton` lines 7-250

Canvas-based square button that draws the ↻ glyph and recentres it from its actual rendered bounding box. It supports normal/hover/pressed/disabled states and palette changes.

`ToggleSwitch` lines 254-418

Canvas-based boolean switch bound to a Tk variable. It toggles the variable, invokes the callback and redraws track/knob colors while preserving fixed dimensions.

mb_gui.py

Main application composition and global UI behavior. SnifferApp combines MasterMixin, SlaveFinderMixin, CaptureMixin and ViewMixin with tk.Tk. It creates all widgets, initializes state, freezes language-independent geometry, performs runtime translation, manages theme/language persistence, prevents mouse-wheel tab switching and keeps Bus Health content responsive.

Source lines	SHA-256
2748	`56a2d86cdc6fb7d298e149b733ab84c3ad473bd7b6dcd3033a6a477cd51fd28f`

Imports: `queue, threading, collections.deque, tkinter, tkinter.ttk, tkinter.font, mb_config.APP_NAME, APP_VERSION, DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS, DISCOVERY_IDLE_CHECK_SECONDS, MAX_PENDING_REQUEST_TIMEOUT_SECONDS, MAX_RAW_TEXT_LINES, MAX_UI_FRAMES, MIN_PENDING_REQUEST_TIMEOUT_SECONDS, SLOW_RESPONSE_THRESHOLD_MS, SNIFFER_SPLIT_FRAME_HOLD_MS, debug_sim, load_ui_settings, resource_path, save_ui_settings, mb_protocol.SessionMetrics, mb_slave_finder.SlaveFinderMixin, mb_master_ui.MasterMixin, mb_slave_ui.SlaveMixin, mb_tcp_ui.ClientTab, ServerTab, mb_capture.CaptureMixin, mb_view.TRAFFIC_MENU_OPEN_IN_MASTER, TRAFFIC_MENU_OPEN_IN_SLAVE, ViewMixin, mb_widgets.RefreshButton, ToggleSwitch, mb_theme.apply_modern_theme, apply_theme, theme_colors, mb_i18n.LANGUAGE_CHOICES, LANGUAGE_ENGLISH, LANGUAGE_PORTUGUESE, canonical_ui_text, english_help_blocks, normalize_language, translate_text, translate_runtime_text`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class SnifferApp`	**49-2747**	**SnifferApp.**	**Class container**
`SnifferApp.__init__` __init__(self)	50-155	Creates the root window, loads persisted preferences before widget construction, initializes capture/discovery/log/history state, constructs the UI, freezes language geometry, applies language, installs event guards/persistence and schedules queue/status polling.	`super().__init__, self.title, load_ui_settings, normalize_language, bool`
`SnifferApp._freeze_language_layout_geometry` _freeze_language_layout_geometry(self)	158-218	Measures the canonical Portuguese layout and freezes key container dimensions/minimums so changing to English changes text only, not panel/control geometry.	`getattr, self.update_idletasks, enumerate, max, overlay.place_configure`

Callable	Lines	Responsibility	Representative direct calls
`SnifferApp._guard_mouse_wheel` _guard_mouse_wheel(self, event)	220-236	Allows normal wheel scrolling only on truly scrollable widgets and consumes wheel events elsewhere so a ttk Notebook cannot change tabs from mouse-wheel input.	`getattr, isinstance`
`SnifferApp._install_mouse_wheel_guard` _install_mouse_wheel_guard(self)	238-272	Prepend one application bindtag to every existing widget.	`stack.pop, self.bind_class, tuple, stack.extend, widget.bindtags`
`SnifferApp._apply_classic_widget_palette` _apply_classic_widget_palette(self, dark)	274-332	Apply palette to the small set of classic Tk widgets.	`theme_colors, hasattr, self.raw_text.configure, self.help_text.configure, getattr`
`SnifferApp.configure_inspector_result_styles` configure_inspector_result_styles(self)	334-380	Match the Inspector Result foreground to the Traffic result palette.	`theme_colors, self.ui_style.configure, hasattr, bool, tag_map.items`
`SnifferApp.tr` tr(self, text)	382-384	Translate one canonical UI string to the currently selected language.	`translate_text`
`SnifferApp.trf` trf(self, text, **values)	386-388	Translate a format template and substitute values afterwards.	`self.tr(text).format, self.tr`
`SnifferApp._capture_portuguese_help` _capture_portuguese_help(self)	390-408	Store the original PT-PT Help text and tag ranges exactly as built.	`hasattr, self.help_text.get, self.help_text.tag_names, list, self.help_text.tag_ranges`
`SnifferApp._render_help_language` _render_help_language(self)	410-438	Render Help in English or restore the original PT-PT text byte-for-byte.	`hasattr, self.help_text.configure, self.help_text.delete, self.help_text.see, f'{MAX_UI_FRAMES:_}'.replace`
`SnifferApp._translate_widget_tree` _translate_widget_tree(self)	440-472	Translate static widget captions without changing widget geometry.	`stack.pop, isinstance, stack.extend, widget.keys, widget.winfo_children`
`SnifferApp._translate_traffic_context_menu` _translate_traffic_context_menu(self)	474-492	Translate traffic context menu.	`getattr, menu.entryconfigure, self.tr`
`SnifferApp._apply_table_language` _apply_table_language(self)	494-546	Refresh headings and filter values that are not normal widget captions.	`hasattr, canonical_ui_text, self.advanced_filter_type_combo.configure, self.advanced_filter_type_var.set, self.time_order_var.set`

Callable	Lines	Responsibility	Representative direct calls
`SnifferApp.apply_language` apply_language(self, save=True)	548-612	Performs runtime translation of static widgets, tables, context menus, help content and live status strings, then re-applies saved/fixed geometry and optionally persists the selected language.	`getattr, self._translate_widget_tree, self._apply_table_language, self._translate_traffic_context_menu, self._render_help_language`
`SnifferApp.on_language_changed` on_language_changed(self, _event=None)	614-615	Event handler for language changed.	`self.apply_language`
`SnifferApp.toggle_dark_mode` toggle_dark_mode(self)	617-634	Applies the alternate palette using the already-created ttk geometry and updates custom Canvas widgets; the control structure itself is not rebuilt.	`bool, apply_theme, self._apply_classic_widget_palette, self.save_current_ui_settings, self.dark_mode_var.get`
`SnifferApp._restore_saved_main_tab` _restore_saved_main_tab(self)	636-644	Restore saved main tab.	`self.ui_settings.get, self.main_notebook.select, pages.get`
`SnifferApp._current_main_tab_name` _current_main_tab_name(self)	646-661	Current main tab name.	`self.main_notebook.select, str`
`SnifferApp._schedule_ui_settings_save` _schedule_ui_settings_save(self, *_args)	663-680	Schedules ui settings save.	`getattr, self.after, self.after_cancel`
`SnifferApp.save_current_ui_settings` save_current_ui_settings(self)	682-765	Collects validated user-facing preferences from both Sniffer and Finder and writes them to the shared settings file.	`getattr, save_ui_settings, self.after_cancel, int, bool`
`SnifferApp._install_ui_settings_persistence` _install_ui_settings_persistence(self)	767-811	Debounced persistence for all roaming-safe user preferences.	`self.main_notebook.bind, variable.trace_add, mapping.values, self._schedule_ui_settings_save`
`SnifferApp._schedule_bus_health_layout_sync` _schedule_bus_health_layout_sync(self, _event=None)	813-827	Debounce Bus Health reflow after notebook/window/Inspector resizing.	`getattr, self.after_idle, self.after_cancel`
`SnifferApp._initialize_bus_health_layout` _initialize_bus_health_layout(self)	829-851	Measure the natural full Bus Health height before adaptive reflow.	`getattr, self._sync_bus_health_layout, self.update_idletasks, card.winfo_reqheight, max`
`SnifferApp._apply_bus_health_layout` _apply_bus_health_layout(self, compact)	853-996	Re-grids Bus Health labels/values into equal-height rows.	`getattr, bool, range, label_widget.grid_forget, value_widget.grid_forget`

Callable	Lines	Responsibility	Representative direct calls
`SnifferApp._sync_bus_health_layout` _sync_bus_health_layout(self)	998-1051	Chooses full versus compact Bus Health arrangement from real available height with hysteresis, then reapplies wrapping/column behavior on width changes.	`getattr, health_frame.winfo_height, max, bool, health_frame.winfo_exists`
`SnifferApp._build_ui` _build_ui(self)	1053-2747	Constructs all top-level tabs and controls.	`self.ui_settings.get, self.columnconfigure, self.rowconfigure, ttk.Notebook, self.main_notebook.grid`

Detailed behavior of the key paths

`SnifferApp.__init__` lines 50-155

Creates the root window, loads persisted preferences before widget construction, initializes capture/discovery/log/history state, constructs the UI, freezes language geometry, applies language, installs event guards/persistence and schedules queue/status polling.

`_freeze_language_layout_geometry` lines 158-218

Measures the canonical Portuguese layout and freezes key container dimensions/minimums so changing to English changes text only, not panel/control geometry.

`_guard_mouse_wheel` lines 220-236

Allows normal wheel scrolling only on truly scrollable widgets and consumes wheel events elsewhere so a ttk Notebook cannot change tabs from mouse-wheel input.

`apply_language` lines 548-612

Performs runtime translation of static widgets, tables, context menus, help content and live status strings, then re-applies saved/fixed geometry and optionally persists the selected language.

`toggle_dark_mode` lines 617-634

Applies the alternate palette using the already-created ttk geometry and updates custom Canvas widgets; the control structure itself is not rebuilt.

`save_current_ui_settings` lines 682-765

Collects validated user-facing preferences from both Sniffer and Finder and writes them to the shared settings file. COM port names, window geometry and safety confirmation are intentionally excluded.

`_apply_bus_health_layout` lines 853-996

Re-grids Bus Health labels/values into equal-height rows. Full mode uses one metric pair per row; compact mode uses two pairs per row and controlled wrapping to keep content inside each rectangle.

`_sync_bus_health_layout` lines 998-1051

Chooses full versus compact Bus Health arrangement from real available height with hysteresis, then reapplies wrapping/column behavior on width changes.

`_build_ui` lines 1053-2747

Constructs all top-level tabs and controls. It is intentionally a large declarative layout method: global language/theme overlay, Sniffer configuration/actions/statistics, Traffic/Raw/Bus Health/Help notebook, filters/context menu, Frame Inspector and the Bus Slave Finder page.

> **Design note.** SnifferApp uses mixins to keep protocol capture, Finder, Master, Slave and view behavior in separate files while sharing one Tk root and common state; the Client and Server are tab objects owned by it.

test_regression.py

Automated regression suite. The tests protect RTU timing, parser/resynchronisation behavior, pairing/timeouts, broadcast and echo-write classification, split-frame reassembly, in-place timeout marking, Slave Finder attempts, idle-bus check and FC43/14 logic, plus reversible Portuguese/English translation. Fake serial classes keep the tests deterministic and hardware-independent.

Source lines	SHA-256
787	`b5a3f7e75c0b385f8cb874fdf3b16c8e867108c59406078d966f333db4dba70c`

Imports: `json, queue, random, tempfile, threading, time, unittest, collections.deque, pathlib.Path, mb_config, mb_slave_finder, mb_capture.CaptureMixin, mb_protocol.SessionMetrics, append_crc, burst_may_continue, crc16_modbus, crc_ok, modbus_rtu_interframe_gap_seconds, parse_burst, read_bits_details, split_capture_buffer, mb_slave_finder.SlaveFinderMixin, mb_view.ViewMixin, mb_diagnostics.inspector_values, mb_i18n.LANGUAGE_ENGLISH, LANGUAGE_PORTUGUESE, english_help_blocks, normalize_language, translate_details, translate_runtime_text, translate_text, mb_slave_finder.build_device_identification_probe, build_discovery_probe, device_identification_timeout_seconds, find_device_identification_response, find_discovery_response, format_device_identification, read_basic_device_identification`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`with_bad_crc` with_bad_crc(frame)	49-52	With bad crc.	`bytearray, bytes`
`device_id_response` device_id_response(slave=1, objects=None, *, more_follows=False, next_object_id=0, conformity=1)	55-83	Device id response.	`bytearray, objects.items, append_crc, bytes, payload.extend`
`class FakeSerial`	**86-116**	**Minimal non-blocking serial fake for active Device ID regression tests.**	**Class container**
`FakeSerial.__init__` __init__(self, response_by_object)	89-92	Init.	`dict, bytearray`
`FakeSerial.in_waiting` in_waiting(self)	95-96	In waiting.	`len`
`FakeSerial.reset_input_buffer` reset_input_buffer(self)	98-99	Reset input buffer.	`self.rx.clear`

Callable	Lines	Responsibility	Representative direct calls
`FakeSerial.reset_output_buffer` reset_output_buffer(self)	101-102	Reset output buffer.	—
`FakeSerial.write` write(self, data)	104-110	Write.	`bytes, self.writes.append, self.response_by_object.get, self.rx.extend, len`
`FakeSerial.read` read(self, size)	112-116	Read.	`min, bytes, int, len`
`class FakeSerialModule`	**119-124**	**FakeSerialModule.**	**Class container**
`class StaticVar`	**127-132**	**StaticVar.**	**Class container**
`StaticVar.__init__` __init__(self, value)	128-129	Init.	—
`StaticVar.get` get(self)	131-132	Get.	—
`class DummyCapture`	**135-144**	**DummyCapture.**	**Class container**
`DummyCapture.__init__` __init__(self, *, mode='Auto', baud='9600', data='8', parity='None', stop='1', manual='2.0')	138-144	Init.	`StaticVar`
`class TimingRegressionTests`	**147-179**	**TimingRegressionTests.**	**Class container**
`TimingRegressionTests.test_t35_uses_character_time_at_or_below_19200` test_t35_uses_character_time_at_or_below_19200(self)	148-158	Regression test: t35 uses character time at or below 19200.	`self.assertAlmostEqual, modbus_rtu_interframe_gap_seconds`
`TimingRegressionTests.test_t35_is_fixed_above_19200` test_t35_is_fixed_above_19200(self)	160-167	Regression test: t35 is fixed above 19200.	`self.subTest, self.assertAlmostEqual, modbus_rtu_interframe_gap_seconds`
`TimingRegressionTests.test_sniffer_auto_gap_uses_fixed_1750_us_above_19200` test_sniffer_auto_gap_uses_fixed_1750_us_above_19200(self)	169-171	Regression test: sniffer auto gap uses fixed 1750 us above 19200.	`DummyCapture, self.assertAlmostEqual, capture.calculated_gap_ms`
`TimingRegressionTests.test_sniffer_manual_gap_is_unchanged` test_sniffer_manual_gap_is_unchanged(self)	173-175	Regression test: sniffer manual gap is unchanged.	`DummyCapture, self.assertAlmostEqual, capture.calculated_gap_ms`
`TimingRegressionTests.test_device_id_timeout_covers_a_full_rtu_frame` test_device_id_timeout_covers_a_full_rtu_frame(self)	177-179	Regression test: device id timeout covers a full rtu frame.	`device_identification_timeout_seconds, self.assertGreaterEqual`
`class ParserRegressionTests`	**182-284**	**ParserRegressionTests.**	**Class container**

Callable	Lines	Responsibility	Representative direct calls
`ParserRegressionTests.test_crc_known_request` test_crc_known_request(self)	183-186	Regression test: crc known request.	`build_discovery_probe, self.assertTrue, self.assertEqual, crc_ok, bytes.fromhex`
`ParserRegressionTests.test_invalid_frame_before_valid_frame_stays_crc_error` test_invalid_frame_before_valid_frame_stays_crc_error(self)	188-193	Regression test: invalid frame before valid frame stays crc error.	`with_bad_crc, build_discovery_probe, split_capture_buffer, self.assertEqual`
`ParserRegressionTests.test_long_unsynchronised_noise_recovers_next_valid_frame` test_long_unsynchronised_noise_recovers_next_valid_frame(self)	195-201	Regression test: long unsynchronised noise recovers next valid frame.	`build_discovery_probe, split_capture_buffer, self.assertEqual, self.assertTrue, bytes`
`ParserRegressionTests.test_supported_function_code_shapes` test_supported_function_code_shapes(self)	203-235	Regression test: supported function code shapes.	`frames.items, append_crc, bytes.fromhex, self.subTest, split_capture_buffer`
`ParserRegressionTests.test_random_noise_never_crashes_and_preserves_all_bytes` test_random_noise_never_crashes_and_preserves_all_bytes(self)	237-244	Regression test: random noise never crashes and preserves all bytes.	`random.Random, range, rng.randint, bytes, split_capture_buffer`
`ParserRegressionTests.test_request_response_pairing_and_response_time` test_request_response_pairing_and_response_time(self)	246-269	Regression test: request response pairing and response time.	`SessionMetrics, metrics.process, self.assertEqual, self.assertAlmostEqual, parse_burst`
`ParserRegressionTests.test_timeout_regression` test_timeout_regression(self)	271-284	Regression test: timeout regression.	`SessionMetrics, metrics.process, self.assertEqual, metrics.pop_expired_requests, parse_burst`
`class SlaveFinderRegressionTests`	**287-369**	**SlaveFinderRegressionTests.**	**Class container**
`SlaveFinderRegressionTests.test_normal_discovery_response` test_normal_discovery_response(self)	288-292	Regression test: normal discovery response.	`append_crc, find_discovery_response, self.assertIsNotNone, self.assertEqual, bytes.fromhex`
`SlaveFinderRegressionTests.test_discovery_exception_counts_as_response` test_discovery_exception_counts_as_response(self)	294-299	Regression test: discovery exception counts as response.	`append_crc, find_discovery_response, self.assertIsNotNone, self.assertEqual, bytes.fromhex`
`SlaveFinderRegressionTests.test_device_identification_request` test_device_identification_request(self)	301-305	Regression test: device identification request.	`build_device_identification_probe, self.assertEqual, self.assertTrue, bytes.fromhex, len`

Callable	Lines	Responsibility	Representative direct calls
`SlaveFinderRegressionTests.test_device_identification_response_parser` test_device_identification_response_parser(self)	307-318	Regression test: device identification response parser.	`device_id_response, find_device_identification_response, self.assertIsNotNone, self.assertEqual, format_device_identification`
`SlaveFinderRegressionTests.test_device_identification_parser_ignores_request_echo` test_device_identification_parser_ignores_request_echo(self)	320-325	Regression test: device identification parser ignores request echo.	`build_device_identification_probe, device_id_response, find_device_identification_response, self.assertIsNotNone, self.assertEqual`
`SlaveFinderRegressionTests.test_device_identification_exception` test_device_identification_exception(self)	327-333	Regression test: device identification exception.	`append_crc, find_device_identification_response, self.assertIsNotNone, self.assertEqual, self.assertIn`
`SlaveFinderRegressionTests.test_device_identification_rejects_bad_crc` test_device_identification_rejects_bad_crc(self)	335-337	Regression test: device identification rejects bad crc.	`with_bad_crc, self.assertIsNone, device_id_response, find_device_identification_response`
`SlaveFinderRegressionTests.test_device_identification_follows_more_follows` test_device_identification_follows_more_follows(self)	339-369	Regression test: device identification follows more follows.	`device_id_response, FakeSerial, read_basic_device_identification, self.assertIsNotNone, self.assertEqual`
`class InternationalizationRegressionTests`	**372-422**	**InternationalizationRegressionTests.**	**Class container**
`InternationalizationRegressionTests.test_language_normalization_defaults_to_portuguese` test_language_normalization_defaults_to_portuguese(self)	373-375	Regression test: language normalization defaults to portuguese.	`self.assertEqual, normalize_language`
`InternationalizationRegressionTests.test_static_ui_translation_is_reversible` test_static_ui_translation_is_reversible(self)	377-383	Regression test: static ui translation is reversible.	`self.assertEqual, translate_text`
`InternationalizationRegressionTests.test_dynamic_status_translation_is_reversible` test_dynamic_status_translation_is_reversible(self)	385-389	Regression test: dynamic status translation is reversible.	`self.assertEqual, translate_runtime_text`

Callable	Lines	Responsibility	Representative direct calls
`InternationalizationRegressionTests.test_register_details_follow_language` test_register_details_follow_language(self)	391-399	Regression test: register details follow language.	`self.assertEqual, translate_details`
`InternationalizationRegressionTests.test_inspector_english_labels_and_crc` test_inspector_english_labels_and_crc(self)	401-411	Regression test: inspector english labels and crc.	`append_crc, row.update, inspector_values, self.assertTrue, self.assertIn`
`InternationalizationRegressionTests.test_english_help_contains_no_portuguese_diacritics` test_english_help_contains_no_portuguese_diacritics(self)	413-422	Regression test: english help contains no portuguese diacritics.	`''.join, self.assertIn, self.assertNotIn, text.casefold, english_help_blocks`
`bitwise_crc16` bitwise_crc16(data)	425-432	Reference CRC used up to v3.0, kept to verify the table-driven CRC.	`range`
`process_frames` process_frames(metrics, frames)	435-442	Run (bytes, channel, epoch) frames through SessionMetrics.	`enumerate, parse_burst, out.append, metrics.process`
`class ProtocolV31RegressionTests`	**445-538**	**ProtocolV31RegressionTests.**	**Class container**
`ProtocolV31RegressionTests.test_crc_table_matches_bitwise_reference` test_crc_table_matches_bitwise_reference(self)	446-450	Regression test: crc table matches bitwise reference.	`random.Random, bytes, self.assertEqual, crc16_modbus, bitwise_crc16`
`ProtocolV31RegressionTests.test_broadcast_request_never_times_out` test_broadcast_request_never_times_out(self)	452-461	Regression test: broadcast request never times out.	`append_crc, SessionMetrics, process_frames, self.assertEqual, metrics.expire_pending`
`ProtocolV31RegressionTests.test_fc06_echo_pair_is_request_then_response` test_fc06_echo_pair_is_request_then_response(self)	463-473	Regression test: fc06 echo pair is request then response.	`append_crc, SessionMetrics, process_frames, self.assertEqual, self.assertAlmostEqual`
`ProtocolV31RegressionTests.test_unanswered_fc05_write_times_out` test_unanswered_fc05_write_times_out(self)	475-480	Regression test: unanswered fc05 write times out.	`append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex`
`ProtocolV31RegressionTests.test_different_fc06_writes_are_both_requests` test_different_fc06_writes_are_both_requests(self)	482-491	Regression test: different fc06 writes are both requests.	`append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex`

Callable	Lines	Responsibility	Representative direct calls
`ProtocolV31RegressionTests.test_dual_rx_echo_must_come_from_opposite_channel` test_dual_rx_echo_must_come_from_opposite_channel(self)	493-504	Regression test: dual rx echo must come from opposite channel.	`append_crc, SessionMetrics, process_frames, self.assertEqual, bytes.fromhex`
`ProtocolV31RegressionTests.test_split_frame_is_held_until_complete` test_split_frame_is_held_until_complete(self)	506-511	Regression test: split frame is held until complete.	`append_crc, self.assertTrue, self.assertFalse, burst_may_continue, bytes.fromhex`
`ProtocolV31RegressionTests.test_merged_frames_keep_their_chunk_arrival_times` test_merged_frames_keep_their_chunk_arrival_times(self)	513-527	Regression test: merged frames keep their chunk arrival times.	`append_crc, parse_burst, self.assertEqual, self.assertAlmostEqual, SessionMetrics`
`ProtocolV31RegressionTests.test_read_bits_details_use_request_qty` test_read_bits_details_use_request_qty(self)	529-538	Regression test: read bits details use request qty.	`append_crc, self.assertEqual, bytes.fromhex, read_bits_details`
`class _ChunkedSerial`	**541-568**	**Delivers byte chunks at fixed delays, like a USB latency timer.**	**Class container**
`_ChunkedSerial.__init__` __init__(self, chunks, stop_event)	544-547	Init.	`time.perf_counter, list`
`_ChunkedSerial._due` _due(self)	549-555	Due.	`time.perf_counter, self.stop_event.set`
`_ChunkedSerial.in_waiting` in_waiting(self)	558-559	In waiting.	`len, self._due`
`_ChunkedSerial.read` read(self, size)	561-565	Read.	`self._due, self.chunks.pop`
`_ChunkedSerial.close` close(self)	567-568	Close.	—
`class _ReaderStub`	**571-574**	**_ReaderStub.**	**Class container**
`_ReaderStub.__init__` __init__(self)	572-574	Init.	`threading.Event, queue.Queue`
`class ReaderV31RegressionTests`	**577-591**	**ReaderV31RegressionTests.**	**Class container**
`ReaderV31RegressionTests.test_frame_split_by_usb_latency_is_reassembled` test_frame_split_by_usb_latency_is_reassembled(self)	578-591	Regression test: frame split by usb latency is reassembled.	`append_crc, _ReaderStub, _ChunkedSerial, reader.reader_loop, self.assertEqual`
`class _FakeVar`	**594-599**	**_FakeVar.**	**Class container**

Callable	Lines	Responsibility	Representative direct calls
`_FakeVar.__init__` __init__(self, value)	595-596	Init.	—
`_FakeVar.get` get(self)	598-599	Get.	—
`class _FakeTree`	**602-607**	**_FakeTree.**	**Class container**
`_FakeTree.__init__` __init__(self)	603-604	Init.	—
`_FakeTree.item` item(self, iid, tags=())	606-607	Item.	—
`class _TimeoutView`	**610-627**	**_TimeoutView.**	**Class container**
`_TimeoutView.__init__` __init__(self, type_filter)	611-621	Init.	`_FakeTree, _FakeVar, set, deque`
`_TimeoutView.render_filtered_view` render_filtered_view(self)	623-624	Render filtered view.	—
`_TimeoutView.selected_record` selected_record(self)	626-627	Selected record.	—
`class ViewV31RegressionTests`	**630-657**	**ViewV31RegressionTests.**	**Class container**
`ViewV31RegressionTests.test_timeout_retags_rows_without_full_rebuild` test_timeout_retags_rows_without_full_rebuild(self)	631-636	Regression test: timeout retags rows without full rebuild.	`_TimeoutView, self.assertEqual, view.mark_timed_out_records`
`ViewV31RegressionTests.test_timeouts_filter_still_rebuilds_view` test_timeouts_filter_still_rebuilds_view(self)	638-641	Regression test: timeouts filter still rebuilds view.	`_TimeoutView, view.mark_timed_out_records, self.assertEqual`
`ViewV31RegressionTests.test_inspector_shows_address_of_each_register` test_inspector_shows_address_of_each_register(self)	643-650	Regression test: inspector shows address of each register.	`append_crc, inspector_values, self.assertIn, bytes.fromhex, parse_burst`
`ViewV31RegressionTests.test_inspector_shows_address_of_each_bit` test_inspector_shows_address_of_each_bit(self)	652-657	Regression test: inspector shows address of each bit.	`append_crc, row.update, inspector_values, self.assertIn, bytes.fromhex`
`class _FakeFinderSerial`	**660-685**	**_FakeFinderSerial.**	**Class container**
`_FakeFinderSerial.__init__` __init__(self, incoming=b'')	661-663	Init.	`bytearray`

Callable	Lines	Responsibility	Representative direct calls
`_FakeFinderSerial.in_waiting` in_waiting(self)	666-667	In waiting.	`len`
`_FakeFinderSerial.read` read(self, size)	669-672	Read.	`bytes`
`_FakeFinderSerial.write` write(self, data)	674-676	Write.	`self.writes.append, len, bytes`
`_FakeFinderSerial.reset_input_buffer` reset_input_buffer(self)	678-679	Reset input buffer.	—
`_FakeFinderSerial.reset_output_buffer` reset_output_buffer(self)	681-682	Reset output buffer.	—
`_FakeFinderSerial.close` close(self)	684-685	Close.	—
`class _FinderStub`	**688-714**	**_FinderStub.**	**Class container**
`_FinderStub.__init__` __init__(self, fake_serial)	691-695	Init.	`queue.Queue, threading.Event`
`_FinderStub.tr` tr(self, text)	697-698	Tr.	`translate_text`
`_FinderStub.trf` trf(self, text, **values)	700-701	Trf.	`self.tr(text).format, self.tr`
`_FinderStub.open_discovery_serial_port` open_discovery_serial_port(self, serial, port, baud, parity, stopbits)	703-704	Open discovery serial port.	—
`_FinderStub.run_worker` run_worker(self, slaves, attempts)	706-714	Run worker.	`self.discovery_worker, self.event_queue.empty, events.append, self.event_queue.get`
`class FinderV31RegressionTests`	**717-763**	**FinderV31RegressionTests.**	**Class container**
`FinderV31RegressionTests.setUp` setUp(self)	718-720	SetUp.	—
`FinderV31RegressionTests.tearDown` tearDown(self)	722-723	TearDown.	—
`FinderV31RegressionTests.test_each_silent_slave_gets_all_attempts` test_each_silent_slave_gets_all_attempts(self)	725-730	Regression test: each silent slave gets all attempts.	`_FinderStub, stub.run_worker, self.assertIsNone, self.assertEqual, _FakeFinderSerial`

Callable	Lines	Responsibility	Representative direct calls
`FinderV31RegressionTests.test_traffic_before_scan_blocks_transmission` test_traffic_before_scan_blocks_transmission(self)	732-737	Regression test: traffic before scan blocks transmission.	`_FinderStub, stub.run_worker, self.assertIn, self.assertEqual, _FakeFinderSerial`
`FinderV31RegressionTests.test_legacy_default_timeout_migrates` test_legacy_default_timeout_migrates(self)	739-763	Regression test: legacy default timeout migrates.	`tempfile.TemporaryDirectory, Path, path.write_text, self.assertEqual, json.dumps`
`class SourceHygieneV31Tests`	**766-783**	**SourceHygieneV31Tests.**	**Class container**
`SourceHygieneV31Tests.test_no_inline_language_branches` test_no_inline_language_branches(self)	767-773	Regression test: no inline language branches.	`package.glob, source.read_text, self.assertNotIn, Path(__file__).resolve, Path`
`SourceHygieneV31Tests.test_message_templates_translate` test_message_templates_translate(self)	775-783	Regression test: message templates translate.	`self.assertEqual, translate_text('Encontrados: {count}', LANGUAGE_ENGLISH).format, translate_text`

test_master.py

Master regression tests. An in-memory Modbus slave behind a fake serial port answers reads and writes, so the tests cover data formats, frames from the Modbus specification, response matching, scheduling, writes, broadcast, timeouts, the idle-bus check and workspace validation without hardware.

Source lines
602

SHA-256
`a01432c6e8951bf1deff66832ffb92e4187fbaf2923d8a0d1e32442a787fda9d`

Imports: `json, os, queue, struct, tempfile, time, unittest, pathlib.Path, mb_config, mb_master_ui, mb_table_ui, mb_master.BYTE_ORDERS, BYTE_ORDERS_64, format_value, format_window_values, raw_text_for_scaled, register_groups, validate_scale, MasterEngine, PollDefinition, build_read_request, build_write_request, encode_coil_value, encode_register_value, find_master_response, format_register_values, mb_master_ui.MasterMixin, MasterWindow, apply_format_to_rows, mb_protocol.append_crc, crc_ok`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class FakeSlaveSerial`	**34-116**	**In-memory Modbus RTU slave behind a fake serial port.**	**Class container**
`FakeSlaveSerial.__init__` __init__(self, slave_id=1, *, echo=False, bus_traffic=b'', exception_for=None)	42-49	Init.	`bytearray, range`
`FakeSlaveSerial.in_waiting` in_waiting(self)	52-53	In waiting.	`len`
`FakeSlaveSerial.read` read(self, size)	55-58	Read.	`bytes`
`FakeSlaveSerial.reset_input_buffer` reset_input_buffer(self)	60-61	Reset input buffer.	`self.rx.clear`
`FakeSlaveSerial.reset_output_buffer` reset_output_buffer(self)	63-64	Reset output buffer.	—
`FakeSlaveSerial.cancel_read` cancel_read(self)	66-67	Cancel read.	—
`FakeSlaveSerial.cancel_write` cancel_write(self)	69-70	Cancel write.	—

Callable	Lines	Responsibility	Representative direct calls
`FakeSlaveSerial.close` close(self)	72-73	Close.	—
`FakeSlaveSerial.write` write(self, data)	75-84	Write.	`bytes, self.requests.append, len, self.rx.extend, crc_ok`
`FakeSlaveSerial._answer` _answer(self, req)	86-116	Answer.	`append_crc, b''.join, bytearray, range, bytes`
`drain` drain(event_queue)	119-123	Drain.	`event_queue.empty, events.append, event_queue.get`
`make_engine` make_engine(serial, **overrides)	126-130	Make engine.	`dict, options.update, MasterEngine, queue.Queue`
`class MasterFormatTests`	**133-169**	**MasterFormatTests.**	**Class container**
`MasterFormatTests.test_32_bit_formats_round_trip_in_every_byte_order` test_32_bit_formats_round_trip_in_every_byte_order(self)	134-141	Regression test: 32 bit formats round trip in every byte order.	`encode_register_value, self.assertEqual, len, format_register_values`
`MasterFormatTests.test_float_byte_orders_match_wire_layout` test_float_byte_orders_match_wire_layout(self)	143-149	Regression test: float byte orders match wire layout.	`struct.pack, self.assertEqual, encode_register_value, bytes.fromhex`
`MasterFormatTests.test_16_bit_formats` test_16_bit_formats(self)	151-158	Regression test: 16 bit formats.	`self.assertEqual, format_register_values, encode_register_value`
`MasterFormatTests.test_values_outside_the_format_are_rejected` test_values_outside_the_format_are_rejected(self)	160-166	Regression test: values outside the format are rejected.	`self.assertRaises, encode_coil_value, encode_register_value`
`MasterFormatTests.test_unpaired_register_in_32_bit_format` test_unpaired_register_in_32_bit_format(self)	168-169	Regression test: unpaired register in 32 bit format.	`self.assertEqual, format_register_values`
`class MasterFrameTests`	**172-217**	**MasterFrameTests.**	**Class container**
`MasterFrameTests.test_write_frames_match_modbus_specification_examples` test_write_frames_match_modbus_specification_examples(self)	173-183	Regression test: write frames match modbus specification examples.	`self.assertEqual, bytes.fromhex, build_write_request`
`MasterFrameTests.test_invalid_definitions_are_rejected` test_invalid_definitions_are_rejected(self)	185-191	Regression test: invalid definitions are rejected.	`self.assertRaises, build_write_request, build_read_request`

Callable	Lines	Responsibility	Representative direct calls
`MasterFrameTests.test_read_response_found_after_adapter_echo` test_read_response_found_after_adapter_echo(self)	193-198	Regression test: read response found after adapter echo.	`build_read_request, append_crc, find_master_response, self.assertEqual, bytes.fromhex`
`MasterFrameTests.test_coil_response_is_trimmed_to_quantity` test_coil_response_is_trimmed_to_quantity(self)	200-203	Regression test: coil response is trimmed to quantity.	`build_read_request, find_master_response, self.assertEqual, append_crc, bytes.fromhex`
`MasterFrameTests.test_exception_and_bad_crc` test_exception_and_bad_crc(self)	205-211	Regression test: exception and bad crc.	`build_read_request, append_crc, self.assertEqual, bytearray, self.assertIsNone`
`MasterFrameTests.test_write_response_must_match_request` test_write_response_must_match_request(self)	213-217	Regression test: write response must match request.	`build_write_request, self.assertIsNotNone, append_crc, self.assertIsNone, find_master_response`
`class MasterEngineTests`	**220-289**	**MasterEngineTests.**	**Class container**
`MasterEngineTests.run_engine` run_engine(self, engine, seconds)	221-225	Run engine.	`engine.start, time.sleep, engine.stop, drain`
`MasterEngineTests.test_scheduler_serves_writes_then_read_once_then_due_polls` test_scheduler_serves_writes_then_read_once_then_due_polls(self)	227-239	Regression test: scheduler serves writes then read once then due polls.	`make_engine, engine.set_poll, engine.read_once, engine.queue_write, self.assertEqual`
`MasterEngineTests.test_fast_and_slow_windows_share_one_bus` test_fast_and_slow_windows_share_one_bus(self)	241-255	Regression test: fast and slow windows share one bus.	`FakeSlaveSerial, make_engine, engine.set_poll, self.run_engine, self.assertGreater`
`MasterEngineTests.test_write_is_executed_and_reported` test_write_is_executed_and_reported(self)	257-263	Regression test: write is executed and reported.	`FakeSlaveSerial, make_engine, engine.queue_write, self.assertEqual, build_write_request`
`MasterEngineTests.test_broadcast_write_does_not_wait_for_a_response` test_broadcast_write_does_not_wait_for_a_response(self)	265-273	Regression test: broadcast write does not wait for a response.	`FakeSlaveSerial, make_engine, engine.queue_write, time.perf_counter, engine.transact`
`MasterEngineTests.test_timeout_exception_and_invalid_results` test_timeout_exception_and_invalid_results(self)	275-280	Regression test: timeout exception and invalid results.	`make_engine, self.assertEqual, engine.transact, FakeSlaveSerial, build_read_request`
`MasterEngineTests.test_traffic_on_the_bus_blocks_the_master` test_traffic_on_the_bus_blocks_the_master(self)	282-289	Regression test: traffic on the bus blocks the master.	`FakeSlaveSerial, make_engine, engine.set_poll, self.run_engine, self.assertEqual`

Callable	Lines	Responsibility	Representative direct calls
`class MasterWindowTests`	**292-309**	**MasterWindowTests.**	**Class container**
`MasterWindowTests.test_workspace_entry_round_trip_and_validation` test_workspace_entry_round_trip_and_validation(self)	293-301	Regression test: workspace entry round trip and validation.	`MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json, self.assertRaises`
`MasterWindowTests.test_write_dialog_builds_fc16_for_float_values` test_write_dialog_builds_fc16_for_float_values(self)	303-309	Regression test: write dialog builds fc16 for float values.	`MasterWindow, MasterMixin.master_build_write, self.assertEqual, bytes.fromhex, self.assertRaises`
`class MasterWorkspaceFolderTests`	**312-386**	**MasterWorkspaceFolderTests.**	**Class container**
`MasterWorkspaceFolderTests.setUp` setUp(self)	313-316	SetUp.	`os.environ.get, tempfile.TemporaryDirectory`
`MasterWorkspaceFolderTests.tearDown` tearDown(self)	318-323	TearDown.	`self._folder.cleanup, os.environ.pop`
`MasterWorkspaceFolderTests.test_workspaces_live_beside_the_settings` test_workspaces_live_beside_the_settings(self)	325-332	Regression test: workspaces live beside the settings.	`mb_config.get_workspaces_folder, self.assertEqual, self.assertTrue, mb_config.get_master_workspaces_folder, folder.is_dir`
`MasterWorkspaceFolderTests.test_loose_master_workspaces_move_into_the_master_folder` test_loose_master_workspaces_move_into_the_master_folder(self)	334-345	Regression test: loose master workspaces move into the master folder.	`mb_config.get_workspaces_folder, (root / 'Linha 1.mbsw').write_text, (root / 'notas.txt').write_text, self.assertEqual, mb_config.get_maste…`
`MasterWorkspaceFolderTests.test_saved_workspaces_are_listed_by_name` test_saved_workspaces_are_listed_by_name(self)	347-352	Regression test: saved workspaces are listed by name.	`(mb_config.get_master_workspaces_folder() / 'notas.txt').write_text, self.assertEqual, mb_master_ui.workspace_path(name).write_text, mb_mas…`
`MasterWorkspaceFolderTests.test_workspace_names_must_be_valid_file_names` test_workspace_names_must_be_valid_file_names(self)	354-358	Regression test: workspace names must be valid file names.	`self.assertEqual, mb_table_ui.validate_workspace_name, self.assertRaises`
`MasterWorkspaceFolderTests.test_mbsniffer_settings_and_workspaces_are_migrated_once` test_mbsniffer_settings_and_workspaces_are_migrated_once(self)	360-376	Regression test: mbsniffer settings and workspaces are migrated once.	`(legacy / 'Workspaces').mkdir, (legacy / 'settings.json').write_text, (legacy / 'Workspaces' / 'Linha 1.mbsw').write_text, mb_config.get_se…`

Callable	Lines	Responsibility	Representative direct calls
`MasterWorkspaceFolderTests.test_legacy_workspace_format_is_accepted` test_legacy_workspace_format_is_accepted(self)	378-380	Regression test: legacy workspace format is accepted.	`self.assertIn, self.assertEqual`
`MasterWorkspaceFolderTests.test_window_title_is_saved_in_the_workspace` test_window_title_is_saved_in_the_workspace(self)	382-386	Regression test: window title is saved in the workspace.	`MasterWindow, self.assertEqual, MasterWindow.validated_kwargs, window.to_json`
`class MasterPerAddressFormatTests`	**389-446**	**MasterPerAddressFormatTests.**	**Class container**
`MasterPerAddressFormatTests.test_64_bit_formats_round_trip_in_every_byte_order` test_64_bit_formats_round_trip_in_every_byte_order(self)	390-397	Regression test: 64 bit formats round trip in every byte order.	`encode_register_value, self.assertEqual, len, format_value`
`MasterPerAddressFormatTests.test_double_byte_orders_match_wire_layout` test_double_byte_orders_match_wire_layout(self)	399-404	Regression test: double byte orders match wire layout.	`self.assertEqual, encode_register_value`
`MasterPerAddressFormatTests.test_per_address_formats_override_the_window_format` test_per_address_formats_override_the_window_format(self)	406-411	Regression test: per address formats override the window format.	`format_window_values, self.assertEqual, register_groups`
`MasterPerAddressFormatTests.test_default_multi_register_values_cut_by_an_override_show_a_dash` test_default_multi_register_values_cut_by_an_override_show_a_dash(self)	413-415	Regression test: default multi register values cut by an override show a dash.	`format_window_values, self.assertEqual`
`MasterPerAddressFormatTests.test_selection_rules_for_multi_register_formats` test_selection_rules_for_multi_register_formats(self)	417-432	Regression test: selection rules for multi register formats.	`MasterWindow, apply_format_to_rows, self.assertEqual, self.assertNotIn, self.assertRaises`
`MasterPerAddressFormatTests.test_formats_are_saved_and_invalid_ones_dropped` test_formats_are_saved_and_invalid_ones_dropped(self)	434-440	Regression test: formats are saved and invalid ones dropped.	`MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json`
`MasterPerAddressFormatTests.test_write_uses_the_format_of_the_clicked_value` test_write_uses_the_format_of_the_clicked_value(self)	442-446	Regression test: write uses the format of the clicked value.	`MasterWindow, MasterMixin.master_build_write, self.assertEqual, bytes.fromhex`
`class MasterScaleTests`	**449-488**	**MasterScaleTests.**	**Class container**

Callable	Lines	Responsibility	Representative direct calls
`MasterScaleTests.test_scaled_values_are_shown_in_engineering_units` test_scaled_values_are_shown_in_engineering_units(self)	452-456	Regression test: scaled values are shown in engineering units.	`format_window_values, self.assertEqual`
`MasterScaleTests.test_scale_applies_to_multi_register_values` test_scale_applies_to_multi_register_values(self)	458-462	Regression test: scale applies to multi register values.	`encode_register_value, validate_scale, self.assertEqual, format_window_values`
`MasterScaleTests.test_written_engineering_values_are_converted_to_raw` test_written_engineering_values_are_converted_to_raw(self)	464-471	Regression test: written engineering values are converted to raw.	`self.assertEqual, MasterWindow, MasterMixin.master_build_write, raw_text_for_scaled, bytes.fromhex`
`MasterScaleTests.test_invalid_scales_are_rejected` test_invalid_scales_are_rejected(self)	473-479	Regression test: invalid scales are rejected.	`self.assertRaises, raw_text_for_scaled, validate_scale`
`MasterScaleTests.test_scales_are_saved_and_invalid_ones_dropped` test_scales_are_saved_and_invalid_ones_dropped(self)	481-488	Regression test: scales are saved and invalid ones dropped.	`MasterWindow, MasterWindow.validated_kwargs, self.assertEqual, window.to_json, list`
`class MasterOpenFromTrafficTests`	**491-513**	**MasterOpenFromTrafficTests.**	**Class container**
`MasterOpenFromTrafficTests.definition` definition(self, record)	492-493	Definition.	`MasterMixin.master_definition_from_record`
`MasterOpenFromTrafficTests.test_request_and_matched_response_give_the_read_definition` test_request_and_matched_response_give_the_read_definition(self)	495-504	Regression test: request and matched response give the read definition.	`self.assertEqual, self.definition`
`MasterOpenFromTrafficTests.test_frames_without_a_read_definition_are_refused` test_frames_without_a_read_definition_are_refused(self)	506-513	Regression test: frames without a read definition are refused.	`self.assertIsNone, self.definition`
`class FlakySlaveSerial`	**516-531**	**Fake slave whose port fails (like an unplugged USB adapter).**	**Class container**
`FlakySlaveSerial.__init__` __init__(self, failures, **kwargs)	519-522	Init.	`super().__init__, super`
`FlakySlaveSerial.write` write(self, data)	524-528	Write.	`super().write, OSError, super`

Callable	Lines	Responsibility	Representative direct calls
`FlakySlaveSerial.close` close(self)	530-531	Close.	—
`class MasterReconnectTests`	**534-581**	**MasterReconnectTests.**	**Class container**
`MasterReconnectTests.test_engine_reopens_the_port_after_a_failure` test_engine_reopens_the_port_after_a_failure(self)	535-553	Regression test: engine reopens the port after a failure.	`FlakySlaveSerial, FakeSlaveSerial, make_engine, engine.set_poll, engine.start`
`MasterReconnectTests.test_engine_keeps_retrying_until_stopped` test_engine_keeps_retrying_until_stopped(self)	555-571	Regression test: engine keeps retrying until stopped.	`make_engine, engine.set_poll, engine.start, time.sleep, engine.stop`
`MasterReconnectTests.test_without_reopen_a_failure_ends_the_session` test_without_reopen_a_failure_ends_the_session(self)	573-581	Regression test: without reopen a failure ends the session.	`make_engine, engine.set_poll, engine.start, time.sleep, engine.stop`
`class MasterSettingsTests`	**584-598**	**MasterSettingsTests.**	**Class container**
`MasterSettingsTests.test_last_workspace_is_validated` test_last_workspace_is_validated(self)	585-598	Regression test: last workspace is validated.	`tempfile.TemporaryDirectory, Path, path.write_text, self.assertEqual, json.dumps`

test_slave.py

Slave regression tests. Requests and replies are checked against the store, and the real Master and Slave engines talk over an in-memory serial link, with adapter echo, foreign traffic, split and garbled requests and every fault type.

Source lines	SHA-256
537	`4139e1e5cba1c5d97a51bb1bd8f4a8a026218392d387ac8de265283c7bbcbe05`

Imports: `os, queue, random, tempfile, threading, time, unittest, mb_master.MasterEngine, PollDefinition, build_read_request, build_write_request, find_master_response, mb_protocol.append_crc, crc_ok, mb_slave.SlaveEngine, SlaveStore, corrupt_crc, fault_active, handle_request, request_length, validate_fault, validate_generator, ValueGenerator, mb_config, mb_i18n.LANGUAGE_ENGLISH, translate_text, mb_slave_ui.SlaveWindow, fault_parts, slave_definition_from_record, validate_area, mb_table_ui.list_workspaces, workspace_path`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`class BusEnd`	**34-92**	**One end of an in-memory serial link: what one end writes, the other reads.**	**Class container**
`BusEnd.__init__` __init__(self, echo=False)	41-46	Init.	`bytearray, threading.Lock`
`BusEnd.pair` pair(echo_a=False, echo_b=False)	49-52	Pair.	`BusEnd`
`BusEnd.in_waiting` in_waiting(self)	55-57	In waiting.	`len`
`BusEnd.read` read(self, size)	59-63	Read.	`bytes`
`BusEnd.write` write(self, data)	65-72	Write.	`bytes, self.written.append, len, self.inject, self.peer.inject`
`BusEnd.inject` inject(self, data)	74-76	Inject.	`self.rx.extend`
`BusEnd.reset_input_buffer` reset_input_buffer(self)	78-80	Reset input buffer.	`self.rx.clear`
`BusEnd.reset_output_buffer` reset_output_buffer(self)	82-83	Reset output buffer.	—

Callable	Lines	Responsibility	Representative direct calls
`BusEnd.cancel_read` cancel_read(self)	85-86	Cancel read.	—
`BusEnd.cancel_write` cancel_write(self)	88-89	Cancel write.	—
`BusEnd.close` close(self)	91-92	Close.	—
`make_store` make_store()	95-103	Make store.	`SlaveStore, store.set_areas, store.write, range`
`drain` drain(event_queue)	106-110	Drain.	`event_queue.empty, events.append, event_queue.get`
`wait_for` wait_for(predicate, timeout=2.0)	113-119	Wait for.	`predicate, time.monotonic, time.sleep`
`class SlaveRequestTests`	**122-206**	**SlaveRequestTests.**	**Class container**
`SlaveRequestTests.test_reads_answer_from_the_tables` test_reads_answer_from_the_tables(self)	123-137	Regression test: reads answer from the tables.	`make_store, handle_request, self.assertEqual, find_master_response, build_read_request`
`SlaveRequestTests.test_writes_change_the_tables_and_answer_like_a_device` test_writes_change_the_tables_and_answer_like_a_device(self)	139-155	Regression test: writes change the tables and answer like a device.	`make_store, build_write_request, handle_request, self.assertEqual, store.read`
`SlaveRequestTests.test_exceptions_like_a_real_device` test_exceptions_like_a_real_device(self)	157-172	Regression test: exceptions like a real device.	`make_store, handle_request, self.assertEqual, build_read_request, build_write_request`
`SlaveRequestTests.test_other_slave_ids_are_ignored` test_other_slave_ids_are_ignored(self)	174-177	Regression test: other slave ids are ignored.	`handle_request, self.assertEqual, self.assertIsNone, make_store, build_read_request`
`SlaveRequestTests.test_broadcast_writes_every_covering_slave_without_answering` test_broadcast_writes_every_covering_slave_without_answering(self)	179-191	Regression test: broadcast writes every covering slave without answering.	`make_store, handle_request, self.assertEqual, self.assertIsNone, build_write_request`
`SlaveRequestTests.test_request_length` test_request_length(self)	193-198	Regression test: request length.	`self.assertIsNone, self.assertEqual, request_length, build_read_request, build_write_request`
`SlaveRequestTests.test_window_areas_define_the_slave` test_window_areas_define_the_slave(self)	200-206	Regression test: window areas define the slave.	`SlaveWindow, SlaveStore, store.set_areas, self.assertEqual, self.assertTrue`

Callable	Lines	Responsibility	Representative direct calls
`class SlaveFaultTests`	**209-232**	**SlaveFaultTests.**	**Class container**
`SlaveFaultTests.test_fault_validation` test_fault_validation(self)	210-219	Regression test: fault validation.	`self.assertFalse, validate_fault, self.assertTrue, self.assertEqual, fault_active`
`SlaveFaultTests.test_inactive_faults_are_not_stored` test_inactive_faults_are_not_stored(self)	221-228	Regression test: inactive faults are not stored.	`make_store, store.set_fault, self.assertIsNone, self.assertTrue, self.assertEqual`
`SlaveFaultTests.test_corrupt_crc` test_corrupt_crc(self)	230-232	Regression test: corrupt crc.	`build_read_request, self.assertFalse, crc_ok, corrupt_crc`
`class SlaveEngineTests`	**235-383**	**The Master engine talking to the Slave engine over an in-memory bus.**	**Class container**
`SlaveEngineTests.setUp` setUp(self)	238-244	SetUp.	`make_store, BusEnd.pair, queue.Queue`
`SlaveEngineTests.tearDown` tearDown(self)	246-249	TearDown.	`engine.stop`
`SlaveEngineTests.start_slave` start_slave(self, **options)	251-254	Start slave.	`SlaveEngine, self.slave.start`
`SlaveEngineTests.start_master` start_master(self, timeout_ms=200)	256-260	Start master.	`MasterEngine, self.master.start`
`SlaveEngineTests.master_results` master_results(self, count, timeout=3.0)	262-272	Master results.	`time.monotonic, len, self.master_events.get, results.append`
`SlaveEngineTests.test_master_reads_and_writes_every_function` test_master_reads_and_writes_every_function(self)	274-294	Regression test: master reads and writes every function.	`self.start_slave, self.start_master, enumerate, self.assertEqual, self.assertIn`
`SlaveEngineTests.test_exception_reaches_the_master` test_exception_reaches_the_master(self)	296-301	Regression test: exception reaches the master.	`self.start_slave, self.start_master, self.master.set_poll, self.assertEqual, PollDefinition`
`SlaveEngineTests.test_adapter_echo_and_foreign_traffic_are_ignored` test_adapter_echo_and_foreign_traffic_are_ignored(self)	303-320	Regression test: adapter echo and foreign traffic are ignored.	`self.start_slave, self.slave_end.inject, self.start_master, self.master.set_poll, self.master_results`
`SlaveEngineTests.test_split_request_is_assembled` test_split_request_is_assembled(self)	322-330	Regression test: split request is assembled.	`self.start_slave, build_read_request, self.slave_end.inject, time.sleep, self.assertTrue`

Callable	Lines	Responsibility	Representative direct calls
`SlaveEngineTests.test_garbage_before_a_request_is_dropped` test_garbage_before_a_request_is_dropped(self)	332-337	Regression test: garbage before a request is dropped.	`self.start_slave, build_read_request, self.slave_end.inject, self.assertTrue, self.assertEqual`
`SlaveEngineTests.test_unknown_function_gets_exception_01_after_silence` test_unknown_function_gets_exception_01_after_silence(self)	339-343	Regression test: unknown function gets exception 01 after silence.	`self.start_slave, self.slave_end.inject, self.assertTrue, self.assertEqual, append_crc`
`SlaveEngineTests.test_faults` test_faults(self)	345-370	Regression test: faults.	`self.start_slave, self.start_master, self.master.set_poll, self.store.set_fault, self.master.read_once`
`SlaveEngineTests.test_fault_probability` test_fault_probability(self)	372-383	Regression test: fault probability.	`make_store, store.set_fault, SlaveEngine, range, statuses.count`
`class SlaveWindowTests`	**386-420**	**SlaveWindowTests.**	**Class container**
`SlaveWindowTests.test_workspace_entry_round_trip_and_validation` test_workspace_entry_round_trip_and_validation(self)	387-398	Regression test: workspace entry round trip and validation.	`SlaveWindow, SlaveWindow.validated_kwargs, self.assertEqual, validate_area, window.to_json`
`SlaveWindowTests.test_table_field_shows_the_read_function_code` test_table_field_shows_the_read_function_code(self)	400-406	Regression test: table field shows the read function code.	`self.assertEqual, table_choice_label`
`SlaveWindowTests.test_window_overlap` test_window_overlap(self)	408-413	Regression test: window overlap.	`SlaveWindow, self.assertEqual, self.assertIsNone, window.overlap`
`SlaveWindowTests.test_fault_description_translates` test_fault_description_translates(self)	415-420	Regression test: fault description translates.	`validate_fault, self.assertEqual, translate_text(template, LANGUAGE_ENGLISH).format, translate_text, fault_parts`
`class SlaveGeneratorTests`	**423-474**	**SlaveGeneratorTests.**	**Class container**
`SlaveGeneratorTests.test_validation` test_validation(self)	424-439	Regression test: validation.	`validate_generator, self.assertEqual, self.assertRaises`
`SlaveGeneratorTests.test_values_over_time` test_values_over_time(self)	441-462	Regression test: values over time.	`ValueGenerator, self.assertEqual, self.assertAlmostEqual, noise.value, self.assertTrue`

Callable	Lines	Responsibility	Representative direct calls
`SlaveGeneratorTests.test_generators_are_saved_and_follow_the_window` test_generators_are_saved_and_follow_the_window(self)	464-474	Regression test: generators are saved and follow the window.	`SlaveWindow, SlaveWindow.validated_kwargs, self.assertEqual, window.to_json, window.drop_invalid_generators`
`class SlaveOpenFromTrafficTests`	**477-496**	**SlaveOpenFromTrafficTests.**	**Class container**
`SlaveOpenFromTrafficTests.test_reads_and_writes_give_the_slave_area` test_reads_and_writes_give_the_slave_area(self)	478-489	Regression test: reads and writes give the slave area.	`self.assertEqual, slave_definition_from_record`
`SlaveOpenFromTrafficTests.test_frames_without_an_area_are_refused` test_frames_without_an_area_are_refused(self)	491-496	Regression test: frames without an area are refused.	`self.assertIsNone, slave_definition_from_record`
`class SlaveSettingsTests`	**499-533**	**SlaveSettingsTests.**	**Class container**
`SlaveSettingsTests.setUp` setUp(self)	500-503	SetUp.	`os.environ.get, tempfile.TemporaryDirectory`
`SlaveSettingsTests.tearDown` tearDown(self)	505-510	TearDown.	`self._folder.cleanup, os.environ.pop`
`SlaveSettingsTests.test_slave_workspaces_have_their_own_folder` test_slave_workspaces_have_their_own_folder(self)	512-518	Regression test: slave workspaces have their own folder.	`mb_config.get_slave_workspaces_folder, self.assertEqual, workspace_path('Banco', folder).write_text, mb_config.get_workspaces_folder, list_…`
`SlaveSettingsTests.test_slave_settings_are_validated` test_slave_settings_are_validated(self)	520-533	Regression test: slave settings are validated.	`mb_config.save_ui_settings, mb_config.load_ui_settings, self.assertEqual`

test_tcp.py

Modbus TCP regression tests. MBAP framing, then the real Client and Server engines over 127.0.0.1: reads, writes, exceptions, faults, several clients, unreachable devices, a port in use, Client windows and settings.

Source lines	SHA-256
270	`08101c93368627aa7ca8da028ff16852f4a4498ec371352c4cd48b3d2fd8c50b`

Imports: `os, queue, random, socket, tempfile, time, unittest, mb_config, mb_master.build_write_request, mb_protocol.append_crc, crc_ok, mb_slave.SlaveStore, validate_fault, mb_tcp.ClientEngine, ClientPollDefinition, ServerEngine, adu_length, adu_to_rtu, build_adu, parse_adu, rtu_to_adu, validate_host, validate_tcp_port, mb_tcp_ui.ClientTab, ClientWindow, ServerTab`

Complete callable reference

The table below includes every top-level function, class and class method in this file. Line numbers refer to the exact v3.5 source snapshot documented here.

Callable	Lines	Responsibility	Representative direct calls
`make_store` make_store()	29-35	Make store.	`SlaveStore, store.set_areas, store.write, range`
`wait_event` wait_event(event_queue, name, timeout=3.0)	38-47	Wait event.	`AssertionError, time.monotonic, event_queue.get`
`class MbapFramingTests`	**50-76**	**MbapFramingTests.**	**Class container**
`MbapFramingTests.test_adu_round_trip` test_adu_round_trip(self)	51-59	Regression test: adu round trip.	`build_write_request, rtu_to_adu, self.assertEqual, parse_adu, self.assertTrue`
`MbapFramingTests.test_incomplete_and_impossible_headers` test_incomplete_and_impossible_headers(self)	61-66	Regression test: incomplete and impossible headers.	`self.assertIsNone, adu_length, self.assertRaises, bytes.fromhex`
`MbapFramingTests.test_host_and_port_validation` test_host_and_port_validation(self)	68-76	Regression test: host and port validation.	`self.assertEqual, validate_host, validate_tcp_port, self.assertRaises`
`class ServerClientTests`	**79-208**	**The Client engine talking to the Server engine over 127.0.0.1.**	**Class container**
`ServerClientTests.setUp` setUp(self)	82-90	SetUp.	`make_store, queue.Queue, ServerEngine, self.server.start, wait_event`

Callable	Lines	Responsibility	Representative direct calls
`ServerClientTests.tearDown` tearDown(self)	92-95	TearDown.	`self.server.stop, self.client.stop`
`ServerClientTests.start_client` start_client(self, timeout_ms=300)	97-100	Start client.	`ClientEngine, self.client.start`
`ServerClientTests.poll` poll(self, window_id, unit, fc, address, qty, port=None)	102-104	Poll.	`self.client.set_poll, ClientPollDefinition`
`ServerClientTests.results` results(self, count, timeout=3.0)	106-116	Results.	`time.monotonic, len, self.client_events.get, found.append`
`ServerClientTests.test_reads_writes_and_exceptions` test_reads_writes_and_exceptions(self)	118-136	Regression test: reads writes and exceptions.	`self.start_client, self.poll, self.assertEqual, self.client.queue_write, build_write_request`
`ServerClientTests.test_server_events` test_server_events(self)	138-146	Regression test: server events.	`self.start_client, self.poll, self.results, wait_event, self.assertTrue`
`ServerClientTests.test_faults` test_faults(self)	148-167	Regression test: faults.	`self.start_client, self.poll, self.results, self.store.set_fault, self.client.read_once`
`ServerClientTests.test_several_clients_at_once` test_several_clients_at_once(self)	169-181	Regression test: several clients at once.	`socket.create_connection, enumerate, range, sock.sendall, sock.recv`
`ServerClientTests.test_unreachable_device_and_reconnection` test_unreachable_device_and_reconnection(self)	183-196	Regression test: unreachable device and reconnection.	`self.start_client, socket.socket, free.bind, free.close, self.poll`
`ServerClientTests.test_port_in_use_is_reported` test_port_in_use_is_reported(self)	198-203	Regression test: port in use is reported.	`queue.Queue, ServerEngine, second.start, wait_event, self.assertIn`
`ServerClientTests.test_unit_ids_above_247` test_unit_ids_above_247(self)	205-208	Regression test: unit ids above 247.	`ClientPollDefinition, self.assertEqual, self.assertTrue, crc_ok, definition.request`
`class ClientWindowTests`	**211-237**	**ClientWindowTests.**	**Class container**
`ClientWindowTests.test_workspace_entry_round_trip_and_validation` test_workspace_entry_round_trip_and_validation(self)	212-223	Regression test: workspace entry round trip and validation.	`ClientWindow, ClientWindow.validated_kwargs, self.assertEqual, ClientWindow(2, 2, **data).definition, window.to_json`

Callable	Lines	Responsibility	Representative direct calls
`ClientWindowTests.test_write_uses_the_unit_id` test_write_uses_the_unit_id(self)	225-231	Regression test: write uses the unit id.	`ClientWindow, ClientTab, tab.master_build_write, self.assertEqual, object`
`ClientWindowTests.test_tabs_keep_their_own_settings_and_folders` test_tabs_keep_their_own_settings_and_folders(self)	233-237	Regression test: tabs keep their own settings and folders.	`self.assertEqual, ClientTab._master_ws_folder, ServerTab._slave_ws_folder`
`class TcpSettingsTests`	**240-266**	**TcpSettingsTests.**	**Class container**
`TcpSettingsTests.setUp` setUp(self)	241-244	SetUp.	`os.environ.get, tempfile.TemporaryDirectory`
`TcpSettingsTests.tearDown` tearDown(self)	246-251	TearDown.	`self._folder.cleanup, os.environ.pop`
`TcpSettingsTests.test_client_and_server_settings_are_validated` test_client_and_server_settings_are_validated(self)	253-266	Regression test: client and server settings are validated.	`mb_config.save_ui_settings, mb_config.load_ui_settings, self.assertEqual`

14. Regression-test suite

The documented package was executed with `python -m unittest discover -s tests -p "test_*.py" -v`. All 132 tests passed.

> Verification result: **132 tests run - 132 passed - 0 failures/errors**.

TimingRegressionTests

Protects RTU timing and automatic/manual gap calculations, including the fixed 1.750 ms t3.5 rule above 19200 bit/s and FC43 response timing.

Test	Lines	What it protects
`test_t35_uses_character_time_at_or_below_19200`	148-158	Regression test: t35 uses character time at or below 19200.
`test_t35_is_fixed_above_19200`	160-167	Regression test: t35 is fixed above 19200.
`test_sniffer_auto_gap_uses_fixed_1750_us_above_19200`	169-171	Regression test: sniffer auto gap uses fixed 1750 us above 19200.
`test_sniffer_manual_gap_is_unchanged`	173-175	Regression test: sniffer manual gap is unchanged.
`test_device_id_timeout_covers_a_full_rtu_frame`	177-179	Regression test: device id timeout covers a full rtu frame.

ParserRegressionTests

Protects CRC, resynchronisation, supported frame shapes, random-noise robustness, request/response pairing and timeout behavior.

Test	Lines	What it protects
`test_crc_known_request`	183-186	Regression test: crc known request.
`test_invalid_frame_before_valid_frame_stays_crc_error`	188-193	Regression test: invalid frame before valid frame stays crc error.
`test_long_unsynchronised_noise_recovers_next_valid_frame`	195-201	Regression test: long unsynchronised noise recovers next valid frame.
`test_supported_function_code_shapes`	203-235	Regression test: supported function code shapes.

Test	Lines	What it protects
`test_random_noise_never_crashes_and_preserves_all_bytes`	237-244	Regression test: random noise never crashes and preserves all bytes.
`test_request_response_pairing_and_response_time`	246-269	Regression test: request response pairing and response time.
`test_timeout_regression`	271-284	Regression test: timeout regression.

SlaveFinderRegressionTests

Protects FC03/FC04 discovery and FC43/14 Device Identification parsing, CRC validation, Exceptions, echo rejection and More Follows paging.

Test	Lines	What it protects
`test_normal_discovery_response`	288-292	Regression test: normal discovery response.
`test_discovery_exception_counts_as_response`	294-299	Regression test: discovery exception counts as response.
`test_device_identification_request`	301-305	Regression test: device identification request.
`test_device_identification_response_parser`	307-318	Regression test: device identification response parser.
`test_device_identification_parser_ignores_request_echo`	320-325	Regression test: device identification parser ignores request echo.
`test_device_identification_exception`	327-333	Regression test: device identification exception.
`test_device_identification_rejects_bad_crc`	335-337	Regression test: device identification rejects bad crc.
`test_device_identification_follows_more_follows`	339-369	Regression test: device identification follows more follows.

InternationalizationRegressionTests

Protects language normalization, reversible static/runtime translation, translated register details, Frame Inspector English text and English Help content.

Test	Lines	What it protects
`test_language_normalization_defaults_to_portuguese`	373-375	Regression test: language normalization defaults to portuguese.

Test	Lines	What it protects
`test_static_ui_translation_is_reversible`	377-383	Regression test: static ui translation is reversible.
`test_dynamic_status_translation_is_reversible`	385-389	Regression test: dynamic status translation is reversible.
`test_register_details_follow_language`	391-399	Regression test: register details follow language.
`test_inspector_english_labels_and_crc`	401-411	Regression test: inspector english labels and crc.
`test_english_help_contains_no_portuguese_diacritics`	413-422	Regression test: english help contains no portuguese diacritics.

ProtocolV31RegressionTests

Protects the v3.1 protocol fixes: table CRC against the bit-wise reference, broadcast accounting, FC05/06 echo pairing, dual-RX direction, split-burst detection, per-read timestamps and FC01/02 bit formatting.

Test	Lines	What it protects
`test_crc_table_matches_bitwise_reference`	446-450	Regression test: crc table matches bitwise reference.
`test_broadcast_request_never_times_out`	452-461	Regression test: broadcast request never times out.
`test_fc06_echo_pair_is_request_then_response`	463-473	Regression test: fc06 echo pair is request then response.
`test_unanswered_fc05_write_times_out`	475-480	Regression test: unanswered fc05 write times out.
`test_different_fc06_writes_are_both_requests`	482-491	Regression test: different fc06 writes are both requests.
`test_dual_rx_echo_must_come_from_opposite_channel`	493-504	Regression test: dual rx echo must come from opposite channel.
`test_split_frame_is_held_until_complete`	506-511	Regression test: split frame is held until complete.
`test_merged_frames_keep_their_chunk_arrival_times`	513-527	Regression test: merged frames keep their chunk arrival times.
`test_read_bits_details_use_request_qty`	529-538	Regression test: read bits details use request qty.

ReaderV31RegressionTests

Feeds a frame split by a simulated 16 ms USB latency into the real reader_loop() and requires one CRC-valid RESPONSE.

Test	Lines	What it protects
`test_frame_split_by_usb_latency_is_reassembled`	578-591	Regression test: frame split by usb latency is reassembled.

ViewV31RegressionTests

Protects in-place timeout retagging, the Timeouts-filter rebuild and the address-mapped register/bit data in the Frame Inspector.

Test	Lines	What it protects
`test_timeout_retags_rows_without_full_rebuild`	631-636	Regression test: timeout retags rows without full rebuild.
`test_timeouts_filter_still_rebuilds_view`	638-641	Regression test: timeouts filter still rebuilds view.
`test_inspector_shows_address_of_each_register`	643-650	Regression test: inspector shows address of each register.
`test_inspector_shows_address_of_each_bit`	652-657	Regression test: inspector shows address of each bit.

FinderV31RegressionTests

Runs the real discovery worker against fake serial ports to check attempts, the idle-bus abort and the 40 to 100 ms settings migration.

Test	Lines	What it protects
`test_each_silent_slave_gets_all_attempts`	725-730	Regression test: each silent slave gets all attempts.
`test_traffic_before_scan_blocks_transmission`	732-737	Regression test: traffic before scan blocks transmission.
`test_legacy_default_timeout_migrates`	739-763	Regression test: legacy default timeout migrates.

SourceHygieneV31Tests

Rejects inline language branches in the package and checks that message templates translate.

Test	Lines	What it protects
`test_no_inline_language_branches`	767-773	Regression test: no inline language branches.
`test_message_templates_translate`	775-783	Regression test: message templates translate.

MasterFormatTests

Protects the 16/32-bit data formats, the four byte orders and the rejection of values that do not fit.

Test	Lines	What it protects
`test_32_bit_formats_round_trip_in_every_byte_order`	134-141	Regression test: 32 bit formats round trip in every byte order.
`test_float_byte_orders_match_wire_layout`	143-149	Regression test: float byte orders match wire layout.
`test_16_bit_formats`	151-158	Regression test: 16 bit formats.
`test_values_outside_the_format_are_rejected`	160-166	Regression test: values outside the format are rejected.
`test_unpaired_register_in_32_bit_format`	168-169	Regression test: unpaired register in 32 bit format.

MasterFrameTests

Checks write frames against the Modbus specification examples, definition limits, adapter echoes, exceptions, bad CRC and write response matching.

Test	Lines	What it protects
`test_write_frames_match_modbus_specification_examples`	173-183	Regression test: write frames match modbus specification examples.
`test_invalid_definitions_are_rejected`	185-191	Regression test: invalid definitions are rejected.
`test_read_response_found_after_adapter_echo`	193-198	Regression test: read response found after adapter echo.
`test_coil_response_is_trimmed_to_quantity`	200-203	Regression test: coil response is trimmed to quantity.
`test_exception_and_bad_crc`	205-211	Regression test: exception and bad crc.
`test_write_response_must_match_request`	213-217	Regression test: write response must match request.

MasterEngineTests

Runs MasterEngine against an in-memory slave: scheduling order, fast and slow windows on one bus, writes, broadcast, timeouts, exceptions and the idle-bus abort.

Test	Lines	What it protects
`test_scheduler_serves_writes_then_read_once_then_due_polls`	227-239	Regression test: scheduler serves writes then read once then due polls.

Test	Lines	What it protects
`test_fast_and_slow_windows_share_one_bus`	241-255	Regression test: fast and slow windows share one bus.
`test_write_is_executed_and_reported`	257-263	Regression test: write is executed and reported.
`test_broadcast_write_does_not_wait_for_a_response`	265-273	Regression test: broadcast write does not wait for a response.
`test_timeout_exception_and_invalid_results`	275-280	Regression test: timeout exception and invalid results.
`test_traffic_on_the_bus_blocks_the_master`	282-289	Regression test: traffic on the bus blocks the master.

MasterWindowTests

Protects workspace entry validation and the FC16 float encoding of the write dialog.

Test	Lines	What it protects
`test_workspace_entry_round_trip_and_validation`	293-301	Regression test: workspace entry round trip and validation.
`test_write_dialog_builds_fc16_for_float_values`	303-309	Regression test: write dialog builds fc16 for float values.

MasterWorkspaceFolderTests

Checks that workspaces live beside settings.json, are listed alphabetically and use valid Windows file names, and that window names are saved.

Test	Lines	What it protects
`test_workspaces_live_beside_the_settings`	325-332	Regression test: workspaces live beside the settings.
`test_loose_master_workspaces_move_into_the_master_folder`	334-345	Regression test: loose master workspaces move into the master folder.
`test_saved_workspaces_are_listed_by_name`	347-352	Regression test: saved workspaces are listed by name.
`test_workspace_names_must_be_valid_file_names`	354-358	Regression test: workspace names must be valid file names.
`test_mbsniffer_settings_and_workspaces_are_migrated_once`	360-376	Regression test: mbsniffer settings and workspaces are migrated once.
`test_legacy_workspace_format_is_accepted`	378-380	Regression test: legacy workspace format is accepted.

Test	Lines	What it protects
`test_window_title_is_saved_in_the_workspace`	382-386	Regression test: window title is saved in the workspace.

MasterPerAddressFormatTests

Protects the 64-bit formats and byte orders, per-address formats, value grouping, the selection rules and the write encoding of the clicked value.

Test	Lines	What it protects
`test_64_bit_formats_round_trip_in_every_byte_order`	390-397	Regression test: 64 bit formats round trip in every byte order.
`test_double_byte_orders_match_wire_layout`	399-404	Regression test: double byte orders match wire layout.
`test_per_address_formats_override_the_window_format`	406-411	Regression test: per address formats override the window format.
`test_default_multi_register_values_cut_by_an_override_show_a_dash`	413-415	Regression test: default multi register values cut by an override show a dash.
`test_selection_rules_for_multi_register_formats`	417-432	Regression test: selection rules for multi register formats.
`test_formats_are_saved_and_invalid_ones_dropped`	434-440	Regression test: formats are saved and invalid ones dropped.
`test_write_uses_the_format_of_the_clicked_value`	442-446	Regression test: write uses the format of the clicked value.

MasterScaleTests

Protects scaled display, multi-register scaling, the conversion of written engineering values and scale validation.

Test	Lines	What it protects
`test_scaled_values_are_shown_in_engineering_units`	452-456	Regression test: scaled values are shown in engineering units.
`test_scale_applies_to_multi_register_values`	458-462	Regression test: scale applies to multi register values.
`test_written_engineering_values_are_converted_to_raw`	464-471	Regression test: written engineering values are converted to raw.
`test_invalid_scales_are_rejected`	473-479	Regression test: invalid scales are rejected.

Test	Lines	What it protects
`test_scales_are_saved_and_invalid_ones_dropped`	481-488	Regression test: scales are saved and invalid ones dropped.

MasterOpenFromTrafficTests

Checks which Traffic records can open a Master window and with which definition.

Test	Lines	What it protects
`test_request_and_matched_response_give_the_read_definition`	495-504	Regression test: request and matched response give the read definition.
`test_frames_without_a_read_definition_are_refused`	506-513	Regression test: frames without a read definition are refused.

MasterReconnectTests

Simulates an unplugged adapter: the engine reopens the port and resumes, keeps retrying until stopped, or ends without reopen.

Test	Lines	What it protects
`test_engine_reopens_the_port_after_a_failure`	535-553	Regression test: engine reopens the port after a failure.
`test_engine_keeps_retrying_until_stopped`	555-571	Regression test: engine keeps retrying until stopped.
`test_without_reopen_a_failure_ends_the_session`	573-581	Regression test: without reopen a failure ends the session.

MasterSettingsTests

Validates the last workspace name stored in settings.json.

Test	Lines	What it protects
`test_last_workspace_is_validated`	585-598	Regression test: last workspace is validated.

SlaveRequestTests

Checks reads and writes of every table, Exceptions 01/02/03, ignored IDs, broadcast writes and request lengths.

Test	Lines	What it protects
`test_reads_answer_from_the_tables`	123-137	Regression test: reads answer from the tables.

Test	Lines	What it protects
`test_writes_change_the_tables_and_answer_like_a_device`	139-155	Regression test: writes change the tables and answer like a device.
`test_exceptions_like_a_real_device`	157-172	Regression test: exceptions like a real device.
`test_other_slave_ids_are_ignored`	174-177	Regression test: other slave ids are ignored.
`test_broadcast_writes_every_covering_slave_without_answering`	179-191	Regression test: broadcast writes every covering slave without answering.
`test_request_length`	193-198	Regression test: request length.
`test_window_areas_define_the_slave`	200-206	Regression test: window areas define the slave.

SlaveFaultTests

Validates fault settings, keeps only active faults and corrupts the CRC.

Test	Lines	What it protects
`test_fault_validation`	210-219	Regression test: fault validation.
`test_inactive_faults_are_not_stored`	221-228	Regression test: inactive faults are not stored.
`test_corrupt_crc`	230-232	Regression test: corrupt crc.

SlaveEngineTests

Runs the Master engine against the Slave engine over an in-memory link: every function, exceptions, adapter echo, foreign traffic, split and garbled requests, unknown functions and each fault, including the probability.

Test	Lines	What it protects
`test_master_reads_and_writes_every_function`	274-294	Regression test: master reads and writes every function.
`test_exception_reaches_the_master`	296-301	Regression test: exception reaches the master.
`test_adapter_echo_and_foreign_traffic_are_ignored`	303-320	Regression test: adapter echo and foreign traffic are ignored.
`test_split_request_is_assembled`	322-330	Regression test: split request is assembled.
`test_garbage_before_a_request_is_dropped`	332-337	Regression test: garbage before a request is dropped.

Test	Lines	What it protects
`test_unknown_function_gets_exception_01_after_silence`	339-343	Regression test: unknown function gets exception 01 after silence.
`test_faults`	345-370	Regression test: faults.
`test_fault_probability`	372-383	Regression test: fault probability.

SlaveWindowTests

Protects Slave workspace entries, area validation, window overlap and the translated fault description.

Test	Lines	What it protects
`test_workspace_entry_round_trip_and_validation`	387-398	Regression test: workspace entry round trip and validation.
`test_table_field_shows_the_read_function_code`	400-406	Regression test: table field shows the read function code.
`test_window_overlap`	408-413	Regression test: window overlap.
`test_fault_description_translates`	415-420	Regression test: fault description translates.

SlaveGeneratorTests

Validates automatic values, checks counter, ramp, sine, toggle and random over time, and their workspace round trip.

Test	Lines	What it protects
`test_validation`	424-439	Regression test: validation.
`test_values_over_time`	441-462	Regression test: values over time.
`test_generators_are_saved_and_follow_the_window`	464-474	Regression test: generators are saved and follow the window.

SlaveOpenFromTrafficTests

Checks which Traffic records can open a Slave window and with which table and area.

Test	Lines	What it protects
`test_reads_and_writes_give_the_slave_area`	478-489	Regression test: reads and writes give the slave area.
`test_frames_without_an_area_are_refused`	491-496	Regression test: frames without an area are refused.

SlaveSettingsTests

Checks the Slave workspace folder and the validated slave section of settings.json.

Test	Lines	What it protects
`test_slave_workspaces_have_their_own_folder`	512-518	Regression test: slave workspaces have their own folder.
`test_slave_settings_are_validated`	520-533	Regression test: slave settings are validated.

MbapFramingTests

Checks the MBAP header against a known ADU, the RTU conversion, impossible lengths and host/port validation.

Test	Lines	What it protects
`test_adu_round_trip`	51-59	Regression test: adu round trip.
`test_incomplete_and_impossible_headers`	61-66	Regression test: incomplete and impossible headers.
`test_host_and_port_validation`	68-76	Regression test: host and port validation.

ServerClientTests

Runs the Client engine against the Server engine on 127.0.0.1: reads, writes, Exceptions 02 and 0B, events, every fault, several clients at once, an unreachable device and a port in use.

Test	Lines	What it protects
`test_reads_writes_and_exceptions`	118-136	Regression test: reads writes and exceptions.
`test_server_events`	138-146	Regression test: server events.
`test_faults`	148-167	Regression test: faults.
`test_several_clients_at_once`	169-181	Regression test: several clients at once.
`test_unreachable_device_and_reconnection`	183-196	Regression test: unreachable device and reconnection.
`test_port_in_use_is_reported`	198-203	Regression test: port in use is reported.
`test_unit_ids_above_247`	205-208	Regression test: unit ids above 247.

ClientWindowTests

Protects Client workspace entries (host, port, Unit ID 0–255), the Unit ID in written requests and the tabs' own settings and folders.

Test	Lines	What it protects
`test_workspace_entry_round_trip_and_validation`	212-223	Regression test: workspace entry round trip and validation.
`test_write_uses_the_unit_id`	225-231	Regression test: write uses the unit id.
`test_tabs_keep_their_own_settings_and_folders`	233-237	Regression test: tabs keep their own settings and folders.

TcpSettingsTests

Validates the client and server sections of settings.json.

Test	Lines	What it protects
`test_client_and_server_settings_are_validated`	253-266	Regression test: client and server settings are validated.

15. Build and launch automation

The package contains three Windows batch files. Their complete source is reproduced in Appendix A.

MBToolkit.bat

Windows batch automation included in the package.

Lines	SHA-256
210	`4dcfe23fa45a1b97e136be2db75b4fb4763cb0db3cfae46edb4464cf19b4f52e`

- Sets the expected versioned source folder and verifies that MBToolkit.py exists.
- Detects Python 3 through py -3, python on PATH, or standard Python 3.13 install locations.
- If Python is absent and WinGet exists, attempts a silent per-user Python 3.13 installation.
- Verifies tkinter; if missing, attempts a Python repair/reinstall.
- Verifies pyserial and installs it with pip/ensurepip when required.
- Prefers pythonw.exe so the GUI opens without a console window, with python as fallback.

MBToolkit v3.5/build_exe.bat

Windows batch automation included in the package.

Lines	SHA-256
151	`5d1b64d6be1e91673938109b0d8495caac3e256065f8d891160f423302b2679c`

- Normalizes application/distribution paths and finds Python 3.
- Installs/upgrades pyserial and PyInstaller.
- Runs the complete regression suite before building; any failure cancels the build.
- Cleans prior build/spec/output artifacts.
- Creates a one-file, windowed MBToolkit.exe with the icon embedded when available.

MBToolkit v3.5/run_tests.bat

Windows batch automation included in the package.

Lines	SHA-256
45	`98c20bdbf89f4f478c4b38b7c8f5a55fae834f328edaf7d2979c8823f2236eae`

- Finds a Python 3 interpreter.
- Runs unittest discovery for tests/test_*.py in verbose mode.
- Returns a non-zero exit code and explicit warning when tests fail, so a release should not proceed.

16. Known boundaries and extension points

What the current v3.5 does not implement

- No Modbus TCP traffic capture (Sniffer) or device scan (Bus Slave Finder): both remain RTU only.
- Master tab: no data logging, conditional colours, charts, Test Center, FC22/23 or OLE-style automation.
- Slave and Server tabs: no FC08/FC22/FC23/FC43 and no traffic view of their own.
- Client and Server: no Modbus RTU over TCP, no UDP and no TCP↔RTU gateway.
- No passive Ethernet packet capture.
- No analog RS485 electrical measurement.

Current parser scope

Known frame-shape recognition covers the Function Codes encoded in `_candidate_prefixes_at()`. An unsupported FC can only be accepted conservatively as `UNKNOWN / OK` when it occupies the complete remaining burst and has a valid CRC. This intentionally favors resynchronisation reliability over aggressive guessing.

Natural refactoring seams

- **Transport abstraction:** `mb_protocol.py` is already mostly transport-independent above RTU framing, making it the natural starting point for a future shared Modbus PDU core.
- **Active client engine:** the Finder already contains robust serial request/write/read/timeout primitives that could inform a future general Master engine, but the thesis-quality design should separate those primitives from the Finder UI.
- **Gateway:** the Server and the serial Master already share the Modbus core; a TCP↔RTU gateway would forward Server requests to a Master engine.
- **Testing:** the existing hardware-independent regression suite is a strong base; future transport/client/server work should extend the same pattern.

> **Documentation intent.** These are architectural extension points inferred from the current code structure, not features claimed to exist in v3.5.

Appendix A - Complete source-code screenshots

Every Python and batch source file in the package is reproduced below as syntax-highlighted screenshots with original line numbers. Very long source lines are visually wrapped inside the screenshot for readability; this does not alter the source file. The SHA-256 values in the module chapters identify the exact files represented.

MBToolkit v3.5/MBToolkit.py

29 source lines.

MBToolkit v3.5/MBToolkit.py — lines 1-29

```python
#!/usr/bin/env python3
# MBToolkit v3.5
"""MBToolkit launcher."""

# Simulação:
#   0 = simulação inativa (o botão não aparece no GUI)
#   1 = simulação ativa  (o botão aparece no GUI)
#
# Mantido neste ficheiro para preservar o mesmo ponto de configuração usado
# antes da modularização.
debug_sim = 0

import mb_config

# Apply the launcher switch before importing GUI/capture modules, which read
# the value during import.
mb_config.debug_sim = debug_sim

from mb_config import set_windows_app_user_model_id
from mb_gui import SnifferApp


def main():
    set_windows_app_user_model_id()
    SnifferApp().mainloop()


if __name__ == "__main__":
    main()
```

MBToolkit v3.5/mb_config.py

637 source lines.

MBToolkit v3.5/mb_config.py — lines 1-40

```
#!/usr/bin/env python3
"""MBToolkit application configuration, constants and filesystem helpers."""

import ctypes
import ipaddress
import json
import os
import shutil
import sys
from pathlib import Path

APP_NAME = "MBToolkit"
# Product name up to v3.2. Its settings folder is migrated on first start.
LEGACY_APP_NAME = "MBSniffer"
APP_VERSION = "3.5"

# Simulação:
#   0 = simulação inativa (o botão não aparece no GUI)
#   1 = simulação ativa  (o botão aparece no GUI)
debug_sim = 0

# Active Modbus RTU slave discovery.
DISCOVERY_BAUD_RATES = (1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200)
DISCOVERY_PARITIES = ("None", "Even", "Odd")
DISCOVERY_STOP_BITS = ("1", "2")
DISCOVERY_DEFAULT_MIN_TIMEOUT_MS = 100
# v3.0 default; settings saved by v3.0 with this value migrate to the new default.
DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS = 40
DISCOVERY_MIN_TIMEOUT_MS = 10
DISCOVERY_MAX_TIMEOUT_MS = 2000
DISCOVERY_PROBE_ADDRESS = 0
DISCOVERY_PROBE_QTY = 1
DISCOVERY_WRITE_TIMEOUT_SECONDS = 1.5
DISCOVERY_WRITE_RETRIES = 1
DISCOVERY_RX_BUFFER_LIMIT = 1024
# Total probe attempts per FC when a Slave does not answer.
DISCOVERY_MIN_ATTEMPTS = 1
DISCOVERY_MAX_ATTEMPTS = 3
DISCOVERY_DEFAULT_ATTEMPTS = 1
# Passive listen before the first transmission; any byte aborts the scan.
```

MBToolkit v3.5/mb_config.py — lines 41-80

```
41 DISCOVERY_IDLE_CHECK_SECONDS = 1.0
42
43 # GUI queue processing limits.
44 UI_QUEUE_BATCH_SIZE = 200
45 UI_QUEUE_IDLE_INTERVAL_MS = 50
46 UI_QUEUE_BUSY_INTERVAL_MS = 10
47
48 # Pending-request timeout.
49 MIN_PENDING_REQUEST_TIMEOUT_SECONDS = 1.0
50 DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS = 10.0
51 MAX_PENDING_REQUEST_TIMEOUT_SECONDS = 300.0
52 PENDING_REQUEST_TIMEOUT_SECONDS = DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS
53
54 # Master (poll) tab. Modbus limits per request; RTU response timeout and the
55 # silent gap between two requests are user settings.
56 MASTER_MAX_READ_BITS = 2000
57 MASTER_MAX_READ_REGISTERS = 125
58 MASTER_MAX_WRITE_BITS = 1968
59 MASTER_MAX_WRITE_REGISTERS = 123
60 MASTER_DEFAULT_RESPONSE_TIMEOUT_MS = 1000
61 MASTER_MIN_RESPONSE_TIMEOUT_MS = 10
62 MASTER_MAX_RESPONSE_TIMEOUT_MS = 10000
63 MASTER_DEFAULT_DELAY_MS = 20
64 MASTER_MIN_DELAY_MS = 0
65 MASTER_MAX_DELAY_MS = 10000
66 MASTER_DEFAULT_SCAN_MS = 1000
67 MASTER_MAX_SCAN_MS = 3600000
68 MASTER_MAX_WINDOWS = 32
69 # When the serial port fails (e.g. USB adapter unplugged), the Master keeps
70 # trying to reopen it at this interval until it succeeds or is disconnected.
71 MASTER_RECONNECT_INTERVAL_SECONDS = 2.0
72 MASTER_WORKSPACE_FORMAT = "MBToolkit Master workspace"
73 # Workspaces saved by MBSniffer v3.2 use this format name and still open.
74 MASTER_WORKSPACE_FORMATS = (MASTER_WORKSPACE_FORMAT, "MBSniffer Master workspace")
75 MASTER_WORKSPACE_VERSION = 1
76
77 # Slave (simulator) tab. Each window is one data table of one simulated Slave
78 # ID; requests are answered from those tables.
79 SLAVE_MAX_WINDOWS = 32
80 SLAVE_MAX_QTY = 2000
```

MBToolkit v3.5/mb_config.py — lines 81-120

```
# A master write is shown highlighted in the table for this long.
SLAVE_HIGHLIGHT_SECONDS = 1.5
SLAVE_FAULT_MAX_DELAY_MS = 10000
# Automatic values (counter, ramp, sine, random, toggle) are recomputed at
# this interval.
SLAVE_GENERATOR_TICK_MS = 100
# Bytes that do not form a request (other devices' responses, noise) are
# discarded once the bus has been silent this long. Long enough for the
# latency-timer chunks of USB-serial adapters (FTDI default 16 ms).
SLAVE_RESYNC_SILENCE_MS = 40.0
SLAVE_WORKSPACE_FORMAT = "MBToolkit Slave workspace"
SLAVE_WORKSPACE_VERSION = 1

# Modbus TCP Client and Server tabs (same windows as Master and Slave).
CLIENT_WORKSPACE_FORMAT = "MBToolkit Client workspace"
SERVER_WORKSPACE_FORMAT = "MBToolkit Server workspace"
TCP_DEFAULT_PORT = 502
# The Server listens only on this PC until another interface is chosen.
SERVER_DEFAULT_BIND = "127.0.0.1"

# Sniffer burst assembly: extra wait when a burst ends with the start of a
# recognised frame. USB-serial adapters deliver bytes in latency-timer
# chunks (FTDI default 16 ms), which can split one RTU frame in two.
SNIFFER_SPLIT_FRAME_HOLD_MS = 40.0

# Diagnostic thresholds.
SLOW_RESPONSE_THRESHOLD_MS = 500.0

# Standard Modbus exception response codes.
MODBUS_EXCEPTION_CODES = {
    0x01: (
        "Illegal Function",
        "A função Modbus pedida não é suportada/permitida pelo dispositivo."
    ),
    0x02: (
        "Illegal Data Address",
        "O endereço ou a gama de endereços pedida não é válida neste dispositivo."
    ),
    0x03: (
        "Illegal Data Value",
```

MBToolkit v3.5/mb_config.py — lines 121-160

```
        "Um valor, quantidade ou campo do pedido é inválido para esta função."
    ),
    0x04: (
        "Server Device Failure",
        "O dispositivo encontrou um erro interno ao tentar executar o pedido."
    ),
    0x05: (
        "Acknowledge",
        "O pedido foi aceite, mas necessita de mais tempo para ser processado."
    ),
    0x06: (
        "Server Device Busy",
        "O dispositivo está ocupado e não consegue processar o pedido neste momento."
    ),
    0x08: (
        "Memory Parity Error",
        "Foi detetado um erro de paridade ao aceder à memória do dispositivo."
    ),
    0x0A: (
        "Gateway Path Unavailable",
        "O gateway não conseguiu estabelecer o caminho para o dispositivo de destino."
    ),
    0x0B: (
        "Gateway Target Device Failed to Respond",
        "O gateway não recebeu resposta do dispositivo de destino."
    ),
}

# Bounded GUI history. The disk log remains complete.
MAX_UI_FRAMES = 20000
MAX_RAW_TEXT_LINES = 20000
UI_PRUNE_CHUNK_FRAMES = 2000
RAW_TEXT_PRUNE_CHUNK_LINES = 2000
RAW_TRANSACTION_SEPARATOR = "—" * 72 + "\n"

SIMULATION_FUNCTION_CODES = (1, 2, 3, 4)
SIMULATION_EXCEPTION_CODES = (1, 2, 3, 4, 6)

def resource_path(filename):
    """Return a bundled/source resource path for normal Python or PyInstaller."""
```

MBToolkit v3.5/mb_config.py — lines 161-200

```
    try:
        base = Path(sys._MEIPASS)
    except (AttributeError, TypeError):
        base = Path(__file__).resolve().parent
    return base / filename

def set_windows_app_user_model_id():
    """
    Give Windows a stable application identity so the taskbar can associate
    the Tk window with MBToolkit instead of a generic Python process.
    """
    if os.name != "nt":
        return
    try:
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID(
            f"{APP_NAME}.{APP_VERSION}"
        )
    except Exception:
        pass



def get_settings_file() -> Path:
    """
    Return the persistent per-user settings file.

    Windows:
        %APPDATA%\\MBToolkit\\settings.json

    Fallback:
        ~/.mbtoolkit/settings.json

    This location is independent from the EXE/BAT folder, so the selected
    Light/Dark mode survives application updates and also works when the
    application folder is read-only.
    """
    try:
        folder = _settings_folder(APP_NAME)
        if not folder.exists():
            _migrate_legacy_settings(folder)
```

MBToolkit v3.5/mb_config.py — lines 201-240

```
201         folder.mkdir(parents=True, exist_ok=True)
202         return folder / "settings.json"
203     except Exception:
204         return Path.home() / ".mbtoolkit_settings.json"
205 
206 
207 def _settings_folder(app_name):
208     if os.name == "nt":
209         roaming_app_data = os.environ.get("APPDATA")
210         if roaming_app_data:
211             return Path(roaming_app_data) / app_name
212         return Path.home() / "AppData" / "Roaming" / app_name
213     return Path.home() / f".{app_name.lower()}"
214 
215 
216 def _migrate_legacy_settings(folder):
217     """Copy MBSniffer's settings and workspaces into the new MBToolkit folder.
218 
219     Runs once, when the MBToolkit folder does not exist yet. The old folder
220     is left untouched. Failures are ignored: settings are convenience state.
221     """
222     legacy = _settings_folder(LEGACY_APP_NAME)
223     if not legacy.is_dir():
224         return
225     try:
226         shutil.copytree(legacy, folder)
227     except Exception:
228         pass
229 
230 
231 def _validated_choice(value, allowed, default):
232     return value if value in allowed else default
233 
234 
235 def _validated_bool(value, default):
236     return value if isinstance(value, bool) else default
237 
238 
239 def _validated_number_string(value, minimum, maximum, default):
240     try:
```

MBToolkit v3.5/mb_config.py — lines 241-280

```
        number = float(value)
        if minimum <= number <= maximum:
            return f"{number:g}"
    except (TypeError, ValueError):
        pass
    return str(default)


def _validated_int_string(value, minimum, maximum, default):
    try:
        number = int(value)
        if minimum <= number <= maximum:
            return str(number)
    except (TypeError, ValueError):
        pass
    return str(default)


def load_ui_settings() -> dict:
    """
    Load validated user preferences from the single Roaming settings file.

    Machine-specific state such as COM-port names and window geometry is
    deliberately excluded.
    """
    path = get_settings_file()
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except Exception:
        return {}

    if not isinstance(data, dict):
        return {}

    result = {}

    if isinstance(data.get("dark_mode"), bool):
        result["dark_mode"] = data["dark_mode"]

    if data.get("language") in ("Português", "English"):
```

MBToolkit v3.5/mb_config.py — lines 281-320

```
        result["language"] = data["language"]

    if data.get("last_main_tab") in ("Sniffer", "Bus Slave Finder", "Master", "Slave",
                                     "Client", "Server"):
        result["last_main_tab"] = data["last_main_tab"]

    sniffer_in = data.get("sniffer")
    if isinstance(sniffer_in, dict):
        # v2.9 migration: the UI option was renamed from "Realçar anomalias"
        # to "Realçar resultados". Accept the old key once so existing user
        # preferences are not lost, but expose/save only the new name.
        highlight_results_value = sniffer_in.get(
            "highlight_results",
            sniffer_in.get("highlight_anomalies"),
        )

        result["sniffer"] = {
            "physical_mode": _validated_choice(
                sniffer_in.get("physical_mode"),
                ("RS485 2-wire", "RS232 single RX", "RS232 dual RX"),
                "RS485 2-wire",
            ),
            "baud": _validated_choice(
                str(sniffer_in.get("baud", "")),
                tuple(str(v) for v in DISCOVERY_BAUD_RATES),
                "9600",
            ),
            "data_bits": _validated_choice(
                str(sniffer_in.get("data_bits", "")),
                ("5", "6", "7", "8"),
                "8",
            ),
            "parity": _validated_choice(
                sniffer_in.get("parity"),
                ("None", "Even", "Odd", "Mark", "Space"),
                "None",
            ),
            "stop_bits": _validated_choice(
                str(sniffer_in.get("stop_bits", "")),
                ("1", "1.5", "2"),
```

MBToolkit v3.5/mb_config.py — lines 321-360

```
                "1",
            ),
            "frame_gap_mode": _validated_choice(
                sniffer_in.get("frame_gap_mode"),
                ("Auto", "Manual"),
                "Auto",
            ),
            "frame_gap_ms": str(sniffer_in.get("frame_gap_ms", "")),
            "pending_timeout_s": _validated_number_string(
                sniffer_in.get("pending_timeout_s"),
                MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
                MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
                DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS,
            ),
            "auto_scroll": _validated_bool(
                sniffer_in.get("auto_scroll"),
                True,
            ),
            "separate_transactions": _validated_bool(
                sniffer_in.get("separate_transactions"),
                True,
            ),
            "highlight_results": _validated_bool(
                highlight_results_value,
                False,
            ),
        }

    finder_in = data.get("bus_slave_finder")
    if isinstance(finder_in, dict):
        saved_bauds = finder_in.get("baud_rates")
        if isinstance(saved_bauds, list):
            baud_rates = [
                int(v) for v in saved_bauds
                if isinstance(v, (int, float, str))
                and str(v).isdigit()
                and int(v) in DISCOVERY_BAUD_RATES
            ]
        else:
            baud_rates = [9600, 19200]
```

MBToolkit v3.5/mb_config.py — lines 361-400

```
361
362         saved_parities = finder_in.get("parities")
363         if isinstance(saved_parities, list):
364             parities = [
365                 v for v in saved_parities
366                 if v in DISCOVERY_PARITIES
367             ]
368         else:
369             parities = ["None", "Even"]
370
371         saved_stops = finder_in.get("stop_bits")
372         if isinstance(saved_stops, list):
373             stop_bits = [
374                 str(v) for v in saved_stops
375                 if str(v) in DISCOVERY_STOP_BITS
376             ]
377         else:
378             stop_bits = ["1"]
379
380         min_timeout_ms = _validated_int_string(
381             finder_in.get("min_timeout_ms"),
382             DISCOVERY_MIN_TIMEOUT_MS,
383             DISCOVERY_MAX_TIMEOUT_MS,
384             DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
385         )
386         # Settings written before v3.1 have no "attempts" key. Their 40 ms
387         # value is the old automatic default, not a user choice.
388         if (
389             "attempts" not in finder_in
390             and min_timeout_ms == str(DISCOVERY_LEGACY_DEFAULT_MIN_TIMEOUT_MS)
391         ):
392             min_timeout_ms = str(DISCOVERY_DEFAULT_MIN_TIMEOUT_MS)
393
394         result["bus_slave_finder"] = {
395             "baud_rates": baud_rates,
396             "parities": parities,
397             "stop_bits": stop_bits,
398             "slave_start": _validated_int_string(
399                 finder_in.get("slave_start"),
400                 1,
```

MBToolkit v3.5/mb_config.py — lines 401-440

```
                247,
                1,
            ),
            "slave_end": _validated_int_string(
                finder_in.get("slave_end"),
                1,
                247,
                247,
            ),
            "min_timeout_ms": min_timeout_ms,
            "attempts": _validated_int_string(
                finder_in.get("attempts"),
                DISCOVERY_MIN_ATTEMPTS,
                DISCOVERY_MAX_ATTEMPTS,
                DISCOVERY_DEFAULT_ATTEMPTS,
            ),
            "fc04_fallback": _validated_bool(
                finder_in.get("fc04_fallback"),
                False,
            ),
            "device_identification": _validated_bool(
                finder_in.get("device_identification"),
                False,
            ),
        }

    master_in = data.get("master")
    if isinstance(master_in, dict):
        result["master"] = {
            "baud": _validated_choice(
                str(master_in.get("baud", "")),
                tuple(str(v) for v in DISCOVERY_BAUD_RATES),
                "9600",
            ),
            "parity": _validated_choice(
                master_in.get("parity"), ("None", "Even", "Odd"), "None"
            ),
            "stop_bits": _validated_choice(
                str(master_in.get("stop_bits", "")), ("1", "2"), "1"
            ),
```

MBToolkit v3.5/mb_config.py — lines 441-480

```
            "response_timeout_ms": _validated_int_string(
                master_in.get("response_timeout_ms"),
                MASTER_MIN_RESPONSE_TIMEOUT_MS,
                MASTER_MAX_RESPONSE_TIMEOUT_MS,
                MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
            ),
            "delay_ms": _validated_int_string(
                master_in.get("delay_ms"),
                MASTER_MIN_DELAY_MS,
                MASTER_MAX_DELAY_MS,
                MASTER_DEFAULT_DELAY_MS,
            ),
            # Workspace reopened at start-up (name of a file in Workspaces).
            "last_workspace": _validated_workspace_name(master_in.get("last_workspace")),
        }

    slave_in = data.get("slave")
    if isinstance(slave_in, dict):
        result["slave"] = {
            "baud": _validated_choice(
                str(slave_in.get("baud", "")),
                tuple(str(v) for v in DISCOVERY_BAUD_RATES),
                "9600",
            ),
            "parity": _validated_choice(
                slave_in.get("parity"), ("None", "Even", "Odd"), "None"
            ),
            "stop_bits": _validated_choice(
                str(slave_in.get("stop_bits", "")), ("1", "2"), "1"
            ),
            "last_workspace": _validated_workspace_name(slave_in.get("last_workspace")),
        }

    client_in = data.get("client")
    if isinstance(client_in, dict):
        result["client"] = {
            "response_timeout_ms": _validated_int_string(
                client_in.get("response_timeout_ms"),
                MASTER_MIN_RESPONSE_TIMEOUT_MS,
                MASTER_MAX_RESPONSE_TIMEOUT_MS,
```

MBToolkit v3.5/mb_config.py — lines 481-520

```
481                 MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
482             ),
483             "delay_ms": _validated_int_string(
484                 client_in.get("delay_ms"),
485                 MASTER_MIN_DELAY_MS,
486                 MASTER_MAX_DELAY_MS,
487                 MASTER_DEFAULT_DELAY_MS,
488             ),
489             "last_workspace": _validated_workspace_name(client_in.get("last_workspace")),
490         }
491
492     server_in = data.get("server")
493     if isinstance(server_in, dict):
494         result["server"] = {
495             "bind": _validated_ipv4(server_in.get("bind"), SERVER_DEFAULT_BIND),
496             "port": _validated_int_string(server_in.get("port"), 1, 65535, TCP_DEFAULT_PORT),
497             "last_workspace": _validated_workspace_name(server_in.get("last_workspace")),
498         }
499
500     return result
501
502
503 def _validated_ipv4(value, default):
504     """IPv4 address text remembered in settings.json, or *default*."""
505     try:
506         return str(ipaddress.IPv4Address(str(value).strip()))
507     except (ipaddress.AddressValueError, ValueError):
508         return default
509
510
511 def _validated_workspace_name(value):
512     """Workspace name remembered in settings.json, or "" when unusable."""
513     if not isinstance(value, str):
514         return ""
515     value = value.strip()
516     if not 0 < len(value) <= 60 or any(ch in '<>:"/\\|?*' for ch in value):
517         return ""
518     return value
519
520
```

MBToolkit v3.5/mb_config.py — lines 521-560

```python
def save_ui_settings(settings=None, **updates) -> None:
    """
    Persist UI preferences atomically in the single Roaming settings file.

    Partial updates merge with the existing file. This lets the Dark-mode
    toggle update immediately without discarding the other remembered options.
    """
    path = get_settings_file()

    current = load_ui_settings()
    if isinstance(settings, dict):
        current.update(settings)
    current.update(updates)

    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        tmp = path.with_suffix(path.suffix + ".tmp")
        tmp.write_text(
            json.dumps(current, indent=2, sort_keys=True),
            encoding="utf-8",
        )
        tmp.replace(path)
    except Exception:
        # Remembered UI state must never break application use.
        pass


def get_application_directory() -> Path:
    """
    Return the user-facing application directory.

    - PyInstaller EXE: directory containing MBToolkit.exe.
    - Packaged Python/BAT mode: directory containing the root MBToolkit.bat.
      The Python sources live inside the versioned child folder, e.g. ``MBToolkit v3.5``.
    - Development Python mode: directory containing this module.
    """
    try:
        if getattr(sys, "frozen", False):
            return Path(sys.executable).resolve().parent

```

MBToolkit v3.5/mb_config.py — lines 561-600

```
        source_dir = Path(__file__).resolve().parent
        launcher_dir = source_dir.parent

        if (launcher_dir / "MBToolkit.bat").is_file():
            return launcher_dir

        return source_dir
    except Exception:
        return Path.cwd()

def get_workspaces_folder() -> Path:
    """
    Return/create the Master workspace folder beside settings.json:

        %APPDATA%\\MBToolkit\\Workspaces
    """
    folder = get_settings_file().parent / "Workspaces"
    folder.mkdir(parents=True, exist_ok=True)
    return folder


def get_master_workspaces_folder() -> Path:
    """
    Return/create the Master workspace folder inside the workspace folder:

        %APPDATA%\\MBToolkit\\Workspaces\\Master

    Up to v3.4 the Master workspaces were kept directly in Workspaces. The
    first time this folder is created, those files are moved into it.
    """
    root = get_workspaces_folder()
    folder = root / "Master"
    if not folder.exists():
        folder.mkdir(parents=True, exist_ok=True)
        for old in root.glob("*.mbsw"):
            try:
                if old.is_file() and not (folder / old.name).exists():
                    shutil.move(str(old), str(folder / old.name))
            except OSError:
                pass
```

MBToolkit v3.5/mb_config.py — lines 601-637

```
601     return folder
602 
603 
604 def get_client_workspaces_folder() -> Path:
605     """Return/create %APPDATA%\\MBToolkit\\Workspaces\\Client (Modbus TCP Client)."""
606     folder = get_workspaces_folder() / "Client"
607     folder.mkdir(parents=True, exist_ok=True)
608     return folder
609 
610 
611 def get_server_workspaces_folder() -> Path:
612     """Return/create %APPDATA%\\MBToolkit\\Workspaces\\Server (Modbus TCP Server)."""
613     folder = get_workspaces_folder() / "Server"
614     folder.mkdir(parents=True, exist_ok=True)
615     return folder
616 
617 
618 def get_slave_workspaces_folder() -> Path:
619     """
620     Return/create the Slave workspace folder inside the workspace folder:
621 
622         %APPDATA%\\MBToolkit\\Workspaces\\Slave
623     """
624     folder = get_workspaces_folder() / "Slave"
625     folder.mkdir(parents=True, exist_ok=True)
626     return folder
627 
628 
629 def get_logs_folder() -> Path:
630     """
631     Return/create the log directory beside MBToolkit.bat or MBToolkit.exe:
632 
633         <launcher/exe directory>/MBToolkit Logs
634     """
635     folder = get_application_directory() / "MBToolkit Logs"
636     folder.mkdir(parents=True, exist_ok=True)
637     return folder
```

MBToolkit v3.5/mb_protocol.py

1007 source lines.

MBToolkit v3.5/mb_protocol.py — lines 1-40

```python
#!/usr/bin/env python3
"""Modbus RTU parsing, CRC, transaction metrics and simulation helpers."""

import math
import random
import time
from bisect import bisect_right
from collections import defaultdict, deque
from datetime import datetime

from mb_config import (
    MODBUS_EXCEPTION_CODES,
    PENDING_REQUEST_TIMEOUT_SECONDS,
    SIMULATION_FUNCTION_CODES,
    SIMULATION_EXCEPTION_CODES,
    SLOW_RESPONSE_THRESHOLD_MS,
)

def _build_crc16_table():
    table = []
    for value in range(256):
        crc = value
        for _ in range(8):
            if crc & 1:
                crc = (crc >> 1) ^ 0xA001
            else:
                crc >>= 1
        table.append(crc)
    return tuple(table)

# Table-driven CRC: one lookup per byte instead of eight shift/XOR steps. The
# resynchronisation scan computes CRCs at many offsets, so this matters on
# noisy buses.
_CRC16_TABLE = _build_crc16_table()

def crc16_modbus(data: bytes) -> int:
    crc = 0xFFFF
    table = _CRC16_TABLE
    for b in data:
        crc = (crc >> 8) ^ table[(crc ^ b) & 0xFF]
```

MBToolkit v3.5/mb_protocol.py — lines 41-80

```
41      return crc
42
43  def crc_ok(frame: bytes) -> bool:
44      if len(frame) < 4:
45          return False
46      expected = crc16_modbus(frame[:-2])
47      received = frame[-2] | (frame[-1] << 8)
48      return expected == received
49
50  def append_crc(payload: bytes) -> bytes:
51      crc = crc16_modbus(payload)
52      return payload + bytes((crc & 0xFF, (crc >> 8) & 0xFF))
53
54  def hex_bytes(data: bytes) -> str:
55      return " ".join(f"{b:02X}" for b in data)
56
57  def modbus_rtu_interframe_gap_seconds(baud: int, bits_per_char: float) -> float:
58      """Return the recommended Modbus RTU t3.5 inter-frame gap.
59
60      Modbus Serial Line V1.02 requires the character-time timers to be
61      respected at baud rates up to and including 19200 bit/s. Above 19200
62      bit/s it recommends a fixed t3.5 value of 1.750 ms.
63      """
64      baud = int(baud)
65      bits_per_char = float(bits_per_char)
66      if baud <= 0:
67          raise ValueError("Baud rate must be greater than zero")
68      if bits_per_char <= 0:
69          raise ValueError("Bits per character must be greater than zero")
70      if baud > 19200:
71          return 0.001750
72      return 3.5 * bits_per_char / float(baud)
73
74  def plausible_request_qty(fc: int, qty: int) -> bool:
75      if fc in (1, 2):
76          return 1 <= qty <= 2000
77      if fc in (3, 4):
78          return 1 <= qty <= 125
79      return True
80
```

MBToolkit v3.5/mb_protocol.py — lines 81-120

```python
# Function Codes whose RTU frame length the parser can derive.
KNOWN_FRAME_FUNCTION_CODES = (1, 2, 3, 4, 5, 6, 7, 8, 15, 16, 22, 23)

# Function Codes whose normal response is an exact copy of the request.
EXACT_ECHO_FUNCTION_CODES = (5, 6, 22)

BROADCAST_UNIT_ID = 0

def _plausible_unit_id(value: int) -> bool:
    """Return True for Modbus RTU unit addresses that can appear on the wire."""
    try:
        return 0 <= int(value) <= 247
    except (TypeError, ValueError):
        return False


def _candidate_prefixes_at(buf, start=0):
    """Return structurally plausible known-frame lengths at *start*."""
    remaining = len(buf) - int(start)
    if remaining < 2:
        return []

    unit = buf[start]
    if not _plausible_unit_id(unit):
        return []

    fc = buf[start + 1]
    out = []

    if fc & 0x80:
        # Exception responses use FC | 0x80 and have a fixed five-byte length.
        base_fc = fc & 0x7F
        if base_fc in KNOWN_FRAME_FUNCTION_CODES:
            out.append((5, "EXCEPTION"))
        return out

    if fc in (1, 2, 3, 4):
        if remaining >= 6:
            qty = (buf[start + 4] << 8) | buf[start + 5]
            if plausible_request_qty(fc, qty):
```

MBToolkit v3.5/mb_protocol.py — lines 121-160

```
121                 out.append((8, "REQUEST"))
122
123         if remaining >= 3:
124             byte_count = buf[start + 2]
125             resp_len = 5 + byte_count
126             if fc in (3, 4):
127                 if byte_count > 0 and byte_count % 2 == 0:
128                     out.append((resp_len, "RESPONSE"))
129             elif byte_count > 0:
130                 out.append((resp_len, "RESPONSE"))
131
132     elif fc in (5, 6, 8):
133         out.append((8, "REQ/RESP"))
134
135     elif fc == 7:
136         # FC07 request: Slave + FC + CRC = 4 bytes.
137         # FC07 response: Slave + FC + Status + CRC = 5 bytes.
138         if remaining >= 5:
139             out.append((5, "RESPONSE"))
140         out.append((4, "REQUEST"))
141
142     elif fc in (15, 16):
143         out.append((8, "RESPONSE"))
144         if remaining >= 7:
145             byte_count = buf[start + 6]
146             if byte_count > 0:
147                 out.append((9 + byte_count, "REQUEST"))
148
149     elif fc == 22:
150         out.append((10, "REQ/RESP"))
151
152     elif fc == 23:
153         if remaining >= 3:
154             bc = buf[start + 2]
155             if bc > 0 and bc % 2 == 0:
156                 out.append((5 + bc, "RESPONSE"))
157         if remaining >= 11:
158             bc = buf[start + 10]
159             if bc > 0:
160                 out.append((13 + bc, "REQUEST"))
```

MBToolkit v3.5/mb_protocol.py — lines 161-200

```
161
162     cleaned = []
163     seen = set()
164     for length, kind in out:
165         if 4 <= length <= 260 and (length, kind) not in seen:
166             seen.add((length, kind))
167             cleaned.append((length, kind))
168     return cleaned
169
170
171 def candidate_prefixes(buf: bytes):
172     """Compatibility wrapper for callers/tests that inspect offset zero."""
173     return _candidate_prefixes_at(buf, 0)
174
175
176 def _crc_ok_at(buf, start: int, length: int) -> bool:
177     """CRC check without copying the whole remaining capture buffer."""
178     if length < 4 or start < 0 or start + length > len(buf):
179         return False
180     end = start + length
181     expected = crc16_modbus(memoryview(buf)[start:end - 2])
182     received = buf[end - 2] | (buf[end - 1] << 8)
183     return expected == received
184
185
186 def find_valid_prefix(buf: bytes, start=0):
187     """Find a CRC-valid, structurally recognised Modbus frame at *start*."""
188     for length, kind in _candidate_prefixes_at(buf, start):
189         if start + length <= len(buf) and _crc_ok_at(buf, start, length):
190             return length, kind
191     return None
192
193
194 def guess_invalid_whole_frame(buf: bytes):
195     """Classify a complete known-shape frame whose CRC is invalid."""
196     if len(buf) < 4 or not _plausible_unit_id(buf[0]):
197         return None
198     for length, kind in _candidate_prefixes_at(buf, 0):
199         if len(buf) == length:
200             return kind
```

MBToolkit v3.5/mb_protocol.py — lines 201-240

```
201     return None
202 
203 
204 def _unknown_whole_frame_is_valid(buf: bytes) -> bool:
205     """Conservative fallback for one unsupported FC occupying the whole burst.
206 
207     Unknown frames cannot be length-validated from their FC, so CRC-only
208     recognition is deliberately restricted to the complete remaining burst.
209     It is never used while scanning arbitrary offsets for resynchronisation.
210     """
211     if not 4 <= len(buf) <= 260:
212         return False
213     if not _plausible_unit_id(buf[0]):
214         return False
215     if _candidate_prefixes_at(buf, 0):
216         return False
217     return crc_ok(buf)
218 
219 
220 def split_capture_buffer(data: bytes):
221     results = []
222     buf = bytearray(data)
223 
224     while buf:
225         match = find_valid_prefix(buf, 0)
226         if match:
227             length, kind = match
228             results.append((bytes(buf[:length]), kind, "OK"))
229             del buf[:length]
230             continue
231 
232         # Search the full remaining burst, but only for structurally recognised
233         # Modbus frames. The former generic CRC scan tried up to 256 candidate
234         # lengths at every offset and became very slow on long noise bursts.
235         # Restricting resynchronisation to known frame shapes makes the scan
236         # effectively linear in the burst size and strongly reduces accidental
237         # CRC-only "phantom" frames in random noise.
238         found_offset = None
239         search_limit = max(len(buf) - 4, 0)
240         for offset in range(1, search_limit + 1):
```

MBToolkit v3.5/mb_protocol.py — lines 241-280

```
241             if find_valid_prefix(buf, offset):
242                 found_offset = offset
243                 break
244 
245         if found_offset is not None:
246             prefix = bytes(buf[:found_offset])
247 
248             # If the bytes immediately before the recovered frame are exactly a
249             # recognised Modbus frame shape, preserve the useful CRC diagnosis
250             # instead of hiding it as RAW/UNSYNC. Otherwise they are genuinely
251             # unsynchronised bytes and remain RAW.
252             guessed = guess_invalid_whole_frame(prefix)
253             if guessed:
254                 results.append((prefix, guessed, "ERROR"))
255             else:
256                 results.append((prefix, "RAW/UNSYNC", "UNKNOWN"))
257             del buf[:found_offset]
258             continue
259 
260         guessed = guess_invalid_whole_frame(bytes(buf))
261         if guessed:
262             results.append((bytes(buf), guessed, "ERROR"))
263         elif _unknown_whole_frame_is_valid(bytes(buf)):
264             results.append((bytes(buf), "UNKNOWN", "OK"))
265         else:
266             results.append((bytes(buf), "RAW/UNPARSED", "UNKNOWN"))
267         break
268 
269     return results
270 
271 def burst_may_continue(data: bytes) -> bool:
272     """Return True when *data* ends with the start of an unfinished frame.
273 
274     USB-serial adapters deliver received bytes in chunks, so a silence longer
275     than t3.5 can occur in the middle of one RTU frame. The reader uses this
276     to wait a little longer before closing such a burst. Only the final
277     unvalidated segment is inspected, and only at its first byte.
278     """
279     segments = split_capture_buffer(data)
280     if not segments:
```

MBToolkit v3.5/mb_protocol.py — lines 281-320

```
281         return False
282
283     tail, _kind, state = segments[-1]
284     if state == "OK":
285         return False
286
287     if not _plausible_unit_id(tail[0]):
288         return False
289     if len(tail) == 1:
290         return True
291
292     fc = tail[1]
293     if (fc & 0x7F) not in KNOWN_FRAME_FUNCTION_CODES:
294         return False
295     if len(tail) < 8 and not fc & 0x80:
296         # Too short for the request/response length fields to be known.
297         return True
298
299     return any(
300         length > len(tail)
301         for length, _kind in _candidate_prefixes_at(tail, 0)
302     )
303
304
305 def read_bits_from_response(frame: bytes, qty=None):
306     """Return FC01/FC02 coil/input states, least-significant bit first.
307
308     Without the matching request quantity the padding bits of the last data
309     byte cannot be distinguished, so every transmitted bit is returned.
310     """
311     if len(frame) < 5:
312         return []
313     byte_count = frame[2]
314     data = frame[3:3 + byte_count]
315     bits = [(byte >> bit) & 1 for byte in data for bit in range(8)]
316     try:
317         qty = int(qty) if qty is not None else None
318     except (TypeError, ValueError):
319         qty = None
320     if qty is not None and 0 < qty <= len(bits):
```

MBToolkit v3.5/mb_protocol.py — lines 321-360

```
321         bits = bits[:qty]
322     return bits
323 
324 
325 # FC01/FC02 can return up to 2000 bits; the Traffic row shows only the first
326 # ones (the Frame Inspector lists them all with their addresses).
327 READ_BITS_DETAILS_LIMIT = 32
328 
329 
330 def read_bits_details(frame: bytes, qty=None) -> str:
331     """Compact Details text: bits in address order, grouped per data byte."""
332     byte_count = frame[2] if len(frame) >= 3 else 0
333     bits = read_bits_from_response(frame, qty)
334     shown = "".join(str(bit) for bit in bits[:READ_BITS_DETAILS_LIMIT])
335     grouped = " ".join(shown[i:i + 8] for i in range(0, len(shown), 8))
336     if len(bits) > READ_BITS_DETAILS_LIMIT:
337         grouped += f" …(+{len(bits) - READ_BITS_DETAILS_LIMIT})"
338     return f"ByteCount={byte_count}  Bits={grouped}"
339 
340 
341 def decode_frame(frame: bytes, kind_hint: str):
342     info = {
343         "kind": kind_hint,
344         "slave": "",
345         "fc": "",
346         "details": "",
347         "pdu_address": None,
348         "one_based": None,
349         "qty": None,
350     }
351 
352     if str(kind_hint).startswith("RAW"):
353         info["details"] = f"{len(frame)} raw bytes"
354         return info
355 
356     if len(frame) < 2:
357         info["details"] = "Raw bytes"
358         return info
359 
360     slave = frame[0]
```

MBToolkit v3.5/mb_protocol.py — lines 361-400

```
    fc = frame[1]
    info["slave"] = str(slave)
    info["fc"] = f"{fc:02X}"

    if fc & 0x80:
        info["kind"] = "EXCEPTION"
        code = frame[2] if len(frame) >= 3 else None

        if code is None:
            info["details"] = "Exception"
            return info

        exception_info = MODBUS_EXCEPTION_CODES.get(code)
        if exception_info is not None:
            name, _description = exception_info
            info["details"] = f"Exception 0x{code:02X} — {name}"
        else:
            info["details"] = f"Exception 0x{code:02X} — Unknown/Reserved Exception"
        return info

    if fc in (1, 2, 3, 4):
        if kind_hint == "REQUEST" and len(frame) >= 8:
            start_addr = (frame[2] << 8) | frame[3]
            qty = (frame[4] << 8) | frame[5]
            info["pdu_address"] = start_addr
            info["one_based"] = start_addr + 1
            info["qty"] = qty
            info["details"] = (
                f"PDU Address={start_addr}  1-based={start_addr + 1}  Qty={qty}"
            )
            return info

        if kind_hint == "RESPONSE" and len(frame) >= 5:
            bc = frame[2]
            details = f"ByteCount={bc}"
            if fc in (3, 4) and bc % 2 == 0 and len(frame) >= 5 + bc:
                data = frame[3:3+bc]
                regs = [(data[i] << 8) | data[i+1] for i in range(0, len(data), 2)]
                details += f"  Registos={regs}"
            elif fc in (1, 2) and len(frame) >= 5 + bc:
```

MBToolkit v3.5/mb_protocol.py — lines 401-440

```
401                 details = read_bits_details(frame)
402             info["details"] = details
403             return info
404
405     if fc in (5, 6, 8) and len(frame) >= 8:
406         addr = (frame[2] << 8) | frame[3]
407         value = (frame[4] << 8) | frame[5]
408         info["pdu_address"] = addr
409         info["one_based"] = addr + 1
410         info["details"] = (
411             f"PDU Address={addr}  1-based={addr + 1}  Value=0x{value:04X}"
412         )
413         return info
414
415     if fc == 7:
416         if kind_hint == "REQUEST":
417             info["details"] = "Read Exception Status"
418         elif kind_hint == "RESPONSE" and len(frame) >= 5:
419             info["details"] = f"Status=0x{frame[2]:02X}"
420         else:
421             info["details"] = "Read Exception Status"
422         return info
423
424     if fc in (15, 16):
425         if kind_hint == "REQUEST" and len(frame) >= 9:
426             start_addr = (frame[2] << 8) | frame[3]
427             qty = (frame[4] << 8) | frame[5]
428             bc = frame[6]
429             info["pdu_address"] = start_addr
430             info["one_based"] = start_addr + 1
431             info["qty"] = qty
432             info["details"] = (
433                 f"PDU Address={start_addr}  1-based={start_addr + 1}  "
434                 f"Qty={qty}  ByteCount={bc}"
435             )
436             return info
437         if len(frame) >= 8:
438             start_addr = (frame[2] << 8) | frame[3]
439             qty = (frame[4] << 8) | frame[5]
440             info["pdu_address"] = start_addr
```

MBToolkit v3.5/mb_protocol.py — lines 441-480

```
            info["one_based"] = start_addr + 1
            info["qty"] = qty
            info["details"] = (
                f"PDU Address={start_addr}  1-based={start_addr + 1}  Qty={qty}"
            )
            return info

    info["details"] = f"{len(frame)} bytes"
    return info

def format_clock_time(epoch: float) -> str:
    return datetime.fromtimestamp(float(epoch)).strftime("%H:%M:%S.%f")[:-3]


def parse_burst(data: bytes, timestamp: str, channel: str = "", event_time=None,
                chunk_times=None):
    """Split one received burst into decoded rows.

    *chunk_times* is an optional ascending list of ``(byte_offset, epoch)``
    pairs recording when each serial read arrived. When given, every frame
    gets the arrival time of the chunk holding its first byte instead of the
    burst start time, so frames merged into one burst keep distinct times.
    """
    rows = []
    if event_time is None:
        event_time = time.time()

    chunk_offsets = [int(o) for o, _t in chunk_times] if chunk_times else []
    offset = 0

    for frame, kind, state in split_capture_buffer(data):
        frame_time = float(event_time)
        frame_timestamp = timestamp
        if chunk_offsets:
            index = bisect_right(chunk_offsets, offset) - 1
            if index > 0:
                frame_time = float(chunk_times[index][1])
                frame_timestamp = format_clock_time(frame_time)
        offset += len(frame)

```

MBToolkit v3.5/mb_protocol.py — lines 481-520

```
        info = decode_frame(frame, kind)
        crc_status = "OK" if state == "OK" else ("ERROR" if state == "ERROR" else "?")
        rows.append({
            "time": frame_timestamp,
            "channel": channel,
            "type": info["kind"],
            "slave": info["slave"],
            "fc": info["fc"],
            "details": info["details"],
            "crc": crc_status,
            "raw": hex_bytes(frame),
            "pdu_address": info["pdu_address"],
            "one_based": info["one_based"],
            "qty": info["qty"],
            "_frame_len": len(frame),
            "_epoch": frame_time,
        })
    return rows

class SessionMetrics:
    """Tracks pairing, counters and bus-health diagnostics for one session."""

    def __init__(self, pending_timeout_seconds=PENDING_REQUEST_TIMEOUT_SECONDS):
        self.pending_timeout_seconds = float(pending_timeout_seconds)
        self.reset()

    def reset(self):
        self.first_event_time = None
        self.last_event_time = None
        self.pending = {}
        self.requests = 0
        # Broadcast requests (Slave 0) never receive a response, so they are
        # never pending and are excluded from the timeout-rate denominator.
        self.broadcast_requests = 0
        self.responses = 0
        self.timeouts = 0
        self.crc_errors = 0
        self.exceptions = 0
        self.raw = 0

```

MBToolkit v3.5/mb_protocol.py — lines 521-560

```
        # Bus Health.
        self.total_bytes = 0
        self.nonvalidated_bytes = 0
        self.active_slaves = set()
        self.exception_by_slave = defaultdict(int)
        self.response_times_ms = deque(maxlen=10000)
        self.response_times_by_slave = defaultdict(lambda: deque(maxlen=2000))

        # Timeout events are exposed to the view so the original request row can
        # be marked/highlighted without inventing a fake serial frame.
        self._expired_requests = []

    def _key(self, row):
        try:
            slave = int(row.get("slave", ""))
            fc = int(row.get("fc", ""), 16)
        except (ValueError, TypeError):
            return None

        if row.get("type") == "EXCEPTION":
            fc &= 0x7F

        return (slave, fc)

    def pending_count(self):
        return sum(len(v) for v in self.pending.values())

    def pop_expired_requests(self):
        items = self._expired_requests
        self._expired_requests = []
        return items

    def expire_pending(self, now=None):
        """
        Expire stale requests and retain their sequence IDs for the GUI.

        Returns the number of newly expired requests, preserving the original
        public behavior used by CaptureMixin.
        """
        if now is None:
```

MBToolkit v3.5/mb_protocol.py — lines 561-600

```
            now = time.time()
        now = float(now)

        expired = 0
        cutoff = now - self.pending_timeout_seconds

        for key in list(self.pending.keys()):
            pending_list = self.pending[key]

            keep_from = 0
            for item in pending_list:
                if item["time"] <= cutoff:
                    expired += 1
                    keep_from += 1
                    event = dict(item)
                    event["slave"] = key[0]
                    event["fc"] = key[1]
                    self._expired_requests.append(event)
                else:
                    break

            if keep_from:
                del pending_list[:keep_from]

            if not pending_list:
                self.pending.pop(key, None)

        if expired:
            self.timeouts += expired

        return expired

    def force_timeout(self, slave, fc):
        """
        Debug/simulation helper: expire the oldest pending request for one key.

        It uses the same pending bookkeeping and returns the expired request
        descriptor so the GUI marks the real REQUEST row as timed out.
        """
        try:
```

MBToolkit v3.5/mb_protocol.py — lines 601-640

```
            key = (int(slave), int(fc))
        except (TypeError, ValueError):
            return []

        pending_list = self.pending.get(key, [])
        if not pending_list:
            return []

        item = pending_list.pop(0)
        if not pending_list:
            self.pending.pop(key, None)

        event = dict(item)
        event["slave"] = key[0]
        event["fc"] = key[1]
        self.timeouts += 1
        return [event]

    def _resolve_echo_direction(self, row):
        """Classify a REQ/RESP frame (FC05/06/08/22) as REQUEST or RESPONSE.

        These requests and their normal responses have the same shape, so the
        parser cannot tell them apart. A frame is the RESPONSE when an older
        pending request with the same (Slave, FC) exists; for exact-echo FCs
        the bytes must also be identical. In RS232 dual RX a response must
        arrive on the opposite channel to its request.
        """
        if row.get("crc") != "OK":
            return row.get("type", "")

        key = self._key(row)
        if key is None:
            return row.get("type", "")
        if key[0] == BROADCAST_UNIT_ID:
            return "REQUEST"

        pending_list = self.pending.get(key)
        if pending_list:
            candidate = pending_list[0]
            channel = row.get("channel", "")
```

MBToolkit v3.5/mb_protocol.py — lines 641-680

```
641             same_direction = (
642                 channel in ("A→B", "B→A")
643                 and candidate.get("channel") == channel
644             )
645             same_bytes = candidate.get("raw") == row.get("raw")
646             if not same_direction and (
647                 key[1] not in EXACT_ECHO_FUNCTION_CODES or same_bytes
648             ):
649                 return "RESPONSE"
650 
651         return "REQUEST"
652 
653     def _record_response_time(self, row, response_ms):
654         if response_ms is None or row.get("crc") != "OK":
655             return
656 
657         response_ms = float(response_ms)
658         self.response_times_ms.append(response_ms)
659 
660         try:
661             slave = int(row.get("slave", ""))
662         except (TypeError, ValueError):
663             return
664         self.response_times_by_slave[slave].append(response_ms)
665 
666     def process(self, row):
667         event_time = float(row.get("_epoch", time.time()))
668 
669         # Expire stale requests BEFORE pairing the current frame.
670         self.expire_pending(event_time)
671         expired_requests = self.pop_expired_requests()
672 
673         if self.first_event_time is None:
674             self.first_event_time = event_time
675 
676         delta_ms = None
677         if self.last_event_time is not None and event_time >= self.last_event_time:
678             delta_ms = (event_time - self.last_event_time) * 1000.0
679         if self.last_event_time is None or event_time >= self.last_event_time:
680             self.last_event_time = event_time
```

MBToolkit v3.5/mb_protocol.py — lines 681-720

```
681
682         if row.get("type") == "REQ/RESP":
683             row["type"] = self._resolve_echo_direction(row)
684
685         row_type = row.get("type", "")
686         crc = row.get("crc", "")
687
688         try:
689             frame_len = max(0, int(row.get("_frame_len", 0)))
690         except (TypeError, ValueError):
691             frame_len = 0
692         self.total_bytes += frame_len
693
694         try:
695             slave_num = int(row.get("slave", ""))
696             if 1 <= slave_num <= 247 and not str(row_type).startswith("RAW"):
697                 self.active_slaves.add(slave_num)
698         except (TypeError, ValueError):
699             slave_num = None
700
701         if crc == "ERROR":
702             self.crc_errors += 1
703
704         # RAW bytes cannot honestly be called CRC errors: there may be no
705         # complete frame on which to validate a CRC. For Bus Health we track
706         # them together with explicit CRC failures as traffic that could not be
707         # validated as a correct Modbus RTU frame.
708         if crc == "ERROR" or str(row_type).startswith("RAW"):
709             self.nonvalidated_bytes += frame_len
710
711         if row_type == "REQUEST":
712             self.requests += 1
713             key = self._key(row)
714             if key is not None and key[0] == BROADCAST_UNIT_ID:
715                 self.broadcast_requests += 1
716             elif key is not None and crc == "OK":
717                 self.pending.setdefault(key, []).append({
718                     "time": event_time,
719                     "seq": row.get("_view_seq"),
720                     "pdu_address": row.get("pdu_address"),
```

MBToolkit v3.5/mb_protocol.py — lines 721-760

```
                    "one_based": row.get("one_based"),
                    "qty": row.get("qty"),
                    "channel": row.get("channel", ""),
                    "raw": row.get("raw", ""),
                })

        elif row_type == "RESPONSE":
            self.responses += 1

        elif row_type == "EXCEPTION":
            self.responses += 1
            self.exceptions += 1
            if slave_num is not None:
                self.exception_by_slave[slave_num] += 1

        elif str(row_type).startswith("RAW"):
            self.raw += 1

        response_ms = None
        matched_request = None

        if row_type in ("RESPONSE", "EXCEPTION"):
            key = self._key(row)
            if key is not None:
                pending_list = self.pending.get(key, [])
                if pending_list:
                    matched_request = pending_list.pop(0)
                    if not pending_list:
                        self.pending.pop(key, None)

                    if event_time >= matched_request["time"]:
                        response_ms = (
                            event_time - matched_request["time"]
                        ) * 1000.0

        self._record_response_time(row, response_ms)

        return {
            "delta_ms": delta_ms,
            "response_ms": response_ms,
```

MBToolkit v3.5/mb_protocol.py — lines 761-800

```
            "matched_request": matched_request,
            "expired_requests": expired_requests,
            "pending": self.pending_count(),
            "timeouts": self.timeouts,
        }

    @staticmethod
    def _p95(values):
        if not values:
            return None
        ordered = sorted(float(v) for v in values)
        index = max(0, min(len(ordered) - 1, math.ceil(0.95 * len(ordered)) - 1))
        return ordered[index]

    def health_summary(
        self,
        *,
        baud=9600,
        data_bits=8,
        parity="None",
        stop_bits=1,
        now=None,
    ):
        """Return compact diagnostic metrics for the Bus Health view."""
        if now is None:
            now = time.time()

        if self.first_event_time is None:
            elapsed = 0.0
        else:
            end = max(
                float(now),
                float(self.last_event_time or self.first_event_time),
            )
            elapsed = max(0.0, end - float(self.first_event_time))

        responses = list(self.response_times_ms)
        response_avg = (
            sum(responses) / len(responses)
            if responses else None
```

MBToolkit v3.5/mb_protocol.py — lines 801-840

```
        )
        response_min = min(responses) if responses else None
        response_max = max(responses) if responses else None
        response_p95 = self._p95(responses)

        slowest_slave = None
        slowest_avg = None
        for slave, samples in self.response_times_by_slave.items():
            if not samples:
                continue
            avg = sum(samples) / len(samples)
            if slowest_avg is None or avg > slowest_avg:
                slowest_slave = slave
                slowest_avg = avg

        request_rate = (
            self.requests / elapsed
            if elapsed > 0.0 else 0.0
        )

        nonvalidated_rate = (
            100.0 * self.nonvalidated_bytes / self.total_bytes
            if self.total_bytes else 0.0
        )
        answerable_requests = self.requests - self.broadcast_requests
        timeout_rate = (
            100.0 * self.timeouts / answerable_requests
            if answerable_requests > 0 else 0.0
        )

        try:
            baud = max(1.0, float(baud))
        except (TypeError, ValueError):
            baud = 9600.0
        try:
            data_bits = float(data_bits)
        except (TypeError, ValueError):
            data_bits = 8.0
        try:
            stop_bits = float(stop_bits)
```

MBToolkit v3.5/mb_protocol.py — lines 841-880

```
        except (TypeError, ValueError):
            stop_bits = 1.0

        parity_bits = 0.0 if str(parity) == "None" else 1.0
        bits_per_char = 1.0 + data_bits + parity_bits + stop_bits
        bus_load = (
            100.0 * (self.total_bytes * bits_per_char) / (baud * elapsed)
            if elapsed > 0.0 else 0.0
        )

        exception_summary = ", ".join(
            f"S{slave}:{count}"
            for slave, count in sorted(self.exception_by_slave.items())
        ) or "-"

        return {
            "elapsed_s": elapsed,
            "request_rate": request_rate,
            "response_avg_ms": response_avg,
            "response_min_ms": response_min,
            "response_max_ms": response_max,
            "response_p95_ms": response_p95,
            "nonvalidated_rate_pct": nonvalidated_rate,
            "timeout_rate_pct": timeout_rate,
            "active_slaves": len(self.active_slaves),
            "bus_load_pct": bus_load,
            "slowest_slave": slowest_slave,
            "slowest_avg_ms": slowest_avg,
            "exception_summary": exception_summary,
            "total_bytes": self.total_bytes,
            "nonvalidated_bytes": self.nonvalidated_bytes,
        }

def u16_bytes(value):
    value = int(value) & 0xFFFF
    return bytes(((value >> 8) & 0xFF, value & 0xFF))

def corrupt_crc(frame):
    damaged = bytearray(frame)
    if len(damaged) >= 2:
```

MBToolkit v3.5/mb_protocol.py — lines 881-920

```
881         damaged[-1] ^= 0xFF
882     return bytes(damaged)
883 
884 def build_simulation_transaction(slave, fc, rng):
885     """
886     Build one syntactically valid Modbus request/response pair.
887 
888     Simulation rules:
889     - FC01..FC04 only.
890     - Address random 1..32.
891     - Qty random 1..8.
892     """
893     slave = int(slave)
894     fc = int(fc)
895     start_addr = rng.randint(1, 32)
896     qty = rng.randint(1, 8)
897 
898     if fc in (1, 2):
899         request = append_crc(
900             bytes((slave, fc)) + u16_bytes(start_addr) + u16_bytes(qty)
901         )
902 
903         byte_count = (qty + 7) // 8
904         data = bytes(rng.randrange(256) for _ in range(byte_count))
905         response = append_crc(bytes((slave, fc, byte_count)) + data)
906 
907     elif fc in (3, 4):
908         request = append_crc(
909             bytes((slave, fc)) + u16_bytes(start_addr) + u16_bytes(qty)
910         )
911 
912         values = [rng.randint(0, 65535) for _ in range(qty)]
913         payload = b"".join(u16_bytes(value) for value in values)
914         response = append_crc(
915             bytes((slave, fc, len(payload))) + payload
916         )
917 
918     else:
919         raise ValueError(f"Unsupported simulation FC: {fc}")
920 
```

MBToolkit v3.5/mb_protocol.py — lines 921-960

```
921     return {
922         "slave": slave,
923         "fc": fc,
924         "address": start_addr,
925         "qty": qty,
926         "request": request,
927         "response": response,
928     }
929 
930 def build_random_simulation_plan(rng=None):
931     """
932     Build exactly four simulation transactions:
933     FC01, FC02, FC03 and FC04, one each.
934 
935     Four unique random Slave IDs are used, one per transaction.
936     """
937     if rng is None:
938         rng = random.Random()
939 
940     slaves = rng.sample(range(1, 248), 4)
941 
942     plan = []
943     for slave, fc in zip(slaves, SIMULATION_FUNCTION_CODES):
944         tx = build_simulation_transaction(slave, fc, rng)
945         tx["outcome"] = "response"
946         tx["reply"] = tx["response"]
947         plan.append(tx)
948 
949     return {
950         "slaves": slaves,
951         "transactions": plan,
952     }
953 
954 
955 
956 def build_anomaly_simulation_plan(rng=None):
957     """
958     Build a deterministic set of anomaly categories for UI colour preview.
959 
960     Used whenever debug simulation is executed. "Realçar resultados" controls
```

MBToolkit v3.5/mb_protocol.py — lines 961-1000

```
    only whether these rows receive colours; it does not change the generated
    scenario.

    It demonstrates:
      - normal response
      - slow response
      - Modbus exception
      - CRC error
      - timeout
      - RAW/UNPARSED bytes
    """
    if rng is None:
        rng = random.Random()

    slaves = rng.sample(range(1, 248), 5)

    normal = build_simulation_transaction(slaves[0], 1, rng)
    normal["outcome"] = "response"
    normal["reply"] = normal["response"]

    slow = build_simulation_transaction(slaves[1], 2, rng)
    slow["outcome"] = "slow"
    slow["reply"] = slow["response"]

    exception = build_simulation_transaction(slaves[2], 3, rng)
    exception["outcome"] = "exception"
    exception["reply"] = append_crc(
        bytes((slaves[2], 0x83, 0x02))
    )

    crc_error = build_simulation_transaction(slaves[3], 4, rng)
    crc_error["outcome"] = "crc"
    crc_error["reply"] = corrupt_crc(crc_error["response"])

    timeout = build_simulation_transaction(slaves[4], 3, rng)
    timeout["outcome"] = "timeout"
    timeout["reply"] = None

    return {
        "slaves": slaves,
```

MBToolkit v3.5/mb_protocol.py — lines 1001-1007

```
        "transactions": [normal, slow, exception, crc_error, timeout],
        # Four bytes are deliberately too short to be accepted as the FC=0xFF
        # exception frame candidate, producing a visible RAW/UNPARSED row.
        "raw_noise": bytes((0x00, 0x00, 0x00, 0x00)),
        "slow_delay_seconds": (SLOW_RESPONSE_THRESHOLD_MS / 1000.0) + 0.15,
    }

```

MBToolkit v3.5/mb_capture.py

913 source lines.

MBToolkit v3.5/mb_capture.py — lines 1-40

```python
#!/usr/bin/env python3
"""Serial-port management, passive capture, simulation and event processing."""

import queue
import random
import threading
import time
import traceback
from datetime import datetime

import tkinter as tk
from tkinter import messagebox

from mb_config import (
    APP_NAME,
    APP_VERSION,
    MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
    MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
    SNIFFER_SPLIT_FRAME_HOLD_MS,
    UI_QUEUE_BATCH_SIZE,
    UI_QUEUE_IDLE_INTERVAL_MS,
    UI_QUEUE_BUSY_INTERVAL_MS,
    debug_sim,
)
from mb_protocol import (
    parse_burst,
    burst_may_continue,
    build_anomaly_simulation_plan,
    modbus_rtu_interframe_gap_seconds,
)
from mb_slave_finder import cancel_serial_io

class CaptureMixin:
    """Methods for serial capture, COM management and simulation."""

    def set_sniffer_refresh_state(self, state):
        """Apply the same state to both Sniffer COM refresh buttons."""
        for name in ("refresh_a_btn", "refresh_b_btn"):
            button = getattr(self, name, None)
            if button is not None:
```

MBToolkit v3.5/mb_capture.py — lines 41-80

```
41                 button.configure(state=state)
42
43     def update_mode_ui(self):
44         mode = self.mode_var.get()
45         dual = mode == "RS232 dual RX"
46
47         state = "readonly" if dual else "disabled"
48         self.port_b_combo.configure(state=state)
49
50         if dual:
51             self.mode_help_var.set(
52                 self.tr("RS232 dual RX: COM A escuta uma direção; COM B escuta a direção oposta.")
53             )
54         elif mode == "RS232 single RX":
55             self.mode_help_var.set(
56                 self.tr("RS232 single RX: uma COM escuta apenas uma direção da ligação RS232.")
57             )
58         else:
59             self.mode_help_var.set(
60                 self.tr("RS485 2-wire: uma COM consegue observar pedidos e respostas no mesmo par diferencial.")
61             )
62
63         self.update_com_status()
64
65     def update_gap_state(self):
66         self.gap_entry.configure(
67             state="normal" if self.gap_mode_var.get() == "Manual" else "disabled"
68         )
69
70     def get_serial_modules(self):
71         try:
72             import serial
73             from serial.tools import list_ports
74             return serial, list_ports
75         except ImportError:
76             return None, None
77
78     def refresh_ports(self):
79         _, list_ports = self.get_serial_modules()
80         ports = [p.device for p in list_ports.comports()] if list_ports else []
```

MBToolkit v3.5/mb_capture.py — lines 81-120

```
81
82          self.port_a_combo["values"] = ports
83          self.port_b_combo["values"] = ports
84
85          if ports:
86              if self.port_a_var.get() not in ports:
87                  self.port_a_var.set(ports[0])
88              if len(ports) > 1:
89                  if self.port_b_var.get() not in ports or self.port_b_var.get() == self.port_a_var.get():
90                      self.port_b_var.set(ports[1])
91              elif not self.port_b_var.get():
92                  self.port_b_var.set("")
93          else:
94              self.port_a_var.set("")
95              self.port_b_var.set("")
96
97          if hasattr(self, "discovery_port_combo"):
98              self.discovery_port_combo["values"] = ports
99              current_discovery = self.discovery_port_var.get().strip()
100             if ports:
101                 if current_discovery not in ports:
102                     self.discovery_port_var.set(ports[0])
103             else:
104                 self.discovery_port_var.set("")
105             self.update_discovery_start_state()
106
107         if hasattr(self, "master_port_combo"):
108             self.master_port_combo["values"] = ports
109             if ports:
110                 if self.master_port_var.get().strip() not in ports:
111                     self.master_port_var.set(ports[0])
112             elif not getattr(self, "master_active", False):
113                 self.master_port_var.set("")
114
115         if hasattr(self, "slave_port_combo"):
116             self.slave_port_combo["values"] = ports
117             if ports:
118                 if self.slave_port_var.get().strip() not in ports:
119                     self.slave_port_var.set(ports[0])
120             elif not getattr(self, "slave_active", False):
```

MBToolkit v3.5/mb_capture.py — lines 121-160

```
121                 self.slave_port_var.set("")
122
123         self.update_com_status()
124
125     def detected_ports(self):
126         """Return the set of COM device names currently reported by Windows/pyserial."""
127         _, list_ports = self.get_serial_modules()
128         if list_ports is None:
129             return set()
130         try:
131             return {p.device for p in list_ports.comports()}
132         except Exception:
133             return set()
134
135     def port_status_text(self, label, port, enabled=True):
136         if not enabled:
137             return f"{label} - {self.tr('Não Utilizada')}"
138         if not port:
139             return f"{label} - {self.tr('Não selecionada')}"
140
141         ser = self.serial_by_port.get(port)
142         if ser is not None:
143             try:
144                 if ser.is_open:
145                     return f"{label} - {self.tr('Aberta')}"
146             except Exception:
147                 pass
148
149         if port in self.detected_ports():
150             return f"{label} - {self.tr('Detetada')}"
151         return f"{label} - {self.tr('Não detetada')}"
152
153     def update_com_status(self):
154         """
155         Update COM status without opening/probing a port.
156
157         Status means:
158           - Aberta: currently opened by MBToolkit
159           - Detetada: reported by Windows/pyserial, but not opened by MBToolkit
160           - Não detetada: selected name is no longer reported by the OS
```

MBToolkit v3.5/mb_capture.py — lines 161-200

```
        """
        if not hasattr(self, "port_a_status_var"):
            return

        mode = self.mode_var.get() if hasattr(self, "mode_var") else "RS485 2-wire"
        dual = mode == "RS232 dual RX"
        self.port_a_status_var.set(
            self.port_status_text("COM A", self.port_a_var.get().strip(), True)
        )
        self.port_b_status_var.set(
            self.port_status_text("COM B", self.port_b_var.get().strip(), dual)
        )

    def poll_com_status(self):
        if self._closing:
            return
        try:
            self.update_com_status()
            if hasattr(self, "stat_requests_var"):
                self.update_stats_labels()
        finally:
            if not self._closing:
                try:
                    self.after(1000, self.poll_com_status)
                except tk.TclError:
                    pass

    def selected_port_plan(self):
        """Return [(channel, port), ...] for the currently selected physical mode."""
        mode = self.mode_var.get()
        port_a = self.port_a_var.get().strip()
        port_b = self.port_b_var.get().strip()

        if not port_a:
            raise ValueError(self.tr("Seleciona a COM A."))

        plan = [("BUS" if mode == "RS485 2-wire" else "A→B", port_a)]

        if mode == "RS232 dual RX":
            if not port_b:
```

MBToolkit v3.5/mb_capture.py — lines 201-240

```
                raise ValueError(self.tr("Seleciona também a COM B."))
            if port_a == port_b:
                raise ValueError(self.tr("COM A e COM B têm de ser diferentes."))
            plan.append(("B→A", port_b))

        return plan

    def open_selected_ports(self, serial, config):
        """Open the selected capture ports and return [(channel, serial_obj), ...]."""
        opened = []
        try:
            for channel, port in self.selected_port_plan():
                opened.append((channel, self.open_serial_port(serial, port, config)))
            return opened
        except Exception:
            for _, ser_obj in opened:
                try:
                    ser_obj.close()
                except Exception:
                    pass
            raise

    def install_opened_ports(self, opened):
        self.serial_objects = [ser_obj for _, ser_obj in opened]
        self.serial_by_port = {
            ser_obj.port: ser_obj for _, ser_obj in opened if getattr(ser_obj, "port", None)
        }
        self.update_com_status()

    def start_reader_threads(self, opened, gap_ms):
        self.reader_threads = []
        generation = self.reader_generation
        for channel, ser_obj in opened:
            t = threading.Thread(
                target=self.reader_loop,
                args=(ser_obj, gap_ms, channel, generation),
                daemon=True
            )
            self.reader_threads.append(t)
            t.start()
```

MBToolkit v3.5/mb_capture.py — lines 241-280

```
241
242     def restart_com_port(self):
243         """
244         Close and reopen the selected COM port(s).
245
246         During an active capture the capture continues in the same session/log.
247         When stopped, the selected COM port(s) are opened and closed once as a
248         restart/test of the application's serial handle.
249         """
250         if self.busy and not self.capture_active:
251             return
252         if self.restart_in_progress:
253             return
254
255         serial, _ = self.get_serial_modules()
256         if serial is None:
257             messagebox.showerror(self.tr("pyserial em falta"), self.tr("pyserial não está instalado."))
258             return
259
260         try:
261             config = self.serial_config(serial)
262             gap_ms = self.calculated_gap_ms()
263             self.selected_port_plan()
264         except Exception as exc:
265             messagebox.showerror(self.tr("Porta COM"), str(exc))
266             return
267
268         self.restart_in_progress = True
269         self.restart_btn.configure(state="disabled")
270
271         if not self.capture_active:
272             opened = []
273             try:
274                 opened = self.open_selected_ports(serial, config)
275                 self.install_opened_ports(opened)
276                 self.update_idletasks()
277             except Exception as exc:
278                 messagebox.showerror(self.tr("Reiniciar COM"), self.trf("Não foi possível reiniciar a porta COM:\n\n{error}", error=exc))
279             finally:
280                 for _, ser_obj in opened:
```

MBToolkit v3.5/mb_capture.py — lines 281-320

```
                    try:
                        ser_obj.close()
                    except Exception:
                        pass
                self.serial_objects = []
                self.serial_by_port = {}
                self.restart_in_progress = False
                self.restart_btn.configure(state="normal")
                self.refresh_ports()
            return

        # Active capture: stop only the current serial-reader generation.
        self.status_var.set(self.tr("● A reiniciar COM"))
        self.reader_generation += 1
        self.stop_event.set()

        for thread in list(self.reader_threads):
            try:
                thread.join(timeout=0.5)
            except Exception:
                pass

        for ser_obj in list(self.serial_objects):
            try:
                ser_obj.close()
            except Exception:
                pass

        self.reader_threads = []
        self.serial_objects = []
        self.serial_by_port = {}
        self.stop_event.clear()

        try:
            opened = self.open_selected_ports(serial, config)
        except Exception as exc:
            self.restart_in_progress = False
            self.capture_active = False
            self.busy = False
            self.stop_event.set()
```

MBToolkit v3.5/mb_capture.py — lines 321-360

```
            self.finish_capture_ui()
            self.update_com_status()
            messagebox.showerror(
                self.tr("Reiniciar COM"),
                self.trf("Não foi possível reabrir a porta COM. A captura foi parada.\n\n{error}", error=exc)
            )
            return

        self.install_opened_ports(opened)
        self.stop_event.clear()
        self.start_reader_threads(opened, gap_ms)
        self.queue_log_note(
            f"# COM restart: {datetime.now().isoformat(timespec='milliseconds')}\n"
        )
        self.status_var.set(self.tr("● A capturar"))
        self.restart_in_progress = False
        self.restart_btn.configure(state="normal")
        self.update_com_status()

    def set_simulation_button_state(self, state):
        """Change Simulation button state only when the optional button exists."""
        if self.sim_btn is not None:
            self.sim_btn.configure(state=state)

    def selected_pending_timeout_seconds(self):
        try:
            value = float(self.pending_timeout_var.get().strip().replace(",", "."))
        except ValueError:
            raise ValueError(self.tr("Pending timeout inválido."))

        if not (
            MIN_PENDING_REQUEST_TIMEOUT_SECONDS
            <= value
            <= MAX_PENDING_REQUEST_TIMEOUT_SECONDS
        ):
            raise ValueError(
                self.trf(
                    "Pending timeout deve estar entre {minimum:g} e {maximum:g} segundos.",
                    minimum=MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
                    maximum=MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
```

MBToolkit v3.5/mb_capture.py — lines 361-400

```
                )
            )

        return value

    def calculated_gap_ms(self):
        if self.gap_mode_var.get() == "Manual":
            value = float(self.gap_ms_var.get().replace(",", "."))
            if value <= 0:
                raise ValueError(self.tr("Frame gap manual inválido."))
            return value

        baud = int(self.baud_var.get())
        data_bits = int(self.data_var.get())
        parity_bits = 0 if self.parity_var.get() == "None" else 1
        stop_bits = float(self.stop_var.get())
        bits_per_char = 1 + data_bits + parity_bits + stop_bits
        return modbus_rtu_interframe_gap_seconds(baud, bits_per_char) * 1000.0

    def serial_config(self, serial):
        byte_map = {
            5: serial.FIVEBITS, 6: serial.SIXBITS,
            7: serial.SEVENBITS, 8: serial.EIGHTBITS,
        }
        parity_map = {
            "None": serial.PARITY_NONE,
            "Even": serial.PARITY_EVEN,
            "Odd": serial.PARITY_ODD,
            "Mark": serial.PARITY_MARK,
            "Space": serial.PARITY_SPACE,
        }
        stop_map = {
            "1": serial.STOPBITS_ONE,
            "1.5": serial.STOPBITS_ONE_POINT_FIVE,
            "2": serial.STOPBITS_TWO,
        }
        return {
            "baudrate": int(self.baud_var.get()),
            "bytesize": byte_map[int(self.data_var.get())],
            "parity": parity_map[self.parity_var.get()],
```

MBToolkit v3.5/mb_capture.py — lines 401-440

```
            "stopbits": stop_map[self.stop_var.get()],
        }

    def open_serial_port(self, serial, port, config):
        ser = serial.Serial()
        ser.port = port
        ser.baudrate = config["baudrate"]
        ser.bytesize = config["bytesize"]
        ser.parity = config["parity"]
        ser.stopbits = config["stopbits"]
        ser.timeout = 0
        ser.write_timeout = 0
        ser.xonxoff = False
        ser.rtscts = False
        ser.dsrdtr = False

        try:
            ser.rts = False
            ser.dtr = False
        except Exception:
            pass

        ser.open()

        try:
            ser.rts = False
            ser.dtr = False
        except Exception:
            pass

        try:
            ser.reset_input_buffer()
        except Exception:
            pass

        return ser

    def start_capture(self):
        if self.busy:
            return
```

MBToolkit v3.5/mb_capture.py — lines 441-480

```
441
442         serial, _ = self.get_serial_modules()
443         if serial is None:
444             messagebox.showerror(
445                 self.tr("pyserial em falta"),
446                 self.tr("pyserial não está instalado.\n\nO build_exe.bat inclui-o automaticamente no EXE.")
447             )
448             return
449
450         mode = self.mode_var.get()
451         port_a = self.port_a_var.get().strip()
452         port_b = self.port_b_var.get().strip()
453
454         try:
455             gap_ms = self.calculated_gap_ms()
456             pending_timeout_s = self.selected_pending_timeout_seconds()
457             config = self.serial_config(serial)
458             opened = self.open_selected_ports(serial, config)
459         except Exception as exc:
460             messagebox.showerror(self.tr("Erro ao abrir porta"), str(exc))
461             self.update_com_status()
462             return
463
464         self.reader_generation += 1
465         self.install_opened_ports(opened)
466         self.reader_threads = []
467         self.stop_event.clear()
468         self.busy = True
469         self.capture_active = True
470         self.metrics.pending_timeout_seconds = pending_timeout_s
471         self.reset_session_metrics()
472
473         capture_started = datetime.now()
474         log_header = (
475             f"# {APP_NAME} v{APP_VERSION}\n"
476             f"# Start: {capture_started.isoformat(timespec='milliseconds')}\n"
477             f"# Mode: {mode}\n"
478             f"# COM A: {port_a}\n"
479             + (f"# COM B: {port_b}\n" if mode == "RS232 dual RX" else "")
480             + f"# Serial: baud={self.baud_var.get()} data={self.data_var.get()} "
```

MBToolkit v3.5/mb_capture.py — lines 481-520

```
481                 f"parity={self.parity_var.get()} stop={self.stop_var.get()} "
482                 f"frame_gap_ms={gap_ms:.3f}\n"
483             + f"# Pending timeout: {pending_timeout_s:g} s\n"
484             + "# Passive/read-only: no Serial.write() calls.\n#\n"
485         )
486         self.prepare_log_session("MBToolkit", capture_started, log_header)
487 
488         self.status_var.set(self.tr("● A capturar"))
489         self.start_btn.configure(state="disabled")
490         self.stop_btn.configure(state="normal")
491         self.restart_btn.configure(state="normal")
492         self.set_simulation_button_state("disabled")
493         self.set_sniffer_refresh_state("disabled")
494 
495         self.start_reader_threads(opened, gap_ms)
496         self.update_com_status()
497 
498     def reader_loop(self, ser, gap_ms, channel, generation):
499         gap_s = gap_ms / 1000.0
500         hold_s = max(gap_s, SNIFFER_SPLIT_FRAME_HOLD_MS / 1000.0)
501         poll_sleep = min(max(gap_s / 8.0, 0.0002), 0.001)
502 
503         buffer = bytearray()
504         buffer_wall = None
505         # (byte offset, epoch) of every read, so frames merged into one burst
506         # by USB latency keep their own arrival times.
507         chunk_times = []
508         last_rx = None
509         # Burst length already checked by burst_may_continue(); avoids
510         # re-parsing an unchanged buffer on every poll of the hold window.
511         hold_checked_len = None
512         hold_active = False
513 
514         def flush_buffer():
515             ts = (
516                 buffer_wall.strftime("%H:%M:%S.%f")[:-3]
517                 if buffer_wall else datetime.now().strftime("%H:%M:%S.%f")[:-3]
518             )
519             event_epoch = (
520                 buffer_wall.timestamp() if buffer_wall else time.time()
```

MBToolkit v3.5/mb_capture.py — lines 521-560

```
            )
            for row in parse_burst(
                bytes(buffer), ts, channel, event_epoch, chunk_times
            ):
                self.event_queue.put(("frame", row))

        try:
            while not self.stop_event.is_set():
                waiting = ser.in_waiting

                if waiting:
                    chunk = ser.read(waiting)
                    if chunk:
                        if not buffer:
                            buffer_wall = datetime.now()
                            chunk_times.append((0, buffer_wall.timestamp()))
                        else:
                            chunk_times.append((len(buffer), time.time()))
                        buffer.extend(chunk)
                        last_rx = time.perf_counter()

                elif buffer and last_rx is not None:
                    idle_s = time.perf_counter() - last_rx
                    if idle_s >= gap_s:
                        if idle_s < hold_s:
                            if hold_checked_len != len(buffer):
                                hold_checked_len = len(buffer)
                                hold_active = burst_may_continue(bytes(buffer))
                            if hold_active:
                                time.sleep(poll_sleep)
                                continue

                        flush_buffer()
                        buffer.clear()
                        chunk_times.clear()
                        buffer_wall = None
                        last_rx = None
                        hold_checked_len = None
                        hold_active = False

```

MBToolkit v3.5/mb_capture.py — lines 561-600

```
561                 time.sleep(poll_sleep)
562
563         except Exception as exc:
564             if not self.stop_event.is_set():
565                 self.event_queue.put(("error", (generation, channel, str(exc))))
566
567         finally:
568             if buffer:
569                 flush_buffer()
570
571             try:
572                 ser.close()
573             except Exception:
574                 pass
575             self.event_queue.put(("reader_stopped", (generation, channel)))
576
577     def stop_capture(self):
578         if self.capture_active:
579             self.status_var.set(self.tr("● A parar"))
580         self.stop_event.set()
581
582     def run_simulation(self):
583         if debug_sim != 1:
584             return
585
586         if self.busy:
587             return
588
589         try:
590             pending_timeout_s = self.selected_pending_timeout_seconds()
591         except Exception as exc:
592             messagebox.showerror(self.tr("Configuração inválida"), str(exc))
593             return
594
595         self.busy = True
596         self.metrics.pending_timeout_seconds = pending_timeout_s
597         self.reset_session_metrics()
598
599         # Simulation is GUI-only. Never create or write a .txt log.
600         self.close_log()
```

MBToolkit v3.5/mb_capture.py — lines 601-640

```
601
602         self.status_var.set(self.tr("● A simular"))
603         self.start_btn.configure(state="disabled")
604         self.restart_btn.configure(state="disabled")
605         self.set_simulation_button_state("disabled")
606         self.set_sniffer_refresh_state("disabled")
607
608         def worker():
609             try:
610                 # Simulation always exercises the complete diagnostic scenario.
611                 # "Realçar resultados" affects only Treeview row colouring; it
612                 # never changes which simulated frames are generated.
613                 rng = random.Random()
614                 plan = build_anomaly_simulation_plan(rng=rng)
615
616                 for tx in plan["transactions"]:
617                     time.sleep(rng.uniform(0.025, 0.085))
618
619                     request_now = datetime.now()
620                     request_ts = request_now.strftime("%H:%M:%S.%f")[:-3]
621
622                     request_channel = "SIM"
623
624                     for row in parse_burst(
625                         tx["request"], request_ts, request_channel,
626                         request_now.timestamp()
627                     ):
628                         self.event_queue.put(("frame", row))
629
630                     outcome = tx.get("outcome", "response")
631
632                     if outcome == "timeout":
633                         # Do not sleep for the user-configured Pending timeout.
634                         # The queue event expires this exact pending request
635                         # through SessionMetrics and marks the REQUEST row.
636                         self.event_queue.put((
637                             "simulation_force_timeout",
638                             (tx["slave"], tx["fc"]),
639                         ))
640                         continue
```

MBToolkit v3.5/mb_capture.py — lines 641-680

```
641
642                     if outcome == "slow":
643                         time.sleep(plan.get("slow_delay_seconds", 0.65))
644                     else:
645                         time.sleep(rng.uniform(0.018, 0.095))
646
647                     reply_now = datetime.now()
648                     reply_ts = reply_now.strftime("%H:%M:%S.%f")[:-3]
649                     reply_channel = "SIM"
650
651                     reply = tx.get("reply")
652                     if reply:
653                         for row in parse_burst(
654                             reply, reply_ts, reply_channel,
655                             reply_now.timestamp()
656                         ):
657                             self.event_queue.put(("frame", row))
658
659                 # One RAW row completes the diagnostic scenario. With anomaly
660                 # highlighting OFF it remains a normal, uncoloured RAW row;
661                 # with highlighting ON the same row receives the RAW colour.
662                 raw_now = datetime.now()
663                 raw_ts = raw_now.strftime("%H:%M:%S.%f")[:-3]
664                 raw_channel = "SIM"
665                 for row in parse_burst(
666                     plan["raw_noise"],
667                     raw_ts,
668                     raw_channel,
669                     raw_now.timestamp(),
670                 ):
671                     self.event_queue.put(("frame", row))
672
673                 self.event_queue.put(("simulation_done", None))
674
675             except Exception as exc:
676                 # Never leave the GUI permanently busy if the simulation worker
677                 # fails unexpectedly.
678                 self.event_queue.put((
679                     "simulation_error",
680                     f"{type(exc).__name__}: {exc}"
```

MBToolkit v3.5/mb_capture.py — lines 681-720

```
                ))

        threading.Thread(target=worker, daemon=True).start()

    def process_queue(self):
        if self._closing:
            return

        delay = UI_QUEUE_IDLE_INTERVAL_MS
        try:
            events = []
            for _ in range(UI_QUEUE_BATCH_SIZE):
                try:
                    events.append(self.event_queue.get_nowait())
                except queue.Empty:
                    break

            frame_batch = []

            def flush_frames():
                nonlocal frame_batch
                if frame_batch:
                    self.add_frame_rows_batch(frame_batch)
                    frame_batch = []

            for event, payload in events:
                if event == "frame":
                    frame_batch.append(payload)
                    continue

                flush_frames()

                if event == "error":
                    generation, channel, error_text = payload
                    if generation != self.reader_generation:
                        continue
                    # Stop all readers in this capture generation after any
                    # reader failure; never leave a half-alive dual-RX capture.
                    self.stop_event.set()
                    self.update_com_status()
```

MBToolkit v3.5/mb_capture.py — lines 721-760

```
                    messagebox.showerror(
                        self.tr("Erro de captura"), f"{channel}: {error_text}"
                    )

                elif event == "reader_stopped":
                    generation, _channel = payload
                    if generation != self.reader_generation:
                        continue
                    if not any(t.is_alive() for t in self.reader_threads):
                        self.finish_capture_ui()

                elif event == "simulation_force_timeout":
                    slave, fc = payload
                    expired = self.metrics.force_timeout(slave, fc)
                    if expired:
                        self.mark_timed_out_records(expired, rerender=True)
                    self.update_stats_labels()

                elif event == "simulation_done":
                    self.busy = False
                    self.status_var.set(self.tr("● Simulação concluída"))
                    self.start_btn.configure(state="normal")
                    self.stop_btn.configure(state="disabled")
                    self.restart_btn.configure(state="normal")
                    self.set_simulation_button_state("normal")
                    self.set_sniffer_refresh_state("normal")
                    self.close_log()
                    self.update_com_status()

                elif event == "simulation_error":
                    self.busy = False
                    self.status_var.set(self.tr("● Erro na simulação"))
                    self.start_btn.configure(state="normal")
                    self.stop_btn.configure(state="disabled")
                    self.restart_btn.configure(state="normal")
                    self.set_simulation_button_state("normal")
                    self.set_sniffer_refresh_state("normal")
                    self.close_log()
                    self.update_com_status()
                    messagebox.showerror(self.tr("Erro na simulação"), str(payload))
```

MBToolkit v3.5/mb_capture.py — lines 761-800

```
761
762                 elif event == "discovery_status":
763                     self.discovery_status_var.set(str(payload))
764
765                 elif event == "discovery_progress":
766                     self.apply_discovery_progress(payload)
767
768                 elif event == "discovery_found":
769                     self.add_discovery_result(payload)
770
771                 elif event == "discovery_done":
772                     self.finish_discovery_ui(payload)
773
774                 elif event.startswith("master_"):
775                     self.handle_master_event(event, payload)
776
777                 elif event.startswith("slave_"):
778                     self.handle_slave_event(event, payload)
779
780                 elif event.startswith("client_"):
781                     self.client_tab.handle_master_event(
782                         "master_" + event[len("client_"):], payload)
783
784                 elif event.startswith("server_"):
785                     self.server_tab.handle_server_event(event, payload)
786
787             flush_frames()
788
789             if self.metrics.expire_pending(time.time()):
790                 expired = self.metrics.pop_expired_requests()
791                 if expired:
792                     self.mark_timed_out_records(expired, rerender=True)
793                 self.update_stats_labels()
794
795             if (
796                 self.capture_active
797                 and not self.restart_in_progress
798                 and self.stop_event.is_set()
799                 and self.reader_threads
800                 and not any(t.is_alive() for t in self.reader_threads)
```

MBToolkit v3.5/mb_capture.py — lines 801-840

```
                and self.stop_btn["state"] != "disabled"
            ):
                self.finish_capture_ui()

            delay = (
                UI_QUEUE_BUSY_INTERVAL_MS
                if len(events) >= UI_QUEUE_BATCH_SIZE
                else UI_QUEUE_IDLE_INTERVAL_MS
            )

        except tk.TclError:
            if self._closing:
                return
            traceback.print_exc()
            delay = UI_QUEUE_IDLE_INTERVAL_MS

        except Exception as exc:
            # One unexpected event/UI error must not permanently kill the
            # periodic queue callback and make the application appear frozen.
            traceback.print_exc()
            if not self._process_queue_error_shown and not self._closing:
                self._process_queue_error_shown = True
                try:
                    messagebox.showerror(
                        self.tr("Erro interno"),
                        self.trf(
                            "Ocorreu um erro ao processar eventos da interface. O processamento continuará.\n\n{error}",
                            error=f"{type(exc).__name__}: {exc}",
                        )
                    )
                except Exception:
                    pass
            delay = UI_QUEUE_IDLE_INTERVAL_MS

        finally:
            if not self._closing:
                try:
                    self.after(delay, self.process_queue)
                except tk.TclError:
                    pass
```

MBToolkit v3.5/mb_capture.py — lines 841-880

```
841
842     def on_close(self):
843         if self._closing:
844             return
845
846         # A changed Master / Slave workspace may be saved first; "Cancel"
847         # keeps the application open.
848         for tab, name in ((self, "master_confirm_close"), (self, "slave_confirm_close"),
849                           (getattr(self, "client_tab", None), "master_confirm_close"),
850                           (getattr(self, "server_tab", None), "slave_confirm_close")):
851             confirm = getattr(tab, name, None) if tab is not None else None
852             if callable(confirm) and not confirm():
853                 return
854
855         try:
856             saver = getattr(self, "save_current_ui_settings", None)
857             if callable(saver):
858                 saver()
859         except Exception:
860             pass
861
862         self._closing = True
863         self.busy = False
864         self.capture_active = False
865         self.stop_event.set()
866         self.discovery_stop_event.set()
867
868         # Cancel driver I/O before joining workers so shutdown is not held up
869         # by a serial read/write timeout.
870         cancel_serial_io(self.discovery_serial)
871         for tab, name in ((self, "master_shutdown"), (self, "slave_shutdown"),
872                           (getattr(self, "client_tab", None), "master_shutdown"),
873                           (getattr(self, "server_tab", None), "slave_shutdown")):
874             stopper = getattr(tab, name, None) if tab is not None else None
875             if callable(stopper):
876                 stopper()
877
878         if self.discovery_thread is not None:
879             try:
880                 if self.discovery_thread.is_alive():
```

MBToolkit v3.5/mb_capture.py — lines 881-913

```
                    self.discovery_thread.join(timeout=0.8)
            except Exception:
                pass

        if self.discovery_serial is not None:
            try:
                self.discovery_serial.close()
            except Exception:
                pass
        self.discovery_serial = None

        for ser in list(self.serial_objects):
            cancel_serial_io(ser)

        for thread in list(self.reader_threads):
            try:
                if thread.is_alive():
                    thread.join(timeout=0.5)
            except Exception:
                pass

        for ser in list(self.serial_objects):
            try:
                ser.close()
            except Exception:
                pass

        self.close_log()
        try:
            self.destroy()
        except tk.TclError:
            pass

```

MBToolkit v3.5/mb_diagnostics.py

286 source lines.

MBToolkit v3.5/mb_diagnostics.py — lines 1-40

```python
#!/usr/bin/env python3
"""Diagnostic helpers for MBToolkit traffic inspection and anomaly display."""

from mb_config import MODBUS_EXCEPTION_CODES, SLOW_RESPONSE_THRESHOLD_MS
from mb_protocol import crc16_modbus, read_bits_from_response
from mb_i18n import LANGUAGE_PORTUGUESE, translate_details, translate_text


FUNCTION_NAMES = {
    0x01: "Read Coils",
    0x02: "Read Discrete Inputs",
    0x03: "Read Holding Registers",
    0x04: "Read Input Registers",
    0x05: "Write Single Coil",
    0x06: "Write Single Register",
    0x07: "Read Exception Status",
    0x08: "Diagnostics",
    0x0F: "Write Multiple Coils",
    0x10: "Write Multiple Registers",
    0x16: "Mask Write Register",
    0x17: "Read/Write Multiple Registers",
    0x2B: "Encapsulated Interface Transport",
}


def raw_hex_to_bytes(raw):
    try:
        return bytes.fromhex(str(raw or ""))
    except (TypeError, ValueError):
        return b""


def fc_number(record):
    text = str(record.get("fc", "") or "").strip()
    try:
        value = int(text, 16)
    except (TypeError, ValueError):
        return None
    if record.get("type") == "EXCEPTION":
        value &= 0x7F
```

MBToolkit v3.5/mb_diagnostics.py — lines 41-80

```
41      return value
42
43
44  def function_label(record, language=LANGUAGE_PORTUGUESE):
45      fc = fc_number(record)
46      if fc is None:
47          return "—"
48      name = FUNCTION_NAMES.get(fc, translate_text("Desconhecida/outra", language))
49      return f"0x{fc:02X} — {name}"
50
51
52  def crc_fields(record):
53      if str(record.get("type", "")).startswith("RAW"):
54          return None, None, str(record.get("crc", "?"))
55
56      frame = raw_hex_to_bytes(record.get("raw"))
57      if len(frame) < 4:
58          return None, None, str(record.get("crc", "?"))
59
60      received = frame[-2] | (frame[-1] << 8)
61      calculated = crc16_modbus(frame[:-2])
62      return received, calculated, ("OK" if received == calculated else "ERROR")
63
64
65  def response_payload(record, language=LANGUAGE_PORTUGUESE):
66      """Return (byte_count, data_hex, decoded_data) where available."""
67      frame = raw_hex_to_bytes(record.get("raw"))
68      if len(frame) < 3:
69          return None, "", ""
70
71      kind = str(record.get("type", ""))
72      fc_raw = frame[1] if len(frame) >= 2 else 0
73      fc = fc_raw & 0x7F
74
75      if kind == "EXCEPTION":
76          code = frame[2] if len(frame) >= 3 else None
77          if code is None:
78              return None, "", "Exception"
79          info = MODBUS_EXCEPTION_CODES.get(code)
80          name = info[0] if info else "Unknown/Reserved Exception"
```

MBToolkit v3.5/mb_diagnostics.py — lines 81-120

```
        return None, "", f"0x{code:02X} — {name}"

    if kind == "RESPONSE" and fc in (1, 2, 3, 4) and len(frame) >= 5:
        byte_count = frame[2]
        data = frame[3:3 + byte_count]
        data_hex = " ".join(f"{b:02X}" for b in data)
        decoded = ""
        start = record.get("request_pdu_address")
        if fc in (3, 4) and byte_count % 2 == 0 and len(data) == byte_count:
            regs = [
                (data[i] << 8) | data[i + 1]
                for i in range(0, len(data), 2)
            ]
            if start is not None:
                # The matched request gives the address of every value.
                pairs = ", ".join(
                    f"{start + i}={value}" for i, value in enumerate(regs)
                )
                decoded = translate_details(
                    f"Registos (PDU Address=valor): {pairs}", language
                )
            else:
                decoded = translate_details(f"Registos={regs}", language)
        elif fc in (1, 2) and len(data) == byte_count:
            bits = read_bits_from_response(frame, record.get("request_qty"))
            if start is not None:
                pairs = ", ".join(
                    f"{start + i}={value}" for i, value in enumerate(bits)
                )
                decoded = translate_details(
                    f"Bits (PDU Address=valor): {pairs}", language
                )
            else:
                decoded = "Bits=" + "".join(str(bit) for bit in bits)
        return byte_count, data_hex, decoded

    if kind == "REQUEST" and fc in (15, 16) and len(frame) >= 9:
        byte_count = frame[6]
        data = frame[7:7 + byte_count]
        return byte_count, " ".join(f"{b:02X}" for b in data), ""
```

MBToolkit v3.5/mb_diagnostics.py — lines 121-160

```
121
122     if kind == "REQUEST" and fc == 23 and len(frame) >= 13:
123         byte_count = frame[10]
124         data = frame[11:11 + byte_count]
125         return byte_count, " ".join(f"{b:02X}" for b in data), ""
126
127     return record.get("byte_count"), str(record.get("data_hex", "") or ""), ""
128
129
130 def anomaly_kind(record, threshold_ms=SLOW_RESPONSE_THRESHOLD_MS):
131     """
132     Return one anomaly class or None.
133
134     Priority avoids ambiguous colouring when one frame satisfies several
135     conditions. CRC is strongest because the frame itself is invalid.
136     """
137     if str(record.get("crc", "")) == "ERROR":
138         return "crc"
139     if bool(record.get("timed_out")):
140         return "timeout"
141     if str(record.get("type", "")) == "EXCEPTION":
142         return "exception"
143     if str(record.get("type", "")).startswith("RAW"):
144         return "raw"
145
146     try:
147         response_ms = float(record.get("response_text", ""))
148     except (TypeError, ValueError):
149         response_ms = None
150
151     if response_ms is not None and response_ms >= float(threshold_ms):
152         return "slow"
153     return None
154
155
156 RESULT_LABELS = {
157     "success": ("✓ Sucesso", "Verde"),
158     "slow": ("⚠ Resposta lenta", "Amarelo"),
159     "exception": ("Exception", "Vermelho"),
160     "crc": ("× CRC Error", "Vermelho"),
```

MBToolkit v3.5/mb_diagnostics.py — lines 161-200

```
    "timeout": ("⚠ Timeout", "Laranja"),
    "raw": ("RAW", "Roxo"),
    "pending": ("Pendente / sem resultado", "Neutro"),
    "neutral": ("Sem classificação", "Neutro"),
}


def result_info(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
    """
    Return (result_key, human_label, colour_name) for one traffic record.

    The semantic result is always available to the Frame Inspector. Whether
    the table/Inspector actually uses the category colour is controlled by
    "Realçar resultados".
    """
    if not record:
        return "neutral", "—", translate_text("Neutro", language)

    kind = anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS)
    if kind:
        label, colour = RESULT_LABELS[kind]
        if kind == "raw":
            # Preserve the exact parser state in the Inspector.
            row_type = str(record.get("type", "RAW") or "RAW")
            label = row_type
        return kind, translate_text(label, language), translate_text(colour, language)

    row_type = str(record.get("type", ""))

    if (
        row_type == "RESPONSE"
        and record.get("crc") == "OK"
        and record.get("matched")
    ):
        label, colour = RESULT_LABELS["success"]
        return "success", translate_text(label, language), translate_text(colour, language)

    if row_type == "REQUEST":
        if successful_request and not record.get("timed_out"):
            label, colour = RESULT_LABELS["success"]
```

MBToolkit v3.5/mb_diagnostics.py — lines 201-240

```
201             return "success", translate_text(label, language), translate_text(colour, language)
202
203         label, colour = RESULT_LABELS["pending"]
204         return "pending", translate_text(label, language), translate_text(colour, language)
205
206     label, colour = RESULT_LABELS["neutral"]
207     return "neutral", translate_text(label, language), translate_text(colour, language)
208
209
210 def inspector_values(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
211     if not record:
212         return {
213             "slave": "Slave: —",
214             "type": f"{translate_text('Tipo', language)}: —",
215             "function": "Function: —",
216             "response": "Resp.: —",
217             "result": f"{translate_text('Resultado', language)}: —",
218             "result_key": "neutral",
219             "pdu": "PDU Address: —",
220             "one_based": "1-based: —",
221             "qty": "Qty: —",
222             "byte_count": "ByteCount: —",
223             "crc": "CRC: —",
224             "data": f"{translate_text('Dados', language)}: —",
225             "raw": "",
226         }
227
228     received, calculated, crc_state = crc_fields(record)
229     if received is None:
230         crc_text = f"CRC: {crc_state or '—'}"
231     else:
232         crc_text = (
233             (f"CRC: received 0x{received:04X} / calculated 0x{calculated:04X} — {crc_state}" if language == "English" else f"CRC: recebido 0x{received:04X} / calculado 0x{calcul
    ated:04X} — {crc_state}")
234         )
235
236     byte_count, data_hex, decoded = response_payload(record, language=language)
237     payload = decoded or data_hex or str(record.get("details", "") or "—")
238
239     response = str(record.get("response_text", "") or "")
240     if response:
```

MBToolkit v3.5/mb_diagnostics.py — lines 241-280

```
        response += " ms"

    timeout_suffix = " — TIMEOUT" if record.get("timed_out") else ""
    result_key, result_label, _result_colour = result_info(
        record,
        successful_request=successful_request,
        language=language,
    )

    return {
        "slave": f"Slave: {record.get('slave') or '—'}",
        "type": f"{translate_text('Tipo', language)}: {record.get('type') or '—'}{timeout_suffix}",
        "function": f"Function: {function_label(record, language=language)}",
        "response": f"Resp.: {response or '—'}",
        "result": f"{translate_text('Resultado', language)}: {result_label}",
        "result_key": result_key,
        "pdu": f"PDU Address: {record.get('pdu_address') if record.get('pdu_address') is not None else '—'}",
        "one_based": f"1-based: {record.get('one_based') if record.get('one_based') is not None else '—'}",
        "qty": f"Qty: {record.get('qty') if record.get('qty') is not None else '—'}",
        "byte_count": f"ByteCount: {byte_count if byte_count is not None else '—'}",
        "crc": crc_text,
        "data": f"{translate_text('Dados', language)}: {translate_details(payload, language)}",
        "raw": str(record.get("raw", "") or ""),
    }


def decoded_record_text(record, successful_request=False, language=LANGUAGE_PORTUGUESE):
    values = inspector_values(
        record,
        successful_request=successful_request,
        language=language,
    )
    return "\n".join((
        values["slave"],
        values["type"],
        values["function"],
        values["response"],
        values["result"],
        values["pdu"],
        values["one_based"],
```

MBToolkit v3.5/mb_diagnostics.py — lines 281-286

```
        values["qty"],
        values["byte_count"],
        values["crc"],
        values["data"],
        f"Raw: {values['raw']}",
    ))
```

MBToolkit v3.5/mb_view.py

1881 source lines.

MBToolkit v3.5/mb_view.py — lines 1-40

```python
#!/usr/bin/env python3
"""Traffic rendering, filtering, Raw Hex view, statistics and log handling."""

import csv
import math
import os
import subprocess
import sys
import time
from datetime import datetime

import tkinter as tk
from tkinter import filedialog, messagebox

from mb_config import (
    MAX_UI_FRAMES,
    MAX_RAW_TEXT_LINES,
    UI_PRUNE_CHUNK_FRAMES,
    RAW_TEXT_PRUNE_CHUNK_LINES,
    SLOW_RESPONSE_THRESHOLD_MS,
    get_application_directory,
    get_logs_folder,
)
from mb_diagnostics import (
    anomaly_kind,
    decoded_record_text,
    inspector_values,
)
from mb_i18n import canonical_ui_text, translate_details
from mb_protocol import read_bits_details

# Index of "Abrir no Master" in the Traffic context menu (built in mb_gui.py).
TRAFFIC_MENU_OPEN_IN_MASTER = 8
TRAFFIC_MENU_OPEN_IN_SLAVE = 9


class ViewMixin:
    """Methods for rendering, filtering, statistics and logging."""

    def reset_session_metrics(self):
```

MBToolkit v3.5/mb_view.py — lines 41-80

```
        self.metrics.reset()
        self.update_stats_labels()

    def update_stats_labels(self):
        self.stat_requests_var.set(f"{self.tr('Pedidos')}: {self.metrics.requests}")
        self.stat_responses_var.set(f"{self.tr('Respostas')}: {self.metrics.responses}")
        self.stat_pending_var.set(f"{self.tr('Pendentes')}: {self.metrics.pending_count()}")
        self.stat_timeouts_var.set(f"Timeouts: {self.metrics.timeouts}")
        self.stat_crc_var.set(f"{self.tr('Erros de CRC')}: {self.metrics.crc_errors}")
        self.stat_exceptions_var.set(f"{self.tr('Exceções')}: {self.metrics.exceptions}")
        self.stat_raw_var.set(f"RAW: {self.metrics.raw}")

        try:
            health_now = (
                time.time()
                if getattr(self, "capture_active", False) or getattr(self, "busy", False)
                else self.metrics.last_event_time
            )
            summary = self.metrics.health_summary(
                baud=self.baud_var.get(),
                data_bits=self.data_var.get(),
                parity=self.parity_var.get(),
                stop_bits=self.stop_var.get(),
                now=health_now,
            )
        except Exception:
            summary = self.metrics.health_summary()

        def ms(value):
            return "–" if value is None else f"{value:.1f} ms"

        if hasattr(self, "stat_slaves_var"):
            self.stat_slaves_var.set(f"Slaves: {summary['active_slaves']}")

        mappings = {
            "health_requests_var": str(self.metrics.requests),
            "health_responses_var": str(self.metrics.responses),
            "health_crc_count_var": str(self.metrics.crc_errors),
            "health_timeout_count_var": str(self.metrics.timeouts),
            "health_exception_count_var": str(self.metrics.exceptions),
```

MBToolkit v3.5/mb_view.py — lines 81-120

```
            "health_req_rate_var": f"{summary['request_rate']:.2f} req/s",
            "health_resp_avg_var": ms(summary["response_avg_ms"]),
            "health_resp_min_var": ms(summary["response_min_ms"]),
            "health_resp_max_var": ms(summary["response_max_ms"]),
            "health_p95_var": ms(summary["response_p95_ms"]),
            "health_nonvalidated_rate_var": f"{summary['nonvalidated_rate_pct']:.2f} %",
            "health_timeout_rate_var": f"{summary['timeout_rate_pct']:.2f} %",
            "health_bus_load_var": f"{summary['bus_load_pct']:.2f} %",
            "health_slaves_var": str(summary["active_slaves"]),
            "health_total_bytes_var": str(summary["total_bytes"]),
            "health_exception_slaves_var": summary["exception_summary"],
        }

        slowest_slave = summary["slowest_slave"]
        if slowest_slave is None:
            mappings["health_slowest_var"] = "—"
        else:
            mappings["health_slowest_var"] = (
                self.trf(
                    "Slave {slave} — média {average:.1f} ms",
                    slave=slowest_slave,
                    average=summary["slowest_avg_ms"],
                )
            )

        for attr, value in mappings.items():
            var = getattr(self, attr, None)
            if var is not None:
                var.set(value)

    # ------------------------------------------------------------------
    # Bus activity, filters, anomaly highlighting and Frame Inspector.
    # ------------------------------------------------------------------
    def pulse_bus_activity(self):
        var = getattr(self, "bus_activity_var", None)
        if var is None:
            return

        var.set("BUS ● RX")
        previous = getattr(self, "_bus_activity_after_id", None)
```

MBToolkit v3.5/mb_view.py — lines 121-160

```
        if previous is not None:
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass

        try:
            self._bus_activity_after_id = self.after(
                180,
                self._clear_bus_activity_pulse,
            )
        except tk.TclError:
            self._bus_activity_after_id = None

    def _clear_bus_activity_pulse(self):
        self._bus_activity_after_id = None
        var = getattr(self, "bus_activity_var", None)
        if var is not None:
            var.set("BUS ○")

    def schedule_advanced_filter_refresh(self, *_args):
        previous = getattr(self, "_advanced_filter_after_id", None)
        if previous is not None:
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass
        try:
            self._advanced_filter_after_id = self.after(
                120,
                self.render_filtered_view,
            )
        except tk.TclError:
            self._advanced_filter_after_id = None

    def clear_advanced_filters(self):
        pending = getattr(self, "_advanced_filter_after_id", None)
        if pending is not None:
            try:
                self.after_cancel(pending)
```

MBToolkit v3.5/mb_view.py — lines 161-200

```
            except tk.TclError:
                pass
            self._advanced_filter_after_id = None

        self.advanced_filter_type_var.set(self.tr("Todos"))
        self.advanced_filter_fc_var.set("")
        self.advanced_filter_response_ms_var.set("")
        self.advanced_filter_text_var.set("")
        self._transaction_filter_seqs = None

        self.slave_filter_all_mode = True
        for var in self.slave_filter_vars.values():
            var.set(True)
        self.update_slave_heading()

        self.render_filtered_view()

    @staticmethod
    def normalized_fc_filter(value):
        text = str(value or "").strip().upper()
        if not text or text == "TODOS":
            return ""
        if text.startswith("0X"):
            text = text[2:]
        try:
            number = int(text, 16)
        except ValueError:
            return "__INVALID__"
        if not 0 <= number <= 0xFF:
            return "__INVALID__"
        return f"{number:02X}"

    def record_passes_advanced_filters(self, record):
        transaction_filter = getattr(self, "_transaction_filter_seqs", None)
        if transaction_filter is not None:
            if record.get("seq") not in transaction_filter:
                return False

        kind = canonical_ui_text(self.advanced_filter_type_var.get())
        row_type = str(record.get("type", ""))
```

MBToolkit v3.5/mb_view.py — lines 201-240

```
201
202         if kind == "Requests" and row_type != "REQUEST":
203             return False
204         if kind == "Responses" and row_type != "RESPONSE":
205             return False
206         if kind == "Exceptions" and row_type != "EXCEPTION":
207             return False
208         if kind == "CRC errors" and record.get("crc") != "ERROR":
209             return False
210         if kind == "Timeouts" and not record.get("timed_out"):
211             return False
212         if kind == "RAW" and not row_type.startswith("RAW"):
213             return False
214
215         fc_filter = self.normalized_fc_filter(
216             self.advanced_filter_fc_var.get()
217         )
218         if fc_filter == "__INVALID__":
219             return False
220         if fc_filter:
221             record_fc_text = str(record.get("fc", "")).upper()
222             candidates = {record_fc_text}
223             if str(record.get("type", "")) == "EXCEPTION":
224                 try:
225                     candidates.add(f"{int(record_fc_text, 16) & 0x7F:02X}")
226                 except ValueError:
227                     pass
228             if fc_filter not in candidates:
229                 return False
230
231         response_limit = self.advanced_filter_response_ms_var.get().strip()
232         if response_limit:
233             try:
234                 minimum_ms = float(response_limit.replace(",", "."))
235                 response_ms = float(record.get("response_text", ""))
236             except (TypeError, ValueError):
237                 return False
238             if response_ms <= minimum_ms:
239                 return False
240
```

MBToolkit v3.5/mb_view.py — lines 241-280

```
241         query = self.advanced_filter_text_var.get().strip().casefold()
242         if query:
243             haystack = " ".join((
244                 str(record.get("details", "")),
245                 str(record.get("raw", "")),
246                 str(record.get("slave", "")),
247                 str(record.get("fc", "")),
248                 str(record.get("type", "")),
249                 str(record.get("channel", "")),
250             )).casefold()
251             if query not in haystack:
252                 return False
253 
254         return True
255 
256     def configure_anomaly_tags(self, dark=None):
257         if not hasattr(self, "tree"):
258             return
259 
260         if dark is None:
261             dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
262 
263         if dark:
264             palette = {
265                 "result_success": ("#20372a", "#a7e4b8"),
266                 "anomaly_exception": ("#3b2728", "#ffc0c5"),
267                 "anomaly_crc": ("#482427", "#ffb3ba"),
268                 "anomaly_timeout": ("#42351f", "#ffd27a"),
269                 "anomaly_slow": ("#3b3422", "#ffe19a"),
270                 "anomaly_raw": ("#30283c", "#d8c5ff"),
271             }
272         else:
273             palette = {
274                 "result_success": ("#e4f4e9", "#1f6b3a"),
275                 "anomaly_exception": ("#ffe8e9", "#8c1d24"),
276                 "anomaly_crc": ("#ffdfe1", "#8c1d24"),
277                 "anomaly_timeout": ("#fff0cf", "#774b00"),
278                 "anomaly_slow": ("#fff6dd", "#6b5200"),
279                 "anomaly_raw": ("#f0e9ff", "#5d3d8f"),
280             }
```

MBToolkit v3.5/mb_view.py — lines 281-320

```
281
282         for tag, (background, foreground) in palette.items():
283             self.tree.tag_configure(
284                 tag,
285                 background=background,
286                 foreground=foreground,
287             )
288
289     def anomaly_tag_for_record(self, record):
290         """
291         Return the optional result-highlight tag for one logical traffic row.
292
293         "Realçar resultados" is a presentation option only:
294         - diagnostic outcomes retain their existing anomaly colors;
295         - a successfully completed normal REQUEST/RESPONSE transaction is green;
296         - unmatched/pending requests remain neutral.
297         """
298         if not getattr(self, "highlight_anomalies_var", None):
299             return ""
300         if not self.highlight_anomalies_var.get():
301             return ""
302
303         kind = anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS)
304         if kind:
305             return f"anomaly_{kind}"
306
307         row_type = str(record.get("type", ""))
308
309         # A normal, CRC-valid matched RESPONSE is a successful result.
310         if (
311             row_type == "RESPONSE"
312             and record.get("crc") == "OK"
313             and record.get("matched")
314         ):
315             return "result_success"
316
317         # Once the normal response has arrived, also mark its REQUEST so the
318         # complete successful transaction pair reads as one result.
319         if (
320             row_type == "REQUEST"
```

MBToolkit v3.5/mb_view.py — lines 321-360

```
            and record.get("seq") in self._successful_request_seqs
            and not record.get("timed_out")
        ):
            return "result_success"

        return ""

    def on_highlight_anomalies_changed(self):
        self.configure_anomaly_tags()
        self.configure_inspector_result_styles()
        self.render_filtered_view()
        self.update_frame_inspector(self.selected_record())
        saver = getattr(self, "_schedule_ui_settings_save", None)
        if callable(saver):
            saver()

    def mark_timed_out_records(self, expired, rerender=True):
        if not expired:
            return 0

        seqs = {
            item.get("seq")
            for item in expired
            if item.get("seq") is not None
        }
        if not seqs:
            return 0

        changed_records = []
        for record in self._frame_history:
            if record.get("seq") in seqs and not record.get("timed_out"):
                record["timed_out"] = True
                changed_records.append(record)

        if changed_records and rerender:
            # The "Timeouts" filter changes which rows are visible, so only
            # then is a full rebuild needed. Otherwise a timeout changes just
            # the row colour: retag the already visible rows in place.
            if canonical_ui_text(self.advanced_filter_type_var.get()) == "Timeouts":
                self.render_filtered_view()
```

MBToolkit v3.5/mb_view.py — lines 361-400

```
            else:
                self.refresh_record_tags(changed_records)
        return len(changed_records)

    def refresh_record_tags(self, records):
        """Re-apply the result tag of visible rows without rebuilding the table."""
        for record in records:
            row_tag = self.anomaly_tag_for_record(record)
            for iid in self._tree_iids_by_seq.get(record.get("seq"), []):
                try:
                    self.tree.item(iid, tags=((row_tag,) if row_tag else ()))
                except tk.TclError:
                    pass

        selected = self.selected_record()
        if selected is not None and any(selected is record for record in records):
            self.update_frame_inspector(selected)

    def selected_record(self):
        selection = self.tree.selection()
        if not selection:
            return None
        return self._record_by_iid.get(selection[0])

    def update_frame_inspector(self, record=None):
        if not hasattr(self, "inspector_vars"):
            return

        successful_request = bool(
            record
            and record.get("type") == "REQUEST"
            and record.get("seq") in self._successful_request_seqs
        )
        values = inspector_values(
            record,
            successful_request=successful_request,
            language=self.current_language,
        )
        for key, var in self.inspector_vars.items():
            if key in values:
```

MBToolkit v3.5/mb_view.py — lines 401-440

```
                var.set(values[key])
        self.selected_raw_var.set(values.get("raw", ""))

        result_label = getattr(self, "inspector_result_label", None)
        if result_label is not None:
            style_key = values.get("result_key", "neutral")
            if not self.highlight_anomalies_var.get():
                style_key = "neutral"
            try:
                result_label.configure(
                    style=f"MBS.Result.{style_key}.TLabel"
                )
            except tk.TclError:
                pass

    def toggle_frame_inspector(self):
        if getattr(self, "inspector_collapsed", False):
            self.inspector_body.grid()
            self.inspector_collapsed = False
            self.inspector_toggle_btn.configure(text=self.tr("Recolher"))
        else:
            self.inspector_body.grid_remove()
            self.inspector_collapsed = True
            self.inspector_toggle_btn.configure(text=self.tr("Expandir"))

        scheduler = getattr(self, "_schedule_bus_health_layout_sync", None)
        if callable(scheduler):
            scheduler()

    def copy_to_clipboard(self, text):
        try:
            self.clipboard_clear()
            self.clipboard_append(str(text))
            self.update_idletasks()
        except tk.TclError:
            pass

    def copy_selected_raw(self):
        record = self.selected_record()
        if record:
```

MBToolkit v3.5/mb_view.py — lines 441-480

```
441             self.copy_to_clipboard(record.get("raw", ""))
442
443     def copy_selected_decoded(self):
444         record = self.selected_record()
445         if record:
446             successful_request = bool(
447                 record.get("type") == "REQUEST"
448                 and record.get("seq") in self._successful_request_seqs
449             )
450             self.copy_to_clipboard(
451                 decoded_record_text(
452                     record,
453                     successful_request=successful_request,
454                     language=self.current_language,
455                 )
456             )
457
458     def context_filter_selected_slave(self):
459         record = self.selected_record()
460         if not record:
461             return
462         slave = str(record.get("slave", "") or "")
463         if not slave or slave not in self.slave_filter_vars:
464             return
465
466         self._transaction_filter_seqs = None
467         self.slave_filter_all_mode = False
468         for key, var in self.slave_filter_vars.items():
469             var.set(key == slave)
470         self.update_slave_heading()
471         self.render_filtered_view()
472
473     def context_filter_selected_fc(self):
474         record = self.selected_record()
475         if not record:
476             return
477         fc = str(record.get("fc", "") or "")
478         if not fc:
479             return
480         if str(record.get("type", "")) == "EXCEPTION":
```

MBToolkit v3.5/mb_view.py — lines 481-520

```
481             try:
482                 fc = f"{int(fc, 16) & 0x7F:02X}"
483             except ValueError:
484                 pass
485         self.advanced_filter_fc_var.set(fc)
486         self._transaction_filter_seqs = None
487         self.render_filtered_view()
488 
489     def context_show_selected_transaction(self):
490         record = self.selected_record()
491         if not record:
492             return
493 
494         seq = record.get("seq")
495         seqs = {seq}
496 
497         if record.get("request_seq") is not None:
498             seqs.add(record["request_seq"])
499         else:
500             for candidate in self._frame_history:
501                 if candidate.get("request_seq") == seq:
502                     seqs.add(candidate.get("seq"))
503                     break
504 
505         self.slave_filter_all_mode = True
506         for var in self.slave_filter_vars.values():
507             var.set(True)
508         self.update_slave_heading()
509 
510         self.advanced_filter_type_var.set(self.tr("Todos"))
511         self.advanced_filter_fc_var.set("")
512         self.advanced_filter_response_ms_var.set("")
513         self.advanced_filter_text_var.set("")
514 
515         self._transaction_filter_seqs = {
516             value for value in seqs if value is not None
517         }
518         self.render_filtered_view()
519 
520     def clear_transaction_filter(self):
```

MBToolkit v3.5/mb_view.py — lines 521-560

```
        self._transaction_filter_seqs = None
        self.render_filtered_view()

    def context_open_selected_in_master(self):
        record = self.selected_record()
        if record is not None:
            self.master_open_from_record(record)

    def context_open_selected_in_slave(self):
        record = self.selected_record()
        if record is not None:
            self.slave_open_from_record(record)

    def show_traffic_context_menu(self, event):
        row_iid = self.tree.identify_row(event.y)
        if not row_iid or row_iid not in self._record_by_iid:
            return

        self.tree.selection_set(row_iid)
        self.tree.focus(row_iid)
        self.on_row_select()

        from mb_slave_ui import slave_definition_from_record

        record = self._record_by_iid[row_iid]
        for index, opener in ((TRAFFIC_MENU_OPEN_IN_MASTER,
                               getattr(self, "master_definition_from_record", None)),
                              (TRAFFIC_MENU_OPEN_IN_SLAVE, slave_definition_from_record)):
            if callable(opener):
                state = "normal" if opener(record) else "disabled"
                try:
                    self.traffic_context_menu.entryconfigure(index, state=state)
                except tk.TclError:
                    pass

        try:
            self.traffic_context_menu.tk_popup(event.x_root, event.y_root)
        finally:
            try:
                self.traffic_context_menu.grab_release()
```

MBToolkit v3.5/mb_view.py — lines 561-600

```
            except tk.TclError:
                pass

    def export_session_csv(self, path=None):
        interactive = path is None
        records = list(self._frame_history)
        if not records:
            if path is None:
                messagebox.showinfo(
                    self.tr("Exportar CSV"),
                    self.tr("Não existem frames na sessão atual para exportar."),
                )
            return None

        if path is None:
            filename = datetime.now().strftime(
                "MBToolkit_session_%Y%m%d_%H%M%S.csv"
            )
            path = filedialog.asksaveasfilename(
                title=self.tr("Exportar sessão para CSV"),
                defaultextension=".csv",
                initialfile=filename,
                filetypes=[("CSV", "*.csv"), (self.tr("Todos os ficheiros"), "*.*")],
            )
            if not path:
                return None

        fieldnames = [
            "Time",
            "Delta ms",
            "Response ms",
            "Channel",
            "Type",
            "Slave",
            "FC",
            "Details",
            "CRC",
            "Raw",
            "PDU Address",
            "1-based Address",
```

MBToolkit v3.5/mb_view.py — lines 601-640

```
            "Qty",
            "Byte Count",
            "Timed out",
            "Anomaly",
        ]

        try:
            with open(path, "w", newline="", encoding="utf-8-sig") as handle:
                writer = csv.DictWriter(handle, fieldnames=fieldnames)
                writer.writeheader()

                for record in records:
                    inspector = inspector_values(record, language=self.current_language)
                    byte_count_text = inspector["byte_count"].split(
                        ":", 1
                    )[-1].strip()
                    writer.writerow({
                        "Time": record.get("time", ""),
                        "Delta ms": record.get("delta_text", ""),
                        "Response ms": record.get("response_text", ""),
                        "Channel": record.get("channel", ""),
                        "Type": record.get("type", ""),
                        "Slave": record.get("slave", ""),
                        "FC": record.get("fc", ""),
                        "Details": translate_details(record.get("details", ""), self.current_language),
                        "CRC": record.get("crc", ""),
                        "Raw": record.get("raw", ""),
                        "PDU Address": (
                            "" if record.get("pdu_address") is None
                            else record.get("pdu_address")
                        ),
                        "1-based Address": (
                            "" if record.get("one_based") is None
                            else record.get("one_based")
                        ),
                        "Qty": (
                            "" if record.get("qty") is None
                            else record.get("qty")
                        ),
                        "Byte Count": (
```

MBToolkit v3.5/mb_view.py — lines 641-680

```
                            "" if byte_count_text == "–"
                            else byte_count_text
                        ),
                        "Timed out": "Yes" if record.get("timed_out") else "No",
                        "Anomaly": anomaly_kind(
                            record,
                            SLOW_RESPONSE_THRESHOLD_MS,
                        ) or "",
                    })

            if interactive:
                try:
                    messagebox.showinfo(
                        self.tr("Exportar CSV"),
                        self.trf("Sessão exportada para:\n\n{path}", path=path),
                    )
                except tk.TclError:
                    pass
            return path

        except OSError as exc:
            messagebox.showerror(
                self.tr("Exportar CSV"),
                self.trf("Não foi possível guardar o CSV:\n\n{error}", error=exc),
            )
            return None

    def tree_record_column_values(self, record):
        return {
            "time": record.get("time", ""),
            "delta": record.get("delta_text", ""),
            "response": record.get("response_text", ""),
            "channel": record.get("channel", ""),
            "type": record.get("type", ""),
            "slave": record.get("slave", ""),
            "fc": record.get("fc", ""),
            "details": translate_details(record.get("details", ""), self.current_language),
            "crc": record.get("crc", ""),
        }

```

MBToolkit v3.5/mb_view.py — lines 681-720

```
    def measure_tree_column_width(self, column, records, heading_text=None):
        """
        Measure the pixel width required by a non-wrapped Treeview column.

        Details is handled separately because it uses the remaining viewport
        width and wraps instead of expanding horizontally without limit.
        """
        heading = (
            heading_text
            if heading_text is not None
            else self.tree_headings.get(column, column)
        )
        max_px = self.tree_heading_font.measure(str(heading))

        for record in records:
            value = self.tree_record_column_values(record).get(column, "")
            max_px = max(max_px, self.tree_cell_font.measure(str(value)))

        return max_px + self.tree_column_padding_px

    def available_details_width(self):
        """
        Return the width that can be assigned to Details without intentionally
        forcing horizontal scrolling in the current Treeview viewport.
        """
        tree_width = int(self.tree.winfo_width())
        if tree_width <= 20:
            tree_width = 1100  # sensible pre-layout fallback

        other_total = 0
        for column in self.tree["columns"]:
            if column == "details":
                continue
            other_total += int(self.tree.column(column, "width"))

        return max(
            self.tree_detail_min_width_px,
            tree_width - other_total - self.tree_detail_border_reserve_px
        )

```

MBToolkit v3.5/mb_view.py — lines 721-760

```
    def wrap_text_to_pixels(self, value, max_px):
        """
        Word-wrap text using the actual Tk font width.

        Long individual tokens are split character-by-character as a fallback,
        so a cell can never force the Details column wider than its target.
        """
        text = str(value or "")
        if not text:
            return ""

        max_px = max(40, int(max_px))
        paragraphs = text.splitlines() or [text]
        output_lines = []

        for paragraph in paragraphs:
            words = paragraph.split()
            if not words:
                output_lines.append("")
                continue

            line = ""
            for word in words:
                candidate = word if not line else f"{line} {word}"
                if self.tree_cell_font.measure(candidate) <= max_px:
                    line = candidate
                    continue

                if line:
                    output_lines.append(line)
                    line = ""

                # The token itself may be wider than the available space.
                if self.tree_cell_font.measure(word) <= max_px:
                    line = word
                    continue

                chunk = ""
                for char in word:
                    candidate_chunk = chunk + char
```

MBToolkit v3.5/mb_view.py — lines 761-800

```
                    if chunk and self.tree_cell_font.measure(candidate_chunk) > max_px:
                        output_lines.append(chunk)
                        chunk = char
                    else:
                        chunk = candidate_chunk
                line = chunk

            if line:
                output_lines.append(line)

        return "\n".join(output_lines)

    def details_horizontal_rule(self):
        """
        Return a rule that fills the Details column as far as the Treeview cell
        permits. A slight overdraw is intentional; Treeview clips it at the
        column boundary.
        """
        details_width = max(
            40,
            int(self.tree.column("details", "width"))
        )
        char_px = max(1, self.tree_cell_font.measure("—"))
        count = max(8, int(math.ceil(details_width / char_px)) + 4)
        return "—" * count

    def completed_pairs_for_records(self, records):
        """
        Return complete visible REQUEST/RESPONSE pairs in current display order.

        Each matched RESPONSE stores the exact request sequence id. This allows
        correct boundaries in both Hora ↑ and Hora ↓ without relying on row
        adjacency alone.
        """
        seq_to_index = {
            record.get("seq"): index
            for index, record in enumerate(records)
        }

        pairs = []
```

MBToolkit v3.5/mb_view.py — lines 801-840

```
        for response in records:
            request_seq = response.get("request_seq")
            response_seq = response.get("seq")

            if request_seq is None or request_seq not in seq_to_index:
                continue

            request_index = seq_to_index[request_seq]
            response_index = seq_to_index.get(response_seq)
            if response_index is None:
                continue

            start_index = min(request_index, response_index)
            end_index = max(request_index, response_index)

            pairs.append({
                "request_seq": request_seq,
                "response_seq": response_seq,
                "start_index": start_index,
                "end_index": end_index,
                # Separator belongs after whichever logical frame is visually
                # the end of the pair.
                "end_seq": records[end_index].get("seq"),
            })

        pairs.sort(key=lambda item: item["start_index"])
        return pairs

    def separator_after_sequences(self, records):
        """
        Return logical frame seq ids that need a separator AFTER them.

        There is intentionally no separator after the final complete visible
        pair because there is nothing below it to divide.
        """
        if not self.separate_transactions_var.get():
            return set()

        pairs = self.completed_pairs_for_records(records)
        if len(pairs) < 2:
```

MBToolkit v3.5/mb_view.py — lines 841-880

```
841             return set()
842 
843         return {
844             pair["end_seq"]
845             for pair in pairs[:-1]
846         }
847 
848     def tree_separator_values(self):
849         return (
850             "", "", "", "", "", "", "",
851             self.details_horizontal_rule(),
852             ""
853         )
854 
855     def insert_tree_separator_after_seq(self, seq):
856         """Insert one Details-width separator after the visual rows of seq."""
857         if seq in self._tree_separator_iid_by_seq:
858             return
859 
860         iids = self._tree_iids_by_seq.get(seq, [])
861         if not iids:
862             return
863 
864         last_iid = iids[-1]
865         insert_at = self.tree.index(last_iid) + 1
866 
867         iid = self.tree.insert(
868             "",
869             insert_at,
870             values=self.tree_separator_values()
871         )
872         self._raw_by_iid[iid] = ""
873         self._tree_separator_iid_by_seq[seq] = iid
874 
875     def sync_tree_transaction_separators(self, records=None):
876         """
877         Synchronize standalone transaction separator rows without rebuilding
878         normal traffic rows.
879         """
880         if records is None:
```

MBToolkit v3.5/mb_view.py — lines 881-920

```
            records = self.records_for_display()

        desired = self.separator_after_sequences(records)
        existing = set(self._tree_separator_iid_by_seq)

        # Remove boundaries that are no longer valid (filter/order changes).
        for seq in existing - desired:
            iid = self._tree_separator_iid_by_seq.pop(seq, None)
            if iid and self.tree.exists(iid):
                self.tree.delete(iid)
            self._raw_by_iid.pop(iid, None)

        # Add newly required boundaries.
        for seq in desired - existing:
            self.insert_tree_separator_after_seq(seq)

    def detail_lines_for_record(self, record):
        """
        Return wrapped Details as separate visual Treeview rows.

        ttk.Treeview has one global row height for the entire widget. Putting
        newline-wrapped text in one item would therefore make every REQUEST and
        RESPONSE row as tall as the largest Details message. Continuation rows
        preserve wrapping while keeping normal rows compact.
        """
        content_width = max(
            40,
            int(self.tree.column("details", "width"))
            - self.tree_column_padding_px
        )
        wrapped = self.wrap_text_to_pixels(
            translate_details(record.get("details", ""), self.current_language),
            content_width
        )

        if not wrapped:
            return [""]

        return wrapped.splitlines()

```

MBToolkit v3.5/mb_view.py — lines 921-960

```
921     def set_tree_rowheight_for_records(self, records):
922         """Treeview row height stays fixed; wrapping uses continuation rows."""
923         self.tree_style.configure(
924             self.tree_style_name,
925             rowheight=self.tree_single_line_rowheight
926         )
927 
928     def update_tree_rowheight_for_record(self, record):
929         """No-op by design: one logical frame may use several compact rows."""
930         return
931 
932     def autofit_tree_columns(self, records=None):
933         """
934         Non-Details columns fit the longest visible value/title.
935 
936         Details deliberately behaves differently: it consumes the remaining
937         viewport width and wraps. This keeps the full row information visible
938         without letting long register lists create an extremely wide table.
939         """
940         if records is None:
941             records = self.records_for_display()
942 
943         # First compact every non-Details column.
944         for column in self.tree["columns"]:
945             if column == "details":
946                 continue
947 
948             width = self.measure_tree_column_width(column, records)
949             self.tree.column(
950                 column,
951                 width=width,
952                 minwidth=width,
953                 anchor="center",
954                 stretch=False
955             )
956 
957         # Then allocate the remaining viewport to Details.
958         heading_min = self.measure_tree_column_width(
959             "details",
960             [],
```

MBToolkit v3.5/mb_view.py — lines 961-1000

```
            heading_text=self.tree_headings["details"]
        )
        details_width = max(
            heading_min,
            self.available_details_width()
        )
        self.tree.column(
            "details",
            width=details_width,
            minwidth=heading_min,
            anchor="w",
            stretch=True
        )

        self.set_tree_rowheight_for_records(records)

    def expand_tree_columns_for_record(self, record):
        """
        Incrementally enlarge only the compact non-Details columns.

        Returns True when another column became wider. If that happens the
        remaining Details width changes, so the caller can rebuild/re-wrap the
        visible table once for the batch.
        """
        values = self.tree_record_column_values(record)
        layout_changed = False

        for column in self.tree["columns"]:
            if column == "details":
                continue

            heading_px = self.tree_heading_font.measure(
                str(self.tree_headings.get(column, column))
            )
            value_px = self.tree_cell_font.measure(str(values.get(column, "")))
            required = max(heading_px, value_px) + self.tree_column_padding_px
            current = int(self.tree.column(column, "width"))

            if required > current:
                self.tree.column(
```

MBToolkit v3.5/mb_view.py — lines 1001-1040

```
                    column,
                    width=required,
                    minwidth=required,
                    anchor="center",
                    stretch=False
                )
                layout_changed = True

        if layout_changed:
            heading_min = self.measure_tree_column_width(
                "details",
                [],
                heading_text=self.tree_headings["details"]
            )
            details_width = max(
                heading_min,
                self.available_details_width()
            )
            self.tree.column(
                "details",
                width=details_width,
                minwidth=heading_min,
                anchor="w",
                stretch=True
            )

        return layout_changed

    def refresh_tree_heading_width(self, column):
        """
        Dynamic Hora/Slave headings remain compact. Details is intentionally
        excluded because it is viewport-driven and wrapped.
        """
        if column == "details":
            return

        records = self.records_for_display()
        width = self.measure_tree_column_width(column, records)
        self.tree.column(
            column,
```

MBToolkit v3.5/mb_view.py — lines 1041-1080

```
            width=width,
            minwidth=width,
            anchor="center",
            stretch=False
        )

    def schedule_tree_reflow(self, _event=None):
        """
        Debounce window/Treeview resizing and then re-wrap Details to the new
        available width.
        """
        if self._tree_reflow_after_id is not None:
            try:
                self.after_cancel(self._tree_reflow_after_id)
            except Exception:
                pass

        self._tree_reflow_after_id = self.after(
            120,
            self.reflow_tree_after_resize
        )

    def reflow_tree_after_resize(self):
        self._tree_reflow_after_id = None
        if not hasattr(self, "tree"):
            return
        self.render_filtered_view()

    def _slave_sort_key(self, slave):
        try:
            return (0, int(slave))
        except (TypeError, ValueError):
            return (1, str(slave))

    def rebuild_slave_filter_menu(self):
        self.slave_filter_menu.delete(0, "end")

        self.slave_filter_menu.add_command(
            label=self.tr("Todos"),
            command=self.select_all_slaves
```

MBToolkit v3.5/mb_view.py — lines 1081-1120

```
        )
        self.slave_filter_menu.add_command(
            label=self.tr("Nenhum"),
            command=self.select_no_slaves
        )

        if self.slave_filter_vars:
            self.slave_filter_menu.add_separator()
            for slave in sorted(self.slave_filter_vars, key=self._slave_sort_key):
                self.slave_filter_menu.add_checkbutton(
                    label=f"Slave {slave}",
                    variable=self.slave_filter_vars[slave],
                    command=self.on_slave_filter_changed
                )
        else:
            self.slave_filter_menu.add_separator()
            self.slave_filter_menu.add_command(
                label=self.tr("Nenhum slave detetado"),
                state="disabled"
            )

        self.update_slave_heading()

    def update_slave_heading(self):
        if self.slave_filter_all_mode:
            label = "Slave ▾"
        else:
            selected = [
                slave for slave, var in self.slave_filter_vars.items()
                if var.get()
            ]
            label = (self.tr("Slave (Filtro) ▾") if selected else self.tr("Slave (Nenhum) ▾"))

        self.tree_headings["slave"] = label
        self.tree.heading(
            "slave",
            text=label,
            anchor="center",
            command=self.show_slave_filter_menu
        )
```

MBToolkit v3.5/mb_view.py — lines 1121-1160

```
1121        self.refresh_tree_heading_width("slave")
1122
1123    def show_slave_filter_menu(self):
1124        """
1125        Open the slave filter menu at the position where the Slave heading
1126        was clicked.
1127        """
1128        try:
1129            x = self.winfo_pointerx()
1130            y = self.winfo_pointery() + 8
1131            self.slave_filter_menu.tk_popup(x, y)
1132        finally:
1133            try:
1134                self.slave_filter_menu.grab_release()
1135            except tk.TclError:
1136                pass
1137
1138    def register_slave_for_filter(self, slave):
1139        slave = str(slave or "").strip()
1140        if not slave:
1141            return
1142
1143        if slave in self.slave_filter_vars:
1144            return
1145
1146        self.slave_filter_vars[slave] = tk.BooleanVar(
1147            value=self.slave_filter_all_mode
1148        )
1149        self.rebuild_slave_filter_menu()
1150
1151    def select_all_slaves(self):
1152        self.slave_filter_all_mode = True
1153        for var in self.slave_filter_vars.values():
1154            var.set(True)
1155        self.update_slave_heading()
1156        self.render_filtered_view()
1157
1158    def select_no_slaves(self):
1159        self.slave_filter_all_mode = False
1160        for var in self.slave_filter_vars.values():
```

MBToolkit v3.5/mb_view.py — lines 1161-1200

```
            var.set(False)
        self.update_slave_heading()
        self.render_filtered_view()

    def on_slave_filter_changed(self):
        if not self.slave_filter_vars:
            self.slave_filter_all_mode = True
        else:
            values = [var.get() for var in self.slave_filter_vars.values()]
            # If every currently known slave is checked, remain in "All" mode
            # so future slaves are automatically displayed too.
            self.slave_filter_all_mode = bool(values) and all(values)

        self.update_slave_heading()
        self.render_filtered_view()

    def selected_slave_ids(self):
        return {
            slave for slave, var in self.slave_filter_vars.items()
            if var.get()
        }

    def record_passes_slave_filter(self, record):
        # "Todos" includes RAW/unsynchronised data where a slave ID could not
        # be identified. "Nenhum" hides absolutely all traffic.
        if self.slave_filter_all_mode:
            return True

        selected = self.selected_slave_ids()
        if not selected:
            return False

        return str(record.get("slave", "")) in selected

    def time_order_descending(self):
        return canonical_ui_text(self.time_order_var.get()).startswith("Decrescente")

    def on_time_order_changed(self):
        label = self.tr("Hora ↓") if self.time_order_descending() else self.tr("Hora ↑")
        self.tree_headings["time"] = label
```

MBToolkit v3.5/mb_view.py — lines 1201-1240

```
        self.tree.heading(
            "time",
            text=label,
            anchor="center",
            command=self.toggle_time_order
        )
        self.render_filtered_view()

    def toggle_time_order(self):
        if self.time_order_descending():
            self.time_order_var.set(self.tr("Crescente — antigo → recente"))
        else:
            self.time_order_var.set(self.tr("Decrescente — recente → antigo"))
        self.on_time_order_changed()

    def records_for_display(self):
        records = [
            record for record in self._frame_history
            if (
                self.record_passes_slave_filter(record)
                and self.record_passes_advanced_filters(record)
            )
        ]
        if self.time_order_descending():
            records.reverse()
        return records

    def raw_line_for_record(self, record):
        # Keep fixed-width timing fields so the rest of every Raw Hex line
        # starts at exactly the same horizontal position. When a value does not
        # exist, show a completely blank field instead of "-", "dt=", "resp="
        # or "ms".
        delta_field = (
            f"dt={record['delta_text']:>7}ms "
            if record.get("delta_text")
            else " " * 13
        )
        response_field = (
            f"resp={record['response_text']:>7}ms "
            if record.get("response_text")
```

MBToolkit v3.5/mb_view.py — lines 1241-1280

```
            else " " * 15
        )

        return (
            f"[{record['time']}] "
            f"{delta_field}"
            f"{response_field}"
            f"{record['channel']:<6} {record['type']:<10} "
            f"Slave={record['slave'] or '-':<3} FC={record['fc'] or '-':<2} "
            f"CRC={record['crc']:<5} | {record['raw']} | {translate_details(record['details'], self.current_language)}\n"
        )

    def raw_separator_width_chars(self, raw_lines=None):
        """
        Width of one Raw Hex separator in visible character cells.

        Raw Hex now wraps to the viewport, so the separator must span only the
        visible width. Using the longest unwrapped data line would make the
        separator wrap onto multiple lines.
        """
        try:
            # Leave a small pixel reserve for Text internal borders/padding so
            # the last separator character does not wrap onto a second line.
            widget_px = max(1, int(self.raw_text.winfo_width()) - 14)
            char_px = max(1, self.raw_text_font.measure("—"))
            viewport_chars = max(
                1,
                int(widget_px // char_px)
            )
        except Exception:
            viewport_chars = 1

        return viewport_chars

    def raw_transaction_separator(self, width_chars=None):
        if width_chars is None:
            width_chars = self._raw_separator_width_chars
        return "—" * max(1, int(width_chars)) + "\n"

    def schedule_raw_separator_reflow(self, _event=None):
```

MBToolkit v3.5/mb_view.py — lines 1281-1320

```
        if self._raw_reflow_after_id is not None:
            try:
                self.after_cancel(self._raw_reflow_after_id)
            except Exception:
                pass

        self._raw_reflow_after_id = self.after(
            120,
            self.reflow_raw_separator_after_resize
        )

    def reflow_raw_separator_after_resize(self):
        self._raw_reflow_after_id = None
        if self.separate_transactions_var.get():
            self.render_raw_text_only()

    def insert_tree_record(self, record, index="end", allow_separator=True):
        detail_lines = self.detail_lines_for_record(record)
        visual_rows = []

        # Main row: all columns + first Details line.
        visual_rows.append({
            "kind": "main",
            "values": (
                record["time"],
                record["delta_text"],
                record["response_text"],
                record["channel"],
                record["type"],
                record["slave"],
                record["fc"],
                detail_lines[0],
                record["crc"],
            ),
            "raw": record["raw"],
        })

        # Wrapped continuation lines keep only the Details text.
        for continuation in detail_lines[1:]:
            visual_rows.append({
```

MBToolkit v3.5/mb_view.py — lines 1321-1360

```
                "kind": "continuation",
                "values": (
                    "", "", "", "", "", "", "",
                    continuation,
                    ""
                ),
                "raw": record["raw"],
            })

        inserted_main_iid = None
        inserted_iids = []

        # index=0 requires reverse insertion to preserve visual block order.
        rows_to_insert = (
            list(reversed(visual_rows))
            if index == 0
            else visual_rows
        )

        row_tag = self.anomaly_tag_for_record(record)

        for visual in rows_to_insert:
            iid = self.tree.insert(
                "",
                index,
                values=visual["values"],
                tags=((row_tag,) if row_tag else ()),
            )

            self._raw_by_iid[iid] = visual["raw"]
            self._record_by_iid[iid] = record

            if index == 0:
                inserted_iids.insert(0, iid)
                self._tree_history.appendleft((iid, True))
            else:
                inserted_iids.append(iid)
                self._tree_history.append((iid, True))

            if visual["kind"] == "main":
```

MBToolkit v3.5/mb_view.py — lines 1361-1400

```
                inserted_main_iid = iid

        self._tree_iids_by_seq[record["seq"]] = inserted_iids
        self._ui_frame_count += 1
        return inserted_main_iid

    def render_raw_text_only(self):
        records = self.records_for_display()
        separator_after = self.separator_after_sequences(records)

        line_by_seq = {
            record["seq"]: self.raw_line_for_record(record)
            for record in records
        }
        all_raw_lines = list(line_by_seq.values())
        self._raw_separator_width_chars = self.raw_separator_width_chars(
            all_raw_lines
        )
        separator_line = self.raw_transaction_separator(
            self._raw_separator_width_chars
        )

        # Respect the visual line cap, counting transaction separators as one
        # line each.
        if not self.time_order_descending():
            selected_reversed = []
            line_count = 0

            for record in reversed(records):
                cost = 1 + (1 if record["seq"] in separator_after else 0)
                if line_count + cost > MAX_RAW_TEXT_LINES:
                    break
                selected_reversed.append(record)
                line_count += cost

            raw_records = list(reversed(selected_reversed))
        else:
            selected = []
            line_count = 0

```

MBToolkit v3.5/mb_view.py — lines 1401-1440

```
            for record in records:
                cost = 1 + (1 if record["seq"] in separator_after else 0)
                if line_count + cost > MAX_RAW_TEXT_LINES:
                    break
                selected.append(record)
                line_count += cost

            raw_records = selected

        parts = []
        for record in raw_records:
            parts.append(line_by_seq[record["seq"]])
            if record["seq"] in separator_after:
                parts.append(separator_line)

        block = "".join(parts)
        self.raw_text.delete("1.0", "end")
        if block:
            self.raw_text.insert("end", block)
        self._raw_text_line_count = block.count("\n")

    def render_filtered_view(self):
        # This is intentionally called on user-driven filter/sort changes and
        # infrequent history-prune events, not once per received frame.
        selected_record = self.selected_record()
        selected_seq = (
            selected_record.get("seq")
            if selected_record is not None
            else None
        )

        for item in self.tree.get_children():
            self.tree.delete(item)

        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._tree_history.clear()
        self._tree_iids_by_seq.clear()
        self._tree_separator_iid_by_seq.clear()
        self._ui_frame_count = 0
```

MBToolkit v3.5/mb_view.py — lines 1441-1480

```
        self.selected_raw_var.set("")

        descending = self.time_order_descending()
        records = self.records_for_display()

        # Column sizing determines the wrap width, so it must happen before
        # inserting the visible rows.
        self.autofit_tree_columns(records)

        for record in records:
            self.insert_tree_record(
                record,
                index="end",
                allow_separator=False
            )

        self.sync_tree_transaction_separators(records)
        self.render_raw_text_only()

        if selected_seq is not None:
            iids = self._tree_iids_by_seq.get(selected_seq, [])
            if iids:
                try:
                    self.tree.selection_set(iids[0])
                    self.tree.focus(iids[0])
                    self.update_frame_inspector(
                        self._record_by_iid.get(iids[0])
                    )
                except tk.TclError:
                    pass
            else:
                self.update_frame_inspector(None)

        if self.auto_scroll_var.get() and self._tree_history:
            if descending:
                iid = self._tree_history[0][0]
                self.tree.see(iid)
                self.raw_text.see("1.0")
            else:
                iid = self._tree_history[-1][0]
```

MBToolkit v3.5/mb_view.py — lines 1481-1520

```
                self.tree.see(iid)
                self.raw_text.see("end")

    def add_frame_row(self, row):
        # Kept as a convenience wrapper for callers/tests.
        self.add_frame_rows_batch([row])

    def add_frame_rows_batch(self, rows):
        if not rows:
            return

        raw_lines_log = []
        any_visible = False
        tree_layout_changed = False
        pair_boundary_changed = False
        expired_requests = []

        self.pulse_bus_activity()

        for row in rows:
            seq = self._next_frame_seq
            self._next_frame_seq += 1
            row["_view_seq"] = seq

            timing = self.metrics.process(row)

            delta_text = (
                f"{timing['delta_ms']:.1f}" if timing["delta_ms"] is not None else ""
            )
            response_text = (
                f"{timing['response_ms']:.1f}"
                if timing["response_ms"] is not None
                else ""
            )

            # Keep pairing for response-time/statistics, but do not copy the
            # request Address/1-based/Qty into RESPONSE/EXCEPTION details.
            # Those fields remain visible only on the REQUEST itself.
            matched = timing.get("matched_request")
            expired_requests.extend(
```

MBToolkit v3.5/mb_view.py — lines 1521-1560

```
                timing.get("expired_requests", [])
            )

            self.register_slave_for_filter(row.get("slave", ""))

            # FC01/FC02: the request Qty tells which bits of the last data
            # byte are padding, so trim them from the response details.
            if (
                matched is not None
                and row.get("type") == "RESPONSE"
                and row.get("fc") in ("01", "02")
            ):
                try:
                    row["details"] = read_bits_details(
                        bytes.fromhex(row.get("raw", "")),
                        matched.get("qty"),
                    )
                except ValueError:
                    pass

            record = {
                "seq": seq,
                "epoch": float(row.get("_epoch", time.time())),
                "time": row["time"],
                "delta_text": delta_text,
                "response_text": response_text,
                "channel": row["channel"],
                "type": row["type"],
                "slave": row["slave"],
                "fc": row["fc"],
                "details": row["details"],
                "crc": row["crc"],
                "raw": row["raw"],
                "pdu_address": row.get("pdu_address"),
                "one_based": row.get("one_based"),
                "qty": row.get("qty"),
                "frame_len": row.get("_frame_len", 0),
                "timed_out": False,
                "matched": matched is not None,
                "request_seq": (
```

MBToolkit v3.5/mb_view.py — lines 1561-1600

```
                    matched.get("seq")
                    if matched is not None
                    else None
                ),
                # Used only by the Frame Inspector to show the address of
                # each returned value; RESPONSE Details stay response-native.
                "request_pdu_address": (
                    matched.get("pdu_address")
                    if matched is not None
                    else None
                ),
                "request_qty": (
                    matched.get("qty")
                    if matched is not None
                    else None
                ),
            }
            self._frame_history.append(record)

            # Track successful normal transaction pairs independently of whether
            # highlighting is currently enabled. This lets the user toggle
            # "Realçar resultados" on/off against the exact same session.
            if (
                record.get("type") == "RESPONSE"
                and record.get("crc") == "OK"
                and record.get("matched")
                and anomaly_kind(record, SLOW_RESPONSE_THRESHOLD_MS) is None
                and record.get("request_seq") is not None
            ):
                self._successful_request_seqs.add(record["request_seq"])

                # If the REQUEST is already visible, update it immediately
                # without rebuilding the entire Treeview.
                if self.highlight_anomalies_var.get():
                    for request_iid in self._tree_iids_by_seq.get(
                        record["request_seq"], []
                    ):
                        try:
                            self.tree.item(
                                request_iid,
```

MBToolkit v3.5/mb_view.py — lines 1601-1640

```
                                tags=("result_success",),
                            )
                        except tk.TclError:
                            pass

            # Disk log is always complete and ignores GUI filters/sorting.
            raw_lines_log.append(self.raw_line_for_record(record))

            if (
                self.record_passes_slave_filter(record)
                and self.record_passes_advanced_filters(record)
            ):
                any_visible = True
                if self.expand_tree_columns_for_record(record):
                    tree_layout_changed = True

                self.update_tree_rowheight_for_record(record)

                if self.time_order_descending():
                    self.insert_tree_record(
                        record,
                        index=0,
                        allow_separator=False
                    )

                    raw_block = self.raw_line_for_record(record)
                    self.raw_text.insert("1.0", raw_block)
                    self._raw_text_line_count += raw_block.count("\n")
                else:
                    self.insert_tree_record(
                        record,
                        index="end",
                        allow_separator=False
                    )

                    raw_block = self.raw_line_for_record(record)
                    self.raw_text.insert("end", raw_block)
                    self._raw_text_line_count += raw_block.count("\n")

                if record.get("matched"):
```

MBToolkit v3.5/mb_view.py — lines 1641-1680

```
                    pair_boundary_changed = True

        if expired_requests:
            # A new frame can itself cause older requests to cross the Pending
            # timeout. Mark those original REQUEST records in place.
            self.mark_timed_out_records(
                expired_requests,
                rerender=True,
            )

        log_block = "".join(raw_lines_log)

        # Only real traffic is allowed to create a .txt log. GUI filters do not
        # change the complete saved traffic.
        if self.capture_active and log_block:
            if self.ensure_log_started():
                self.log_line(log_block)

        self.update_stats_labels()
        rebuilt = self.prune_ui_history()

        if tree_layout_changed and not rebuilt:
            self.render_filtered_view()
            rebuilt = True

        if not rebuilt and pair_boundary_changed:
            visible_records = self.records_for_display()
            self.sync_tree_transaction_separators(visible_records)
            self.render_raw_text_only()

        # Raw Hex has its own bounded line count. Rebuild it in the correct
        # chronological direction when its hysteresis threshold is reached.
        if (
            not rebuilt
            and self._raw_text_line_count
            > MAX_RAW_TEXT_LINES + RAW_TEXT_PRUNE_CHUNK_LINES
        ):
            self.render_raw_text_only()

        if self.auto_scroll_var.get() and any_visible and self._tree_history:
```

MBToolkit v3.5/mb_view.py — lines 1681-1720

```
            if self.time_order_descending():
                self.tree.see(self._tree_history[0][0])
                self.raw_text.see("1.0")
            else:
                self.tree.see(self._tree_history[-1][0])
                self.raw_text.see("end")

    def prune_ui_history(self):
        """
        Keep the GUI source history bounded.

        The full disk log is independent and remains complete. Pruning happens
        by deque chunks and only triggers a full GUI rebuild occasionally.
        """
        trigger_frames = MAX_UI_FRAMES + UI_PRUNE_CHUNK_FRAMES

        if len(self._frame_history) <= trigger_frames:
            return False

        while len(self._frame_history) > MAX_UI_FRAMES:
            self._frame_history.popleft()

        retained_sequences = {
            record.get("seq")
            for record in self._frame_history
            if record.get("seq") is not None
        }
        self._successful_request_seqs.intersection_update(retained_sequences)

        self.render_filtered_view()
        return True

    def on_row_select(self, _event=None):
        record = self.selected_record()
        self.update_frame_inspector(record)

    def finish_capture_ui(self):
        self.busy = False
        self.capture_active = False
        self.status_var.set(self.tr("● Parado"))
```

MBToolkit v3.5/mb_view.py — lines 1721-1760

```
        self.start_btn.configure(state="normal")
        self.stop_btn.configure(state="disabled")
        self.restart_btn.configure(state="normal")
        self.set_simulation_button_state("normal")
        self.set_sniffer_refresh_state("normal")
        self.serial_objects = []
        self.serial_by_port = {}
        self.reader_threads = []
        self.close_log()
        self.update_com_status()

    def app_directory(self):
        return get_application_directory()

    def prepare_log_session(self, prefix, started_at, header):
        """
        Prepare a real-capture log session without creating a file.

        The .txt file is created only after the first received frame/RAW block.
        """
        self.close_log()
        self.pending_log_filename = started_at.strftime(
            prefix + "_%Y%m%d_%H%M%S.txt"
        )
        self.pending_log_header = header
        self.pending_log_notes = []
        self.log_start_failed = False
        self.log_path = None

    def ensure_log_started(self):
        """
        Create the pending real-capture log on first actual traffic.

        Returns True when the log is open. A creation failure is reported only
        once for the current capture session.
        """
        if self.log_file:
            return True

        if not self.pending_log_filename or self.log_start_failed:
```

MBToolkit v3.5/mb_view.py — lines 1761-1800

```
            return False

        try:
            target = get_logs_folder() / self.pending_log_filename
            self.log_file = target.open("w", encoding="utf-8", buffering=1)
            self.log_path = target

            if self.pending_log_header:
                self.log_file.write(self.pending_log_header)

            if self.pending_log_notes:
                self.log_file.write("".join(self.pending_log_notes))

            self.pending_log_notes = []
            return True

        except OSError as exc:
            self.log_start_failed = True
            self.log_file = None
            self.log_path = None
            messagebox.showerror(
                self.tr("Erro ao criar log"),
                self.trf(
                    "Foi detetado tráfego, mas não foi possível criar o ficheiro de log em:\n\n{folder}\n\n{error}",
                    folder=get_logs_folder(),
                    error=exc,
                )
            )
            return False

    def queue_log_note(self, text):
        """
        Write a capture note to an already-open log, or keep it in memory until
        the first traffic frame creates the log.
        """
        if self.log_file:
            self.log_line(text)
        elif self.capture_active and self.pending_log_filename:
            self.pending_log_notes.append(text)

```

MBToolkit v3.5/mb_view.py — lines 1801-1840

```
    def log_line(self, text):
        if self.log_file:
            try:
                self.log_file.write(text)
            except Exception:
                pass

    def close_log(self):
        if self.log_file:
            try:
                self.log_file.write(
                    f"# Stop: {datetime.now().isoformat(timespec='milliseconds')}\n"
                )
                self.log_file.close()
            except Exception:
                pass

        self.log_file = None
        self.log_path = None
        self.pending_log_filename = None
        self.pending_log_header = ""
        self.pending_log_notes = []
        self.log_start_failed = False

    def open_log_folder(self):
        try:
            folder = get_logs_folder()
            if os.name == "nt":
                os.startfile(str(folder))
            elif sys.platform == "darwin":
                subprocess.Popen(["open", str(folder)])
            else:
                subprocess.Popen(["xdg-open", str(folder)])
        except Exception as exc:
            messagebox.showerror(self.tr("Abrir pasta"), str(exc))

    def clear_view(self):
        for item in self.tree.get_children():
            self.tree.delete(item)
        self.raw_text.delete("1.0", "end")
```

MBToolkit v3.5/mb_view.py — lines 1841-1880

```
        self.selected_raw_var.set("")
        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._tree_history.clear()
        self._tree_iids_by_seq.clear()
        self._tree_separator_iid_by_seq.clear()
        self._frame_history.clear()
        self._next_frame_seq = 1
        self._ui_frame_count = 0
        self._raw_text_line_count = 0
        self._raw_separator_width_chars = 1

        self.slave_filter_vars.clear()
        self.slave_filter_all_mode = True
        self.rebuild_slave_filter_menu()

        self._transaction_filter_seqs = None
        self._successful_request_seqs.clear()
        if hasattr(self, "advanced_filter_type_var"):
            self.advanced_filter_type_var.set(self.tr("Todos"))
            self.advanced_filter_fc_var.set("")
            self.advanced_filter_response_ms_var.set("")
            self.advanced_filter_text_var.set("")
        self.update_frame_inspector(None)
        if hasattr(self, "bus_activity_var"):
            self.bus_activity_var.set("BUS ○")

        self.time_order_var.set(self.tr("Crescente — antigo → recente"))
        self.tree_headings["time"] = self.tr("Hora ↑")
        self.tree.heading(
            "time",
            text=self.tree_headings["time"],
            anchor="center",
            command=self.toggle_time_order
        )

        # With no traffic, widths/row height return to their default layout.
        self.autofit_tree_columns([])

        self.reset_session_metrics()
```

MBToolkit v3.5/mb_view.py — lines 1881-1881

```
1881
```

MBToolkit v3.5/mb_slave_finder.py

1625 source lines.

MBToolkit v3.5/mb_slave_finder.py — lines 1-40

```python
#!/usr/bin/env python3
"""Active Modbus RTU Bus Slave Finder logic and GUI mixin."""

import threading
import time
import tkinter as tk
from tkinter import ttk, messagebox

from mb_config import (
    DISCOVERY_BAUD_RATES,
    DISCOVERY_PARITIES,
    DISCOVERY_STOP_BITS,
    DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
    DISCOVERY_MIN_TIMEOUT_MS,
    DISCOVERY_MAX_TIMEOUT_MS,
    DISCOVERY_PROBE_ADDRESS,
    DISCOVERY_PROBE_QTY,
    DISCOVERY_WRITE_TIMEOUT_SECONDS,
    DISCOVERY_WRITE_RETRIES,
    DISCOVERY_RX_BUFFER_LIMIT,
    DISCOVERY_MIN_ATTEMPTS,
    DISCOVERY_MAX_ATTEMPTS,
    DISCOVERY_DEFAULT_ATTEMPTS,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MODBUS_EXCEPTION_CODES,
)
from mb_protocol import (
    append_crc,
    crc_ok,
    hex_bytes,
    modbus_rtu_interframe_gap_seconds,
)
from mb_widgets import RefreshButton

def build_discovery_probe(slave: int, fc: int) -> bytes:
    """Build a read-only discovery probe: FC03/FC04, Address 0, Qty 1."""
    if not (1 <= int(slave) <= 247):
        raise ValueError("Slave ID fora do intervalo 1..247")
    if int(fc) not in (3, 4):
        raise ValueError("A descoberta ativa suporta apenas FC03/FC04")
```

MBToolkit v3.5/mb_slave_finder.py — lines 41-80

```
41
42      payload = bytes((
43          int(slave),
44          int(fc),
45          0x00, DISCOVERY_PROBE_ADDRESS & 0xFF,
46          0x00, DISCOVERY_PROBE_QTY & 0xFF,
47      ))
48      return append_crc(payload)
49
50  DEVICE_ID_FC = 0x2B
51  DEVICE_ID_MEI_TYPE = 0x0E
52  DEVICE_ID_READ_BASIC = 0x01
53  DEVICE_ID_MORE_FOLLOWS = 0xFF
54  DEVICE_ID_BASIC_OBJECTS = (0x00, 0x01, 0x02)
55  DEVICE_ID_MAX_PAGES = 8
56
57  def build_device_identification_probe(slave: int, object_id: int = 0x00) -> bytes:
58      """Build FC43/MEI 14 Basic Device Identification request."""
59      slave = int(slave)
60      object_id = int(object_id)
61      if not (1 <= slave <= 247):
62          raise ValueError("Slave ID fora do intervalo 1..247")
63      if not (0 <= object_id <= 0xFF):
64          raise ValueError("Object ID fora do intervalo 0..255")
65      return append_crc(bytes((
66          slave,
67          DEVICE_ID_FC,
68          DEVICE_ID_MEI_TYPE,
69          DEVICE_ID_READ_BASIC,
70          object_id,
71      )))
72
73  def _decode_device_id_value(value: bytes) -> str:
74      """Decode a Modbus Device Identification ASCII object for display."""
75      return bytes(value).decode("ascii", errors="replace").strip("\x00 ")
76
77  def find_device_identification_response(buffer: bytes, slave: int):
78      """Find one CRC-valid FC43/14 response or Modbus exception in *buffer*."""
79      data = bytes(buffer)
80      slave = int(slave)
```

MBToolkit v3.5/mb_slave_finder.py — lines 81-120

```
81
82      for offset in range(max(0, len(data) - 3)):
83          if data[offset] != slave or offset + 2 > len(data):
84              continue
85
86          response_fc = data[offset + 1]
87
88          # FC43 exception response: Slave + 0xAB + Exception + CRC.
89          if response_fc == (DEVICE_ID_FC | 0x80):
90              end = offset + 5
91              if end <= len(data):
92                  frame = data[offset:end]
93                  if crc_ok(frame):
94                      return {
95                          "kind": "EXCEPTION",
96                          "frame": frame,
97                          "exception_code": frame[2],
98                          "objects": {},
99                          "more_follows": False,
100                         "next_object_id": 0,
101                     }
102             continue
103
104         if response_fc != DEVICE_ID_FC:
105             continue
106
107         # Normal response fixed header before the object list:
108         # Slave, FC, MEI, ReadCode, Conformity, MoreFollows, NextObject, Count.
109         if offset + 8 > len(data):
110             continue
111         if data[offset + 2] != DEVICE_ID_MEI_TYPE:
112             continue
113         if data[offset + 3] != DEVICE_ID_READ_BASIC:
114             continue
115
116         more_follows = data[offset + 5] == DEVICE_ID_MORE_FOLLOWS
117         next_object_id = data[offset + 6]
118         object_count = data[offset + 7]
119         pos = offset + 8
120         objects = {}
```

MBToolkit v3.5/mb_slave_finder.py — lines 121-160

```
121         complete = True
122 
123         for _ in range(object_count):
124             if pos + 2 > len(data):
125                 complete = False
126                 break
127             object_id = data[pos]
128             object_len = data[pos + 1]
129             pos += 2
130             if pos + object_len > len(data):
131                 complete = False
132                 break
133             objects[object_id] = _decode_device_id_value(
134                 data[pos:pos + object_len]
135             )
136             pos += object_len
137 
138         if not complete:
139             continue
140 
141         end = pos + 2
142         if end > len(data):
143             continue
144         frame = data[offset:end]
145         if not crc_ok(frame):
146             continue
147 
148         return {
149             "kind": "DEVICE_ID",
150             "frame": frame,
151             "exception_code": None,
152             "objects": objects,
153             "more_follows": more_follows,
154             "next_object_id": next_object_id,
155             "conformity_level": data[offset + 4],
156         }
157 
158     return None
159 
160 def read_device_identification_response(ser, slave, timeout_s, stop_event):
```

MBToolkit v3.5/mb_slave_finder.py — lines 161-200

```
161     """Read until a valid FC43/14 response/exception is received or timed out."""
162     deadline = time.perf_counter() + max(0.001, float(timeout_s))
163     buffer = bytearray()
164 
165     while time.perf_counter() < deadline:
166         if stop_event.is_set():
167             return None
168 
169         waiting = int(getattr(ser, "in_waiting", 0) or 0)
170         if waiting:
171             chunk = ser.read(waiting)
172             if chunk:
173                 buffer.extend(chunk)
174                 if len(buffer) > DISCOVERY_RX_BUFFER_LIMIT:
175                     del buffer[:-DISCOVERY_RX_BUFFER_LIMIT]
176                 found = find_device_identification_response(buffer, slave)
177                 if found is not None:
178                     return found
179         elif stop_event.wait(0.001):
180             return None
181 
182     return find_device_identification_response(buffer, slave)
183 
184 def device_identification_timeout_seconds(
185     baud: int,
186     parity: str,
187     stopbits: str,
188     minimum_timeout_ms: float = DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
189 ) -> float:
190     """Conservative timeout for one FC43/14 response, up to one RTU frame."""
191     baud = max(1, int(baud))
192     bits_per_char = discovery_bits_per_char(parity, stopbits)
193     char_s = bits_per_char / float(baud)
194     # A Modbus RTU ADU is at most 256 bytes. Allow one full response frame,
195     # the recommended t3.5 gap and a small device-processing margin.
196     calculated = (
197         256.0 * char_s
198         + modbus_rtu_interframe_gap_seconds(baud, bits_per_char)
199         + 0.020
200     )
```

MBToolkit v3.5/mb_slave_finder.py — lines 201-240

```
    return max(float(minimum_timeout_ms) / 1000.0, calculated)

def device_identification_request_transmit_seconds(
    baud: int, parity: str, stopbits: str
) -> float:
    """Estimated on-wire duration of the 7-byte FC43/14 request."""
    return 7.0 * discovery_bits_per_char(parity, stopbits) / float(max(1, int(baud)))

def format_device_identification(result) -> str:
    """Return compact Device Identification text for the Finder result table."""
    if result is None:
        return "No response"
    if result.get("kind") == "EXCEPTION":
        code = int(result.get("exception_code", 0))
        name = MODBUS_EXCEPTION_CODES.get(
            code, ("Unknown/Reserved Exception", "")
        )[0]
        return f"Exception 0x{code:02X} – {name}"

    objects = result.get("objects", {}) or {}
    values = [objects.get(obj_id, "") for obj_id in DEVICE_ID_BASIC_OBJECTS]
    if any(values):
        return " | ".join(value or "—" for value in values)
    return "Response – no Basic Device Identification objects"

def read_basic_device_identification(
    ser, slave, baud, parity, stopbits, minimum_timeout_ms, stop_event, serial_module
):
    """Read Basic Device Identification, following More Follows when required."""
    object_id = 0x00
    merged_objects = {}
    frames = []
    conformity_level = None

    for _page in range(DEVICE_ID_MAX_PAGES):
        if stop_event.is_set():
            return None

        try:
            ser.reset_input_buffer()
```

MBToolkit v3.5/mb_slave_finder.py — lines 241-280

```
        except Exception:
            raise

        request = build_device_identification_probe(slave, object_id)
        sent = write_discovery_request(
            ser, request, baud, parity, stopbits, stop_event, serial_module
        )
        if not sent or stop_event.is_set():
            return None

        result = read_device_identification_response(
            ser,
            slave,
            device_identification_timeout_seconds(
                baud, parity, stopbits, minimum_timeout_ms
            ),
            stop_event,
        )
        if result is None or result.get("kind") == "EXCEPTION":
            return result

        frames.append(result["frame"])
        merged_objects.update(result.get("objects", {}))
        conformity_level = result.get("conformity_level")

        # Basic identification is complete as soon as VendorName, ProductCode
        # and MajorMinorRevision have all been received.
        if all(obj_id in merged_objects for obj_id in DEVICE_ID_BASIC_OBJECTS):
            return {
                "kind": "DEVICE_ID",
                "frame": b"".join(frames),
                "exception_code": None,
                "objects": merged_objects,
                "more_follows": bool(result.get("more_follows")),
                "next_object_id": int(result.get("next_object_id", 0)),
                "conformity_level": conformity_level,
            }

        if not result.get("more_follows"):
            break
```

MBToolkit v3.5/mb_slave_finder.py — lines 281-320

```
281
282         next_object_id = int(result.get("next_object_id", 0))
283         if next_object_id == object_id:
284             break
285         object_id = next_object_id
286
287         gap_s = modbus_rtu_interframe_gap_seconds(
288             baud, discovery_bits_per_char(parity, stopbits)
289         )
290         if stop_event.wait(gap_s):
291             return None
292
293     return {
294         "kind": "DEVICE_ID",
295         "frame": b"".join(frames),
296         "exception_code": None,
297         "objects": merged_objects,
298         "more_follows": False,
299         "next_object_id": object_id,
300         "conformity_level": conformity_level,
301     } if frames else None
302
303 def discovery_bits_per_char(parity: str, stopbits: str) -> float:
304     """Return serial bits/character: start + 8 data + parity + stop."""
305     parity_bits = 0 if parity == "None" else 1
306     stop_value = float(stopbits)
307     return 1.0 + 8.0 + parity_bits + stop_value
308
309 def discovery_response_timeout_seconds(
310     baud: int,
311     parity: str,
312     stopbits: str,
313     minimum_timeout_ms: float = DISCOVERY_DEFAULT_MIN_TIMEOUT_MS,
314 ) -> float:
315     """
316     Adaptive response deadline after the request has left the serial driver.
317
318     The deadline includes the on-wire duration of a normal 7-byte FC03/FC04
319     response plus processing/inter-frame margin, with a user-configurable
320     minimum.
```

MBToolkit v3.5/mb_slave_finder.py — lines 321-360

```
    """
    baud = max(1, int(baud))
    bits_per_char = discovery_bits_per_char(parity, stopbits)
    char_s = bits_per_char / float(baud)
    normal_response_s = 7.0 * char_s
    serial_margin_s = max(
        0.012, modbus_rtu_interframe_gap_seconds(baud, bits_per_char)
    )
    calculated = normal_response_s + serial_margin_s
    return max(float(minimum_timeout_ms) / 1000.0, calculated)

def discovery_request_transmit_seconds(baud: int, parity: str, stopbits: str) -> float:
    """Estimated on-wire duration of the fixed 8-byte discovery request."""
    return 8.0 * discovery_bits_per_char(parity, stopbits) / float(max(1, int(baud)))

def find_discovery_response(buffer: bytes, slave: int, fc: int):
    """
    Find a valid normal or exception response for one discovery probe.

    Any CRC-valid Modbus exception counts as proof that the slave exists.
    The scanner intentionally ignores other traffic and request echoes.
    """
    data = bytes(buffer)
    slave = int(slave)
    fc = int(fc)

    for offset in range(max(0, len(data) - 3)):
        if data[offset] != slave or offset + 2 > len(data):
            continue

        response_fc = data[offset + 1]

        if response_fc == (fc | 0x80):
            end = offset + 5
            if end <= len(data):
                frame = data[offset:end]
                if crc_ok(frame):
                    code = frame[2]
                    return {
                        "kind": "EXCEPTION",
```

MBToolkit v3.5/mb_slave_finder.py — lines 361-400

```
                        "frame": frame,
                        "exception_code": code,
                    }

        if response_fc == fc and offset + 3 <= len(data):
            byte_count = data[offset + 2]
            # FC03/04 register responses must contain an even positive byte count.
            if byte_count > 0 and byte_count % 2 == 0 and byte_count <= 250:
                end = offset + 5 + byte_count
                if end <= len(data):
                    frame = data[offset:end]
                    if crc_ok(frame):
                        return {
                            "kind": "RESPONSE",
                            "frame": frame,
                            "exception_code": None,
                        }

    return None

def read_discovery_response(ser, slave, fc, timeout_s, stop_event):
    """Read non-blocking serial bytes until a valid probe response or timeout."""
    deadline = time.perf_counter() + max(0.001, float(timeout_s))
    buffer = bytearray()

    while time.perf_counter() < deadline:
        if stop_event.is_set():
            return None

        # Do not hide serial-driver failures as ordinary slave timeouts.
        waiting = int(getattr(ser, "in_waiting", 0) or 0)

        if waiting:
            chunk = ser.read(waiting)
            if chunk:
                buffer.extend(chunk)
                if len(buffer) > DISCOVERY_RX_BUFFER_LIMIT:
                    del buffer[:-DISCOVERY_RX_BUFFER_LIMIT]
                found = find_discovery_response(buffer, slave, fc)
                if found is not None:
```

MBToolkit v3.5/mb_slave_finder.py — lines 401-440

```
                    return found
        else:
            # Event.wait() keeps the Stop button responsive.
            if stop_event.wait(0.001):
                return None

    return find_discovery_response(buffer, slave, fc)

def bus_activity_detected(ser, listen_s, stop_event):
    """Listen passively for *listen_s* seconds; return the bytes received.

    Any byte, even one framed at another baud rate, means something else is
    transmitting, so the active scan must not start.
    """
    deadline = time.perf_counter() + max(0.0, float(listen_s))
    received = bytearray()
    while time.perf_counter() < deadline:
        if stop_event.is_set():
            break
        waiting = int(getattr(ser, "in_waiting", 0) or 0)
        if waiting:
            received.extend(ser.read(waiting) or b"")
            if received:
                break
        elif stop_event.wait(0.005):
            break
    return bytes(received)


def cancel_serial_io(ser):
    """Best-effort cancellation of pending serial I/O."""
    if ser is None:
        return
    for method_name in ("cancel_read", "cancel_write"):
        method = getattr(ser, method_name, None)
        if callable(method):
            try:
                method()
            except Exception:
                pass
```

MBToolkit v3.5/mb_slave_finder.py — lines 441-480

```
441
442 def recover_discovery_write(ser):
443     """Best-effort cleanup after a discovery write timeout."""
444     cancel = getattr(ser, "cancel_write", None)
445     if callable(cancel):
446         try:
447             cancel()
448         except Exception:
449             pass
450     try:
451         ser.reset_output_buffer()
452     except Exception:
453         pass
454     try:
455         ser.reset_input_buffer()
456     except Exception:
457         pass
458
459 def write_discovery_request(
460     ser, request, baud, parity, stopbits, stop_event, serial_module
461 ):
462     """
463     Transmit one discovery request robustly.
464
465     A write timeout is retried once after cancelling/clearing TX. ser.flush()
466     is deliberately avoided because some USB-serial drivers can block there.
467     The helper also waits for the request's estimated physical on-wire time so
468     the slave response timeout starts at a sensible point on slow baud rates.
469     """
470     request = bytes(request)
471     timeout_exc = getattr(serial_module, "SerialTimeoutException", TimeoutError)
472     serial_exc = getattr(serial_module, "SerialException", OSError)
473     attempts = DISCOVERY_WRITE_RETRIES + 1
474     last_error = None
475     bits_per_char = discovery_bits_per_char(parity, stopbits)
476     char_time_s = bits_per_char / float(max(1, int(baud)))
477     request_wire_s = len(request) * char_time_s
478     retry_silence_s = max(
479         0.050,
480         request_wire_s + modbus_rtu_interframe_gap_seconds(baud, bits_per_char),
```

MBToolkit v3.5/mb_slave_finder.py — lines 481-520

```
481     )
482
483     for attempt in range(attempts):
484         if stop_event.is_set():
485             return False
486
487         started_write = time.perf_counter()
488         try:
489             written_total = 0
490             while written_total < len(request):
491                 if stop_event.is_set():
492                     return False
493                 written = ser.write(request[written_total:])
494                 written = 0 if written is None else int(written)
495                 if written <= 0:
496                     raise timeout_exc(
497                         "A porta série não aceitou bytes para transmissão."
498                     )
499                 written_total += written
500
501             elapsed_write_s = time.perf_counter() - started_write
502             remaining_wire_s = max(0.0, request_wire_s - elapsed_write_s)
503             if remaining_wire_s > 0 and stop_event.wait(remaining_wire_s):
504                 return False
505             return True
506
507         except timeout_exc as exc:
508             last_error = exc
509             recover_discovery_write(ser)
510             if attempt + 1 >= attempts or stop_event.is_set():
511                 raise
512             # If any fragment reached the wire before the driver timed out,
513             # leave enough silence for that fragment to finish and for a full
514             # Modbus RTU frame gap before retrying.
515             if stop_event.wait(retry_silence_s):
516                 return False
517
518         except serial_exc:
519             raise
520
```

MBToolkit v3.5/mb_slave_finder.py — lines 521-560

```
521     if last_error is not None:
522         raise last_error
523     return False
524 
525 def format_duration(seconds: float) -> str:
526     total = max(0, int(round(float(seconds))))
527     hours, rem = divmod(total, 3600)
528     minutes, secs = divmod(rem, 60)
529     if hours:
530         return f"{hours:d}:{minutes:02d}:{secs:02d}"
531     return f"{minutes:02d}:{secs:02d}"
532 
533 
534 class SlaveFinderMixin:
535     """Methods that implement the active Bus Slave Finder tab."""
536 
537     def build_discovery_tab(self, page):
538         """Build the full-window active Modbus RTU slave-discovery workspace."""
539         finder_saved = getattr(self, "ui_settings", {}).get(
540             "bus_slave_finder", {}
541         )
542         if not isinstance(finder_saved, dict):
543             finder_saved = {}
544 
545         saved_bauds = set(
546             finder_saved.get("baud_rates", [9600, 19200])
547         )
548         saved_parities = set(
549             finder_saved.get("parities", ["None", "Even"])
550         )
551         saved_stops = set(
552             finder_saved.get("stop_bits", ["1"])
553         )
554 
555         page.columnconfigure(0, weight=1)
556         page.rowconfigure(3, weight=1)
557 
558         warning = ttk.LabelFrame(page, text="Pesquisa ativa — transmite no barramento")
559         warning.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
560         warning.columnconfigure(0, weight=1)
```

MBToolkit v3.5/mb_slave_finder.py — lines 561-600

```
        ttk.Label(
            warning,
            text=(
                "Esta ferramenta atua temporariamente como master Modbus RTU e envia apenas "
                "pedidos de leitura. Não a utilizes com outro master ativo no mesmo barramento."
            ),
            wraplength=1100,
        ).grid(row=0, column=0, sticky="w", padx=10, pady=(8, 4))

        self.discovery_safety_var = tk.BooleanVar(value=False)
        safety_cb = ttk.Checkbutton(
            warning,
            text="Confirmo que não existe outro master ativo no barramento.",
            variable=self.discovery_safety_var,
            command=self.update_discovery_start_state,
        )
        safety_cb.grid(row=1, column=0, sticky="w", padx=10, pady=(0, 8))

        options = ttk.LabelFrame(page, text="Configuração da pesquisa")
        self._discovery_options_frame = options
        options.grid(row=1, column=0, sticky="ew", padx=12, pady=6)
        options.columnconfigure(0, weight=1)

        top_row = ttk.Frame(options)
        top_row.grid(row=0, column=0, sticky="ew", padx=8, pady=(8, 4))
        top_row.columnconfigure(3, weight=1)

        ttk.Label(top_row, text="Porta COM").grid(
            row=0, column=0, sticky="w", padx=(0, 4)
        )
        self.discovery_port_var = tk.StringVar()
        self.discovery_port_combo = ttk.Combobox(
            top_row,
            textvariable=self.discovery_port_var,
            width=13,
            state="readonly",
        )
        self.discovery_port_combo.grid(
            row=0, column=1, sticky="w", padx=(0, 0)
        )
```

MBToolkit v3.5/mb_slave_finder.py — lines 601-640

```
601         self.discovery_port_combo.bind(
602             "<<ComboboxSelected>>",
603             lambda _e: self.update_discovery_start_state(),
604         )
605 
606         self.discovery_refresh_btn = RefreshButton(
607             top_row,
608             command=self.refresh_ports,
609             size=24,
610         )
611         self.discovery_refresh_btn.grid(
612             row=0, column=2, sticky="w", padx=(4, 0)
613         )
614 
615         def _sync_discovery_refresh_square(_event=None):
616             height = self.discovery_port_combo.winfo_height()
617             if height > 1:
618                 self.discovery_refresh_btn.configure(
619                     width=height, height=height
620                 )
621 
622         self.discovery_port_combo.bind(
623             "<Configure>", _sync_discovery_refresh_square, add="+"
624         )
625 
626         ttk.Label(
627             top_row,
628             text="Data bits: 8 (fixo)",
629         ).grid(row=0, column=4, sticky="w", padx=(28, 20))
630         ttk.Label(
631             top_row,
632             text="Probe: FC03 / Address 0 / Qty 1",
633         ).grid(row=0, column=5, sticky="w")
634 
635         ttk.Label(top_row, text="Tentativas").grid(
636             row=0, column=6, sticky="w", padx=(28, 4)
637         )
638         self.discovery_attempts_var = tk.StringVar(
639             value=finder_saved.get("attempts", str(DISCOVERY_DEFAULT_ATTEMPTS))
640         )
```

MBToolkit v3.5/mb_slave_finder.py — lines 641-680

```
        self.discovery_attempts_spin = ttk.Spinbox(
            top_row,
            from_=DISCOVERY_MIN_ATTEMPTS,
            to=DISCOVERY_MAX_ATTEMPTS,
            increment=1,
            width=4,
            textvariable=self.discovery_attempts_var,
        )
        self.discovery_attempts_spin.grid(row=0, column=7, sticky="w")

        # Baud-rate checkboxes. Defaults keep a normal industrial scan quick.
        baud_frame = ttk.LabelFrame(options, text="Baud rate")
        baud_frame.grid(row=1, column=0, sticky="ew", padx=8, pady=4)
        baud_frame.rowconfigure(0, weight=1)
        self.discovery_baud_vars = {}
        for idx in range(len(DISCOVERY_BAUD_RATES)):
            baud_frame.columnconfigure(idx, weight=1, uniform="baud")
        for idx, baud in enumerate(DISCOVERY_BAUD_RATES):
            var = tk.BooleanVar(value=baud in saved_bauds)
            self.discovery_baud_vars[baud] = var
            cb = ttk.Checkbutton(baud_frame, text=str(baud), variable=var)
            cb.grid(row=0, column=idx, sticky="", padx=4, pady=(8, 7))
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())

        serial_frame = ttk.Frame(options)
        serial_frame.grid(row=2, column=0, sticky="ew", padx=8, pady=4)
        # Equal thirds independent of translated child captions.
        for column in range(3):
            serial_frame.columnconfigure(
                column, weight=1, uniform="finder_serial_columns"
            )

        parity_box = ttk.LabelFrame(serial_frame, text="Parity")
        parity_box.grid(row=0, column=0, sticky="nsew", padx=(0, 5))
        parity_box.rowconfigure(0, weight=1)
        for idx in range(len(DISCOVERY_PARITIES)):
            parity_box.columnconfigure(idx, weight=1, uniform="parity")
        self.discovery_parity_vars = {}
        for idx, parity in enumerate(DISCOVERY_PARITIES):
            var = tk.BooleanVar(value=parity in saved_parities)
```

MBToolkit v3.5/mb_slave_finder.py — lines 681-720

```
            self.discovery_parity_vars[parity] = var
            ttk.Checkbutton(parity_box, text=parity, variable=var).grid(
                row=0, column=idx, sticky="", padx=4, pady=(8, 7)
            )
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())
        stop_box = ttk.LabelFrame(serial_frame, text="Stop bits")
        stop_box.grid(row=0, column=1, sticky="nsew", padx=5)
        stop_box.rowconfigure(0, weight=1)
        for idx in range(len(DISCOVERY_STOP_BITS)):
            stop_box.columnconfigure(idx, weight=1, uniform="stop")
        self.discovery_stop_vars = {}
        for idx, stopbits in enumerate(DISCOVERY_STOP_BITS):
            var = tk.BooleanVar(value=stopbits in saved_stops)
            self.discovery_stop_vars[stopbits] = var
            ttk.Checkbutton(stop_box, text=stopbits, variable=var).grid(
                row=0, column=idx, sticky="", padx=4, pady=(8, 7)
            )
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())
        range_box = ttk.LabelFrame(serial_frame, text="Slave IDs / velocidade")
        range_box.grid(row=0, column=2, sticky="nsew", padx=(5, 0))
        range_box.columnconfigure(0, weight=1)
        range_box.rowconfigure(0, weight=1)

        # Keep the range and timeout controls on one centred line so this
        # LabelFrame stays as compact as the neighbouring Parity/Stop bits
        # boxes.  The option checkboxes keep their own second line so the long
        # Device Identification caption remains fully inside the rectangle.
        range_content = ttk.Frame(range_box)
        range_content.grid(
            row=0, column=0, sticky="nsew", padx=8, pady=(8, 7)
        )
        range_content.columnconfigure(0, weight=1)
        for row in range(2):
            range_content.rowconfigure(row, weight=1, uniform="range_rows")

        id_row = ttk.Frame(range_content)
        id_row.grid(row=0, column=0, sticky="")
        ttk.Label(id_row, text="De").grid(row=0, column=0, padx=(0, 3))
        self.discovery_slave_start_var = tk.StringVar(value=finder_saved.get("slave_start", "1"))
        self.discovery_slave_start_spin = ttk.Spinbox(
```

MBToolkit v3.5/mb_slave_finder.py — lines 721-760

```
            id_row, from_=1, to=247, width=5, textvariable=self.discovery_slave_start_var
        )
        self.discovery_slave_start_spin.grid(row=0, column=1, padx=3)
        ttk.Label(id_row, text="até").grid(row=0, column=2, padx=3)
        self.discovery_slave_end_var = tk.StringVar(value=finder_saved.get("slave_end", "247"))
        self.discovery_slave_end_spin = ttk.Spinbox(
            id_row, from_=1, to=247, width=5, textvariable=self.discovery_slave_end_var
        )
        self.discovery_slave_end_spin.grid(row=0, column=3, padx=3)

        ttk.Label(id_row, text="Timeout mín.").grid(
            row=0, column=4, padx=(14, 3)
        )
        self.discovery_min_timeout_var = tk.StringVar(
            value=finder_saved.get(
                "min_timeout_ms",
                str(DISCOVERY_DEFAULT_MIN_TIMEOUT_MS),
            )
        )
        self.discovery_min_timeout_spin = ttk.Spinbox(
            id_row,
            from_=DISCOVERY_MIN_TIMEOUT_MS,
            to=DISCOVERY_MAX_TIMEOUT_MS,
            increment=5,
            width=6,
            textvariable=self.discovery_min_timeout_var,
        )
        self.discovery_min_timeout_spin.grid(row=0, column=5, padx=3)
        ttk.Label(id_row, text="ms").grid(row=0, column=6, padx=3)

        toggles = ttk.Frame(range_content)
        # Small vertical separation from the Slave range/timeout row.
        # Keep this deliberately modest so the panel remains compact.
        toggles.grid(row=1, column=0, sticky="", pady=(3, 0))

        self.discovery_fc04_fallback_var = tk.BooleanVar(value=finder_saved.get("fc04_fallback", False))
        fallback_cb = ttk.Checkbutton(
            toggles,
            text="FC04 fallback",
            variable=self.discovery_fc04_fallback_var,
```

MBToolkit v3.5/mb_slave_finder.py — lines 761-800

```
        )
        fallback_cb.grid(row=0, column=0, sticky="w", padx=(0, 10))

        self.discovery_device_identification_var = tk.BooleanVar(
            value=finder_saved.get("device_identification", False)
        )
        device_id_cb = ttk.Checkbutton(
            toggles,
            text="Device Identification (FC43/14)",
            variable=self.discovery_device_identification_var,
        )
        device_id_cb.grid(row=0, column=1, sticky="w")

        for var in (
            self.discovery_slave_start_var,
            self.discovery_slave_end_var,
            self.discovery_min_timeout_var,
            self.discovery_attempts_var,
            self.discovery_fc04_fallback_var,
            self.discovery_device_identification_var,
        ):
            var.trace_add("write", lambda *_a: self.on_discovery_options_changed())

        ttk.Label(
            options,
            text=(
                "FC03 é o modo rápido. Uma resposta normal OU uma Modbus Exception com CRC válido "
                "conta como slave encontrado. Ativa FC04 fallback apenas para dispositivos que "
                "possam ignorar FC03 sem devolver Exception. Device Identification envia FC43/14 "
                "apenas aos slaves encontrados e tenta ler VendorName, ProductCode e Revision. "
                "Tentativas repete cada pedido sem resposta. Antes de transmitir, a pesquisa escuta "
                "o barramento durante 1 s e não arranca se detetar tráfego."
            ),
            wraplength=1120,
        ).grid(row=3, column=0, sticky="w", padx=10, pady=(4, 8))

        controls = ttk.LabelFrame(page, text="Pesquisa")
        controls.grid(row=2, column=0, sticky="ew", padx=12, pady=6)
        controls.columnconfigure(4, weight=1)

```

MBToolkit v3.5/mb_slave_finder.py — lines 801-840

```
        self.discovery_start_btn = ttk.Button(
            controls, text="Iniciar pesquisa", command=self.start_discovery_scan
        )
        self.discovery_start_btn.grid(row=0, column=0, padx=(10, 5), pady=8)
        self.discovery_stop_btn = ttk.Button(
            controls, text="Parar", command=self.stop_discovery_scan, state="disabled"
        )
        self.discovery_stop_btn.grid(row=0, column=1, padx=5, pady=8)
        self.discovery_clear_btn = ttk.Button(
            controls, text="Limpar resultados", command=self.clear_discovery_results
        )
        self.discovery_clear_btn.grid(row=0, column=2, padx=5, pady=8)

        self.discovery_found_var = tk.StringVar(value="Encontrados: 0")
        ttk.Label(controls, textvariable=self.discovery_found_var).grid(
            row=0, column=3, padx=(18, 8), pady=8, sticky="w"
        )

        self.discovery_status_var = tk.StringVar(value="Parado")
        ttk.Label(controls, textvariable=self.discovery_status_var).grid(
            row=0, column=4, padx=8, pady=8, sticky="w"
        )

        self.discovery_progress_var = tk.DoubleVar(value=0.0)
        self.discovery_progress = ttk.Progressbar(
            controls, variable=self.discovery_progress_var, maximum=100.0
        )
        self.discovery_progress.grid(
            row=1, column=0, columnspan=5, sticky="ew", padx=10, pady=(0, 5)
        )

        self.discovery_progress_text_var = tk.StringVar(value="0 / 0")
        self.discovery_estimate_var = tk.StringVar(value="")
        ttk.Label(controls, textvariable=self.discovery_progress_text_var).grid(
            row=2, column=0, columnspan=2, sticky="w", padx=10, pady=(0, 8)
        )
        ttk.Label(controls, textvariable=self.discovery_estimate_var).grid(
            row=2, column=2, columnspan=3, sticky="e", padx=10, pady=(0, 8)
        )

```

MBToolkit v3.5/mb_slave_finder.py — lines 841-880

```
        results = ttk.LabelFrame(page, text="Slaves encontrados")
        results.grid(row=3, column=0, sticky="nsew", padx=12, pady=(6, 12))
        results.columnconfigure(0, weight=1)
        results.rowconfigure(0, weight=1)

        columns = (
            "slave", "baud", "config", "fc", "result", "resp", "device_id", "raw"
        )
        self.discovery_tree = ttk.Treeview(
            results, columns=columns, show="headings", selectmode="browse"
        )
        headings = {
            "slave": "Slave",
            "baud": "Baud",
            "config": "Config.",
            "fc": "FC",
            "result": "Resultado",
            "resp": "Resp. (ms)",
            "device_id": "Device Identification",
            "raw": "Raw Hex",
        }
        # Match the Traffic table behaviour: no horizontal scrollbar. Keep
        # short fields compact and let the three text-heavy columns share the
        # remaining viewport width. Column widths are recalculated whenever
        # the Treeview is resized so the complete table remains visible.
        self.discovery_fixed_widths = {
            "slave": 70,
            "baud": 90,
            "config": 85,
            "fc": 55,
            "resp": 95,
        }
        self.discovery_flex_columns = ("result", "device_id", "raw")
        self.discovery_flex_weights = {
            "result": 0.27,
            "device_id": 0.38,
            "raw": 0.35,
        }
        self.discovery_flex_min_width = 90

```

MBToolkit v3.5/mb_slave_finder.py — lines 881-920

```
        initial_widths = {
            **self.discovery_fixed_widths,
            "result": 220,
            "device_id": 330,
            "raw": 300,
        }
        for col in columns:
            self.discovery_tree.heading(col, text=headings[col], anchor="center")
            self.discovery_tree.column(
                col,
                width=initial_widths[col],
                minwidth=(
                    self.discovery_flex_min_width
                    if col in self.discovery_flex_columns
                    else self.discovery_fixed_widths[col]
                ),
                anchor="w" if col in self.discovery_flex_columns else "center",
                stretch=False,
            )

        discovery_y = ttk.Scrollbar(
            results, orient="vertical", command=self.discovery_tree.yview
        )
        self.discovery_tree.configure(yscrollcommand=discovery_y.set)
        self.discovery_tree.grid(row=0, column=0, sticky="nsew")
        discovery_y.grid(row=0, column=1, sticky="ns")
        self.discovery_tree.bind(
            "<Configure>", self.fit_discovery_columns_to_viewport, add="+"
        )
        # Right-click / double-click a found Slave to read it in the Master tab.
        self.discovery_context_menu = tk.Menu(self, tearoff=False)
        self.discovery_context_menu.add_command(
            label="Abrir no Master", command=self.discovery_open_selected_in_master
        )
        self.discovery_tree.bind("<Button-3>", self._show_discovery_context_menu)
        self.discovery_tree.bind(
            "<Double-1>", lambda _e: self.discovery_open_selected_in_master()
        )

        self.update_discovery_estimate()
```

MBToolkit v3.5/mb_slave_finder.py — lines 921-960

```
        self.update_discovery_start_state()

    def _show_discovery_context_menu(self, event):
        row = self.discovery_tree.identify_row(event.y)
        if not row:
            return
        self.discovery_tree.selection_set(row)
        try:
            self.discovery_context_menu.entryconfigure(0, label=self.tr("Abrir no Master"))
            self.discovery_context_menu.tk_popup(event.x_root, event.y_root)
        finally:
            self.discovery_context_menu.grab_release()

    def discovery_open_selected_in_master(self):
        """Open the selected found Slave in a new Master window."""
        selection = self.discovery_tree.selection()
        if not selection:
            return
        values = self.discovery_tree.item(selection[0], "values")
        try:
            slave, baud, config, fc = int(values[0]), str(values[1]), str(values[2]), int(values[3])
        except (IndexError, ValueError):
            return
        parity = {"N": "None", "E": "Even", "O": "Odd"}.get(config[1:2], "None")
        stopbits = config[2:] or "1"
        self.master_open_definition(
            {"slave": slave, "fc": fc, "address": 0, "qty": 10},
            port=self.discovery_port_var.get().strip(),
            baud=baud, parity=parity, stopbits=stopbits,
        )

    def fit_discovery_columns_to_viewport(self, _event=None):
        """Fit Slave Finder columns to the visible Treeview width."""
        if not hasattr(self, "discovery_tree"):
            return

        viewport_width = int(self.discovery_tree.winfo_width())
        if viewport_width <= 1:
            return

```

MBToolkit v3.5/mb_slave_finder.py — lines 961-1000

```
        # Reserve a few pixels for Treeview borders so Tk never needs a hidden
        # horizontal overflow area. The vertical scrollbar is a sibling and is
        # therefore already excluded from winfo_width().
        usable_width = max(1, viewport_width - 4)
        fixed_total = sum(self.discovery_fixed_widths.values())
        flex_count = len(self.discovery_flex_columns)
        min_flex_total = self.discovery_flex_min_width * flex_count

        flex_total = max(min_flex_total, usable_width - fixed_total)
        allocated = 0
        for index, col in enumerate(self.discovery_flex_columns):
            if index == flex_count - 1:
                width = flex_total - allocated
            else:
                width = int(round(flex_total * self.discovery_flex_weights[col]))
                allocated += width
            width = max(self.discovery_flex_min_width, width)
            self.discovery_tree.column(col, width=width)

        for col, width in self.discovery_fixed_widths.items():
            self.discovery_tree.column(col, width=width)

    def on_discovery_options_changed(self):
        if hasattr(self, "discovery_estimate_var"):
            self.update_discovery_estimate()
            self.update_discovery_start_state()

    def selected_discovery_configs(self):
        bauds = [baud for baud in DISCOVERY_BAUD_RATES if self.discovery_baud_vars[baud].get()]
        parities = [p for p in DISCOVERY_PARITIES if self.discovery_parity_vars[p].get()]
        stops = [s for s in DISCOVERY_STOP_BITS if self.discovery_stop_vars[s].get()]
        return [
            (baud, parity, stopbits)
            for baud in bauds
            for parity in parities
            for stopbits in stops
        ]

    def selected_discovery_slaves(self):
        start = int(self.discovery_slave_start_var.get())
```

MBToolkit v3.5/mb_slave_finder.py — lines 1001-1040

```
        end = int(self.discovery_slave_end_var.get())
        if not (1 <= start <= 247 and 1 <= end <= 247):
            raise ValueError(self.tr("Os Slave IDs têm de estar entre 1 e 247."))
        if start > end:
            raise ValueError(self.tr("O Slave ID inicial não pode ser superior ao final."))
        return list(range(start, end + 1))

    def selected_discovery_min_timeout_ms(self):
        value = float(self.discovery_min_timeout_var.get())
        if not (DISCOVERY_MIN_TIMEOUT_MS <= value <= DISCOVERY_MAX_TIMEOUT_MS):
            raise ValueError(
                self.trf(
                    "O timeout mínimo tem de estar entre {minimum} e {maximum} ms.",
                    minimum=DISCOVERY_MIN_TIMEOUT_MS,
                    maximum=DISCOVERY_MAX_TIMEOUT_MS,
                )
            )
        return value

    def selected_discovery_attempts(self):
        value = int(self.discovery_attempts_var.get())
        if not (DISCOVERY_MIN_ATTEMPTS <= value <= DISCOVERY_MAX_ATTEMPTS):
            raise ValueError(
                self.trf(
                    "Tentativas tem de estar entre {minimum} e {maximum}.",
                    minimum=DISCOVERY_MIN_ATTEMPTS,
                    maximum=DISCOVERY_MAX_ATTEMPTS,
                )
            )
        return value

    def discovery_config_label(self, parity, stopbits):
        parity_letter = {"None": "N", "Even": "E", "Odd": "O"}[parity]
        return f"8{parity_letter}{stopbits}"

    def update_discovery_estimate(self):
        try:
            configs = self.selected_discovery_configs()
            slaves = self.selected_discovery_slaves()
            minimum_timeout_ms = self.selected_discovery_min_timeout_ms()
```

MBToolkit v3.5/mb_slave_finder.py — lines 1041-1080

```
            attempts = self.selected_discovery_attempts()
        except Exception:
            self.discovery_estimate_var.set(self.tr("Máx. estimado: —"))
            self.discovery_progress_text_var.set("0 / 0")
            return

        total_units = len(configs) * len(slaves)
        probes_per_unit = 2 if self.discovery_fc04_fallback_var.get() else 1
        identify_devices = bool(self.discovery_device_identification_var.get())

        seconds = 0.0
        for baud, parity, stopbits in configs:
            per_probe = (
                discovery_request_transmit_seconds(baud, parity, stopbits)
                + discovery_response_timeout_seconds(
                    baud, parity, stopbits, minimum_timeout_ms
                )
            )
            # Worst case: no Slave answers, so every probe uses all attempts.
            seconds += len(slaves) * probes_per_unit * attempts * per_probe
            if identify_devices:
                # Worst case: every probed Slave exists and therefore receives
                # one optional Basic Device Identification query.
                seconds += len(slaves) * (
                    device_identification_request_transmit_seconds(
                        baud, parity, stopbits
                    )
                    + device_identification_timeout_seconds(
                        baud, parity, stopbits, minimum_timeout_ms
                    )
                )
            seconds += 0.025  # small reopen/reconfigure allowance
        seconds += DISCOVERY_IDLE_CHECK_SECONDS if configs else 0.0

        self.discovery_progress_text_var.set(f"0 / {total_units}")
        suffix_parts = []
        if probes_per_unit == 2:
            suffix_parts.append("FC04 fallback")
        if identify_devices:
            suffix_parts.append("Device Identification")
```

MBToolkit v3.5/mb_slave_finder.py — lines 1081-1120

```
        suffix = (
            self.trf(" (com {options})", options=" + ".join(suffix_parts))
            if suffix_parts else ""
        )
        self.discovery_estimate_var.set(
            self.trf(
                "{configs} config. × {slaves} slaves — máx. estimado: {duration}",
                configs=len(configs),
                slaves=len(slaves),
                duration=format_duration(seconds),
            )
            + suffix
        )

    def update_discovery_start_state(self):
        if not hasattr(self, "discovery_start_btn"):
            return
        enabled = (
            not self.discovery_active
            and not self.busy
            and bool(self.discovery_port_var.get().strip())
            and bool(self.discovery_safety_var.get())
        )
        try:
            enabled = enabled and bool(self.selected_discovery_configs())
            self.selected_discovery_slaves()
            self.selected_discovery_min_timeout_ms()
            self.selected_discovery_attempts()
        except Exception:
            enabled = False
        self.discovery_start_btn.configure(state="normal" if enabled else "disabled")

    def clear_discovery_results(self):
        if self.discovery_active:
            return
        for iid in self.discovery_tree.get_children():
            self.discovery_tree.delete(iid)
        self.discovery_found_count = 0
        self.discovery_found_var.set(self.trf("Encontrados: {count}", count=0))
        self.discovery_progress_var.set(0.0)
```

MBToolkit v3.5/mb_slave_finder.py — lines 1121-1160

```
1121        self.discovery_status_var.set(self.tr("Parado"))
1122        self.update_discovery_estimate()
1123
1124    def open_discovery_serial_port(self, serial, port, baud, parity, stopbits):
1125        parity_map = {
1126            "None": serial.PARITY_NONE,
1127            "Even": serial.PARITY_EVEN,
1128            "Odd": serial.PARITY_ODD,
1129        }
1130        stop_map = {
1131            "1": serial.STOPBITS_ONE,
1132            "2": serial.STOPBITS_TWO,
1133        }
1134
1135        ser = serial.Serial()
1136        ser.port = port
1137        ser.baudrate = int(baud)
1138        ser.bytesize = serial.EIGHTBITS
1139        ser.parity = parity_map[parity]
1140        ser.stopbits = stop_map[stopbits]
1141        ser.timeout = 0
1142
1143        ser.write_timeout = DISCOVERY_WRITE_TIMEOUT_SECONDS
1144        ser.xonxoff = False
1145        ser.rtscts = False
1146        ser.dsrdtr = False
1147
1148        # Do not force RTS/DTR levels. Many USB-RS485 adapters manage direction
1149        # internally and forcing modem-control lines can interfere with some
1150        # adapters/drivers. Hardware flow control is disabled above.
1151        ser.open()
1152
1153        try:
1154            ser.reset_input_buffer()
1155        except Exception:
1156            pass
1157        try:
1158            ser.reset_output_buffer()
1159        except Exception:
1160            pass
```

MBToolkit v3.5/mb_slave_finder.py — lines 1161-1200

```
1161
1162        return ser
1163
1164    def start_discovery_scan(self):
1165        if self.busy or self.discovery_active:
1166            return
1167
1168        serial, _ = self.get_serial_modules()
1169        if serial is None:
1170            messagebox.showerror(
1171                self.tr("pyserial em falta"),
1172                self.tr("pyserial não está instalado. O build_exe.bat inclui-o automaticamente no EXE.")
1173            )
1174            return
1175
1176        if not self.discovery_safety_var.get():
1177            messagebox.showwarning(
1178                self.tr("Pesquisa ativa"),
1179                self.tr("Confirma primeiro que não existe outro master ativo no barramento.")
1180            )
1181            return
1182
1183        try:
1184            port = self.discovery_port_var.get().strip()
1185            if not port:
1186                raise ValueError(self.tr("Seleciona uma porta COM."))
1187            configs = self.selected_discovery_configs()
1188            if not configs:
1189                raise ValueError(self.tr("Seleciona pelo menos uma configuração série."))
1190            slaves = self.selected_discovery_slaves()
1191            minimum_timeout_ms = self.selected_discovery_min_timeout_ms()
1192            attempts = self.selected_discovery_attempts()
1193        except Exception as exc:
1194            messagebox.showerror(self.tr("Configuração inválida"), str(exc))
1195            return
1196
1197        fallback_fc04 = bool(self.discovery_fc04_fallback_var.get())
1198        identify_devices = bool(self.discovery_device_identification_var.get())
1199
1200        self.clear_discovery_results()
```

MBToolkit v3.5/mb_slave_finder.py — lines 1201-1240

```
        self.discovery_active = True
        self.busy = True
        self.discovery_stop_event.clear()
        self.discovery_found_count = 0
        self.discovery_progress_var.set(0.0)
        self.discovery_status_var.set(self.tr("A iniciar pesquisa…"))

        self.discovery_start_btn.configure(state="disabled")
        self.discovery_stop_btn.configure(state="normal")
        self.discovery_clear_btn.configure(state="disabled")
        self.discovery_refresh_btn.configure(state="disabled")
        self.discovery_port_combo.configure(state="disabled")

        # Prevent passive capture/simulation from being started concurrently.
        self.start_btn.configure(state="disabled")
        self.stop_btn.configure(state="disabled")
        self.restart_btn.configure(state="disabled")
        self.set_sniffer_refresh_state("disabled")
        self.set_simulation_button_state("disabled")

        immutable_configs = list(configs)
        immutable_slaves = list(slaves)

        self.discovery_thread = threading.Thread(
            target=self.discovery_worker,
            args=(
                serial, port, immutable_configs, immutable_slaves,
                minimum_timeout_ms, fallback_fc04, identify_devices, attempts
            ),
            daemon=True,
        )
        try:
            self.discovery_thread.start()
        except Exception as exc:
            # Restore the interface if the worker cannot even be started.
            self.discovery_active = False
            self.busy = False
            self.discovery_thread = None
            self.discovery_stop_btn.configure(state="disabled")
            self.discovery_clear_btn.configure(state="normal")
```

MBToolkit v3.5/mb_slave_finder.py — lines 1241-1280

```
            self.discovery_refresh_btn.configure(state="normal")
            self.discovery_port_combo.configure(state="readonly")
            self.start_btn.configure(state="normal")
            self.restart_btn.configure(state="normal")
            self.set_sniffer_refresh_state("normal")
            self.set_simulation_button_state("normal")
            self.discovery_status_var.set(self.tr("Erro ao iniciar pesquisa"))
            self.update_discovery_start_state()
            messagebox.showerror(
                self.tr("Erro na Bus Slave Finder"),
                self.trf(
                    "Não foi possível iniciar a thread de pesquisa:\n\n{error}",
                    error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                )
            )

    def stop_discovery_scan(self):
        if self.discovery_active:
            self.discovery_status_var.set(self.tr("A parar…"))
            self.discovery_stop_event.set()
            self.discovery_stop_btn.configure(state="disabled")
            # Wake/cancel pending driver I/O without closing the handle from
            # the Tk thread while the discovery worker may still be using it.
            cancel_serial_io(self.discovery_serial)

    def discovery_worker(
        self, serial, port, configs, slaves, minimum_timeout_ms, fallback_fc04,
        identify_devices, attempts=DISCOVERY_DEFAULT_ATTEMPTS
    ):
        started = time.perf_counter()
        total_units = len(configs) * len(slaves)
        completed = 0
        found_count = 0
        fatal_error = None
        context = str(port)
        idle_checked = False

        try:
            for baud, parity, stopbits in configs:
                if self.discovery_stop_event.is_set():
```

MBToolkit v3.5/mb_slave_finder.py — lines 1281-1320

```
                    break

                config_label = self.discovery_config_label(parity, stopbits)
                context = f"{port} — {baud} / {config_label}"
                self.event_queue.put((
                    "discovery_status",
                    self.trf(
                        "A configurar {baud} / {config}…",
                        baud=baud,
                        config=config_label,
                    )
                ))

                ser = None
                try:
                    try:
                        ser = self.open_discovery_serial_port(
                            serial, port, baud, parity, stopbits
                        )
                    except Exception as exc:
                        raise RuntimeError(
                            self.trf(
                                "Não foi possível abrir {port} em {baud} / {config}: {error}",
                                port=port,
                                baud=baud,
                                config=config_label,
                                error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                            )
                        ) from exc

                    self.discovery_serial = ser
                    if self.discovery_stop_event.wait(0.025):
                        break

                    if not idle_checked:
                        # Last line of defence behind the safety checkbox:
                        # never transmit onto a bus that is already busy.
                        idle_checked = True
                        self.event_queue.put((
                            "discovery_status",
```

MBToolkit v3.5/mb_slave_finder.py — lines 1321-1360

```
                            self.tr("A verificar se o barramento está livre…"),
                        ))
                        traffic = bus_activity_detected(
                            ser,
                            DISCOVERY_IDLE_CHECK_SECONDS,
                            self.discovery_stop_event,
                        )
                        if traffic:
                            raise RuntimeError(
                                self.trf(
                                    "Foi detetado tráfego em {port} antes de transmitir "
                                    "({count} bytes: {raw}). Pode existir outro master "
                                    "ativo no barramento; a pesquisa não foi iniciada.",
                                    port=port,
                                    count=len(traffic),
                                    raw=hex_bytes(traffic[:16]),
                                )
                            )
                        if self.discovery_stop_event.is_set():
                            break
                        self.event_queue.put((
                            "discovery_status",
                            self.trf(
                                "A configurar {baud} / {config}…",
                                baud=baud,
                                config=config_label,
                            ),
                        ))

                    response_timeout_s = discovery_response_timeout_seconds(
                        baud, parity, stopbits, minimum_timeout_ms
                    )
                    char_gap_s = modbus_rtu_interframe_gap_seconds(
                        baud, discovery_bits_per_char(parity, stopbits)
                    )

                    for slave in slaves:
                        if self.discovery_stop_event.is_set():
                            break

```

MBToolkit v3.5/mb_slave_finder.py — lines 1361-1400

```
                    result = None
                    used_fc = None
                    elapsed_ms = None
                    probe_fcs = [3, 4] if fallback_fc04 else [3]

                    probe_plan = [
                        (fc, attempt)
                        for fc in probe_fcs
                        for attempt in range(1, attempts + 1)
                    ]
                    for fc, attempt in probe_plan:
                        if self.discovery_stop_event.is_set():
                            break

                        context = (
                            f"{port} — {baud} / {config_label} / "
                            f"Slave {slave} / FC{fc:02X}"
                        )
                        if attempts > 1:
                            context += f" / {attempt}/{attempts}"

                        # If this fails, the COM/driver is unhealthy. Do not
                        # disguise it as a normal no-response slave timeout.
                        try:
                            ser.reset_input_buffer()
                        except Exception as exc:
                            raise RuntimeError(
                                self.trf(
                                    "Falha ao preparar a receção em {context}: {error}",
                                    context=context,
                                    error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                )
                            ) from exc

                        request = build_discovery_probe(slave, fc)
                        started_probe = time.perf_counter()
                        try:
                            sent = write_discovery_request(
                                ser, request, baud, parity, stopbits,
                                self.discovery_stop_event, serial
```

MBToolkit v3.5/mb_slave_finder.py — lines 1401-1440

```
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha ao transmitir em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc

                            if not sent or self.discovery_stop_event.is_set():
                                break

                            try:
                                result = read_discovery_response(
                                    ser, slave, fc, response_timeout_s,
                                    self.discovery_stop_event
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha ao receber em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc

                            if result is not None:
                                elapsed_ms = (
                                    time.perf_counter() - started_probe
                                ) * 1000.0
                                used_fc = fc
                                if char_gap_s > 0:
                                    self.discovery_stop_event.wait(
                                        min(char_gap_s, 0.05)
                                    )
                                break

                        if result is not None and used_fc is not None:
                            found_count += 1
```

MBToolkit v3.5/mb_slave_finder.py — lines 1441-1480

```
                        if result["kind"] == "EXCEPTION":
                            code = result["exception_code"]
                            name = MODBUS_EXCEPTION_CODES.get(
                                code, ("Unknown/Reserved Exception", "")
                            )[0]
                            result_text = f"Exception 0x{code:02X} – {name}"
                        else:
                            result_text = "Response"

                        device_id_text = ""
                        if identify_devices and not self.discovery_stop_event.is_set():
                            context = (
                                f"{port} – {baud} / {config_label} / "
                                f"Slave {slave} / FC43/14"
                            )
                            try:
                                device_result = read_basic_device_identification(
                                    ser,
                                    slave,
                                    baud,
                                    parity,
                                    stopbits,
                                    minimum_timeout_ms,
                                    self.discovery_stop_event,
                                    serial,
                                )
                            except Exception as exc:
                                raise RuntimeError(
                                    self.trf(
                                        "Falha em Device Identification em {context}: {error}",
                                        context=context,
                                        error=f"{type(exc).__name__}: {self.tr(str(exc))}",
                                    )
                                ) from exc
                            device_id_text = format_device_identification(
                                device_result
                            )
                            if char_gap_s > 0:
                                self.discovery_stop_event.wait(
                                    min(char_gap_s, 0.05)
```

MBToolkit v3.5/mb_slave_finder.py — lines 1481-1520

```
                                    )

                            self.event_queue.put((
                                "discovery_found",
                                {
                                    "slave": slave,
                                    "baud": baud,
                                    "config": config_label,
                                    "fc": f"{used_fc:02X}",
                                    "result": result_text,
                                    "response_ms": elapsed_ms,
                                    "device_id": device_id_text,
                                    "raw": hex_bytes(result["frame"]),
                                }
                            ))

                        completed += 1
                        elapsed = time.perf_counter() - started
                        eta = (
                            max(
                                0.0,
                                (elapsed / completed) * (total_units - completed)
                            )
                            if completed > 0 else 0.0
                        )

                        self.event_queue.put((
                            "discovery_progress",
                            {
                                "completed": completed,
                                "total": total_units,
                                "baud": baud,
                                "config": config_label,
                                "slave": slave,
                                "elapsed": elapsed,
                                "eta": eta,
                                "found": found_count,
                            }
                        ))

```

MBToolkit v3.5/mb_slave_finder.py — lines 1521-1560

```
                finally:
                    if ser is not None:
                        cancel_serial_io(ser)
                        try:
                            ser.close()
                        except Exception:
                            pass
                    self.discovery_serial = None

        except Exception as exc:
            if not self.discovery_stop_event.is_set():
                fatal_error = str(exc) or f"{context}: {type(exc).__name__}"

        elapsed = time.perf_counter() - started
        self.event_queue.put((
            "discovery_done",
            {
                "stopped": self.discovery_stop_event.is_set(),
                "error": fatal_error,
                "elapsed": elapsed,
                "completed": completed,
                "total": total_units,
                "found": found_count,
            }
        ))

    def apply_discovery_progress(self, payload):
        completed = int(payload["completed"])
        total = max(1, int(payload["total"]))
        self.discovery_progress_var.set(100.0 * completed / total)
        self.discovery_progress_text_var.set(
            f"{completed} / {payload['total']} — "
            f"{payload['baud']} / {payload['config']} / Slave {payload['slave']}"
        )
        self.discovery_found_var.set(self.trf("Encontrados: {count}", count=payload["found"]))
        self.discovery_status_var.set(
            self.trf(
                "Decorrido {elapsed} — restante ~{remaining}",
                elapsed=format_duration(payload["elapsed"]),
                remaining=format_duration(payload["eta"]),
```

MBToolkit v3.5/mb_slave_finder.py — lines 1561-1600

```
            )
        )

    def add_discovery_result(self, payload):
        self.discovery_found_count += 1
        self.discovery_found_var.set(
            self.trf("Encontrados: {count}", count=self.discovery_found_count)
        )
        self.discovery_tree.insert(
            "", "end",
            values=(
                payload["slave"],
                payload["baud"],
                payload["config"],
                payload["fc"],
                payload["result"],
                f"{payload['response_ms']:.1f}" if payload.get("response_ms") is not None else "",
                payload.get("device_id", ""),
                payload["raw"],
            )
        )

    def finish_discovery_ui(self, payload):
        self.discovery_active = False
        self.busy = False
        self.discovery_stop_event.clear()
        self.discovery_serial = None
        self.discovery_thread = None

        self.discovery_stop_btn.configure(state="disabled")
        self.discovery_clear_btn.configure(state="normal")
        self.discovery_refresh_btn.configure(state="normal")
        self.discovery_port_combo.configure(state="readonly")

        self.start_btn.configure(state="normal")
        self.stop_btn.configure(state="disabled")
        self.restart_btn.configure(state="normal")
        self.set_sniffer_refresh_state("normal")
        self.set_simulation_button_state("normal")

```

MBToolkit v3.5/mb_slave_finder.py — lines 1601-1625

```
        if payload.get("error"):
            self.discovery_status_var.set(self.tr("Erro na pesquisa"))
            messagebox.showerror(self.tr("Erro na Bus Slave Finder"), payload["error"])
        elif payload.get("stopped"):
            self.discovery_status_var.set(
                self.trf(
                    "Pesquisa parada — {found} encontrados — {elapsed}",
                    found=payload["found"],
                    elapsed=format_duration(payload["elapsed"]),
                )
            )
        else:
            self.discovery_progress_var.set(100.0)
            self.discovery_status_var.set(
                self.trf(
                    "Pesquisa concluída — {found} encontrados — {elapsed}",
                    found=payload["found"],
                    elapsed=format_duration(payload["elapsed"]),
                )
            )

        self.update_discovery_estimate()
        self.update_discovery_start_state()
        self.update_com_status()

```

MBToolkit v3.5/mb_master.py

695 source lines.

MBToolkit v3.5/mb_master.py — lines 1-40

```python
#!/usr/bin/env python3
"""Modbus RTU master (poll) engine: frames, data formats and scheduling.

This module has no widgets. ``MasterEngine`` owns one open serial port and
serves every poll window over it: Modbus RTU allows a single request on the
bus at a time, so windows are polled in turn according to their scan rate,
with queued writes and "read once" requests served first. Results are
posted to the application's ``event_queue`` like the other workers.
"""

import math
import struct
import threading
import time
from collections import deque
from dataclasses import dataclass

from mb_config import (
    MASTER_RECONNECT_INTERVAL_SECONDS,
    MASTER_MAX_READ_BITS,
    MASTER_MAX_READ_REGISTERS,
    MASTER_MAX_WRITE_BITS,
    MASTER_MAX_WRITE_REGISTERS,
)
from mb_protocol import append_crc, crc_ok, hex_bytes, modbus_rtu_interframe_gap_seconds
from mb_slave_finder import (
    bus_activity_detected,
    cancel_serial_io,
    discovery_bits_per_char,
    write_discovery_request,
)

READ_FUNCTION_CODES = (1, 2, 3, 4)
BIT_FUNCTION_CODES = (1, 2)
WRITABLE_READ_FCS = {1: (5, 15), 3: (6, 16)}  # read window FC -> write FCs

# ---------------------------------------------------------------------------
# Data formats
# ---------------------------------------------------------------------------

```

MBToolkit v3.5/mb_master.py — lines 41-80

```
41 BYTE_ORDERS = ("ABCD", "CDAB", "BADC", "DCBA")
42 # 64-bit orders, in the same sequence: big-endian, word swap (registers in
43 # reverse order), byte swap inside each register, little-endian.
44 BYTE_ORDERS_64 = ("ABCDEFGH", "GHEFCDAB", "BADCFEHG", "HGFEDCBA")
45 WIRE_LETTERS = "ABCDEFGH"
46
47 # (key, label, registers per value).  Labels are technical terms and are not
48 # translated.  Byte order letters follow the wire order of the registers:
49 # register n = A B, register n+1 = C D, ...; "ABCD" / "ABCDEFGH" is big-endian.
50 REGISTER_FORMATS = (
51     ("int16", "Signed 16-bit", 1),
52     ("uint16", "Unsigned 16-bit", 1),
53     ("hex16", "Hex 16-bit", 1),
54     ("bin16", "Binary 16-bit", 1),
55 ) + tuple(
56     (f"{kind}_{order}", f"{label} {order}", 2)
57     for kind, label in (
58         ("int32", "Signed 32-bit"),
59         ("uint32", "Unsigned 32-bit"),
60         ("float32", "Float 32-bit"),
61     )
62     for order in BYTE_ORDERS
63 ) + tuple(
64     (f"{kind}_{order}", f"{label} {order}", 4)
65     for kind, label in (
66         ("int64", "Signed 64-bit"),
67         ("uint64", "Unsigned 64-bit"),
68         ("float64", "Double 64-bit"),
69     )
70     for order in BYTE_ORDERS_64
71 )
72 FORMAT_LABELS = {key: label for key, label, _size in REGISTER_FORMATS}
73 FORMAT_KEYS_BY_LABEL = {label: key for key, label, _size in REGISTER_FORMATS}
74 FORMAT_SIZES = {key: size for key, _label, size in REGISTER_FORMATS}
75 DEFAULT_FORMAT = "int16"
76
77 _STRUCT_CODES = {"int32": ">i", "uint32": ">I", "float32": ">f",
78                  "int64": ">q", "uint64": ">Q", "float64": ">d"}
79
80
```

MBToolkit v3.5/mb_master.py — lines 81-120

```
 81 def _value_bytes(registers, order):
 82     """Reorder the wire bytes of 2 or 4 registers into big-endian value bytes."""
 83     wire = b"".join(bytes(((r >> 8) & 0xFF, r & 0xFF)) for r in registers)
 84     return bytes(wire[WIRE_LETTERS.index(letter)] for letter in order)
 85 
 86 
 87 def _registers_from_value_bytes(value, order):
 88     wire = bytearray(len(order))
 89     for index, letter in enumerate(order):
 90         wire[WIRE_LETTERS.index(letter)] = value[index]
 91     return [(wire[i] << 8) | wire[i + 1] for i in range(0, len(wire), 2)]
 92 
 93 
 94 def format_float(value, digits=7):
 95     if math.isnan(value):
 96         return "NaN"
 97     if math.isinf(value):
 98         return "-Inf" if value < 0 else "Inf"
 99     return f"{value:.{digits}g}"
100 
101 
102 def decode_value(registers, fmt):
103     """Numeric value of one value (None for the Hex/Binary display formats)."""
104     if fmt in ("hex16", "bin16"):
105         return None
106     if FORMAT_SIZES[fmt] == 1:
107         value = registers[0] & 0xFFFF
108         return value - 0x10000 if fmt == "int16" and value & 0x8000 else value
109     kind, order = fmt.split("_")
110     return struct.unpack(_STRUCT_CODES[kind], _value_bytes(registers, order))[0]
111 
112 
113 SCALE_MAX_DECIMALS = 6
114 SCALE_MAX_UNIT_LENGTH = 12
115 
116 
117 def validate_scale(raw_min, raw_max, eng_min, eng_max, decimals=1, unit=""):
118     """Return a scale dict from user text/numbers; raise ValueError (PT text)."""
119     try:
120         values = [float(str(v).strip().replace(",", ".")) for v in
```

MBToolkit v3.5/mb_master.py — lines 121-160

```
                  (raw_min, raw_max, eng_min, eng_max)]
        decimals = int(str(decimals).strip())
    except ValueError:
        raise ValueError("A escala tem de ter números válidos.") from None
    if any(math.isnan(v) or math.isinf(v) for v in values):
        raise ValueError("A escala tem de ter números válidos.")
    if values[0] == values[1]:
        raise ValueError("O bruto mínimo e o bruto máximo têm de ser diferentes.")
    if not 0 <= decimals <= SCALE_MAX_DECIMALS:
        raise ValueError("As casas decimais têm de estar entre 0 e 6.")
    unit = str(unit or "").strip()[:SCALE_MAX_UNIT_LENGTH]
    return {"raw": [values[0], values[1]], "eng": [values[2], values[3]],
            "decimals": decimals, "unit": unit}


def scale_value(number, scale):
    """Linear map from raw units to engineering units through two points."""
    (r1, r2), (e1, e2) = scale["raw"], scale["eng"]
    return e1 + (number - r1) * (e2 - e1) / (r2 - r1)


def unscale_value(value, scale):
    """Inverse of scale_value(): engineering units back to raw units."""
    (r1, r2), (e1, e2) = scale["raw"], scale["eng"]
    if e1 == e2:
        raise ValueError("Esta escala não permite escrever: os valores de engenharia são iguais.")
    return r1 + (value - e1) * (r2 - r1) / (e2 - e1)


def format_scaled(number, scale):
    text = f"{scale_value(number, scale):.{scale['decimals']}f}"
    return f"{text} {scale['unit']}" if scale.get("unit") else text


def format_value(registers, fmt):
    """Display text of one value occupying ``FORMAT_SIZES[fmt]`` registers."""
    if FORMAT_SIZES[fmt] == 1:
        value = registers[0] & 0xFFFF
        if fmt == "int16":
            return str(value - 0x10000 if value & 0x8000 else value)
```

MBToolkit v3.5/mb_master.py — lines 161-200

```
        if fmt == "uint16":
            return str(value)
        if fmt == "hex16":
            return f"0x{value:04X}"
        return f"{value >> 8:08b} {value & 0xFF:08b}"

    kind, order = fmt.split("_")
    number = struct.unpack(_STRUCT_CODES[kind], _value_bytes(registers, order))[0]
    if kind == "float32":
        return format_float(number, 7)
    if kind == "float64":
        return format_float(number, 15)
    return str(number)


def register_groups(qty, start_address=0, default_fmt=DEFAULT_FORMAT, overrides=None):
    """Split a window's registers into values: [(offset, size, fmt or None)].

    *overrides* maps a PDU address to the format of the value starting there
    (per-address formats). Other registers use *default_fmt*. A value that
    would run past the window, or a default multi-register value cut by an
    override, becomes ``(offset, size, None)`` and is shown as "—".
    """
    overrides = overrides or {}
    default_fmt = default_fmt if default_fmt in FORMAT_SIZES else DEFAULT_FORMAT
    groups = []
    offset = 0
    while offset < qty:
        fmt = overrides.get(start_address + offset)
        if fmt not in FORMAT_SIZES:
            fmt = default_fmt
            size = FORMAT_SIZES[fmt]
            cut = next((j for j in range(offset + 1, min(offset + size, qty))
                        if start_address + j in overrides), None)
            if cut is not None:
                groups.append((offset, cut - offset, None))
                offset = cut
                continue
        size = FORMAT_SIZES[fmt]
        if offset + size > qty:
```

MBToolkit v3.5/mb_master.py — lines 201-240

```
            groups.append((offset, qty - offset, None))
            break
        groups.append((offset, size, fmt))
        offset += size
    return groups


def format_window_values(registers, start_address=0, default_fmt=DEFAULT_FORMAT,
                         overrides=None, scales=None):
    """Return one display string per register, honouring per-address formats.

    A multi-register value is shown on the row of its first register; the
    other rows of the group are empty. Incomplete groups show "—". *scales*
    maps the PDU address where a value starts to a scale (validate_scale());
    scaled values are shown in engineering units (Hex/Binary stay raw).
    """
    scales = scales or {}
    out = []
    for offset, size, fmt in register_groups(len(registers), start_address,
                                             default_fmt, overrides):
        if fmt is None:
            out.extend(["—"] + [""] * (size - 1))
            continue
        group = registers[offset:offset + size]
        scale = scales.get(start_address + offset)
        number = decode_value(group, fmt) if scale else None
        if scale and number is not None and not (isinstance(number, float)
                                                 and (math.isnan(number) or math.isinf(number))):
            out.append(format_scaled(number, scale))
        else:
            out.append(format_value(group, fmt))
        out.extend([""] * (size - 1))
    return out


def raw_text_for_scaled(value_text, scale, fmt):
    """Convert an engineering value typed by the user into raw text for *fmt*."""
    try:
        value = float(str(value_text).strip().replace(",", "."))
    except ValueError:
```

MBToolkit v3.5/mb_master.py — lines 241-280

```
        raise ValueError("Valor fora do formato selecionado.") from None
    raw = unscale_value(value, scale)
    if fmt.split("_")[0].startswith("float"):
        return repr(raw)
    return str(int(round(raw)))


def format_register_values(registers, fmt):
    """Display strings for registers that all use one format."""
    return format_window_values(registers, 0, fmt, None)


def _parse_int(text):
    text = str(text).strip().replace(" ", "").replace("_", "")
    lowered = text.lower()
    if lowered.startswith(("0x", "-0x")):
        return int(text, 16)
    if lowered.startswith(("0b", "-0b")):
        return int(text, 2)
    return int(text, 10)


def encode_register_value(text, fmt):
    """Convert one user-entered value to the registers it occupies.

    Raises ValueError with a message key (PT canonical text) when the value
    does not fit the format.
    """
    fmt = fmt if fmt in FORMAT_SIZES else DEFAULT_FORMAT
    text = str(text).strip()
    if not text:
        raise ValueError("Valor vazio.")
    try:
        if fmt == "hex16":
            value = int(text[2:] if text.lower().startswith("0x") else text, 16)
            if not 0 <= value <= 0xFFFF:
                raise ValueError
            return [value]
        if fmt == "bin16":
            value = int(text.replace(" ", "")[2:] if text.lower().startswith("0b")
```

MBToolkit v3.5/mb_master.py — lines 281-320

```
                        else text.replace(" ", ""), 2)
            if not 0 <= value <= 0xFFFF:
                raise ValueError
            return [value]
        if fmt == "int16":
            value = _parse_int(text)
            if not -0x8000 <= value <= 0x7FFF:
                raise ValueError
            return [value & 0xFFFF]
        if fmt == "uint16":
            value = _parse_int(text)
            if not 0 <= value <= 0xFFFF:
                raise ValueError
            return [value]

        kind, order = fmt.split("_")
        code = _STRUCT_CODES[kind]
        if kind.startswith("float"):
            number = float(text.replace(",", "."))
            raw = struct.pack(code, number)
            if math.isinf(struct.unpack(code, raw)[0]) and not math.isinf(number):
                raise ValueError
        else:
            raw = struct.pack(code, _parse_int(text))
        return _registers_from_value_bytes(raw, order)
    except (ValueError, OverflowError, struct.error):
        raise ValueError("Valor fora do formato selecionado.") from None


def encode_coil_value(text):
    value = str(text).strip().lower()
    if value in ("1", "on", "true"):
        return 1
    if value in ("0", "off", "false"):
        return 0
    raise ValueError("Uma coil só aceita 0 ou 1.")


# ---------------------------------------------------------------------------
# Request frames
```

MBToolkit v3.5/mb_master.py — lines 321-360

```
321 # ---------------------------------------------------------------------------
322
323 def _u16(value):
324     return bytes(((value >> 8) & 0xFF, value & 0xFF))
325
326
327 def read_limit(fc):
328     return MASTER_MAX_READ_BITS if fc in BIT_FUNCTION_CODES else MASTER_MAX_READ_REGISTERS
329
330
331 def validate_read_definition(slave, fc, address, qty):
332     """Raise ValueError (PT canonical message) for an invalid poll definition."""
333     if not 1 <= int(slave) <= 247:
334         raise ValueError("O Slave ID de leitura tem de estar entre 1 e 247.")
335     if int(fc) not in READ_FUNCTION_CODES:
336         raise ValueError("Função de leitura não suportada.")
337     if not 0 <= int(address) <= 0xFFFF:
338         raise ValueError("O endereço tem de estar entre 0 e 65535.")
339     if not 1 <= int(qty) <= read_limit(int(fc)):
340         raise ValueError("Quantidade fora do limite desta função.")
341     if int(address) + int(qty) > 0x10000:
342         raise ValueError("O intervalo de endereços ultrapassa 65535.")
343
344
345 def build_read_request(slave, fc, address, qty):
346     validate_read_definition(slave, fc, address, qty)
347     return append_crc(bytes((int(slave), int(fc))) + _u16(int(address)) + _u16(int(qty)))
348
349
350 def build_write_request(slave, fc, address, values):
351     """Build an FC05/06/15/16 request. *values* are 0/1 bits or 16-bit registers."""
352     slave, fc, address = int(slave), int(fc), int(address)
353     values = [int(v) for v in values]
354     if not 0 <= slave <= 247:
355         raise ValueError("O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.")
356     if not 0 <= address <= 0xFFFF or address + max(1, len(values)) > 0x10000:
357         raise ValueError("O endereço tem de estar entre 0 e 65535.")
358     if fc == 5:
359         if len(values) != 1:
360             raise ValueError("FC05 escreve exatamente uma coil.")
```

MBToolkit v3.5/mb_master.py — lines 361-400

```
        payload = _u16(address) + (b"\xFF\x00" if values[0] else b"\x00\x00")
    elif fc == 6:
        if len(values) != 1:
            raise ValueError("FC06 escreve exatamente um registo.")
        payload = _u16(address) + _u16(values[0] & 0xFFFF)
    elif fc == 15:
        if not 1 <= len(values) <= MASTER_MAX_WRITE_BITS:
            raise ValueError("Quantidade fora do limite desta função.")
        data = bytearray((len(values) + 7) // 8)
        for index, bit in enumerate(values):
            if bit:
                data[index // 8] |= 1 << (index % 8)
        payload = _u16(address) + _u16(len(values)) + bytes((len(data),)) + bytes(data)
    elif fc == 16:
        if not 1 <= len(values) <= MASTER_MAX_WRITE_REGISTERS:
            raise ValueError("Quantidade fora do limite desta função.")
        data = b"".join(_u16(v & 0xFFFF) for v in values)
        payload = _u16(address) + _u16(len(values)) + bytes((len(data),)) + data
    else:
        raise ValueError("Função de escrita não suportada.")
    return append_crc(bytes((slave, fc)) + payload)


def expected_response_length(request):
    fc = request[1]
    if fc in BIT_FUNCTION_CODES:
        qty = (request[4] << 8) | request[5]
        return 5 + (qty + 7) // 8
    if fc in (3, 4):
        qty = (request[4] << 8) | request[5]
        return 5 + 2 * qty
    return 8  # FC05/06/15/16


# ---------------------------------------------------------------------------
# Response matching
# ---------------------------------------------------------------------------

def find_master_response(buffer, request):
    """Find the response to *request* in the received bytes.
```

MBToolkit v3.5/mb_master.py — lines 401-440

```
401
402     Returns a dict with ``kind`` RESPONSE or EXCEPTION, the ``frame`` and,
403     for reads, the decoded ``values`` (bits or registers); None when no
404     complete, CRC-valid, matching frame is present yet. A request echoed by
405     an RS485 adapter is skipped for reads because its length and CRC do not
406     fit the expected response. For FC05/06 a slave reply is itself an exact
407     echo, so an adapter echo cannot be distinguished from it.
408     """
409     data = bytes(buffer)
410     slave, fc = request[0], request[1]
411     length = expected_response_length(request)
412
413     for offset in range(len(data)):
414         if data[offset] != slave or offset + 2 > len(data):
415             continue
416         response_fc = data[offset + 1]
417
418         if response_fc == (fc | 0x80) and offset + 5 <= len(data):
419             frame = data[offset:offset + 5]
420             if crc_ok(frame):
421                 return {"kind": "EXCEPTION", "frame": frame, "exception_code": frame[2],
422                         "values": None}
423
424         if response_fc != fc or offset + length > len(data):
425             continue
426         frame = data[offset:offset + length]
427         if not crc_ok(frame):
428             continue
429
430         if fc in READ_FUNCTION_CODES:
431             if frame[2] != length - 5:
432                 continue
433             payload = frame[3:-2]
434             qty = (request[4] << 8) | request[5]
435             if fc in BIT_FUNCTION_CODES:
436                 values = [(payload[i // 8] >> (i % 8)) & 1 for i in range(qty)]
437             else:
438                 values = [(payload[i] << 8) | payload[i + 1] for i in range(0, len(payload), 2)]
439             return {"kind": "RESPONSE", "frame": frame, "exception_code": None, "values": values}
440
```

MBToolkit v3.5/mb_master.py — lines 441-480

```
        if fc in (5, 6) and frame != request:
            continue
        if fc in (15, 16) and frame[2:6] != request[2:6]:
            continue
        return {"kind": "RESPONSE", "frame": frame, "exception_code": None, "values": None}

    return None


# ---------------------------------------------------------------------------
# Poll definitions and engine
# ---------------------------------------------------------------------------

@dataclass
class PollDefinition:
    window_id: int
    slave: int
    fc: int
    address: int
    qty: int
    scan_ms: int
    enabled: bool = True

    def request(self):
        return build_read_request(self.slave, self.fc, self.address, self.qty)


class MasterEngine:
    """Serve all poll windows over one serial port, one transaction at a time.

    Order of service: queued writes, then "read once" requests, then the
    enabled poll whose next due time is earliest. A poll becomes due again
    ``scan_ms`` after its previous transaction started (0 = continuously).
    After every transaction the bus is left silent for at least the
    configured delay between polls, and never less than Modbus t3.5.

    With *reopen* (a callable returning a new open serial object), a serial or
    driver failure does not end the session: the engine closes the port, posts
    ``master_reconnecting`` and retries every MASTER_RECONNECT_INTERVAL_SECONDS
    until the port opens again (``master_reconnected``) or the engine stops.
```

MBToolkit v3.5/mb_master.py — lines 481-520

```
481
482     Events are named ``<event_prefix>_ready``, ``_result``, ... so a subclass
483     serving another tab (the Modbus TCP Client) posts its own events.
484     """
485
486     def __init__(self, ser, event_queue, *, baud, parity, stopbits,
487                  response_timeout_ms, delay_ms, serial_module=None,
488                  idle_check_seconds=0.0, reopen=None,
489                  reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS,
490                  event_prefix="master"):
491         self.ser = ser
492         self.event_prefix = event_prefix
493         self.reopen = reopen
494         self.reconnect_interval_s = float(reconnect_interval_s)
495         self.event_queue = event_queue
496         self.baud = int(baud)
497         self.parity = parity
498         self.stopbits = stopbits
499         self.response_timeout_s = max(0.001, float(response_timeout_ms) / 1000.0)
500         gap_s = modbus_rtu_interframe_gap_seconds(
501             self.baud, discovery_bits_per_char(parity, stopbits)
502         )
503         self.delay_s = max(gap_s, float(delay_ms) / 1000.0)
504         self.serial_module = serial_module
505         self.idle_check_seconds = float(idle_check_seconds)
506
507         self._lock = threading.Lock()
508         self._polls = {}           # window_id -> [PollDefinition, next_due]
509         self._writes = deque()     # (window_id, request, description)
510         self._once = deque()       # window_id
511         self._wake = threading.Event()
512         self.stop_event = threading.Event()
513         self.thread = None
514
515     # -- control (any thread) ------------------------------------------------
516     def set_poll(self, definition):
517         with self._lock:
518             self._polls[definition.window_id] = [definition, 0.0]
519         self._wake.set()
520
```

MBToolkit v3.5/mb_master.py — lines 521-560

```
521     def remove_poll(self, window_id):
522         with self._lock:
523             self._polls.pop(window_id, None)
524             self._once = deque(w for w in self._once if w != window_id)
525 
526     def read_once(self, window_id):
527         with self._lock:
528             if window_id in self._polls and window_id not in self._once:
529                 self._once.append(window_id)
530         self._wake.set()
531 
532     def queue_write(self, window_id, request, description=""):
533         with self._lock:
534             self._writes.append((window_id, bytes(request), description))
535         self._wake.set()
536 
537     def start(self):
538         self.thread = threading.Thread(target=self.run, daemon=True)
539         self.thread.start()
540 
541     def stop(self, join_timeout=1.0):
542         self.stop_event.set()
543         self._wake.set()
544         cancel_serial_io(self.ser)
545         thread = self.thread
546         if thread is not None and thread.is_alive() and thread is not threading.current_thread():
547             thread.join(timeout=join_timeout)
548 
549     # -- scheduling ---------------------------------------------------------
550     def next_job(self, now=None):
551         """Return ('write', item), ('read', definition, once) or ('wait', seconds)."""
552         now = time.perf_counter() if now is None else now
553         with self._lock:
554             if self._writes:
555                 return ("write", self._writes.popleft())
556             while self._once:
557                 window_id = self._once.popleft()
558                 entry = self._polls.get(window_id)
559                 if entry is not None:
560                     return ("read", entry[0], True)
```

MBToolkit v3.5/mb_master.py — lines 561-600

```
            due = [
                (entry[1], window_id)
                for window_id, entry in self._polls.items()
                if entry[0].enabled
            ]
            if not due:
                return ("wait", 0.5)
            next_due, window_id = min(due)
            if next_due > now:
                return ("wait", min(0.5, next_due - now))
            entry = self._polls[window_id]
            entry[1] = now + max(0, entry[0].scan_ms) / 1000.0
            return ("read", entry[0], False)

    # -- worker ------------------------------------------------------------
    def run(self):
        error = None
        try:
            if self.idle_check_seconds > 0:
                traffic = bus_activity_detected(
                    self.ser, self.idle_check_seconds, self.stop_event
                )
                if traffic:
                    error = ("traffic", hex_bytes(traffic[:16]), len(traffic))
                    return
            self.event_queue.put((f"{self.event_prefix}_ready", None))

            while not self.stop_event.is_set():
                try:
                    self._serve_jobs()
                except Exception as exc:  # serial/driver failure
                    if self.stop_event.is_set():
                        break
                    detail = f"{type(exc).__name__}: {exc}"
                    if self.reopen is None:
                        error = ("serial", detail, 0)
                        break
                    if not self._reconnect(detail):
                        break
        except Exception as exc:  # failure during the idle check
```

MBToolkit v3.5/mb_master.py — lines 601-640

```
            if not self.stop_event.is_set():
                error = ("serial", f"{type(exc).__name__}: {exc}", 0)
        finally:
            self.event_queue.put((f"{self.event_prefix}_stopped", {"error": error}))

    def _serve_jobs(self):
        """Serve writes / reads until stopped; serial errors propagate."""
        while not self.stop_event.is_set():
            job = self.next_job()
            if job[0] == "wait":
                self._wake.wait(job[1])
                self._wake.clear()
                continue

            if job[0] == "write":
                window_id, request, description = job[1]
                result = self.transact_for(window_id, request)
                result.update(kind="write", window_id=window_id, description=description)
            else:
                definition, once = job[1], job[2]
                result = self.transact_for(definition.window_id, definition.request())
                result.update(kind="read", window_id=definition.window_id, once=once,
                              fc=definition.fc, address=definition.address)
            if self.stop_event.is_set():
                return
            self.event_queue.put((f"{self.event_prefix}_result", result))
            if self.stop_event.wait(self.delay_s):
                return

    def _reconnect(self, detail):
        """Close the failed port and reopen it; False when stopped meanwhile."""
        try:
            self.ser.close()
        except Exception:
            pass
        self.event_queue.put((f"{self.event_prefix}_reconnecting", {"error": detail}))
        while not self.stop_event.wait(self.reconnect_interval_s):
            try:
                ser = self.reopen()
            except Exception:
```

MBToolkit v3.5/mb_master.py — lines 641-680

```
                continue
            self.ser = ser
            self.event_queue.put((f"{self.event_prefix}_reconnected", None))
            return True
        return False

    def transact_for(self, window_id, request):
        """Transact *request* for a window (the TCP Client picks its device here)."""
        return self.transact(request)

    def transact(self, request):
        """Send one request and wait for its response (no wait for broadcast)."""
        try:
            self.ser.reset_input_buffer()
        except Exception:
            pass
        sent = write_discovery_request(
            self.ser, request, self.baud, self.parity, self.stopbits,
            self.stop_event, self.serial_module,
        )
        # write_discovery_request() returns once the request has left the
        # wire, so response time and timeout count from the end of the request.
        started = time.perf_counter()
        result = {"tx": hex_bytes(request), "rx": "", "status": "stopped",
                  "values": None, "exception_code": None, "response_ms": None}
        if not sent:
            return result
        if request[0] == 0:
            result["status"] = "broadcast"
            return result

        deadline = time.perf_counter() + self.response_timeout_s
        buffer = bytearray()
        while time.perf_counter() < deadline and not self.stop_event.is_set():
            waiting = int(getattr(self.ser, "in_waiting", 0) or 0)
            if waiting:
                buffer.extend(self.ser.read(waiting) or b"")
                found = find_master_response(buffer, request)
                if found is not None:
                    result.update(
```

MBToolkit v3.5/mb_master.py — lines 681-695

```
                        status="exception" if found["kind"] == "EXCEPTION" else "ok",
                        values=found["values"],
                        exception_code=found["exception_code"],
                        rx=hex_bytes(found["frame"]),
                        response_ms=(time.perf_counter() - started) * 1000.0,
                    )
                    return result
            elif self.stop_event.wait(0.001):
                break

        if self.stop_event.is_set():
            return result
        result["rx"] = hex_bytes(buffer)
        result["status"] = "invalid" if buffer else "timeout"
        return result
```

MBToolkit v3.5/mb_master_ui.py

1479 source lines.

MBToolkit v3.5/mb_master_ui.py — lines 1-40

```python
#!/usr/bin/env python3
"""Master (poll) tab: one window per Slave / data area, polled over one COM.

The protocol and scheduling live in ``mb_master.py``; this mixin owns the
widgets. Each poll window has a model (``MasterWindow``) and a pane that can
sit in the tab's grid or be popped out into its own top-level window.
"""

import time
import tkinter as tk
from tkinter import font as tkfont
from tkinter import messagebox, ttk

from mb_config import (
    DISCOVERY_BAUD_RATES,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MASTER_DEFAULT_DELAY_MS,
    MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
    MASTER_DEFAULT_SCAN_MS,
    MASTER_MAX_DELAY_MS,
    MASTER_MAX_RESPONSE_TIMEOUT_MS,
    MASTER_MAX_SCAN_MS,
    MASTER_MAX_WINDOWS,
    MASTER_MIN_DELAY_MS,
    MASTER_MIN_RESPONSE_TIMEOUT_MS,
    MASTER_WORKSPACE_FORMATS,
    MASTER_WORKSPACE_VERSION,
    MODBUS_EXCEPTION_CODES,
    get_master_workspaces_folder,
)
from mb_diagnostics import FUNCTION_NAMES
from mb_master import (
    BIT_FUNCTION_CODES,
    DEFAULT_FORMAT,
    FORMAT_KEYS_BY_LABEL,
    FORMAT_LABELS,
    FORMAT_SIZES,
    READ_FUNCTION_CODES,
    WRITABLE_READ_FCS,
    MasterEngine,
```

MBToolkit v3.5/mb_master_ui.py — lines 41-80

```
    PollDefinition,
    build_write_request,
    encode_coil_value,
    encode_register_value,
    format_window_values,
    raw_text_for_scaled,
    register_groups,
    validate_read_definition,
)
from mb_table_ui import (
    SERIAL_PARITIES,
    SERIAL_STOP_BITS,
    TABLE_MAX_TITLE_LENGTH,
    TABLE_PANE_MAX_ROWS,
    TABLE_PANE_MIN_ROWS,
    TABLE_PANE_MIN_WIDTH,
    TableUiMixin,
    TableWorkspace,
    apply_format_to_rows,
    parse_int_field,
    validated_formats,
    validated_names,
    validated_scales,
)
from mb_table_ui import list_workspaces as _list_workspaces
from mb_table_ui import workspace_path as _workspace_path
from mb_theme import theme_colors
from mb_widgets import RefreshButton

MASTER_PARITIES = SERIAL_PARITIES
MASTER_STOP_BITS = SERIAL_STOP_BITS
MASTER_PANE_MIN_WIDTH = TABLE_PANE_MIN_WIDTH
MASTER_PANE_MIN_ROWS = TABLE_PANE_MIN_ROWS
MASTER_PANE_MAX_ROWS = TABLE_PANE_MAX_ROWS
MASTER_OK_MESSAGE_SECONDS = 3.0
MASTER_MAX_TITLE_LENGTH = TABLE_MAX_TITLE_LENGTH
MASTER_WHEEL_TAG = "MBToolkit.MasterWheel"


def workspace_path(name):
```

MBToolkit v3.5/mb_master_ui.py — lines 81-120

```
    """File of a Master workspace (in Workspaces\\Master)."""
    return _workspace_path(name, get_master_workspaces_folder())


def list_workspaces():
    """Names of the saved Master workspaces, sorted alphabetically."""
    return _list_workspaces(get_master_workspaces_folder())


def fc_choice_label(fc):
    return f"{fc:02d} {FUNCTION_NAMES[fc]}"


FC_CHOICES = tuple(fc_choice_label(fc) for fc in READ_FUNCTION_CODES)


class MasterWindow:
    """State of one poll window (independent of where its pane is shown)."""

    def __init__(self, window_id, number, slave=1, fc=3, address=0, qty=10,
                 scan_ms=MASTER_DEFAULT_SCAN_MS, base=0, fmt=DEFAULT_FORMAT,
                 names=None, enabled=True, title="", formats=None, scales=None):
        self.window_id = window_id
        self.number = number
        # User-chosen window name; empty means the default "Janela N".
        self.title = str(title or "").strip()[:MASTER_MAX_TITLE_LENGTH]
        self.slave = slave
        self.fc = fc
        self.address = address
        self.qty = qty
        self.scan_ms = scan_ms
        self.base = base
        self.fmt = fmt
        self.names = dict(names or {})  # PDU address -> label
        # PDU address -> format of the value that starts there (per-address
        # formats chosen with the right-click menu); others use self.fmt.
        self.formats = {int(a): f for a, f in (formats or {}).items() if f in FORMAT_SIZES}
        # PDU address -> scale (raw/engineering points, decimals, unit) of the
        # value that starts there.
        self.scales = {int(a): dict(s) for a, s in (scales or {}).items()}
```

MBToolkit v3.5/mb_master_ui.py — lines 121-160

```
        self.enabled = enabled
        self.popped = False
        self.reset_counters()

    def reset_counters(self):
        self.tx = 0
        self.errors = 0
        self.timeouts = 0
        self.response_ms = None
        self.values = None
        # True when the latest read failed: the shown values are out of date.
        self.stale = False
        # (canonical PT template, values, style) shown on the status line.
        self.message = ("", {}, "normal")
        self.message_at = 0.0

    def definition(self):
        return PollDefinition(self.window_id, self.slave, self.fc, self.address,
                              self.qty, self.scan_ms, self.enabled)

    def to_json(self):
        return {
            "title": self.title,
            "slave": self.slave, "fc": self.fc, "address": self.address,
            "qty": self.qty, "scan_ms": self.scan_ms, "base": self.base,
            "format": self.fmt, "enabled": self.enabled,
            "names": {str(address): name for address, name in sorted(self.names.items())},
            "formats": {str(address): fmt for address, fmt in sorted(self.formats.items())},
            "scales": {str(address): scale for address, scale in sorted(self.scales.items())},
        }

    def groups(self):
        """[(row offset, size, fmt or None)] of the values shown in this window."""
        return register_groups(self.qty, self.address, self.fmt, self.formats)

    def group_of_row(self, row):
        for offset, size, fmt in self.groups():
            if offset <= row < offset + size:
                return offset, size, fmt
        return row, 1, None
```

MBToolkit v3.5/mb_master_ui.py — lines 161-200

```
161
162     @staticmethod
163     def validated_kwargs(data):
164         """Validate one workspace window entry; raise ValueError if invalid."""
165         if not isinstance(data, dict):
166             raise ValueError("Janela inválida.")
167         slave, fc = int(data.get("slave", 1)), int(data.get("fc", 3))
168         address, qty = int(data.get("address", 0)), int(data.get("qty", 10))
169         validate_read_definition(slave, fc, address, qty)
170         scan_ms = int(data.get("scan_ms", MASTER_DEFAULT_SCAN_MS))
171         if not 0 <= scan_ms <= MASTER_MAX_SCAN_MS:
172             raise ValueError("Scan rate fora do intervalo.")
173         fmt = data.get("format", DEFAULT_FORMAT)
174         names = data.get("names", {})
175         formats = data.get("formats", {})
176         return {
177             "title": str(data.get("title", "") or "").strip()[:MASTER_MAX_TITLE_LENGTH],
178             "slave": slave, "fc": fc, "address": address, "qty": qty,
179             "scan_ms": scan_ms, "base": 1 if data.get("base") == 1 else 0,
180             "fmt": fmt if fmt in FORMAT_SIZES else DEFAULT_FORMAT,
181             "enabled": bool(data.get("enabled", True)),
182             "names": validated_names(names),
183             "formats": validated_formats(formats),
184             "scales": validated_scales(data.get("scales", {})),
185         }
186
187
188 class MasterMixin(TableUiMixin):
189     """Master (poll) tab GUI.
190
191     The class attributes below describe this tab. The Modbus TCP Client tab
192     (``mb_tcp_ui.ClientTab``) is a second instance of this mixin with its own
193     state; it overrides them and the connection-specific methods
194     (``_master_build_connection``, ``master_connect``, ...).
195     """
196
197     _master_tab_name = "Master"
198     _master_settings_key = "master"
199     _master_page_attr = "master_page"
200     _master_wheel_tag = MASTER_WHEEL_TAG
```

MBToolkit v3.5/mb_master_ui.py — lines 201-240

```
    _master_window_class = MasterWindow
    _master_ws_folder = staticmethod(get_master_workspaces_folder)
    _master_ws_formats = MASTER_WORKSPACE_FORMATS
    _master_ws_close_question = "Guardar alterações ao workspace \"{name}\"?"
    # Caption, range and range message of the Slave field of the windows.
    _master_id_label = "Slave"
    _master_id_range = (1, 247)
    _master_id_message = "O Slave ID de leitura tem de estar entre 1 e 247."
    _master_write_id_label = "Slave (0 = broadcast)"
    _master_write_id_range = (0, 247)
    _master_connect_first = "Liga primeiro o Master."
    # Row menu entry that opens the window's area in the simulator tab:
    # (PT caption, name of the method on the application).
    _master_open_in_slave = ("Abrir no Slave", "slave_open_from_master")

    # -- construction ---------------------------------------------------------
    def build_master_tab(self, page):
        saved = getattr(self, "ui_settings", {}).get(self._master_settings_key, {})
        if not isinstance(saved, dict):
            saved = {}

        self.master_windows = {}
        self._master_panes = {}
        self._master_next_id = 1
        self.master_engine = None
        self.master_serial = None
        self.master_active = False
        self._master_reflow_after_id = None

        page.columnconfigure(0, weight=1)
        page.rowconfigure(2, weight=1)

        self._master_build_connection(page, saved)

        toolbar = ttk.Frame(page)
        toolbar.grid(row=1, column=0, sticky="ew", padx=12, pady=(0, 6))
        toolbar.columnconfigure(3, weight=1)
        ttk.Button(toolbar, text="Nova janela", command=self.master_add_window).grid(
            row=0, column=0, padx=(0, 6))
        # Workspace menu: the saved workspaces are listed directly in it.
```

MBToolkit v3.5/mb_master_ui.py — lines 241-280

```
        self.master_workspace_btn = ttk.Menubutton(toolbar, text="Workspace")
        self.master_workspace_btn.grid(row=0, column=1, padx=(0, 6))
        self.master_workspace_menu = tk.Menu(
            self.master_workspace_btn, tearoff=False,
            postcommand=self._master_build_workspace_menu,
        )
        self.master_workspace_btn.configure(menu=self.master_workspace_menu)
        self.master_workspace_var = tk.StringVar(value="")
        ttk.Label(toolbar, textvariable=self.master_workspace_var).grid(
            row=0, column=2, padx=(0, 6))
        self.master_workspace = TableWorkspace(
            self, folder=self._master_ws_folder,
            formats=self._master_ws_formats, data=self.master_workspace_data,
            load=self._master_load_workspace_data, label_var=self.master_workspace_var,
            close_question=self._master_ws_close_question,
        )
        title_font = tkfont.nametofont("TkDefaultFont").copy()
        title_font.configure(weight="bold")
        self._master_title_font = title_font
        self.ui_style.configure("MBS.Master.Title.TLabel", font=title_font)
        ttk.Label(
            toolbar,
            text=("Duplo clique num valor para escrever, num nome para o editar. "
                  "Clique direito: formato, copiar e base."),
        ).grid(row=0, column=3, sticky="w", padx=(12, 0))

        self.master_canvas, self.master_grid, self._master_grid_item = \
            self._table_build_scroll_grid(page, 2)
        self.master_canvas.bind("<Configure>", self._master_schedule_reflow, add="+")
        self.master_empty_label = ttk.Label(
            self.master_grid,
            text="Sem janelas. Usa \"Nova janela\" para começar a ler um Slave.",
        )

        for sequence in ("<MouseWheel>", "<Button-4>", "<Button-5>"):
            self.bind_class(self._master_wheel_tag, sequence, self._master_wheel)

        self._master_build_row_menu()

        # Reopen the workspace used last time; it is loaded once the window is
```

MBToolkit v3.5/mb_master_ui.py — lines 281-320

```
        # complete so the COM port list is already known. Never connects.
        last = saved.get("last_workspace", "")
        if last and last in self.master_workspace.names():
            self.after(10, lambda: self._master_restore_workspace(last))
        else:
            self.master_add_window()

    def _master_build_connection(self, page, saved):
        """Connection bar of the serial Master (COM port and serial format)."""
        connection = ttk.LabelFrame(page, text="Ligação — o Master transmite no barramento")
        connection.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
        self._master_connection_frame = connection
        row = ttk.Frame(connection)
        row.grid(row=0, column=0, sticky="w", padx=8, pady=(8, 4))

        ttk.Label(row, text="Porta COM").grid(row=0, column=0, padx=(0, 4))
        self.master_port_var = tk.StringVar()
        self.master_port_combo = ttk.Combobox(
            row, textvariable=self.master_port_var, width=13, state="readonly"
        )
        self.master_port_combo.grid(row=0, column=1)
        self.master_refresh_btn = RefreshButton(row, command=self.refresh_ports, size=24)
        self.master_refresh_btn.grid(row=0, column=2, padx=(4, 18))

        def _sync_refresh_square(_event=None):
            height = self.master_port_combo.winfo_height()
            if height > 1:
                self.master_refresh_btn.configure(width=height, height=height)

        self.master_port_combo.bind("<Configure>", _sync_refresh_square, add="+")

        self.master_baud_var = tk.StringVar(value=str(saved.get("baud", "9600")))
        self.master_parity_var = tk.StringVar(value=saved.get("parity", "None"))
        self.master_stop_var = tk.StringVar(value=str(saved.get("stop_bits", "1")))
        self.master_timeout_var = tk.StringVar(
            value=str(saved.get("response_timeout_ms", MASTER_DEFAULT_RESPONSE_TIMEOUT_MS))
        )
        self.master_delay_var = tk.StringVar(
            value=str(saved.get("delay_ms", MASTER_DEFAULT_DELAY_MS))
        )
```

MBToolkit v3.5/mb_master_ui.py — lines 321-360

```
        fields = (
            ("Baud rate", ttk.Combobox(row, textvariable=self.master_baud_var, width=8,
                                       state="readonly",
                                       values=[str(b) for b in DISCOVERY_BAUD_RATES])),
            ("Parity", ttk.Combobox(row, textvariable=self.master_parity_var, width=6,
                                    state="readonly", values=MASTER_PARITIES)),
            ("Stop bits", ttk.Combobox(row, textvariable=self.master_stop_var, width=3,
                                       state="readonly", values=MASTER_STOP_BITS)),
            ("Timeout resposta (ms)", ttk.Spinbox(
                row, textvariable=self.master_timeout_var, width=6, increment=50,
                from_=MASTER_MIN_RESPONSE_TIMEOUT_MS, to=MASTER_MAX_RESPONSE_TIMEOUT_MS)),
            ("Atraso entre pedidos (ms)", ttk.Spinbox(
                row, textvariable=self.master_delay_var, width=6, increment=5,
                from_=MASTER_MIN_DELAY_MS, to=MASTER_MAX_DELAY_MS)),
        )
        column = 3
        self._master_setting_widgets = [self.master_port_combo]
        for caption, widget in fields:
            ttk.Label(row, text=caption).grid(row=0, column=column, padx=(0, 4))
            widget.grid(row=0, column=column + 1, padx=(0, 16))
            self._master_setting_widgets.append(widget)
            column += 2
        ttk.Label(row, text="Data bits: 8").grid(row=0, column=column)

        controls = ttk.Frame(connection)
        controls.grid(row=1, column=0, sticky="w", padx=8, pady=(0, 8))
        self.master_safety_var = tk.BooleanVar(value=False)
        ttk.Checkbutton(
            controls,
            text="Confirmo que não existe outro master ativo no barramento.",
            variable=self.master_safety_var,
        ).grid(row=0, column=0, padx=(0, 18))
        self.master_connect_btn = ttk.Button(
            controls, text="Ligar", command=self.master_connect, style="MBS.Primary.TButton"
        )
        self.master_connect_btn.grid(row=0, column=1, padx=(0, 6))
        self.master_disconnect_btn = ttk.Button(
            controls, text="Desligar", command=self.master_disconnect, state="disabled"
        )
        self.master_disconnect_btn.grid(row=0, column=2, padx=(0, 18))
```

MBToolkit v3.5/mb_master_ui.py — lines 361-400

```
361         self.master_status_var = tk.StringVar(value=self.tr("Desligado"))
362         ttk.Label(controls, textvariable=self.master_status_var).grid(row=0, column=3)
363 
364         for variable in (self.master_baud_var, self.master_parity_var, self.master_stop_var,
365                          self.master_timeout_var, self.master_delay_var):
366             variable.trace_add("write", lambda *_a: self._schedule_ui_settings_save())
367 
368     def _master_restore_workspace(self, name):
369         if not self.master_open_workspace(name, quiet=True):
370             self.master_add_window()
371 
372     # -- helpers ---------------------------------------------------------
373     def _master_install_wheel_tags(self, root_widget):
374         self._table_install_wheel_tags(root_widget, self._master_wheel_tag)
375 
376     def _master_wheel(self, event):
377         return self._table_wheel(getattr(self, "master_canvas", None), event)
378 
379     def _master_window_name(self, window):
380         return window.title or self.trf("Janela {number}", number=window.number)
381 
382     def _master_window_summary(self, window):
383         return (f"{self._master_id_label} {window.slave} · "
384                 f"FC{window.fc:02d} {FUNCTION_NAMES[window.fc]} · "
385                 f"{window.address + window.base} · {window.qty}")
386 
387     def _master_window_title(self, window):
388         return f"{self._master_window_name(window)} — {self._master_window_summary(window)}"
389 
390     def _master_counters_text(self, window):
391         response = f"{window.response_ms:.1f} ms" if window.response_ms is not None else "—"
392         return (f"Tx: {window.tx}   Err: {window.errors}   "
393                 f"Timeouts: {window.timeouts}   Resp.: {response}")
394 
395     def _master_set_message(self, window, template, style="normal", **values):
396         window.message = (template, values, style)
397         window.message_at = time.monotonic()
398         self._master_render_status(window)
399 
400     def _master_render_status(self, window):
```

MBToolkit v3.5/mb_master_ui.py — lines 401-440

```
        pane = self._master_panes.get(window.window_id)
        if pane is None:
            return
        template, values, style = window.message
        if not window.enabled and style != "error":
            template, values, style = "Pausada", {}, "normal"
        text = self.trf(template, **values) if template else ""
        try:
            pane["counters_var"].set(self._master_counters_text(window))
            pane["message_var"].set(text)
            pane["message_label"].configure(style={
                "error": "MBS.Master.Error.TLabel",
                "ok": "MBS.Master.Ok.TLabel",
            }.get(style, "TLabel"))
            pane["name_var"].set(self._master_window_name(window))
            pane["summary_var"].set(self._master_window_summary(window))
            pane["pause_btn"].configure(text=self.tr("Retomar" if not window.enabled else "Pausar"))
            pane["popout_btn"].configure(text=self.tr("Ancorar" if window.popped else "Destacar"))
            if window.popped and pane.get("toplevel") is not None:
                pane["toplevel"].title(f"MBToolkit — {self._master_window_title(window)}")
        except tk.TclError:
            pass

    # -- panes ---------------------------------------------------------------
    def _master_build_pane(self, window, parent):
        frame = ttk.LabelFrame(parent, text="")
        frame.columnconfigure(0, weight=1)
        pane = {"frame": frame}
        window_id = window.window_id

        header_buttons = self._table_build_header(
            frame, pane,
            rename=lambda: self.master_rename_window(window_id),
            popout=lambda: self.master_toggle_popout(window_id),
            close=lambda: self.master_close_window(window_id),
        )

        # Row 1 holds the device of a TCP Client window; the serial Master has none.
        target = self._master_build_target(frame, pane, window)
        row = 2 if target is not None else 1
```

MBToolkit v3.5/mb_master_ui.py — lines 441-480

```
441         frame.rowconfigure(row + 4, weight=1)
442
443         definition = ttk.Frame(frame)
444         definition.grid(row=row, column=0, sticky="w", padx=6, pady=(2, 2))
445         pane["slave_var"] = tk.StringVar(value=str(window.slave))
446         pane["fc_var"] = tk.StringVar(value=fc_choice_label(window.fc))
447         pane["address_var"] = tk.StringVar(value=str(window.address + window.base))
448         pane["qty_var"] = tk.StringVar(value=str(window.qty))
449         pane["scan_var"] = tk.StringVar(value=str(window.scan_ms))
450         pane["format_var"] = tk.StringVar(value=FORMAT_LABELS[window.fmt])
451
452         layout = (
453             (self._master_id_label, ttk.Spinbox(
454                 definition, from_=self._master_id_range[0], to=self._master_id_range[1],
455                 width=4, textvariable=pane["slave_var"])),
456             ("FC", ttk.Combobox(definition, values=FC_CHOICES, width=24, state="readonly",
457                                 textvariable=pane["fc_var"])),
458             ("Endereço", ttk.Entry(definition, width=6, textvariable=pane["address_var"])),
459             ("Qtd.", ttk.Entry(definition, width=5, textvariable=pane["qty_var"])),
460         )
461         for index, (caption, widget) in enumerate(layout):
462             ttk.Label(definition, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
463             widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))
464
465         definition2 = ttk.Frame(frame)
466         definition2.grid(row=row + 1, column=0, sticky="w", padx=6, pady=2)
467         format_combo = ttk.Combobox(
468             definition2, values=[FORMAT_LABELS[k] for k in FORMAT_LABELS], width=20,
469             state="readonly", textvariable=pane["format_var"],
470         )
471         layout2 = (
472             ("Scan (ms)", ttk.Entry(definition2, width=7, textvariable=pane["scan_var"])),
473             ("Formato", format_combo),
474         )
475         for index, (caption, widget) in enumerate(layout2):
476             ttk.Label(definition2, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
477             widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))
478         pane["format_combo"] = format_combo
479
480         buttons = ttk.Frame(frame)
```

MBToolkit v3.5/mb_master_ui.py — lines 481-520

```
        buttons.grid(row=row + 2, column=0, sticky="w", padx=6, pady=(2, 2))
        actions = (
            ("Aplicar", lambda: self.master_apply_window(window_id)),
            ("Pausar", lambda: self.master_toggle_pause(window_id)),
            ("Ler 1x", lambda: self.master_read_once(window_id)),
            ("Duplicar", lambda: self.master_duplicate_window(window_id)),
        )
        for index, (caption, command) in enumerate(actions):
            button = ttk.Button(buttons, text=caption, command=command, width=10)
            button.grid(row=0, column=index, padx=(0, 3))
            if caption == "Pausar":
                pane["pause_btn"] = button
            elif caption == "Aplicar":
                pane["apply_btn"] = button

        status = ttk.Frame(frame)
        status.grid(row=row + 3, column=0, sticky="ew", padx=6, pady=(2, 2))
        status.columnconfigure(1, weight=1)
        pane["counters_var"] = tk.StringVar()
        pane["message_var"] = tk.StringVar()
        ttk.Label(status, textvariable=pane["counters_var"]).grid(row=0, column=0, sticky="w")
        pane["message_label"] = ttk.Label(status, textvariable=pane["message_var"])
        pane["message_label"].grid(row=0, column=1, sticky="w", padx=(14, 0))

        self._table_build_tree(
            frame, pane, row + 4,
            on_double=lambda e: self._master_on_double_click(window_id, e),
            on_return=lambda: self._master_edit_value(window_id),
            on_menu=lambda e: self._master_show_row_menu(window_id, e),
        )

        rows = (definition, definition2) + ((target,) if target is not None else ())
        for widget in rows:
            for child in widget.winfo_children():
                if isinstance(child, (ttk.Entry, ttk.Spinbox)):
                    child.bind("<Return>", lambda _e: self.master_apply_window(window_id))

        self._table_translate_captions(*rows, buttons, header_buttons)

        self._master_panes[window_id] = pane
```

MBToolkit v3.5/mb_master_ui.py — lines 521-560

```
        for name in ("slave_var", "fc_var", "address_var", "qty_var", "scan_var", "format_var",
                     *pane.get("target_vars", ())):
            pane[name].trace_add("write",
                                 lambda *_a: self._master_update_apply_state(window_id))
        self._master_fill_rows(window)
        self._master_update_format_state(window)
        self._master_render_status(window)
        self._master_update_apply_state(window_id)
        self._master_install_wheel_tags(frame)
        return frame

    def _master_build_target(self, frame, pane, window):
        """Row with the device of the window (TCP Client only); None here."""
        return None

    def _master_parse_target(self, pane):
        """Device fields of a window as a dict (TCP Client only; ValueError if invalid)."""
        return {}

    def _master_validate_definition(self, slave, fc, address, qty):
        validate_read_definition(slave, fc, address, qty)

    def _master_update_format_state(self, window):
        pane = self._master_panes.get(window.window_id)
        if pane is not None:
            state = "disabled" if window.fc in BIT_FUNCTION_CODES else "readonly"
            pane["format_combo"].configure(state=state)

    def _master_fill_rows(self, window):
        pane = self._master_panes.get(window.window_id)
        if pane is None:
            return
        tree = pane["tree"]
        tree.delete(*tree.get_children())
        tree.configure(height=max(MASTER_PANE_MIN_ROWS, min(MASTER_PANE_MAX_ROWS, window.qty)))
        for index in range(window.qty):
            pdu = window.address + index
            tree.insert("", "end", iid=f"r{index}", tags=self._table_row_tags(index),
                        values=(pdu + window.base, window.names.get(pdu, ""), ""))
        self._master_render_values(window)
```

MBToolkit v3.5/mb_master_ui.py — lines 561-600

```
561
562     def _master_render_values(self, window):
563         pane = self._master_panes.get(window.window_id)
564         if pane is None:
565             return
566         tree = pane["tree"]
567         stale = ("stale",) if window.stale else ()
568         if window.values is None:
569             texts = [""] * window.qty
570         elif window.fc in BIT_FUNCTION_CODES:
571             texts = [str(v) for v in window.values]
572         else:
573             texts = format_window_values(window.values, window.address, window.fmt,
574                                          window.formats, window.scales)
575         for index, text in enumerate(texts[:window.qty]):
576             try:
577                 tree.item(f"r{index}", tags=self._table_row_tags(index) + stale)
578                 tree.set(f"r{index}", "value", text)
579             except tk.TclError:
580                 pass
581
582     def _master_schedule_reflow(self, _event=None):
583         if self._master_reflow_after_id is not None:
584             try:
585                 self.after_cancel(self._master_reflow_after_id)
586             except tk.TclError:
587                 pass
588         self._master_reflow_after_id = self.after(80, self._master_reflow)
589
590     def _master_reflow(self):
591         """Arrange docked panes in as many equal columns as the width allows."""
592         self._master_reflow_after_id = None
593         docked = [w for w in sorted(self.master_windows.values(), key=lambda w: w.number)
594                   if not w.popped]
595         frames = [self._master_panes[w.window_id]["frame"] for w in docked
596                   if w.window_id in self._master_panes]
597         self._table_reflow(self.master_canvas, self.master_grid, self._master_grid_item,
598                            frames, self.master_empty_label, "master_panes")
599
600     # -- window actions ------------------------------------------------------
```

MBToolkit v3.5/mb_master_ui.py — lines 601-640

```
    def master_add_window(self, kwargs=None):
        if len(self.master_windows) >= MASTER_MAX_WINDOWS:
            messagebox.showwarning(
                self.tr(self._master_tab_name),
                self.trf("Máximo de {count} janelas.", count=MASTER_MAX_WINDOWS),
            )
            return None
        window_id = self._master_next_id
        self._master_next_id += 1
        numbers = {w.number for w in self.master_windows.values()}
        number = next(n for n in range(1, MASTER_MAX_WINDOWS + 2) if n not in numbers)
        window = self._master_window_class(window_id, number, **(kwargs or {}))
        self.master_windows[window_id] = window
        self._master_build_pane(window, self.master_grid)
        self._master_reflow()
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        self._master_mark_changed()
        return window

    def master_duplicate_window(self, window_id):
        window = self.master_windows.get(window_id)
        if window is not None:
            data = self._master_window_class.validated_kwargs(window.to_json())
            self.master_add_window(data)

    def master_close_window(self, window_id):
        window = self.master_windows.pop(window_id, None)
        if window is None:
            return
        if self.master_engine is not None:
            self.master_engine.remove_poll(window_id)
        pane = self._master_panes.pop(window_id, None)
        if pane is not None:
            top = pane.get("toplevel")
            try:
                (top if top is not None else pane["frame"]).destroy()
            except tk.TclError:
                pass
        self._master_reflow()
```

MBToolkit v3.5/mb_master_ui.py — lines 641-680

```
        if not self.master_windows:
            # Closing every window leaves no workspace open.
            self.master_workspace.forget()
        else:
            self._master_mark_changed()

    def master_apply_window(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None:
            return False
        try:
            base = window.base
            slave = parse_int_field(pane["slave_var"].get(), *self._master_id_range,
                                    self._master_id_message)
            fc = int(pane["fc_var"].get()[:2])
            address = parse_int_field(pane["address_var"].get(), base, 0xFFFF + base,
                                      "O endereço tem de estar entre 0 e 65535.") - base
            qty = parse_int_field(pane["qty_var"].get(), 1, 2000,
                                  "Quantidade fora do limite desta função.")
            scan_ms = parse_int_field(pane["scan_var"].get(), 0, MASTER_MAX_SCAN_MS,
                                      "Scan rate fora do intervalo.")
            self._master_validate_definition(slave, fc, address, qty)
            target = self._master_parse_target(pane)
        except ValueError as exc:
            messagebox.showerror(self.tr("Janela inválida"), self.tr(str(exc)))
            return False

        fmt = FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get(), DEFAULT_FORMAT)
        changed_area = (slave, fc, address, qty) != (window.slave, window.fc,
                                                     window.address, window.qty) or any(
            getattr(window, key) != value for key, value in target.items())
        changed = changed_area or (scan_ms, fmt) != (window.scan_ms, window.fmt)
        for key, value in target.items():
            setattr(window, key, value)
        window.slave, window.fc, window.address, window.qty = slave, fc, address, qty
        window.scan_ms, window.fmt = scan_ms, fmt
        if changed_area:
            window.reset_counters()
        self._master_fill_rows(window)
```

MBToolkit v3.5/mb_master_ui.py — lines 681-720

```
        self._master_update_format_state(window)
        self._master_render_status(window)
        self._master_update_apply_state(window_id)
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        if changed:
            self._master_mark_changed()
        return True

    def master_toggle_pause(self, window_id):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        window.enabled = not window.enabled
        if window.enabled and window.message[0] == "Pausada":
            window.message = ("", {}, "normal")
        self._master_render_status(window)
        if self.master_engine is not None:
            self.master_engine.set_poll(window.definition())
        self._master_mark_changed()

    def master_read_once(self, window_id):
        if self.master_engine is None:
            messagebox.showinfo(self.tr(self._master_tab_name), self.tr(self._master_connect_first))
            return
        self.master_engine.read_once(window_id)

    def master_toggle_popout(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.pop(window_id, None)
        if window is None or pane is None:
            return
        old_top = pane.get("toplevel")
        try:
            pane["frame"].destroy()
            if old_top is not None:
                old_top.destroy()
        except tk.TclError:
            pass

```

MBToolkit v3.5/mb_master_ui.py — lines 721-760

```
        window.popped = not window.popped
        if window.popped:
            top = tk.Toplevel(self._table_root())
            top.geometry("660x420")
            top.minsize(MASTER_PANE_MIN_WIDTH, 300)
            top.columnconfigure(0, weight=1)
            top.rowconfigure(0, weight=1)
            top.protocol("WM_DELETE_WINDOW", lambda: self.master_toggle_popout(window_id))
            frame = self._master_build_pane(window, top)
            frame.grid(row=0, column=0, sticky="nsew", padx=8, pady=8)
            self._master_panes[window_id]["toplevel"] = top
            self._table_apply_toplevel_palette(top)
        else:
            self._master_build_pane(window, self.master_grid)
        self._master_render_status(window)
        self._master_reflow()

    # -- value / name editing ------------------------------------------------
    def _master_on_double_click(self, window_id, event):
        pane = self._master_panes.get(window_id)
        if pane is None:
            return "break"
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if not row:
            return "break"
        tree.selection_set(row)
        if tree.identify_column(event.x) == "#2":
            self._master_edit_name(window_id, row)
        else:
            self._master_edit_value(window_id)
        return "break"

    def _master_edit_name(self, window_id, row):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        pdu = window.address + int(row[1:])
        dialog, body = self._table_dialog(self.tr("Nome"))
        ttk.Label(body, text=self.trf("Nome do endereço {address}",
```

MBToolkit v3.5/mb_master_ui.py — lines 761-800

```
                                    address=pdu + window.base)).grid(row=0, column=0, sticky="w")
        name_var = tk.StringVar(value=window.names.get(pdu, ""))
        entry = ttk.Entry(body, textvariable=name_var, width=36)
        entry.grid(row=1, column=0, pady=(4, 8))

        def save(_event=None):
            name = name_var.get().strip()
            if name:
                window.names[pdu] = name
            else:
                window.names.pop(pdu, None)
            pane = self._master_panes.get(window_id)
            if pane is not None:
                pane["tree"].set(row, "name", name)
            self._master_mark_changed()
            dialog.destroy()

        ttk.Button(body, text=self.tr("Guardar"), command=save).grid(row=2, column=0, sticky="e")
        entry.bind("<Return>", save)
        entry.focus_set()
        dialog.grab_set()

    def _master_edit_value(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None:
            return
        selection = pane["tree"].selection()
        if not selection:
            return
        index = int(selection[0][1:])
        write_fcs = WRITABLE_READ_FCS.get(window.fc)
        if write_fcs is None:
            messagebox.showinfo(
                self.tr("Escrita"),
                self.tr("FC02 e FC04 são só de leitura: abre uma janela FC01 ou FC03 para escrever."),
            )
            return
        bits = window.fc in BIT_FUNCTION_CODES
        if bits:
```

MBToolkit v3.5/mb_master_ui.py — lines 801-840

```
            size, fmt = 1, None
        else:
            # Multi-register values start on the first register of their group.
            index, size, fmt = window.group_of_row(index)
            if fmt is None:
                messagebox.showinfo(
                    self.tr("Escrita"),
                    self.tr("Este endereço não tem um valor completo no formato atual."),
                )
                return
        current = ""
        try:
            current = pane["tree"].set(f"r{index}", "value")
        except tk.TclError:
            pass
        if not bits and window.scales.get(window.address + index, {}).get("unit"):
            current = current.rsplit(" ", 1)[0]

        dialog, body = self._table_dialog(self.tr("Escrever"))
        slave_var = tk.StringVar(value=str(window.slave))
        address_var = tk.StringVar(value=str(window.address + index + window.base))
        value_var = tk.StringVar(value=current)
        single_fc, multiple_fc = write_fcs
        fc_var = tk.IntVar(value=single_fc if size == 1 else multiple_fc)

        rows = (
            (self._master_write_id_label, ttk.Spinbox(body, from_=self._master_write_id_range[0],
                                                      to=self._master_write_id_range[1], width=6,
                                                  textvariable=slave_var)),
            ("Endereço", ttk.Entry(body, width=8, textvariable=address_var)),
            ("Valor(es)", ttk.Entry(body, width=30, textvariable=value_var)),
        )
        for row_index, (caption, widget) in enumerate(rows):
            ttk.Label(body, text=self.tr(caption)).grid(row=row_index, column=0, sticky="w",
                                                        pady=2, padx=(0, 8))
            widget.grid(row=row_index, column=1, sticky="w", pady=2)
        functions = ttk.Frame(body)
        functions.grid(row=3, column=0, columnspan=2, sticky="w", pady=(6, 2))
        single = ttk.Radiobutton(functions, text=fc_choice_label_write(single_fc),
                                 variable=fc_var, value=single_fc)
```

MBToolkit v3.5/mb_master_ui.py — lines 841-880

```
        single.grid(row=0, column=0, sticky="w")
        if size != 1:
            single.configure(state="disabled")
        ttk.Radiobutton(functions, text=fc_choice_label_write(multiple_fc),
                        variable=fc_var, value=multiple_fc).grid(row=1, column=0, sticky="w")
        scale = None if bits else window.scales.get(window.address + index)
        if scale is not None and fmt in ("hex16", "bin16"):
            scale = None
        format_text = "0 / 1" if bits else FORMAT_LABELS[fmt]
        if scale is not None:
            format_text += " · " + self.trf("valores em unidades da escala {unit}",
                                            unit=scale.get("unit") or "")
        ttk.Label(
            body,
            text=self.trf("Formato: {fmt}. Vários valores (FC15/FC16): separar com ;",
                          fmt=format_text),
        ).grid(row=4, column=0, columnspan=2, sticky="w", pady=(6, 8))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=5, column=0, columnspan=2, sticky="w")

        def send(_event=None):
            try:
                request, description = self.master_build_write(
                    window, slave_var.get(), address_var.get(), value_var.get(),
                    fc_var.get(), fmt, scale,
                )
            except ValueError as exc:
                message_var.set(self.tr(str(exc)))
                return
            if self.master_engine is None:
                message_var.set(self.tr(self._master_connect_first))
                return
            self.master_engine.queue_write(window.window_id, request, description)
            dialog.destroy()

        buttons = ttk.Frame(body)
        buttons.grid(row=6, column=0, columnspan=2, sticky="e", pady=(8, 0))
        ttk.Button(buttons, text=self.tr("Enviar"), command=send,
                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
```

MBToolkit v3.5/mb_master_ui.py — lines 881-920

```
881         ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(row=0, column=1)
882         rows[2][1].bind("<Return>", send)
883         rows[2][1].focus_set()
884         rows[2][1].select_range(0, "end")
885         dialog.grab_set()
886 
887     def master_build_write(self, window, slave_text, address_text, values_text, fc,
888                            fmt=None, scale=None):
889         """Validate the write dialog fields and return (request, description).
890 
891         *fmt* is the format of the clicked value (per-address or window format).
892         With *scale*, the typed values are engineering units and are converted
893         back to raw values before encoding.
894         """
895         fmt = fmt or window.fmt
896         slave = parse_int_field(slave_text, 0, 247,
897                                 "O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.")
898         address = parse_int_field(address_text, window.base, 0xFFFF + window.base,
899                                   "O endereço tem de estar entre 0 e 65535.") - window.base
900         parts = [p for p in str(values_text).split(";") if p.strip()]
901         if not parts:
902             raise ValueError("Valor vazio.")
903         if window.fc in BIT_FUNCTION_CODES:
904             values = [encode_coil_value(p) for p in parts]
905         else:
906             values = []
907             for part in parts:
908                 if scale is not None:
909                     part = raw_text_for_scaled(part, scale, fmt)
910                 values.extend(encode_register_value(part, fmt))
911         if fc in (5, 6) and len(values) != 1:
912             raise ValueError("FC05/FC06 escrevem um único valor; usa FC15/FC16 para vários.")
913         request = build_write_request(slave, fc, address, values)
914         description = f"FC{fc:02d} @ {address + window.base} × {len(values)}"
915         return request, description
916 
917     # -- connection ------------------------------------------------------------
918     def _master_connection_settings(self):
919         timeout = parse_int_field(
920             self.master_timeout_var.get(), MASTER_MIN_RESPONSE_TIMEOUT_MS,
```

MBToolkit v3.5/mb_master_ui.py — lines 921-960

```
            MASTER_MAX_RESPONSE_TIMEOUT_MS,
            "O timeout de resposta tem de estar entre 10 e 10000 ms.")
        delay = parse_int_field(
            self.master_delay_var.get(), MASTER_MIN_DELAY_MS, MASTER_MAX_DELAY_MS,
            "O atraso entre pedidos tem de estar entre 0 e 10000 ms.")
        return {
            "port": self.master_port_var.get().strip(),
            "baud": int(self.master_baud_var.get()),
            "parity": self.master_parity_var.get(),
            "stopbits": self.master_stop_var.get(),
            "response_timeout_ms": timeout,
            "delay_ms": delay,
        }

    def master_connect(self):
        if self.master_active:
            return
        if self.busy:
            messagebox.showwarning(
                self.tr(self._master_tab_name),
                self.tr("Para a captura, a pesquisa ou o Slave em curso antes de ligar o Master."),
            )
            return
        if not self.master_safety_var.get():
            messagebox.showwarning(
                self.tr(self._master_tab_name),
                self.tr("Confirma primeiro que não existe outro master ativo no barramento."),
            )
            return
        serial, _ = self.get_serial_modules()
        if serial is None:
            messagebox.showerror(self.tr("pyserial em falta"), self.tr("pyserial não está instalado."))
            return
        try:
            settings = self._master_connection_settings()
            if not settings["port"]:
                raise ValueError("Seleciona uma porta COM.")
        except ValueError as exc:
            messagebox.showerror(self.tr("Configuração inválida"), self.tr(str(exc)))
            return
```

MBToolkit v3.5/mb_master_ui.py — lines 961-1000

```
        for window_id in list(self.master_windows):
            if not self.master_apply_window(window_id):
                return
        try:
            ser = self.open_discovery_serial_port(
                serial, settings["port"], settings["baud"], settings["parity"],
                settings["stopbits"],
            )
        except Exception as exc:
            messagebox.showerror(
                self.tr(self._master_tab_name),
                self.trf("Não foi possível abrir {port}: {error}",
                         port=settings["port"], error=f"{type(exc).__name__}: {exc}"),
            )
            return

        self.master_serial = ser
        self.master_engine = MasterEngine(
            ser, self.event_queue, baud=settings["baud"], parity=settings["parity"],
            stopbits=settings["stopbits"],
            response_timeout_ms=settings["response_timeout_ms"],
            delay_ms=settings["delay_ms"], serial_module=serial,
            idle_check_seconds=DISCOVERY_IDLE_CHECK_SECONDS,
            reopen=lambda: self.open_discovery_serial_port(
                serial, settings["port"], settings["baud"], settings["parity"],
                settings["stopbits"],
            ),
        )
        for window in self.master_windows.values():
            window.reset_counters()
            self._master_render_status(window)
            self.master_engine.set_poll(window.definition())
        self.master_active = True
        self.busy = True
        self._master_lock_other_features(True)
        self.master_status_var.set(self.tr("A verificar se o barramento está livre…"))
        self.master_engine.start()

    def master_disconnect(self):
        if self.master_engine is None:
```

MBToolkit v3.5/mb_master_ui.py — lines 1001-1040

```
            return
        self.master_status_var.set(self.tr("A desligar…"))
        self.master_disconnect_btn.configure(state="disabled")
        self.master_engine.stop()

    def _master_lock_other_features(self, locked):
        state = "disabled" if locked else "normal"
        self.master_connect_btn.configure(state=state)
        self.master_disconnect_btn.configure(state="normal" if locked else "disabled")
        for widget in self._master_setting_widgets:
            try:
                widget.configure(state="disabled" if locked else "readonly"
                                 if isinstance(widget, ttk.Combobox) else "normal")
            except tk.TclError:
                pass
        self.master_refresh_btn.configure(state=state)
        slave_button = getattr(self, "slave_connect_btn", None)
        if slave_button is not None:
            slave_button.configure(state=state)
        self.start_btn.configure(state=state)
        self.restart_btn.configure(state=state)
        self.set_sniffer_refresh_state(state)
        self.set_simulation_button_state(state)
        self.update_discovery_start_state()

    def _master_finish_session(self, payload):
        if not self.master_active:
            return  # already finished (a late master_stopped event)
        engine = self.master_engine
        self.master_engine = None
        if engine is not None:
            engine.stop(join_timeout=0.5)
            self.master_serial = engine.ser
        if self.master_serial is not None:
            try:
                self.master_serial.close()
            except Exception:
                pass
        self.master_serial = None
        self.master_active = False
```

MBToolkit v3.5/mb_master_ui.py — lines 1041-1080

```
        self.busy = False
        self._master_lock_other_features(False)
        self.master_status_var.set(self.tr("Desligado"))
        for window in self.master_windows.values():
            if window.values is not None and not window.stale:
                window.stale = True
                self._master_render_values(window)

        error = (payload or {}).get("error")
        if error:
            kind, detail, count = error
            if kind == "traffic":
                text = self.trf(
                    "Foi detetado tráfego em {port} antes de transmitir "
                    "({count} bytes: {raw}). Pode existir outro master "
                    "ativo no barramento; a pesquisa não foi iniciada.",
                    port=self.master_port_var.get(), count=count, raw=detail,
                )
            else:
                text = self.trf("Erro na porta série: {error}", error=detail)
            messagebox.showerror(self.tr(self._master_tab_name), text)

    # -- worker events ------------------------------------------------------
    def handle_master_event(self, event, payload):
        if event == "master_ready":
            self.master_status_var.set(self.tr("● Ligado"))
        elif event == "master_reconnecting":
            self.master_status_var.set(self.trf(
                "A religar a {port}… (porta falhou: {error})",
                port=self.master_port_var.get(), error=(payload or {}).get("error", ""),
            ))
            for window in self.master_windows.values():
                if window.values is not None and not window.stale:
                    window.stale = True
                    self._master_render_values(window)
        elif event == "master_reconnected":
            engine = self.master_engine
            if engine is not None:
                self.master_serial = engine.ser
            self.master_status_var.set(self.tr("● Ligado"))
```

MBToolkit v3.5/mb_master_ui.py — lines 1081-1120

```
        elif event == "master_stopped":
            self._master_finish_session(payload)
        elif event == "master_result":
            self._master_apply_result(payload)

    def _master_apply_result(self, result):
        window = self.master_windows.get(result.get("window_id"))
        if window is None:
            return
        status = result["status"]
        if status == "stopped":
            return
        window.tx += 1
        if result.get("response_ms") is not None:
            window.response_ms = result["response_ms"]

        if result["kind"] == "read":
            if (result.get("fc"), result.get("address")) != (window.fc, window.address):
                return  # definition changed while the request was on the bus
            if status == "ok":
                window.values = result["values"]
                window.stale = False
                self._master_render_values(window)
                # A successful read clears errors; a write confirmation stays
                # visible for a few seconds so the user can see it.
                if (window.message[2] != "ok"
                        or time.monotonic() - window.message_at > MASTER_OK_MESSAGE_SECONDS):
                    self._master_set_message(window, "")
                return
        elif status == "ok":
            self._master_set_message(window, "Escrita OK: {what}", "ok",
                                     what=result.get("description", ""))
            if self.master_engine is not None:
                self.master_engine.read_once(window.window_id)
            return
        elif status == "broadcast":
            self._master_set_message(window, "Escrita broadcast enviada: {what}", "ok",
                                     what=result.get("description", ""))
            return

```

MBToolkit v3.5/mb_master_ui.py — lines 1121-1160

```
        window.errors += 1
        if result["kind"] == "read" and not window.stale:
            window.stale = True
            self._master_render_values(window)
        if status == "timeout":
            window.timeouts += 1
            self._master_set_message(window, "Timeout", "error")
        elif status == "exception":
            code = result.get("exception_code") or 0
            name = MODBUS_EXCEPTION_CODES.get(code, ("Unknown/Reserved Exception", ""))[0]
            self._master_set_message(window, "Exception 0x{code:02X} — {name}", "error",
                                     code=code, name=name)
        elif status == "no_connection":
            self._master_set_message(window, "Sem ligação: {error}", "error",
                                     error=result.get("error", "")[:60])
        else:
            self._master_set_message(window, "Resposta inválida (CRC ou formato): {rx}",
                                     "error", rx=result.get("rx", "")[:47])

    # -- workspace ------------------------------------------------------------
    def master_workspace_data(self):
        return {
            "format": self._master_ws_formats[0],
            "version": MASTER_WORKSPACE_VERSION,
            "connection": self._master_connection_json(),
            "windows": [w.to_json() for w in sorted(self.master_windows.values(),
                                                    key=lambda w: w.number)],
        }

    def _master_connection_json(self):
        return {
            "port": self.master_port_var.get(),
            "baud": self.master_baud_var.get(),
            "parity": self.master_parity_var.get(),
            "stop_bits": self.master_stop_var.get(),
            "response_timeout_ms": self.master_timeout_var.get(),
            "delay_ms": self.master_delay_var.get(),
        }

    def _master_apply_connection(self, connection):
```

MBToolkit v3.5/mb_master_ui.py — lines 1161-1200

```
        """Connection settings of an opened workspace (only while disconnected)."""
        if str(connection.get("baud")) in [str(b) for b in DISCOVERY_BAUD_RATES]:
            self.master_baud_var.set(str(connection["baud"]))
        if connection.get("parity") in MASTER_PARITIES:
            self.master_parity_var.set(connection["parity"])
        if str(connection.get("stop_bits")) in MASTER_STOP_BITS:
            self.master_stop_var.set(str(connection["stop_bits"]))
        for key, var in (("response_timeout_ms", self.master_timeout_var),
                         ("delay_ms", self.master_delay_var)):
            if str(connection.get(key, "")).isdigit():
                var.set(str(connection[key]))
        if connection.get("port") in tuple(self.master_port_combo["values"]):
            self.master_port_var.set(connection["port"])

    def _master_mark_changed(self):
        self.master_workspace.mark_changed()

    def _master_build_workspace_menu(self):
        """Rebuild the Workspace menu from the files in the workspace folder."""
        self.master_workspace.fill_menu(self.master_workspace_menu)
        self._master_apply_menu_palette()

    def master_save_workspace(self, name=None):
        return self.master_workspace.save(name)

    def master_delete_workspace(self, name, confirm=True):
        return self.master_workspace.delete(name, confirm)

    def master_open_workspace(self, name, quiet=False):
        return self.master_workspace.open(name, quiet)

    def _master_load_workspace_data(self, data):
        """Replace the windows with those of *data*; returns the invalid count."""
        entries, skipped = [], 0
        for item in data.get("windows", [])[:MASTER_MAX_WINDOWS]:
            try:
                entries.append(self._master_window_class.validated_kwargs(item))
            except (ValueError, TypeError):
                skipped += 1

```

MBToolkit v3.5/mb_master_ui.py — lines 1201-1240

```
        for window_id in list(self.master_windows):
            self.master_close_window(window_id)
        connection = data.get("connection", {})
        if isinstance(connection, dict) and not self.master_active:
            self._master_apply_connection(connection)
        for kwargs in entries:
            self.master_add_window(kwargs)
        return skipped

    # -- window rename ---------------------------------------------------------
    def master_rename_window(self, window_id, name=None):
        window = self.master_windows.get(window_id)
        if window is None:
            return
        if name is None:
            name = self._table_ask_text(
                self.tr("Renomear janela"),
                self.tr("Nome da janela (vazio = nome por defeito)"),
                self._master_window_name(window),
            )
            if name is None:
                return
        name = name.strip()[:MASTER_MAX_TITLE_LENGTH]
        if name == self.trf("Janela {number}", number=window.number):
            name = ""
        window.title = name
        self._master_render_status(window)
        self._master_mark_changed()

    # -- apply highlight / table tags ------------------------------------------
    def _master_fields_changed(self, window, pane):
        """True when the definition fields differ from what was last applied."""
        try:
            return (
                pane["slave_var"].get().strip() != str(window.slave)
                or pane["fc_var"].get() != fc_choice_label(window.fc)
                or pane["address_var"].get().strip() != str(window.address + window.base)
                or pane["qty_var"].get().strip() != str(window.qty)
                or pane["scan_var"].get().strip() != str(window.scan_ms)
                or FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get()) != window.fmt
```

MBToolkit v3.5/mb_master_ui.py — lines 1241-1280

```
            )
        except tk.TclError:
            return False

    def _master_update_apply_state(self, window_id):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None or "apply_btn" not in pane:
            return
        style = ("MBS.Primary.TButton" if self._master_fields_changed(window, pane)
                 else "TButton")
        try:
            pane["apply_btn"].configure(style=style)
        except tk.TclError:
            pass

    # -- right-click menu: per-address format, copy, base ---------------------------
    def _master_build_row_menu(self):
        self._master_row_menu = tk.Menu(self._table_root(), tearoff=False)

    def _master_selected_rows(self, window_id):
        return self._table_selected_rows(self._master_panes.get(window_id))

    def _master_show_row_menu(self, window_id, event):
        window = self.master_windows.get(window_id)
        pane = self._master_panes.get(window_id)
        if window is None or pane is None:
            return "break"
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if row and row not in tree.selection():
            tree.selection_set(row)
        self._table_fill_row_menu(
            self._master_row_menu, pane, window, window.fc not in BIT_FUNCTION_CODES,
            set_format=lambda fmt: self.master_set_row_format(window_id, fmt),
            edit_scale=lambda: self.master_edit_scale(window_id),
            remove_scale=lambda: self.master_remove_scale(window_id),
            copy=lambda: self.master_copy_rows(window_id),
            set_base=lambda base: self.master_set_base(window_id, base),
            extra=self._master_row_menu_extra(window),
```

MBToolkit v3.5/mb_master_ui.py — lines 1281-1320

```
        )
        self._master_apply_menu_palette()
        self._table_popup(self._master_row_menu, event)
        return "break"

    def _master_row_menu_extra(self, window):
        label, method = self._master_open_in_slave
        opener = getattr(self, method, None)
        return ((label, lambda: opener(window)),) if callable(opener) else ()

    def master_set_row_format(self, window_id, fmt):
        """Apply *fmt* (None = window format) to the selected rows."""
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        try:
            apply_format_to_rows(window, self._master_selected_rows(window_id), fmt)
        except ValueError as exc:
            size = FORMAT_SIZES.get(fmt, 1)
            messagebox.showwarning(self.tr("Formato"), self.trf(str(exc), size=size))
            return False
        self._master_render_values(window)
        self._master_mark_changed()
        return True

    # -- opening windows from Sniffer / Bus Slave Finder ------------------------------
    def master_definition_from_record(self, record):
        """Read definition (slave, fc, address, qty) behind a Traffic record, or None.

        A REQUEST carries its own address/quantity; a matched RESPONSE or
        EXCEPTION carries the ones of its request. Only FC01–FC04 qualify.
        """
        try:
            slave = int(record.get("slave", ""))
            fc = int(str(record.get("fc", "")), 16) & 0x7F
        except (TypeError, ValueError):
            return None
        if fc not in READ_FUNCTION_CODES or not 1 <= slave <= 247:
            return None
        if record.get("type") == "REQUEST":
```

MBToolkit v3.5/mb_master_ui.py — lines 1321-1360

```
            address, qty = record.get("pdu_address"), record.get("qty")
        else:
            address, qty = record.get("request_pdu_address"), record.get("request_qty")
        if address is None or qty is None:
            return None
        try:
            validate_read_definition(slave, fc, address, qty)
        except ValueError:
            return None
        return {"slave": slave, "fc": fc, "address": int(address), "qty": int(qty)}

    def master_open_from_record(self, record):
        definition = self.master_definition_from_record(record)
        if definition is None:
            messagebox.showinfo(
                self.tr("Abrir no Master"),
                self.tr("Esta trama não tem um pedido de leitura FC01–FC04 com endereço "
                        "conhecido."),
            )
            return None
        port = ""
        if not getattr(self, "capture_active", False):
            port = self.port_a_var.get().strip()
        return self.master_open_definition(
            definition, port=port, baud=self.baud_var.get(),
            parity=self.parity_var.get(), stopbits=self.stop_var.get(),
        )

    def master_open_definition(self, definition, port="", baud="", parity="", stopbits=""):
        """Add a Master window and show the Master tab.

        The serial settings (and port) are copied to the Master connection only
        when it is not connected and they are valid for the Master.
        """
        if not self.master_active:
            if str(baud) in [str(b) for b in DISCOVERY_BAUD_RATES]:
                self.master_baud_var.set(str(baud))
            if parity in MASTER_PARITIES:
                self.master_parity_var.set(parity)
            if str(stopbits) in MASTER_STOP_BITS:
```

MBToolkit v3.5/mb_master_ui.py — lines 1361-1400

```
                self.master_stop_var.set(str(stopbits))
            if port and port in tuple(self.master_port_combo["values"]):
                self.master_port_var.set(port)
        window = self.master_add_window(definition)
        try:
            self.main_notebook.select(getattr(self, self._master_page_attr))
        except tk.TclError:
            pass
        return window

    def _master_selected_group_starts(self, window):
        """PDU addresses where the values of the selected rows start."""
        return self._table_selected_group_starts(window, self._master_panes.get(window.window_id))

    def master_edit_scale(self, window_id):
        """Ask for a linear scale and apply it to the selected values."""
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        starts = self._master_selected_group_starts(window)
        if not starts:
            messagebox.showwarning(self.tr("Escala"),
                                   self.tr("Seleciona pelo menos um endereço."))
            return False
        if not self._table_ask_scale(window, starts):
            return False
        self._master_render_values(window)
        self._master_mark_changed()
        return True

    def master_remove_scale(self, window_id):
        window = self.master_windows.get(window_id)
        if window is None:
            return False
        removed = False
        for address in self._master_selected_group_starts(window):
            removed = window.scales.pop(address, None) is not None or removed
        if removed:
            self._master_render_values(window)
            self._master_mark_changed()
```

MBToolkit v3.5/mb_master_ui.py — lines 1401-1440

```
1401        return removed
1402
1403    def master_copy_rows(self, window_id):
1404        """Copy the selected rows (all rows if none) as tab-separated text."""
1405        self._table_copy_rows(self._master_panes.get(window_id))
1406
1407    def master_set_base(self, window_id, base):
1408        window = self.master_windows.get(window_id)
1409        pane = self._master_panes.get(window_id)
1410        if window is None or pane is None or window.base == base:
1411            return
1412        window.base = base
1413        pane["address_var"].set(str(window.address + window.base))
1414        self._master_fill_rows(window)
1415        self._master_render_status(window)
1416        self._master_update_apply_state(window_id)
1417        self._master_mark_changed()
1418
1419    # -- closing the application ---------------------------------------------------
1420    def master_confirm_close(self):
1421        """Ask to save a changed workspace; False cancels closing the application."""
1422        workspace = getattr(self, "master_workspace", None)
1423        return workspace is None or workspace.confirm_close()
1424
1425    # -- language / palette / shutdown ---------------------------------------------
1426    def master_apply_language(self):
1427        if not hasattr(self, "master_windows"):
1428            return
1429        self.master_workspace.update_label()
1430        for window in self.master_windows.values():
1431            pane = self._master_panes.get(window.window_id)
1432            if pane is None:
1433                continue
1434            self._table_translate_headings(pane)
1435            self._master_render_status(window)
1436        if not self.master_active:
1437            self.master_status_var.set(self.tr("Desligado"))
1438        elif self.master_engine is not None:
1439            self.master_status_var.set(self.tr("● Ligado"))
1440
```

MBToolkit v3.5/mb_master_ui.py — lines 1441-1479

```
1441    def _master_apply_menu_palette(self):
1442        self._table_apply_menus_palette(getattr(self, "master_workspace_menu", None),
1443                                        getattr(self, "_master_row_menu", None))
1444
1445    def master_apply_palette(self, dark):
1446        self._master_apply_menu_palette()
1447        colors = theme_colors(self, dark)
1448        if hasattr(self, "master_canvas"):
1449            self.master_canvas.configure(background=colors["root"])
1450        if hasattr(self, "ui_style"):
1451            self.ui_style.configure("MBS.Master.Error.TLabel",
1452                                    foreground="#ff8a80" if dark else "#b42318")
1453            self.ui_style.configure("MBS.Master.Ok.TLabel",
1454                                    foreground="#a7e4b8" if dark else "#1f6b3a")
1455        for pane in getattr(self, "_master_panes", {}).values():
1456            if pane.get("toplevel") is not None:
1457                self._table_apply_toplevel_palette(pane["toplevel"])
1458            self._table_configure_tree_tags(pane["tree"])
1459        button = getattr(self, "master_refresh_btn", None)
1460        if button is not None:
1461            button.set_dark_mode(dark)
1462
1463    def master_shutdown(self):
1464        engine = getattr(self, "master_engine", None)
1465        if engine is not None:
1466            engine.stop(join_timeout=0.8)
1467            self.master_serial = engine.ser
1468        ser = getattr(self, "master_serial", None)
1469        if ser is not None:
1470            try:
1471                ser.close()
1472            except Exception:
1473                pass
1474        self.master_engine = None
1475        self.master_serial = None
1476
1477
1478 def fc_choice_label_write(fc):
1479     return f"FC{fc:02d} {FUNCTION_NAMES[fc]}"
```

MBToolkit v3.5/mb_slave.py

629 source lines.

MBToolkit v3.5/mb_slave.py — lines 1-40

```python
#!/usr/bin/env python3
"""Modbus RTU slave (simulator) engine: data tables, replies and the serial loop.

This module has no widgets. ``SlaveStore`` holds the data tables of every
simulated Slave ID, the address areas the windows define and the fault
settings of each Slave ID. ``handle_request()`` executes one request frame
against the store and builds the reply, the way a real device would.
``SlaveEngine`` listens on one serial port, assembles the request frames,
answers them and posts what it did to the application's ``event_queue``.
"""

import math
import random
import threading
import time
from dataclasses import dataclass, field

from mb_config import (
    MASTER_MAX_READ_BITS,
    MASTER_MAX_READ_REGISTERS,
    MASTER_MAX_WRITE_BITS,
    MASTER_MAX_WRITE_REGISTERS,
    MASTER_RECONNECT_INTERVAL_SECONDS,
    SLAVE_FAULT_MAX_DELAY_MS,
    SLAVE_RESYNC_SILENCE_MS,
)
from mb_protocol import append_crc, crc_ok, modbus_rtu_interframe_gap_seconds
from mb_slave_finder import cancel_serial_io, discovery_bits_per_char, write_discovery_request

# Data tables of a Modbus device. Labels are technical terms (not translated).
TABLES = ("coils", "discrete", "holding", "input")
TABLE_LABELS = {
    "coils": "Coils",
    "discrete": "Discrete Inputs",
    "holding": "Holding Registers",
    "input": "Input Registers",
}
BIT_TABLES = ("coils", "discrete")
# Table read by FC01–FC04 and written by FC05/06/15/16.
FC_TABLES = {1: "coils", 2: "discrete", 3: "holding", 4: "input",
```

MBToolkit v3.5/mb_slave.py — lines 41-80

```
            5: "coils", 15: "coils", 6: "holding", 16: "holding"}
SUPPORTED_FUNCTION_CODES = (1, 2, 3, 4, 5, 6, 15, 16)
WRITE_FUNCTION_CODES = (5, 6, 15, 16)
# Exception codes offered by the fault simulation.
FAULT_EXCEPTION_CODES = (1, 2, 3, 4, 6)

EXCEPTION_ILLEGAL_FUNCTION = 0x01
EXCEPTION_ILLEGAL_ADDRESS = 0x02
EXCEPTION_ILLEGAL_VALUE = 0x03


# Function code that reads each table (shown with the table name).
TABLE_READ_FC = {"coils": 1, "discrete": 2, "holding": 3, "input": 4}


def table_label(table):
    return TABLE_LABELS.get(table, table)


def table_choice_label(table):
    """Table field text: read function code + name, e.g. "03 Holding Registers"."""
    return f"{TABLE_READ_FC[table]:02d} {TABLE_LABELS[table]}"


# ---------------------------------------------------------------------------
# Fault simulation
# ---------------------------------------------------------------------------

def default_fault():
    return {"delay_ms": 0, "no_response": False, "exception": 0, "bad_crc": False,
            "probability": 100}


def validate_fault(data):
    """Return a clean fault dict from user/workspace data; raise ValueError (PT)."""
    if not isinstance(data, dict):
        raise ValueError("Falha inválida.")
    try:
        delay_ms = int(str(data.get("delay_ms", 0)).strip() or 0)
        probability = int(str(data.get("probability", 100)).strip() or 0)
```

MBToolkit v3.5/mb_slave.py — lines 81-120

```
        exception = int(data.get("exception", 0) or 0)
    except ValueError:
        raise ValueError("A falha tem de ter números válidos.") from None
    if not 0 <= delay_ms <= SLAVE_FAULT_MAX_DELAY_MS:
        raise ValueError("O atraso tem de estar entre 0 e 10000 ms.")
    if not 1 <= probability <= 100:
        raise ValueError("A probabilidade tem de estar entre 1 e 100 %.")
    if exception and exception not in FAULT_EXCEPTION_CODES:
        raise ValueError("Código de exceção não suportado.")
    return {"delay_ms": delay_ms, "no_response": bool(data.get("no_response")),
            "exception": exception, "bad_crc": bool(data.get("bad_crc")),
            "probability": probability}


def fault_active(fault):
    """True when the fault settings change how the slave answers."""
    return bool(fault) and bool(fault.get("delay_ms") or fault.get("no_response")
                                or fault.get("exception") or fault.get("bad_crc"))


# --------------------------------------------------------------------------
# Automatic values
# --------------------------------------------------------------------------

# Register values: counter, ramp (sawtooth), sine and random. Bits (Coils /
# Discrete Inputs): toggle and random 0/1.
REGISTER_GENERATORS = ("counter", "ramp", "sine", "random")
BIT_GENERATORS = ("toggle", "random")
GENERATOR_MIN_PERIOD_S = 0.1
GENERATOR_MAX_PERIOD_S = 3600.0


def _number(value, message):
    try:
        number = float(str(value).strip().replace(",", "."))
    except ValueError:
        raise ValueError(message) from None
    if math.isnan(number) or math.isinf(number):
        raise ValueError(message)
    return number
```

MBToolkit v3.5/mb_slave.py — lines 121-160

```
121
122
123 def validate_generator(data, bits=False):
124     """Return a clean automatic-value dict; raise ValueError (PT canonical text).
125
126     ``period_s`` is the full cycle of a ramp / sine and the update interval of
127     a counter, random value or toggle. Registers use ``min`` / ``max`` (and
128     ``step`` for a counter) in the units shown in the window.
129     """
130     if not isinstance(data, dict):
131         raise ValueError("Valor automático inválido.")
132     kind = data.get("kind")
133     if kind not in (BIT_GENERATORS if bits else REGISTER_GENERATORS):
134         raise ValueError("Tipo de valor automático inválido.")
135     period_s = _number(data.get("period_s", 1), "O período tem de ser um número.")
136     if not GENERATOR_MIN_PERIOD_S <= period_s <= GENERATOR_MAX_PERIOD_S:
137         raise ValueError("O período tem de estar entre 0,1 e 3600 s.")
138     result = {"kind": kind, "period_s": period_s}
139     if bits:
140         return result
141     low = _number(data.get("min", 0), "O mínimo e o máximo têm de ser números.")
142     high = _number(data.get("max", 100), "O mínimo e o máximo têm de ser números.")
143     if low >= high:
144         raise ValueError("O mínimo tem de ser menor que o máximo.")
145     result.update(min=low, max=high)
146     if kind == "counter":
147         step = _number(data.get("step", 1), "O passo tem de ser um número.")
148         if step <= 0:
149             raise ValueError("O passo tem de ser maior que 0.")
150         result["step"] = step
151     return result
152
153
154 class ValueGenerator:
155     """Value of one automatic address as a function of the elapsed time.
156
157     Counter, ramp, sine and toggle are deterministic: ``value(t)`` depends only
158     on the seconds since the generator started. A random value changes once
159     per period. Wrapping: a counter returns to ``min`` after ``max``; a ramp
160     rises from ``min`` to ``max`` in one period and starts again.
```

MBToolkit v3.5/mb_slave.py — lines 161-200

```
161     """
162
163     def __init__(self, config, rng=None):
164         self.config = dict(config)
165         self.rng = rng or random.Random()
166         self._random_index = None
167         self._random_value = None
168
169     def value(self, elapsed_s):
170         c = self.config
171         period = c["period_s"]
172         cycles = max(0.0, elapsed_s) / period
173         kind = c["kind"]
174         if kind == "toggle":
175             return int(cycles) % 2
176         if kind == "random":
177             index = int(cycles)
178             if index != self._random_index:
179                 self._random_index = index
180                 if "min" in c:
181                     self._random_value = self.rng.uniform(c["min"], c["max"])
182                 else:
183                     self._random_value = self.rng.randint(0, 1)
184             return self._random_value
185         low, high = c["min"], c["max"]
186         if kind == "ramp":
187             return low + (high - low) * (cycles - math.floor(cycles))
188         if kind == "sine":
189             middle, amplitude = (low + high) / 2.0, (high - low) / 2.0
190             return middle + amplitude * math.sin(2.0 * math.pi * cycles)
191         # counter
192         count = int(math.floor((high - low) / c["step"] + 1e-9)) + 1
193         return low + (int(cycles) % count) * c["step"]
194
195
196 # ---------------------------------------------------------------------------
197 # Data store
198 # ---------------------------------------------------------------------------
199
200 class SlaveStore:
```

MBToolkit v3.5/mb_slave.py — lines 201-240

```
    """Data tables, defined areas and faults of every simulated Slave ID.

    Values live per (Slave ID, table) and address and start at 0. A request
    is valid only for addresses inside an area defined by a window; windows
    over the same addresses share the values. Thread-safe: the engine thread
    reads and writes while the GUI edits.
    """

    def __init__(self):
        self._lock = threading.RLock()
        self._values = {}    # (slave, table) -> {address: value}
        self._defined = {}   # (slave, table) -> set of addresses
        self._faults = {}    # slave -> fault dict

    def set_areas(self, areas):
        """Replace the defined areas: iterable of (slave, table, address, qty)."""
        defined = {}
        for slave, table, address, qty in areas:
            defined.setdefault((int(slave), table), set()).update(
                range(int(address), int(address) + int(qty)))
        with self._lock:
            self._defined = defined

    def slave_ids(self):
        with self._lock:
            return {slave for slave, _table in self._defined}

    def covers(self, slave, table, address, qty):
        with self._lock:
            defined = self._defined.get((slave, table))
            return bool(defined) and all(a in defined for a in range(address, address + qty))

    def read(self, slave, table, address, qty):
        with self._lock:
            values = self._values.get((slave, table), {})
            return [values.get(a, 0) for a in range(address, address + qty)]

    def write(self, slave, table, address, values):
        mask = 1 if table in BIT_TABLES else 0xFFFF
        with self._lock:
```

MBToolkit v3.5/mb_slave.py — lines 241-280

```
            target = self._values.setdefault((slave, table), {})
            for offset, value in enumerate(values):
                target[address + offset] = int(value) & mask

    def clear_values(self):
        with self._lock:
            self._values = {}

    def set_fault(self, slave, fault):
        with self._lock:
            if fault_active(fault):
                self._faults[int(slave)] = dict(fault)
            else:
                self._faults.pop(int(slave), None)

    def fault(self, slave):
        with self._lock:
            fault = self._faults.get(slave)
            return dict(fault) if fault else None

    def faults(self):
        with self._lock:
            return {slave: dict(fault) for slave, fault in self._faults.items()}

    def clear_faults(self):
        with self._lock:
            self._faults = {}


# ---------------------------------------------------------------------------
# Request handling
# ---------------------------------------------------------------------------

def request_length(buffer):
    """Length of the request frame starting at buffer[0]; None if not known yet.

    FC01–FC06 requests are 8 bytes; FC15/FC16 carry a byte count in byte 6.
    Unknown function codes have no known length (None).
    """
    if len(buffer) < 2:
```

MBToolkit v3.5/mb_slave.py — lines 281-320

```
        return None
    fc = buffer[1]
    if fc in (1, 2, 3, 4, 5, 6):
        return 8
    if fc in (15, 16):
        return 9 + buffer[6] if len(buffer) >= 7 else None
    return None


def exception_frame(slave, fc, code):
    return append_crc(bytes((slave, fc | 0x80, code)))


@dataclass
class SlaveReply:
    """Outcome of one request: what to send back and what it touched."""
    slave: int
    fc: int
    status: str                 # ok, exception, broadcast, ignored
    table: str = None
    address: int = None
    qty: int = None
    exception_code: int = None
    response: bytes = None
    written: list = field(default_factory=list)   # [(slave, table, address, qty)]


def _u16(data, index):
    return (data[index] << 8) | data[index + 1]


def _parse_request(frame):
    """Return (table, address, qty, values or None) or raise _ExceptionReply."""
    fc = frame[1]
    if fc not in SUPPORTED_FUNCTION_CODES:
        raise _ExceptionReply(EXCEPTION_ILLEGAL_FUNCTION)
    table = FC_TABLES[fc]
    if fc in (15, 16):
        if len(frame) < 9 or len(frame) != 9 + frame[6]:
            raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
```

MBToolkit v3.5/mb_slave.py — lines 321-360

```
321     elif len(frame) != 8:
322         raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
323     address = _u16(frame, 2)
324 
325     if fc in (1, 2, 3, 4):
326         qty = _u16(frame, 4)
327         limit = MASTER_MAX_READ_BITS if fc in (1, 2) else MASTER_MAX_READ_REGISTERS
328         if not 1 <= qty <= limit:
329             raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
330         return table, address, qty, None
331     if fc == 5:
332         raw = _u16(frame, 4)
333         if raw not in (0xFF00, 0x0000):
334             raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
335         return table, address, 1, [1 if raw else 0]
336     if fc == 6:
337         return table, address, 1, [_u16(frame, 4)]
338 
339     qty, byte_count = _u16(frame, 4), frame[6]
340     data = frame[7:-2]
341     if fc == 15:
342         if not 1 <= qty <= MASTER_MAX_WRITE_BITS or byte_count != (qty + 7) // 8:
343             raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
344         return table, address, qty, [(data[i // 8] >> (i % 8)) & 1 for i in range(qty)]
345     if not 1 <= qty <= MASTER_MAX_WRITE_REGISTERS or byte_count != 2 * qty:
346         raise _ExceptionReply(EXCEPTION_ILLEGAL_VALUE)
347     return table, address, qty, [_u16(data, 2 * i) for i in range(qty)]
348 
349 
350 class _ExceptionReply(Exception):
351     def __init__(self, code):
352         super().__init__(code)
353         self.code = code
354 
355 
356 def handle_request(store, frame):
357     """Execute one CRC-valid request frame and return a SlaveReply.
358 
359     Requests for Slave IDs that are not simulated are ``ignored``. Broadcast
360     (Slave 0) writes are executed on every simulated Slave ID whose areas
```

MBToolkit v3.5/mb_slave.py — lines 361-400

```
    cover the addresses, and never answered. Errors answer like a real
    device: Exception 01 (function), 02 (address outside the defined areas)
    or 03 (quantity / value / byte count).
    """
    slave, fc = frame[0], frame[1] & 0x7F
    reply = SlaveReply(slave=slave, fc=fc, status="ignored")
    if slave != 0 and slave not in store.slave_ids():
        return reply

    try:
        table, address, qty, values = _parse_request(frame)
        reply.table, reply.address, reply.qty = table, address, qty
        if address + qty > 0x10000:
            raise _ExceptionReply(EXCEPTION_ILLEGAL_ADDRESS)
    except _ExceptionReply as exc:
        if slave == 0:
            reply.status = "broadcast"
            return reply
        reply.status = "exception"
        reply.exception_code = exc.code
        reply.response = exception_frame(slave, fc, exc.code)
        return reply

    if slave == 0:
        reply.status = "broadcast"
        if values is not None:
            for target in sorted(store.slave_ids()):
                if store.covers(target, table, address, qty):
                    store.write(target, table, address, values)
                    reply.written.append((target, table, address, qty))
        return reply

    if not store.covers(slave, table, address, qty):
        reply.status = "exception"
        reply.exception_code = EXCEPTION_ILLEGAL_ADDRESS
        reply.response = exception_frame(slave, fc, EXCEPTION_ILLEGAL_ADDRESS)
        return reply

    reply.status = "ok"
    if values is not None:
```

MBToolkit v3.5/mb_slave.py — lines 401-440

```
401         store.write(slave, table, address, values)
402         reply.written.append((slave, table, address, qty))
403         reply.response = bytes(frame) if fc in (5, 6) else append_crc(bytes(frame[:6]))
404         return reply
405 
406     current = store.read(slave, table, address, qty)
407     if fc in (1, 2):
408         data = bytearray((qty + 7) // 8)
409         for index, bit in enumerate(current):
410             if bit:
411                 data[index // 8] |= 1 << (index % 8)
412     else:
413         data = b"".join(bytes(((v >> 8) & 0xFF, v & 0xFF)) for v in current)
414     reply.response = append_crc(bytes((slave, fc, len(data))) + bytes(data))
415     return reply
416 
417 
418 def corrupt_crc(frame):
419     """Same frame with a wrong CRC (fault simulation)."""
420     frame = bytearray(frame)
421     frame[-1] ^= 0xFF
422     return bytes(frame)
423 
424 
425 # ---------------------------------------------------------------------------
426 # Engine
427 # ---------------------------------------------------------------------------
428 
429 class SlaveEngine:
430     """Answer the requests of a real master on one serial port.
431 
432     Request frames are taken from the received bytes by their length (FC01–06
433     and FC15/16 have a known size), so a request is answered as soon as it is
434     complete. Bytes that do not form a CRC-valid request (other devices'
435     responses, noise, unknown function codes) are discarded one at a time
436     until a request lines up, at the latest after SLAVE_RESYNC_SILENCE_MS of
437     silence. The echo of our own response from an RS485 adapter is removed.
438 
439     Before answering, the engine waits the Modbus t3.5 gap, and the fault
440     simulation of the Slave ID (if any) may delay, drop, replace by an
```

MBToolkit v3.5/mb_slave.py — lines 441-480

```
    exception or corrupt the response.

    With *reopen*, a serial failure does not end the session: the engine posts
    ``slave_reconnecting`` and reopens the port like the Master does.
    """

    def __init__(self, ser, event_queue, store, *, baud, parity, stopbits,
                 serial_module=None, reopen=None,
                 reconnect_interval_s=MASTER_RECONNECT_INTERVAL_SECONDS,
                 silence_ms=SLAVE_RESYNC_SILENCE_MS, rng=None):
        self.ser = ser
        self.event_queue = event_queue
        self.store = store
        self.baud = int(baud)
        self.parity = parity
        self.stopbits = stopbits
        self.serial_module = serial_module
        self.reopen = reopen
        self.reconnect_interval_s = float(reconnect_interval_s)
        bits_per_char = discovery_bits_per_char(parity, stopbits)
        self.char_s = bits_per_char / float(max(1, self.baud))
        self.gap_s = modbus_rtu_interframe_gap_seconds(self.baud, bits_per_char)
        self.silence_s = max(self.gap_s, float(silence_ms) / 1000.0)
        self.rng = rng or random.Random()
        self.stop_event = threading.Event()
        self.thread = None
        self._buffer = bytearray()
        self._echo = b""
        self._echo_deadline = 0.0

    def start(self):
        self.thread = threading.Thread(target=self.run, daemon=True)
        self.thread.start()

    def stop(self, join_timeout=1.0):
        self.stop_event.set()
        cancel_serial_io(self.ser)
        thread = self.thread
        if thread is not None and thread.is_alive() and thread is not threading.current_thread():
            thread.join(timeout=join_timeout)
```

MBToolkit v3.5/mb_slave.py — lines 481-520

```
481
482     # -- worker ------------------------------------------------------------
483     def run(self):
484         error = None
485         try:
486             self.event_queue.put(("slave_ready", None))
487             while not self.stop_event.is_set():
488                 try:
489                     self._listen()
490                 except Exception as exc:  # serial/driver failure
491                     if self.stop_event.is_set():
492                         break
493                     detail = f"{type(exc).__name__}: {exc}"
494                     if self.reopen is None:
495                         error = detail
496                         break
497                     if not self._reconnect(detail):
498                         break
499         finally:
500             self.event_queue.put(("slave_stopped", {"error": error}))
501
502     def _reconnect(self, detail):
503         try:
504             self.ser.close()
505         except Exception:
506             pass
507         self._buffer.clear()
508         self.event_queue.put(("slave_reconnecting", {"error": detail}))
509         while not self.stop_event.wait(self.reconnect_interval_s):
510             try:
511                 ser = self.reopen()
512             except Exception:
513                 continue
514             self.ser = ser
515             self.event_queue.put(("slave_reconnected", None))
516             return True
517         return False
518
519     def _listen(self):
520         """Read and answer until stopped; serial errors propagate."""
```

MBToolkit v3.5/mb_slave.py — lines 521-560

```
        last_rx = time.perf_counter()
        while not self.stop_event.is_set():
            waiting = int(getattr(self.ser, "in_waiting", 0) or 0)
            if waiting:
                data = self._strip_echo(self.ser.read(waiting) or b"")
                last_rx = time.perf_counter()
                if data:
                    self._buffer.extend(data)
                    self.process(final=False)
                continue
            if self._buffer and time.perf_counter() - last_rx >= self.silence_s:
                self.process(final=True)
            if self.stop_event.wait(0.001):
                return

    def process(self, final):
        """Answer every complete request in the buffer.

        *final* (the bus went silent) also resolves what is left: a frame of
        unknown length is answered if the whole rest is CRC-valid, anything
        else is dropped byte by byte.
        """
        buffer = self._buffer
        while buffer and not self.stop_event.is_set():
            length = request_length(buffer)
            if length is None or len(buffer) < length:
                if not final:
                    return
                if length is None and len(buffer) >= 4 and crc_ok(bytes(buffer)):
                    frame = bytes(buffer)
                    buffer.clear()
                    self._answer(frame)
                    return
                del buffer[0]
                continue
            frame = bytes(buffer[:length])
            if crc_ok(frame):
                del buffer[:length]
                self._answer(frame)
            else:
```

MBToolkit v3.5/mb_slave.py — lines 561-600

```
561                 del buffer[0]
562
563     def _strip_echo(self, data):
564         """Remove the RS485 adapter echo of our last response from *data*."""
565         if not self._echo:
566             return data
567         if time.perf_counter() > self._echo_deadline:
568             self._echo = b""
569             return data
570         matched = 0
571         while (matched < len(data) and matched < len(self._echo)
572                and data[matched] == self._echo[matched]):
573             matched += 1
574         if matched == len(data) or matched == len(self._echo):
575             self._echo = self._echo[matched:]
576             return data[matched:]
577         self._echo = b""
578         return data
579
580     def _answer(self, frame):
581         slave = frame[0]
582         fault = None
583         if slave != 0 and slave in self.store.slave_ids():
584             fault = self.store.fault(slave)
585             if fault and self.rng.random() * 100.0 >= fault["probability"]:
586                 fault = None
587
588         if fault and fault["no_response"]:
589             reply = SlaveReply(slave=slave, fc=frame[1] & 0x7F, status="no_response")
590         elif fault and fault["exception"]:
591             code = fault["exception"]
592             reply = SlaveReply(slave=slave, fc=frame[1] & 0x7F, status="exception",
593                                exception_code=code,
594                                response=exception_frame(slave, frame[1] & 0x7F, code))
595         else:
596             reply = handle_request(self.store, frame)
597         if reply.table is None and reply.fc in FC_TABLES and len(frame) >= 6:
598             # Fault replies and ignored requests still name the addresses, so
599             # the windows over them can count the request.
600             reply.table, reply.address = FC_TABLES[reply.fc], _u16(frame, 2)
```

MBToolkit v3.5/mb_slave.py — lines 601-629

```
601             reply.qty = 1 if reply.fc in (5, 6) else _u16(frame, 4)
602
603         response = reply.response
604         if response is not None and fault and fault["bad_crc"]:
605             response = corrupt_crc(response)
606         info = {
607             "slave": reply.slave, "fc": reply.fc, "table": reply.table,
608             "address": reply.address, "qty": reply.qty, "status": reply.status,
609             "exception_code": reply.exception_code, "written": list(reply.written),
610             "fault": bool(fault), "time": time.time(),
611         }
612         if response is not None:
613             delay_s = self.gap_s + (fault["delay_ms"] / 1000.0 if fault else 0.0)
614             if self.stop_event.wait(delay_s):
615                 return
616             # A request that arrived meanwhile belongs to another exchange; a
617             # real slave would not answer over it, but answering late is the
618             # best a PC can do. Anything received before our reply is dropped.
619             self._buffer.clear()
620             sent = write_discovery_request(
621                 self.ser, response, self.baud, self.parity, self.stopbits,
622                 self.stop_event, self.serial_module,
623             )
624             if not sent:
625                 return
626             self._echo = bytes(response)
627             self._echo_deadline = time.perf_counter() + len(response) * self.char_s + 0.2
628             info["response"] = len(response)
629         self.event_queue.put(("slave_request", info))
```

MBToolkit v3.5/mb_slave_ui.py

1668 source lines.

MBToolkit v3.5/mb_slave_ui.py — lines 1-40

```python
#!/usr/bin/env python3
"""Slave (simulator) tab: the application answers a real master as Modbus slaves.

The data tables, request handling and serial loop live in ``mb_slave.py``;
this mixin owns the widgets. Each window shows one data table (Coils,
Discrete Inputs, Holding or Input Registers) of one simulated Slave ID; the
windows define which addresses exist. Values start at 0, are edited by
double-click and change when the master writes. Fault simulation is set per
Slave ID. The window grid, value table and Workspace menu are shared with the
Master tab (``mb_table_ui.py``).
"""

import time
import tkinter as tk
from tkinter import messagebox, ttk

from mb_config import (
    DISCOVERY_BAUD_RATES,
    MODBUS_EXCEPTION_CODES,
    SLAVE_FAULT_MAX_DELAY_MS,
    SLAVE_GENERATOR_TICK_MS,
    SLAVE_HIGHLIGHT_SECONDS,
    SLAVE_MAX_QTY,
    SLAVE_MAX_WINDOWS,
    SLAVE_WORKSPACE_FORMAT,
    SLAVE_WORKSPACE_VERSION,
    get_slave_workspaces_folder,
)
from mb_master import (
    DEFAULT_FORMAT,
    FORMAT_KEYS_BY_LABEL,
    FORMAT_LABELS,
    FORMAT_SIZES,
    encode_coil_value,
    encode_register_value,
    format_window_values,
    READ_FUNCTION_CODES,
    raw_text_for_scaled,
    register_groups,
    unscale_value,
```

MBToolkit v3.5/mb_slave_ui.py — lines 41-80

```
)
from mb_slave import (
    BIT_GENERATORS,
    BIT_TABLES,
    FAULT_EXCEPTION_CODES,
    FC_TABLES,
    REGISTER_GENERATORS,
    TABLE_READ_FC,
    TABLES,
    SlaveEngine,
    SlaveStore,
    default_fault,
    fault_active,
    table_choice_label,
    table_label,
    validate_fault,
    validate_generator,
    ValueGenerator,
)
from mb_table_ui import (
    SERIAL_PARITIES,
    SERIAL_STOP_BITS,
    TABLE_MAX_TITLE_LENGTH,
    TABLE_PANE_MAX_ROWS,
    TABLE_PANE_MIN_ROWS,
    TABLE_PANE_MIN_WIDTH,
    TableUiMixin,
    TableWorkspace,
    apply_format_to_rows,
    parse_int_field,
    validated_formats,
    validated_names,
    validated_scales,
)
from mb_theme import theme_colors
from mb_widgets import RefreshButton

SLAVE_WHEEL_TAG = "MBToolkit.SlaveWheel"
# Automatic value kinds as shown in the dialog (PT canonical captions).
GENERATOR_LABELS = {"counter": "Contador", "ramp": "Rampa", "sine": "Senoide",
```

MBToolkit v3.5/mb_slave_ui.py — lines 81-120

```
                    "random": "Aleatório", "toggle": "Alternar 0/1"}
TABLE_CHOICES = tuple(table_choice_label(t) for t in TABLES)
TABLE_KEYS_BY_LABEL = {table_choice_label(t): t for t in TABLES}


def fault_parts(fault, bad_reply="CRC errado"):
    """(PT template, values) pieces describing an active fault.

    *bad_reply* names the corrupted-response fault (wrong CRC on RTU, invalid
    MBAP header on TCP).
    """
    parts = []
    if fault.get("no_response"):
        parts.append(("sem resposta", {}))
    if fault.get("exception"):
        parts.append(("Exception {code:02X}", {"code": fault["exception"]}))
    if fault.get("bad_crc"):
        parts.append((bad_reply, {}))
    if fault.get("delay_ms"):
        parts.append(("atraso {ms} ms", {"ms": fault["delay_ms"]}))
    return parts


class SlaveWindow:
    """State of one simulated data table (independent of where it is shown)."""

    def __init__(self, window_id, number, slave=1, table="holding", address=0, qty=10,
                 base=0, fmt=DEFAULT_FORMAT, names=None, title="", formats=None, scales=None,
                 generators=None):
        self.window_id = window_id
        self.number = number
        # User-chosen window name; empty means the default "Janela N".
        self.title = str(title or "").strip()[:TABLE_MAX_TITLE_LENGTH]
        self.slave = slave
        self.table = table
        self.address = address
        self.qty = qty
        self.base = base
        self.fmt = fmt
        self.names = dict(names or {})  # PDU address -> label
```

MBToolkit v3.5/mb_slave_ui.py — lines 121-160

```
        self.formats = {int(a): f for a, f in (formats or {}).items() if f in FORMAT_SIZES}
        self.scales = {int(a): dict(s) for a, s in (scales or {}).items()}
        # PDU address -> automatic value (validate_generator()) of the value
        # that starts there.
        self.generators = {int(a): dict(g) for a, g in (generators or {}).items()}
        self.popped = False
        # Row index -> time.monotonic() until which it is shown as written.
        self.highlight = {}
        self.reset_counters()

    def reset_counters(self):
        self.requests = 0
        self.exceptions = 0
        self.last = ""

    @property
    def bits(self):
        return self.table in BIT_TABLES

    def area(self):
        return (self.slave, self.table, self.address, self.qty)

    def overlap(self, address, qty):
        """Row range of this window covered by [address, address + qty), or None."""
        first = max(self.address, address)
        last = min(self.address + self.qty, address + qty)
        return (first - self.address, last - self.address) if first < last else None

    def groups(self):
        return register_groups(self.qty, self.address, self.fmt, self.formats)

    def group_of_row(self, row):
        for offset, size, fmt in self.groups():
            if offset <= row < offset + size:
                return offset, size, fmt
        return row, 1, None

    def to_json(self):
        return {
            "title": self.title, "slave": self.slave, "table": self.table,
```

MBToolkit v3.5/mb_slave_ui.py — lines 161-200

```
            "address": self.address, "qty": self.qty, "base": self.base,
            "format": self.fmt,
            "names": {str(a): n for a, n in sorted(self.names.items())},
            "formats": {str(a): f for a, f in sorted(self.formats.items())},
            "scales": {str(a): s for a, s in sorted(self.scales.items())},
            "generators": {str(a): g for a, g in sorted(self.generators.items())},
        }

    def drop_invalid_generators(self):
        """Keep only automatic values that still fit the window's area and table."""
        kept = {}
        for address, config in self.generators.items():
            if self.address <= address < self.address + self.qty:
                try:
                    kept[address] = validate_generator(config, self.bits)
                except ValueError:
                    continue
        self.generators = kept

    @staticmethod
    def validated_kwargs(data):
        """Validate one workspace window entry; raise ValueError if invalid."""
        if not isinstance(data, dict):
            raise ValueError("Janela inválida.")
        slave, table = int(data.get("slave", 1)), data.get("table", "holding")
        address, qty = int(data.get("address", 0)), int(data.get("qty", 10))
        validate_area(slave, table, address, qty)
        fmt = data.get("format", DEFAULT_FORMAT)
        return {
            "title": str(data.get("title", "") or "").strip()[:TABLE_MAX_TITLE_LENGTH],
            "slave": slave, "table": table, "address": address, "qty": qty,
            "base": 1 if data.get("base") == 1 else 0,
            "fmt": fmt if fmt in FORMAT_SIZES else DEFAULT_FORMAT,
            "names": validated_names(data.get("names", {})),
            "formats": validated_formats(data.get("formats", {})),
            "scales": validated_scales(data.get("scales", {})),
            "generators": validated_generators(data.get("generators", {}), table in BIT_TABLES),
        }


```

MBToolkit v3.5/mb_slave_ui.py — lines 201-240

```
201 def validated_generators(generators, bits):
202     """Automatic values of a workspace window entry; invalid ones are dropped."""
203     result = {}
204     for key, item in (generators.items() if isinstance(generators, dict) else ()):
205         try:
206             if str(key).isdigit() and int(key) <= 0xFFFF:
207                 result[int(key)] = validate_generator(item, bits)
208         except ValueError:
209             continue
210     return result
211 
212 
213 def slave_definition_from_record(record):
214     """Slave window (slave, table, address, qty) behind a Traffic record, or None.
215 
216     Reads (FC01–FC04) give their table and requested area; writes (FC05/06/
217     15/16) the written Coils / Holding Registers. A REQUEST carries its own
218     address; a RESPONSE/EXCEPTION of a read carries the one of its request.
219     """
220     try:
221         slave = int(record.get("slave", ""))
222         fc = int(str(record.get("fc", "")), 16) & 0x7F
223     except (TypeError, ValueError):
224         return None
225     if fc not in FC_TABLES or not 1 <= slave <= 247:
226         return None
227     if record.get("type") == "REQUEST" or (fc not in READ_FUNCTION_CODES
228                                            and record.get("type") == "RESPONSE"):
229         address, qty = record.get("pdu_address"), record.get("qty")
230     else:
231         address, qty = record.get("request_pdu_address"), record.get("request_qty")
232     if fc in (5, 6):
233         qty = 1
234     if address is None or qty is None:
235         return None
236     try:
237         validate_area(slave, FC_TABLES[fc], address, qty)
238     except (ValueError, TypeError):
239         return None
240     return {"slave": slave, "table": FC_TABLES[fc], "address": int(address), "qty": int(qty)}
```

MBToolkit v3.5/mb_slave_ui.py — lines 241-280

```
241
242
243 def validate_area(slave, table, address, qty):
244     """Raise ValueError (PT canonical message) for an invalid window area."""
245     if not 1 <= int(slave) <= 247:
246         raise ValueError("O Slave ID tem de estar entre 1 e 247.")
247     if table not in TABLES:
248         raise ValueError("Tabela inválida.")
249     if not 0 <= int(address) <= 0xFFFF:
250         raise ValueError("O endereço tem de estar entre 0 e 65535.")
251     if not 1 <= int(qty) <= SLAVE_MAX_QTY:
252         raise ValueError("A quantidade tem de estar entre 1 e 2000.")
253     if int(address) + int(qty) > 0x10000:
254         raise ValueError("O intervalo de endereços ultrapassa 65535.")
255
256
257 class SlaveMixin(TableUiMixin):
258     """Slave (simulator) tab GUI.
259
260     The class attributes below describe this tab. The Modbus TCP Server tab
261     (``mb_tcp_ui.ServerTab``) is a second instance of this mixin with its own
262     state and store; it overrides them and the connection-specific methods
263     (``_slave_build_connection``, ``slave_connect``, ...).
264     """
265
266     _slave_tab_name = "Slave"
267     _slave_settings_key = "slave"
268     _slave_page_attr = "slave_page"
269     _slave_wheel_tag = SLAVE_WHEEL_TAG
270     _slave_ws_folder = staticmethod(get_slave_workspaces_folder)
271     _slave_ws_format = SLAVE_WORKSPACE_FORMAT
272     _slave_ws_close_question = "Guardar alterações ao workspace do Slave \"{name}\"?"
273     _slave_id_label = "Slave"
274     _slave_fault_title = "Falhas do Slave {slave}"
275     _slave_fault_intro = ("Simulação de falhas do Slave {slave} (aplica-se a todas as janelas "
276                           "deste ID).")
277     # Corrupted-response fault: header text and dialog option.
278     _slave_bad_reply = "CRC errado"
279     _slave_bad_reply_option = "CRC errado nas respostas"
280
```

MBToolkit v3.5/mb_slave_ui.py — lines 281-320

```
    # -- construction -----------------------------------------------------------
    def build_slave_tab(self, page):
        saved = getattr(self, "ui_settings", {}).get(self._slave_settings_key, {})
        if not isinstance(saved, dict):
            saved = {}

        self.slave_windows = {}
        self._slave_panes = {}
        self._slave_next_id = 1
        self.slave_store = SlaveStore()
        # Slave ID -> fault settings (active ones only).
        self.slave_faults = {}
        self.slave_engine = None
        self.slave_serial = None
        self.slave_active = False
        self._slave_reflow_after_id = None
        self._slave_tick_after_id = None
        # (window_id, PDU address) -> (ValueGenerator, start time) of the
        # automatic values; recomputed every SLAVE_GENERATOR_TICK_MS.
        self._slave_generators = {}
        self._slave_generator_after_id = None
        self._slave_reset_stats()

        page.columnconfigure(0, weight=1)
        page.rowconfigure(2, weight=1)

        self._slave_build_connection(page, saved)
        self._slave_render_stats()

        toolbar = ttk.Frame(page)
        toolbar.grid(row=1, column=0, sticky="ew", padx=12, pady=(0, 6))
        toolbar.columnconfigure(3, weight=1)
        ttk.Button(toolbar, text="Nova janela", command=self.slave_add_window).grid(
            row=0, column=0, padx=(0, 6))
        self.slave_workspace_btn = ttk.Menubutton(toolbar, text="Workspace")
        self.slave_workspace_btn.grid(row=0, column=1, padx=(0, 6))
        self.slave_workspace_menu = tk.Menu(
            self.slave_workspace_btn, tearoff=False,
            postcommand=self._slave_build_workspace_menu,
        )
```

MBToolkit v3.5/mb_slave_ui.py — lines 321-360

```
        self.slave_workspace_btn.configure(menu=self.slave_workspace_menu)
        self.slave_workspace_var = tk.StringVar(value="")
        ttk.Label(toolbar, textvariable=self.slave_workspace_var).grid(
            row=0, column=2, padx=(0, 6))
        self.slave_workspace = TableWorkspace(
            self, folder=self._slave_ws_folder, formats=(self._slave_ws_format,),
            data=self.slave_workspace_data, load=self._slave_load_workspace_data,
            label_var=self.slave_workspace_var,
            close_question=self._slave_ws_close_question,
        )
        ttk.Label(
            toolbar,
            text=("Duplo clique num valor para o alterar, num nome para o editar. "
                  "Clique direito: formato, escala, copiar e base."),
        ).grid(row=0, column=3, sticky="w", padx=(12, 0))

        self.slave_canvas, self.slave_grid, self._slave_grid_item = \
            self._table_build_scroll_grid(page, 2)
        self.slave_canvas.bind("<Configure>", self._slave_schedule_reflow, add="+")
        self.slave_empty_label = ttk.Label(
            self.slave_grid,
            text="Sem janelas. Usa \"Nova janela\" para simular um Slave.",
        )
        for sequence in ("<MouseWheel>", "<Button-4>", "<Button-5>"):
            self.bind_class(self._slave_wheel_tag, sequence,
                            lambda e: self._table_wheel(self.slave_canvas, e))

        self._slave_row_menu = tk.Menu(self._table_root(), tearoff=False)

        # Reopen the workspace used last time (never starts listening).
        last = saved.get("last_workspace", "")
        if last and last in self.slave_workspace.names():
            self.after(10, lambda: self._slave_restore_workspace(last))
        else:
            self.slave_add_window()

    def _slave_build_connection(self, page, saved):
        """Connection bar of the serial Slave (COM port, serial format, counters)."""
        connection = ttk.LabelFrame(
            page, text="Ligação — o Slave responde aos pedidos de um master real")
```

MBToolkit v3.5/mb_slave_ui.py — lines 361-400

```
361         connection.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
362         row = ttk.Frame(connection)
363         row.grid(row=0, column=0, sticky="w", padx=8, pady=(8, 4))
364 
365         ttk.Label(row, text="Porta COM").grid(row=0, column=0, padx=(0, 4))
366         self.slave_port_var = tk.StringVar()
367         self.slave_port_combo = ttk.Combobox(
368             row, textvariable=self.slave_port_var, width=13, state="readonly"
369         )
370         self.slave_port_combo.grid(row=0, column=1)
371         self.slave_refresh_btn = RefreshButton(row, command=self.refresh_ports, size=24)
372         self.slave_refresh_btn.grid(row=0, column=2, padx=(4, 18))
373 
374         def _sync_refresh_square(_event=None):
375             height = self.slave_port_combo.winfo_height()
376             if height > 1:
377                 self.slave_refresh_btn.configure(width=height, height=height)
378 
379         self.slave_port_combo.bind("<Configure>", _sync_refresh_square, add="+")
380 
381         self.slave_baud_var = tk.StringVar(value=str(saved.get("baud", "9600")))
382         self.slave_parity_var = tk.StringVar(value=saved.get("parity", "None"))
383         self.slave_stop_var = tk.StringVar(value=str(saved.get("stop_bits", "1")))
384         fields = (
385             ("Baud rate", ttk.Combobox(row, textvariable=self.slave_baud_var, width=8,
386                                        state="readonly",
387                                        values=[str(b) for b in DISCOVERY_BAUD_RATES])),
388             ("Parity", ttk.Combobox(row, textvariable=self.slave_parity_var, width=6,
389                                     state="readonly", values=SERIAL_PARITIES)),
390             ("Stop bits", ttk.Combobox(row, textvariable=self.slave_stop_var, width=3,
391                                        state="readonly", values=SERIAL_STOP_BITS)),
392         )
393         column = 3
394         self._slave_setting_widgets = [self.slave_port_combo]
395         for caption, widget in fields:
396             ttk.Label(row, text=caption).grid(row=0, column=column, padx=(0, 4))
397             widget.grid(row=0, column=column + 1, padx=(0, 16))
398             self._slave_setting_widgets.append(widget)
399             column += 2
400         ttk.Label(row, text="Data bits: 8").grid(row=0, column=column)
```

MBToolkit v3.5/mb_slave_ui.py — lines 401-440

```
401
402         controls = ttk.Frame(connection)
403         controls.grid(row=1, column=0, sticky="w", padx=8, pady=(0, 4))
404         self.slave_safety_var = tk.BooleanVar(value=False)
405         ttk.Checkbutton(
406             controls,
407             text="Confirmo que nenhum equipamento no barramento usa os Slave IDs simulados.",
408             variable=self.slave_safety_var,
409         ).grid(row=0, column=0, padx=(0, 18))
410         self.slave_connect_btn = ttk.Button(
411             controls, text="Ligar", command=self.slave_connect, style="MBS.Primary.TButton"
412         )
413         self.slave_connect_btn.grid(row=0, column=1, padx=(0, 6))
414         self.slave_disconnect_btn = ttk.Button(
415             controls, text="Desligar", command=self.slave_disconnect, state="disabled"
416         )
417         self.slave_disconnect_btn.grid(row=0, column=2, padx=(0, 18))
418         self.slave_status_var = tk.StringVar(value=self.tr("Desligado"))
419         ttk.Label(controls, textvariable=self.slave_status_var).grid(row=0, column=3)
420         self.slave_stats_var = tk.StringVar()
421         ttk.Label(connection, textvariable=self.slave_stats_var).grid(
422             row=2, column=0, sticky="w", padx=8, pady=(0, 8))
423
424         for variable in (self.slave_baud_var, self.slave_parity_var, self.slave_stop_var):
425             variable.trace_add("write", lambda *_a: self._schedule_ui_settings_save())
426
427     def _slave_restore_workspace(self, name):
428         if not self.slave_open_workspace(name, quiet=True):
429             self.slave_add_window()
430
431     # -- texts -------------------------------------------------------------
432     def _slave_reset_stats(self):
433         self.slave_stats = {"requests": 0, "responses": 0, "exceptions": 0,
434                             "broadcasts": 0, "others": 0}
435
436     def _slave_render_stats(self):
437         stats = self.slave_stats
438         self.slave_stats_var.set(self.trf(
439             "Pedidos: {requests}   Respostas: {responses}   Exceções: {exceptions}   "
440             "Broadcast: {broadcasts}   Para outros IDs: {others}", **stats))
```

MBToolkit v3.5/mb_slave_ui.py — lines 441-480

```
441
442     def _slave_window_name(self, window):
443         return window.title or self.trf("Janela {number}", number=window.number)
444
445     def _slave_window_summary(self, window):
446         return (f"{self._slave_id_label} {window.slave} · FC{TABLE_READ_FC[window.table]:02d} "
447                 f"{table_label(window.table)} · "
448                 f"{window.address + window.base} · {window.qty}")
449
450     def _slave_fault_text(self, slave):
451         fault = self.slave_faults.get(slave)
452         if not fault_active(fault):
453             return ""
454         text = ", ".join(self.trf(template, **values)
455                          for template, values in fault_parts(fault, self._slave_bad_reply))
456         if fault["probability"] < 100:
457             text += f" · {fault['probability']}%"
458         return self.trf("⚠ Falha simulada: {fault}", fault=text)
459
460     def _slave_render_status(self, window):
461         pane = self._slave_panes.get(window.window_id)
462         if pane is None:
463             return
464         try:
465             pane["counters_var"].set(self.trf(
466                 "Pedidos: {requests}   Exceções: {exceptions}",
467                 requests=window.requests, exceptions=window.exceptions))
468             pane["last_var"].set(self.trf("Último: {request}", request=window.last)
469                                  if window.last else "")
470             pane["name_var"].set(self._slave_window_name(window))
471             pane["summary_var"].set(self._slave_window_summary(window))
472             fault = self._slave_fault_text(window.slave)
473             pane["fault_var"].set(fault)
474             if fault:
475                 pane["fault_label"].grid()
476             else:
477                 pane["fault_label"].grid_remove()
478             pane["popout_btn"].configure(text=self.tr("Ancorar" if window.popped else "Destacar"))
479             if window.popped and pane.get("toplevel") is not None:
480                 pane["toplevel"].title(
```

MBToolkit v3.5/mb_slave_ui.py — lines 481-520

```
                    f"MBToolkit — {self._slave_window_name(window)} — "
                    f"{self._slave_window_summary(window)}")
        except tk.TclError:
            pass

    # -- panes ---------------------------------------------------------------
    def _slave_build_pane(self, window, parent):
        frame = ttk.LabelFrame(parent, text="")
        frame.columnconfigure(0, weight=1)
        frame.rowconfigure(5, weight=1)
        pane = {"frame": frame}
        window_id = window.window_id

        header_buttons = self._table_build_header(
            frame, pane,
            rename=lambda: self.slave_rename_window(window_id),
            popout=lambda: self.slave_toggle_popout(window_id),
            close=lambda: self.slave_close_window(window_id),
        )
        # A fault of this Slave ID is shown under the name, never hidden.
        pane["fault_var"] = tk.StringVar()
        pane["fault_label"] = ttk.Label(pane["header"], textvariable=pane["fault_var"],
                                        style="MBS.Master.Error.TLabel")
        pane["fault_label"].grid(row=1, column=0, columnspan=3, sticky="w")

        definition = ttk.Frame(frame)
        definition.grid(row=1, column=0, sticky="w", padx=6, pady=(2, 2))
        pane["slave_var"] = tk.StringVar(value=str(window.slave))
        pane["table_var"] = tk.StringVar(value=table_choice_label(window.table))
        pane["address_var"] = tk.StringVar(value=str(window.address + window.base))
        pane["qty_var"] = tk.StringVar(value=str(window.qty))
        pane["format_var"] = tk.StringVar(value=FORMAT_LABELS[window.fmt])
        layout = (
            (self._slave_id_label, ttk.Spinbox(definition, from_=1, to=247, width=4,
                                   textvariable=pane["slave_var"])),
            ("Tabela", ttk.Combobox(definition, values=TABLE_CHOICES, width=22,
                                    state="readonly", textvariable=pane["table_var"])),
            ("Endereço", ttk.Entry(definition, width=6, textvariable=pane["address_var"])),
            ("Qtd.", ttk.Entry(definition, width=5, textvariable=pane["qty_var"])),
        )
```

MBToolkit v3.5/mb_slave_ui.py — lines 521-560

```
        for index, (caption, widget) in enumerate(layout):
            ttk.Label(definition, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
            widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))

        definition2 = ttk.Frame(frame)
        definition2.grid(row=2, column=0, sticky="w", padx=6, pady=2)
        pane["format_combo"] = ttk.Combobox(
            definition2, values=[FORMAT_LABELS[k] for k in FORMAT_LABELS], width=20,
            state="readonly", textvariable=pane["format_var"],
        )
        ttk.Label(definition2, text="Formato").grid(row=0, column=0, padx=(0, 3))
        pane["format_combo"].grid(row=0, column=1, padx=(0, 10))

        buttons = ttk.Frame(frame)
        buttons.grid(row=3, column=0, sticky="w", padx=6, pady=(2, 2))
        actions = (
            ("Aplicar", lambda: self.slave_apply_window(window_id)),
            ("Duplicar", lambda: self.slave_duplicate_window(window_id)),
            ("Falhas…", lambda: self.slave_edit_faults(window_id)),
        )
        for index, (caption, command) in enumerate(actions):
            button = ttk.Button(buttons, text=caption, command=command, width=10)
            button.grid(row=0, column=index, padx=(0, 3))
            if caption == "Aplicar":
                pane["apply_btn"] = button

        status = ttk.Frame(frame)
        status.grid(row=4, column=0, sticky="ew", padx=6, pady=(2, 2))
        status.columnconfigure(1, weight=1)
        pane["counters_var"] = tk.StringVar()
        pane["last_var"] = tk.StringVar()
        ttk.Label(status, textvariable=pane["counters_var"]).grid(row=0, column=0, sticky="w")
        ttk.Label(status, textvariable=pane["last_var"]).grid(
            row=0, column=1, sticky="w", padx=(14, 0))

        self._table_build_tree(
            frame, pane, 5,
            on_double=lambda e: self._slave_on_double_click(window_id, e),
            on_return=lambda: self._slave_edit_value(window_id),
            on_menu=lambda e: self._slave_show_row_menu(window_id, e),
```

MBToolkit v3.5/mb_slave_ui.py — lines 561-600

```
        )
        for widget in (definition, definition2):
            for child in widget.winfo_children():
                if isinstance(child, (ttk.Entry, ttk.Spinbox)):
                    child.bind("<Return>", lambda _e: self.slave_apply_window(window_id))
        self._table_translate_captions(definition, definition2, buttons, header_buttons)

        self._slave_panes[window_id] = pane
        for name in ("slave_var", "table_var", "address_var", "qty_var", "format_var"):
            pane[name].trace_add("write", lambda *_a: self._slave_update_apply_state(window_id))
        self._slave_fill_rows(window)
        self._slave_update_format_state(window)
        self._slave_render_status(window)
        self._slave_update_apply_state(window_id)
        self._table_install_wheel_tags(frame, self._slave_wheel_tag)
        return frame

    def _slave_update_format_state(self, window):
        pane = self._slave_panes.get(window.window_id)
        if pane is not None:
            pane["format_combo"].configure(state="disabled" if window.bits else "readonly")

    def _slave_fill_rows(self, window):
        pane = self._slave_panes.get(window.window_id)
        if pane is None:
            return
        tree = pane["tree"]
        tree.delete(*tree.get_children())
        tree.configure(height=max(TABLE_PANE_MIN_ROWS, min(TABLE_PANE_MAX_ROWS, window.qty)))
        for index in range(window.qty):
            pdu = window.address + index
            tree.insert("", "end", iid=f"r{index}", tags=self._table_row_tags(index),
                        values=(pdu + window.base, window.names.get(pdu, ""), ""))
        self._slave_render_values(window)

    def _slave_render_values(self, window):
        pane = self._slave_panes.get(window.window_id)
        if pane is None:
            return
        tree = pane["tree"]
```

MBToolkit v3.5/mb_slave_ui.py — lines 601-640

```
601         values = self.slave_store.read(window.slave, window.table, window.address, window.qty)
602         if window.bits:
603             texts = [str(v) for v in values]
604         else:
605             texts = format_window_values(values, window.address, window.fmt,
606                                          window.formats, window.scales)
607         now = time.monotonic()
608         generated = set()
609         for address in window.generators:
610             offset, size, _fmt = window.group_of_row(address - window.address)
611             generated.update(range(offset, offset + size))
612         for index, text in enumerate(texts[:window.qty]):
613             # The zebra tag would win over "written", so a written row has only it.
614             tags = (("written",) if window.highlight.get(index, 0) > now
615                     else self._table_row_tags(index))
616             if index in generated:
617                 tags += ("generated",)
618             try:
619                 tree.item(f"r{index}", tags=tags)
620                 tree.set(f"r{index}", "value", text)
621             except tk.TclError:
622                 pass
623 
624     def _slave_render_table(self, slave, table):
625         for window in self.slave_windows.values():
626             if (window.slave, window.table) == (slave, table):
627                 self._slave_render_values(window)
628 
629     def _slave_schedule_reflow(self, _event=None):
630         if self._slave_reflow_after_id is not None:
631             try:
632                 self.after_cancel(self._slave_reflow_after_id)
633             except tk.TclError:
634                 pass
635         self._slave_reflow_after_id = self.after(80, self._slave_reflow)
636 
637     def _slave_reflow(self):
638         self._slave_reflow_after_id = None
639         docked = [w for w in sorted(self.slave_windows.values(), key=lambda w: w.number)
640                   if not w.popped]
```

MBToolkit v3.5/mb_slave_ui.py — lines 641-680

```
641         frames = [self._slave_panes[w.window_id]["frame"] for w in docked
642                   if w.window_id in self._slave_panes]
643         self._table_reflow(self.slave_canvas, self.slave_grid, self._slave_grid_item,
644                            frames, self.slave_empty_label, "slave_panes")
645 
646     # -- window actions ------------------------------------------------------
647     def _slave_sync_areas(self):
648         """Tell the store which addresses exist (what the windows show)."""
649         self.slave_store.set_areas(w.area() for w in self.slave_windows.values())
650 
651     def slave_add_window(self, kwargs=None):
652         if len(self.slave_windows) >= SLAVE_MAX_WINDOWS:
653             messagebox.showwarning(
654                 self.tr(self._slave_tab_name),
655                 self.trf("Máximo de {count} janelas.", count=SLAVE_MAX_WINDOWS),
656             )
657             return None
658         window_id = self._slave_next_id
659         self._slave_next_id += 1
660         numbers = {w.number for w in self.slave_windows.values()}
661         number = next(n for n in range(1, SLAVE_MAX_WINDOWS + 2) if n not in numbers)
662         window = SlaveWindow(window_id, number, **(kwargs or {}))
663         self.slave_windows[window_id] = window
664         window.drop_invalid_generators()
665         self._slave_sync_areas()
666         self._slave_sync_generators()
667         self._slave_build_pane(window, self.slave_grid)
668         self._slave_reflow()
669         self._slave_mark_changed()
670         return window
671 
672     def slave_duplicate_window(self, window_id):
673         window = self.slave_windows.get(window_id)
674         if window is not None:
675             self.slave_add_window(SlaveWindow.validated_kwargs(window.to_json()))
676 
677     def slave_close_window(self, window_id):
678         window = self.slave_windows.pop(window_id, None)
679         if window is None:
680             return
```

MBToolkit v3.5/mb_slave_ui.py — lines 681-720

```
        self._slave_sync_areas()
        self._slave_sync_generators()
        pane = self._slave_panes.pop(window_id, None)
        if pane is not None:
            top = pane.get("toplevel")
            try:
                (top if top is not None else pane["frame"]).destroy()
            except tk.TclError:
                pass
        self._slave_reflow()
        if not self.slave_windows:
            self.slave_workspace.forget()
        else:
            self._slave_mark_changed()

    def _slave_fields_changed(self, window, pane):
        try:
            return (
                pane["slave_var"].get().strip() != str(window.slave)
                or pane["table_var"].get() != table_choice_label(window.table)
                or pane["address_var"].get().strip() != str(window.address + window.base)
                or pane["qty_var"].get().strip() != str(window.qty)
                or FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get()) != window.fmt
            )
        except tk.TclError:
            return False

    def _slave_update_apply_state(self, window_id):
        window = self.slave_windows.get(window_id)
        pane = self._slave_panes.get(window_id)
        if window is None or pane is None or "apply_btn" not in pane:
            return
        style = "MBS.Primary.TButton" if self._slave_fields_changed(window, pane) else "TButton"
        try:
            pane["apply_btn"].configure(style=style)
        except tk.TclError:
            pass

    def slave_apply_window(self, window_id):
        window = self.slave_windows.get(window_id)
```

MBToolkit v3.5/mb_slave_ui.py — lines 721-760

```
        pane = self._slave_panes.get(window_id)
        if window is None or pane is None:
            return False
        try:
            base = window.base
            slave = parse_int_field(pane["slave_var"].get(), 1, 247,
                                    "O Slave ID tem de estar entre 1 e 247.")
            table = TABLE_KEYS_BY_LABEL.get(pane["table_var"].get(), window.table)
            address = parse_int_field(pane["address_var"].get(), base, 0xFFFF + base,
                                      "O endereço tem de estar entre 0 e 65535.") - base
            qty = parse_int_field(pane["qty_var"].get(), 1, SLAVE_MAX_QTY,
                                  "A quantidade tem de estar entre 1 e 2000.")
            validate_area(slave, table, address, qty)
        except ValueError as exc:
            messagebox.showerror(self.tr("Janela inválida"), self.tr(str(exc)))
            return False
        fmt = FORMAT_KEYS_BY_LABEL.get(pane["format_var"].get(), DEFAULT_FORMAT)
        changed_area = (slave, table, address, qty) != window.area()
        changed = changed_area or fmt != window.fmt
        window.slave, window.table, window.address, window.qty = slave, table, address, qty
        window.fmt = fmt
        if changed_area:
            window.reset_counters()
            window.highlight = {}
            window.drop_invalid_generators()
            self._slave_sync_areas()
            self._slave_sync_generators()
        self._slave_fill_rows(window)
        self._slave_update_format_state(window)
        self._slave_render_status(window)
        self._slave_update_apply_state(window_id)
        if changed:
            self._slave_mark_changed()
        return True

    def slave_toggle_popout(self, window_id):
        window = self.slave_windows.get(window_id)
        pane = self._slave_panes.pop(window_id, None)
        if window is None or pane is None:
            return
```

MBToolkit v3.5/mb_slave_ui.py — lines 761-800

```
        old_top = pane.get("toplevel")
        try:
            pane["frame"].destroy()
            if old_top is not None:
                old_top.destroy()
        except tk.TclError:
            pass
        window.popped = not window.popped
        if window.popped:
            top = tk.Toplevel(self._table_root())
            top.geometry("660x420")
            top.minsize(TABLE_PANE_MIN_WIDTH, 300)
            top.columnconfigure(0, weight=1)
            top.rowconfigure(0, weight=1)
            top.protocol("WM_DELETE_WINDOW", lambda: self.slave_toggle_popout(window_id))
            frame = self._slave_build_pane(window, top)
            frame.grid(row=0, column=0, sticky="nsew", padx=8, pady=8)
            self._slave_panes[window_id]["toplevel"] = top
            self._table_apply_toplevel_palette(top)
        else:
            self._slave_build_pane(window, self.slave_grid)
        self._slave_render_status(window)
        self._slave_reflow()

    def slave_rename_window(self, window_id, name=None):
        window = self.slave_windows.get(window_id)
        if window is None:
            return
        if name is None:
            name = self._table_ask_text(
                self.tr("Renomear janela"),
                self.tr("Nome da janela (vazio = nome por defeito)"),
                self._slave_window_name(window),
            )
            if name is None:
                return
        name = name.strip()[:TABLE_MAX_TITLE_LENGTH]
        if name == self.trf("Janela {number}", number=window.number):
            name = ""
        window.title = name
```

MBToolkit v3.5/mb_slave_ui.py — lines 801-840

```
801         self._slave_render_status(window)
802         self._slave_mark_changed()
803 
804     # -- values and names ------------------------------------------------------
805     def _slave_on_double_click(self, window_id, event):
806         pane = self._slave_panes.get(window_id)
807         if pane is None:
808             return "break"
809         tree = pane["tree"]
810         row = tree.identify_row(event.y)
811         if not row:
812             return "break"
813         tree.selection_set(row)
814         if tree.identify_column(event.x) == "#2":
815             self._slave_edit_name(window_id, row)
816         else:
817             self._slave_edit_value(window_id)
818         return "break"
819 
820     def _slave_edit_name(self, window_id, row):
821         window = self.slave_windows.get(window_id)
822         if window is None:
823             return
824         pdu = window.address + int(row[1:])
825         name = self._table_ask_text(
826             self.tr("Nome"), self.trf("Nome do endereço {address}", address=pdu + window.base),
827             window.names.get(pdu, ""),
828         )
829         if name is None:
830             return
831         name = name.strip()
832         if name:
833             window.names[pdu] = name
834         else:
835             window.names.pop(pdu, None)
836         pane = self._slave_panes.get(window_id)
837         if pane is not None:
838             pane["tree"].set(row, "name", name)
839         self._slave_mark_changed()
840 
```

MBToolkit v3.5/mb_slave_ui.py — lines 841-880

```
841     def slave_set_value(self, window, row, text):
842         """Store the value typed for *row* (first register of its value).
843 
844         Coils / discrete inputs accept 0 or 1; registers use the format (and
845         scale) of the value. Raises ValueError (PT canonical message).
846         """
847         address = window.address + row
848         if window.bits:
849             values = [encode_coil_value(text)]
850         else:
851             offset, size, fmt = window.group_of_row(row)
852             if fmt is None:
853                 raise ValueError("Este endereço não tem um valor completo no formato atual.")
854             address = window.address + offset
855             scale = window.scales.get(address)
856             if scale is not None and fmt not in ("hex16", "bin16"):
857                 text = raw_text_for_scaled(text, scale, fmt)
858             values = encode_register_value(text, fmt)
859         self.slave_store.write(window.slave, window.table, address, values)
860         self._slave_render_table(window.slave, window.table)
861         return values
862 
863     def _slave_edit_value(self, window_id):
864         window = self.slave_windows.get(window_id)
865         pane = self._slave_panes.get(window_id)
866         if window is None or pane is None:
867             return
868         selection = pane["tree"].selection()
869         if not selection:
870             return
871         row = int(selection[0][1:])
872         if window.bits:
873             # A bit is simply toggled.
874             current = self.slave_store.read(window.slave, window.table, window.address + row, 1)
875             self.slave_set_value(window, row, "0" if current[0] else "1")
876             return
877         offset, _size, fmt = window.group_of_row(row)
878         if fmt is None:
879             messagebox.showinfo(self.tr("Valor"), self.tr(
880                 "Este endereço não tem um valor completo no formato atual."))
```

MBToolkit v3.5/mb_slave_ui.py — lines 881-920

```
881             return
882         current = pane["tree"].set(f"r{offset}", "value")
883         scale = window.scales.get(window.address + offset)
884         if scale is not None and fmt in ("hex16", "bin16"):
885             scale = None
886         if scale is not None and scale.get("unit"):
887             current = current.rsplit(" ", 1)[0]
888 
889         dialog, body = self._table_dialog(self.tr("Valor"))
890         format_text = FORMAT_LABELS[fmt]
891         if scale is not None:
892             format_text += " · " + self.trf("valores em unidades da escala {unit}",
893                                             unit=scale.get("unit") or "")
894         ttk.Label(body, text=self.trf("Valor do endereço {address} ({fmt})",
895                                       address=window.address + offset + window.base,
896                                       fmt=format_text)).grid(row=0, column=0, sticky="w")
897         value_var = tk.StringVar(value=current)
898         entry = ttk.Entry(body, textvariable=value_var, width=30)
899         entry.grid(row=1, column=0, sticky="w", pady=(4, 4))
900         message_var = tk.StringVar()
901         ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
902             row=2, column=0, sticky="w")
903 
904         def accept(_event=None):
905             try:
906                 self.slave_set_value(window, offset, value_var.get())
907             except ValueError as exc:
908                 message_var.set(self.tr(str(exc)))
909                 return
910             dialog.destroy()
911 
912         buttons = ttk.Frame(body)
913         buttons.grid(row=3, column=0, sticky="e", pady=(8, 0))
914         ttk.Button(buttons, text=self.tr("Guardar"), command=accept,
915                    style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
916         ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(row=0, column=1)
917         entry.bind("<Return>", accept)
918         entry.bind("<Escape>", lambda _e: dialog.destroy())
919         entry.focus_set()
920         entry.select_range(0, "end")
```

MBToolkit v3.5/mb_slave_ui.py — lines 921-960

```
        dialog.grab_set()

    # -- right-click menu ------------------------------------------------------------
    def _slave_show_row_menu(self, window_id, event):
        window = self.slave_windows.get(window_id)
        pane = self._slave_panes.get(window_id)
        if window is None or pane is None:
            return "break"
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if row and row not in tree.selection():
            tree.selection_set(row)
        self._table_fill_row_menu(
            self._slave_row_menu, pane, window, not window.bits,
            set_format=lambda fmt: self.slave_set_row_format(window_id, fmt),
            edit_scale=lambda: self.slave_edit_scale(window_id),
            remove_scale=lambda: self.slave_remove_scale(window_id),
            copy=lambda: self._table_copy_rows(self._slave_panes.get(window_id)),
            set_base=lambda base: self.slave_set_base(window_id, base),
            extra=(("Valor automático…", lambda: self.slave_edit_generator(window_id)),
                   ("Parar valor automático", lambda: self.slave_remove_generator(window_id))),
        )
        self._slave_apply_menu_palette()
        self._table_popup(self._slave_row_menu, event)
        return "break"

    def _slave_selected_rows(self, window_id):
        return self._table_selected_rows(self._slave_panes.get(window_id))

    def slave_set_row_format(self, window_id, fmt):
        window = self.slave_windows.get(window_id)
        if window is None:
            return False
        try:
            apply_format_to_rows(window, self._slave_selected_rows(window_id), fmt)
        except ValueError as exc:
            messagebox.showwarning(self.tr("Formato"),
                                   self.trf(str(exc), size=FORMAT_SIZES.get(fmt, 1)))
            return False
        self._slave_render_values(window)
```

MBToolkit v3.5/mb_slave_ui.py — lines 961-1000

```
961         self._slave_mark_changed()
962         return True
963 
964     def slave_edit_scale(self, window_id):
965         window = self.slave_windows.get(window_id)
966         if window is None:
967             return False
968         starts = self._table_selected_group_starts(window, self._slave_panes.get(window_id))
969         if not starts:
970             messagebox.showwarning(self.tr("Escala"), self.tr("Seleciona pelo menos um endereço."))
971             return False
972         if not self._table_ask_scale(window, starts):
973             return False
974         self._slave_render_values(window)
975         self._slave_mark_changed()
976         return True
977 
978     def slave_remove_scale(self, window_id):
979         window = self.slave_windows.get(window_id)
980         if window is None:
981             return False
982         removed = False
983         for address in self._table_selected_group_starts(window, self._slave_panes.get(window_id)):
984             removed = window.scales.pop(address, None) is not None or removed
985         if removed:
986             self._slave_render_values(window)
987             self._slave_mark_changed()
988         return removed
989 
990     def slave_set_base(self, window_id, base):
991         window = self.slave_windows.get(window_id)
992         pane = self._slave_panes.get(window_id)
993         if window is None or pane is None or window.base == base:
994             return
995         window.base = base
996         pane["address_var"].set(str(window.address + window.base))
997         self._slave_fill_rows(window)
998         self._slave_render_status(window)
999         self._slave_update_apply_state(window_id)
1000        self._slave_mark_changed()
```

MBToolkit v3.5/mb_slave_ui.py — lines 1001-1040

```
1001
1002    # -- automatic values ------------------------------------------------------------
1003    def _slave_sync_generators(self):
1004        """Rebuild the running automatic values from the windows' settings."""
1005        now = time.monotonic()
1006        running = {}
1007        for window in self.slave_windows.values():
1008            for address, config in window.generators.items():
1009                key = (window.window_id, address)
1010                current = self._slave_generators.get(key)
1011                if current is not None and current[0].config == config:
1012                    running[key] = current
1013                else:
1014                    running[key] = (ValueGenerator(config), now)
1015        self._slave_generators = running
1016        if running and self._slave_generator_after_id is None:
1017            self._slave_generator_after_id = self.after(
1018                SLAVE_GENERATOR_TICK_MS, self._slave_generate)
1019
1020    def slave_generator_registers(self, window, address, value):
1021        """Registers (or bit) for an automatic *value* at *address*; None if it does not fit."""
1022        if window.bits:
1023            return [1 if value else 0]
1024        offset, _size, fmt = window.group_of_row(address - window.address)
1025        if fmt is None or window.address + offset != address:
1026            return None
1027        scale = window.scales.get(address)
1028        if scale is not None and fmt not in ("hex16", "bin16"):
1029            try:
1030                value = unscale_value(value, scale)
1031            except ValueError:
1032                return None
1033        if fmt.split("_")[0].startswith("float"):
1034            text, fmt_used = repr(float(value)), fmt
1035        else:
1036            text = str(int(round(value)))
1037            fmt_used = "uint16" if fmt in ("hex16", "bin16") else fmt
1038        try:
1039            return encode_register_value(text, fmt_used)
1040        except ValueError:
```

MBToolkit v3.5/mb_slave_ui.py — lines 1041-1080

```
1041            return None
1042
1043    def _slave_generate(self):
1044        """Write the current automatic values into the store and show them."""
1045        self._slave_generator_after_id = None
1046        if not self._slave_generators or getattr(self, "_closing", False):
1047            return
1048        now = time.monotonic()
1049        changed = set()
1050        for (window_id, address), (generator, started) in list(self._slave_generators.items()):
1051            window = self.slave_windows.get(window_id)
1052            if window is None:
1053                continue
1054            registers = self.slave_generator_registers(window, address,
1055                                                       generator.value(now - started))
1056            if registers is None:
1057                continue
1058            current = self.slave_store.read(window.slave, window.table, address, len(registers))
1059            if current != registers:
1060                self.slave_store.write(window.slave, window.table, address, registers)
1061                changed.add((window.slave, window.table))
1062        for slave, table in changed:
1063            self._slave_render_table(slave, table)
1064        try:
1065            self._slave_generator_after_id = self.after(SLAVE_GENERATOR_TICK_MS,
1066                                                        self._slave_generate)
1067        except tk.TclError:
1068            pass
1069
1070    def slave_set_generator(self, window_id, starts, config):
1071        """Set (config) or remove (None) the automatic value of the given addresses."""
1072        window = self.slave_windows.get(window_id)
1073        if window is None:
1074            return
1075        for address in starts:
1076            if config is None:
1077                window.generators.pop(address, None)
1078            else:
1079                window.generators[address] = dict(config)
1080        self._slave_sync_generators()
```

MBToolkit v3.5/mb_slave_ui.py — lines 1081-1120

```
        self._slave_render_values(window)
        self._slave_mark_changed()

    def slave_remove_generator(self, window_id):
        window = self.slave_windows.get(window_id)
        if window is None:
            return False
        starts = [a for a in self._table_selected_group_starts(
            window, self._slave_panes.get(window_id)) if a in window.generators]
        if starts:
            self.slave_set_generator(window_id, starts, None)
        return bool(starts)

    def slave_edit_generator(self, window_id):
        """Dialog: automatic value for the selected values."""
        window = self.slave_windows.get(window_id)
        if window is None:
            return
        starts = self._table_selected_group_starts(window, self._slave_panes.get(window_id))
        if not starts:
            messagebox.showwarning(self.tr("Valor automático"),
                                   self.tr("Seleciona pelo menos um endereço."))
            return
        kinds = BIT_GENERATORS if window.bits else REGISTER_GENERATORS
        current = window.generators.get(starts[0]) or {
            "kind": kinds[0], "min": 0, "max": 100, "step": 1, "period_s": 1}
        labels = [self.tr(GENERATOR_LABELS[k]) for k in kinds]

        dialog, body = self._table_dialog(self.tr("Valor automático"))
        ttk.Label(body, text=self.trf("Valor automático de {count} valor(es) selecionado(s):",
                                      count=len(starts))).grid(
            row=0, column=0, columnspan=4, sticky="w", pady=(0, 6))

        def number_text(value):
            return f"{value:g}"

        kind_var = tk.StringVar(value=labels[kinds.index(current["kind"])]
                                if current["kind"] in kinds else labels[0])
        variables = {
            "min": tk.StringVar(value=number_text(current.get("min", 0))),
```

MBToolkit v3.5/mb_slave_ui.py — lines 1121-1160

```
            "max": tk.StringVar(value=number_text(current.get("max", 100))),
            "step": tk.StringVar(value=number_text(current.get("step", 1))),
            "period_s": tk.StringVar(value=number_text(current.get("period_s", 1))),
        }
        ttk.Label(body, text=self.tr("Tipo")).grid(row=1, column=0, sticky="w", padx=(0, 6), pady=2)
        ttk.Combobox(body, values=labels, textvariable=kind_var, width=16,
                     state="readonly").grid(row=1, column=1, columnspan=3, sticky="w", pady=2)
        entries = {}
        for row, column, caption, key in ((2, 0, "Mínimo", "min"), (2, 2, "Máximo", "max"),
                                          (3, 0, "Passo", "step"), (3, 2, "Período (s)", "period_s")):
            ttk.Label(body, text=self.tr(caption)).grid(row=row, column=column, sticky="w",
                                                        padx=(0, 6), pady=2)
            entries[key] = ttk.Entry(body, width=10, textvariable=variables[key])
            entries[key].grid(row=row, column=column + 1, sticky="w", padx=(0, 14), pady=2)
        help_var = tk.StringVar()
        ttk.Label(body, textvariable=help_var).grid(row=4, column=0, columnspan=4, sticky="w",
                                                    pady=(6, 2))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=5, column=0, columnspan=4, sticky="w")
        help_texts = {
            "counter": "Soma o passo em cada período; depois do máximo volta ao mínimo.",
            "ramp": "Sobe do mínimo ao máximo durante o período e recomeça.",
            "sine": "Oscila entre o mínimo e o máximo; o período é uma volta completa.",
            "random": "Um valor ao acaso entre o mínimo e o máximo em cada período.",
            "toggle": "Alterna entre 0 e 1 em cada período.",
        }
        if window.bits:
            help_texts["random"] = "0 ou 1 ao acaso em cada período."

        def selected_kind():
            return kinds[labels.index(kind_var.get())]

        def refresh(*_args):
            kind = selected_kind()
            for key in ("min", "max"):
                entries[key].configure(state="disabled" if window.bits else "normal")
            entries["step"].configure(state="normal" if kind == "counter" else "disabled")
            help_var.set(self.tr(help_texts[kind]))

```

MBToolkit v3.5/mb_slave_ui.py — lines 1161-1200

```
1161        kind_var.trace_add("write", refresh)
1162        refresh()
1163
1164        def accept(_event=None):
1165            data = {key: var.get() for key, var in variables.items()}
1166            data["kind"] = selected_kind()
1167            try:
1168                config = validate_generator(data, window.bits)
1169            except ValueError as exc:
1170                message_var.set(self.tr(str(exc)))
1171                return
1172            self.slave_set_generator(window_id, starts, config)
1173            dialog.destroy()
1174
1175        buttons = ttk.Frame(body)
1176        buttons.grid(row=6, column=0, columnspan=4, sticky="e", pady=(8, 0))
1177        ttk.Button(buttons, text=self.tr("Aplicar"), command=accept,
1178                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
1179        ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(row=0, column=1)
1180        dialog.bind("<Return>", accept)
1181        dialog.bind("<Escape>", lambda _e: dialog.destroy())
1182        dialog.grab_set()
1183
1184    # -- opening windows from the Sniffer / Master ------------------------------------------
1185    def slave_open_definition(self, definition, port="", baud="", parity="", stopbits=""):
1186        """Add a Slave window and show the Slave tab.
1187
1188        The serial settings (and port) are copied to the Slave connection only
1189        when it is not listening and they are valid.
1190        """
1191        if not self.slave_active:
1192            self._slave_apply_connection({"port": port, "baud": baud, "parity": parity,
1193                                          "stop_bits": stopbits})
1194        window = self.slave_add_window(definition)
1195        try:
1196            self.main_notebook.select(getattr(self, self._slave_page_attr))
1197        except tk.TclError:
1198            pass
1199        return window
1200
```

MBToolkit v3.5/mb_slave_ui.py — lines 1201-1240

```
1201    def slave_open_from_record(self, record):
1202        definition = slave_definition_from_record(record)
1203        if definition is None:
1204            messagebox.showinfo(
1205                self.tr("Abrir no Slave"),
1206                self.tr("Esta trama não tem um pedido FC01–FC06/FC15/FC16 com endereço "
1207                        "conhecido."),
1208            )
1209            return None
1210        port = "" if getattr(self, "capture_active", False) else self.port_a_var.get().strip()
1211        return self.slave_open_definition(
1212            definition, port=port, baud=self.baud_var.get(),
1213            parity=self.parity_var.get(), stopbits=self.stop_var.get(),
1214        )
1215
1216    def slave_open_from_master(self, master_window):
1217        """Simulate the data area of a Master window (same names, formats and scales)."""
1218        definition = {
1219            # A TCP Client may use Unit IDs 0 and 248–255; simulate them as 1.
1220            "slave": master_window.slave if 1 <= master_window.slave <= 247 else 1,
1221            "table": FC_TABLES[master_window.fc],
1222            "address": master_window.address, "qty": master_window.qty,
1223            "base": master_window.base, "fmt": master_window.fmt,
1224            "title": master_window.title, "names": dict(master_window.names),
1225            "formats": dict(master_window.formats),
1226            "scales": {a: dict(s) for a, s in master_window.scales.items()},
1227        }
1228        serial = getattr(self, "master_baud_var", None) is not None
1229        return self.slave_open_definition(
1230            definition, baud=self.master_baud_var.get() if serial else "",
1231            parity=self.master_parity_var.get() if serial else "",
1232            stopbits=self.master_stop_var.get() if serial else "",
1233        )
1234
1235    # -- fault simulation ------------------------------------------------------
1236    def slave_set_fault(self, slave, fault):
1237        """Set (or clear, with None / an inactive fault) the fault of a Slave ID."""
1238        if fault_active(fault):
1239            self.slave_faults[slave] = dict(fault)
1240        else:
```

MBToolkit v3.5/mb_slave_ui.py — lines 1241-1280

```
            self.slave_faults.pop(slave, None)
        self.slave_store.set_fault(slave, self.slave_faults.get(slave))
        for window in self.slave_windows.values():
            if window.slave == slave:
                self._slave_render_status(window)

    def slave_edit_faults(self, window_id):
        window = self.slave_windows.get(window_id)
        if window is None:
            return
        slave = window.slave
        current = self.slave_faults.get(slave) or default_fault()
        dialog, body = self._table_dialog(self.trf(self._slave_fault_title, slave=slave))
        ttk.Label(body, text=self.trf(
            self._slave_fault_intro,
            slave=slave)).grid(row=0, column=0, columnspan=2, sticky="w", pady=(0, 8))

        delay_var = tk.StringVar(value=str(current["delay_ms"]))
        silent_var = tk.BooleanVar(value=current["no_response"])
        exception_labels = [self.tr("Nenhuma")] + [
            f"{code:02X} {MODBUS_EXCEPTION_CODES.get(code, ('', ''))[0]}"
            for code in FAULT_EXCEPTION_CODES]
        exception_var = tk.StringVar(value=exception_labels[
            (FAULT_EXCEPTION_CODES.index(current["exception"]) + 1)
            if current["exception"] in FAULT_EXCEPTION_CODES else 0])
        crc_var = tk.BooleanVar(value=current["bad_crc"])
        probability_var = tk.StringVar(value=str(current["probability"]))

        ttk.Label(body, text=self.tr("Atraso na resposta (ms)")).grid(
            row=1, column=0, sticky="w", padx=(0, 8), pady=2)
        ttk.Spinbox(body, from_=0, to=SLAVE_FAULT_MAX_DELAY_MS, increment=50, width=8,
                    textvariable=delay_var).grid(row=1, column=1, sticky="w", pady=2)
        ttk.Checkbutton(body, text=self.tr("Não responder"), variable=silent_var).grid(
            row=2, column=0, columnspan=2, sticky="w", pady=2)
        ttk.Label(body, text=self.tr("Responder com Exception")).grid(
            row=3, column=0, sticky="w", padx=(0, 8), pady=2)
        ttk.Combobox(body, values=exception_labels, textvariable=exception_var, width=28,
                     state="readonly").grid(row=3, column=1, sticky="w", pady=2)
        ttk.Checkbutton(body, text=self.tr(self._slave_bad_reply_option), variable=crc_var).grid(
            row=4, column=0, columnspan=2, sticky="w", pady=2)
```

MBToolkit v3.5/mb_slave_ui.py — lines 1281-1320

```
        ttk.Label(body, text=self.tr("Probabilidade (%)")).grid(
            row=5, column=0, sticky="w", padx=(0, 8), pady=2)
        ttk.Spinbox(body, from_=1, to=100, increment=5, width=8,
                    textvariable=probability_var).grid(row=5, column=1, sticky="w", pady=2)
        ttk.Label(body, text=self.tr(
            "A probabilidade decide, em cada pedido, se a falha acontece.")).grid(
            row=6, column=0, columnspan=2, sticky="w", pady=(6, 2))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=7, column=0, columnspan=2, sticky="w")

        def accept(_event=None):
            index = exception_labels.index(exception_var.get())
            try:
                fault = validate_fault({
                    "delay_ms": delay_var.get(), "no_response": silent_var.get(),
                    "exception": FAULT_EXCEPTION_CODES[index - 1] if index else 0,
                    "bad_crc": crc_var.get(), "probability": probability_var.get(),
                })
            except ValueError as exc:
                message_var.set(self.tr(str(exc)))
                return
            self.slave_set_fault(slave, fault)
            self._slave_mark_changed()
            dialog.destroy()

        def clear():
            self.slave_set_fault(slave, None)
            self._slave_mark_changed()
            dialog.destroy()

        buttons = ttk.Frame(body)
        buttons.grid(row=8, column=0, columnspan=2, sticky="e", pady=(8, 0))
        ttk.Button(buttons, text=self.tr("Aplicar"), command=accept,
                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
        ttk.Button(buttons, text=self.tr("Sem falhas"), command=clear).grid(
            row=0, column=1, padx=(0, 6))
        ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(
            row=0, column=2)
        dialog.bind("<Return>", accept)
```

MBToolkit v3.5/mb_slave_ui.py — lines 1321-1360

```
1321        dialog.bind("<Escape>", lambda _e: dialog.destroy())
1322        dialog.grab_set()
1323
1324    # -- connection ------------------------------------------------------------
1325    def slave_connect(self):
1326        if self.slave_active:
1327            return
1328        if self.busy:
1329            messagebox.showwarning(
1330                self.tr(self._slave_tab_name),
1331                self.tr("Para a captura, a pesquisa ou o Master em curso antes de ligar o Slave."),
1332            )
1333            return
1334        if not self.slave_safety_var.get():
1335            messagebox.showwarning(
1336                self.tr(self._slave_tab_name),
1337                self.tr("Confirma primeiro que nenhum equipamento no barramento usa os "
1338                        "Slave IDs simulados."),
1339            )
1340            return
1341        serial, _ = self.get_serial_modules()
1342        if serial is None:
1343            messagebox.showerror(self.tr("pyserial em falta"), self.tr("pyserial não está instalado."))
1344            return
1345        port = self.slave_port_var.get().strip()
1346        if not port:
1347            messagebox.showerror(self.tr("Configuração inválida"),
1348                                 self.tr("Seleciona uma porta COM."))
1349            return
1350        for window_id in list(self.slave_windows):
1351            if not self.slave_apply_window(window_id):
1352                return
1353        if not self.slave_windows:
1354            messagebox.showwarning(self.tr(self._slave_tab_name),
1355                                   self.tr("Cria primeiro uma janela com o Slave a simular."))
1356            return
1357        baud, parity, stopbits = (int(self.slave_baud_var.get()), self.slave_parity_var.get(),
1358                                  self.slave_stop_var.get())
1359        try:
1360            ser = self.open_discovery_serial_port(serial, port, baud, parity, stopbits)
```

MBToolkit v3.5/mb_slave_ui.py — lines 1361-1400

```
        except Exception as exc:
            messagebox.showerror(
                self.tr(self._slave_tab_name),
                self.trf("Não foi possível abrir {port}: {error}",
                         port=port, error=f"{type(exc).__name__}: {exc}"),
            )
            return

        self.slave_serial = ser
        self.slave_engine = SlaveEngine(
            ser, self.event_queue, self.slave_store, baud=baud, parity=parity,
            stopbits=stopbits, serial_module=serial,
            reopen=lambda: self.open_discovery_serial_port(serial, port, baud, parity, stopbits),
        )
        self._slave_reset_stats()
        self._slave_render_stats()
        for window in self.slave_windows.values():
            window.reset_counters()
            self._slave_render_status(window)
        self.slave_active = True
        self.busy = True
        self._slave_lock_other_features(True)
        self.slave_status_var.set(self.tr("A abrir a porta…"))
        self.slave_engine.start()

    def slave_disconnect(self):
        if self.slave_engine is None:
            return
        self.slave_status_var.set(self.tr("A desligar…"))
        self.slave_disconnect_btn.configure(state="disabled")
        self.slave_engine.stop()

    def _slave_lock_other_features(self, locked):
        """While the Slave listens, nothing else may use a serial port."""
        state = "disabled" if locked else "normal"
        self.slave_connect_btn.configure(state=state)
        self.slave_disconnect_btn.configure(state="normal" if locked else "disabled")
        for widget in self._slave_setting_widgets:
            try:
                widget.configure(state="disabled" if locked else "readonly")
```

MBToolkit v3.5/mb_slave_ui.py — lines 1401-1440

```
            except tk.TclError:
                pass
        self.slave_refresh_btn.configure(state=state)
        for name in ("start_btn", "restart_btn", "master_connect_btn"):
            widget = getattr(self, name, None)
            if widget is not None:
                widget.configure(state=state)
        self.set_sniffer_refresh_state(state)
        self.set_simulation_button_state(state)
        self.update_discovery_start_state()

    def _slave_finish_session(self, payload):
        if not self.slave_active:
            return  # already finished (a late slave_stopped event)
        engine = self.slave_engine
        self.slave_engine = None
        if engine is not None:
            engine.stop(join_timeout=0.5)
            self.slave_serial = engine.ser
        if self.slave_serial is not None:
            try:
                self.slave_serial.close()
            except Exception:
                pass
        self.slave_serial = None
        self.slave_active = False
        self.busy = False
        self._slave_lock_other_features(False)
        self.slave_status_var.set(self.tr("Desligado"))
        error = (payload or {}).get("error")
        if error:
            messagebox.showerror(self.tr(self._slave_tab_name),
                                 self.trf("Erro na porta série: {error}", error=error))

    # -- worker events ---------------------------------------------------------
    def handle_slave_event(self, event, payload):
        if event == "slave_ready":
            self.slave_status_var.set(self.tr("● A responder"))
        elif event == "slave_reconnecting":
            self.slave_status_var.set(self.trf(
```

MBToolkit v3.5/mb_slave_ui.py — lines 1441-1480

```
                "A religar a {port}… (porta falhou: {error})",
                port=self.slave_port_var.get(), error=(payload or {}).get("error", ""),
            ))
        elif event == "slave_reconnected":
            engine = self.slave_engine
            if engine is not None:
                self.slave_serial = engine.ser
            self.slave_status_var.set(self.tr("● A responder"))
        elif event == "slave_stopped":
            self._slave_finish_session(payload)
        elif event == "slave_request":
            self._slave_apply_request(payload)

    def slave_request_text(self, info):
        """Short description of a request: "FC03 @ 0 × 10 → Exception 02"."""
        text = f"FC{info['fc']:02d}"
        if info.get("address") is not None:
            text += f" @ {info['address']}"
            if info.get("qty"):
                text += f" × {info['qty']}"
        status = info.get("status")
        if status == "exception":
            code = info.get("exception_code") or 0
            text += f" → Exception {code:02X}"
        elif status == "no_response":
            text += " → " + self.tr("sem resposta")
        elif status == "broadcast":
            text += " → broadcast"
        if info.get("fault"):
            text += " ⚠"
        return text

    def _slave_apply_request(self, info):
        stats = self.slave_stats
        status = info.get("status")
        if status == "ignored":
            stats["others"] += 1
            self._slave_render_stats()
            return
        stats["requests"] += 1
```

MBToolkit v3.5/mb_slave_ui.py — lines 1481-1520

```
        if status == "broadcast":
            stats["broadcasts"] += 1
        elif status == "exception":
            stats["exceptions"] += 1
            stats["responses"] += 1
        elif status == "ok":
            stats["responses"] += 1
        self._slave_render_stats()

        stamp = time.strftime("%H:%M:%S", time.localtime(info.get("time", time.time())))
        text = f"{self.slave_request_text(info)}  {stamp}"
        address, qty = info.get("address"), info.get("qty") or 1
        targets = {(slave, table) for slave, table, _a, _q in info.get("written", [])}
        if status != "broadcast":
            targets.add((info.get("slave"), info.get("table")))
        until = time.monotonic() + SLAVE_HIGHLIGHT_SECONDS
        touched = False
        for window in self.slave_windows.values():
            if (window.slave, window.table) not in targets or address is None:
                continue
            rows = window.overlap(address, qty)
            if rows is None:
                continue
            window.requests += 1
            if status == "exception":
                window.exceptions += 1
            window.last = text
            for slave, table, w_address, w_qty in info.get("written", []):
                if (slave, table) == (window.slave, window.table):
                    written = window.overlap(w_address, w_qty)
                    if written is not None:
                        for row in range(*written):
                            window.highlight[row] = until
                        touched = True
            self._slave_render_status(window)
            if info.get("written"):
                self._slave_render_values(window)
        if touched:
            self._slave_schedule_tick()

```

MBToolkit v3.5/mb_slave_ui.py — lines 1521-1560

```
    def _slave_schedule_tick(self):
        if self._slave_tick_after_id is None:
            self._slave_tick_after_id = self.after(250, self._slave_tick)

    def _slave_tick(self):
        """Remove expired write highlights."""
        self._slave_tick_after_id = None
        now = time.monotonic()
        pending = False
        for window in self.slave_windows.values():
            expired = [row for row, until in window.highlight.items() if until <= now]
            for row in expired:
                del window.highlight[row]
            if expired:
                self._slave_render_values(window)
            pending = pending or bool(window.highlight)
        if pending:
            self._slave_schedule_tick()

    # -- workspace ------------------------------------------------------------
    def slave_workspace_data(self):
        """Configuration only: the register values are not saved."""
        used = {w.slave for w in self.slave_windows.values()}
        return {
            "format": self._slave_ws_format,
            "version": SLAVE_WORKSPACE_VERSION,
            "connection": self._slave_connection_json(),
            "windows": [w.to_json() for w in sorted(self.slave_windows.values(),
                                                    key=lambda w: w.number)],
            "faults": {str(slave): fault for slave, fault in sorted(self.slave_faults.items())
                       if slave in used},
        }

    def _slave_connection_json(self):
        return {
            "port": self.slave_port_var.get(),
            "baud": self.slave_baud_var.get(),
            "parity": self.slave_parity_var.get(),
            "stop_bits": self.slave_stop_var.get(),
        }
```

MBToolkit v3.5/mb_slave_ui.py — lines 1561-1600

```
1561
1562    def _slave_apply_connection(self, connection):
1563        """Serial settings from a workspace or another tab (valid ones only)."""
1564        if str(connection.get("baud")) in [str(b) for b in DISCOVERY_BAUD_RATES]:
1565            self.slave_baud_var.set(str(connection["baud"]))
1566        if connection.get("parity") in SERIAL_PARITIES:
1567            self.slave_parity_var.set(connection["parity"])
1568        if str(connection.get("stop_bits")) in SERIAL_STOP_BITS:
1569            self.slave_stop_var.set(str(connection["stop_bits"]))
1570        if connection.get("port") and connection["port"] in tuple(self.slave_port_combo["values"]):
1571            self.slave_port_var.set(connection["port"])
1572
1573    def _slave_mark_changed(self):
1574        self.slave_workspace.mark_changed()
1575
1576    def _slave_build_workspace_menu(self):
1577        self.slave_workspace.fill_menu(self.slave_workspace_menu)
1578        self._slave_apply_menu_palette()
1579
1580    def slave_save_workspace(self, name=None):
1581        return self.slave_workspace.save(name)
1582
1583    def slave_delete_workspace(self, name, confirm=True):
1584        return self.slave_workspace.delete(name, confirm)
1585
1586    def slave_open_workspace(self, name, quiet=False):
1587        """Replace the windows and faults with a saved workspace; values restart at 0."""
1588        return self.slave_workspace.open(name, quiet)
1589
1590    def _slave_load_workspace_data(self, data):
1591        """Replace windows and faults with those of *data*; returns the invalid count."""
1592        entries, skipped = [], 0
1593        for item in data.get("windows", [])[:SLAVE_MAX_WINDOWS]:
1594            try:
1595                entries.append(SlaveWindow.validated_kwargs(item))
1596            except (ValueError, TypeError):
1597                skipped += 1
1598        for window_id in list(self.slave_windows):
1599            self.slave_close_window(window_id)
1600        self.slave_store.clear_values()
```

MBToolkit v3.5/mb_slave_ui.py — lines 1601-1640

```
        self.slave_store.clear_faults()
        self.slave_faults = {}
        connection = data.get("connection", {})
        if isinstance(connection, dict) and not self.slave_active:
            self._slave_apply_connection(connection)
        faults = data.get("faults", {})
        for key, item in (faults.items() if isinstance(faults, dict) else ()):
            try:
                if str(key).isdigit() and 1 <= int(key) <= 247:
                    self.slave_set_fault(int(key), validate_fault(item))
            except ValueError:
                continue
        for kwargs in entries:
            self.slave_add_window(kwargs)
        return skipped

    def slave_confirm_close(self):
        """Ask to save a changed Slave workspace; False cancels closing."""
        workspace = getattr(self, "slave_workspace", None)
        return workspace is None or workspace.confirm_close()

    # -- language / palette / shutdown ------------------------------------------------
    def slave_apply_language(self):
        if not hasattr(self, "slave_windows"):
            return
        self.slave_workspace.update_label()
        self._slave_render_stats()
        for window in self.slave_windows.values():
            pane = self._slave_panes.get(window.window_id)
            if pane is not None:
                self._table_translate_headings(pane)
                self._slave_render_status(window)
        if not self.slave_active:
            self.slave_status_var.set(self.tr("Desligado"))
        elif self.slave_engine is not None:
            self.slave_status_var.set(self.tr("● A responder"))

    def _slave_apply_menu_palette(self):
        self._table_apply_menus_palette(getattr(self, "slave_workspace_menu", None),
                                        getattr(self, "_slave_row_menu", None))
```

MBToolkit v3.5/mb_slave_ui.py — lines 1641-1668

```
1641
1642    def slave_apply_palette(self, dark):
1643        self._slave_apply_menu_palette()
1644        canvas = getattr(self, "slave_canvas", None)
1645        if canvas is not None:
1646            canvas.configure(background=theme_colors(self, dark)["root"])
1647        for pane in getattr(self, "_slave_panes", {}).values():
1648            if pane.get("toplevel") is not None:
1649                self._table_apply_toplevel_palette(pane["toplevel"])
1650            self._table_configure_tree_tags(pane["tree"])
1651        button = getattr(self, "slave_refresh_btn", None)
1652        if button is not None:
1653            button.set_dark_mode(dark)
1654
1655    def slave_shutdown(self):
1656        self._slave_generators = {}
1657        engine = getattr(self, "slave_engine", None)
1658        if engine is not None:
1659            engine.stop(join_timeout=0.8)
1660            self.slave_serial = engine.ser
1661        ser = getattr(self, "slave_serial", None)
1662        if ser is not None:
1663            try:
1664                ser.close()
1665            except Exception:
1666                pass
1667        self.slave_engine = None
1668        self.slave_serial = None
```

MBToolkit v3.5/mb_table_ui.py

792 source lines.

MBToolkit v3.5/mb_table_ui.py — lines 1-40

```python
#!/usr/bin/env python3
"""Shared parts of the Master and Slave window grids.

Both tabs show a grid of windows; each window has a header (name, summary,
Rename / Pop out / close), a value table with per-address formats, scales,
names, copy, address base, drag selection and zebra rows, and the tab has a
Workspace menu. This module holds what is common to both: module helpers
for workspace files and formats, ``TableWorkspace`` (the workspace files and
state of one tab) and ``TableUiMixin`` with the widget code.
A "pane" is the dict of widgets of one window; a "window" is its model,
with ``address``, ``qty``, ``base``, ``fmt``, ``formats``, ``scales`` and
``group_of_row()``.
"""

import json
import os
import subprocess
import sys
import tkinter as tk
from tkinter import messagebox, ttk

from mb_master import FORMAT_SIZES, REGISTER_FORMATS, validate_scale
from mb_theme import theme_colors

TABLE_PANE_MIN_WIDTH = 600
TABLE_PANE_MIN_ROWS = 3
TABLE_PANE_MAX_ROWS = 15
# Background of every second table row: one step from the table background.
TABLE_STRIPE_LIGHT = "#f2f4f6"
TABLE_STRIPE_DARK = "#262d34"
# Rows just written by a remote master (Slave tab).
TABLE_WRITTEN_LIGHT = "#fff1b8"
TABLE_WRITTEN_DARK = "#5a4b12"
# Text of values that change automatically (Slave tab).
TABLE_GENERATED_LIGHT = "#1d4ed8"
TABLE_GENERATED_DARK = "#7cc4ff"
TABLE_MAX_TITLE_LENGTH = 40
SERIAL_PARITIES = ("None", "Even", "Odd")
SERIAL_STOP_BITS = ("1", "2")
WORKSPACE_SUFFIX = ".mbsw"
```

MBToolkit v3.5/mb_table_ui.py — lines 41-80

```
41 # Characters Windows does not allow in file names.
42 INVALID_FILENAME_CHARS = '<>:"/\\|?*'
43 RESERVED_FILENAMES = ({"CON", "PRN", "AUX", "NUL"} | {f"COM{i}" for i in range(1, 10)}
44                       | {f"LPT{i}" for i in range(1, 10)})
45 WHEEL_GUARD_TAG = "MBToolkit.MouseWheelGuard"
46
47
48 def parse_int_field(text, minimum, maximum, message):
49     try:
50         value = int(str(text).strip())
51     except ValueError:
52         raise ValueError(message) from None
53     if not minimum <= value <= maximum:
54         raise ValueError(message)
55     return value
56
57
58 def validate_workspace_name(name):
59     """Return a cleaned workspace name usable as a Windows file name."""
60     name = str(name or "").strip().rstrip(".")
61     if not name:
62         raise ValueError("O nome do workspace não pode ficar vazio.")
63     if len(name) > 60 or any(ch in INVALID_FILENAME_CHARS for ch in name) \
64             or any(ord(ch) < 32 for ch in name):
65         raise ValueError("O nome do workspace não pode ter os caracteres < > : \" / \\ | ? *.")
66     if name.split(".")[0].upper() in RESERVED_FILENAMES:
67         raise ValueError("Esse nome está reservado pelo Windows.")
68     return name
69
70
71 def workspace_path(name, folder):
72     return folder / f"{validate_workspace_name(name)}{WORKSPACE_SUFFIX}"
73
74
75 def list_workspaces(folder):
76     """Names of the saved workspaces in *folder*, sorted alphabetically."""
77     try:
78         files = folder.glob(f"*{WORKSPACE_SUFFIX}")
79         return sorted((f.stem for f in files if f.is_file()), key=str.casefold)
80     except OSError:
```

MBToolkit v3.5/mb_table_ui.py — lines 81-120

```
        return []


def apply_format_to_rows(window, rows, fmt):
    """Set the per-address format of the selected rows of *window*.

    16-bit formats apply to every selected row. Multi-register formats need
    consecutive rows in a multiple of the format size and are applied in
    groups (2 registers for 32-bit, 4 for 64-bit). *fmt* None restores the
    window format. Existing per-address formats overlapping the selection are
    removed. Raises ValueError (PT canonical message) for an invalid selection.
    """
    rows = sorted(set(int(r) for r in rows))
    if not rows:
        raise ValueError("Seleciona pelo menos um endereço.")
    size = FORMAT_SIZES[fmt] if fmt else 1
    if size > 1:
        if rows != list(range(rows[0], rows[0] + len(rows))):
            raise ValueError("Seleciona registos seguidos para este formato.")
        if len(rows) % size:
            raise ValueError(
                "Este formato ocupa {size} registos: seleciona {size} registos seguidos "
                "(ou um múltiplo de {size})."
            )
        if rows[-1] >= window.qty:
            raise ValueError("Seleciona registos seguidos para este formato.")

    covered = {window.address + r for r in rows}
    for start, existing in list(window.formats.items()):
        span = set(range(start, start + FORMAT_SIZES[existing]))
        if span & covered:
            del window.formats[start]
    if fmt is not None:
        for index in range(0, len(rows), size):
            window.formats[window.address + rows[index]] = fmt


def validated_names(names):
    return {int(k): str(v) for k, v in (names.items() if isinstance(names, dict) else ())
            if str(k).isdigit() and str(v).strip()}
```

MBToolkit v3.5/mb_table_ui.py — lines 121-160

```
121
122
123 def validated_formats(formats):
124     return {int(k): v for k, v in (formats.items() if isinstance(formats, dict) else ())
125             if str(k).isdigit() and int(k) <= 0xFFFF and v in FORMAT_SIZES}
126
127
128 def validated_scales(scales):
129     """Scales of a workspace window entry; invalid ones are dropped."""
130     result = {}
131     for key, item in (scales.items() if isinstance(scales, dict) else ()):
132         try:
133             if not str(key).isdigit() or int(key) > 0xFFFF or not isinstance(item, dict):
134                 raise ValueError
135             raw, eng = item.get("raw", []), item.get("eng", [])
136             result[int(key)] = validate_scale(raw[0], raw[1], eng[0], eng[1],
137                                               item.get("decimals", 1), item.get("unit", ""))
138         except (ValueError, TypeError, IndexError):
139             continue
140     return result
141
142
143 class TableWorkspace:
144     """Named workspace files of one tab (Master or Slave) and their state.
145
146     Holds the name of the open workspace, whether it changed since it was
147     opened or saved (``dirty``) and a ``loading`` guard that suppresses change
148     tracking while a workspace is being loaded. The tab supplies its folder,
149     the accepted ``format`` names (the first is written), a ``data()`` callable
150     that returns the JSON to save and a ``load(data)`` callable that replaces
151     the tab's windows and returns the number of invalid windows skipped.
152     """
153
154     def __init__(self, app, *, folder, formats, data, load, label_var, close_question):
155         self.app = app
156         self.folder = folder
157         self.formats = tuple(formats)
158         self.data = data
159         self.load = load
160         self.label_var = label_var
```

MBToolkit v3.5/mb_table_ui.py — lines 161-200

```
161         # PT template with {name}, asked when closing the program with changes.
162         self.close_question = close_question
163         self.current = ""
164         self.dirty = False
165         self.loading = False
166 
167     def names(self):
168         return list_workspaces(self.folder())
169 
170     def path(self, name):
171         return workspace_path(name, self.folder())
172 
173     def update_label(self):
174         app = self.app
175         if not self.current:
176             text = ""
177         elif self.dirty:
178             text = app.trf("Workspace atual: {name} (alterado)", name=self.current)
179         else:
180             text = app.trf("Workspace atual: {name}", name=self.current)
181         self.label_var.set(text)
182 
183     def _set(self, name, save_settings=True):
184         self.current = name
185         self.dirty = False
186         self.update_label()
187         if save_settings:
188             self.app._schedule_ui_settings_save()
189 
190     def mark_changed(self):
191         """Record a change to the open workspace (ignored while loading one)."""
192         if self.loading:
193             return
194         if self.current and not self.dirty:
195             self.dirty = True
196             self.update_label()
197 
198     def forget(self):
199         """No workspace open any more (e.g. every window was closed)."""
200         if not self.loading:
```

MBToolkit v3.5/mb_table_ui.py — lines 201-240

```
            self._set("")

    def fill_menu(self, menu):
        self.app._table_fill_workspace_menu(
            menu, self.names(), save=self.save, open_=self.open, delete=self.delete,
            folder=lambda: self.app._table_open_folder(self.folder()),
        )

    def save(self, name=None):
        """Save the tab under *name* (asked when None); True when saved."""
        app = self.app
        if name is None:
            name = app._table_ask_text(app.tr("Guardar workspace"), app.tr("Nome do workspace"),
                                       self.current or app.tr("Workspace 1"))
            if name is None:
                return False
        try:
            name = validate_workspace_name(name)
        except ValueError as exc:
            messagebox.showerror(app.tr("Guardar workspace"), app.tr(str(exc)))
            return False
        path = self.path(name)
        if path.exists() and name != self.current:
            if not messagebox.askyesno(
                app.tr("Guardar workspace"),
                app.trf("Já existe um workspace chamado \"{name}\". Substituir?", name=name),
            ):
                return False
        try:
            with open(path, "w", encoding="utf-8") as handle:
                json.dump(self.data(), handle, indent=2, ensure_ascii=False)
        except OSError as exc:
            messagebox.showerror(app.tr("Guardar workspace"), str(exc))
            return False
        self._set(name)
        return True

    def delete(self, name, confirm=True):
        app = self.app
        if confirm and not messagebox.askyesno(
```

MBToolkit v3.5/mb_table_ui.py — lines 241-280

```
            app.tr("Apagar workspace"), app.trf("Apagar o workspace \"{name}\"?", name=name),
        ):
            return False
        try:
            self.path(name).unlink()
        except OSError as exc:
            messagebox.showerror(app.tr("Apagar workspace"), str(exc))
            return False
        if name == self.current:
            self._set("")
        return True

    def open(self, name, quiet=False):
        """Replace the tab's windows with a saved workspace (by name).

        *quiet* (start-up restore) suppresses the error dialog when the file
        cannot be read.
        """
        app = self.app
        try:
            with open(self.path(name), encoding="utf-8") as handle:
                data = json.load(handle)
            if not isinstance(data, dict) or data.get("format") not in self.formats:
                raise ValueError(app.tr("O ficheiro não é um workspace do MBToolkit."))
        except (OSError, ValueError) as exc:
            if not quiet:
                messagebox.showerror(app.tr("Abrir workspace"), str(exc))
            return False
        self.loading = True
        try:
            skipped = self.load(data)
        finally:
            self.loading = False
        self._set(name)
        if skipped:
            messagebox.showwarning(
                app.tr("Abrir workspace"),
                app.trf("{count} janela(s) inválida(s) foram ignoradas.", count=skipped),
            )
        return True
```

MBToolkit v3.5/mb_table_ui.py — lines 281-320

```
281
282     def confirm_close(self):
283         """Ask to save a changed workspace; False cancels closing the program.
284
285         Windows never saved as a workspace close silently.
286         """
287         if not (self.current and self.dirty):
288             return True
289         app = self.app
290         answer = messagebox.askyesnocancel(
291             app.tr("Workspace"), app.trf(self.close_question, name=self.current))
292         if answer is None:
293             return False
294         return self.save(self.current) if answer else True
295
296
297 class TabInstance:
298     """Second copy of a tab mixin with its own state, sharing the application.
299
300     The Modbus TCP Client and Server tabs are the Master and Slave mixins run
301     on a ``TabInstance``: attributes set by the tab (its windows, engine,
302     widgets, ``busy`` flag...) live on the instance, while everything else
303     (``tr``, ``after``, ``event_queue``, ``ui_settings``, the theme...) is
304     looked up on the application. Tk widgets are created with
305     ``_table_root()`` as parent, never with the instance itself.
306     """
307
308     def __init__(self, app):
309         self.app = app
310
311     def __getattr__(self, name):
312         if name == "app":
313             raise AttributeError(name)
314         return getattr(self.app, name)
315
316
317 class TableUiMixin:
318     """Widget code shared by the Master and Slave window grids."""
319
320     def _table_root(self):
```

MBToolkit v3.5/mb_table_ui.py — lines 321-360

```
        """Tk parent for new top-level windows and menus of this tab."""
        return getattr(self, "app", self)

    # -- dialogs -------------------------------------------------------------
    def _table_dialog(self, title):
        dialog = tk.Toplevel(self._table_root())
        dialog.title(title)
        dialog.transient(self._table_root())
        dialog.resizable(False, False)
        self._table_apply_toplevel_palette(dialog)
        body = ttk.Frame(dialog, padding=12)
        body.grid(row=0, column=0, sticky="nsew")
        return dialog, body

    def _table_ask_text(self, title, caption, initial):
        """Small modal text prompt; returns the text or None when cancelled."""
        dialog, body = self._table_dialog(title)
        ttk.Label(body, text=caption).grid(row=0, column=0, columnspan=2, sticky="w")
        value = tk.StringVar(value=initial)
        entry = ttk.Entry(body, textvariable=value, width=36)
        entry.grid(row=1, column=0, columnspan=2, pady=(4, 10), sticky="ew")
        result = {"text": None}

        def accept(_event=None):
            result["text"] = value.get()
            dialog.destroy()

        ttk.Button(body, text=self.tr("OK"), command=accept,
                   style="MBS.Primary.TButton").grid(row=2, column=0, sticky="e", padx=(0, 6))
        ttk.Button(body, text=self.tr("Cancelar"), command=dialog.destroy).grid(
            row=2, column=1, sticky="e")
        entry.bind("<Return>", accept)
        entry.bind("<Escape>", lambda _e: dialog.destroy())
        entry.focus_set()
        entry.select_range(0, "end")
        dialog.grab_set()
        self.wait_window(dialog)
        return result["text"]

    def _table_ask_scale(self, window, starts):
```

MBToolkit v3.5/mb_table_ui.py — lines 361-400

```
        """Ask for a linear scale and set it on the values starting at *starts*."""
        current = window.scales.get(starts[0]) or {
            "raw": [0, 65535], "eng": [0, 100], "decimals": 1, "unit": ""}

        dialog, body = self._table_dialog(self.tr("Escala"))
        ttk.Label(body, text=self.trf("Escala de {count} valor(es) selecionado(s):",
                                      count=len(starts))).grid(
            row=0, column=0, columnspan=4, sticky="w", pady=(0, 6))

        def number_text(value):
            return f"{value:g}"

        variables = {
            "raw_min": tk.StringVar(value=number_text(current["raw"][0])),
            "raw_max": tk.StringVar(value=number_text(current["raw"][1])),
            "eng_min": tk.StringVar(value=number_text(current["eng"][0])),
            "eng_max": tk.StringVar(value=number_text(current["eng"][1])),
            "decimals": tk.StringVar(value=str(current["decimals"])),
            "unit": tk.StringVar(value=current.get("unit", "")),
        }
        layout = (
            (1, 0, "Bruto mín.", "raw_min"), (1, 2, "Bruto máx.", "raw_max"),
            (2, 0, "Eng. mín.", "eng_min"), (2, 2, "Eng. máx.", "eng_max"),
            (3, 0, "Casas decimais", "decimals"), (3, 2, "Unidade", "unit"),
        )
        first_entry = None
        for row, column, caption, key in layout:
            ttk.Label(body, text=self.tr(caption)).grid(row=row, column=column, sticky="w",
                                                        padx=(0, 6), pady=2)
            entry = ttk.Entry(body, width=10, textvariable=variables[key])
            entry.grid(row=row, column=column + 1, sticky="w", padx=(0, 14), pady=2)
            first_entry = first_entry or entry
        ttk.Label(body, text=self.tr("Valor = Eng. mín. + (bruto − Bruto mín.) × "
                                     "(Eng. máx. − Eng. mín.) / (Bruto máx. − Bruto mín.)"),
                  style="TLabel").grid(row=4, column=0, columnspan=4, sticky="w", pady=(6, 4))
        message_var = tk.StringVar()
        ttk.Label(body, textvariable=message_var, style="MBS.Master.Error.TLabel").grid(
            row=5, column=0, columnspan=4, sticky="w")
        result = {"ok": False}

```

MBToolkit v3.5/mb_table_ui.py — lines 401-440

```
        def accept(_event=None):
            try:
                scale = validate_scale(
                    variables["raw_min"].get(), variables["raw_max"].get(),
                    variables["eng_min"].get(), variables["eng_max"].get(),
                    variables["decimals"].get(), variables["unit"].get(),
                )
            except ValueError as exc:
                message_var.set(self.tr(str(exc)))
                return
            for address in starts:
                window.scales[address] = dict(scale)
            result["ok"] = True
            dialog.destroy()

        buttons = ttk.Frame(body)
        buttons.grid(row=6, column=0, columnspan=4, sticky="e", pady=(8, 0))
        ttk.Button(buttons, text=self.tr("Aplicar"), command=accept,
                   style="MBS.Primary.TButton").grid(row=0, column=0, padx=(0, 6))
        ttk.Button(buttons, text=self.tr("Cancelar"), command=dialog.destroy).grid(
            row=0, column=1)
        dialog.bind("<Return>", accept)
        dialog.bind("<Escape>", lambda _e: dialog.destroy())
        if first_entry is not None:
            first_entry.focus_set()
        dialog.grab_set()
        self.wait_window(dialog)
        return result["ok"]

    # -- pane parts ---------------------------------------------------------
    def _table_build_header(self, frame, pane, rename, popout, close):
        """Window name + summary on the left; Rename / Pop out / □ on the right.

        Closing a window never depends on the pane being wide.
        """
        header = ttk.Frame(frame)
        header.grid(row=0, column=0, sticky="ew", padx=6, pady=(4, 2))
        header.columnconfigure(1, weight=1)
        pane["name_var"] = tk.StringVar()
        pane["summary_var"] = tk.StringVar()
```

MBToolkit v3.5/mb_table_ui.py — lines 441-480

```
441         name_label = ttk.Label(header, textvariable=pane["name_var"],
442                                style="MBS.Master.Title.TLabel")
443         name_label.grid(row=0, column=0, sticky="w")
444         name_label.bind("<Double-1>", lambda _e: rename())
445         ttk.Label(header, textvariable=pane["summary_var"]).grid(
446             row=0, column=1, sticky="w", padx=(10, 0))
447         header_buttons = ttk.Frame(header)
448         header_buttons.grid(row=0, column=2, sticky="e")
449         ttk.Button(header_buttons, text="Renomear", width=10, command=rename).grid(
450             row=0, column=0, padx=(0, 3))
451         pane["popout_btn"] = ttk.Button(header_buttons, text="Destacar", width=10,
452                                         command=popout)
453         pane["popout_btn"].grid(row=0, column=1, padx=(0, 3))
454         ttk.Button(header_buttons, text="×", width=3, command=close).grid(row=0, column=2)
455         pane["header"] = header
456         return header_buttons
457
458     def _table_build_tree(self, frame, pane, row, on_double, on_return, on_menu):
459         """Value table (address / name / value) with a thin 1 px frame."""
460         # Classic Tk highlight ring, coloured in _table_configure_tree_tags().
461         table = tk.Frame(frame, borderwidth=0, highlightthickness=1)
462         table.grid(row=row, column=0, sticky="nsew", padx=6, pady=(2, 6))
463         table.columnconfigure(0, weight=1)
464         table.rowconfigure(0, weight=1)
465         pane["table_frame"] = table
466         tree = ttk.Treeview(table, columns=("address", "name", "value"), show="headings",
467                             height=TABLE_PANE_MAX_ROWS, selectmode="extended")
468         for column, caption, width, anchor in (
469             ("address", "Endereço", 90, "center"),
470             ("name", "Nome", 200, "w"),
471             ("value", "Valor", 160, "e"),
472         ):
473             tree.heading(column, text=self.tr(caption), anchor="center")
474             tree.column(column, width=width, minwidth=60, anchor=anchor, stretch=column != "address")
475         tree.grid(row=0, column=0, sticky="nsew")
476         scrollbar = ttk.Scrollbar(table, orient="vertical", command=tree.yview)
477         scrollbar.grid(row=0, column=1, sticky="ns")
478         tree.configure(yscrollcommand=scrollbar.set)
479         tree.bind("<Double-1>", on_double)
480         tree.bind("<Return>", lambda _e: on_return())
```

MBToolkit v3.5/mb_table_ui.py — lines 481-520

```
        tree.bind("<Button-3>", on_menu)
        tree.bind("<ButtonPress-1>", lambda e: self._table_drag_start(pane, e), add="+")
        tree.bind("<B1-Motion>", lambda e: self._table_drag_move(pane, e))
        tree.bind("<ButtonRelease-1>", lambda _e: self._table_drag_stop(pane), add="+")
        for sequence in ("<Control-c>", "<Control-C>"):
            tree.bind(sequence, lambda _e: self._table_copy_rows(pane) or "break")
        pane["tree"] = tree
        self._table_configure_tree_tags(tree)
        return tree

    def _table_translate_captions(self, *containers):
        """Captions are written in canonical PT; show them in the current language."""
        for widget in containers:
            for child in widget.winfo_children():
                try:
                    if "text" in child.keys() and child.cget("text"):
                        child.configure(text=self.tr(child.cget("text")))
                except tk.TclError:
                    pass

    def _table_translate_headings(self, pane):
        for column, caption in (("address", "Endereço"), ("name", "Nome"), ("value", "Valor")):
            try:
                pane["tree"].heading(column, text=self.tr(caption))
            except tk.TclError:
                pass

    @staticmethod
    def _table_row_tags(index):
        """Alternate rows get a slightly darker background (zebra striping)."""
        return ("odd",) if index % 2 else ()

    def _table_configure_tree_tags(self, tree):
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        colors = theme_colors(self, dark)
        try:
            tree.tag_configure("stale", foreground=colors["disabled"])
            tree.tag_configure("odd", background=TABLE_STRIPE_DARK if dark
                               else TABLE_STRIPE_LIGHT)
            tree.tag_configure("generated", foreground=TABLE_GENERATED_DARK if dark
```

MBToolkit v3.5/mb_table_ui.py — lines 521-560

```
                              else TABLE_GENERATED_LIGHT)
            # A written row carries only this tag (no zebra "odd" tag).
            tree.tag_configure("written", background=TABLE_WRITTEN_DARK if dark
                               else TABLE_WRITTEN_LIGHT)
            frame = tree.master
            if isinstance(frame, tk.Frame):
                frame.configure(highlightbackground=colors["border"],
                                highlightcolor=colors["border"],
                                background=colors["input"])
        except tk.TclError:
            pass

    # -- grid ---------------------------------------------------------------
    def _table_reflow(self, canvas, grid, grid_item, frames, empty_label, uniform):
        """Arrange docked pane *frames* in as many equal columns as the width allows."""
        width = max(1, canvas.winfo_width())
        canvas.itemconfigure(grid_item, width=width)
        # Use the real requested pane width (it grows with Windows display
        # scaling) so columns never become narrower than their contents.
        pane_width = TABLE_PANE_MIN_WIDTH
        for frame in frames:
            pane_width = max(pane_width, frame.winfo_reqwidth() + 8)
        columns = max(1, width // pane_width)
        for column in range(8):
            grid.columnconfigure(column, weight=1 if column < columns else 0,
                                 uniform=uniform if column < columns else "")
        if not frames:
            empty_label.grid(row=0, column=0, sticky="w", padx=4, pady=8)
        else:
            empty_label.grid_remove()
        for index, frame in enumerate(frames):
            frame.grid(row=index // columns, column=index % columns,
                       sticky="nsew", padx=(0, 8), pady=(0, 8))

    def _table_build_scroll_grid(self, page, row):
        """Scrollable area that holds the window grid; returns (canvas, grid, item)."""
        holder = ttk.Frame(page)
        holder.grid(row=row, column=0, sticky="nsew", padx=12, pady=(0, 12))
        holder.columnconfigure(0, weight=1)
        holder.rowconfigure(0, weight=1)
```

MBToolkit v3.5/mb_table_ui.py — lines 561-600

```
        canvas = tk.Canvas(holder, highlightthickness=0, borderwidth=0)
        canvas.grid(row=0, column=0, sticky="nsew")
        scrollbar = ttk.Scrollbar(holder, orient="vertical", command=canvas.yview)
        scrollbar.grid(row=0, column=1, sticky="ns")
        canvas.configure(yscrollcommand=scrollbar.set)
        grid = ttk.Frame(canvas)
        item = canvas.create_window((0, 0), window=grid, anchor="nw")
        grid.bind("<Configure>",
                  lambda _e: canvas.configure(scrollregion=canvas.bbox("all")))
        return canvas, grid, item

    def _table_install_wheel_tags(self, root_widget, wheel_tag):
        """Give new widgets the global wheel guard; plain panels scroll the grid."""
        stack = [root_widget]
        while stack:
            widget = stack.pop()
            try:
                stack.extend(widget.winfo_children())
                tags = tuple(t for t in widget.bindtags()
                             if t not in (WHEEL_GUARD_TAG, wheel_tag))
                if isinstance(widget, (ttk.Treeview, ttk.Scrollbar, tk.Toplevel)):
                    widget.bindtags((WHEEL_GUARD_TAG,) + tags)
                else:
                    widget.bindtags((wheel_tag, WHEEL_GUARD_TAG) + tags)
            except tk.TclError:
                continue

    @staticmethod
    def _table_wheel(canvas, event):
        if canvas is None:
            return "break"
        try:
            top, bottom = canvas.yview()
            if top <= 0.0 and bottom >= 1.0:
                return "break"
            if getattr(event, "num", None) == 4:
                step = -1
            elif getattr(event, "num", None) == 5:
                step = 1
            else:
```

MBToolkit v3.5/mb_table_ui.py — lines 601-640

```
                step = -1 if event.delta > 0 else 1
            canvas.yview_scroll(step * 3, "units")
        except tk.TclError:
            pass
        return "break"

    # -- drag selection ------------------------------------------------------
    # ttk.Treeview has no rubber-band selection: pressing on a row and dragging
    # selects every row from that row to the one under the pointer, scrolling
    # when the pointer leaves the table. Ctrl+drag adds the block to the
    # selection that existed before the press.
    def _table_drag_start(self, pane, event):
        tree = pane["tree"]
        row = tree.identify_row(event.y)
        if not row or tree.identify_region(event.x, event.y) != "cell":
            pane["drag"] = None
            return
        ctrl = bool(event.state & 0x0004)
        before = tuple(tree.selection())
        pane["drag"] = {"anchor": row, "base": before if ctrl else (), "y": event.y,
                        "after_id": None}

    def _table_drag_move(self, pane, event):
        drag = pane.get("drag")
        if not drag:
            return None
        drag["y"] = event.y
        self._table_drag_update(pane)
        return "break"

    def _table_drag_update(self, pane):
        drag = pane.get("drag")
        if not drag:
            return
        tree = pane["tree"]
        drag["after_id"] = None
        height = tree.winfo_height()
        y = drag["y"]
        if y < 0 or y > height:
            # Pointer outside the rows: scroll and keep extending the block.
```

MBToolkit v3.5/mb_table_ui.py — lines 641-680

```
            tree.yview_scroll(-1 if y < 0 else 1, "units")
            y = 1 if y < 0 else height - 2
            drag["after_id"] = self.after(60, lambda: self._table_drag_update(pane))
        row = tree.identify_row(y)
        rows = tree.get_children()
        if not row or drag["anchor"] not in rows:
            return
        first, last = sorted((rows.index(drag["anchor"]), rows.index(row)))
        block = rows[first:last + 1]
        tree.selection_set(tuple(dict.fromkeys(drag["base"] + tuple(block))))

    def _table_drag_stop(self, pane):
        drag = pane.get("drag")
        if drag and drag.get("after_id"):
            try:
                self.after_cancel(drag["after_id"])
            except tk.TclError:
                pass
        pane["drag"] = None

    # -- selection / copy / row menu ------------------------------------------------
    @staticmethod
    def _table_selected_rows(pane):
        if pane is None:
            return []
        return sorted(int(iid[1:]) for iid in pane["tree"].selection() if iid.startswith("r"))

    def _table_selected_group_starts(self, window, pane):
        """PDU addresses where the values of the selected rows start."""
        starts = []
        for row in self._table_selected_rows(pane):
            offset, _size, fmt = window.group_of_row(row)
            address = window.address + offset
            if fmt is not None and address not in starts:
                starts.append(address)
        return starts

    def _table_copy_rows(self, pane):
        """Copy the selected rows (all rows if none) as tab-separated text."""
        if pane is None:
```

MBToolkit v3.5/mb_table_ui.py — lines 681-720

```
            return
        tree = pane["tree"]
        rows = list(tree.selection()) or list(tree.get_children())
        lines = ["\t".join(str(v) for v in tree.item(iid, "values")) for iid in rows]
        if lines:
            self.clipboard_clear()
            self.clipboard_append("\n".join(lines))

    def _table_fill_row_menu(self, menu, pane, window, registers, *, set_format,
                             edit_scale, remove_scale, copy, set_base, extra=()):
        """Right-click menu: Formato ▸, Repor, Escala…, Remover escala, Copiar, Base ▸.

        *extra* adds tab-specific entries (PT label, command) after the scale
        entries, in their own section.
        """
        menu.delete(0, "end")
        for child in menu.winfo_children():
            child.destroy()
        state = "normal" if registers else "disabled"
        format_menu = tk.Menu(menu, tearoff=False)
        sub_menus = {2: tk.Menu(format_menu, tearoff=False),
                     4: tk.Menu(format_menu, tearoff=False)}
        for key, label, size in REGISTER_FORMATS:
            target = format_menu if size == 1 else sub_menus[size]
            target.add_command(label=label, command=lambda k=key: set_format(k))
        format_menu.add_separator()
        format_menu.add_cascade(label=self.tr("32 bits (2 registos)"), menu=sub_menus[2])
        format_menu.add_cascade(label=self.tr("64 bits (4 registos)"), menu=sub_menus[4])
        menu.add_cascade(label=self.tr("Formato"), menu=format_menu, state=state)
        menu.add_command(label=self.tr("Repor formato da janela"),
                         command=lambda: set_format(None), state=state)
        menu.add_command(label=self.tr("Escala…"), command=edit_scale, state=state)
        menu.add_command(label=self.tr("Remover escala"), command=remove_scale, state=state)
        if extra:
            menu.add_separator()
            for label, command in extra:
                menu.add_command(label=self.tr(label), command=command)
        menu.add_separator()
        menu.add_command(label=self.tr("Copiar"), accelerator="Ctrl+C", command=copy)
        menu.add_separator()
```

MBToolkit v3.5/mb_table_ui.py — lines 721-760

```
        base_menu = tk.Menu(menu, tearoff=False)
        base_var = tk.IntVar(value=window.base)
        pane["menu_base_var"] = base_var  # keep a reference while the menu lives
        base_menu.add_radiobutton(label=self.tr("0 — PDU Address"), variable=base_var, value=0,
                                  command=lambda: set_base(0))
        base_menu.add_radiobutton(label=self.tr("1 — numeração dos manuais"), variable=base_var,
                                  value=1, command=lambda: set_base(1))
        menu.add_cascade(label=self.tr("Base de endereços"), menu=base_menu)

    @staticmethod
    def _table_popup(menu, event):
        try:
            menu.tk_popup(event.x_root, event.y_root)
        finally:
            menu.grab_release()

    # -- workspace menu ------------------------------------------------------
    def _table_fill_workspace_menu(self, menu, names, *, save, open_, delete, folder):
        """Workspace menu with the saved workspaces listed directly in it."""
        menu.delete(0, "end")
        for child in menu.winfo_children():
            child.destroy()
        menu.add_command(label=self.tr("Guardar workspace…"), command=save)
        open_menu = tk.Menu(menu, tearoff=False)
        delete_menu = tk.Menu(menu, tearoff=False)
        for name in names:
            open_menu.add_command(label=name, command=lambda n=name: open_(n))
            delete_menu.add_command(label=name, command=lambda n=name: delete(n))
        if not names:
            for submenu in (open_menu, delete_menu):
                submenu.add_command(label=self.tr("(nenhum workspace guardado)"),
                                    state="disabled")
        menu.add_cascade(label=self.tr("Abrir workspace"), menu=open_menu)
        menu.add_cascade(label=self.tr("Apagar workspace"), menu=delete_menu)
        menu.add_separator()
        menu.add_command(label=self.tr("Abrir pasta dos workspaces"), command=folder)

    def _table_open_folder(self, folder):
        try:
            if os.name == "nt":
```

MBToolkit v3.5/mb_table_ui.py — lines 761-792

```
                os.startfile(str(folder))
            elif sys.platform == "darwin":
                subprocess.Popen(["open", str(folder)])
            else:
                subprocess.Popen(["xdg-open", str(folder)])
        except Exception as exc:
            messagebox.showerror(self.tr("Abrir pasta dos workspaces"), str(exc))

    # -- palette ------------------------------------------------------------------
    def _table_apply_toplevel_palette(self, top):
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        try:
            top.configure(background=theme_colors(self, dark)["root"])
        except tk.TclError:
            pass

    def _table_apply_menus_palette(self, *menus):
        """Colour menus and every submenu below them (classic Tk widgets)."""
        dark = bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get())
        colors = theme_colors(self, dark)
        stack = [m for m in menus if m is not None]
        while stack:
            widget = stack.pop()
            try:
                stack.extend(widget.winfo_children())
                widget.configure(
                    background=colors["panel"], foreground=colors["text"],
                    activebackground=colors["select"], activeforeground=colors["select_text"],
                    disabledforeground=colors["disabled"],
                )
            except tk.TclError:
                pass
```

MBToolkit v3.5/mb_tcp.py

500 source lines.

MBToolkit v3.5/mb_tcp.py — lines 1-40

```
#!/usr/bin/env python3
"""Modbus TCP transport: MBAP framing, Server engine and Client engine.

This module has no widgets. Modbus TCP carries the same PDU as Modbus RTU
(function code + data) inside an MBAP header instead of Slave ID + CRC:

    Transaction ID (2) | Protocol ID = 0 (2) | Length (2) | Unit ID (1) | PDU

The Modbus logic is not repeated here: a TCP request is turned into the
equivalent RTU frame (Unit ID as Slave ID, CRC added) and served by
``mb_slave.handle_request()``; a Client request is built by ``mb_master`` and
its response is matched by ``find_master_response()``. Only the envelope,
the sockets and the per-connection bookkeeping are TCP-specific.
"""

import ipaddress
import json
import os
import random
import selectors
import socket
import subprocess
import threading
import time
from dataclasses import dataclass

from mb_master import MasterEngine, PollDefinition, build_read_request, find_master_response
from mb_protocol import append_crc, hex_bytes
from mb_slave import FC_TABLES, handle_request

MBAP_LENGTH = 7
MODBUS_TCP_PORT = 502
# Largest Length field: Unit ID + a 253-byte PDU.
MBAP_MAX_LENGTH = 254
# Exception sent by the Server for a Unit ID it does not simulate.
EXCEPTION_GATEWAY_TARGET_FAILED = 0x0B
SERVER_MAX_CLIENTS = 16
# A Client does not retry a device that refused the connection for this long.
CLIENT_RETRY_SECONDS = 1.0

```

MBToolkit v3.5/mb_tcp.py — lines 41-80

```
41
42 # ---------------------------------------------------------------------------
43 # MBAP framing
44 # ---------------------------------------------------------------------------
45
46 def build_adu(transaction, unit, pdu, protocol=0):
47     """MBAP header + PDU."""
48     pdu = bytes(pdu)
49     return (transaction.to_bytes(2, "big") + protocol.to_bytes(2, "big")
50             + (len(pdu) + 1).to_bytes(2, "big") + bytes((unit & 0xFF,)) + pdu)
51
52
53 def adu_length(buffer):
54     """Total length of the ADU starting at buffer[0]; None if the header is incomplete.
55
56     Raises ValueError when the Length field is impossible (the connection is
57     then out of step and must be closed).
58     """
59     if len(buffer) < 6:
60         return None
61     length = (buffer[4] << 8) | buffer[5]
62     if not 2 <= length <= MBAP_MAX_LENGTH:
63         raise ValueError(f"MBAP Length {length}")
64     return 6 + length
65
66
67 def parse_adu(adu):
68     """(transaction, protocol, unit, pdu) of one complete ADU."""
69     return ((adu[0] << 8) | adu[1], (adu[2] << 8) | adu[3], adu[6], bytes(adu[7:]))
70
71
72 def rtu_to_adu(rtu_frame, transaction):
73     """ADU for an RTU frame: Slave ID becomes Unit ID, the CRC is dropped."""
74     return build_adu(transaction, rtu_frame[0], rtu_frame[1:-2])
75
76
77 def adu_to_rtu(unit, pdu):
78     """Equivalent RTU frame (Unit ID as Slave ID, valid CRC) of a TCP PDU."""
79     return append_crc(bytes((unit & 0xFF,)) + bytes(pdu))
80
```

MBToolkit v3.5/mb_tcp.py — lines 81-120

```
 81
 82 def validate_host(text):
 83     """Return a cleaned IP address or host name; raise ValueError (PT)."""
 84     host = str(text or "").strip()
 85     if not host or len(host) > 253 or any(ch.isspace() for ch in host):
 86         raise ValueError("Indica um endereço IP ou nome válido.")
 87     return host
 88
 89
 90 def validate_tcp_port(text):
 91     try:
 92         port = int(str(text).strip())
 93     except ValueError:
 94         raise ValueError("A porta TCP tem de estar entre 1 e 65535.") from None
 95     if not 1 <= port <= 65535:
 96         raise ValueError("A porta TCP tem de estar entre 1 e 65535.")
 97     return port
 98
 99
100 def list_ipv4_interfaces():
101     """[(ip, interface name)] of this PC's IPv4 addresses, loopback excluded.
102
103     On Windows the names come from Get-NetIPAddress (run hidden, 5 s limit);
104     elsewhere, or if that fails, the addresses of the host name are used
105     without names.
106     """
107     found = []
108     if os.name == "nt":
109         try:
110             command = ("[Console]::OutputEncoding = [Text.Encoding]::UTF8; "
111                        "Get-NetIPAddress -AddressFamily IPv4 | "
112                        "Select-Object IPAddress,InterfaceAlias | ConvertTo-Json -Compress")
113             output = subprocess.run(
114                 ["powershell", "-NoProfile", "-NonInteractive", "-Command", command],
115                 capture_output=True, timeout=5,
116                 creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0),
117             ).stdout.decode("utf-8", "replace").strip()
118             data = json.loads(output) if output else []
119             for item in data if isinstance(data, list) else [data]:
120                 found.append((str(item.get("IPAddress", "")), str(item.get("InterfaceAlias", ""))))
```

MBToolkit v3.5/mb_tcp.py — lines 121-160

```
        except Exception:
            found = []
    if not found:
        try:
            for info in socket.getaddrinfo(socket.gethostname(), None, socket.AF_INET):
                found.append((info[4][0], ""))
        except OSError:
            pass
    result, seen = [], set()
    for ip, name in found:
        try:
            if ipaddress.ip_address(ip).is_loopback or ip in seen:
                continue
        except ValueError:
            continue
        seen.add(ip)
        result.append((ip, name))
    # Link-local addresses (169.254.x.x, adapters without DHCP) go last.
    return sorted(result, key=lambda item: (ipaddress.ip_address(item[0]).is_link_local,
                                            ipaddress.ip_address(item[0])))


# --------------------------------------------------------------------------
# Server
# --------------------------------------------------------------------------

def _request_area(fc, pdu):
    """(table, address, qty) named by a request PDU, for the window counters."""
    if fc not in FC_TABLES or len(pdu) < 5:
        return None, None, None
    address = (pdu[1] << 8) | pdu[2]
    qty = 1 if fc in (5, 6) else (pdu[3] << 8) | pdu[4]
    return FC_TABLES[fc], address, qty


class ServerEngine:
    """Modbus TCP server: answers several clients from one SlaveStore.

    One thread serves the listening socket and every client connection with
    ``selectors``. Each request is answered from the store like the RTU Slave
```

MBToolkit v3.5/mb_tcp.py — lines 161-200

```
    (``handle_request``); a Unit ID that is not simulated gets Exception 0B
    (Gateway Target Device Failed to Respond). The fault simulation of the
    Unit ID applies: no response, a chosen Exception, a corrupted MBAP header
    (Protocol ID 1) and a delay, which is scheduled so other clients are not
    held up. Events: ``server_ready``, ``server_client``, ``server_request``
    and ``server_stopped``.
    """

    def __init__(self, event_queue, store, *, host, port, rng=None,
                 max_clients=SERVER_MAX_CLIENTS):
        self.event_queue = event_queue
        self.store = store
        self.host = host
        self.port = int(port)
        self.rng = rng or random.Random()
        self.max_clients = max_clients
        self.stop_event = threading.Event()
        self.thread = None
        self.bound_port = None
        self._clients = {}    # socket -> {"peer": "ip:port", "rx": bytearray}
        self._pending = []    # [(due, socket, data)] delayed responses

    def start(self):
        self.thread = threading.Thread(target=self.run, daemon=True)
        self.thread.start()

    def stop(self, join_timeout=1.0):
        self.stop_event.set()
        thread = self.thread
        if thread is not None and thread.is_alive() and thread is not threading.current_thread():
            thread.join(timeout=join_timeout)

    def run(self):
        error = None
        listener = None
        selector = selectors.DefaultSelector()
        try:
            try:
                listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                if hasattr(socket, "SO_EXCLUSIVEADDRUSE"):
```

MBToolkit v3.5/mb_tcp.py — lines 201-240

```
                    # Windows: never share the port with another server.
                    listener.setsockopt(socket.SOL_SOCKET, socket.SO_EXCLUSIVEADDRUSE, 1)
                listener.bind((self.host, self.port))
                listener.listen(self.max_clients)
                listener.setblocking(False)
            except OSError as exc:
                error = f"{type(exc).__name__}: {exc}"
                return
            self.bound_port = listener.getsockname()[1]
            selector.register(listener, selectors.EVENT_READ, None)
            self.event_queue.put(("server_ready", {"host": self.host, "port": self.bound_port}))
            while not self.stop_event.is_set():
                timeout = 0.1
                if self._pending:
                    timeout = max(0.0, min(timeout, min(p[0] for p in self._pending)
                                           - time.perf_counter()))
                for key, _mask in selector.select(timeout):
                    if key.data is None:
                        self._accept(listener, selector)
                    else:
                        self._receive(key.fileobj, selector)
                self._send_due(selector)
        finally:
            for sock in list(self._clients):
                self._drop(sock, selector, notify=False)
            if listener is not None:
                try:
                    listener.close()
                except OSError:
                    pass
            selector.close()
            self.event_queue.put(("server_stopped", {"error": error}))

    # -- connections ------------------------------------------------------
    def _accept(self, listener, selector):
        try:
            sock, address = listener.accept()
        except OSError:
            return
        if len(self._clients) >= self.max_clients:
```

MBToolkit v3.5/mb_tcp.py — lines 241-280

```
            sock.close()
            return
        sock.setblocking(False)
        peer = f"{address[0]}:{address[1]}"
        self._clients[sock] = {"peer": peer, "rx": bytearray()}
        selector.register(sock, selectors.EVENT_READ, peer)
        self.event_queue.put(("server_client", {"peer": peer, "connected": True,
                                                "count": len(self._clients)}))

    def _drop(self, sock, selector, notify=True):
        client = self._clients.pop(sock, None)
        try:
            selector.unregister(sock)
        except (KeyError, ValueError, OSError):
            pass
        try:
            sock.close()
        except OSError:
            pass
        self._pending = [p for p in self._pending if p[1] is not sock]
        if notify and client is not None:
            self.event_queue.put(("server_client", {"peer": client["peer"], "connected": False,
                                                    "count": len(self._clients)}))

    def _receive(self, sock, selector):
        client = self._clients.get(sock)
        if client is None:
            return
        try:
            data = sock.recv(4096)
        except (BlockingIOError, InterruptedError):
            return
        except OSError:
            data = b""
        if not data:
            self._drop(sock, selector)
            return
        buffer = client["rx"]
        buffer.extend(data)
        while True:
```

MBToolkit v3.5/mb_tcp.py — lines 281-320

```
            try:
                length = adu_length(buffer)
            except ValueError:
                self._drop(sock, selector)  # out of step: close like a real server
                return
            if length is None or len(buffer) < length:
                return
            adu = bytes(buffer[:length])
            del buffer[:length]
            self._answer(sock, client["peer"], adu)

    def _send(self, sock, data):
        try:
            sock.sendall(data)
        except OSError:
            pass

    def _send_due(self, selector):
        if not self._pending:
            return
        now = time.perf_counter()
        due = [p for p in self._pending if p[0] <= now]
        self._pending = [p for p in self._pending if p[0] > now]
        for _due, sock, data in due:
            if sock in self._clients:
                self._send(sock, data)

    # -- requests ------------------------------------------------------------
    def _answer(self, sock, peer, adu):
        transaction, protocol, unit, pdu = parse_adu(adu)
        if protocol != 0 or not pdu:
            return  # not Modbus: ignored, as a real server does
        fc = pdu[0] & 0x7F
        table, address, qty = _request_area(fc, pdu)
        info = {"slave": unit, "fc": fc, "table": table, "address": address, "qty": qty,
                "status": "ok", "exception_code": None, "written": [], "fault": False,
                "time": time.time(), "peer": peer}

        response_pdu = None
        fault = None
```

MBToolkit v3.5/mb_tcp.py — lines 321-360

```
        if unit not in self.store.slave_ids():
            info.update(status="exception", exception_code=EXCEPTION_GATEWAY_TARGET_FAILED)
            response_pdu = bytes((fc | 0x80, EXCEPTION_GATEWAY_TARGET_FAILED))
        else:
            fault = self.store.fault(unit)
            if fault and self.rng.random() * 100.0 >= fault["probability"]:
                fault = None
            info["fault"] = bool(fault)
            if fault and fault["no_response"]:
                info["status"] = "no_response"
            elif fault and fault["exception"]:
                info.update(status="exception", exception_code=fault["exception"])
                response_pdu = bytes((fc | 0x80, fault["exception"]))
            else:
                reply = handle_request(self.store, adu_to_rtu(unit, pdu))
                info.update(status=reply.status, exception_code=reply.exception_code,
                            written=list(reply.written))
                if reply.table is not None:
                    info.update(table=reply.table, address=reply.address, qty=reply.qty)
                if reply.response is not None:
                    response_pdu = reply.response[1:-2]

        if response_pdu is not None:
            # A corrupted response carries Protocol ID 1 (clients reject it).
            protocol_out = 1 if fault and fault["bad_crc"] else 0
            data = build_adu(transaction, unit, response_pdu, protocol_out)
            delay_s = fault["delay_ms"] / 1000.0 if fault else 0.0
            if delay_s > 0:
                self._pending.append((time.perf_counter() + delay_s, sock, data))
            else:
                self._send(sock, data)
            info["response"] = len(data)
        self.event_queue.put(("server_request", info))


# ---------------------------------------------------------------------------
# Client
# ---------------------------------------------------------------------------

@dataclass
```

MBToolkit v3.5/mb_tcp.py — lines 361-400

```python
class ClientPollDefinition(PollDefinition):
    """Poll window of the TCP Client: the device's address and Unit ID 0–255."""
    host: str = "127.0.0.1"
    port: int = MODBUS_TCP_PORT

    def request(self):
        # RTU form with the Unit ID in the Slave byte: mb_master validates
        # Slave IDs 1–247, Modbus TCP allows Unit IDs 0–255.
        frame = build_read_request(1, self.fc, self.address, self.qty)
        return append_crc(bytes((self.slave & 0xFF,)) + frame[1:-2])


class ClientEngine(MasterEngine):
    """Modbus TCP client: the Master scheduler over one connection per device.

    Windows are served one request at a time, exactly like the Master (writes,
    then "read once", then the earliest due poll). Each window names its
    device (host:port); connections are opened on first use, kept open and
    reopened after a failure, at most once per CLIENT_RETRY_SECONDS. A device
    that cannot be reached gives the status ``no_connection``. Events are
    ``client_ready``, ``client_result`` and ``client_stopped``.
    """

    def __init__(self, event_queue, *, response_timeout_ms, delay_ms):
        super().__init__(None, event_queue, baud=115200, parity="None", stopbits="1",
                         response_timeout_ms=response_timeout_ms, delay_ms=delay_ms,
                         event_prefix="client")
        self.delay_s = max(0.0, float(delay_ms) / 1000.0)
        self._sockets = {}      # (host, port) -> socket
        self._failed = {}       # (host, port) -> (time, error text)
        self._transaction = 0

    def run(self):
        try:
            super().run()
        finally:
            for sock in self._sockets.values():
                try:
                    sock.close()
                except OSError:
```

MBToolkit v3.5/mb_tcp.py — lines 401-440

```
                    pass
            self._sockets = {}

    def _connection(self, target):
        sock = self._sockets.get(target)
        if sock is not None:
            return sock
        failed = self._failed.get(target)
        if failed and time.monotonic() - failed[0] < CLIENT_RETRY_SECONDS:
            raise ConnectionError(failed[1])
        try:
            sock = socket.create_connection(target, timeout=self.response_timeout_s)
            sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        except OSError as exc:
            text = f"{type(exc).__name__}: {exc}"
            self._failed[target] = (time.monotonic(), text)
            raise ConnectionError(text) from None
        self._failed.pop(target, None)
        self._sockets[target] = sock
        return sock

    def _close(self, target):
        sock = self._sockets.pop(target, None)
        if sock is not None:
            try:
                sock.close()
            except OSError:
                pass

    def transact_for(self, window_id, request):
        with self._lock:
            entry = self._polls.get(window_id)
        result = {"tx": "", "rx": "", "status": "stopped", "values": None,
                  "exception_code": None, "response_ms": None}
        if entry is None or self.stop_event.is_set():
            return result
        definition = entry[0]
        target = (definition.host, int(definition.port))
        request = append_crc(bytes(request[:-2]))  # valid CRC for the Unit ID byte
        self._transaction = (self._transaction + 1) & 0xFFFF
```

MBToolkit v3.5/mb_tcp.py — lines 441-480

```
        transaction = self._transaction
        adu = rtu_to_adu(request, transaction)
        result["tx"] = hex_bytes(adu)
        try:
            sock = self._connection(target)
            sock.sendall(adu)
        except (OSError, ConnectionError) as exc:
            self._close(target)
            result.update(status="no_connection", error=str(exc))
            return result

        started = time.perf_counter()
        deadline = started + self.response_timeout_s
        buffer = bytearray()
        while not self.stop_event.is_set():
            remaining = deadline - time.perf_counter()
            if remaining <= 0:
                result["status"] = "timeout"
                result["rx"] = hex_bytes(buffer)
                return result
            try:
                sock.settimeout(min(remaining, 0.1))
                data = sock.recv(4096)
            except socket.timeout:
                continue
            except OSError as exc:
                self._close(target)
                result.update(status="no_connection", error=f"{type(exc).__name__}: {exc}")
                return result
            if not data:
                self._close(target)
                result.update(status="no_connection", error="ligação fechada pelo equipamento")
                return result
            buffer.extend(data)
            while True:
                try:
                    length = adu_length(buffer)
                except ValueError:
                    self._close(target)
                    result.update(status="invalid", rx=hex_bytes(buffer))
```

MBToolkit v3.5/mb_tcp.py — lines 481-500

```
481                     return result
482                 if length is None or len(buffer) < length:
483                     break
484                 answer = bytes(buffer[:length])
485                 del buffer[:length]
486                 got_transaction, protocol, _unit, pdu = parse_adu(answer)
487                 if got_transaction != transaction:
488                     continue  # late answer to an earlier request
489                 result["rx"] = hex_bytes(answer)
490                 result["response_ms"] = (time.perf_counter() - started) * 1000.0
491                 found = None if protocol != 0 else find_master_response(
492                     adu_to_rtu(request[0], pdu), request)
493                 if found is None:
494                     result["status"] = "invalid"
495                 else:
496                     result.update(status="exception" if found["kind"] == "EXCEPTION" else "ok",
497                                   values=found["values"],
498                                   exception_code=found["exception_code"])
499                 return result
500         return result
```

MBToolkit v3.5/mb_tcp_ui.py

509 source lines.

MBToolkit v3.5/mb_tcp_ui.py — lines 1-40

```
#!/usr/bin/env python3
"""Modbus TCP Client and Server tabs.

Both tabs are second instances of the serial tabs, run on ``TabInstance``:
``ClientTab`` is the Master tab over ``mb_tcp.ClientEngine`` and
``ServerTab`` is the Slave tab over ``mb_tcp.ServerEngine``. Windows,
formats, scales, names, writes, faults, automatic values and workspaces are
the same code as in the serial tabs; this module only replaces the
connection bar, the device of each Client window and the engines. The TCP
tabs do not use a serial port, so they run independently of the Sniffer,
the Finder, the Master and the Slave.
"""

import ipaddress
import threading
import tkinter as tk
from tkinter import messagebox, ttk

from mb_config import (
    CLIENT_WORKSPACE_FORMAT,
    MASTER_DEFAULT_DELAY_MS,
    MASTER_DEFAULT_RESPONSE_TIMEOUT_MS,
    MASTER_MAX_DELAY_MS,
    MASTER_MAX_RESPONSE_TIMEOUT_MS,
    MASTER_MIN_DELAY_MS,
    MASTER_MIN_RESPONSE_TIMEOUT_MS,
    SERVER_DEFAULT_BIND,
    SERVER_WORKSPACE_FORMAT,
    TCP_DEFAULT_PORT,
    get_client_workspaces_folder,
    get_server_workspaces_folder,
)
from mb_master import validate_read_definition
from mb_master_ui import MasterMixin, MasterWindow
from mb_slave_ui import SlaveMixin
from mb_table_ui import TabInstance, parse_int_field
from mb_tcp import (
    ClientEngine,
    ClientPollDefinition,
    ServerEngine,
```

MBToolkit v3.5/mb_tcp_ui.py — lines 41-80

```
41      list_ipv4_interfaces,
42      validate_host,
43      validate_tcp_port,
44  )
45  from mb_widgets import RefreshButton
46
47  LOCALHOST = "127.0.0.1"
48  ALL_INTERFACES = "0.0.0.0"
49
50
51  # ---------------------------------------------------------------------------
52  # Client
53  # ---------------------------------------------------------------------------
54
55  class ClientWindow(MasterWindow):
56      """Poll window of the Client: a Master window plus the device (host:port).
57
58      ``slave`` holds the Unit ID, 0–255 in Modbus TCP.
59      """
60
61      def __init__(self, window_id, number, host=LOCALHOST, port=TCP_DEFAULT_PORT, **kwargs):
62          super().__init__(window_id, number, **kwargs)
63          self.host = host
64          self.port = port
65
66      def definition(self):
67          return ClientPollDefinition(self.window_id, self.slave, self.fc, self.address, self.qty,
68                                      self.scan_ms, self.enabled, host=self.host, port=self.port)
69
70      def to_json(self):
71          data = super().to_json()
72          data.update(host=self.host, port=self.port)
73          return data
74
75      @staticmethod
76      def validated_kwargs(data):
77          if not isinstance(data, dict):
78              raise ValueError("Janela inválida.")
79          unit = int(data.get("slave", 1))
80          if not 0 <= unit <= 255:
```

MBToolkit v3.5/mb_tcp_ui.py — lines 81-120

```
            raise ValueError("O Unit ID tem de estar entre 0 e 255.")
        kwargs = MasterWindow.validated_kwargs(dict(data, slave=1))
        kwargs.update(slave=unit, host=validate_host(data.get("host", LOCALHOST)),
                      port=validate_tcp_port(data.get("port", TCP_DEFAULT_PORT)))
        return kwargs


class ClientTab(MasterMixin, TabInstance):
    """Modbus TCP Client tab: the Master tab with one TCP device per window."""

    _master_tab_name = "Client"
    _master_settings_key = "client"
    _master_page_attr = "client_page"
    _master_wheel_tag = "MBToolkit.ClientWheel"
    _master_window_class = ClientWindow
    _master_ws_folder = staticmethod(get_client_workspaces_folder)
    _master_ws_formats = (CLIENT_WORKSPACE_FORMAT,)
    _master_ws_close_question = "Guardar alterações ao workspace do Client \"{name}\"?"
    _master_id_label = "Unit ID"
    _master_id_range = (0, 255)
    _master_id_message = "O Unit ID tem de estar entre 0 e 255."
    _master_write_id_label = "Unit ID"
    _master_write_id_range = (0, 255)
    _master_connect_first = "Liga primeiro o Client."
    _master_open_in_slave = ("Abrir no Server", "open_in_server")

    def __init__(self, app):
        super().__init__(app)
        # The TCP tabs never take the application's serial busy lock.
        self.busy = False

    # -- connection bar -------------------------------------------------------
    def _master_build_connection(self, page, saved):
        connection = ttk.LabelFrame(
            page, text="Ligação — o Client liga-se a equipamentos Modbus TCP")
        connection.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
        row = ttk.Frame(connection)
        row.grid(row=0, column=0, sticky="w", padx=8, pady=(8, 4))
        self.master_timeout_var = tk.StringVar(
            value=str(saved.get("response_timeout_ms", MASTER_DEFAULT_RESPONSE_TIMEOUT_MS)))
```

MBToolkit v3.5/mb_tcp_ui.py — lines 121-160

```
121         self.master_delay_var = tk.StringVar(
122             value=str(saved.get("delay_ms", MASTER_DEFAULT_DELAY_MS)))
123         fields = (
124             ("Timeout resposta (ms)", ttk.Spinbox(
125                 row, textvariable=self.master_timeout_var, width=6, increment=50,
126                 from_=MASTER_MIN_RESPONSE_TIMEOUT_MS, to=MASTER_MAX_RESPONSE_TIMEOUT_MS)),
127             ("Atraso entre pedidos (ms)", ttk.Spinbox(
128                 row, textvariable=self.master_delay_var, width=6, increment=5,
129                 from_=MASTER_MIN_DELAY_MS, to=MASTER_MAX_DELAY_MS)),
130         )
131         self._master_setting_widgets = []
132         for index, (caption, widget) in enumerate(fields):
133             ttk.Label(row, text=caption).grid(row=0, column=index * 2, padx=(0, 4))
134             widget.grid(row=0, column=index * 2 + 1, padx=(0, 16))
135             self._master_setting_widgets.append(widget)
136         ttk.Label(row, text="O IP, a porta e o Unit ID de cada equipamento ficam em cada janela.") \
137             .grid(row=0, column=len(fields) * 2)
138 
139         controls = ttk.Frame(connection)
140         controls.grid(row=1, column=0, sticky="w", padx=8, pady=(0, 8))
141         self.master_connect_btn = ttk.Button(
142             controls, text="Ligar", command=self.master_connect, style="MBS.Primary.TButton")
143         self.master_connect_btn.grid(row=0, column=0, padx=(0, 6))
144         self.master_disconnect_btn = ttk.Button(
145             controls, text="Desligar", command=self.master_disconnect, state="disabled")
146         self.master_disconnect_btn.grid(row=0, column=1, padx=(0, 18))
147         self.master_status_var = tk.StringVar(value=self.tr("Desligado"))
148         ttk.Label(controls, textvariable=self.master_status_var).grid(row=0, column=2)
149 
150         for variable in (self.master_timeout_var, self.master_delay_var):
151             variable.trace_add("write", lambda *_a: self._schedule_ui_settings_save())
152 
153     def _master_connection_json(self):
154         return {"response_timeout_ms": self.master_timeout_var.get(),
155                 "delay_ms": self.master_delay_var.get()}
156 
157     def _master_apply_connection(self, connection):
158         for key, var in (("response_timeout_ms", self.master_timeout_var),
159                          ("delay_ms", self.master_delay_var)):
160             if str(connection.get(key, "")).isdigit():
```

MBToolkit v3.5/mb_tcp_ui.py — lines 161-200

```
161                 var.set(str(connection[key]))
162
163     def settings_json(self):
164         data = self._master_connection_json()
165         data["last_workspace"] = self.master_workspace.current
166         return data
167
168     # -- device of each window ------------------------------------------------------
169     def _master_build_target(self, frame, pane, window):
170         target = ttk.Frame(frame)
171         target.grid(row=1, column=0, sticky="w", padx=6, pady=(2, 0))
172         pane["host_var"] = tk.StringVar(value=window.host)
173         pane["port_var"] = tk.StringVar(value=str(window.port))
174         pane["target_vars"] = ("host_var", "port_var")
175         for index, (caption, widget) in enumerate((
176             ("IP", ttk.Entry(target, width=16, textvariable=pane["host_var"])),
177             ("Porta", ttk.Entry(target, width=6, textvariable=pane["port_var"])),
178         )):
179             ttk.Label(target, text=caption).grid(row=0, column=index * 2, padx=(0, 3))
180             widget.grid(row=0, column=index * 2 + 1, padx=(0, 10))
181         return target
182
183     def _master_parse_target(self, pane):
184         return {"host": validate_host(pane["host_var"].get()),
185                 "port": validate_tcp_port(pane["port_var"].get())}
186
187     def _master_validate_definition(self, slave, fc, address, qty):
188         # The Unit ID (0–255) was checked by the field; validate the rest.
189         validate_read_definition(1, fc, address, qty)
190
191     def _master_fields_changed(self, window, pane):
192         try:
193             if (pane["host_var"].get().strip() != window.host
194                     or pane["port_var"].get().strip() != str(window.port)):
195                 return True
196         except (KeyError, tk.TclError):
197             return False
198         return super()._master_fields_changed(window, pane)
199
200     def _master_window_summary(self, window):
```

MBToolkit v3.5/mb_tcp_ui.py — lines 201-240

```
201         return f"{window.host}:{window.port} · {super()._master_window_summary(window)}"
202
203     def master_build_write(self, window, slave_text, address_text, values_text, fc,
204                            fmt=None, scale=None):
205         """Write request for the Unit ID (0–255); never a broadcast in Modbus TCP."""
206         unit = parse_int_field(slave_text, 0, 255, "O Unit ID tem de estar entre 0 e 255.")
207         request, description = super().master_build_write(
208             window, "1", address_text, values_text, fc, fmt, scale)
209         return bytes((unit,)) + request[1:], description
210
211     def open_in_server(self, window):
212         return self.app.server_tab.slave_open_from_master(window)
213
214     # -- connect ---------------------------------------------------------------
215     def _master_connection_settings(self):
216         return {
217             "response_timeout_ms": parse_int_field(
218                 self.master_timeout_var.get(), MASTER_MIN_RESPONSE_TIMEOUT_MS,
219                 MASTER_MAX_RESPONSE_TIMEOUT_MS,
220                 "O timeout de resposta tem de estar entre 10 e 10000 ms."),
221             "delay_ms": parse_int_field(
222                 self.master_delay_var.get(), MASTER_MIN_DELAY_MS, MASTER_MAX_DELAY_MS,
223                 "O atraso entre pedidos tem de estar entre 0 e 10000 ms."),
224         }
225
226     def master_connect(self):
227         if self.master_active:
228             return
229         try:
230             settings = self._master_connection_settings()
231         except ValueError as exc:
232             messagebox.showerror(self.tr("Configuração inválida"), self.tr(str(exc)))
233             return
234         for window_id in list(self.master_windows):
235             if not self.master_apply_window(window_id):
236                 return
237         self.master_engine = ClientEngine(self.event_queue, **settings)
238         for window in self.master_windows.values():
239             window.reset_counters()
240             self._master_render_status(window)
```

MBToolkit v3.5/mb_tcp_ui.py — lines 241-280

```
241             self.master_engine.set_poll(window.definition())
242         self.master_active = True
243         self.busy = True
244         self._master_lock_other_features(True)
245         self.master_status_var.set(self.tr("A ligar…"))
246         self.master_engine.start()
247
248     def _master_lock_other_features(self, locked):
249         """Only this tab's own controls: TCP does not lock the serial features."""
250         self.master_connect_btn.configure(state="disabled" if locked else "normal")
251         self.master_disconnect_btn.configure(state="normal" if locked else "disabled")
252         for widget in self._master_setting_widgets:
253             try:
254                 widget.configure(state="disabled" if locked else "normal")
255             except tk.TclError:
256                 pass
257
258
259 # ---------------------------------------------------------------------------
260 # Server
261 # ---------------------------------------------------------------------------
262
263 class ServerTab(SlaveMixin, TabInstance):
264     """Modbus TCP Server tab: the Slave tab listening on an IP and TCP port."""
265
266     _slave_tab_name = "Server"
267     _slave_settings_key = "server"
268     _slave_page_attr = "server_page"
269     _slave_wheel_tag = "MBToolkit.ServerWheel"
270     _slave_ws_folder = staticmethod(get_server_workspaces_folder)
271     _slave_ws_format = SERVER_WORKSPACE_FORMAT
272     _slave_ws_close_question = "Guardar alterações ao workspace do Server \"{name}\"?"
273     _slave_id_label = "Unit ID"
274     _slave_fault_title = "Falhas do Unit ID {slave}"
275     _slave_fault_intro = ("Simulação de falhas do Unit ID {slave} (aplica-se a todas as janelas "
276                           "deste ID).")
277     _slave_bad_reply = "MBAP inválido"
278     _slave_bad_reply_option = "Cabeçalho MBAP inválido nas respostas"
279
280     def __init__(self, app):
```

MBToolkit v3.5/mb_tcp_ui.py — lines 281-320

```
281         super().__init__(app)
282         self.busy = False
283         self._server_clients = {}     # peer -> True while connected
284         self._server_interfaces = []  # [(ip, interface name)] found on this PC
285         self._server_address = ""
286 
287     # -- connection bar ---------------------------------------------------------
288     def _slave_build_connection(self, page, saved):
289         connection = ttk.LabelFrame(
290             page, text="Ligação — o Server responde a clientes Modbus TCP")
291         connection.grid(row=0, column=0, sticky="ew", padx=12, pady=(12, 6))
292         row = ttk.Frame(connection)
293         row.grid(row=0, column=0, sticky="w", padx=8, pady=(8, 4))
294 
295         self._server_bind = saved.get("bind", SERVER_DEFAULT_BIND)
296         self.server_bind_var = tk.StringVar()
297         ttk.Label(row, text="Escutar em").grid(row=0, column=0, padx=(0, 4))
298         self.server_bind_combo = ttk.Combobox(row, textvariable=self.server_bind_var, width=36,
299                                               state="readonly")
300         self.server_bind_combo.grid(row=0, column=1)
301         self.server_bind_combo.bind("<<ComboboxSelected>>", self._server_bind_selected)
302         self.slave_refresh_btn = RefreshButton(row, command=self.server_refresh_interfaces,
303                                                size=24)
304         self.slave_refresh_btn.grid(row=0, column=2, padx=(4, 18))
305 
306         def _sync_refresh_square(_event=None):
307             height = self.server_bind_combo.winfo_height()
308             if height > 1:
309                 self.slave_refresh_btn.configure(width=height, height=height)
310 
311         self.server_bind_combo.bind("<Configure>", _sync_refresh_square, add="+")
312         self.server_port_var = tk.StringVar(value=str(saved.get("port", TCP_DEFAULT_PORT)))
313         ttk.Label(row, text="Porta TCP").grid(row=0, column=3, padx=(0, 4))
314         port_entry = ttk.Entry(row, width=7, textvariable=self.server_port_var)
315         port_entry.grid(row=0, column=4)
316         self._slave_setting_widgets = [self.server_bind_combo, port_entry]
317 
318         controls = ttk.Frame(connection)
319         controls.grid(row=1, column=0, sticky="w", padx=8, pady=(0, 4))
320         self.slave_safety_var = tk.BooleanVar(value=False)
```

MBToolkit v3.5/mb_tcp_ui.py — lines 321-360

```
321         ttk.Checkbutton(
322             controls,
323             text="Confirmo que o Server pode ficar acessível na rede escolhida.",
324             variable=self.slave_safety_var,
325         ).grid(row=0, column=0, padx=(0, 18))
326         self.slave_connect_btn = ttk.Button(
327             controls, text="Ligar", command=self.slave_connect, style="MBS.Primary.TButton")
328         self.slave_connect_btn.grid(row=0, column=1, padx=(0, 6))
329         self.slave_disconnect_btn = ttk.Button(
330             controls, text="Desligar", command=self.slave_disconnect, state="disabled")
331         self.slave_disconnect_btn.grid(row=0, column=2, padx=(0, 18))
332         self.slave_status_var = tk.StringVar(value=self.tr("Desligado"))
333         ttk.Label(controls, textvariable=self.slave_status_var).grid(row=0, column=3)
334         self.slave_stats_var = tk.StringVar()
335         ttk.Label(connection, textvariable=self.slave_stats_var).grid(
336             row=2, column=0, sticky="w", padx=8, pady=(0, 8))
337 
338         self.server_port_var.trace_add("write", lambda *_a: self._schedule_ui_settings_save())
339         self._server_fill_interfaces()
340         self.server_refresh_interfaces()
341 
342     # -- network interfaces ---------------------------------------------------------
343     def _server_choices(self):
344         """[(ip, caption)] offered in "Escutar em", this PC first, all last."""
345         choices = [(LOCALHOST, self.tr("Só este PC (127.0.0.1)"))]
346         for ip, name in self._server_interfaces:
347             choices.append((ip, f"{ip} — {name}" if name else ip))
348         known = {ip for ip, _caption in choices} | {ALL_INTERFACES}
349         if self._server_bind not in known:
350             # Remembered address of an adapter that is not present now.
351             choices.append((self._server_bind,
352                             self.trf("{ip} (não encontrado)", ip=self._server_bind)))
353         choices.append((ALL_INTERFACES, self.tr("Todas as interfaces (0.0.0.0)")))
354         return choices
355 
356     def _server_fill_interfaces(self):
357         choices = self._server_choices()
358         self.server_bind_combo["values"] = [caption for _ip, caption in choices]
359         caption = next((c for ip, c in choices if ip == self._server_bind), choices[0][1])
360         self.server_bind_var.set(caption)
```

MBToolkit v3.5/mb_tcp_ui.py — lines 361-400

```
361
362     def _server_bind_selected(self, _event=None):
363         caption = self.server_bind_var.get()
364         for ip, text in self._server_choices():
365             if text == caption:
366                 self._server_bind = ip
367         self._schedule_ui_settings_save()
368
369     def server_refresh_interfaces(self):
370         """List this PC's IPv4 interfaces in the background (PowerShell can take 1 s)."""
371         event_queue = self.event_queue
372
373         def worker():
374             event_queue.put(("server_interfaces", list_ipv4_interfaces()))
375
376         threading.Thread(target=worker, daemon=True).start()
377
378     def _slave_connection_json(self):
379         return {"bind": self._server_bind, "port": self.server_port_var.get()}
380
381     def _slave_apply_connection(self, connection):
382         bind = connection.get("bind")
383         try:
384             if bind is not None:
385                 self._server_bind = str(ipaddress.IPv4Address(str(bind)))
386                 self._server_fill_interfaces()
387         except ValueError:
388             pass
389         if str(connection.get("port", "")).isdigit():
390             self.server_port_var.set(str(connection["port"]))
391
392     def settings_json(self):
393         data = self._slave_connection_json()
394         data["last_workspace"] = self.slave_workspace.current
395         return data
396
397     # -- texts ---------------------------------------------------------------
398     def _slave_render_stats(self):
399         stats = self.slave_stats
400         self.slave_stats_var.set(self.trf(
```

MBToolkit v3.5/mb_tcp_ui.py — lines 401-440

```
            "Clientes ligados: {clients}   Pedidos: {requests}   Respostas: {responses}   "
            "Exceções: {exceptions}", clients=len(getattr(self, "_server_clients", {})),
            **stats))

    def _server_status_text(self):
        return self.trf("● A responder em {address}", address=self._server_address)

    # -- connect ---------------------------------------------------------------
    def slave_connect(self):
        if self.slave_active:
            return
        try:
            port = validate_tcp_port(self.server_port_var.get())
        except ValueError as exc:
            messagebox.showerror(self.tr("Configuração inválida"), self.tr(str(exc)))
            return
        if self._server_bind != LOCALHOST and not self.slave_safety_var.get():
            messagebox.showwarning(
                self.tr(self._slave_tab_name),
                self.tr("Escolheste uma interface de rede: confirma primeiro que o Server pode "
                        "ficar acessível nessa rede."),
            )
            return
        for window_id in list(self.slave_windows):
            if not self.slave_apply_window(window_id):
                return
        if not self.slave_windows:
            messagebox.showwarning(self.tr(self._slave_tab_name),
                                   self.tr("Cria primeiro uma janela com o Slave a simular."))
            return
        self.slave_engine = ServerEngine(self.event_queue, self.slave_store,
                                         host=self._server_bind, port=port)
        self._slave_reset_stats()
        self._server_clients = {}
        self._slave_render_stats()
        for window in self.slave_windows.values():
            window.reset_counters()
            self._slave_render_status(window)
        self.slave_active = True
        self.busy = True
```

MBToolkit v3.5/mb_tcp_ui.py — lines 441-480

```
        self._slave_lock_other_features(True)
        self.slave_status_var.set(self.tr("A iniciar o Server…"))
        self.slave_engine.start()

    def _slave_lock_other_features(self, locked):
        """Only this tab's own controls: TCP does not lock the serial features."""
        self.slave_connect_btn.configure(state="disabled" if locked else "normal")
        self.slave_disconnect_btn.configure(state="normal" if locked else "disabled")
        self.slave_refresh_btn.configure(state="disabled" if locked else "normal")
        for widget in self._slave_setting_widgets:
            try:
                widget.configure(state="disabled" if locked
                                 else "readonly" if isinstance(widget, ttk.Combobox) else "normal")
            except tk.TclError:
                pass

    def _slave_finish_session(self, payload):
        if not self.slave_active:
            return
        engine = self.slave_engine
        self.slave_engine = None
        if engine is not None:
            engine.stop(join_timeout=0.5)
        self.slave_active = False
        self.busy = False
        self._server_clients = {}
        self._slave_lock_other_features(False)
        self.slave_status_var.set(self.tr("Desligado"))
        self._slave_render_stats()
        error = (payload or {}).get("error")
        if error:
            messagebox.showerror(
                self.tr(self._slave_tab_name),
                self.trf("Não foi possível escutar em {host}:{port}: {error}",
                         host=self._server_bind, port=self.server_port_var.get(), error=error),
            )

    def handle_server_event(self, event, payload):
        if event == "server_ready":
            self._server_address = f"{payload['host']}:{payload['port']}"
```

MBToolkit v3.5/mb_tcp_ui.py — lines 481-509

```
481             self.slave_status_var.set(self._server_status_text())
482         elif event == "server_client":
483             if payload.get("connected"):
484                 self._server_clients[payload["peer"]] = True
485             else:
486                 self._server_clients.pop(payload["peer"], None)
487             self._slave_render_stats()
488         elif event == "server_request":
489             self._slave_apply_request(payload)
490         elif event == "server_stopped":
491             self._slave_finish_session(payload)
492         elif event == "server_interfaces":
493             self._server_interfaces = list(payload or [])
494             if not self.slave_active:
495                 self._server_fill_interfaces()
496 
497     def slave_apply_language(self):
498         super().slave_apply_language()
499         if hasattr(self, "server_bind_combo"):
500             self._server_fill_interfaces()
501         if self.slave_active and self._server_address:
502             self.slave_status_var.set(self._server_status_text())
503 
504     def slave_shutdown(self):
505         self._slave_generators = {}
506         engine = getattr(self, "slave_engine", None)
507         if engine is not None:
508             engine.stop(join_timeout=0.8)
509         self.slave_engine = None
```

MBToolkit v3.5/mb_i18n.py

729 source lines.

MBToolkit v3.5/mb_i18n.py — lines 1-40

```python
#!/usr/bin/env python3
"""Runtime language support for MBToolkit.

Portuguese (Portugal) is the canonical UI language.  English translations are
kept here so switching language never mutates protocol data or application
logic.
"""

from mb_config import DISCOVERY_IDLE_CHECK_SECONDS, SNIFFER_SPLIT_FRAME_HOLD_MS

LANGUAGE_PORTUGUESE = "Português"
LANGUAGE_ENGLISH = "English"
LANGUAGE_CHOICES = (LANGUAGE_PORTUGUESE, LANGUAGE_ENGLISH)

# Static UI text. Technical terms that are already English are intentionally
# left out: translate_text() returns unknown strings unchanged.
PT_TO_EN = {
    "Linguagem": "Language",
    "Modo escuro": "Dark mode",
    "Configuração": "Configuration",
    "Modo físico": "Physical mode",
    "RS232 dual RX: COM A escuta uma direção; COM B escuta a direção oposta.": "RS232 dual RX: COM A monitors one direction; COM B monitors the opposite direction.",
    "RS232 single RX: uma COM escuta apenas uma direção da ligação RS232.": "RS232 single RX: one COM port monitors only one direction of the RS232 link.",
    "RS485 2-wire: uma COM consegue observar pedidos e respostas no mesmo par diferencial.": "RS485 2-wire: one COM port can monitor requests and responses on the same differential pair.",
    "Estado": "Status",
    "Visualização": "Display",
    "Gap manual (ms)": "Manual gap (ms)",
    "Estado COM": "COM status",
    "Controlos": "Controls",
    "Iniciar Captura": "Start Capture",
    "Parar": "Stop",
    "Reiniciar COM": "Restart COM",
    "Limpar": "Clear",
    "Abrir pasta de logs": "Open logs folder",
    "Simulação": "Simulation",
    "Separar transações": "Separate transactions",
    "Realçar resultados": "Highlight results",
    "Estatísticas da sessão": "Session statistics",
    "Tráfego": "Traffic",
    "Ajuda / Ligações": "Help / Connections",
```

MBToolkit v3.5/mb_i18n.py — lines 41-80

```
    "Filtros": "Filters",
    "Todos": "All",
    "Pesquisa": "Search",
    "Limpar filtros": "Clear filters",
    "Exportar CSV": "Export CSV",
    "Hora ↑": "Time ↑",
    "Hora ↓": "Time ↓",
    "Tipo": "Type",
    "Detalhes": "Details",
    "Copiar Raw Hex": "Copy Raw Hex",
    "Copiar frame descodificado": "Copy decoded frame",
    "Filtrar por este Slave": "Filter by this Slave",
    "Filtrar por este FC": "Filter by this FC",
    "Mostrar apenas esta transação": "Show only this transaction",
    "Limpar filtro de transação": "Clear transaction filter",
    "Tempos de resposta": "Response times",
    "Qualidade": "Quality",
    "Pedidos": "Requests",
    "Pendentes": "Pending",
    "Slave (Filtro) ▾": "Slave (Filter) ▾",
    "Slave (Nenhum) ▾": "Slave (None) ▾",
    "✓ Sucesso": "✓ Success",
    "⚠ Resposta lenta": "⚠ Slow response",
    "● Parado": "● Stopped",
    "● A reiniciar COM": "● Restarting COM",
    "● A capturar": "● Capturing",
    "● A parar": "● Stopping",
    "● A simular": "● Simulating",
    "● Simulação concluída": "● Simulation complete",
    "● Erro na simulação": "● Simulation error",
    "Parado": "Stopped",
    "A iniciar pesquisa…": "Starting scan…",
    "A parar…": "Stopping…",
    "Erro ao iniciar pesquisa": "Error starting scan",
    "Respostas": "Responses",
    "Taxa de pedidos": "Request rate",
    "Slaves ativos": "Active Slaves",
    "Bytes observados": "Observed bytes",
    "Utilização do bus ~": "Bus utilization ~",
    "Média": "Average",
```

MBToolkit v3.5/mb_i18n.py — lines 81-120

```
    "Mínimo": "Minimum",
    "Máximo": "Maximum",
    "Slave mais lento": "Slowest Slave",
    "Erros de CRC": "CRC errors",
    "Bytes não validados": "Unvalidated bytes",
    "Taxa de timeout": "Timeout rate",
    "Exceções": "Exceptions",
    "Exceções / Slave": "Exceptions / Slave",
    "Seleciona uma linha no Tráfego para ver a descodificação estruturada do frame.":
        "Select a row in Traffic to view the structured frame decoding.",
    "Recolher": "Collapse",
    "Expandir": "Expand",
    "Resultado": "Result",
    "Dados": "Data",
    "Sucesso": "Success",
    "Resposta lenta": "Slow response",
    "Pendente / sem resultado": "Pending / no result",
    "Sem classificação": "Unclassified",
    "Verde": "Green",
    "Amarelo": "Yellow",
    "Vermelho": "Red",
    "Laranja": "Orange",
    "Roxo": "Purple",
    "Neutro": "Neutral",
    "Desconhecida/outra": "Unknown/other",
    "Não Utilizada": "Not Used",
    "Não selecionada": "Not selected",
    "Aberta": "Open",
    "Detetada": "Detected",
    "Não detetada": "Not detected",
    "Configuração inválida": "Invalid configuration",
    "pyserial em falta": "pyserial missing",
    "Porta COM": "COM port",
    "A porta série não aceitou bytes para transmissão.": "The serial port did not accept bytes for transmission.",
    "Erro ao abrir porta": "Error opening port",
    "Erro de captura": "Capture error",
    "Erro na simulação": "Simulation error",
    "Erro interno": "Internal error",
    "Abrir pasta": "Open folder",
    "Pesquisa ativa — transmite no barramento": "Active scan — transmits on the bus",
```

MBToolkit v3.5/mb_i18n.py — lines 121-160

```
    "Esta ferramenta atua temporariamente como master Modbus RTU e envia apenas pedidos de leitura. Não a utilizes com outro master ativo no mesmo barramento.": "This tool temporarily acts as a Modbus RTU master and sends read requests only. Do not use it while another master is active on the same bus.",
    "Confirmo que não existe outro master ativo no barramento.": "I confirm that no other master is active on the bus.",
    "FC03 é o modo rápido. Uma resposta normal OU uma Modbus Exception com CRC válido conta como slave encontrado. Ativa FC04 fallback apenas para dispositivos que possam ignorar FC03 sem devolver Exception. Device Identification envia FC43/14 apenas aos slaves encontrados e tenta ler VendorName, ProductCode e Revision. Tentativas repete cada pedido sem resposta. Antes de transmitir, a pesquisa escuta o barramento durante 1 s e não arranca se detetar tráfego.": "FC03 is the fast mode. A normal response OR a CRC-valid Modbus Exception counts as a found Slave. Enable FC04 fallback only for devices that may ignore FC03 without returning an Exception. Device Identification sends FC43/14 only to found Slaves and attempts to read VendorName, ProductCode and Revision. Attempts repeats every unanswered request. Before transmitting, the scan listens to the bus for 1 s and does not start if traffic is detected.",
    "Configuração da pesquisa": "Scan configuration",
    "Porta COM": "COM port",
    "A porta série não aceitou bytes para transmissão.": "The serial port did not accept bytes for transmission.",
    "Data bits: 8 (fixo)": "Data bits: 8 (fixed)",
    "Slave IDs / velocidade": "Slave IDs / speed",
    "De": "From",
    "até": "to",
    "Timeout mín.": "Min. timeout",
    "Iniciar pesquisa": "Start scan",
    "Limpar resultados": "Clear results",
    "Slaves encontrados": "Slaves found",
    "Encontrados: 0": "Found: 0",
    "Resultado": "Result",
    "Pesquisa ativa": "Active scan",
    "Erro ao iniciar pesquisa": "Error starting scan",
    "Erro na Bus Slave Finder": "Bus Slave Finder error",
    "Erro na pesquisa": "Scan error",
    "Nenhum": "None",
    "Nenhum slave detetado": "No Slave detected",
    "Crescente — antigo → recente": "Ascending — oldest → newest",
    "Decrescente — recente → antigo": "Descending — newest → oldest",
    "Todos os ficheiros": "All files",
    "Exportar sessão para CSV": "Export session to CSV",
    # Messages with live values (templates used with SnifferApp.trf()).
    'Seleciona a COM A.': 'Select COM A.',
    'Seleciona também a COM B.': 'Select COM B as well.',
    'COM A e COM B têm de ser diferentes.': 'COM A and COM B must be different.',
    'pyserial não está instalado.': 'pyserial is not installed.',
    'Não foi possível reiniciar a porta COM:\n\n{error}': 'Could not restart the COM port:\n\n{error}',
    'Não foi possível reabrir a porta COM. A captura foi parada.\n\n{error}': 'Could not reopen the COM port. Capture was stopped.\n\n{error}',
    'Pending timeout inválido.': 'Invalid Pending timeout.',
    'Pending timeout deve estar entre {minimum:g} e {maximum:g} segundos.': 'Pending timeout must be between {minimum:g} and {maximum:g} seconds.',
    'Frame gap manual inválido.': 'Invalid manual Frame gap.',
    'pyserial não está instalado.\n\nO build_exe.bat inclui-o automaticamente no EXE.': 'pyserial is not installed.\n\nbuild_exe.bat includes it automatically in the EXE.',
    'Ocorreu um erro ao processar eventos da interface. O processamento continuará.\n\n{error}': 'An error occurred while processing interface events. Processing will continue.\n\n{error}',
    'Slave {slave} — média {average:.1f} ms': 'Slave {slave} — average {average:.1f} ms',
    'Não existem frames na sessão atual para exportar.': 'There are no frames in the current session to export.',
```

MBToolkit v3.5/mb_i18n.py — lines 161-200

```
    'Sessão exportada para:\n\n{path}': 'Session exported to:\n\n{path}',
    'Não foi possível guardar o CSV:\n\n{error}': 'Could not save the CSV:\n\n{error}',
    'Erro ao criar log': 'Error creating log',
    'Foi detetado tráfego, mas não foi possível criar o ficheiro de log em:\n\n{folder}\n\n{error}': 'Traffic was detected, but the log file could not be created at:\n\n{folder}\n\n{error}',
    'Os Slave IDs têm de estar entre 1 e 247.': 'Slave IDs must be between 1 and 247.',
    'O Slave ID inicial não pode ser superior ao final.': 'The starting Slave ID cannot be greater than the ending Slave ID.',
    'O timeout mínimo tem de estar entre {minimum} e {maximum} ms.': 'The minimum timeout must be between {minimum} and {maximum} ms.',
    'Tentativas tem de estar entre {minimum} e {maximum}.': 'Attempts must be between {minimum} and {maximum}.',
    'Tentativas': 'Attempts',
    'Máx. estimado: —': 'Estimated max.: —',
    ' (com {options})': ' (with {options})',
    '{configs} config. × {slaves} slaves — máx. estimado: {duration}': '{configs} config. × {slaves} slaves — estimated max.: {duration}',
    'Encontrados: {count}': 'Found: {count}',
    'pyserial não está instalado. O build_exe.bat inclui-o automaticamente no EXE.': 'pyserial is not installed. build_exe.bat includes it automatically in the EXE.',
    'Confirma primeiro que não existe outro master ativo no barramento.': 'First confirm that no other master is active on the bus.',
    'Seleciona uma porta COM.': 'Select a COM port.',
    'Seleciona pelo menos uma configuração série.': 'Select at least one serial configuration.',
    'Não foi possível iniciar a thread de pesquisa:\n\n{error}': 'Could not start the scan thread:\n\n{error}',
    'A configurar {baud} / {config}…': 'Configuring {baud} / {config}…',
    'Foi detetado tráfego em {port} antes de transmitir ({count} bytes: {raw}). Pode existir outro master ativo no barramento; a pesquisa não foi iniciada.': 'Traffic was detected on {port} before transmitting ({count} bytes: {raw}). Another master may be active on the bus; the scan was not started.',
    'A verificar se o barramento está livre…': 'Checking that the bus is idle…',
    'Não foi possível abrir {port} em {baud} / {config}: {error}': 'Could not open {port} at {baud} / {config}: {error}',
    'Falha ao preparar a receção em {context}: {error}': 'Failed to prepare reception at {context}: {error}',
    'Falha ao transmitir em {context}: {error}': 'Failed to transmit at {context}: {error}',
    'Falha ao receber em {context}: {error}': 'Failed to receive at {context}: {error}',
    'Falha em Device Identification em {context}: {error}': 'Device Identification failed at {context}: {error}',
    'Decorrido {elapsed} — restante ~{remaining}': 'Elapsed {elapsed} — remaining ~{remaining}',
    'Pesquisa parada — {found} encontrados — {elapsed}': 'Scan stopped — {found} found — {elapsed}',
    'Pesquisa concluída — {found} encontrados — {elapsed}': 'Scan complete — {found} found — {elapsed}',
    # Master (poll) tab — v3.2.
    'Ligação — o Master transmite no barramento': 'Connection — the Master transmits on the bus',
    'Timeout resposta (ms)': 'Response timeout (ms)',
    'Atraso entre pedidos (ms)': 'Delay between polls (ms)',
    'Ligar': 'Connect',
    'Desligar': 'Disconnect',
    'Desligado': 'Disconnected',
    '● Ligado': '● Connected',
    'A desligar…': 'Disconnecting…',
    'Nova janela': 'New window',
    'Abrir workspace': 'Open workspace',
```

MBToolkit v3.5/mb_i18n.py — lines 201-240

```
    'Guardar workspace': 'Save workspace',
    'Sem janelas. Usa "Nova janela" para começar a ler um Slave.': 'No windows. Use "New window" to start reading a Slave.',
    'Endereço': 'Address',
    'Qtd.': 'Qty',
    'Formato': 'Format',
    'Aplicar': 'Apply',
    'Pausar': 'Pause',
    'Retomar': 'Resume',
    'Pausada': 'Paused',
    'Ler 1x': 'Read once',
    'Duplicar': 'Duplicate',
    'Destacar': 'Pop out',
    'Ancorar': 'Dock',
    'Fechar': 'Close',
    'Nome': 'Name',
    'Valor': 'Value',
    'Máximo de {count} janelas.': 'Maximum of {count} windows.',
    'Janela inválida': 'Invalid window',
    'Janela inválida.': 'Invalid window.',
    'Liga primeiro o Master.': 'Connect the Master first.',
    'Escrita': 'Write',
    'FC02 e FC04 são só de leitura: abre uma janela FC01 ou FC03 para escrever.': 'FC02 and FC04 are read-only: open an FC01 or FC03 window to write.',
    'Escrever': 'Write value',
    'Slave (0 = broadcast)': 'Slave (0 = broadcast)',
    'Valor(es)': 'Value(s)',
    'Formato: {fmt}. Vários valores (FC15/FC16): separar com ;': 'Format: {fmt}. Several values (FC15/FC16): separate with ;',
    'Enviar': 'Send',
    'Cancelar': 'Cancel',
    'Nome do endereço {address}': 'Name of address {address}',
    'Guardar': 'Save',
    'Para a captura, a pesquisa ou o Slave em curso antes de ligar o Master.': 'Stop the running capture, scan or Slave before connecting the Master.',
    'Não foi possível abrir {port}: {error}': 'Could not open {port}: {error}',
    'Erro na porta série: {error}': 'Serial port error: {error}',
    'Escrita OK: {what}': 'Write OK: {what}',
    'Escrita broadcast enviada: {what}': 'Broadcast write sent: {what}',
    'Resposta inválida (CRC ou formato): {rx}': 'Invalid response (CRC or format): {rx}',
    'O ficheiro não é um workspace do MBToolkit.': 'The file is not an MBToolkit workspace.',
    '{count} janela(s) inválida(s) foram ignoradas.': '{count} invalid window(s) were skipped.',
    'O timeout de resposta tem de estar entre 10 e 10000 ms.': 'The response timeout must be between 10 and 10000 ms.',
    'O atraso entre pedidos tem de estar entre 0 e 10000 ms.': 'The delay between polls must be between 0 and 10000 ms.',
```

MBToolkit v3.5/mb_i18n.py — lines 241-280

```
    'Scan rate fora do intervalo.': 'Scan rate out of range.',
    'Valor vazio.': 'Empty value.',
    'Valor fora do formato selecionado.': 'Value does not fit the selected format.',
    'Uma coil só aceita 0 ou 1.': 'A coil only accepts 0 or 1.',
    'O Slave ID de leitura tem de estar entre 1 e 247.': 'The read Slave ID must be between 1 and 247.',
    'Função de leitura não suportada.': 'Unsupported read function.',
    'O endereço tem de estar entre 0 e 65535.': 'The address must be between 0 and 65535.',
    'Quantidade fora do limite desta função.': 'Quantity outside the limit of this function.',
    'O intervalo de endereços ultrapassa 65535.': 'The address range goes beyond 65535.',
    'O Slave ID de escrita tem de estar entre 0 (broadcast) e 247.': 'The write Slave ID must be between 0 (broadcast) and 247.',
    'FC05 escreve exatamente uma coil.': 'FC05 writes exactly one coil.',
    'FC06 escreve exatamente um registo.': 'FC06 writes exactly one register.',
    'Função de escrita não suportada.': 'Unsupported write function.',
    'FC05/FC06 escrevem um único valor; usa FC15/FC16 para vários.': 'FC05/FC06 write a single value; use FC15/FC16 for several.',
    # Master windows and workspaces — v3.2.
    'Janela {number}': 'Window {number}',
    'Renomear': 'Rename',
    'Renomear janela': 'Rename window',
    'Nome da janela (vazio = nome por defeito)': 'Window name (empty = default name)',
    'Workspace atual: {name}': 'Current workspace: {name}',
    'Guardar workspace…': 'Save workspace…',
    'Apagar workspace': 'Delete workspace',
    '(nenhum workspace guardado)': '(no saved workspace)',
    'Abrir pasta dos workspaces': 'Open workspaces folder',
    'Nome do workspace': 'Workspace name',
    'Já existe um workspace chamado "{name}". Substituir?': 'A workspace named "{name}" already exists. Replace it?',
    'Apagar o workspace "{name}"?': 'Delete workspace "{name}"?',
    'O nome do workspace não pode ficar vazio.': 'The workspace name cannot be empty.',
    'O nome do workspace não pode ter os caracteres < > : " / \\ | ? *.': 'The workspace name cannot contain the characters < > : " / \\ | ? *.',
    'Esse nome está reservado pelo Windows.': 'That name is reserved by Windows.',
    # Master per-address formats, row menu, workspace changes — v3.2.
    'Duplo clique num valor para escrever, num nome para o editar. Clique direito: formato, copiar e base.': 'Double-click a value to write it, a name to edit it. Right-click: f
ormat, copy and base.',
    'Workspace atual: {name} (alterado)': 'Current workspace: {name} (modified)',
    'Seleciona pelo menos um endereço.': 'Select at least one address.',
    'Seleciona registos seguidos para este formato.': 'Select consecutive registers for this format.',
    'Este formato ocupa {size} registos: seleciona {size} registos seguidos (ou um múltiplo de {size}).': 'This format uses {size} registers: select {size} consecutive registers
 (or a multiple of {size}).',
    'Este endereço não tem um valor completo no formato atual.': 'This address does not hold a complete value in the current format.',
    '32 bits (2 registos)': '32-bit (2 registers)',
    '64 bits (4 registos)': '64-bit (4 registers)',
    'Repor formato da janela': 'Reset to window format',
```

MBToolkit v3.5/mb_i18n.py — lines 281-320

```
    'Copiar': 'Copy',
    '1 – numeração dos manuais': '1 – manual numbering',
    'Base de endereços': 'Address base',
    'Guardar alterações ao workspace "{name}"?': 'Save changes to workspace "{name}"?',
    # Master scale, open in Master, reconnection – v3.2.
    'Escala…': 'Scale…',
    'Remover escala': 'Remove scale',
    'Escala': 'Scale',
    'Escala de {count} valor(es) selecionado(s):': 'Scale of {count} selected value(s):',
    'Bruto mín.': 'Raw min.',
    'Bruto máx.': 'Raw max.',
    'Eng. mín.': 'Eng. min.',
    'Eng. máx.': 'Eng. max.',
    'Casas decimais': 'Decimals',
    'Unidade': 'Unit',
    'Valor = Eng. mín. + (bruto − Bruto mín.) × (Eng. máx. − Eng. mín.) / (Bruto máx. − Bruto mín.)': 'Value = Eng. min. + (raw − Raw min.) × (Eng. max. − Eng. min.) / (Raw max. − Raw min.)',
    'valores em unidades da escala {unit}': 'values in scale units {unit}',
    'A escala tem de ter números válidos.': 'The scale needs valid numbers.',
    'O bruto mínimo e o bruto máximo têm de ser diferentes.': 'Raw min. and raw max. must be different.',
    'As casas decimais têm de estar entre 0 e 6.': 'Decimals must be between 0 and 6.',
    'Esta escala não permite escrever: os valores de engenharia são iguais.': 'This scale does not allow writing: the engineering values are equal.',
    'Abrir no Master': 'Open in Master',
    'Esta trama não tem um pedido de leitura FC01–FC04 com endereço conhecido.': 'This frame has no FC01–FC04 read request with a known address.',
    'A religar a {port}… (porta falhou: {error})': 'Reconnecting to {port}… (port failed: {error})',
    # Slave (simulator) tab – v3.4.
    'Ligação – o Slave responde aos pedidos de um master real': 'Connection – the Slave answers the requests of a real master',
    'Confirmo que nenhum equipamento no barramento usa os Slave IDs simulados.': 'I confirm that no device on the bus uses the simulated Slave IDs.',
    'Confirma primeiro que nenhum equipamento no barramento usa os Slave IDs simulados.': 'First confirm that no device on the bus uses the simulated Slave IDs.',
    'Para a captura, a pesquisa ou o Master em curso antes de ligar o Slave.': 'Stop the running capture, scan or Master before connecting the Slave.',
    'Cria primeiro uma janela com o Slave a simular.': 'First create a window with the Slave to simulate.',
    'Sem janelas. Usa "Nova janela" para simular um Slave.': 'No windows. Use "New window" to simulate a Slave.',
    'Duplo clique num valor para o alterar, num nome para o editar. Clique direito: formato, escala, copiar e base.': 'Double-click a value to change it, a name to edit it. Right-click: format, scale, copy and base.',
    'Pedidos: {requests}   Respostas: {responses}   Exceções: {exceptions}   Broadcast: {broadcasts}   Para outros IDs: {others}': 'Requests: {requests}   Responses: {responses}   Exceptions: {exceptions}   Broadcast: {broadcasts}   For other IDs: {others}',
    'Pedidos: {requests}   Exceções: {exceptions}': 'Requests: {requests}   Exceptions: {exceptions}',
    'Último: {request}': 'Last: {request}',
    '● A responder': '● Answering',
    'A abrir a porta…': 'Opening the port…',
    'Tabela': 'Table',
    'Tabela inválida.': 'Invalid table.',
    'O Slave ID tem de estar entre 1 e 247.': 'The Slave ID must be between 1 and 247.',
```

MBToolkit v3.5/mb_i18n.py — lines 321-360

```
    'A quantidade tem de estar entre 1 e 2000.': 'The quantity must be between 1 and 2000.',
    'Valor do endereço {address} ({fmt})': 'Value of address {address} ({fmt})',
    'Guardar alterações ao workspace do Slave "{name}"?': 'Save changes to the Slave workspace "{name}"?',
    'Falhas…': 'Faults…',
    'Falhas do Slave {slave}': 'Slave {slave} faults',
    'Simulação de falhas do Slave {slave} (aplica-se a todas as janelas deste ID).': 'Fault simulation for Slave {slave} (applies to every window of this ID).',
    'Atraso na resposta (ms)': 'Response delay (ms)',
    'Não responder': 'Do not respond',
    'Responder com Exception': 'Respond with Exception',
    'Nenhuma': 'None',
    'CRC errado nas respostas': 'Wrong CRC in responses',
    'Probabilidade (%)': 'Probability (%)',
    'A probabilidade decide, em cada pedido, se a falha acontece.': 'The probability decides, for each request, whether the fault happens.',
    'Sem falhas': 'No faults',
    '⚠ Falha simulada: {fault}': '⚠ Simulated fault: {fault}',
    'sem resposta': 'no response',
    'CRC errado': 'wrong CRC',
    'atraso {ms} ms': 'delay {ms} ms',
    'Falha inválida.': 'Invalid fault.',
    'A falha tem de ter números válidos.': 'The fault needs valid numbers.',
    'O atraso tem de estar entre 0 e 10000 ms.': 'The delay must be between 0 and 10000 ms.',
    'A probabilidade tem de estar entre 1 e 100 %.': 'The probability must be between 1 and 100 %.',
    'Código de exceção não suportado.': 'Unsupported exception code.',
    # Modbus TCP Client and Server — v3.5.
    'Sem ligação: {error}': 'No connection: {error}',
    'Guardar alterações ao workspace do Client "{name}"?': 'Save changes to the Client workspace "{name}"?',
    'Guardar alterações ao workspace do Server "{name}"?': 'Save changes to the Server workspace "{name}"?',
    'O Unit ID tem de estar entre 0 e 255.': 'The Unit ID must be between 0 and 255.',
    'Liga primeiro o Client.': 'Connect the Client first.',
    'Abrir no Server': 'Open in Server',
    'Falhas do Unit ID {slave}': 'Unit ID {slave} faults',
    'Simulação de falhas do Unit ID {slave} (aplica-se a todas as janelas deste ID).': 'Fault simulation for Unit ID {slave} (applies to every window of this ID).',
    'MBAP inválido': 'invalid MBAP',
    'Cabeçalho MBAP inválido nas respostas': 'Invalid MBAP header in responses',
    'Ligação — o Client liga-se a equipamentos Modbus TCP': 'Connection — the Client connects to Modbus TCP devices',
    'Ligação — o Server responde a clientes Modbus TCP': 'Connection — the Server answers Modbus TCP clients',
    '● A responder em {address}': '● Answering on {address}',
    'A ligar…': 'Connecting…',
    'Clientes ligados: {clients}   Pedidos: {requests}   Respostas: {responses}   Exceções: {exceptions}': 'Connected clients: {clients}   Requests: {requests}   Responses: {responses}   Exceptions: {exceptions}',
    'A iniciar o Server…': 'Starting the Server…',
```

MBToolkit v3.5/mb_i18n.py — lines 361-400

```
    'O IP, a porta e o Unit ID de cada equipamento ficam em cada janela.': 'Each window holds the IP, port and Unit ID of its device.',
    'Porta': 'Port',
    'Escutar em': 'Listen on',
    'Porta TCP': 'TCP port',
    'Confirmo que o Server pode ficar acessível na rede escolhida.': 'I confirm that the Server may be reachable on the chosen network.',
    'Só este PC (127.0.0.1)': 'This PC only (127.0.0.1)',
    'Todas as interfaces (0.0.0.0)': 'All interfaces (0.0.0.0)',
    'Escolheste uma interface de rede: confirma primeiro que o Server pode ficar acessível nessa rede.': 'You chose a network interface: first confirm that the Server may be reachable on that network.',
    'Não foi possível escutar em {host}:{port}: {error}': 'Could not listen on {host}:{port}: {error}',
    '{ip} (não encontrado)': '{ip} (not found)',
    'Indica um endereço IP ou nome válido.': 'Enter a valid IP address or name.',
    'A porta TCP tem de estar entre 1 e 65535.': 'The TCP port must be between 1 and 65535.',
    # Slave automatic values, Open in Slave — v3.4.
    'Valor automático': 'Automatic value',
    'Valor automático…': 'Automatic value…',
    'Parar valor automático': 'Stop automatic value',
    'Valor automático de {count} valor(es) selecionado(s):': 'Automatic value of {count} selected value(s):',
    'Tipo': 'Type',
    'Contador': 'Counter',
    'Rampa': 'Ramp',
    'Senoide': 'Sine',
    'Aleatório': 'Random',
    'Alternar 0/1': 'Toggle 0/1',
    'Mínimo': 'Minimum',
    'Máximo': 'Maximum',
    'Passo': 'Step',
    'Período (s)': 'Period (s)',
    'Soma o passo em cada período; depois do máximo volta ao mínimo.': 'Adds the step every period; after the maximum it returns to the minimum.',
    'Sobe do mínimo ao máximo durante o período e recomeça.': 'Rises from the minimum to the maximum over the period and starts again.',
    'Oscila entre o mínimo e o máximo; o período é uma volta completa.': 'Swings between the minimum and the maximum; the period is one full cycle.',
    'Um valor ao acaso entre o mínimo e o máximo em cada período.': 'A random value between the minimum and the maximum every period.',
    'Alterna entre 0 e 1 em cada período.': 'Toggles between 0 and 1 every period.',
    '0 ou 1 ao acaso em cada período.': 'A random 0 or 1 every period.',
    'Valor automático inválido.': 'Invalid automatic value.',
    'Tipo de valor automático inválido.': 'Invalid automatic value type.',
    'O período tem de ser um número.': 'The period must be a number.',
    'O período tem de estar entre 0,1 e 3600 s.': 'The period must be between 0.1 and 3600 s.',
    'O mínimo e o máximo têm de ser números.': 'The minimum and maximum must be numbers.',
    'O mínimo tem de ser menor que o máximo.': 'The minimum must be lower than the maximum.',
    'O passo tem de ser um número.': 'The step must be a number.',
```

MBToolkit v3.5/mb_i18n.py — lines 401-440

```
401     'O passo tem de ser maior que 0.': 'The step must be greater than 0.',
402     'Abrir no Slave': 'Open in Slave',
403     'Esta trama não tem um pedido FC01–FC06/FC15/FC16 com endereço conhecido.': 'This frame has no FC01–FC06/FC15/FC16 request with a known address.',
404 }
405
406 EN_TO_PT = {english: portuguese for portuguese, english in PT_TO_EN.items()}
407 # "None" is also a canonical technical serial-parity value in the Portuguese UI.
408 # Do not reverse-translate it to "Nenhum" when restoring Portuguese; menu entries
409 # that need "Nenhum" are rendered from their Portuguese canonical source text.
410 EN_TO_PT.pop("None", None)
411
412 EXCEPTION_DESCRIPTION_EN = {
413     0x01: "The requested Modbus function is not supported or permitted by the device.",
414     0x02: "The requested address or address range is not valid for this device.",
415     0x03: "A value, quantity or request field is invalid for this function.",
416     0x04: "The device encountered an internal error while attempting to execute the request.",
417     0x05: "The request was accepted but requires more time to be processed.",
418     0x06: "The device is busy and cannot process the request at this time.",
419     0x08: "A parity error was detected while accessing the device memory.",
420     0x0A: "The gateway could not establish a path to the target device.",
421     0x0B: "The gateway did not receive a response from the target device.",
422 }
423
424
425 def normalize_language(value):
426     return value if value in LANGUAGE_CHOICES else LANGUAGE_PORTUGUESE
427
428
429 def canonical_ui_text(text):
430     """Return the canonical PT-PT form of a known static UI string."""
431     value = str(text)
432     return EN_TO_PT.get(value, value)
433
434
435 def translate_text(text, language):
436     """Translate a known static string; leave technical/unknown text intact."""
437     canonical = canonical_ui_text(text)
438     if normalize_language(language) == LANGUAGE_ENGLISH:
439         return PT_TO_EN.get(canonical, canonical)
440     return canonical
```

MBToolkit v3.5/mb_i18n.py — lines 441-480

```
441
442
443 # Human-readable fragments embedded in protocol Details / Inspector data.
444 DETAIL_FRAGMENTS_PT_TO_EN = (
445     ("Registos (PDU Address=valor):", "Registers (PDU Address=value):"),
446     ("Bits (PDU Address=valor):", "Bits (PDU Address=value):"),
447     ("Registos=", "Registers="),
448 )
449
450
451 def translate_details(text, language):
452     """Translate only human-readable fragments embedded in protocol details."""
453     value = str(text or "")
454     if normalize_language(language) == LANGUAGE_ENGLISH:
455         for portuguese, english in DETAIL_FRAGMENTS_PT_TO_EN:
456             value = value.replace(portuguese, english)
457     else:
458         for portuguese, english in DETAIL_FRAGMENTS_PT_TO_EN:
459             value = value.replace(english, portuguese)
460     return value
461
462
463 def exception_description(code, portuguese_text, language):
464     if normalize_language(language) == LANGUAGE_ENGLISH:
465         return EXCEPTION_DESCRIPTION_EN.get(code, portuguese_text)
466     return portuguese_text
467
468
469 # English Help content. Tags match the tags configured by mb_gui.py.
470 def english_help_blocks(max_ui_frames_text, max_raw_lines_text,
471                         min_pending, max_pending, debug_sim=False):
472     blocks = [
473         ("title", "MBToolkit — Help and connections\n"),
474         (None, "MBToolkit is designed for Modbus RTU communications diagnostics. The Sniffer tab is passive: it opens the COM port(s) for reception and does not transmit request
    s. Bus Slave Finder is an active scan tool and must only be used when no other master is active on the same bus.\n"),
475         ("heading", "Quick start — Sniffer\n"),
476         ("bullet", "1. Select the correct physical mode: RS485 2-wire, RS232 single RX or RS232 dual RX.\n"),
477         ("bullet", "2. Select the COM port(s) and serial parameters used by the communication being observed.\n"),
478         ("bullet", "3. Keep Frame gap set to Auto unless you have a specific reason to change it.\n"),
479         ("bullet", "4. Click “Start Capture” and confirm activity through BUS ● RX.\n"),
480         ("bullet", "5. Use Traffic for quick diagnostics and Raw Hex / Log to confirm the actual bytes.\n"),
```

MBToolkit v3.5/mb_i18n.py — lines 481-520

```
481         ("bullet", "6. Select a frame to inspect its detailed analysis in Frame Inspector.\n"),
482 
483         ("heading", "1. RS485 2-wire\n"),
484         (None, "In this mode, one COM port observes requests and responses on the same differential pair. The sniffer adapter must be connected in parallel and must not act as a
     master.\n"),
485         ("subheading", "Typical connection:\n"),
486         ("mono", "        Device A              Device B\n            D+ ---------------- D+\n            D- ---------------- D-\n              \\              /\n
       \\             /\n                 D+          D-\n                 USB-RS485 sniffer\n\n"),
487         ("bullet", "• Connect sniffer D+ in parallel with bus D+.\n"),
488         ("bullet", "• Connect sniffer D− in parallel with bus D−.\n"),
489         ("bullet", "• If a C/COM/0 V reference is available, connect it only when you know it is the correct RS485 interface reference.\n"),
490         ("warning", "• Do not add a third 120 Ω termination resistor if the bus is already terminated at both ends.\n"),
491         ("bullet", "• If the USB-RS485 adapter has selectable termination or bias, leave them disabled during passive capture unless they are intentionally required by the insta
    llation.\n"),
492         ("bullet", "• A/B naming is not universal: different manufacturers may use opposite polarities. Prefer D+/D− and the equipment documentation.\n"),
493 
494         ("heading", "2. RS232 single RX\n"),
495         (None, "Observes only one RS232 direction. RS232 is full-duplex: each direction uses an independent TX line, so one RX input cannot observe both directions simultaneousl
    y.\n"),
496         ("mono", "        Device A                    Device B\n            TX ---------------------------> RX\n              \\\n              \\----> USB-RS232 sniffer R
    X (COM A)\n\n            GND -------------------------- GND\n            \\--------------------------> sniffer GND\n\n        Sniffer TX: DO NOT CONNECT\n\n"),
497         ("bullet", "• Connect sniffer RX to the TX line you want to observe.\n"),
498         ("bullet", "• Connect the common GND as well.\n"),
499         ("warning", "• Leave the TX pin of the adapter used as a sniffer physically disconnected.\n"),
500 
501         ("heading", "3. RS232 dual RX\n"),
502         (None, "Allows both RS232 directions to be observed at the same time. Two independent RX channels are required, normally two USB-RS232 adapters.\n"),
503         ("mono", "        Device A                    Device B\n            TX ---------------------------> RX\n              \\----> sniffer RX COM A   (A → B)\n\n
        RX <--------------------------- TX\n                 \\----> sniffer RX COM B   (B → A)\n\n            GND -------------------------- GND\n
           \\--------------------------- COM A GND\n               \\--------------------------- COM B GND\n\n        TX on both sniffers: DO NOT CONNECT\n\n"),
504         ("bullet", "• COM A is shown as A→B and COM B as B→A.\n"),
505         ("bullet", "• Both COM ports must use the same serial parameters as the observed link.\n"),
506 
507         ("heading", "RS232 is not TTL\n"),
508         ("warning", "Do not connect RS232 signals directly to 3.3 V or 5 V USB-TTL/UART interfaces. Use a USB-RS232 adapter with a transceiver suitable for RS232 voltage levels.
    \n"),
509 
510         ("heading", "Serial communication parameters\n"),
511         (None, "Baud rate, Data bits, Parity and Stop bits must match the actual communication. Incorrect parameters may produce apparently random bytes, frames with invalid CRC
    , or no recognizable frames at all.\n"),
512         ("bullet", "• Baud rate — speed in bit/s, for example 9600 or 19200.\n"),
513         ("bullet", "• Data bits — normally 8 for Modbus RTU.\n"),
514         ("bullet", "• Parity — None, Even, Odd, Mark or Space, according to the installation.\n"),
515         ("bullet", "• Stop bits — 1, 1.5 or 2, according to the observed configuration.\n"),
516 
517         ("heading", "Frame gap\n"),
518         (None, "In Auto mode, MBToolkit uses 3.5 character times up to and including 19200 bit/s. Above 19200 bit/s it uses the fixed 1.750 ms t3.5 recommended for Modbus RTU. M
    anual mode allows another millisecond value to be used when diagnosing equipment or drivers with particular behavior.\n"),
519         (None, "USB adapters and Windows may group bytes and introduce latency. MBToolkit timestamps are therefore useful for sequence diagnostics and relative timing, but they
    do not replace a logic analyzer when high-precision timing measurements are required.\n"),
520         (None, f"When a burst ends with the start of a known frame that is still incomplete, MBToolkit waits up to {SNIFFER_SPLIT_FRAME_HOLD_MS:g} ms more before closing it. A f
    rame split in two by the USB adapter therefore does not appear as RAW. Frames merged into one burst keep the arrival time of their own block of bytes.\n"),
```

MBToolkit v3.5/mb_i18n.py — lines 521-560

```
521
522         ("heading", "Pending timeout\n"),
523         (None, "Defines how long a CRC-valid REQUEST may remain waiting for a matchable RESPONSE/EXCEPTION. When it expires, the original REQUEST is marked as timeout and the Timeout/Timeout rate counters increase.\n"),
524         (None, f"Allowed range: {min_pending:g} to {max_pending:g} s. The value should be greater than the normal maximum response time of the installation to avoid false timeouts.\n"),
525         (None, "Broadcast requests (Slave 0) never have a response by definition: they are not kept pending, never count as a timeout and are excluded from the Timeout rate.\n"),
526
527         ("heading", "How to read the Traffic table\n"),
528         ("bullet", "• REQUEST — Modbus RTU request recognized by the parser.\n"),
529         ("bullet", "• RESPONSE — recognized Modbus RTU response that is not an EXCEPTION.\n"),
530         ("bullet", "• FC05/FC06/FC08/FC22 — the request and the response have the same format. The frame is a RESPONSE when a pending request from the same Slave/FC exists (for FC05/06/22 the bytes must be identical; in RS232 dual RX it must come from the opposite channel). Otherwise it is a REQUEST.\n"),
531         ("bullet", "• FC01/FC02 responses show the bits in address order, grouped per byte. In Frame Inspector, matched read responses (FC01–FC04) show the PDU Address of every value.\n"),
532         ("bullet", "• EXCEPTION — Modbus response with an exception Function Code (bit 7 set). Details shows the code and normalized description.\n"),
533         ("bullet", "• CRC OK — received CRC matches the calculated CRC.\n"),
534         ("bullet", "• CRC ERROR — the structure resembles a supported frame, but the received CRC does not match the calculated CRC.\n"),
535         ("bullet", "• RAW/UNSYNC — bytes discarded during resynchronization before a structurally recognized, CRC-valid frame is found later in the same burst.\n"),
536         ("bullet", "• RAW/UNPARSED — remaining bytes that could not be interpreted or resynchronized as a valid Modbus frame. This is the state normally shown for a noise block with no later valid frame in the same burst.\n"),
537         (None, "RAW/UNSYNC and RAW/UNPARSED are different conditions and both may appear in the Type column. Neither represents a valid Modbus response.\n"),
538
539         ("heading", "Traffic table columns\n"),
540         ("bullet", "• Time — timestamp for the start of the processed burst/frame.\n"),
541         ("bullet", "• Δt — interval since the previous observed frame.\n"),
542         ("bullet", "• Resp. — time between a REQUEST and its matched RESPONSE/EXCEPTION. Blank when no valid pair exists.\n"),
543         ("bullet", "• COM — capture channel: BUS in RS485, A→B/B→A in RS232 dual RX and SIM in development mode. It does not necessarily represent the Windows COM number.\n"),
544         ("bullet", "• Type — REQUEST, RESPONSE, EXCEPTION or a RAW state.\n"),
545         ("bullet", "• Slave — Modbus Unit/Slave ID when identifiable.\n"),
546         ("bullet", "• FC — hexadecimal Function Code observed in the frame.\n"),
547         ("bullet", "• Details — address, quantity, data or operation description.\n"),
548         ("bullet", "• CRC — OK, ERROR or ? when a complete Modbus frame is not available.\n"),
549
550         ("heading", "REQUEST/RESPONSE pairing\n"),
551         (None, "Modbus RTU has no Transaction ID. MBToolkit pairs frames by (Slave ID, Function Code) in FIFO order. This is appropriate for normal RTU traffic, but can become ambiguous when requests using the same Slave and FC overlap before their corresponding responses arrive.\n"),
552         (None, "An EXCEPTION uses its base Function Code for pairing. For example, response 0x83 is associated with a pending FC03 REQUEST when a compatible request exists.\n"),
553
554         ("heading", "PDU Address, 1-based and Qty\n"),
555         (None, "PDU Address is the address encoded in the Modbus request. 1-based is simply PDU Address + 1 and helps compare with manuals that number the first register/coil as 1. Qty is the requested quantity.\n"),
556         (None, "These fields belong to the REQUEST. A RESPONSE does not automatically repeat Address or Qty in the protocol, so MBToolkit does not invent those fields on the response row.\n"),
557
558         ("heading", "Modbus exception codes\n"),
559         ("bullet", "• 0x01 — Illegal Function: function not supported or not permitted.\n"),
560         ("bullet", "• 0x02 — Illegal Data Address: invalid address/address range.\n"),
```

MBToolkit v3.5/mb_i18n.py — lines 561-600

```
561         ("bullet", "• 0x03 — Illegal Data Value: invalid value, quantity or field.\n"),
562         ("bullet", "• 0x04 — Server Device Failure: device-side execution failure.\n"),
563         ("bullet", "• 0x05 — Acknowledge.\n"),
564         ("bullet", "• 0x06 — Server Device Busy.\n"),
565         ("bullet", "• 0x08 — Memory Parity Error.\n"),
566         ("bullet", "• 0x0A — Gateway Path Unavailable.\n"),
567         ("bullet", "• 0x0B — Gateway Target Device Failed to Respond.\n"),
568 
569         ("heading", "Advanced filters\n"),
570         (None, "The bar at the top of Traffic can combine several filters without changing the original session data:\n"),
571         ("bullet", "• Type — All, Requests, Responses, Exceptions, CRC errors, Timeouts or RAW.\n"),
572         ("bullet", "• FC (hex) — Function Code, with or without the 0x prefix.\n"),
573         ("bullet", "• Resp. > — only completed responses above the specified time in ms.\n"),
574         ("bullet", "• Search — searches Details, Raw Hex, Slave, FC, Type and COM/channel.\n"),
575         (None, "The Slave filter remains available in the column header and combines with the advanced filters. “Clear filters” resets all display filters. Filters do not change statistics or the capture log.\n"),
576 
577         ("heading", "Highlight results\n"),
578         (None, "This option is disabled by default to keep the table neutral. When enabled, results are differentiated by color: a completed normal transaction with a CRC-valid RESPONSE is green; CRC error, timeout, Exception, RAW and slow response retain their respective diagnostic colors.\n"),
579         (None, "In a successfully completed normal transaction, both the REQUEST and its RESPONSE are green. A still-pending REQUEST remains neutral. A slow response keeps the slow-response color even if valid so that the timing result is not hidden.\n"),
580 
581         ("heading", "Frame Inspector\n"),
582         (None, "Select a Traffic row to analyze the frame without manually interpreting every byte. Frame Inspector shows Slave, Type, Function, response time, Result, PDU Address, 1-based address, Qty, ByteCount, CRC, decoded Data and Raw Hex.\n"),
583         (None, "The CRC field shows both received and calculated values for a complete frame. Result uses the same diagnostic classification as Traffic, independently of whether Highlight results is enabled.\n"),
584 
585         ("heading", "Traffic context menu\n"),
586         (None, "Right-click a Traffic row to:\n"),
587         ("bullet", "• Copy Raw Hex.\n"),
588         ("bullet", "• Copy the decoded frame.\n"),
589         ("bullet", "• Filter by that frame's Slave.\n"),
590         ("bullet", "• Filter by that frame's Function Code.\n"),
591         ("bullet", "• Show only the matching REQUEST/RESPONSE transaction.\n"),
592         ("bullet", "• Clear the temporary transaction filter.\n"),
593 
594         ("heading", "Bus Health\n"),
595         (None, "Bus Health summarizes session quality and performance without replacing the frame table.\n"),
596         ("bullet", "• Requests / Responses — session counters.\n"),
597         ("bullet", "• Request rate — average number of observed requests per second.\n"),
598         ("bullet", "• Active Slaves — valid Modbus IDs observed in the session.\n"),
599         ("bullet", "• Observed bytes — sum of bytes delivered to the parser.\n"),
600         ("bullet", "• Bus utilization ~ — estimate based on observed bytes, baud rate, data bits, parity and stop bits. It is approximate and depends on the capture point.\n"),
```

MBToolkit v3.5/mb_i18n.py — lines 601-640

```
601         ("bullet", "• Average / Minimum / Maximum – paired response times.\n"),
602         ("bullet", "• P95 – 95% of measured responses were at or below this value.\n"),
603         ("bullet", "• Slowest Slave – Slave with the highest average response time among observed samples.\n"),
604         ("bullet", "• Unvalidated bytes – percentage of observed bytes belonging to invalid-CRC frames or RAW blocks. By itself this does not prove electrical noise; incorrect s
    erial parameters, incomplete capture or parser-unsupported functions can also produce RAW.\n"),
605         ("bullet", "• Timeout rate – percentage of REQUESTs that expired without a matched response.\n"),
606         ("bullet", "• Exceptions / Slave – distribution of Exception responses by Slave ID.\n"),
607 
608         ("heading", "BUS RX indicator\n"),
609         (None, "BUS ● RX pulses briefly when a batch of frames reaches the interface. At rest, BUS ○ is displayed. It is only a visual confirmation of received activity; it does
     not indicate electrical quality or guarantee that frames are valid.\n"),
610 
611         ("heading", "Raw Hex / Log\n"),
612         (None, "Shows captured bytes in hexadecimal with context information. Long lines wrap to the available width. The view follows the same display filters used in Traffic;
    the on-disk TXT file, when present, keeps the complete capture and is not truncated by filters.\n"),
613 
614         ("heading", "Export CSV\n"),
615         (None, "Export CSV saves the frames currently retained in the UI history using Excel-compatible UTF-8. It includes Time, Δt, Response time, Channel, Type, Slave, FC, Det
    ails, CRC, Raw, PDU Address, 1-based, Qty, ByteCount, Timeout and anomaly class.\n"),
616         (None, f"The interface retains approximately the latest {max_ui_frames_text} frames. In longer sessions, the CSV therefore represents the history still retained by the G
    UI, while the TXT file from a real capture remains the complete on-disk record.\n"),
617 
618         ("heading", "Capture logs\n"),
619         (None, "The TXT log is created only after real traffic arrives. Starting and stopping without receiving frames does not create an empty file.\n"),
620         (None, "Logs folder:\n"),
621         ("mono", "        <application folder>\\MBToolkit Logs\n"),
622         (None, "Filters, sorting, Highlight results and Frame Inspector are display-only features and do not change frames written to the log.\n"),
623 
624         ("heading", "Visible history limits\n"),
625         (None, f"To keep the application responsive, the interface retains approximately {max_ui_frames_text} frames and {max_raw_lines_text} Raw Hex lines. Pruning is performed
     in blocks. The capture TXT log, when active, is independent of this limit.\n"),
626 
627         ("heading", "COM status and ports\n"),
628         (None, "COM status distinguishes a port opened by MBToolkit, a port merely detected by Windows and a port that is no longer detected. Status is refreshed periodically.\n
    "),
629         (None, "Restart COM closes and reopens the port(s) used by Sniffer. This restarts the serial-port handle inside MBToolkit; it does not restart the Windows USB/COM driver
    .\n"),
630         (None, "The ↻ buttons beside the COM selectors refresh the available port enumeration in Sniffer and Bus Slave Finder.\n"),
631 
632         ("heading", "Bus Slave Finder – active scan\n"),
633         (None, "Bus Slave Finder transmits Modbus RTU requests, so it is not passive. Before starting, confirm that no other master is active on the same bus.\n"),
634         (None, "Select COM, baud rates, parities, stop bits and the Slave ID range. Data bits is fixed at 8. The scan tries FC03 Address 0 Qty 1; either a normal response or a C
    RC-valid Modbus Exception confirms that the Slave exists.\n"),
635         (None, "FC04 fallback is optional and increases the maximum number of attempts. The minimum timeout is adapted to baud rate. A write timeout means transmission through t
    he port/driver failed and must not be interpreted simply as “slave did not respond”.\n"),
636         (None, f"Attempts (1 to 3) repeats every unanswered request before moving on to the next Slave. Before the first transmission, the scan listens to the bus for {DISCOVERY
    _IDLE_CHECK_SECONDS:g} s; if any byte is received, it does not transmit and reports that another master may be active.\n"),
637         (None, "Device Identification (FC43/14) is optional. When enabled, it is sent only after a Slave has been found and attempts to read Basic Device Identification: VendorN
    ame, ProductCode and MajorMinorRevision. An Exception or no response to FC43/14 does not invalidate Slave discovery.\n"),
638         (None, "Finder and Sniffer share the global activity lock so the same application cannot actively scan and capture at the same time.\n"),
639 
640         ("heading", "Master – read and write as a Modbus RTU master\n"),
```

MBToolkit v3.5/mb_i18n.py — lines 641-680

```
641         (None, f"The Master tab transmits requests: use it only when no other master is active on the bus. When connecting, the application listens to the bus for {DISCOVERY_IDLE_CHECK_SECONDS:g} s and does not transmit if traffic is detected. While the Master is connected, Sniffer, Bus Slave Finder and Slave are locked.\n"),
642         (None, "Each window reads one data area of a Slave: Slave ID, function (FC01 Coils, FC02 Discrete Inputs, FC03 Holding Registers or FC04 Input Registers), address, quantity and scan rate. Changes take effect only after “Apply”, which is highlighted in blue while some fields are not applied yet. The table shows up to 15 rows; beyond that it scrolls.\n"),
643         (None, "All windows share the same COM port: Modbus RTU allows only one request at a time, so windows are served in turn according to their scan rate. Writes and “Read once” go ahead of cyclic reads. The “Delay between polls” (at least t3.5) is always left between two requests.\n"),
644         (None, "Formats: signed and unsigned 16-bit, hex and binary; signed and unsigned 32-bit and float, in byte orders ABCD (big-endian), CDAB (word swap), BADC (byte swap) and DCBA (little-endian); signed and unsigned 64-bit and double, in orders ABCDEFGH, GHEFCDAB, BADCFEHG and HGFEDCBA. 32 and 64-bit values use 2 and 4 registers and appear on the row of the first one.\n"),
645         (None, "The “Format” field is the window's default format. Right-clicking one or more addresses (drag with the mouse, or Ctrl/Shift, to select several) sets the format of those addresses only: 32-bit formats need 2 consecutive registers and 64-bit formats 4 (or a multiple, applied in groups). “Reset to window format” returns to the default. The same menu has “Copy” (also Ctrl+C), which copies addresses, names and values to paste into Excel, and “Address base”: 0 shows the PDU Address; 1 numbers from 1, as in many manuals.\n"),
646         (None, "When a read fails (Timeout, Exception or invalid response), the values turn grey until a correct read arrives; they also turn grey when disconnecting.\n"),
647         (None, "Scale: in the right-click menu, “Scale…” converts the raw value into engineering units through two points (Raw min./max. → Eng. min./max.), with decimals and a unit (for example 2000–10000 → −180–180 °). When writing, the value is typed in units and converted back to raw. “Remove scale” returns to the raw value. Scales are saved in the workspace.\n"),
648         (None, "“Open in Master”: in the Sniffer Traffic table (right-click an FC01–FC04 request or its matched response) and in the Bus Slave Finder results (right-click or double-click) it creates a Master window with the same Slave, function, address and quantity, and copies the serial parameters to the Master connection (when it is not connected).\n"),
649         (None, "If the COM port fails while connected (for example, the USB adapter was unplugged), the Master does not stop: it tries to reopen the port every 2 s and resumes reading when the port is back. To stop, click “Disconnect”.\n"),
650         (None, "Writing: double-click a value in an FC01 or FC03 window. Coils with FC05 or FC15, registers with FC06 or FC16; several values separated by “;” are written to consecutive addresses with FC15/FC16. Slave 0 sends a broadcast, with no response. After an accepted write, the window is read again.\n"),
651         (None, "Each window shows Tx, errors, timeouts, the last response time and the last error (Timeout, Exception or invalid response). In each window header, “Rename” changes the window name (also by double-clicking the name), “Pop out” opens it outside the tab (“Dock” or closing that window returns it) and “×” closes the window.\n"),
652         (None, "The “Workspace” menu saves the connection, every window, names and formats under a name of your choice, and lists the saved workspaces so they can be opened or deleted directly. The files are kept in %APPDATA%\\MBToolkit\\Workspaces\\Master, next to the preferences.\n"),
653         (None, "The last workspace opened or saved is reopened when the program starts, without connecting to the bus. While there are unsaved changes, “(modified)” appears beside its name; closing the program in that state asks whether to save. Windows never saved in a workspace close without asking, and closing every window leaves no workspace open.\n"),
654
655         ("heading", "Slave — Modbus RTU device simulator\n"),
656         (None, "The Slave tab makes the application behave as one or more Modbus RTU devices: it listens on the COM port and answers the requests of a real master (a PLC, a SCADA or another Modbus Poll). It is used to test the master without the device. It answers FC01–FC06, FC15 and FC16.\n"),
657         (None, "Each window is one table of a simulated Slave: Slave ID, table (Coils, Discrete Inputs, Holding Registers or Input Registers), start address and quantity (up to 2000). Several Slave IDs can exist, each with several windows. Only the addresses defined in the windows exist: a request outside them gets Exception 02 (Illegal Data Address), an unsupported function Exception 01 and an invalid quantity or value Exception 03, as on a real device. Requests for other Slave IDs are ignored.\n"),
658         (None, "Values start at 0. Double-clicking a value changes it (Coils and Discrete Inputs toggle between 0 and 1); changes apply at once, without going through the bus. When the master writes (FC05/06/15/16), the written registers are highlighted in yellow for 1.5 s. Windows over the same addresses show the same values. Broadcast (Slave 0) writes are executed on every simulated Slave that has those addresses, without a response.\n"),
659         (None, "Per-address formats, scale, names, Copy, Address base, mouse selection, Rename, Pop out and × work as in the Master.\n"),
660         (None, "Automatic values: in the right-click menu, “Automatic value…” makes the selected values change by themselves, to test trends and alarms in the SCADA. Registers: Counter (adds the step every period and returns to the minimum after the maximum), Ramp (rises from the minimum to the maximum over the period), Sine (swings between the minimum and the maximum; the period is one cycle) and Random (a random value every period). Coils and Discrete Inputs: Toggle 0/1 and Random. The minimum and maximum are in the units shown (format and scale of the value). Automatic values appear in blue, are saved in the workspace and overwrite what the master or the user writes to those addresses. “Stop automatic value” keeps the last value.\n"),
661         (None, "“Open in Slave”: in the Sniffer Traffic table (right-click an FC01–FC06, FC15 or FC16 request, or its matched response) it creates a Slave window with the same Slave ID, table, address and quantity; in the Master, right-clicking a window creates a Slave window with the same area, names, formats and scales. The serial parameters are copied to the Slave connection when it is not connected.\n"),
662         (None, "Fault simulation: “Faults…” sets faults for the window's Slave ID, which apply to every window of that ID: response delay (ms), do not respond, always respond with a chosen Exception (01, 02, 03, 04 or 06), wrong CRC in responses and the probability (%) of the fault happening on each request. An active fault appears in red in the header of that Slave's windows. “No faults” switches them off.\n"),
663         (None, "The connection bar shows the requests received, responses, exceptions, broadcasts and requests for other IDs; each window shows the requests and exceptions on its addresses and the last request. The RS485 adapter echo is ignored. Before answering, the Slave waits the t3.5 silence; Windows and the USB adapter usually add a few ms, so a master with a very short timeout may get timeouts.\n"),
```

```
aster with a very short timeout may get timeouts.\n"),
        (None, "Before connecting, confirm that no real device on the bus uses the same Slave IDs: if two devices answer the same request, the responses are corrupted. The Slave never runs at the same time as the Sniffer, the Bus Slave Finder or the Master. If the port fails, it tries to reopen it every 2 s, like the Master.\n"),
        (None, "The Slave “Workspace” menu saves the configuration only: connection, windows, names, formats, scales, automatic values and faults. Register values are not saved and return to 0 when a workspace is opened. The files are kept in %APPDATA%\\MBToolkit\\Workspaces\\Slave; the last Slave workspace is also reopened at start-up.\n"),

        ("heading", "Client and Server — Modbus TCP\n"),
        (None, "The Client and Server tabs do the same as the Master and the Slave, but over Modbus TCP (Ethernet or Wi-Fi network) instead of RS485/RS232. Windows, formats, scale, names, writing, faults, automatic values and workspaces work the same way. They do not use the serial port, so they can run at the same time as the Sniffer, the Finder, the Master and the Slave, and at the same time as each other.\n"),
        (None, "Client: each window holds the IP address and TCP port of the device (502 by default) and the Unit ID (0 to 255). Windows are served one at a time, as in the Master, with the response timeout and delay between polls of the connection bar. The connection to each device stays open and is reopened if it drops; a device that does not accept the connection shows “No connection”. Modbus TCP has no broadcast: Unit ID 0 is an ordinary Unit ID. In the right-click menu, “Open in Server” creates the same area in the Server tab.\n"),
        (None, "Server: “Listen on” chooses where the Server accepts connections: “This PC only (127.0.0.1)” (default; only programs on the same PC, no firewall prompt), one of the PC's network interfaces (with the IP and the adapter name; the ↻ button reads the list again) or “All interfaces (0.0.0.0)”. On a network interface you must confirm that the Server may be reachable on that network; the first time, Windows Firewall asks for permission. It accepts several clients at once and shows how many are connected.\n"),
        (None, "In the Server, the Unit ID plays the role of the Slave ID (1 to 247). A request for a Unit ID that is not simulated gets Exception 0B (Gateway Target Device Failed to Respond). In the fault simulation, the RTU “wrong CRC” becomes “Invalid MBAP header in responses”. If the port is already used by another program, a warning appears and another port can be chosen.\n"),
        (None, "Client and Server workspaces are kept in %APPDATA%\\MBToolkit\\Workspaces\\Client and \\Server, and the last one of each is reopened at start-up.\n"),

        ("heading", "Saved preferences\n"),
        (None, "User preferences are stored in %APPDATA%\\MBToolkit\\settings.json. They include Light/Dark, Language, last tab, serial parameters, Frame gap, Pending timeout, Auto-scroll, Separate transactions, Highlight results, Bus Slave Finder options and Master, Slave, Client and Server connection parameters. Windows are saved in workspaces, in the %APPDATA%\\MBToolkit\\Workspaces folder.\n"),
        (None, "COM ports are not saved because they are specific to each computer. The Bus Slave Finder safety confirmation is not saved either.\n"),
        (None, "Up to v3.2 the program was called MBSniffer. The first time MBToolkit starts, it copies the preferences and workspaces from %APPDATA%\\MBSniffer to %APPDATA%\\MBToolkit; the old folder is not deleted. New logs go to “MBToolkit Logs”; old ones stay in “MBSniffer Logs”.\n"),

        ("heading", "Diagnostic best practices\n"),
        ("bullet", "• Confirm wiring, polarity and serial parameters first.\n"),
```

MBToolkit v3.5/mb_i18n.py - lines 681-720

MBToolkit v3.5/mb_i18n.py — lines 681-720

```
        ("bullet", "• On RS485, keep the sniffer stub short and avoid changing the existing termination.\n"),
        ("bullet", "• Make sure no other program has the same COM port open.\n"),
        ("bullet", "• Treat CRC ERROR and RAW as indicators of framing, noise, incorrect parameters or incomplete capture.\n"),
        ("bullet", "• Treat Exceptions as valid protocol responses: they are not equivalent to CRC errors.\n"),
        ("bullet", "• Compare response times with normal equipment behavior, not only with a generic threshold.\n"),
        ("bullet", "• Keep the TXT log when you need later analysis of the complete capture.\n"),
    ]
    if debug_sim:
        blocks.extend([
            ("heading", "Simulation mode\n"),
            (None, "This development mode appears only when debug_sim = 1 in MBToolkit.py. It does not open serial ports and never creates TXT logs.\n"),
            (None, "Each run uses the same diagnostic scenario types: one normal response, one slow response, one Modbus Exception, one CRC error, one timeout and one RAW/UNPARSED block. This makes it possible to compare the same information directly with and without visual highlighting.\n"),
            (None, "Highlight results does not change Simulation data. When disabled, all rows use normal table colors; when enabled, the completed normal transaction is green and Exception, CRC error, timeout, slow response and RAW rows use their respective diagnostic colors.\n"),
            (None, "Frames in this scenario use the SIM channel so they are not confused with real physical capture.\n"),
        ])
    blocks.extend([
        ("heading", "Important limitation\n"),
        (None, "MBToolkit interprets bytes, CRC, Function Code, addresses, quantities, pairing and timing observed by the operating system. It does not directly measure electrical levels, ringing, reflections, bias, common-mode, rise/fall times or analog signal integrity. Those problems still require an oscilloscope or an analyzer suitable for the physical interface.\n"),
    ])
    return blocks


def translate_runtime_text(text, language):
    """Translate status strings that contain live values."""
    import re

    value = str(text or "")
    target_en = normalize_language(language) == LANGUAGE_ENGLISH

    # Normalize either language to canonical PT where practical, then render.
    patterns = [
        (r"^Encontrados: (\d+)$", r"Found: \1", target_en),
        (r"^Found: (\d+)$", r"Encontrados: \1", not target_en),
        (r"^Decorrido (.+) — restante ~(.+)$", r"Elapsed \1 — remaining ~\2", target_en),
        (r"^Elapsed (.+) — remaining ~(.+)$", r"Decorrido \1 — restante ~\2", not target_en),
        (r"^A configurar (.+)…$", r"Configuring \1…", target_en),
        (r"^Configuring (.+)…$", r"A configurar \1…", not target_en),
        (r"^Pesquisa parada — (\d+) encontrados — (.+)$", r"Scan stopped — \1 found — \2", target_en),
        (r"^Scan stopped — (\d+) found — (.+)$", r"Pesquisa parada — \1 encontrados — \2", not target_en),
        (r"^Pesquisa concluída — (\d+) encontrados — (.+)$", r"Scan complete — \1 found — \2", target_en),
```

MBToolkit v3.5/mb_i18n.py — lines 721-729

```
        (r"^Scan complete — (\d+) found — (.+)$", r"Pesquisa concluída — \1 encontrados — \2", not target_en),
        (r"^(\d+) config\. × (\d+) slaves — máx\. estimado: (.+)$", r"\1 config. × \2 slaves — estimated max.: \3", target_en),
        (r"^(\d+) config\. × (\d+) slaves — estimated max\.: (.+)$", r"\1 config. × \2 slaves — máx. estimado: \3", not target_en),
    ]
    for pattern, replacement, enabled in patterns:
        if enabled and re.match(pattern, value):
            return re.sub(pattern, replacement, value)

    return translate_text(value, language)
```

MBToolkit v3.5/mb_theme.py

692 source lines.

MBToolkit v3.5/mb_theme.py — lines 1-40

```
#!/usr/bin/env python3
"""MBToolkit fixed-geometry Light/Dark theme.

The application uses one ttk construction for its entire lifetime. Light/Dark
switching changes colours only; it never changes ttk theme engine, layouts,
padding, borders, margins, widget metrics or row heights.
"""

import tkinter as tk
from tkinter import ttk


BOOTSTRAP_ACTIVE = False
THEME_ENGINE = "clam"

LIGHT = {
    "root": "#f3f4f6",
    "panel": "#f3f4f6",
    "panel_alt": "#ffffff",
    "input": "#ffffff",
    "input_readonly": "#f5f6f7",
    "text": "#1f2328",
    "muted": "#59636e",
    "disabled": "#8b949e",
    "border": "#c3c9d0",
    "divider": "#d6dbe0",
    "button": "#ffffff",
    "button_hover": "#f0f5fa",
    "button_pressed": "#e3edf6",
    "accent": "#2f80d1",
    "accent_hover": "#3b8edb",
    "accent_pressed": "#246eB8",
    "select": "#2f80d1",
    "select_text": "#ffffff",
    "scroll_trough": "#e8ebee",
    "scroll_thumb": "#b9c2ca",
    "scroll_thumb_hover": "#929faa",
}

DARK = {
```

MBToolkit v3.5/mb_theme.py — lines 41-80

```
    "root": "#171a1f",
    "panel": "#171a1f",
    "panel_alt": "#1f242a",
    "input": "#20262c",
    "input_readonly": "#252c33",
    "text": "#e8eaed",
    "muted": "#aab2ba",
    "disabled": "#707982",
    "border": "#46525d",
    "divider": "#46525d",
    "button": "#252c33",
    "button_hover": "#303942",
    "button_pressed": "#1f252b",
    "accent": "#2f8fd8",
    "accent_hover": "#43a2e8",
    "accent_pressed": "#287bb8",
    "select": "#285f80",
    "select_text": "#ffffff",
    "scroll_trough": "#1d2329",
    "scroll_thumb": "#65717c",
    "scroll_thumb_hover": "#7d8994",
}

_CHECK_IMAGES = None
_CHECK_ELEMENT = "MBS.Check.indicator"
_FIXED_GEOMETRY_READY = False
_INTERP_ID = None


def theme_colors(root=None, dark=False):
    return dict(DARK if dark else LIGHT)


def _ensure_engine(style):
    """Select/configure the fixed engine once per Tk interpreter."""
    global _FIXED_GEOMETRY_READY, _CHECK_IMAGES, _INTERP_ID

    # A normal MBToolkit process owns one Tk root. Handling a replaced Tk
    # interpreter as well makes tests/relaunches in the same Python process
    # safe and prevents stale PhotoImage command references.
```

MBToolkit v3.5/mb_theme.py — lines 81-120

```
    current_interp = id(style.master.tk)
    if _INTERP_ID != current_interp:
        _INTERP_ID = current_interp
        _FIXED_GEOMETRY_READY = False
        _CHECK_IMAGES = None

    try:
        if THEME_ENGINE in style.theme_names() and style.theme_use() != THEME_ENGINE:
            style.theme_use(THEME_ENGINE)
    except Exception:
        pass

    if not _FIXED_GEOMETRY_READY:
        _configure_fixed_geometry(style)
        _FIXED_GEOMETRY_READY = True


def _configure_fixed_geometry(style):
    """
    Define every geometry/state rule once.

    Nothing in this function is called as part of a Light/Dark toggle after
    startup. This is what guarantees that the page construction cannot move.
    """

    # ------------------------------------------------------------------
    # LabelFrames: titles above the rectangular border in both modes.
    # ------------------------------------------------------------------
    style.configure(
        "TLabelframe",
        borderwidth=1,
        relief="solid",
        padding=0,
        labeloutside=True,
        labelmargins=(0, 0, 0, 4),
    )
    style.configure(
        "TLabelframe.Label",
        borderwidth=0,
        padding=0,
```

MBToolkit v3.5/mb_theme.py — lines 121-160

```
        font=("Segoe UI", 9, "bold"),
    )

    # ------------------------------------------------------------------
    # Notebook/tabs: fixed Vista-like top geometry.
    # ------------------------------------------------------------------
    style.configure(
        "TNotebook",
        borderwidth=1,
        padding=0,
        tabmargins=(2, 2, 2, 0),
    )
    style.configure(
        "TNotebook.Tab",
        borderwidth=1,
        padding=(12, 6),
        font=("Segoe UI", 9),
    )
    style.map(
        "TNotebook.Tab",
        expand=[("selected", (2, 2, 2, 2))],
        padding=[("selected", (12, 6))],
        relief=[
            ("selected", "flat"),
            ("active", "flat"),
            ("pressed", "flat"),
        ],
    )

    # ------------------------------------------------------------------
    # Buttons.
    # ------------------------------------------------------------------
    style.configure(
        "TButton",
        borderwidth=1,
        relief="solid",
        padding=(7, 4),
        font=("Segoe UI", 9),
    )
    style.configure(
```

MBToolkit v3.5/mb_theme.py — lines 161-200

```
        "MBS.Toolbar.TButton",
        borderwidth=1,
        relief="solid",
        padding=(9, 5),
        font=("Segoe UI", 9),
    )
    style.configure(
        "MBS.Primary.TButton",
        borderwidth=1,
        relief="solid",
        padding=(9, 5),
        font=("Segoe UI", 9, "bold"),
    )
    style.configure(
        "MBS.Compact.TButton",
        borderwidth=1,
        relief="solid",
        padding=(7, 4),
        font=("Segoe UI", 9),
    )

    # ------------------------------------------------------------------
    # Input controls.
    # ------------------------------------------------------------------
    style.configure("TEntry", borderwidth=1, relief="solid", padding=1)
    style.configure("TCombobox", borderwidth=1, relief="solid", padding=1, arrowsize=10)
    style.configure("TSpinbox", borderwidth=1, relief="solid", padding=1, arrowsize=7)

    # ------------------------------------------------------------------
    # Checkboxes: fixed 15x15 image indicator. The image pixels are recoloured
    # on toggle; the element/layout never changes.
    # ------------------------------------------------------------------
    style.configure(
        "TCheckbutton",
        borderwidth=0,
        relief="flat",
        padding=(1, 1),
        font=("Segoe UI", 9),
    )

```

MBToolkit v3.5/mb_theme.py — lines 201-240

```
    # Create the indicator element from the active clam layout once.
    _install_fixed_checkbutton_element(style)

    # ------------------------------------------------------------------
    # Treeviews.
    # ------------------------------------------------------------------
    style.configure(
        "Treeview",
        borderwidth=1,
        relief="solid",
        font=("Segoe UI", 9),
    )
    style.configure(
        "MBToolkit.Treeview",
        borderwidth=1,
        relief="solid",
        font=("Segoe UI", 9),
    )

    # Important: right + bottom border only. This reproduces Light mode's
    # vertical column divider lines without making every heading look like a
    # separate raised rectangular button.
    for heading in ("Treeview.Heading", "MBToolkit.Treeview.Heading"):
        style.configure(
            heading,
            borderwidth=(0, 0, 1, 1),
            relief="solid",
            padding=(5, 4),
            font=("Segoe UI", 9),
        )
        style.map(
            heading,
            relief=[
                ("active", "solid"),
                ("pressed", "solid"),
            ],
        )

    # Keep MBToolkit.Treeview on the same heading layout as Treeview.
    try:
```

MBToolkit v3.5/mb_theme.py — lines 241-280

```
        heading_layout = style.layout("Treeview.Heading")
        if heading_layout:
            style.layout("MBToolkit.Treeview.Heading", heading_layout)
    except Exception:
        pass

    # ------------------------------------------------------------------
    # Scrollbars and named labels.
    # ------------------------------------------------------------------
    # Minimal scrollbars: slim track + thumb only, without arrow buttons or extra controls.
    style.layout(
        "Vertical.TScrollbar",
        [(
            "Vertical.Scrollbar.trough",
            {
                "sticky": "ns",
                "children": [
                    ("Vertical.Scrollbar.thumb", {"sticky": "nswe"})
                ],
            },
        )],
    )
    style.layout(
        "Horizontal.TScrollbar",
        [(
            "Horizontal.Scrollbar.trough",
            {
                "sticky": "we",
                "children": [
                    ("Horizontal.Scrollbar.thumb", {"sticky": "nswe"})
                ],
            },
        )],
    )
    style.configure(
        "Vertical.TScrollbar",
        width=6,
        borderwidth=0,
        relief="flat",
    )
```

MBToolkit v3.5/mb_theme.py — lines 281-320

```
    style.configure(
        "Horizontal.TScrollbar",
        width=6,
        borderwidth=0,
        relief="flat",
    )

    style.configure(
        "Horizontal.TProgressbar",
        borderwidth=0,
        thickness=8,
        relief="flat",
    )

    style.configure("MBS.Stat.TLabel", font=("Segoe UI", 9))
    style.configure("MBS.Status.TLabel", font=("Segoe UI", 9, "bold"))


def _paint_checkbox_image(image, *, fill, border, check=None):
    """Repaint one fixed-size checkbox image without changing its dimensions."""
    size = 15
    image.put(border, to=(0, 0, size, size))
    image.put(fill, to=(1, 1, size - 1, size - 1))

    if check:
        # White/black check mark. Coordinates remain identical in both modes.
        points = (
            (3, 7), (4, 8), (5, 9), (6, 10),
            (7, 9), (8, 8), (9, 7), (10, 6), (11, 5), (12, 4),
        )
        for x, y in points:
            image.put(check, to=(x, y, x + 1, y + 2))
            if x + 1 < size - 1:
                image.put(check, to=(x + 1, y, x + 2, y + 1))


def _install_fixed_checkbutton_element(style):
    global _CHECK_IMAGES

    root = style.master
```

MBToolkit v3.5/mb_theme.py — lines 321-360

```
    if _CHECK_IMAGES is None:
        _CHECK_IMAGES = {
            "off": tk.PhotoImage(master=root, width=15, height=15),
            "on": tk.PhotoImage(master=root, width=15, height=15),
            "disabled_off": tk.PhotoImage(master=root, width=15, height=15),
            "disabled_on": tk.PhotoImage(master=root, width=15, height=15),
        }

    try:
        if _CHECK_ELEMENT not in style.element_names():
            style.element_create(
                _CHECK_ELEMENT,
                "image",
                _CHECK_IMAGES["off"],
                ("disabled selected", _CHECK_IMAGES["disabled_on"]),
                ("disabled", _CHECK_IMAGES["disabled_off"]),
                ("selected", _CHECK_IMAGES["on"]),
                sticky="",
            )
    except tk.TclError:
        pass

    try:
        layout = style.layout("TCheckbutton")
    except Exception:
        layout = None

    if not layout:
        return

    def replace_indicator(nodes):
        result = []
        for name, options in nodes:
            options = dict(options)
            if "children" in options:
                options["children"] = replace_indicator(options["children"])
            if name == "Checkbutton.indicator":
                name = _CHECK_ELEMENT
            result.append((name, options))
        return result
```

MBToolkit v3.5/mb_theme.py — lines 361-400

```
361
362     # Avoid repeatedly replacing an already-custom element.
363     if _CHECK_ELEMENT not in repr(layout):
364         try:
365             style.layout("TCheckbutton", replace_indicator(layout))
366         except tk.TclError:
367             pass
368
369
370 def _repaint_checkboxes(colors, dark):
371     if not _CHECK_IMAGES:
372         return
373
374     if dark:
375         face = colors["input_readonly"]
376         border = colors["disabled"]
377         check = colors["select_text"]
378         disabled_face = colors["panel_alt"]
379         disabled_check = colors["disabled"]
380     else:
381         face = colors["input"]
382         border = "#7a7a7a"
383         check = "#111111"
384         disabled_face = "#e4e4e4"
385         disabled_check = "#999999"
386
387     _paint_checkbox_image(
388         _CHECK_IMAGES["off"],
389         fill=face,
390         border=border,
391     )
392     _paint_checkbox_image(
393         _CHECK_IMAGES["on"],
394         fill=face,
395         border=border,
396         check=check,
397     )
398     _paint_checkbox_image(
399         _CHECK_IMAGES["disabled_off"],
400         fill=disabled_face,
```

MBToolkit v3.5/mb_theme.py — lines 401-440

```
        border=colors["disabled"],
    )
    _paint_checkbox_image(
        _CHECK_IMAGES["disabled_on"],
        fill=disabled_face,
        border=colors["disabled"],
        check=disabled_check,
    )


def _apply_palette(root, style, colors, dark):
    """
    Apply colours only.

    No call in this function changes:
    - theme engine
    - layout
    - padding
    - border width
    - relief
    - margins
    - font
    - widget dimensions
    """
    c = colors

    style.configure(".", background=c["root"], foreground=c["text"])
    style.configure("TFrame", background=c["root"])
    style.configure("TLabel", background=c["root"], foreground=c["text"])
    style.configure(
        "TLabelframe",
        background=c["root"],
        foreground=c["text"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.configure(
        "TLabelframe.Label",
        background=c["root"],
```

MBToolkit v3.5/mb_theme.py — lines 441-480

```
        foreground=c["text"],
    )

    # Notebook.
    style.configure(
        "TNotebook",
        background=c["root"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.configure(
        "TNotebook.Tab",
        background=c["panel"],
        foreground=c["text"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.map(
        "TNotebook.Tab",
        background=[
            ("selected", c["panel_alt"]),
            ("active", c["panel_alt"]),
        ],
        foreground=[
            ("selected", c["text"]),
            ("active", c["text"]),
        ],
    )

    # Buttons: neutral controls have clear contrast; the primary action uses
    # the same restrained accent in Light and Dark.
    for name in (
        "TButton",
        "MBS.Toolbar.TButton",
        "MBS.Compact.TButton",
    ):
        style.configure(
            name,
```

MBToolkit v3.5/mb_theme.py — lines 481-520

```
            background=c["button"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
            focuscolor=c["accent"],
        )
        style.map(
            name,
            background=[
                ("pressed", c["button_pressed"]),
                ("active", c["button_hover"]),
                ("disabled", c["panel"]),
            ],
            foreground=[("disabled", c["disabled"])],
        )

    style.configure(
        "MBS.Primary.TButton",
        background=c["accent"],
        foreground=c["select_text"],
        bordercolor=c["accent"],
        lightcolor=c["accent"],
        darkcolor=c["accent"],
        focuscolor=c["accent"],
    )
    style.map(
        "MBS.Primary.TButton",
        background=[
            ("pressed", c["accent_pressed"]),
            ("active", c["accent_hover"]),
            ("disabled", c["panel"]),
        ],
        foreground=[
            ("disabled", c["disabled"]),
        ],
        bordercolor=[
            ("pressed", c["accent_pressed"]),
            ("active", c["accent_hover"]),
            ("disabled", c["border"]),
```

MBToolkit v3.5/mb_theme.py — lines 521-560

```
        ],
    )

    # Entry / Spinbox.
    for name in ("TEntry", "TSpinbox"):
        style.configure(
            name,
            fieldbackground=c["input"],
            background=c["input"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
            insertcolor=c["text"],
        )
        style.map(
            name,
            fieldbackground=[
                ("readonly", c["input_readonly"]),
                ("disabled", c["panel"]),
            ],
            foreground=[("disabled", c["disabled"])],
        )

    # Combobox.
    style.configure(
        "TCombobox",
        fieldbackground=c["input"],
        background=c["button"],
        foreground=c["text"],
        arrowcolor=c["text"],
        bordercolor=c["border"],
        lightcolor=c["border"],
        darkcolor=c["border"],
    )
    style.map(
        "TCombobox",
        fieldbackground=[
            ("readonly", c["input_readonly"]),
            ("disabled", c["panel"]),
```

MBToolkit v3.5/mb_theme.py — lines 561-600

```
        ],
        foreground=[
            ("readonly", c["text"]),
            ("disabled", c["disabled"]),
        ],
        background=[
            ("active", c["button_hover"]),
            ("disabled", c["panel"]),
        ],
        arrowcolor=[("disabled", c["disabled"])],
    )

    # Checkbox label/background only; indicator pixels are repainted separately.
    style.configure(
        "TCheckbutton",
        background=c["root"],
        foreground=c["text"],
    )
    style.map(
        "TCheckbutton",
        background=[("active", c["root"])],
        foreground=[("disabled", c["disabled"])],
    )
    _repaint_checkboxes(c, dark)

    # Treeview body and flat heading.
    for name in ("Treeview", "MBToolkit.Treeview"):
        style.configure(
            name,
            background=c["input"],
            fieldbackground=c["input"],
            foreground=c["text"],
            bordercolor=c["border"],
            lightcolor=c["border"],
            darkcolor=c["border"],
        )
        style.map(
            name,
            background=[("selected", c["select"])],
            foreground=[("selected", c["select_text"])],
```

MBToolkit v3.5/mb_theme.py — lines 601-640

```
        )

    for name in ("Treeview.Heading", "MBToolkit.Treeview.Heading"):
        style.configure(
            name,
            background=c["input"],
            foreground=c["text"],
            bordercolor=c["divider"],
            lightcolor=c["divider"],
            darkcolor=c["divider"],
        )
        style.map(
            name,
            background=[
                ("active", c["input_readonly"]),
                ("pressed", c["input_readonly"]),
            ],
            foreground=[("active", c["text"])],
        )

    # Scrollbars: only the slim thumb should be visually apparent.
    # The required trough element remains for ttk mechanics, but is painted
    # exactly like the content background so it disappears visually.
    for name in ("Vertical.TScrollbar", "Horizontal.TScrollbar", "TScrollbar"):
        style.configure(
            name,
            background=c["scroll_thumb"],
            troughcolor=c["input"],
            bordercolor=c["input"],
            darkcolor=c["scroll_thumb"],
            lightcolor=c["scroll_thumb"],
        )
        style.map(
            name,
            background=[
                ("active", c["scroll_thumb_hover"]),
                ("pressed", c["scroll_thumb_hover"]),
            ],
            darkcolor=[
                ("active", c["scroll_thumb_hover"]),
```

MBToolkit v3.5/mb_theme.py — lines 641-680

```
641                 ("pressed", c["scroll_thumb_hover"]),
642             ],
643             lightcolor=[
644                 ("active", c["scroll_thumb_hover"]),
645                 ("pressed", c["scroll_thumb_hover"]),
646             ],
647         )
648 
649     style.configure(
650         "Horizontal.TProgressbar",
651         background=c["accent"],
652         troughcolor=c["scroll_trough"],
653         bordercolor=c["scroll_trough"],
654         lightcolor=c["accent"],
655         darkcolor=c["accent"],
656     )
657 
658     style.configure("MBS.Stat.TLabel", background=c["root"], foreground=c["text"])
659     style.configure("MBS.Status.TLabel", background=c["root"], foreground=c["text"])
660 
661     try:
662         root.configure(background=c["root"])
663     except Exception:
664         pass
665 
666     return style
667 
668 
669 def apply_theme(root, dark=False, style=None):
670     """
671     Change palette only.
672 
673     The ttk engine and all geometry were fixed before the UI was built.
674     """
675     style = style or ttk.Style(root)
676     _ensure_engine(style)
677     return _apply_palette(root, style, DARK if dark else LIGHT, bool(dark))
678 
679 
680 def apply_light_theme(root, style=None):
```

MBToolkit v3.5/mb_theme.py — lines 681-692

```
681     return apply_theme(root, dark=False, style=style)
682
683
684 def apply_dark_theme(root, style=None):
685     return apply_theme(root, dark=True, style=style)
686
687
688 def apply_modern_theme(root, dark=False):
689     """Initial application setup with the same engine used for both modes."""
690     style = ttk.Style(root)
691     _ensure_engine(style)
692     return _apply_palette(root, style, DARK if dark else LIGHT, bool(dark))
```

MBToolkit v3.5/mb_widgets.py

418 source lines.

MBToolkit v3.5/mb_widgets.py — lines 1-40

```python
#!/usr/bin/env python3
"""Small reusable Tk widgets for MBToolkit."""

import tkinter as tk


class RefreshButton(tk.Canvas):
    # The fixed clam UI gives the refresh control stable dimensions, so the ↻
    # can now use its measured rendered-bbox centre without manual offsets.
    OPTICAL_OFFSET_X = 0
    OPTICAL_OFFSET_Y = 0

    """
    Square refresh button using the Unicode ↻ glyph.

    The glyph is centered from its actual rendered Tk bounding box. No manual
    X/Y optical offset is applied.
    """

    def __init__(
        self,
        master,
        command,
        size=26,
        cursor="hand2",
        **kwargs,
    ):
        self._command = command
        self._hover = False
        self._pressed = False
        self._state = "normal"
        self._dark_mode = False
        self._final_redraw_after_id = None

        super().__init__(
            master,
            width=size,
            height=size,
            bd=0,
            highlightthickness=0,
```

MBToolkit v3.5/mb_widgets.py — lines 41-80

```
41            relief="flat",
42            cursor=cursor,
43            takefocus=0,
44            **kwargs,
45        )
46
47        self.bind("<Configure>", self._redraw, add="+")
48        self.bind("<Enter>", self._on_enter, add="+")
49        self.bind("<Leave>", self._on_leave, add="+")
50        self.bind("<ButtonPress-1>", self._on_press, add="+")
51        self.bind("<ButtonRelease-1>", self._on_release, add="+")
52        self._redraw()
53        self._schedule_final_redraw()
54
55    @staticmethod
56    def _system_color(widget, name, fallback):
57        try:
58            widget.winfo_rgb(name)
59            return name
60        except Exception:
61            return fallback
62
63    def _schedule_final_redraw(self):
64        """
65        Redraw after Tk has settled requested widget dimensions.
66
67        Some themed comboboxes report their final height only after the first
68        layout pass. A short deferred redraw prevents the Unicode glyph from
69        staying centered for an earlier 24 px square after the control becomes
70        26 px high.
71        """
72        try:
73            if self._final_redraw_after_id is not None:
74                self.after_cancel(self._final_redraw_after_id)
75            self._final_redraw_after_id = self.after(20, self._final_redraw)
76        except tk.TclError:
77            self._final_redraw_after_id = None
78
79    def _final_redraw(self):
80        self._final_redraw_after_id = None
```

MBToolkit v3.5/mb_widgets.py — lines 81-120

```
81          try:
82              self._redraw()
83          except tk.TclError:
84              pass
85
86      def configure(self, cnf=None, **kwargs):
87          state = kwargs.pop("state", None)
88          if state is not None:
89              self._state = str(state)
90
91          result = super().configure(cnf, **kwargs)
92          try:
93              self._redraw()
94              self._schedule_final_redraw()
95          except tk.TclError:
96              pass
97          return result
98
99      config = configure
100
101     def cget(self, key):
102         if key == "state":
103             return self._state
104         return super().cget(key)
105
106     def invoke(self):
107         if self._state != "disabled" and callable(self._command):
108             return self._command()
109         return None
110
111     def set_dark_mode(self, enabled):
112         self._dark_mode = bool(enabled)
113         self._redraw()
114         self._schedule_final_redraw()
115
116     def _palette(self):
117         if self._dark_mode:
118             face = "#2a3138"
119             hover = "#36404a"
120             pressed = "#20262c"
```

MBToolkit v3.5/mb_widgets.py — lines 121-160

```
            text = "#e8eaed"
            gray = "#707982"
            border = "#4a5560"
            highlight = "#2f8fd8"

            if self._state == "disabled":
                return face, gray, border
            if self._pressed:
                return pressed, text, highlight
            if self._hover:
                return hover, text, highlight
            return face, text, border

        face = self._system_color(self, "SystemButtonFace", "#f0f0f0")
        text = self._system_color(self, "SystemButtonText", "#202020")
        gray = self._system_color(self, "SystemGrayText", "#8a8a8a")
        shadow = self._system_color(self, "System3DShadow", "#adadad")
        highlight = self._system_color(self, "SystemHighlight", "#7a9fc2")

        if self._state == "disabled":
            return face, gray, shadow
        if self._pressed:
            return face, text, highlight
        if self._hover:
            return face, text, highlight
        return face, text, shadow

    def _center_item_bbox(self, item, width, height):
        """Center the visible Tk bbox of one Canvas item on the square."""
        target_x = width / 2.0
        target_y = height / 2.0

        # Tk text bboxes land on integer pixels. Repeat because a fractional
        # first move may be rounded by the Canvas implementation.
        for _ in range(4):
            bbox = self.bbox(item)
            if not bbox:
                return
            current_x = (bbox[0] + bbox[2]) / 2.0
            current_y = (bbox[1] + bbox[3]) / 2.0
```

MBToolkit v3.5/mb_widgets.py — lines 161-200

```
            dx = target_x - current_x
            dy = target_y - current_y

            if abs(dx) <= 0.5 and abs(dy) <= 0.5:
                break

            # Move by integral pixels when a full-pixel correction is needed.
            move_x = round(dx) if abs(dx) > 0.5 else 0
            move_y = round(dy) if abs(dy) > 0.5 else 0
            if move_x == 0 and move_y == 0:
                break
            self.move(item, move_x, move_y)

        # Keep the explicit offset hook at zero so all three refresh buttons
        # use the same measured centre.
        if self.OPTICAL_OFFSET_X or self.OPTICAL_OFFSET_Y:
            self.move(
                item,
                self.OPTICAL_OFFSET_X,
                self.OPTICAL_OFFSET_Y,
            )

    def _redraw(self, _event=None):
        if not self.winfo_exists():
            return

        # During a <Configure> callback winfo_width()/height() may still
        # report the previous geometry on some Tk builds. Prefer the event's
        # new dimensions so centering is calculated for the final square.
        if _event is not None and hasattr(_event, "width") and hasattr(_event, "height"):
            width = max(2, int(_event.width))
            height = max(2, int(_event.height))
        else:
            width = max(2, int(self.winfo_width()))
            height = max(2, int(self.winfo_height()))

        background, foreground, border = self._palette()

        self.delete("all")
        self.create_rectangle(
```

MBToolkit v3.5/mb_widgets.py — lines 201-240

```
            0,
            0,
            width - 1,
            height - 1,
            fill=background,
            outline=border,
            width=1,
            tags=("background",),
        )

        glyph = self.create_text(
            width / 2.0,
            height / 2.0,
            text="↻",
            fill=foreground,
            font=("Segoe UI Symbol", 13),
            anchor="center",
            tags=("glyph",),
        )
        self._center_item_bbox(glyph, width, height)

    def _on_enter(self, _event):
        if self._state != "disabled":
            self._hover = True
            self._redraw()

    def _on_leave(self, _event):
        self._hover = False
        self._pressed = False
        self._redraw()

    def _on_press(self, _event):
        if self._state != "disabled":
            self._pressed = True
            self._redraw()

    def _on_release(self, event):
        if self._state == "disabled":
            return

```

MBToolkit v3.5/mb_widgets.py — lines 241-280

```
241         was_pressed = self._pressed
242         self._pressed = False
243         self._redraw()
244 
245         inside = (
246             0 <= event.x < self.winfo_width()
247             and 0 <= event.y < self.winfo_height()
248         )
249         if was_pressed and inside and callable(self._command):
250             self._command()
251 
252 
253 
254 class ToggleSwitch(tk.Canvas):
255     """Compact on/off slider used for the Light/Dark mode control."""
256 
257     def __init__(
258         self,
259         master,
260         variable,
261         command=None,
262         width=42,
263         height=22,
264         cursor="hand2",
265         **kwargs,
266     ):
267         self.variable = variable
268         self.command = command
269         self._dark_mode = False
270         self._hover = False
271         self._final_redraw_after_id = None
272 
273         super().__init__(
274             master,
275             width=width,
276             height=height,
277             bd=0,
278             highlightthickness=0,
279             relief="flat",
280             cursor=cursor,
```

MBToolkit v3.5/mb_widgets.py — lines 281-320

```
            takefocus=0,
            **kwargs,
        )

        self.bind("<Configure>", self._redraw, add="+")
        self.bind("<ButtonPress-1>", self._press, add="+")
        self.bind("<ButtonRelease-1>", self._toggle, add="+")
        self.bind("<Enter>", self._enter, add="+")
        self.bind("<Leave>", self._leave, add="+")
        self.variable.trace_add("write", lambda *_a: self._redraw())
        self._redraw()
        self._schedule_final_redraw()

    def _schedule_final_redraw(self):
        try:
            if self._final_redraw_after_id is not None:
                self.after_cancel(self._final_redraw_after_id)
            self._final_redraw_after_id = self.after(20, self._final_redraw)
        except tk.TclError:
            self._final_redraw_after_id = None

    def _final_redraw(self):
        self._final_redraw_after_id = None
        try:
            self._redraw()
        except tk.TclError:
            pass

    def set_dark_mode(self, enabled):
        self._dark_mode = bool(enabled)
        self._redraw()
        self._schedule_final_redraw()

    def _parent_bg(self):
        if self._dark_mode:
            return "#171a1f"
        try:
            self.winfo_rgb("SystemButtonFace")
            return "SystemButtonFace"
        except Exception:
```

MBToolkit v3.5/mb_widgets.py — lines 321-360

```
321             return "#f0f0f0"
322
323     def _press(self, _event=None):
324         # Deliberately do not focus the Canvas. This removes the white focus
325         # rectangle that appeared around the switch after mouse clicks.
326         return "break"
327
328     def _toggle(self, _event=None):
329         self.variable.set(not bool(self.variable.get()))
330         if callable(self.command):
331             self.command()
332         return "break"
333
334     def _enter(self, _event=None):
335         self._hover = True
336         self._redraw()
337
338     def _leave(self, _event=None):
339         self._hover = False
340         self._redraw()
341
342     def _redraw(self, _event=None):
343         if not self.winfo_exists():
344             return
345
346         # Use the final dimensions delivered by <Configure>. Without this, the
347         # first draw can happen while Tk still reports the initial 1x1 Canvas,
348         # which made the switch look small until the first mouse hover.
349         if _event is not None and hasattr(_event, "width") and hasattr(_event, "height"):
350             width = max(22, int(_event.width))
351             height = max(14, int(_event.height))
352         else:
353             width = max(22, int(self.winfo_width()))
354             height = max(14, int(self.winfo_height()))
355
356         on = bool(self.variable.get())
357
358         self.configure(background=self._parent_bg())
359         self.delete("all")
360
```

MBToolkit v3.5/mb_widgets.py — lines 361-400

```
        pad = 2
        radius = (height - 2 * pad) / 2.0
        x0, y0 = pad, pad
        x1, y1 = width - pad, height - pad

        if self._dark_mode:
            off_track = "#5a646e"
            on_track = "#2f8fd8"
            hover_track = "#43a2e8"
            knob = "#f4f6f8"
            outline = "#68737d"
        else:
            off_track = "#a8a8a8"
            on_track = "#2689d9"
            hover_track = "#3b9be5"
            knob = "#ffffff"
            outline = "#8f8f8f"

        track = hover_track if self._hover and on else (on_track if on else off_track)

        # Rounded track: rectangle + two circles.
        self.create_rectangle(
            x0 + radius,
            y0,
            x1 - radius,
            y1,
            fill=track,
            outline=track,
        )
        self.create_oval(
            x0,
            y0,
            x0 + 2 * radius,
            y1,
            fill=track,
            outline=track,
        )
        self.create_oval(
            x1 - 2 * radius,
            y0,
```

MBToolkit v3.5/mb_widgets.py — lines 401-418

```
            x1,
            y1,
            fill=track,
            outline=track,
        )

        knob_radius = max(4.0, radius - 2)
        knob_cx = (x1 - radius) if on else (x0 + radius)
        knob_cy = height / 2.0
        self.create_oval(
            knob_cx - knob_radius,
            knob_cy - knob_radius,
            knob_cx + knob_radius,
            knob_cy + knob_radius,
            fill=knob,
            outline=outline,
            width=1,
        )
```

MBToolkit v3.5/mb_gui.py

2748 source lines.

MBToolkit v3.5/mb_gui.py — lines 1-40

```python
#!/usr/bin/env python3
"""Main Tkinter window composition for MBToolkit."""

import queue
import threading
from collections import deque

import tkinter as tk
from tkinter import ttk
import tkinter.font as tkfont

from mb_config import (
    APP_NAME,
    APP_VERSION,
    DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS,
    DISCOVERY_IDLE_CHECK_SECONDS,
    MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
    MAX_RAW_TEXT_LINES,
    MAX_UI_FRAMES,
    MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
    SLOW_RESPONSE_THRESHOLD_MS,
    SNIFFER_SPLIT_FRAME_HOLD_MS,
    debug_sim,
    load_ui_settings,
    resource_path,
    save_ui_settings,
)
from mb_protocol import SessionMetrics
from mb_slave_finder import SlaveFinderMixin
from mb_master_ui import MasterMixin
from mb_slave_ui import SlaveMixin
from mb_tcp_ui import ClientTab, ServerTab
from mb_capture import CaptureMixin
from mb_view import TRAFFIC_MENU_OPEN_IN_MASTER, TRAFFIC_MENU_OPEN_IN_SLAVE, ViewMixin
from mb_widgets import RefreshButton, ToggleSwitch
from mb_theme import apply_modern_theme, apply_theme, theme_colors
from mb_i18n import (
    LANGUAGE_CHOICES,
    LANGUAGE_ENGLISH,
    LANGUAGE_PORTUGUESE,
```

MBToolkit v3.5/mb_gui.py — lines 41-80

```
    canonical_ui_text,
    english_help_blocks,
    normalize_language,
    translate_text,
    translate_runtime_text,
)


class SnifferApp(MasterMixin, SlaveMixin, SlaveFinderMixin, CaptureMixin, ViewMixin, tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(f"{APP_NAME} v{APP_VERSION}")

        # Load the remembered mode before any ttk widgets are constructed.
        # Light and Dark use one fixed ttk construction; only colours differ.
        self.ui_settings = load_ui_settings()
        self.current_language = normalize_language(
            self.ui_settings.get("language", LANGUAGE_PORTUGUESE)
        )
        self.initial_dark_mode = bool(self.ui_settings.get("dark_mode", False))
        self.ui_style = apply_modern_theme(
            self,
            dark=self.initial_dark_mode,
        )

        # Explicit application/window icon. This works when launched through
        # MBToolkit.bat (Python) and when bundled as a PyInstaller EXE.
        try:
            icon_file = resource_path("MBToolkit.ico")
            if icon_file.exists():
                self.iconbitmap(default=str(icon_file))
        except Exception:
            pass

        self.geometry("1280x780")
        self.minsize(1050, 650)

        self.serial_objects = []
        self.serial_by_port = {}
        self.reader_threads = []
```

MBToolkit v3.5/mb_gui.py — lines 81-120

```
        self.reader_generation = 0
        self.stop_event = threading.Event()
        self.event_queue = queue.Queue()
        self.busy = False
        self.capture_active = False
        self.restart_in_progress = False

        # Active slave-discovery state. This is deliberately independent from
        # passive capture/simulation state, but shares the global busy lock so
        # the application can never act as scanner and sniffer at the same time.
        self.discovery_active = False
        self.discovery_stop_event = threading.Event()
        self.discovery_thread = None
        self.discovery_serial = None
        self.discovery_found_count = 0
        self._closing = False
        self._process_queue_error_shown = False

        self.log_file = None
        self.log_path = None

        # Real-capture logs are created lazily: starting/stopping a capture with
        # zero received traffic must not create an empty .txt file.
        self.pending_log_filename = None
        self.pending_log_header = ""
        self.pending_log_notes = []
        self.log_start_failed = False
        self._raw_by_iid = {}
        self._record_by_iid = {}
        self._transaction_filter_seqs = None
        self._successful_request_seqs = set()
        self._advanced_filter_after_id = None
        self._bus_activity_after_id = None

        # Bounded GUI-history bookkeeping.
        self._tree_history = deque()   # current visible Treeview rows
        self._ui_frame_count = 0
        self._raw_text_line_count = 0

        # Source-of-truth for the bounded GUI history. Filtering/sorting only
```

MBToolkit v3.5/mb_gui.py — lines 121-160

```
        # changes the view; it never changes statistics or the disk log.
        self._frame_history = deque()
        self._next_frame_seq = 1

        # Visible Treeview bookkeeping by logical frame sequence.
        self._tree_iids_by_seq = {}
        self._tree_separator_iid_by_seq = {}

        # Raw Hex separator width/reflow bookkeeping.
        self._raw_separator_width_chars = 1
        self._raw_reflow_after_id = None

        # Slave filter state. "All mode" also controls whether newly discovered
        # slaves are automatically selected.
        self.slave_filter_vars = {}
        self.slave_filter_all_mode = True

        self.metrics = SessionMetrics()

        self._build_ui()
        self._freeze_language_layout_geometry()
        self._capture_portuguese_help()
        self.apply_language(save=False)
        self._install_mouse_wheel_guard()
        self.rebuild_slave_filter_menu()
        self.refresh_ports()
        self.update_mode_ui()
        self.update_gap_state()
        self.update_com_status()
        self._restore_saved_main_tab()
        self._install_ui_settings_persistence()

        self.protocol("WM_DELETE_WINDOW", self.on_close)
        self.after(50, self.process_queue)
        self.after(1000, self.poll_com_status)


    def _freeze_language_layout_geometry(self):
        """Freeze language-independent container geometry using the PT layout.

```

MBToolkit v3.5/mb_gui.py — lines 161-200

```
        The UI is constructed from the canonical Portuguese captions. Capture
        the natural geometry before any runtime translation and use those
        dimensions as layout constraints. Language changes may replace text,
        but must never move or resize the surrounding controls/panels.
        """
        try:
            self.update_idletasks()
        except tk.TclError:
            return

        # The global language/theme overlay is anchored to the top-right.
        # Freeze the canonical PT grid-cell widths. English captions are shorter
        # here, so this preserves the exact existing PT positions and prevents
        # the Dark Mode switch / language combobox from sliding left or right.
        overlay = getattr(self, "theme_overlay", None)
        if overlay is not None:
            try:
                self.update_idletasks()
                canonical_column_widths = [
                    max(1, int(overlay.grid_bbox(column, 0)[2]))
                    for column in range(4)
                ]
                for column, width in enumerate(canonical_column_widths):
                    overlay.columnconfigure(column, minsize=width)
                self.update_idletasks()
                self._language_overlay_width = max(1, overlay.winfo_reqwidth())
                self._language_overlay_height = max(1, overlay.winfo_reqheight())
                overlay.place_configure(
                    width=self._language_overlay_width,
                    height=self._language_overlay_height,
                )
            except (tk.TclError, IndexError, TypeError):
                pass

        # The Finder configuration includes wrapped explanatory text. It must
        # be measured while the Finder page is actually laid out; measuring a
        # hidden Notebook page can under-estimate the PT height by one line.
        page = getattr(self, "discovery_page", None)
        options = getattr(self, "_discovery_options_frame", None)
        if page is not None and options is not None:
```

MBToolkit v3.5/mb_gui.py — lines 201-240

```
            try:
                selected_tab = self.main_notebook.select()
                self.main_notebook.select(page)
                self.update_idletasks()
                height = max(options.winfo_height(), options.winfo_reqheight(), 1)
                # grid row height also includes the widget's external pady.
                # Preserve the complete PT cell, not only the LabelFrame body.
                try:
                    cell_height = max(height, int(page.grid_bbox(0, 1)[3]))
                except (tk.TclError, TypeError, IndexError):
                    cell_height = height + 12
                self._language_discovery_options_height = height
                page.rowconfigure(1, minsize=cell_height)
                options.grid_configure(sticky="nsew")
                self.main_notebook.select(selected_tab)
                self.update_idletasks()
            except tk.TclError:
                pass

    def _guard_mouse_wheel(self, event):
        """
        Keep wheel input local to genuinely scrollable content widgets.

        Treeviews and Text widgets retain their normal vertical wheel scrolling,
        as do the scrollbar widgets themselves.  Everywhere else the event is
        consumed before ttk class bindings see it; in particular, this prevents
        Windows ttk Notebook tabs from changing when the user turns the wheel
        over a tab or over non-scrollable page content.
        """
        widget = getattr(event, "widget", None)
        if isinstance(
            widget,
            (ttk.Treeview, tk.Text, tk.Listbox, ttk.Scrollbar, tk.Scrollbar),
        ):
            return None
        return "break"

    def _install_mouse_wheel_guard(self):
        """
        Prepend one application bindtag to every existing widget.
```

MBToolkit v3.5/mb_gui.py — lines 241-280

```
241
242         The UI is statically constructed before this method is called, so this
243         covers the Sniffer, Bus Health, Help and Bus Slave Finder pages without
244         changing individual widget behavior or notebook keyboard navigation.
245         """
246         tag = "MBToolkit.MouseWheelGuard"
247
248         for sequence in (
249             "<MouseWheel>",
250             "<Shift-MouseWheel>",
251             "<Button-4>",
252             "<Button-5>",
253         ):
254             try:
255                 self.bind_class(
256                     tag,
257                     sequence,
258                     self._guard_mouse_wheel,
259                 )
260             except tk.TclError:
261                 pass
262
263         stack = [self]
264         while stack:
265             widget = stack.pop()
266             try:
267                 tags = tuple(widget.bindtags())
268                 if tag not in tags:
269                     widget.bindtags((tag,) + tags)
270                 stack.extend(widget.winfo_children())
271             except tk.TclError:
272                 continue
273
274     def _apply_classic_widget_palette(self, dark):
275         """Apply palette to the small set of classic Tk widgets."""
276         colors = theme_colors(self, dark)
277
278         if hasattr(self, "raw_text"):
279             self.raw_text.configure(
280                 background=colors["input"],
```

MBToolkit v3.5/mb_gui.py — lines 281-320

```
                foreground=colors["text"],
                insertbackground=colors["text"],
                selectbackground=colors["select"],
                selectforeground=colors["select_text"],
            )

        if hasattr(self, "help_text"):
            self.help_text.configure(
                background=colors["input"],
                foreground=colors["text"],
                insertbackground=colors["text"],
                selectbackground=colors["select"],
                selectforeground=colors["select_text"],
            )

        for menu_name in ("slave_filter_menu", "traffic_context_menu", "discovery_context_menu"):
            menu = getattr(self, menu_name, None)
            if menu is None:
                continue
            try:
                menu.configure(
                    background=colors["panel"],
                    foreground=colors["text"],
                    activebackground=colors["select"],
                    activeforeground=colors["select_text"],
                    selectcolor=colors["text"],
                )
            except tk.TclError:
                pass

        if hasattr(self, "tree"):
            self.configure_anomaly_tags(dark)
            self.configure_inspector_result_styles()

        for name in (
            "refresh_a_btn",
            "refresh_b_btn",
            "discovery_refresh_btn",
            "dark_mode_switch",
        ):
```

MBToolkit v3.5/mb_gui.py — lines 321-360

```
            widget = getattr(self, name, None)
            setter = getattr(widget, "set_dark_mode", None)
            if callable(setter):
                setter(dark)

        if hasattr(self, "master_canvas"):
            self.master_apply_palette(dark)
        if hasattr(self, "slave_canvas"):
            self.slave_apply_palette(dark)
        if hasattr(self, "client_tab"):
            self.client_tab.master_apply_palette(dark)
            self.server_tab.slave_apply_palette(dark)

    def configure_inspector_result_styles(self):
        """
        Match the Inspector Result foreground to the Traffic result palette.

        Only colour values are configured here; no padding/font/layout metrics
        are changed, so Light/Dark construction remains identical.
        """
        if not hasattr(self, "ui_style"):
            return

        colors = theme_colors(
            self,
            bool(getattr(self, "dark_mode_var", tk.BooleanVar(value=False)).get()),
        )

        tag_map = {
            "success": "result_success",
            "slow": "anomaly_slow",
            "exception": "anomaly_exception",
            "crc": "anomaly_crc",
            "timeout": "anomaly_timeout",
            "raw": "anomaly_raw",
        }

        self.ui_style.configure(
            "MBS.Result.neutral.TLabel",
            background=colors["root"],
```

MBToolkit v3.5/mb_gui.py — lines 361-400

```
            foreground=colors["text"],
        )
        self.ui_style.configure(
            "MBS.Result.pending.TLabel",
            background=colors["root"],
            foreground=colors["text"],
        )

        if hasattr(self, "tree"):
            for key, tag in tag_map.items():
                try:
                    foreground = self.tree.tag_configure(tag)["foreground"]
                except Exception:
                    foreground = colors["text"]

                self.ui_style.configure(
                    f"MBS.Result.{key}.TLabel",
                    background=colors["root"],
                    foreground=foreground,
                )

    def tr(self, text):
        """Translate one canonical UI string to the currently selected language."""
        return translate_text(text, self.current_language)

    def trf(self, text, **values):
        """Translate a format template and substitute values afterwards."""
        return self.tr(text).format(**values)

    def _capture_portuguese_help(self):
        """Store the original PT-PT Help text and tag ranges exactly as built."""
        if not hasattr(self, "help_text"):
            return
        try:
            self._help_pt_text = self.help_text.get("1.0", "end-1c")
            ranges = {}
            for tag in self.help_text.tag_names():
                if tag == "sel":
                    continue
                tag_ranges = list(self.help_text.tag_ranges(tag))
```

MBToolkit v3.5/mb_gui.py — lines 401-440

```
401                 ranges[tag] = [
402                     (str(tag_ranges[i]), str(tag_ranges[i + 1]))
403                     for i in range(0, len(tag_ranges), 2)
404                 ]
405             self._help_pt_tag_ranges = ranges
406         except tk.TclError:
407             self._help_pt_text = ""
408             self._help_pt_tag_ranges = {}
409 
410     def _render_help_language(self):
411         """Render Help in English or restore the original PT-PT text byte-for-byte."""
412         if not hasattr(self, "help_text"):
413             return
414         try:
415             self.help_text.configure(state="normal")
416             self.help_text.delete("1.0", "end")
417 
418             if self.current_language == LANGUAGE_ENGLISH:
419                 max_ui_frames_text = f"{MAX_UI_FRAMES:_}".replace("_", " ")
420                 max_raw_lines_text = f"{MAX_RAW_TEXT_LINES:_}".replace("_", " ")
421                 for tag, text in english_help_blocks(
422                     max_ui_frames_text,
423                     max_raw_lines_text,
424                     MIN_PENDING_REQUEST_TIMEOUT_SECONDS,
425                     MAX_PENDING_REQUEST_TIMEOUT_SECONDS,
426                     debug_sim=(debug_sim == 1),
427                 ):
428                     self.help_text.insert("end", text, tag)
429             else:
430                 self.help_text.insert("1.0", getattr(self, "_help_pt_text", ""))
431                 for tag, ranges in getattr(self, "_help_pt_tag_ranges", {}).items():
432                     for start, end in ranges:
433                         self.help_text.tag_add(tag, start, end)
434 
435             self.help_text.configure(state="disabled")
436             self.help_text.see("1.0")
437         except tk.TclError:
438             pass
439 
440     def _translate_widget_tree(self):
```

MBToolkit v3.5/mb_gui.py — lines 441-480

```
441         """Translate static widget captions without changing widget geometry."""
442         stack = [self]
443         while stack:
444             widget = stack.pop()
445             try:
446                 stack.extend(widget.winfo_children())
447             except tk.TclError:
448                 continue
449 
450             # Text widgets contain documents, not captions.
451             if isinstance(widget, tk.Text):
452                 continue
453 
454             try:
455                 keys = widget.keys()
456             except Exception:
457                 keys = ()
458             if "text" in keys:
459                 try:
460                     current = widget.cget("text")
461                     if current:
462                         widget.configure(text=self.tr(current))
463                 except (tk.TclError, TypeError):
464                     pass
465 
466             if isinstance(widget, ttk.Notebook):
467                 try:
468                     for tab_id in widget.tabs():
469                         current = widget.tab(tab_id, "text")
470                         widget.tab(tab_id, text=self.tr(current))
471                 except tk.TclError:
472                     pass
473 
474     def _translate_traffic_context_menu(self):
475         menu = getattr(self, "traffic_context_menu", None)
476         if menu is None:
477             return
478         labels = (
479             (0, "Copiar Raw Hex"),
480             (1, "Copiar frame descodificado"),
```

MBToolkit v3.5/mb_gui.py — lines 481-520

```
            (3, "Filtrar por este Slave"),
            (4, "Filtrar por este FC"),
            (5, "Mostrar apenas esta transação"),
            (6, "Limpar filtro de transação"),
            (TRAFFIC_MENU_OPEN_IN_MASTER, "Abrir no Master"),
            (TRAFFIC_MENU_OPEN_IN_SLAVE, "Abrir no Slave"),
        )
        for index, canonical in labels:
            try:
                menu.entryconfigure(index, label=self.tr(canonical))
            except tk.TclError:
                pass

    def _apply_table_language(self):
        """Refresh headings and filter values that are not normal widget captions."""
        if hasattr(self, "advanced_filter_type_combo"):
            current = canonical_ui_text(self.advanced_filter_type_var.get())
            canonical_values = (
                "Todos", "Requests", "Responses", "Exceptions",
                "CRC errors", "Timeouts", "RAW",
            )
            self.advanced_filter_type_combo.configure(
                values=tuple(self.tr(v) for v in canonical_values)
            )
            self.advanced_filter_type_var.set(self.tr(current))

        if hasattr(self, "time_order_var"):
            canonical_order = canonical_ui_text(self.time_order_var.get())
            self.time_order_var.set(self.tr(canonical_order))

        if hasattr(self, "tree") and hasattr(self, "tree_headings"):
            canonical_headings = {
                "time": "Hora ↓" if self.time_order_descending() else "Hora ↑",
                "delta": "Δt (ms)",
                "response": "Resp. (ms)",
                "channel": "COM",
                "type": "Tipo",
                "slave": canonical_ui_text(self.tree_headings.get("slave", "Slave ▾")),
                "fc": "FC",
                "details": "Detalhes",
```

MBToolkit v3.5/mb_gui.py — lines 521-560

```
                "crc": "CRC",
            }
            for col, canonical in canonical_headings.items():
                translated = self.tr(canonical)
                self.tree_headings[col] = translated
                try:
                    self.tree.heading(col, text=translated)
                except tk.TclError:
                    pass

        if hasattr(self, "discovery_tree"):
            canonical = {
                "slave": "Slave",
                "baud": "Baud",
                "config": "Config.",
                "fc": "FC",
                "result": "Resultado",
                "resp": "Resp. (ms)",
                "device_id": "Device Identification",
                "raw": "Raw Hex",
            }
            for col, value in canonical.items():
                try:
                    self.discovery_tree.heading(col, text=self.tr(value))
                except tk.TclError:
                    pass

    def apply_language(self, save=True):
        """Apply the selected language immediately to the complete UI."""
        selected = getattr(self, "language_var", None)
        if selected is not None:
            self.current_language = normalize_language(selected.get())
            if selected.get() != self.current_language:
                selected.set(self.current_language)

        self._translate_widget_tree()
        self._apply_table_language()
        self._translate_traffic_context_menu()
        self._render_help_language()

```

MBToolkit v3.5/mb_gui.py — lines 561-600

```
        # Rebuild dynamic captions/values in the selected language.
        try:
            self.update_mode_ui()
            self.update_com_status()
            self.update_stats_labels()
            self.rebuild_slave_filter_menu()
            self.update_slave_heading()
            self.update_frame_inspector(self.selected_record())
            self.update_discovery_estimate()
            self.master_apply_language()
            self.slave_apply_language()
            self.client_tab.master_apply_language()
            self.server_tab.slave_apply_language()
        except (AttributeError, tk.TclError):
            pass

        # Translate idle/current status variables without altering state.
        for name in ("status_var", "discovery_status_var", "discovery_found_var"):
            var = getattr(self, name, None)
            if var is not None:
                try:
                    var.set(translate_runtime_text(var.get(), self.current_language))
                except tk.TclError:
                    pass

        # Refresh translated row contents without allowing the language change
        # itself to alter table/card geometry.  Column widths are restored after
        # rendering; normal viewport resizing and new traffic may still use the
        # existing responsive/autofit behaviour.
        traffic_widths = {}
        discovery_widths = {}
        try:
            if hasattr(self, "tree"):
                traffic_widths = {
                    col: int(self.tree.column(col, "width"))
                    for col in self.tree["columns"]
                }
            if hasattr(self, "discovery_tree"):
                discovery_widths = {
                    col: int(self.discovery_tree.column(col, "width"))
```

MBToolkit v3.5/mb_gui.py — lines 601-640

```
                    for col in self.discovery_tree["columns"]
                }
            self.render_filtered_view()
            for col, width in traffic_widths.items():
                self.tree.column(col, width=width)
            for col, width in discovery_widths.items():
                self.discovery_tree.column(col, width=width)
        except (AttributeError, tk.TclError):
            pass

        if save:
            self.save_current_ui_settings()

    def on_language_changed(self, _event=None):
        self.apply_language(save=True)

    def toggle_dark_mode(self):
        """
        Switch Light/Dark colours only.

        No ttk theme replacement, layout change, padding change or row-height
        change occurs here. The page construction is immutable after startup.
        """
        dark = bool(self.dark_mode_var.get())

        self.ui_style = apply_theme(
            self,
            dark=dark,
            style=self.ui_style,
        )
        self._apply_classic_widget_palette(dark)

        # Remember Dark mode together with the rest of the user's preferences.
        self.save_current_ui_settings()

    def _restore_saved_main_tab(self):
        saved = self.ui_settings.get("last_main_tab", "Sniffer")
        pages = {"Bus Slave Finder": self.discovery_page, "Master": self.master_page,
                 "Slave": self.slave_page, "Client": self.client_page,
                 "Server": self.server_page}
```

MBToolkit v3.5/mb_gui.py — lines 641-680

```
641         try:
642             self.main_notebook.select(pages.get(saved, self.sniffer_page))
643         except tk.TclError:
644             pass
645 
646     def _current_main_tab_name(self):
647         try:
648             selected = self.main_notebook.select()
649             if selected == str(self.discovery_page):
650                 return "Bus Slave Finder"
651             if selected == str(self.master_page):
652                 return "Master"
653             if selected == str(self.slave_page):
654                 return "Slave"
655             if selected == str(self.client_page):
656                 return "Client"
657             if selected == str(self.server_page):
658                 return "Server"
659             return "Sniffer"
660         except Exception:
661             return "Sniffer"
662 
663     def _schedule_ui_settings_save(self, *_args):
664         if getattr(self, "_closing", False):
665             return
666 
667         previous = getattr(self, "_settings_save_after_id", None)
668         if previous is not None:
669             try:
670                 self.after_cancel(previous)
671             except tk.TclError:
672                 pass
673 
674         try:
675             self._settings_save_after_id = self.after(
676                 300,
677                 self.save_current_ui_settings,
678             )
679         except tk.TclError:
680             self._settings_save_after_id = None
```

MBToolkit v3.5/mb_gui.py — lines 681-720

```
681
682     def save_current_ui_settings(self):
683         """Persist all roaming-safe user preferences in one settings.json."""
684         pending = getattr(self, "_settings_save_after_id", None)
685         if pending is not None:
686             try:
687                 self.after_cancel(pending)
688             except tk.TclError:
689                 pass
690         self._settings_save_after_id = None
691
692         try:
693             finder_bauds = [
694                 int(baud)
695                 for baud, var in self.discovery_baud_vars.items()
696                 if bool(var.get())
697             ]
698             finder_parities = [
699                 parity
700                 for parity, var in self.discovery_parity_vars.items()
701                 if bool(var.get())
702             ]
703             finder_stops = [
704                 stop
705                 for stop, var in self.discovery_stop_vars.items()
706                 if bool(var.get())
707             ]
708
709             payload = {
710                 "dark_mode": bool(self.dark_mode_var.get()),
711                 "language": normalize_language(self.current_language),
712                 "last_main_tab": self._current_main_tab_name(),
713                 "sniffer": {
714                     "physical_mode": self.mode_var.get(),
715                     "baud": self.baud_var.get(),
716                     "data_bits": self.data_var.get(),
717                     "parity": self.parity_var.get(),
718                     "stop_bits": self.stop_var.get(),
719                     "frame_gap_mode": self.gap_mode_var.get(),
720                     "frame_gap_ms": self.gap_ms_var.get(),
```

MBToolkit v3.5/mb_gui.py — lines 721-760

```
                    "pending_timeout_s": self.pending_timeout_var.get(),
                    "auto_scroll": bool(self.auto_scroll_var.get()),
                    "separate_transactions": bool(
                        self.separate_transactions_var.get()
                    ),
                    "highlight_results": bool(
                        self.highlight_anomalies_var.get()
                    ),
                },
                "bus_slave_finder": {
                    "baud_rates": finder_bauds,
                    "parities": finder_parities,
                    "stop_bits": finder_stops,
                    "slave_start": self.discovery_slave_start_var.get(),
                    "slave_end": self.discovery_slave_end_var.get(),
                    "min_timeout_ms": self.discovery_min_timeout_var.get(),
                    "attempts": self.discovery_attempts_var.get(),
                    "fc04_fallback": bool(
                        self.discovery_fc04_fallback_var.get()
                    ),
                    "device_identification": bool(
                        self.discovery_device_identification_var.get()
                    ),
                },
                "master": {
                    "baud": self.master_baud_var.get(),
                    "parity": self.master_parity_var.get(),
                    "stop_bits": self.master_stop_var.get(),
                    "response_timeout_ms": self.master_timeout_var.get(),
                    "delay_ms": self.master_delay_var.get(),
                    "last_workspace": self.master_workspace.current,
                },
                "slave": {
                    "baud": self.slave_baud_var.get(),
                    "parity": self.slave_parity_var.get(),
                    "stop_bits": self.slave_stop_var.get(),
                    "last_workspace": self.slave_workspace.current,
                },
                "client": self.client_tab.settings_json(),
                "server": self.server_tab.settings_json(),
```

MBToolkit v3.5/mb_gui.py — lines 761-800

```
            }
            save_ui_settings(settings=payload)
        except Exception:
            # Persistence is convenience state only.
            pass

    def _install_ui_settings_persistence(self):
        """Debounced persistence for all roaming-safe user preferences."""
        self._settings_save_after_id = None

        variables = (
            self.mode_var,
            self.baud_var,
            self.data_var,
            self.parity_var,
            self.stop_var,
            self.gap_mode_var,
            self.gap_ms_var,
            self.pending_timeout_var,
            self.auto_scroll_var,
            self.separate_transactions_var,
            self.highlight_anomalies_var,
            self.discovery_slave_start_var,
            self.discovery_slave_end_var,
            self.discovery_min_timeout_var,
            self.discovery_fc04_fallback_var,
            self.discovery_device_identification_var,
        )

        for variable in variables:
            variable.trace_add(
                "write",
                lambda *_a: self._schedule_ui_settings_save(),
            )

        for mapping in (
            self.discovery_baud_vars,
            self.discovery_parity_vars,
            self.discovery_stop_vars,
        ):
```

MBToolkit v3.5/mb_gui.py — lines 801-840

```
            for variable in mapping.values():
                variable.trace_add(
                    "write",
                    lambda *_a: self._schedule_ui_settings_save(),
                )

        self.main_notebook.bind(
            "<<NotebookTabChanged>>",
            lambda _e: self._schedule_ui_settings_save(),
            add="+",
        )

    def _schedule_bus_health_layout_sync(self, _event=None):
        """Debounce Bus Health reflow after notebook/window/Inspector resizing."""
        previous = getattr(self, "_bus_health_layout_after_id", None)
        if previous is not None:
            try:
                self.after_cancel(previous)
            except tk.TclError:
                pass

        try:
            self._bus_health_layout_after_id = self.after_idle(
                self._sync_bus_health_layout
            )
        except tk.TclError:
            self._bus_health_layout_after_id = None

    def _initialize_bus_health_layout(self):
        """Measure the natural full Bus Health height before adaptive reflow."""
        self._bus_health_layout_after_id = None

        cards = getattr(self, "_health_cards", ())
        if not cards:
            return

        try:
            self.update_idletasks()
        except tk.TclError:
            return
```

MBToolkit v3.5/mb_gui.py — lines 841-880

```
841
842         if getattr(self, "_health_full_required_card_height", None) is None:
843             requested = [
844                 card.winfo_reqheight()
845                 for card in cards
846                 if card.winfo_exists()
847             ]
848             if requested:
849                 self._health_full_required_card_height = max(requested)
850
851         self._sync_bus_health_layout()
852
853     def _apply_bus_health_layout(self, compact):
854         """
855         Re-grid Bus Health metrics inside equal vertical bands.
856
857         Full mode uses one metric/value pair per row.  Compact mode uses two
858         pairs per row.  In both modes every used row receives the same weight,
859         so the metrics are distributed evenly over the card height instead of
860         accumulating at the top.  Compact columns use proportional uniform
861         widths (label/value + label/value), which keeps both halves aligned and
862         prevents one metric group from consuming the neighbouring group.
863         """
864         groups = getattr(self, "_health_metric_groups", ())
865         if not groups:
866             return
867
868         for card, pairs in groups:
869             for label_widget, value_widget in pairs:
870                 label_widget.grid_forget()
871                 value_widget.grid_forget()
872
873             # Clear weights left by the other layout before applying the new
874             # one.  Uniform groups are local to each LabelFrame.
875             for column in range(4):
876                 card.columnconfigure(column, weight=0, minsize=0, uniform="")
877             for row in range(max(6, len(pairs))):
878                 card.rowconfigure(row, weight=0, minsize=0, uniform="")
879
880             if compact:
```

MBToolkit v3.5/mb_gui.py — lines 881-920

```
881                 # Size the four compact columns from their real content.
882                 # The two label columns absorb any spare width equally; value
883                 # columns keep their natural size.  This avoids the clipping
884                 # caused by forcing every card into one fixed percentage split.
885                 for column in (0, 2):
886                     card.columnconfigure(column, weight=1, minsize=0, uniform="")
887                 for column in (1, 3):
888                     card.columnconfigure(column, weight=0, minsize=0, uniform="")
889 
890                 used_rows = max(1, (len(pairs) + 1) // 2)
891                 for row in range(used_rows):
892                     card.rowconfigure(
893                         row,
894                         weight=1,
895                         minsize=0,
896                         uniform="health_metric_rows",
897                     )
898 
899                 pair_count = len(pairs)
900                 card_width = max(320, int(card.winfo_width()))
901                 # At normal widths the technical labels/values remain on one
902                 # line.  Near the application's minimum width labels may wrap
903                 # so the pair stays inside the LabelFrame instead of colliding
904                 # with the neighbouring pair.
905                 compact_label_wrap = (
906                     0 if card_width >= 390 else max(80, int(card_width * 0.29))
907                 )
908 
909                 for index, (label_widget, value_widget) in enumerate(pairs):
910                     row = index // 2
911                     group = index % 2
912 
913                     try:
914                         label_widget.configure(wraplength=compact_label_wrap)
915                         value_widget.configure(wraplength=0)
916                     except tk.TclError:
917                         pass
918 
919                     # Odd final metrics use the complete last row, leaving the
920                     # value the whole right-hand side of the card.
```

MBToolkit v3.5/mb_gui.py — lines 921-960

```
                    if index == pair_count - 1 and pair_count % 2:
                        label_widget.grid(
                            row=row,
                            column=0,
                            sticky="w",
                            padx=(6, 3),
                            pady=1,
                        )
                        value_widget.grid(
                            row=row,
                            column=1,
                            columnspan=3,
                            sticky="e",
                            padx=(3, 6),
                            pady=1,
                        )
                        continue

                    base_column = group * 2
                    label_widget.grid(
                        row=row,
                        column=base_column,
                        sticky="w",
                        padx=((6 if base_column == 0 else 5), 3),
                        pady=1,
                    )
                    value_widget.grid(
                        row=row,
                        column=base_column + 1,
                        sticky="e",
                        padx=(3, (5 if base_column == 0 else 6)),
                        pady=1,
                    )
            else:
                # One pair per row, with every row sharing the card height
                # equally.  Proportional columns keep labels and values aligned
                # while still allowing long values to use the right-hand side.
                card.columnconfigure(0, weight=1, minsize=0, uniform="")
                card.columnconfigure(1, weight=0, minsize=0, uniform="")

```

MBToolkit v3.5/mb_gui.py — lines 961-1000

```
                used_rows = max(1, len(pairs))
                for row in range(used_rows):
                    card.rowconfigure(
                        row,
                        weight=1,
                        minsize=0,
                        uniform="health_metric_rows",
                    )

                card_width = max(320, int(card.winfo_width()))
                full_label_wrap = (
                    0 if card_width >= 330 else max(100, int(card_width * 0.55))
                )

                for row, (label_widget, value_widget) in enumerate(pairs):
                    try:
                        label_widget.configure(wraplength=full_label_wrap)
                        value_widget.configure(wraplength=0)
                    except tk.TclError:
                        pass
                    label_widget.grid(
                        row=row,
                        column=0,
                        sticky="w",
                        padx=(10, 6),
                        pady=1,
                    )
                    value_widget.grid(
                        row=row,
                        column=1,
                        sticky="e",
                        padx=(6, 10),
                        pady=1,
                    )

        self._health_layout_compact = bool(compact)

    def _sync_bus_health_layout(self):
        """
        Keep Bus Health metrics inside their rectangles at every available height.
```

MBToolkit v3.5/mb_gui.py — lines 1001-1040

```
        The natural/full card height is measured from the real widgets. If the
        Frame Inspector expands and the notebook becomes too short, the metrics
        reflow automatically. When enough vertical room returns, the original
        layout is restored. Hysteresis prevents resize oscillation.
        """
        self._bus_health_layout_after_id = None

        health_frame = getattr(self, "health_frame", None)
        cards = getattr(self, "_health_cards", ())
        full_required = getattr(
            self,
            "_health_full_required_card_height",
            None,
        )
        if health_frame is None or not cards:
            return

        try:
            if not health_frame.winfo_exists():
                return

            available_frame_height = health_frame.winfo_height()
            if available_frame_height <= 1:
                return

            # Each card has 6 px external top and bottom grid padding.
            available_card_height = max(0, available_frame_height - 12)

            if full_required is None:
                self.update_idletasks()
                full_required = max(card.winfo_reqheight() for card in cards)
                self._health_full_required_card_height = full_required

            compact = bool(getattr(self, "_health_layout_compact", False))
            if compact:
                should_compact = available_card_height < (full_required + 8)
            else:
                should_compact = available_card_height < (full_required + 2)

```

MBToolkit v3.5/mb_gui.py — lines 1041-1080

```
            if should_compact != compact:
                self._apply_bus_health_layout(should_compact)
                self.update_idletasks()
            else:
                # Width can change without crossing the vertical compact/full
                # threshold. Reapply the current layout so wrap limits and the
                # equal-width grid continue to match the real card size.
                self._apply_bus_health_layout(compact)

        except tk.TclError:
            return

    def _build_ui(self):
        sniffer_saved = self.ui_settings.get("sniffer", {})
        if not isinstance(sniffer_saved, dict):
            sniffer_saved = {}

        self.columnconfigure(0, weight=1)
        self.rowconfigure(0, weight=1)

        # Refresh controls use a custom Canvas widget whose arrow geometry is
        # centered explicitly; this avoids ttk theme/font/image alignment drift.

        self.main_notebook = ttk.Notebook(self)
        self.main_notebook.grid(row=0, column=0, sticky="nsew")

        self.sniffer_page = ttk.Frame(self.main_notebook)
        self.sniffer_page.columnconfigure(0, weight=1)
        self.sniffer_page.rowconfigure(1, weight=1)
        self.main_notebook.add(self.sniffer_page, text="Sniffer")

        self.discovery_page = ttk.Frame(self.main_notebook)
        self.discovery_page.columnconfigure(0, weight=1)
        self.discovery_page.rowconfigure(3, weight=1)
        self.main_notebook.add(self.discovery_page, text="Bus Slave Finder")

        self.master_page = ttk.Frame(self.main_notebook)
        self.main_notebook.add(self.master_page, text="Master")

        self.slave_page = ttk.Frame(self.main_notebook)
```

MBToolkit v3.5/mb_gui.py — lines 1081-1120

```
        self.main_notebook.add(self.slave_page, text="Slave")

        # Modbus TCP: second instances of the Master and Slave tabs.
        self.client_page = ttk.Frame(self.main_notebook)
        self.main_notebook.add(self.client_page, text="Client")
        self.server_page = ttk.Frame(self.main_notebook)
        self.main_notebook.add(self.server_page, text="Server")
        self.client_tab = ClientTab(self)
        self.server_tab = ServerTab(self)

        # Global Light/Dark control: root-level overlay in the top-right corner.
        # It is outside both tabs and therefore applies equally to Sniffer and
        # Bus Slave Finder. `place()` means it consumes no layout height.
        self.theme_overlay = ttk.Frame(self)
        self.theme_overlay.place(
            relx=1.0,
            x=-12,
            y=4,
            anchor="ne",
        )

        ttk.Label(
            self.theme_overlay,
            text="Modo escuro",
        ).grid(row=0, column=0, sticky="e", padx=(0, 6))

        self.dark_mode_var = tk.BooleanVar(value=self.initial_dark_mode)
        self.dark_mode_switch = ToggleSwitch(
            self.theme_overlay,
            variable=self.dark_mode_var,
            command=self.toggle_dark_mode,
            width=42,
            height=22,
        )
        self.dark_mode_switch.grid(row=0, column=1, sticky="e")

        self.language_label = ttk.Label(
            self.theme_overlay,
            text="Linguagem",
        )
```

MBToolkit v3.5/mb_gui.py — lines 1121-1160

```
        self.language_label.grid(
            row=0, column=2, sticky="e", padx=(18, 6)
        )
        self.language_var = tk.StringVar(value=self.current_language)
        self.language_combo = ttk.Combobox(
            self.theme_overlay,
            textvariable=self.language_var,
            values=LANGUAGE_CHOICES,
            state="readonly",
            width=10,
        )
        self.language_combo.grid(row=0, column=3, sticky="e")
        self.language_combo.bind(
            "<<ComboboxSelected>>", self.on_language_changed
        )
        self.theme_overlay.lift()

        # ================================================================
        # Compact top workspace: configuration on the left, controls/stats
        # on the right.  This reclaims vertical space for the traffic view.
        # ================================================================
        top_area = ttk.Frame(self.sniffer_page)
        top_area.grid(row=0, column=0, sticky="ew", padx=10, pady=(10, 5))
        # Keep the 3:2 workspace split independent of translated text widths.
        # Without a shared uniform group Tk first honours each child widget's
        # requested width, so changing language moves the two main rectangles.
        top_area.columnconfigure(0, weight=3, uniform="top_workspace")
        top_area.columnconfigure(1, weight=2, uniform="top_workspace")
        # The left Configuração card defines the natural top-workspace height.
        # The right stack stretches to exactly the same top/bottom limits.
        top_area.rowconfigure(0, weight=1)

        cfg = ttk.LabelFrame(top_area, text="Configuração")
        cfg.grid(row=0, column=0, sticky="nsew", padx=(0, 6))
        cfg.columnconfigure(0, weight=1)

        # Four compact equal columns.  Widgets keep the same fixed widths as
        # v2.6; only the card itself is narrower, eliminating wasted whitespace.
        config_grid = ttk.Frame(cfg)
        config_grid.grid(row=0, column=0, sticky="ew", padx=10, pady=(8, 4))
```

MBToolkit v3.5/mb_gui.py — lines 1161-1200

```
        for col in range(4):
            config_grid.columnconfigure(col, weight=1, uniform="sniffer_cfg")

        def config_cell(row, column):
            cell = ttk.Frame(config_grid)
            cell.grid(
                row=row, column=column, sticky="w",
                padx=(0 if column == 0 else 8, 8 if column < 3 else 0),
                pady=(0, 7),
            )
            return cell

        # Row 1 — physical mode / COM A / COM B / capture status
        cell = config_cell(0, 0)
        ttk.Label(cell, text="Modo físico").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.mode_var = tk.StringVar(value=sniffer_saved.get("physical_mode", "RS485 2-wire"))
        self.mode_combo = ttk.Combobox(
            cell, textvariable=self.mode_var, width=13, state="readonly",
            values=["RS485 2-wire", "RS232 single RX", "RS232 dual RX"],
        )
        self.mode_combo.grid(row=1, column=0, sticky="w")
        self.mode_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_mode_ui())

        cell = config_cell(0, 1)
        ttk.Label(cell, text="COM A").grid(row=0, column=0, sticky="w", pady=(0, 2))
        com_a_controls = ttk.Frame(cell)
        com_a_controls.grid(row=1, column=0, sticky="w")
        self.port_a_var = tk.StringVar()
        self.port_a_combo = ttk.Combobox(
            com_a_controls, textvariable=self.port_a_var, width=13, state="readonly"
        )
        self.port_a_combo.grid(row=0, column=0, sticky="w")
        self.port_a_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_com_status())
        self.refresh_a_btn = RefreshButton(com_a_controls, command=self.refresh_ports, size=24)
        self.refresh_a_btn.grid(row=0, column=1, sticky="w", padx=(4, 0))

        def _sync_sniffer_refresh_a_square(_event=None):
            height = self.port_a_combo.winfo_height()
            if height > 1:
                self.refresh_a_btn.configure(width=height, height=height)
```

MBToolkit v3.5/mb_gui.py — lines 1201-1240

```
1201
1202        self.port_a_combo.bind("<Configure>", _sync_sniffer_refresh_a_square, add="+")
1203
1204        cell = config_cell(0, 2)
1205        self.port_b_label = ttk.Label(cell, text="COM B")
1206        self.port_b_label.grid(row=0, column=0, sticky="w", pady=(0, 2))
1207        com_b_controls = ttk.Frame(cell)
1208        com_b_controls.grid(row=1, column=0, sticky="w")
1209        self.port_b_var = tk.StringVar()
1210        self.port_b_combo = ttk.Combobox(
1211            com_b_controls, textvariable=self.port_b_var, width=13, state="readonly"
1212        )
1213        self.port_b_combo.grid(row=0, column=0, sticky="w")
1214        self.port_b_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_com_status())
1215        self.refresh_b_btn = RefreshButton(com_b_controls, command=self.refresh_ports, size=24)
1216        self.refresh_b_btn.grid(row=0, column=1, sticky="w", padx=(4, 0))
1217
1218        def _sync_sniffer_refresh_b_square(_event=None):
1219            height = self.port_b_combo.winfo_height()
1220            if height > 1:
1221                self.refresh_b_btn.configure(width=height, height=height)
1222
1223        self.port_b_combo.bind("<Configure>", _sync_sniffer_refresh_b_square, add="+")
1224
1225        cell = config_cell(0, 3)
1226        ttk.Label(cell, text="Estado").grid(row=0, column=0, sticky="w", pady=(0, 2))
1227        self.status_var = tk.StringVar(value="● Parado")
1228        ttk.Label(
1229            cell,
1230            textvariable=self.status_var,
1231            style="MBS.Status.TLabel",
1232            width=22,
1233            anchor="w",
1234        ).grid(
1235            row=1, column=0, sticky="w"
1236        )
1237
1238        # Row 2 — serial format
1239        cell = config_cell(1, 0)
1240        ttk.Label(cell, text="Baud rate").grid(row=0, column=0, sticky="w", pady=(0, 2))
```

MBToolkit v3.5/mb_gui.py — lines 1241-1280

```
        self.baud_var = tk.StringVar(value=sniffer_saved.get("baud", "9600"))
        ttk.Combobox(
            cell, textvariable=self.baud_var, width=13,
            values=["1200", "2400", "4800", "9600", "19200", "38400", "57600", "115200"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 1)
        ttk.Label(cell, text="Data bits").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.data_var = tk.StringVar(value=sniffer_saved.get("data_bits", "8"))
        ttk.Combobox(
            cell, textvariable=self.data_var, width=13, state="readonly",
            values=["5", "6", "7", "8"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 2)
        ttk.Label(cell, text="Parity").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.parity_var = tk.StringVar(value=sniffer_saved.get("parity", "None"))
        ttk.Combobox(
            cell, textvariable=self.parity_var, width=13, state="readonly",
            values=["None", "Even", "Odd", "Mark", "Space"],
        ).grid(row=1, column=0, sticky="w")

        cell = config_cell(1, 3)
        ttk.Label(cell, text="Stop bits").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.stop_var = tk.StringVar(value=sniffer_saved.get("stop_bits", "1"))
        ttk.Combobox(
            cell, textvariable=self.stop_var, width=13, state="readonly",
            values=["1", "1.5", "2"],
        ).grid(row=1, column=0, sticky="w")

        # Row 3 — timing and display options
        cell = config_cell(2, 0)
        ttk.Label(cell, text="Frame gap").grid(row=0, column=0, sticky="w", pady=(0, 2))
        self.gap_mode_var = tk.StringVar(value=sniffer_saved.get("frame_gap_mode", "Auto"))
        self.gap_combo = ttk.Combobox(
            cell, textvariable=self.gap_mode_var, width=13, state="readonly",
            values=["Auto", "Manual"],
        )
        self.gap_combo.grid(row=1, column=0, sticky="w")
        self.gap_combo.bind("<<ComboboxSelected>>", lambda _e: self.update_gap_state())
```

MBToolkit v3.5/mb_gui.py — lines 1281-1320

```
1281
1282        cell = config_cell(2, 1)
1283        ttk.Label(cell, text="Gap manual (ms)").grid(row=0, column=0, sticky="w", pady=(0, 2))
1284        self.gap_ms_var = tk.StringVar(value=sniffer_saved.get("frame_gap_ms", ""))
1285        self.gap_entry = ttk.Entry(cell, textvariable=self.gap_ms_var, width=15, state="disabled")
1286        self.gap_entry.grid(row=1, column=0, sticky="w")
1287
1288        cell = config_cell(2, 2)
1289        ttk.Label(cell, text="Visualização").grid(row=0, column=0, sticky="w", pady=(0, 2))
1290        self.auto_scroll_var = tk.BooleanVar(value=sniffer_saved.get("auto_scroll", True))
1291        ttk.Checkbutton(cell, text="Auto-scroll", variable=self.auto_scroll_var).grid(
1292            row=1, column=0, sticky="w"
1293        )
1294
1295        cell = config_cell(2, 3)
1296        ttk.Label(cell, text="Pending timeout (s)").grid(row=0, column=0, sticky="w", pady=(0, 2))
1297        self.pending_timeout_var = tk.StringVar(value=sniffer_saved.get("pending_timeout_s", f"{DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS:g}"))
1298        # Direct numeric entry is cleaner here than tiny spin arrows. Range
1299        # validation still happens in get_pending_timeout_seconds().
1300        self.pending_timeout_entry = ttk.Entry(
1301            cell,
1302            textvariable=self.pending_timeout_var,
1303            width=13,
1304        )
1305        self.pending_timeout_entry.grid(row=1, column=0, sticky="w")
1306        ttk.Label(
1307            cell,
1308            text=(
1309                f"min {MIN_PENDING_REQUEST_TIMEOUT_SECONDS:g} / "
1310                f"default {DEFAULT_PENDING_REQUEST_TIMEOUT_SECONDS:g} / "
1311                f"max {MAX_PENDING_REQUEST_TIMEOUT_SECONDS:g}"
1312            ),
1313        ).grid(row=2, column=0, sticky="w", pady=(2, 0))
1314
1315        # COM status + mode help: compact footer inside the configuration card.
1316        com_status = ttk.Frame(cfg)
1317        com_status.grid(row=1, column=0, sticky="w", padx=10, pady=(0, 3))
1318        ttk.Label(com_status, text="Estado COM").grid(
1319            row=0, column=0, rowspan=2, sticky="nw", padx=(0, 14)
1320        )
```

MBToolkit v3.5/mb_gui.py — lines 1321-1360

```
        self.port_a_status_var = tk.StringVar(value="COM A - —")
        self.port_b_status_var = tk.StringVar(value="COM B - —")
        ttk.Label(com_status, textvariable=self.port_a_status_var).grid(row=0, column=1, sticky="w")
        ttk.Label(com_status, textvariable=self.port_b_status_var).grid(
            row=1, column=1, sticky="w", pady=(1, 0)
        )

        self.mode_help_var = tk.StringVar()
        ttk.Label(cfg, textvariable=self.mode_help_var).grid(
            row=2, column=0, sticky="w", padx=10, pady=(2, 7)
        )

        # ------------------------------------------------------------------
        # Right-hand command/status panel.
        # ------------------------------------------------------------------
        side_panel = ttk.Frame(top_area)
        side_panel.grid(row=0, column=1, sticky="nsew", padx=(6, 0))
        side_panel.columnconfigure(0, weight=1)
        side_panel.rowconfigure(0, weight=0)
        side_panel.rowconfigure(1, weight=1)

        actions = ttk.LabelFrame(side_panel, text="Controlos")
        # Same top coordinate as Configuração; only the gap below Controlos
        # separates it from Estatísticas.
        actions.grid(row=0, column=0, sticky="ew", pady=(0, 6))
        actions.columnconfigure(0, weight=1, uniform="action_columns")
        actions.columnconfigure(1, weight=1, uniform="action_columns")

        self.start_btn = ttk.Button(
            actions, text="Iniciar Captura", command=self.start_capture,
            style="MBS.Primary.TButton"
        )
        self.start_btn.grid(row=0, column=0, sticky="ew", padx=(8, 4), pady=(7, 3))

        self.stop_btn = ttk.Button(
            actions, text="Parar", command=self.stop_capture, state="disabled",
            style="MBS.Toolbar.TButton"
        )
        self.stop_btn.grid(row=0, column=1, sticky="ew", padx=(4, 8), pady=(7, 3))

```

MBToolkit v3.5/mb_gui.py — lines 1361-1400

```
        self.restart_btn = ttk.Button(
            actions, text="Reiniciar COM", command=self.restart_com_port,
            style="MBS.Toolbar.TButton"
        )
        self.restart_btn.grid(row=1, column=0, sticky="ew", padx=(8, 4), pady=3)

        ttk.Button(
            actions, text="Limpar", command=self.clear_view,
            style="MBS.Toolbar.TButton"
        ).grid(row=1, column=1, sticky="ew", padx=(4, 8), pady=3)

        ttk.Button(
            actions, text="Abrir pasta de logs", command=self.open_log_folder,
            style="MBS.Toolbar.TButton"
        ).grid(row=2, column=0, columnspan=2, sticky="ew", padx=8, pady=3)

        self.sim_btn = None
        if debug_sim == 1:
            self.sim_btn = ttk.Button(
                actions, text="Simulação", command=self.run_simulation,
                style="MBS.Toolbar.TButton"
            )
            self.sim_btn.grid(row=3, column=0, columnspan=2, sticky="ew", padx=8, pady=3)
            options_row = 4
        else:
            options_row = 3

        options_bar = ttk.Frame(actions)
        options_bar.grid(
            row=options_row,
            column=0,
            columnspan=2,
            sticky="ew",
            padx=8,
            pady=(3, 7),
        )
        options_bar.columnconfigure(2, weight=1)

        self.separate_transactions_var = tk.BooleanVar(
            value=sniffer_saved.get("separate_transactions", True)
```

MBToolkit v3.5/mb_gui.py — lines 1401-1440

```
        )
        ttk.Checkbutton(
            options_bar,
            text="Separar transações",
            variable=self.separate_transactions_var,
            command=self.render_filtered_view,
        ).grid(row=0, column=0, sticky="w", padx=(0, 12))

        self.highlight_anomalies_var = tk.BooleanVar(
            value=sniffer_saved.get("highlight_results", False)
        )
        ttk.Checkbutton(
            options_bar,
            text="Realçar resultados",
            variable=self.highlight_anomalies_var,
            command=self.on_highlight_anomalies_changed,
        ).grid(row=0, column=1, sticky="w")

        self.bus_activity_var = tk.StringVar(value="BUS ○")
        ttk.Label(
            options_bar,
            textvariable=self.bus_activity_var,
            style="MBS.Status.TLabel",
            anchor="e",
            width=9,
        ).grid(row=0, column=2, sticky="e")

        stats = ttk.LabelFrame(side_panel, text="Estatísticas da sessão")
        # Expand vertically into all remaining space so the lower border always
        # lines up with the lower border of Configuração, even if Configuração
        # changes height in the future.
        stats.grid(row=1, column=0, sticky="nsew")
        for col in range(4):
            stats.columnconfigure(col, weight=1, uniform="stats")
        # Keep the eight counters centred in a fixed 4 x 2 matrix.  The
        # labels must not hug the LabelFrame border or shift with translation.
        for row in range(2):
            stats.rowconfigure(row, weight=1, uniform="stats_rows")

        self.stat_requests_var = tk.StringVar(value="Pedidos: 0")
```

MBToolkit v3.5/mb_gui.py — lines 1441-1480

```
1441        self.stat_responses_var = tk.StringVar(value="Respostas: 0")
1442        self.stat_pending_var = tk.StringVar(value="Pendentes: 0")
1443        self.stat_timeouts_var = tk.StringVar(value="Timeouts: 0")
1444        self.stat_crc_var = tk.StringVar(value="Erros de CRC: 0")
1445        self.stat_exceptions_var = tk.StringVar(value="Exceções: 0")
1446        self.stat_raw_var = tk.StringVar(value="RAW: 0")
1447        self.stat_slaves_var = tk.StringVar(value="Slaves: 0")
1448
1449        stat_vars = [
1450            self.stat_requests_var,
1451            self.stat_responses_var,
1452            self.stat_pending_var,
1453            self.stat_timeouts_var,
1454            self.stat_crc_var,
1455            self.stat_exceptions_var,
1456            self.stat_raw_var,
1457            self.stat_slaves_var,
1458        ]
1459        for idx, var in enumerate(stat_vars):
1460            ttk.Label(
1461                stats,
1462                textvariable=var,
1463                style="MBS.Stat.TLabel",
1464                anchor="center",
1465            ).grid(
1466                row=idx // 4,
1467                column=idx % 4,
1468                sticky="",
1469                padx=6,
1470                pady=3,
1471            )
1472
1473        # Notebook
1474        notebook = ttk.Notebook(self.sniffer_page)
1475        notebook.grid(row=1, column=0, sticky="nsew", padx=10, pady=(5, 8))
1476
1477        parsed = ttk.Frame(notebook)
1478        parsed.columnconfigure(0, weight=1)
1479        parsed.rowconfigure(1, weight=1)
1480        notebook.add(parsed, text="Tráfego")
```

MBToolkit v3.5/mb_gui.py — lines 1481-1520

```
1481
1482        filter_bar = ttk.Frame(parsed)
1483        filter_bar.grid(
1484            row=0,
1485            column=0,
1486            columnspan=2,
1487            sticky="ew",
1488            padx=4,
1489            pady=(4, 3),
1490        )
1491        filter_bar.columnconfigure(8, weight=1)
1492        # "Pesquisa" is wider than "Search". Keep its canonical PT cell width
1493        # reserved so the search Entry never moves when language changes.
1494        filter_bar.columnconfigure(7, minsize=65)
1495
1496        ttk.Label(filter_bar, text="Filtros").grid(
1497            row=0, column=0, sticky="w", padx=(2, 6)
1498        )
1499
1500        self.advanced_filter_type_var = tk.StringVar(value="Todos")
1501        type_filter = ttk.Combobox(
1502            filter_bar,
1503            textvariable=self.advanced_filter_type_var,
1504            state="readonly",
1505            width=11,
1506            values=(
1507                "Todos",
1508                "Requests",
1509                "Responses",
1510                "Exceptions",
1511                "CRC errors",
1512                "Timeouts",
1513                "RAW",
1514            ),
1515        )
1516        type_filter.grid(row=0, column=1, sticky="w", padx=(0, 8))
1517        self.advanced_filter_type_combo = type_filter
1518        type_filter.bind(
1519            "<<ComboboxSelected>>",
1520            lambda _e: self.render_filtered_view(),
```

MBToolkit v3.5/mb_gui.py — lines 1521-1560

```
        )

        ttk.Label(filter_bar, text="FC (hex)").grid(
            row=0, column=2, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_fc_var = tk.StringVar(value="")
        fc_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_fc_var,
            width=7,
        )
        fc_filter.grid(row=0, column=3, sticky="w", padx=(0, 8))

        ttk.Label(filter_bar, text="Resp. >").grid(
            row=0, column=4, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_response_ms_var = tk.StringVar(value="")
        response_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_response_ms_var,
            width=7,
        )
        response_filter.grid(row=0, column=5, sticky="w")
        ttk.Label(filter_bar, text="ms").grid(
            row=0, column=6, sticky="w", padx=(3, 8)
        )

        ttk.Label(filter_bar, text="Pesquisa").grid(
            row=0, column=7, sticky="w", padx=(0, 4)
        )
        self.advanced_filter_text_var = tk.StringVar(value="")
        search_filter = ttk.Entry(
            filter_bar,
            textvariable=self.advanced_filter_text_var,
            width=22,
        )
        search_filter.grid(row=0, column=8, sticky="ew")

        filter_actions = ttk.Frame(filter_bar)
        filter_actions.grid(row=0, column=9, sticky="e")
```

MBToolkit v3.5/mb_gui.py — lines 1561-1600

```
        ttk.Button(
            filter_actions,
            text="Limpar filtros",
            command=self.clear_advanced_filters,
            style="MBS.Compact.TButton",
        ).grid(row=0, column=0, padx=(6, 4))
        ttk.Button(
            filter_actions,
            text="Exportar CSV",
            command=self.export_session_csv,
            style="MBS.Compact.TButton",
        ).grid(row=0, column=1)

        for variable in (
            self.advanced_filter_fc_var,
            self.advanced_filter_response_ms_var,
            self.advanced_filter_text_var,
        ):
            variable.trace_add(
                "write",
                self.schedule_advanced_filter_refresh,
            )

        cols = ("time", "delta", "response", "channel", "type", "slave", "fc", "details", "crc")
        # Dedicated style so wrapped multi-line rows do not affect unrelated
        # widgets. Treeview has one row height for the whole table, therefore
        # the row height is adjusted to the largest wrapped Details cell that
        # is currently visible.
        self.tree_style_name = "MBToolkit.Treeview"
        self.tree_style = ttk.Style(self)
        self.tree = ttk.Treeview(
            parsed,
            columns=cols,
            show="headings",
            selectmode="browse",
            style=self.tree_style_name
        )

        self._tree_reflow_after_id = None
        self.tree_detail_min_width_px = 220
```

MBToolkit v3.5/mb_gui.py — lines 1601-1640

```
        self.tree_detail_border_reserve_px = 18

        # Internal sort state. The user changes it only by clicking Hora.
        self.time_order_var = tk.StringVar(
            value="Crescente — antigo → recente"
        )

        self.tree_headings = {
            "time": "Hora ↑",
            "delta": "Δt (ms)",
            "response": "Resp. (ms)",
            "channel": "COM",
            "type": "Tipo",
            "slave": "Slave ▾",
            "fc": "FC",
            "details": "Detalhes",
            "crc": "CRC",
        }

        # Keep columns as compact as possible while leaving breathing room
        # between the longest text and the column borders.
        self.tree_column_padding_px = 28  # total horizontal padding
        self.tree_cell_font = tkfont.nametofont("TkDefaultFont")
        try:
            heading_font_name = ttk.Style(self).lookup("Treeview.Heading", "font")
            self.tree_heading_font = (
                tkfont.nametofont(heading_font_name)
                if heading_font_name
                else self.tree_cell_font
            )
        except Exception:
            self.tree_heading_font = self.tree_cell_font

        self.tree_font_linespace = max(
            1,
            int(self.tree_cell_font.metrics("linespace"))
        )
        self.tree_single_line_rowheight = max(
            20,
            self.tree_font_linespace + 5
```

MBToolkit v3.5/mb_gui.py — lines 1641-1680

```
        )
        self.tree_style.configure(
            self.tree_style_name,
            rowheight=self.tree_single_line_rowheight
        )

        for col in cols:
            align = "w" if col == "details" else "center"
            self.tree.heading(
                col,
                text=self.tree_headings[col],
                anchor=align
            )
            initial_width = self.measure_tree_column_width(
                col,
                [],
                heading_text=self.tree_headings[col]
            )
            self.tree.column(
                col,
                width=initial_width,
                minwidth=initial_width,
                anchor=align,
                stretch=(col == "details")
            )

        self.tree.heading(
            "time",
            text=self.tree_headings["time"],
            anchor="center",
            command=self.toggle_time_order
        )
        self.tree.heading(
            "slave",
            text=self.tree_headings["slave"],
            anchor="center",
            command=self.show_slave_filter_menu
        )

        # Popup menu opened directly from the Slave column heading.
```

MBToolkit v3.5/mb_gui.py — lines 1681-1720

```
        self.slave_filter_menu = tk.Menu(self, tearoff=False)

        # Right-click actions for the selected traffic frame.
        self.traffic_context_menu = tk.Menu(self, tearoff=False)
        self.traffic_context_menu.add_command(
            label="Copiar Raw Hex",
            command=self.copy_selected_raw,
        )
        self.traffic_context_menu.add_command(
            label="Copiar frame descodificado",
            command=self.copy_selected_decoded,
        )
        self.traffic_context_menu.add_separator()
        self.traffic_context_menu.add_command(
            label="Filtrar por este Slave",
            command=self.context_filter_selected_slave,
        )
        self.traffic_context_menu.add_command(
            label="Filtrar por este FC",
            command=self.context_filter_selected_fc,
        )
        self.traffic_context_menu.add_command(
            label="Mostrar apenas esta transação",
            command=self.context_show_selected_transaction,
        )
        self.traffic_context_menu.add_command(
            label="Limpar filtro de transação",
            command=self.clear_transaction_filter,
        )
        self.traffic_context_menu.add_separator()
        self.traffic_context_menu.add_command(
            label="Abrir no Master",
            command=self.context_open_selected_in_master,
        )
        self.traffic_context_menu.add_command(
            label="Abrir no Slave",
            command=self.context_open_selected_in_slave,
        )

        ys = ttk.Scrollbar(parsed, orient="vertical", command=self.tree.yview)
```

MBToolkit v3.5/mb_gui.py — lines 1721-1760

```
        self.tree.configure(yscrollcommand=ys.set)
        self.tree.grid(row=1, column=0, sticky="nsew")
        ys.grid(row=1, column=1, sticky="ns")
        self.tree.bind("<<TreeviewSelect>>", self.on_row_select)
        self.tree.bind("<Button-3>", self.show_traffic_context_menu, add="+")
        self.tree.bind("<Configure>", self.schedule_tree_reflow)
        self.configure_anomaly_tags(self.initial_dark_mode)

        raw = ttk.Frame(notebook)
        raw.columnconfigure(0, weight=1)
        raw.rowconfigure(0, weight=1)
        notebook.add(raw, text="Raw Hex / Log")

        self.raw_text = tk.Text(
            raw,
            wrap="word",
            font=("Consolas", 10)
        )
        self.raw_text_font = tkfont.Font(font=self.raw_text.cget("font"))
        self.raw_text.bind("<Configure>", self.schedule_raw_separator_reflow)

        raw_y = ttk.Scrollbar(raw, orient="vertical", command=self.raw_text.yview)
        self.raw_text.configure(yscrollcommand=raw_y.set)
        self.raw_text.grid(row=0, column=0, sticky="nsew")
        raw_y.grid(row=0, column=1, sticky="ns")

        # Bus Health — session-level diagnostic metrics.
        self.health_frame = ttk.Frame(notebook)
        health_frame = self.health_frame
        health_frame.columnconfigure(0, weight=1, uniform="health")
        health_frame.columnconfigure(1, weight=1, uniform="health")
        health_frame.columnconfigure(2, weight=1, uniform="health")
        health_frame.rowconfigure(0, weight=1)
        notebook.add(health_frame, text="Bus Health")

        traffic_health = ttk.LabelFrame(
            health_frame,
            text="Tráfego",
        )
        traffic_health.grid(
```

MBToolkit v3.5/mb_gui.py — lines 1761-1800

```
            row=0, column=0, sticky="nsew", padx=(6, 3), pady=6
        )
        traffic_health.columnconfigure(1, weight=1)

        response_health = ttk.LabelFrame(
            health_frame,
            text="Tempos de resposta",
        )
        response_health.grid(
            row=0, column=1, sticky="nsew", padx=3, pady=6
        )
        response_health.columnconfigure(1, weight=1)

        quality_health = ttk.LabelFrame(
            health_frame,
            text="Qualidade",
        )
        quality_health.grid(
            row=0, column=2, sticky="nsew", padx=(3, 6), pady=6
        )
        quality_health.columnconfigure(1, weight=1)

        self.health_requests_var = tk.StringVar(value="0")
        self.health_responses_var = tk.StringVar(value="0")
        self.health_crc_count_var = tk.StringVar(value="0")
        self.health_timeout_count_var = tk.StringVar(value="0")
        self.health_exception_count_var = tk.StringVar(value="0")
        self.health_req_rate_var = tk.StringVar(value="0.00 req/s")
        self.health_resp_avg_var = tk.StringVar(value="—")
        self.health_resp_min_var = tk.StringVar(value="—")
        self.health_resp_max_var = tk.StringVar(value="—")
        self.health_p95_var = tk.StringVar(value="—")
        self.health_nonvalidated_rate_var = tk.StringVar(value="0.00 %")
        self.health_timeout_rate_var = tk.StringVar(value="0.00 %")
        self.health_bus_load_var = tk.StringVar(value="0.00 %")
        self.health_slaves_var = tk.StringVar(value="0")
        self.health_total_bytes_var = tk.StringVar(value="0")
        self.health_slowest_var = tk.StringVar(value="—")
        self.health_exception_slaves_var = tk.StringVar(value="—")

```

MBToolkit v3.5/mb_gui.py — lines 1801-1840

```
        self._health_metric_groups = []

        def health_item(parent, label, variable, *, wraplength=0):
            return (
                ttk.Label(parent, text=label),
                ttk.Label(
                    parent,
                    textvariable=variable,
                    anchor="e",
                    wraplength=wraplength,
                    justify="right",
                ),
            )

        traffic_pairs = [
            health_item(traffic_health, "Pedidos", self.health_requests_var),
            health_item(traffic_health, "Respostas", self.health_responses_var),
            health_item(traffic_health, "Taxa de pedidos", self.health_req_rate_var),
            health_item(traffic_health, "Slaves ativos", self.health_slaves_var),
            health_item(traffic_health, "Bytes observados", self.health_total_bytes_var),
            health_item(traffic_health, "Utilização do bus ~", self.health_bus_load_var),
        ]
        response_pairs = [
            health_item(response_health, "Média", self.health_resp_avg_var),
            health_item(response_health, "Mínimo", self.health_resp_min_var),
            health_item(response_health, "Máximo", self.health_resp_max_var),
            health_item(response_health, "P95", self.health_p95_var),
            health_item(response_health, "Slave mais lento", self.health_slowest_var),
        ]
        quality_pairs = [
            health_item(quality_health, "Erros de CRC", self.health_crc_count_var),
            health_item(quality_health, "Bytes não validados", self.health_nonvalidated_rate_var),
            health_item(quality_health, "Timeouts", self.health_timeout_count_var),
            health_item(quality_health, "Taxa de timeout", self.health_timeout_rate_var),
            health_item(quality_health, "Exceções", self.health_exception_count_var),
            health_item(
                quality_health,
                "Exceções / Slave",
                self.health_exception_slaves_var,
                wraplength=190,
```

MBToolkit v3.5/mb_gui.py — lines 1841-1880

```
            ),
        ]

        self._health_metric_groups = [
            (traffic_health, traffic_pairs),
            (response_health, response_pairs),
            (quality_health, quality_pairs),
        ]
        self._health_cards = (
            traffic_health,
            response_health,
            quality_health,
        )
        self._health_layout_compact = False
        self._health_full_required_card_height = None
        self._bus_health_layout_after_id = None

        # Start with the normal layout, then adapt only if the real available
        # height is insufficient.
        self._apply_bus_health_layout(False)

        health_frame.bind(
            "<Configure>",
            self._schedule_bus_health_layout_sync,
            add="+",
        )
        notebook.bind(
            "<<NotebookTabChanged>>",
            self._schedule_bus_health_layout_sync,
            add="+",
        )
        self.after_idle(self._initialize_bus_health_layout)

        # Help tab
        help_frame = ttk.Frame(notebook)
        help_frame.columnconfigure(0, weight=1)
        help_frame.rowconfigure(0, weight=1)
        notebook.add(help_frame, text="Ajuda / Ligações")

        self.help_text = tk.Text(
```

MBToolkit v3.5/mb_gui.py — lines 1881-1920

```
            help_frame,
            wrap="word",
            font=("Segoe UI", 10),
            padx=14,
            pady=12,
            spacing1=2,
            spacing3=5
        )
        help_scroll = ttk.Scrollbar(help_frame, orient="vertical", command=self.help_text.yview)
        self.help_text.configure(yscrollcommand=help_scroll.set)
        self.help_text.grid(row=0, column=0, sticky="nsew")
        help_scroll.grid(row=0, column=1, sticky="ns")

        self.help_text.tag_configure("title", font=("Segoe UI", 14, "bold"), spacing1=8, spacing3=8)
        self.help_text.tag_configure("heading", font=("Segoe UI", 11, "bold"), spacing1=10, spacing3=4)
        self.help_text.tag_configure("subheading", font=("Segoe UI", 10, "bold"), spacing1=6, spacing3=2)
        self.help_text.tag_configure("mono", font=("Consolas", 10), lmargin1=18, lmargin2=18)
        self.help_text.tag_configure("warning", font=("Segoe UI", 10, "bold"), lmargin1=12, lmargin2=12)
        self.help_text.tag_configure("bullet", lmargin1=12, lmargin2=28)

        def h(text_value, tag=None):
            self.help_text.insert("end", text_value, tag)

        max_ui_frames_text = f"{MAX_UI_FRAMES:_}".replace("_", " ")
        max_raw_lines_text = f"{MAX_RAW_TEXT_LINES:_}".replace("_", " ")

        h("MBToolkit — Ajuda e ligações\n", "title")
        h(
            "O MBToolkit foi pensado para diagnóstico de comunicações Modbus RTU. "
            "O separador Sniffer é passivo: abre a(s) porta(s) COM para receção e "
            "não transmite pedidos. O separador Bus Slave Finder é uma ferramenta "
            "ativa de pesquisa e deve ser utilizado apenas quando não existe outro "
            "master ativo no mesmo barramento.\n"
        )

        h("Início rápido — Sniffer\n", "heading")
        h("1. Escolhe o modo físico correto: RS485 2-wire, RS232 single RX ou RS232 dual RX.\n", "bullet")
        h("2. Seleciona a(s) porta(s) COM e os parâmetros série da comunicação observada.\n", "bullet")
        h("3. Mantém Frame gap em Auto, salvo quando tens uma razão concreta para o ajustar.\n", "bullet")
        h("4. Clica em “Iniciar Captura” e confirma atividade através de BUS ● RX.\n", "bullet")
```

MBToolkit v3.5/mb_gui.py — lines 1921-1960

```
1921        h("5. Usa Tráfego para diagnóstico rápido e Raw Hex / Log para confirmar os bytes reais.\n", "bullet")
1922        h("6. Seleciona um frame para obter a análise detalhada no Frame Inspector.\n", "bullet")
1923
1924        h("1. RS485 2-wire\n", "heading")
1925        h(
1926            "Neste modo uma única COM observa pedidos e respostas no mesmo par diferencial. "
1927            "O adaptador do sniffer deve ser ligado em paralelo e não deve atuar como master.\n"
1928        )
1929        h("Ligação típica:\n", "subheading")
1930        h(
1931            "        Equipamento A          Equipamento B\n"
1932            "             D+ --------------- D+\n"
1933            "             D- --------------- D-\n"
1934            "               \\               /\n"
1935            "                \\             /\n"
1936            "                 D+           D-\n"
1937            "                  USB-RS485 sniffer\n\n",
1938            "mono"
1939        )
1940        h("• Liga D+ do sniffer em paralelo com D+ do barramento.\n", "bullet")
1941        h("• Liga D− do sniffer em paralelo com D− do barramento.\n", "bullet")
1942        h(
1943            "• Se existir C/COM/0 V de referência, liga-o apenas quando souberes que "
1944            "corresponde à referência correta da interface RS485.\n",
1945            "bullet"
1946        )
1947        h(
1948            "• Não acrescentes uma terceira resistência de 120 Ω se o barramento já "
1949            "estiver terminado nas duas extremidades.\n",
1950            "warning"
1951        )
1952        h(
1953            "• Se o adaptador USB-RS485 tiver terminação ou bias selecionáveis, deixa-os "
1954            "desativados durante uma captura passiva, salvo se forem intencionalmente "
1955            "necessários na instalação.\n",
1956            "bullet"
1957        )
1958        h(
1959            "• A/B não é uma nomenclatura universal: fabricantes diferentes podem usar "
1960            "polaridades opostas. Dá prioridade a D+/D− e à documentação do equipamento.\n",
```

MBToolkit v3.5/mb_gui.py — lines 1961-2000

```
            "bullet"
        )

        h("2. RS232 single RX\n", "heading")
        h(
            "Observa apenas uma direção RS232. O RS232 é full-duplex: cada direção utiliza "
            "uma linha TX independente, pelo que uma única entrada RX não consegue observar "
            "simultaneamente os dois sentidos.\n"
        )
        h(
            "        Equipamento A                    Equipamento B\n"
            "             TX ----------------------------> RX\n"
            "              \\\n"
            "               \\----> RX do USB-RS232 sniffer (COM A)\n"
            "\n"
            "             GND --------------------------- GND\n"
            "               \\---------------------------> GND sniffer\n\n"
            "        TX do sniffer: NÃO LIGAR\n\n",
            "mono"
        )
        h("• Liga RX do sniffer ao TX da direção que pretendes observar.\n", "bullet")
        h("• Liga também o GND comum.\n", "bullet")
        h("• Deixa fisicamente desligado o TX do adaptador usado como sniffer.\n", "warning")

        h("3. RS232 dual RX\n", "heading")
        h(
            "Permite observar as duas direções RS232 ao mesmo tempo. São necessários "
            "dois canais RX independentes, normalmente dois adaptadores USB-RS232.\n"
        )
        h(
            "        Equipamento A                    Equipamento B\n"
            "             TX ----------------------------> RX\n"
            "              \\----> RX sniffer COM A   (A → B)\n"
            "\n"
            "             RX <---------------------------- TX\n"
            "                                  \\----> RX sniffer COM B   (B → A)\n"
            "\n"
            "             GND --------------------------- GND\n"
            "               \\---------------------------- GND COM A\n"
            "                \\--------------------------- GND COM B\n\n"
```

MBToolkit v3.5/mb_gui.py — lines 2001-2040

```
            "       TX de ambos os sniffers: NÃO LIGAR\n\n",
            "mono"
        )
        h("• COM A é apresentada como A→B e COM B como B→A.\n", "bullet")
        h("• As duas COM devem usar os mesmos parâmetros série da ligação observada.\n", "bullet")

        h("RS232 não é TTL\n", "heading")
        h(
            "Não ligues sinais RS232 diretamente a interfaces USB-TTL/UART de 3,3 V ou "
            "5 V. Usa um adaptador USB-RS232 com transceiver apropriado aos níveis RS232.\n",
            "warning"
        )

        h("Parâmetros da comunicação série\n", "heading")
        h(
            "Baud, Data bits, Parity e Stop bits têm de coincidir com a comunicação real. "
            "Parâmetros errados podem produzir bytes aparentemente aleatórios, frames com "
            "CRC inválido ou ausência total de frames reconhecíveis.\n"
        )
        h("• Baud rate — velocidade em bit/s, por exemplo 9600 ou 19200.\n", "bullet")
        h("• Data bits — normalmente 8 em Modbus RTU.\n", "bullet")
        h("• Parity — None, Even, Odd, Mark ou Space, conforme a instalação.\n", "bullet")
        h("• Stop bits — 1, 1.5 ou 2, conforme a configuração observada.\n", "bullet")

        h("Frame gap\n", "heading")
        h(
            "Em Auto, o programa usa 3,5 tempos de carácter até 19200 bit/s, inclusive. "
            "Acima de 19200 bit/s usa o t3.5 fixo de 1,750 ms recomendado para Modbus RTU. "
            "O modo Manual permite definir outro valor em milissegundos para diagnóstico "
            "de equipamentos ou drivers com comportamento particular.\n"
        )
        h(
            "Os adaptadores USB e o Windows podem agrupar bytes e introduzir latência. "
            "Por isso, os timestamps do MBToolkit são úteis para diagnóstico de sequência "
            "e tempos relativos, mas não substituem um analisador lógico quando são "
            "necessárias medições temporais de elevada precisão.\n"
        )
        h(
            "Quando um burst termina com o início de um frame conhecido ainda incompleto, "
            f"o MBToolkit espera até {SNIFFER_SPLIT_FRAME_HOLD_MS:g} ms adicionais antes de o "
```

MBToolkit v3.5/mb_gui.py — lines 2041-2080

```
            "fechar. Assim, um frame partido em dois pelo adaptador USB não aparece como RAW. "
            "Frames juntos no mesmo burst mantêm a hora de chegada do seu próprio bloco de bytes.\n"
        )

        h("Pending timeout\n", "heading")
        h(
            "Define durante quanto tempo um REQUEST CRC-válido pode permanecer à espera de "
            "uma RESPONSE/EXCEPTION emparelhável. Ao expirar, o REQUEST original é marcado "
            "como timeout e aumenta os contadores de Timeout/Timeout rate.\n"
        )
        h(
            f"Intervalo permitido: {MIN_PENDING_REQUEST_TIMEOUT_SECONDS:g} a "
            f"{MAX_PENDING_REQUEST_TIMEOUT_SECONDS:g} s. O valor deve ser superior ao "
            "tempo de resposta máximo normal da instalação para evitar falsos timeouts.\n"
        )
        h(
            "Pedidos broadcast (Slave 0) não têm resposta por definição: não ficam pendentes, "
            "nunca contam como timeout e não entram no cálculo da Timeout rate.\n"
        )

        h("Como interpretar a tabela Tráfego\n", "heading")
        h("• REQUEST — pedido Modbus RTU reconhecido pelo parser.\n", "bullet")
        h("• RESPONSE — resposta Modbus RTU reconhecida que não é uma EXCEPTION.\n", "bullet")
        h(
            "• FC05/FC06/FC08/FC22 — o pedido e a resposta têm o mesmo formato. A trama é "
            "RESPONSE quando existe um pedido pendente do mesmo Slave/FC (em FC05/06/22 os "
            "bytes têm de ser iguais; em RS232 dual RX tem de vir do canal oposto). Caso "
            "contrário é REQUEST.\n",
            "bullet"
        )
        h(
            "• Respostas FC01/FC02 mostram os bits por ordem de endereço, agrupados por byte. "
            "No Frame Inspector, as respostas de leitura emparelhadas (FC01–FC04) mostram o "
            "PDU Address de cada valor.\n",
            "bullet"
        )
        h(
            "• EXCEPTION — resposta Modbus com Function Code de exceção (bit 7 ativo). "
            "Detalhes mostra o código e a descrição normalizada.\n",
            "bullet"
```

MBToolkit v3.5/mb_gui.py — lines 2081-2120

```
        )
        h("• CRC OK — CRC recebido coincide com o CRC calculado.\n", "bullet")
        h(
            "• CRC ERROR — a estrutura parece um frame suportado, mas o CRC recebido não "
            "coincide com o CRC calculado.\n",
            "bullet"
        )
        h(
            "• RAW/UNSYNC — bytes descartados durante uma recuperação de sincronismo antes "
            "de ser encontrado, mais à frente no mesmo burst, um frame de estrutura "
            "reconhecida com CRC válido.\n",
            "bullet"
        )
        h(
            "• RAW/UNPARSED — restante conjunto de bytes que não pôde ser interpretado nem "
            "ressincronizado como um frame Modbus válido. É este estado que aparece, por "
            "exemplo, num bloco de ruído sem um frame válido posterior no mesmo burst.\n",
            "bullet"
        )
        h(
            "RAW/UNSYNC e RAW/UNPARSED são portanto situações diferentes e ambas podem "
            "aparecer na coluna Tipo. Nenhuma delas representa uma resposta Modbus válida.\n"
        )

        h("Colunas da tabela Tráfego\n", "heading")
        h("• Hora — timestamp do início do burst/frame processado.\n", "bullet")
        h("• Δt — intervalo desde o frame anterior observada.\n", "bullet")
        h(
            "• Resp. — tempo entre um REQUEST e a RESPONSE/EXCEPTION emparelhada. "
            "Fica vazio quando não existe um par válido.\n",
            "bullet"
        )
        h("• COM — identifica o canal de captura: BUS em RS485, A→B/B→A em RS232 dual RX e SIM no modo de desenvolvimento. Não representa necessariamente o número COM do Windows
    .\n", "bullet")
        h("• Tipo — REQUEST, RESPONSE, EXCEPTION ou um estado RAW.\n", "bullet")
        h("• Slave — Unit/Slave ID Modbus quando identificável.\n", "bullet")
        h("• FC — Function Code hexadecimal observado no frame.\n", "bullet")
        h("• Detalhes — endereço, quantidade, dados ou descrição da operação.\n", "bullet")
        h("• CRC — OK, ERROR ou ? quando não existe um frame Modbus completo.\n", "bullet")

        h("Emparelhamento REQUEST / RESPONSE\n", "heading")
```

MBToolkit v3.5/mb_gui.py — lines 2121-2160

```
2121        h(
2122            "Modbus RTU não possui Transaction ID. O MBToolkit emparelha frames por "
2123            "(Slave ID, Function Code) em ordem FIFO. Isto é apropriado para tráfego RTU "
2124            "normal, mas pode tornar-se ambíguo se existirem pedidos sobrepostos com o mesmo "
2125            "Slave e FC antes das respostas correspondentes.\n"
2126        )
2127        h(
2128            "Uma EXCEPTION usa internamente o Function Code base para emparelhamento. "
2129            "Exemplo: uma resposta 0x83 é associada a um REQUEST FC03 quando existe um "
2130            "pedido pendente compatível.\n"
2131        )
2132
2133        h("PDU Address, 1-based e Qty\n", "heading")
2134        h(
2135            "PDU Address é o endereço codificado no pedido Modbus. 1-based é simplesmente "
2136            "PDU Address + 1 e serve para comparação com manuais que numeram o primeiro "
2137            "registo/coil como 1. Qty é a quantidade pedida.\n"
2138        )
2139        h(
2140            "Estes campos pertencem ao REQUEST. Uma RESPONSE não repete automaticamente "
2141            "Address ou Qty no protocolo, por isso o MBToolkit não inventa esses campos na "
2142            "linha da resposta.\n"
2143        )
2144
2145        h("Códigos de exceção Modbus\n", "heading")
2146        h("• 0x01 — Illegal Function: função não suportada ou não permitida.\n", "bullet")
2147        h("• 0x02 — Illegal Data Address: endereço/gama de endereços inválida.\n", "bullet")
2148        h("• 0x03 — Illegal Data Value: valor, quantidade ou campo inválido.\n", "bullet")
2149        h("• 0x04 — Server Device Failure: falha interna do dispositivo.\n", "bullet")
2150        h("• 0x05 — Acknowledge: pedido aceite; processamento ainda em curso.\n", "bullet")
2151        h("• 0x06 — Server Device Busy: dispositivo temporariamente ocupado.\n", "bullet")
2152        h("• 0x08 — Memory Parity Error.\n", "bullet")
2153        h("• 0x0A — Gateway Path Unavailable.\n", "bullet")
2154        h("• 0x0B — Gateway Target Device Failed to Respond.\n", "bullet")
2155
2156        h("Filtros avançados\n", "heading")
2157        h(
2158            "A barra no topo de Tráfego permite combinar vários filtros sem alterar os "
2159            "dados originais da sessão:\n"
2160        )
```

MBToolkit v3.5/mb_gui.py — lines 2161-2200

```
        h("• Tipo – Todos, Requests, Responses, Exceptions, CRC errors, Timeouts ou RAW.\n", "bullet")
        h("• FC (hex) – por exemplo 03, 04, 10 ou 0x04.\n", "bullet")
        h("• Resp. > – mostra apenas respostas acima do limite indicado em ms.\n", "bullet")
        h(
            "• Pesquisa – procura texto em Detalhes, Raw Hex, Slave, FC, Tipo e COM/canal.\n",
            "bullet"
        )
        h(
            "O filtro Slave continua disponível no cabeçalho da coluna e combina-se com "
            "os filtros avançados. “Limpar filtros” repõe todos os filtros de visualização. "
            "Os filtros não alteram as estatísticas nem o conteúdo do log de captura.\n"
        )

        h("Realçar resultados\n", "heading")
        h(
            "Esta opção vem desligada por defeito para manter a tabela neutra. Quando está "
            "ativa, os resultados passam a ser diferenciados por cor: uma transação normal "
            "concluída com RESPONSE CRC-válida fica verde; CRC error, timeout, Exception, "
            "RAW e resposta lenta mantêm as respetivas cores de diagnóstico.\n"
        )
        h(
            "Numa transação normal concluída com sucesso, tanto o REQUEST como a RESPONSE "
            "correspondente ficam verdes. Um REQUEST ainda pendente permanece neutro. "
            "Uma resposta lenta continua com a cor de resposta lenta, mesmo sendo válida, "
            "para não esconder o resultado de diagnóstico.\n"
        )
        h(
            f"Uma resposta é considerada lenta a partir de "
            f"{SLOW_RESPONSE_THRESHOLD_MS:g} ms. Este limite é uma referência visual do "
            "MBToolkit; deve ser interpretado no contexto do equipamento e do ciclo de poll.\n"
        )

        h("Código de cores dos resultados\n", "heading")
        h("• Verde – Sucesso: REQUEST/RESPONSE normal, emparelhada e com CRC válido.\n", "bullet")
        h(
            f"• Amarelo – Resposta lenta: tempo de resposta >= "
            f"{SLOW_RESPONSE_THRESHOLD_MS:g} ms.\n",
            "bullet"
        )
        h("• Vermelho – CRC Error ou resposta Modbus Exception.\n", "bullet")
```

MBToolkit v3.5/mb_gui.py — lines 2201-2240

```
        h("• Laranja — Timeout: REQUEST expirou sem resposta emparelhada.\n", "bullet")
        h("• Roxo — RAW/UNSYNC ou RAW/UNPARSED: bytes sem frame Modbus válido.\n", "bullet")
        h(
            "Com “Realçar resultados” desligado, a tabela permanece neutra. O Frame "
            "Inspector continua a indicar a classificação do resultado em texto; quando "
            "o destaque está ligado, o campo Resultado também usa a respetiva cor.\n"
        )

        h("Frame Inspector\n", "heading")
        h(
            "Seleciona uma linha de Tráfego para analisar o frame sem ter de interpretar "
            "manualmente todos os bytes. O Inspector mostra Slave, Tipo, Function Code + "
            "nome, tempo de resposta, Resultado, PDU Address, 1-based, "
            "Qty, ByteCount, CRC recebido, CRC calculado, dados/registos/Exception e Raw Hex.\n"
        )
        h(
            "“Recolher” esconde o conteúdo do Inspector e devolve espaço vertical à tabela. "
            "“Expandir” volta a apresentar a análise.\n"
        )

        h("Menu de contexto do Tráfego\n", "heading")
        h("Clica com o botão direito numa frame real para:\n")
        h("• Copiar Raw Hex.\n", "bullet")
        h("• Copiar o frame descodificado.\n", "bullet")
        h("• Filtrar pelo Slave dessa frame.\n", "bullet")
        h("• Filtrar pelo Function Code dessa frame.\n", "bullet")
        h("• Mostrar apenas a transação REQUEST/RESPONSE correspondente.\n", "bullet")
        h("• Limpar o filtro temporário de transação.\n", "bullet")

        h("Bus Health\n", "heading")
        h(
            "O separador Bus Health resume a qualidade e o desempenho da sessão sem "
            "substituir a tabela de frames.\n"
        )
        h("• Pedidos / Respostas — contadores da sessão.\n", "bullet")
        h("• Taxa de pedidos — número médio de pedidos observados por segundo.\n", "bullet")
        h("• Slaves ativos — IDs Modbus válidos observados na sessão.\n", "bullet")
        h("• Bytes observados — soma dos bytes entregues ao parser.\n", "bullet")
        h(
            "• Utilização bus ~ — estimativa com base em bytes observados, baud, data bits, "
```

MBToolkit v3.5/mb_gui.py — lines 2241-2280

```
            "parity e stop bits. É aproximada e depende do ponto de captura.\n",
            "bullet"
        )
        h("• Média / Mínimo / Máximo — tempos de resposta emparelhados.\n", "bullet")
        h("• P95 — 95% das respostas medidas ficaram abaixo ou iguais a este valor.\n", "bullet")
        h(
            "• Slave mais lento — Slave com maior tempo médio de resposta entre as amostras "
            "observadas.\n",
            "bullet"
        )
        h("• Bytes não validados — percentagem dos bytes observados que pertencem a frames com CRC inválido ou a blocos RAW. Não prova, por si só, a existência de ruído elétrico
    ; parâmetros série errados, captura incompleta ou funções não suportadas pelo parser também podem produzir RAW.\n", "bullet")
        h("• Taxa de timeout — percentagem de REQUEST que expiraram sem resposta emparelhada.\n", "bullet")
        h("• Exceções / Slave — distribuição das respostas Exception por Slave ID.\n", "bullet")

        h("Indicador BUS RX\n", "heading")
        h(
            "BUS ● RX pulsa brevemente quando chega um lote de frames à interface. Em repouso "
            "é apresentado BUS ○. Serve apenas como confirmação visual de atividade recebida; "
            "não representa qualidade elétrica nem garante que os frames sejam válidos.\n"
        )

        h("Raw Hex / Log\n", "heading")
        h(
            "Mostra os bytes capturados em hexadecimal com informação de contexto. As linhas "
            "longas fazem wrap à largura disponível. A vista acompanha os mesmos filtros "
            "visuais usados em Tráfego; o ficheiro TXT em disco, quando existe, mantém a "
            "captura completa e não é truncado pelos filtros.\n"
        )

        h("Exportar CSV\n", "heading")
        h(
            "Exportar CSV grava os frames atualmente retidos no histórico da interface em "
            "UTF-8 compatível com Excel. Inclui Time, Δt, Response time, Channel, Type, "
            "Slave, FC, Details, CRC, Raw, PDU Address, 1-based, Qty, ByteCount, Timeout "
            "e classe de anomalia.\n"
        )
        h(
            f"A interface mantém aproximadamente os últimos {max_ui_frames_text} frames. "
            "Por isso, em sessões maiores o CSV representa o histórico ainda retido pela GUI, "
            "enquanto o TXT de uma captura real continua a ser o registo completo em disco.\n"
```

MBToolkit v3.5/mb_gui.py — lines 2281-2320

```
        )

        h("Logs de captura\n", "heading")
        h(
            "O log TXT só é criado depois de chegar tráfego real. Iniciar e parar sem receber "
            "frames não cria um ficheiro vazio.\n"
        )
        h("Pasta dos logs:\n")
        h("        <pasta da aplicação>\\MBToolkit Logs\n", "mono")
        h(
            "Os filtros, ordenação, Realçar resultados e o Frame Inspector são apenas funções "
            "de visualização e não alteram os frames gravados no log.\n"
        )

        h("Limites do histórico visível\n", "heading")
        h(
            f"Para manter a aplicação responsiva, a interface conserva aproximadamente "
            f"{max_ui_frames_text} frames e {max_raw_lines_text} linhas de Raw Hex. "
            "A remoção é feita por blocos. O log TXT de captura, quando ativo, é independente "
            "deste limite.\n"
        )

        h("Estado e portas COM\n", "heading")
        h(
            "Estado COM distingue uma porta aberta pelo MBToolkit, uma porta apenas detetada "
            "pelo Windows e uma porta que deixou de ser detetada. O estado é atualizado "
            "periodicamente.\n"
        )
        h(
            "Reiniciar COM fecha e volta a abrir a(s) porta(s) usada(s) pelo Sniffer. "
            "Isto reinicia o handle da porta série dentro do MBToolkit; não reinicia o "
            "driver USB/COM do Windows.\n"
        )
        h(
            "Os botões ↻ ao lado das COM atualizam a enumeração de portas disponível no "
            "Sniffer e no Bus Slave Finder.\n"
        )

        h("Bus Slave Finder — pesquisa ativa\n", "heading")
        h(
```

MBToolkit v3.5/mb_gui.py — lines 2321-2360

```
            "O Bus Slave Finder transmite pedidos Modbus RTU, por isso não é uma função "
            "passiva. Antes de iniciar, confirma que não existe outro master ativo no mesmo "
            "barramento.\n"
        )
        h(
            "Seleciona COM, baud rates, paridades, stop bits e intervalo de Slave IDs. "
            "Data bits é fixo em 8. A pesquisa tenta FC03 Address 0 Qty 1; uma resposta "
            "normal ou uma Exception Modbus com CRC válido confirma que o Slave existe.\n"
        )
        h(
            "FC04 fallback é opcional e aumenta o número máximo de tentativas. O timeout "
            "mínimo é adaptado ao baud rate. Um write timeout significa falha ao transmitir "
            "pela porta/driver e não deve ser interpretado simplesmente como “slave sem resposta”.\n"
        )
        h(
            "Tentativas (1 a 3) repete cada pedido sem resposta antes de passar ao Slave "
            "seguinte. Antes da primeira transmissão, a pesquisa escuta o barramento durante "
            f"{DISCOVERY_IDLE_CHECK_SECONDS:g} s; se receber qualquer byte, não transmite e "
            "indica que pode existir outro master ativo.\n"
        )
        h(
            "Device Identification (FC43/14) é opcional. Quando ativo, só é enviado depois "
            "de um Slave ter sido encontrado e tenta ler Basic Device Identification: "
            "VendorName, ProductCode e MajorMinorRevision. Uma Exception ou ausência de "
            "resposta a FC43/14 não invalida a descoberta do Slave.\n"
        )
        h(
            "O Finder e o Sniffer partilham o bloqueio global de atividade para impedir que "
            "a mesma aplicação faça pesquisa ativa e captura ao mesmo tempo.\n"
        )

        h("Master — leitura e escrita como master Modbus RTU\n", "heading")
        h(
            "O separador Master transmite pedidos: só deve ser usado quando não existe "
            "outro master ativo no barramento. Ao ligar, a aplicação escuta o barramento "
            f"durante {DISCOVERY_IDLE_CHECK_SECONDS:g} s e não transmite se detetar tráfego. "
            "Enquanto o Master está ligado, o Sniffer, o Bus Slave Finder e o Slave ficam "
            "bloqueados.\n"
        )
        h(
```

MBToolkit v3.5/mb_gui.py — lines 2361-2400

```
            "Cada janela lê uma zona de memória de um Slave: Slave ID, função (FC01 Coils, "
            "FC02 Discrete Inputs, FC03 Holding Registers ou FC04 Input Registers), endereço, "
            "quantidade e scan rate. As alterações só valem depois de “Aplicar”, que fica "
            "destacado a azul enquanto houver campos por aplicar. A tabela mostra até 15 "
            "linhas; acima disso tem scroll.\n"
        )
        h(
            "Todas as janelas partilham a mesma porta COM: o Modbus RTU só admite um pedido "
            "de cada vez, por isso as janelas são servidas à vez conforme o scan rate. "
            "Escritas e “Ler 1x” passam à frente das leituras cíclicas. Entre dois pedidos "
            "fica sempre o “Atraso entre pedidos” (no mínimo t3.5).\n"
        )
        h(
            "Formatos: 16 bits com e sem sinal, hex e binário; 32 bits com e sem sinal e "
            "float, nas ordens de bytes ABCD (big-endian), CDAB (palavras trocadas), BADC "
            "(bytes trocados) e DCBA (little-endian); 64 bits com e sem sinal e double, nas "
            "ordens ABCDEFGH, GHEFCDAB, BADCFEHG e HGFEDCBA. Os valores de 32 e 64 bits "
            "ocupam 2 e 4 registos e aparecem na linha do primeiro.\n"
        )
        h(
            "O campo “Formato” é o formato por defeito da janela. Com clique direito num ou "
            "mais endereços (arrastar com o rato, ou Ctrl/Shift, para selecionar vários) escolhe-se o formato só "
            "desses endereços: os de 32 bits exigem 2 registos seguidos e os de 64 bits 4 "
            "(ou um múltiplo, aplicado em grupos). “Repor formato da janela” volta ao "
            "formato por defeito. O mesmo menu tem “Copiar” (também Ctrl+C), que copia "
            "endereços, nomes e valores para colar no Excel, e “Base de endereços”: 0 mostra "
            "o PDU Address; 1 numera a partir de 1, como em muitos manuais.\n"
        )
        h(
            "Quando uma leitura falha (Timeout, Exception ou resposta inválida), os valores "
            "ficam a cinzento até voltar a haver uma leitura correta; ao desligar também.\n"
        )
        h(
            "Escala: no clique direito, “Escala…” converte o valor bruto em unidades de "
            "engenharia com dois pontos (Bruto mín./máx. → Eng. mín./máx.), com casas "
            "decimais e unidade (por exemplo 2000–10000 → −180–180 °). Na escrita, o valor "
            "é escrito em unidades e convertido para bruto. “Remover escala” volta ao valor "
            "bruto. As escalas ficam guardadas no workspace.\n"
        )
        h(
```

MBToolkit v3.5/mb_gui.py — lines 2401-2440

```
            "“Abrir no Master”: no Tráfego do Sniffer (clique direito num pedido FC01–FC04 "
            "ou na resposta correspondente) e nos resultados do Bus Slave Finder (clique "
            "direito ou duplo clique) cria uma janela do Master com o mesmo Slave, função, "
            "endereço e quantidade, e copia os parâmetros série para a ligação do Master "
            "(se não estiver ligado).\n"
        )
        h(
            "Se a porta COM falhar durante a ligação (por exemplo, o adaptador USB foi "
            "desligado), o Master não para: tenta voltar a abrir a porta a cada 2 s e "
            "retoma as leituras quando a porta volta. Para parar, carregar em “Desligar”.\n"
        )
        h(
            "Escrita: duplo clique num valor de uma janela FC01 ou FC03. Coils com FC05 ou "
            "FC15, registos com FC06 ou FC16; vários valores separados por “;” são escritos "
            "em endereços consecutivos com FC15/FC16. Slave 0 envia em broadcast, sem "
            "resposta. Depois de uma escrita aceite, a janela é lida de novo.\n"
        )
        h(
            "Cada janela mostra Tx, erros, timeouts, o último tempo de resposta e o último "
            "erro (Timeout, Exception ou resposta inválida). No cabeçalho de cada janela, "
            "“Renomear” muda o nome da janela (também com duplo clique no nome), "
            "“Destacar” abre-a à parte do separador (“Ancorar” ou fechar essa janela "
            "devolve-a) e “x” fecha a janela.\n"
        )
        h(
            "O menu “Workspace” guarda a ligação, todas as janelas, os nomes e os formatos "
            "com um nome à escolha, e lista os workspaces guardados para os abrir ou apagar "
            "diretamente. Os ficheiros ficam em %APPDATA%\\MBToolkit\\Workspaces\\Master, junto das "
            "preferências.\n"
        )
        h(
            "O último workspace aberto ou guardado é reaberto ao arrancar o programa, sem "
            "ligar ao barramento. Enquanto houver alterações por guardar, aparece "
            "“(alterado)” ao lado do nome; ao fechar o programa nessa situação, pergunta se "
            "quer guardar. Janelas que nunca foram guardadas num workspace fecham sem "
            "perguntar, e fechar todas as janelas deixa de ter workspace aberto.\n"
        )

        h("Slave — simulador de equipamentos Modbus RTU\n", "heading")
        h(
```

MBToolkit v3.5/mb_gui.py — lines 2441-2480

```
            "O separador Slave faz a aplicação comportar-se como um ou mais equipamentos Modbus "
            "RTU: fica à escuta na porta COM e responde aos pedidos de um master real (um PLC, um "
            "SCADA ou outro Modbus Poll). Serve para testar o master sem o equipamento. Responde "
            "a FC01–FC06, FC15 e FC16.\n"
        )
        h(
            "Cada janela é uma tabela de um Slave simulado: Slave ID, tabela (Coils, Discrete "
            "Inputs, Holding Registers ou Input Registers), endereço inicial e quantidade (até "
            "2000). Podem existir vários Slave IDs, cada um com várias janelas. Só existem os "
            "endereços definidos nas janelas: um pedido fora deles recebe a Exception 02 (Illegal "
            "Data Address), uma função não suportada a Exception 01 e uma quantidade ou valor "
            "inválido a Exception 03, como num equipamento real. Pedidos para outros Slave IDs "
            "são ignorados.\n"
        )
        h(
            "Os valores começam a 0. Duplo clique num valor altera-o (nas Coils e Discrete Inputs "
            "alterna entre 0 e 1); as alterações valem logo, sem passar pelo barramento. Quando o "
            "master escreve (FC05/06/15/16), os registos escritos ficam destacados a amarelo "
            "durante 1,5 s. Janelas sobre os mesmos endereços mostram os mesmos valores. Pedidos "
            "broadcast (Slave 0) de escrita são executados em todos os Slaves simulados que têm "
            "esses endereços, sem resposta.\n"
        )
        h(
            "Formatos por endereço, escala, nomes, Copiar, Base de endereços, seleção com o rato, "
            "Renomear, Destacar e × funcionam como no Master.\n"
        )
        h(
            "Valores automáticos: no clique direito, “Valor automático…” faz os valores "
            "selecionados mudarem sozinhos, para testar trends e alarmes no SCADA. Nos "
            "registos: Contador (soma o passo em cada período e volta ao mínimo depois do "
            "máximo), Rampa (sobe do mínimo ao máximo durante o período), Senoide (oscila "
            "entre o mínimo e o máximo; o período é uma volta) e Aleatório (um valor ao acaso "
            "em cada período). Nas Coils e Discrete Inputs: Alternar 0/1 e Aleatório. O "
            "mínimo e o máximo estão nas unidades mostradas (formato e escala do valor). Os "
            "valores automáticos aparecem a azul, são guardados no workspace e substituem "
            "o que o master ou o utilizador escreverem nesses endereços. “Parar valor "
            "automático” deixa o último valor.\n"
        )
        h(
            "“Abrir no Slave”: no Tráfego do Sniffer (clique direito num pedido FC01–FC06, "
```

MBToolkit v3.5/mb_gui.py — lines 2481-2520

```
            "FC15 ou FC16, ou na resposta correspondente) cria uma janela do Slave com o mesmo "
            "Slave ID, tabela, endereço e quantidade; no Master, o clique direito numa janela "
            "cria uma janela do Slave com a mesma zona, nomes, formatos e escalas. Os "
            "parâmetros série são copiados para a ligação do Slave se este não estiver ligado.\n"
        )
        h(
            "Simulação de falhas: “Falhas…” configura falhas para o Slave ID da janela, que valem "
            "para todas as janelas desse ID: atraso na resposta (ms), não responder, responder "
            "sempre com uma Exception à escolha (01, 02, 03, 04 ou 06), CRC errado nas respostas "
            "e a probabilidade (%) de a falha acontecer em cada pedido. Uma falha ativa aparece a "
            "vermelho no cabeçalho das janelas desse Slave. “Sem falhas” desliga-as.\n"
        )
        h(
            "A barra de ligação mostra os pedidos recebidos, as respostas, as exceções, os "
            "broadcasts e os pedidos para outros IDs; cada janela mostra os pedidos e exceções "
            "sobre os seus endereços e o último pedido. O eco do adaptador RS485 é ignorado. "
            "Antes de responder, o Slave espera o silêncio t3.5; o Windows e o adaptador USB "
            "juntam normalmente alguns ms, por isso um master com timeout muito curto pode dar "
            "timeouts.\n"
        )
        h(
            "Antes de ligar, confirma que nenhum equipamento real no barramento usa os mesmos "
            "Slave IDs: se dois responderem ao mesmo pedido, as respostas corrompem-se. O Slave "
            "nunca funciona ao mesmo tempo que o Sniffer, o Bus Slave Finder ou o Master. Se a "
            "porta falhar, tenta voltar a abri-la a cada 2 s, como o Master.\n"
        )
        h(
            "O menu “Workspace” do Slave guarda só a configuração: ligação, janelas, nomes, "
            "formatos, escalas, valores automáticos e falhas. Os valores dos registos não são guardados e voltam a 0 "
            "ao abrir um workspace. Os ficheiros ficam em %APPDATA%\\MBToolkit\\Workspaces\\Slave; o "
            "último workspace do Slave também é reaberto ao arrancar.\n"
        )

        h("Client e Server — Modbus TCP\n", "heading")
        h(
            "Os separadores Client e Server fazem o mesmo que o Master e o Slave, mas em Modbus "
            "TCP (rede Ethernet ou Wi-Fi) em vez de RS485/RS232. As janelas, formatos, escala, "
            "nomes, escrita, falhas, valores automáticos e workspaces funcionam da mesma forma. "
            "Não usam a porta série, por isso podem correr ao mesmo tempo que o Sniffer, o "
            "Finder, o Master e o Slave, e também ao mesmo tempo um do outro.\n"
```

MBToolkit v3.5/mb_gui.py — lines 2521-2560

```
        )
        h(
            "Client: cada janela tem o IP e a porta TCP do equipamento (502 por defeito) e o Unit "
            "ID (0 a 255). As janelas são servidas uma de cada vez, como no Master, com o timeout "
            "de resposta e o atraso entre pedidos da barra de ligação. A ligação a cada "
            "equipamento fica aberta e volta a ser feita se cair; um equipamento que não responde "
            "à ligação aparece como “Sem ligação”. Em Modbus TCP não há broadcast: o Unit ID 0 é "
            "um Unit ID normal. No clique direito, “Abrir no Server” cria a mesma zona no "
            "separador Server.\n"
        )
        h(
            "Server: “Escutar em” escolhe onde o Server aceita ligações: “Só este PC (127.0.0.1)” "
            "(por defeito; só programas no mesmo PC, sem pedido da firewall), uma das interfaces "
            "de rede do PC (com o IP e o nome da placa; o botão ↻ volta a ler a lista) ou “Todas "
            "as interfaces (0.0.0.0)”. Numa interface de rede é preciso confirmar que o Server "
            "pode ficar acessível nessa rede; da primeira vez, a firewall do Windows pede "
            "autorização. Aceita vários clientes ao mesmo tempo e mostra quantos estão ligados.\n"
        )
        h(
            "No Server, o Unit ID faz o papel do Slave ID (1 a 247). Um pedido para um Unit ID "
            "que não está simulado recebe a Exception 0B (Gateway Target Device Failed to "
            "Respond). Na simulação de falhas, o “CRC errado” do RTU passa a “Cabeçalho MBAP "
            "inválido nas respostas”. Se a porta já estiver a ser usada por outro programa, "
            "aparece um aviso e pode escolher-se outra porta.\n"
        )
        h(
            "Os workspaces do Client e do Server ficam em %APPDATA%\\MBToolkit\\Workspaces\\Client e "
            "\\Server, e o último de cada um é reaberto ao arrancar.\n"
        )

        h("Preferências guardadas\n", "heading")
        h(
            "As preferências do utilizador são guardadas em "
            "%APPDATA%\\MBToolkit\\settings.json. Incluem Light/Dark, último separador, "
            "parâmetros série, Frame gap, Pending timeout, Auto-scroll, Separar transações, "
            "Realçar resultados, opções do Bus Slave Finder e parâmetros de ligação do "
            "Master, do Slave, do Client e do Server. As janelas guardam-se em "
            "workspaces, na pasta %APPDATA%\\MBToolkit\\Workspaces.\n"
        )
        h(
```

MBToolkit v3.5/mb_gui.py — lines 2561-2600

```
            "As portas COM não são guardadas porque são específicas de cada computador. "
            "A confirmação de segurança do Bus Slave Finder também não é persistida.\n"
        )
        h(
            "Até à v3.2 o programa chamava-se MBSniffer. Na primeira vez que o MBToolkit "
            "arranca, copia as preferências e os workspaces de %APPDATA%\\MBSniffer para "
            "%APPDATA%\\MBToolkit; a pasta antiga não é apagada. Os logs novos ficam em "
            "“MBToolkit Logs”; os antigos continuam em “MBSniffer Logs”.\n"
        )

        h("Boas práticas de diagnóstico\n", "heading")
        h("• Confirma primeiro cablagem, polaridade e parâmetros série.\n", "bullet")
        h("• Em RS485, mantém a derivação do sniffer curta e evita alterar a terminação existente.\n", "bullet")
        h("• Garante que nenhum outro programa tem a mesma COM aberta.\n", "bullet")
        h("• Usa CRC ERROR e RAW como indícios de framing, ruído, parâmetros errados ou captura incompleta.\n", "bullet")
        h("• Usa Exceptions como respostas válidas do protocolo: não são equivalentes a CRC error.\n", "bullet")
        h("• Compara tempos de resposta com o comportamento normal do equipamento, não apenas com um limite genérico.\n", "bullet")
        h("• Guarda o TXT quando precisares de análise posterior da captura completa.\n", "bullet")

        if debug_sim == 1:
            h("Modo Simulação\n", "heading")
            h(
                "Este modo de desenvolvimento aparece apenas quando debug_sim = 1 em "
                "MBToolkit.py. Não abre portas série e nunca cria logs TXT.\n"
            )
            h(
                "Cada execução usa sempre o mesmo tipo de cenário de diagnóstico: uma "
                "resposta normal, uma resposta lenta, uma Modbus Exception, um CRC error, "
                "um timeout e um bloco RAW/UNPARSED. Assim é possível comparar diretamente "
                "a mesma informação com e sem destaque visual.\n"
            )
            h(
                "“Realçar resultados” não altera os dados gerados pela Simulação. Desligado, "
                "todas as linhas usam as cores normais da tabela; ligado, a transação normal "
                "concluída fica verde e as linhas de Exception, CRC error, timeout, resposta "
                "lenta e RAW recebem as respetivas cores de diagnóstico.\n"
            )
            h(
                "As frames deste cenário usam o canal SIM para não serem confundidas com uma "
                "captura física real.\n"
```

MBToolkit v3.5/mb_gui.py — lines 2601-2640

```
            )

        h("Limitação importante\n", "heading")
        h(
            "O MBToolkit interpreta bytes, CRC, Function Code, endereços, quantidades, "
            "emparelhamento e tempos observados pelo sistema operativo. Não mede diretamente "
            "níveis elétricos, ringing, reflexões, bias, common-mode, tempos de subida/descida "
            "ou integridade analógica do sinal. Para esses problemas continua a ser necessário "
            "um osciloscópio ou analisador adequado à interface física.\n"
        )

        self.help_text.configure(state="disabled")

        detail = ttk.LabelFrame(self.sniffer_page, text="Frame Inspector")
        detail.grid(row=2, column=0, sticky="ew", padx=10, pady=(0, 10))
        detail.columnconfigure(0, weight=1)

        inspector_header = ttk.Frame(detail)
        inspector_header.grid(
            row=0, column=0, sticky="ew", padx=8, pady=(4, 2)
        )
        inspector_header.columnconfigure(0, weight=1)
        ttk.Label(
            inspector_header,
            text=(
                "Seleciona uma linha no Tráfego para ver a descodificação "
                "estruturada do frame."
            ),
        ).grid(row=0, column=0, sticky="w")

        self.inspector_collapsed = False
        self.inspector_toggle_btn = ttk.Button(
            inspector_header,
            text="Recolher",
            command=self.toggle_frame_inspector,
            style="MBS.Compact.TButton",
        )
        self.inspector_toggle_btn.grid(row=0, column=1, sticky="e")

        self.inspector_body = ttk.Frame(detail)
```

MBToolkit v3.5/mb_gui.py — lines 2641-2680

```
        self.inspector_body.grid(
            row=1, column=0, sticky="ew", padx=8, pady=(2, 7)
        )
        for col in range(4):
            self.inspector_body.columnconfigure(
                col,
                weight=1,
                uniform="inspector",
            )

        self.inspector_vars = {
            "slave": tk.StringVar(value="Slave: —"),
            "type": tk.StringVar(value="Tipo: —"),
            "function": tk.StringVar(value="Function: —"),
            "response": tk.StringVar(value="Resp.: —"),
            "result": tk.StringVar(value="Resultado: —"),
            "pdu": tk.StringVar(value="PDU Address: —"),
            "one_based": tk.StringVar(value="1-based: —"),
            "qty": tk.StringVar(value="Qty: —"),
            "byte_count": tk.StringVar(value="ByteCount: —"),
            "crc": tk.StringVar(value="CRC: —"),
            "data": tk.StringVar(value="Dados: —"),
        }

        inspector_grid = (
            ("slave", 0, 0),
            ("type", 0, 1),
            ("function", 0, 2),
            ("response", 0, 3),
            ("pdu", 1, 0),
            ("one_based", 1, 1),
            ("qty", 1, 2),
            ("byte_count", 1, 3),
        )
        for key, row, col in inspector_grid:
            ttk.Label(
                self.inspector_body,
                textvariable=self.inspector_vars[key],
                anchor="w",
            ).grid(
```

MBToolkit v3.5/mb_gui.py — lines 2681-2720

```
                row=row,
                column=col,
                sticky="ew",
                padx=(0 if col == 0 else 8, 8 if col < 3 else 0),
                pady=2,
            )

        self.inspector_result_label = ttk.Label(
            self.inspector_body,
            textvariable=self.inspector_vars["result"],
            anchor="w",
            style="MBS.Result.neutral.TLabel",
        )
        self.inspector_result_label.grid(
            row=2,
            column=0,
            columnspan=4,
            sticky="ew",
            pady=(3, 2),
        )

        ttk.Label(
            self.inspector_body,
            textvariable=self.inspector_vars["crc"],
            anchor="w",
        ).grid(
            row=3, column=0, columnspan=2, sticky="ew", pady=2
        )
        # Data/Register payloads can be very long. A readonly Entry keeps the
        # inspector compact instead of letting content change requested width.
        ttk.Entry(
            self.inspector_body,
            textvariable=self.inspector_vars["data"],
            state="readonly",
        ).grid(
            row=3,
            column=2,
            columnspan=2,
            sticky="ew",
            padx=(8, 0),
```

MBToolkit v3.5/mb_gui.py — lines 2721-2748

```
            pady=2,
        )

        self.selected_raw_var = tk.StringVar()
        ttk.Entry(
            self.inspector_body,
            textvariable=self.selected_raw_var,
            state="readonly",
        ).grid(
            row=4,
            column=0,
            columnspan=4,
            sticky="ew",
            pady=(4, 0),
        )

        self.build_discovery_tab(self.discovery_page)
        self.build_master_tab(self.master_page)
        self.build_slave_tab(self.slave_page)
        self.client_tab.build_master_tab(self.client_page)
        self.server_tab.build_slave_tab(self.server_page)
        self.update_stats_labels()
        self.update_frame_inspector(None)

        # Classic Tk/Canvas widgets are outside ttk.Style; apply the remembered
        # startup palette after every such widget has been created.
        self._apply_classic_widget_palette(self.initial_dark_mode)

```

MBToolkit v3.5/tests/test_regression.py

787 source lines.

MBToolkit v3.5/tests/test_regression.py — lines 1-40

```
import json
import queue
import random
import tempfile
import threading
import time
import unittest
from collections import deque
from pathlib import Path

import mb_config
import mb_slave_finder
from mb_capture import CaptureMixin
from mb_protocol import (
    SessionMetrics,
    append_crc,
    burst_may_continue,
    crc16_modbus,
    crc_ok,
    modbus_rtu_interframe_gap_seconds,
    parse_burst,
    read_bits_details,
    split_capture_buffer,
)
from mb_slave_finder import SlaveFinderMixin
from mb_view import ViewMixin
from mb_diagnostics import inspector_values
from mb_i18n import (
    LANGUAGE_ENGLISH,
    LANGUAGE_PORTUGUESE,
    english_help_blocks,
    normalize_language,
    translate_details,
    translate_runtime_text,
    translate_text,
)

from mb_slave_finder import (
    build_device_identification_probe,
    build_discovery_probe,
```

MBToolkit v3.5/tests/test_regression.py — lines 41-80

```
    device_identification_timeout_seconds,
    find_device_identification_response,
    find_discovery_response,
    format_device_identification,
    read_basic_device_identification,
)


def with_bad_crc(frame: bytes) -> bytes:
    damaged = bytearray(frame)
    damaged[-1] ^= 0xFF
    return bytes(damaged)


def device_id_response(
    slave=1,
    objects=None,
    *,
    more_follows=False,
    next_object_id=0,
    conformity=0x01,
):
    if objects is None:
        objects = {
            0x00: b"Schneider Electric",
            0x01: b"TEST-PRODUCT",
            0x02: b"1.2.3",
        }
    payload = bytearray((
        slave,
        0x2B,
        0x0E,
        0x01,
        conformity,
        0xFF if more_follows else 0x00,
        next_object_id,
        len(objects),
    ))
    for object_id, value in objects.items():
        value = bytes(value)
```

MBToolkit v3.5/tests/test_regression.py — lines 81-120

```
81          payload.extend((int(object_id) & 0xFF, len(value)))
82          payload.extend(value)
83      return append_crc(bytes(payload))
84
85
86  class FakeSerial:
87      """Minimal non-blocking serial fake for active Device ID regression tests."""
88
89      def __init__(self, response_by_object):
90          self.response_by_object = dict(response_by_object)
91          self.rx = bytearray()
92          self.writes = []
93
94      @property
95      def in_waiting(self):
96          return len(self.rx)
97
98      def reset_input_buffer(self):
99          self.rx.clear()
100
101     def reset_output_buffer(self):
102         pass
103
104     def write(self, data):
105         data = bytes(data)
106         self.writes.append(data)
107         object_id = data[4] if len(data) >= 5 else 0
108         response = self.response_by_object.get(object_id, b"")
109         self.rx.extend(response)
110         return len(data)
111
112     def read(self, size):
113         size = min(int(size), len(self.rx))
114         chunk = bytes(self.rx[:size])
115         del self.rx[:size]
116         return chunk
117
118
119 class FakeSerialModule:
120     class SerialTimeoutException(TimeoutError):
```

MBToolkit v3.5/tests/test_regression.py — lines 121-160

```
121         pass
122
123     class SerialException(OSError):
124         pass
125
126
127 class StaticVar:
128     def __init__(self, value):
129         self.value = value
130
131     def get(self):
132         return self.value
133
134
135 class DummyCapture:
136     calculated_gap_ms = CaptureMixin.calculated_gap_ms
137
138     def __init__(self, *, mode="Auto", baud="9600", data="8", parity="None", stop="1", manual="2.0"):
139         self.gap_mode_var = StaticVar(mode)
140         self.baud_var = StaticVar(baud)
141         self.data_var = StaticVar(data)
142         self.parity_var = StaticVar(parity)
143         self.stop_var = StaticVar(stop)
144         self.gap_ms_var = StaticVar(manual)
145
146
147 class TimingRegressionTests(unittest.TestCase):
148     def test_t35_uses_character_time_at_or_below_19200(self):
149         self.assertAlmostEqual(
150             modbus_rtu_interframe_gap_seconds(9600, 11),
151             3.5 * 11 / 9600,
152             places=9,
153         )
154         self.assertAlmostEqual(
155             modbus_rtu_interframe_gap_seconds(19200, 11),
156             3.5 * 11 / 19200,
157             places=9,
158         )
159
160     def test_t35_is_fixed_above_19200(self):
```

MBToolkit v3.5/tests/test_regression.py — lines 161-200

```
161         for baud in (38400, 57600, 115200):
162             with self.subTest(baud=baud):
163                 self.assertAlmostEqual(
164                     modbus_rtu_interframe_gap_seconds(baud, 10),
165                     0.001750,
166                     places=9,
167                 )
168 
169     def test_sniffer_auto_gap_uses_fixed_1750_us_above_19200(self):
170         capture = DummyCapture(baud="115200", parity="None", stop="1")
171         self.assertAlmostEqual(capture.calculated_gap_ms(), 1.750, places=6)
172 
173     def test_sniffer_manual_gap_is_unchanged(self):
174         capture = DummyCapture(mode="Manual", manual="2,75")
175         self.assertAlmostEqual(capture.calculated_gap_ms(), 2.75, places=6)
176 
177     def test_device_id_timeout_covers_a_full_rtu_frame(self):
178         timeout = device_identification_timeout_seconds(9600, "None", "1", 40)
179         self.assertGreaterEqual(timeout, 256 * 10 / 9600 + 0.001750)
180 
181 
182 class ParserRegressionTests(unittest.TestCase):
183     def test_crc_known_request(self):
184         frame = build_discovery_probe(1, 3)
185         self.assertTrue(crc_ok(frame))
186         self.assertEqual(frame[:6], bytes.fromhex("01 03 00 00 00 01"))
187 
188     def test_invalid_frame_before_valid_frame_stays_crc_error(self):
189         first = with_bad_crc(build_discovery_probe(1, 3))
190         second = build_discovery_probe(2, 4)
191         parts = split_capture_buffer(first + second)
192         self.assertEqual(parts[0], (first, "REQUEST", "ERROR"))
193         self.assertEqual(parts[1], (second, "REQUEST", "OK"))
194 
195     def test_long_unsynchronised_noise_recovers_next_valid_frame(self):
196         noise = bytes([0xFF]) * 4000
197         frame = build_discovery_probe(7, 3)
198         parts = split_capture_buffer(noise + frame)
199         self.assertEqual(parts[-1], (frame, "REQUEST", "OK"))
200         self.assertTrue(parts[0][1].startswith("RAW/"))
```

MBToolkit v3.5/tests/test_regression.py — lines 201-240

```
201         self.assertEqual(sum(len(part[0]) for part in parts), len(noise) + len(frame))
202
203     def test_supported_function_code_shapes(self):
204         frames = {
205             "fc01_req": append_crc(bytes.fromhex("01 01 00 00 00 08")),
206             "fc01_resp": append_crc(bytes.fromhex("01 01 01 5A")),
207             "fc02_req": append_crc(bytes.fromhex("01 02 00 00 00 08")),
208             "fc02_resp": append_crc(bytes.fromhex("01 02 01 A5")),
209             "fc03_req": append_crc(bytes.fromhex("01 03 00 00 00 01")),
210             "fc03_resp": append_crc(bytes.fromhex("01 03 02 00 2A")),
211             "fc04_req": append_crc(bytes.fromhex("01 04 00 00 00 01")),
212             "fc04_resp": append_crc(bytes.fromhex("01 04 02 00 2A")),
213             "fc05": append_crc(bytes.fromhex("01 05 00 01 FF 00")),
214             "fc06": append_crc(bytes.fromhex("01 06 00 01 00 2A")),
215             "fc07_req": append_crc(bytes.fromhex("01 07")),
216             "fc07_resp": append_crc(bytes.fromhex("01 07 AA")),
217             "fc08": append_crc(bytes.fromhex("01 08 00 00 12 34")),
218             "fc15_req": append_crc(bytes.fromhex("01 0F 00 00 00 08 01 5A")),
219             "fc15_resp": append_crc(bytes.fromhex("01 0F 00 00 00 08")),
220             "fc16_req": append_crc(bytes.fromhex("01 10 00 00 00 01 02 00 2A")),
221             "fc16_resp": append_crc(bytes.fromhex("01 10 00 00 00 01")),
222             "fc22": append_crc(bytes.fromhex("01 16 00 01 FF 00 00 2A")),
223             "fc23_req": append_crc(bytes.fromhex(
224                 "01 17 00 00 00 01 00 10 00 01 02 00 2A"
225             )),
226             "fc23_resp": append_crc(bytes.fromhex("01 17 02 00 2A")),
227             "exception": append_crc(bytes.fromhex("01 83 02")),
228         }
229         for name, frame in frames.items():
230             with self.subTest(name=name):
231                 parts = split_capture_buffer(frame)
232                 self.assertEqual(len(parts), 1)
233                 self.assertEqual(parts[0][0], frame)
234                 self.assertEqual(parts[0][2], "OK")
235                 self.assertFalse(parts[0][1].startswith("RAW"))
236
237     def test_random_noise_never_crashes_and_preserves_all_bytes(self):
238         rng = random.Random(20260910)
239         for _ in range(250):
240             length = rng.randint(1, 512)
```

MBToolkit v3.5/tests/test_regression.py — lines 241-280

```
            data = bytes(rng.randrange(256) for _ in range(length))
            parts = split_capture_buffer(data)
            self.assertEqual(sum(len(part[0]) for part in parts), len(data))
            self.assertTrue(all(len(part[0]) > 0 for part in parts))

    def test_request_response_pairing_and_response_time(self):
        req = parse_burst(
            append_crc(bytes.fromhex("01 03 00 00 00 01")),
            "12:00:00.000",
            "BUS",
            100.000,
        )[0]
        resp = parse_burst(
            append_crc(bytes.fromhex("01 03 02 00 2A")),
            "12:00:00.125",
            "BUS",
            100.125,
        )[0]
        req["_view_seq"] = 10
        resp["_view_seq"] = 11

        metrics = SessionMetrics(pending_timeout_seconds=1.0)
        first = metrics.process(req)
        second = metrics.process(resp)

        self.assertEqual(first["pending"], 1)
        self.assertEqual(second["pending"], 0)
        self.assertEqual(second["matched_request"]["seq"], 10)
        self.assertAlmostEqual(second["response_ms"], 125.0, places=6)

    def test_timeout_regression(self):
        req = parse_burst(
            append_crc(bytes.fromhex("05 04 00 00 00 01")),
            "12:00:00.000",
            "BUS",
            10.0,
        )[0]
        req["_view_seq"] = 77
        metrics = SessionMetrics(pending_timeout_seconds=1.0)
        metrics.process(req)
```

MBToolkit v3.5/tests/test_regression.py — lines 281-320

```
281         self.assertEqual(metrics.expire_pending(11.1), 1)
282         expired = metrics.pop_expired_requests()
283         self.assertEqual(expired[0]["seq"], 77)
284         self.assertEqual(metrics.timeouts, 1)
285
286
287 class SlaveFinderRegressionTests(unittest.TestCase):
288     def test_normal_discovery_response(self):
289         response = append_crc(bytes.fromhex("04 03 02 12 34"))
290         found = find_discovery_response(response, 4, 3)
291         self.assertIsNotNone(found)
292         self.assertEqual(found["kind"], "RESPONSE")
293
294     def test_discovery_exception_counts_as_response(self):
295         response = append_crc(bytes.fromhex("04 83 02"))
296         found = find_discovery_response(response, 4, 3)
297         self.assertIsNotNone(found)
298         self.assertEqual(found["kind"], "EXCEPTION")
299         self.assertEqual(found["exception_code"], 2)
300
301     def test_device_identification_request(self):
302         request = build_device_identification_probe(4)
303         self.assertEqual(request[:5], bytes.fromhex("04 2B 0E 01 00"))
304         self.assertEqual(len(request), 7)
305         self.assertTrue(crc_ok(request))
306
307     def test_device_identification_response_parser(self):
308         response = device_id_response(slave=4)
309         found = find_device_identification_response(response, 4)
310         self.assertIsNotNone(found)
311         self.assertEqual(found["kind"], "DEVICE_ID")
312         self.assertEqual(found["objects"][0x00], "Schneider Electric")
313         self.assertEqual(found["objects"][0x01], "TEST-PRODUCT")
314         self.assertEqual(found["objects"][0x02], "1.2.3")
315         self.assertEqual(
316             format_device_identification(found),
317             "Schneider Electric | TEST-PRODUCT | 1.2.3",
318         )
319
320     def test_device_identification_parser_ignores_request_echo(self):
```

MBToolkit v3.5/tests/test_regression.py — lines 321-360

```
        request = build_device_identification_probe(4)
        response = device_id_response(slave=4)
        found = find_device_identification_response(request + response, 4)
        self.assertIsNotNone(found)
        self.assertEqual(found["objects"][0x01], "TEST-PRODUCT")

    def test_device_identification_exception(self):
        response = append_crc(bytes.fromhex("04 AB 01"))
        found = find_device_identification_response(response, 4)
        self.assertIsNotNone(found)
        self.assertEqual(found["kind"], "EXCEPTION")
        self.assertEqual(found["exception_code"], 1)
        self.assertIn("Illegal Function", format_device_identification(found))

    def test_device_identification_rejects_bad_crc(self):
        response = with_bad_crc(device_id_response(slave=4))
        self.assertIsNone(find_device_identification_response(response, 4))

    def test_device_identification_follows_more_follows(self):
        page0 = device_id_response(
            slave=4,
            objects={0x00: b"Vendor"},
            more_follows=True,
            next_object_id=0x01,
        )
        page1 = device_id_response(
            slave=4,
            objects={0x01: b"Product", 0x02: b"2.0"},
            more_follows=False,
            next_object_id=0x00,
        )
        serial = FakeSerial({0x00: page0, 0x01: page1})
        result = read_basic_device_identification(
            serial,
            4,
            115200,
            "None",
            "1",
            10,
            threading.Event(),
```

MBToolkit v3.5/tests/test_regression.py — lines 361-400

```
361             FakeSerialModule,
362         )
363         self.assertIsNotNone(result)
364         self.assertEqual(result["objects"][0x00], "Vendor")
365         self.assertEqual(result["objects"][0x01], "Product")
366         self.assertEqual(result["objects"][0x02], "2.0")
367         self.assertEqual(len(serial.writes), 2)
368         self.assertEqual(serial.writes[0][4], 0x00)
369         self.assertEqual(serial.writes[1][4], 0x01)
370
371
372 class InternationalizationRegressionTests(unittest.TestCase):
373     def test_language_normalization_defaults_to_portuguese(self):
374         self.assertEqual(normalize_language("invalid"), LANGUAGE_PORTUGUESE)
375         self.assertEqual(normalize_language(LANGUAGE_ENGLISH), LANGUAGE_ENGLISH)
376
377     def test_static_ui_translation_is_reversible(self):
378         self.assertEqual(translate_text("Configuração", LANGUAGE_ENGLISH), "Configuration")
379         self.assertEqual(translate_text("Configuration", LANGUAGE_PORTUGUESE), "Configuração")
380         self.assertEqual(translate_text("Frame gap", LANGUAGE_ENGLISH), "Frame gap")
381         # "None" is a technical parity value and must remain English in the PT UI.
382         self.assertEqual(translate_text("None", LANGUAGE_PORTUGUESE), "None")
383         self.assertEqual(translate_text("Nenhum", LANGUAGE_ENGLISH), "None")
384
385     def test_dynamic_status_translation_is_reversible(self):
386         pt = "Pesquisa concluída — 7 encontrados — 00:12"
387         en = "Scan complete — 7 found — 00:12"
388         self.assertEqual(translate_runtime_text(pt, LANGUAGE_ENGLISH), en)
389         self.assertEqual(translate_runtime_text(en, LANGUAGE_PORTUGUESE), pt)
390
391     def test_register_details_follow_language(self):
392         self.assertEqual(
393             translate_details("ByteCount=2  Registos=[42]", LANGUAGE_ENGLISH),
394             "ByteCount=2  Registers=[42]",
395         )
396         self.assertEqual(
397             translate_details("ByteCount=2  Registers=[42]", LANGUAGE_PORTUGUESE),
398             "ByteCount=2  Registos=[42]",
399         )
400
```

MBToolkit v3.5/tests/test_regression.py — lines 401-440

```
    def test_inspector_english_labels_and_crc(self):
        frame = append_crc(bytes.fromhex("01 03 02 00 2A"))
        row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
        row.update({"response_text": "12.3", "matched": True})
        values = inspector_values(row, language=LANGUAGE_ENGLISH)
        self.assertTrue(values["type"].startswith("Type:"))
        self.assertTrue(values["result"].startswith("Result:"))
        self.assertTrue(values["data"].startswith("Data:"))
        self.assertIn("Registers=[42]", values["data"])
        self.assertIn("received", values["crc"])
        self.assertIn("calculated", values["crc"])

    def test_english_help_contains_no_portuguese_diacritics(self):
        text = "".join(
            block for _tag, block in english_help_blocks(
                "20 000", "20 000", 1.0, 300.0, debug_sim=True
            )
        )
        for character in "ãõçáéíóúâê":
            self.assertNotIn(character, text.casefold())
        self.assertIn("Bus Slave Finder — active scan", text)
        self.assertIn("Device Identification (FC43/14)", text)


def bitwise_crc16(data: bytes) -> int:
    """Reference CRC used up to v3.0, kept to verify the table-driven CRC."""
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc


def process_frames(metrics, frames):
    """Run (bytes, channel, epoch) frames through SessionMetrics."""
    out = []
    for index, (frame, channel, epoch) in enumerate(frames):
        for row in parse_burst(frame, "12:00:00.000", channel, epoch):
            row["_view_seq"] = index
```

MBToolkit v3.5/tests/test_regression.py — lines 441-480

```
            out.append((row, metrics.process(row)))
    return out


class ProtocolV31RegressionTests(unittest.TestCase):
    def test_crc_table_matches_bitwise_reference(self):
        rng = random.Random(31)
        for length in (0, 1, 2, 7, 64, 256):
            data = bytes(rng.randrange(256) for _ in range(length))
            self.assertEqual(crc16_modbus(data), bitwise_crc16(data))

    def test_broadcast_request_never_times_out(self):
        request = append_crc(bytes.fromhex("00 10 00 00 00 01 02 00 05"))
        metrics = SessionMetrics(pending_timeout_seconds=1.0)
        process_frames(metrics, [(request, "BUS", 10.0)])
        self.assertEqual(metrics.requests, 1)
        self.assertEqual(metrics.broadcast_requests, 1)
        self.assertEqual(metrics.pending_count(), 0)
        metrics.expire_pending(20.0)
        self.assertEqual(metrics.timeouts, 0)
        self.assertEqual(metrics.health_summary(now=20.0)["timeout_rate_pct"], 0.0)

    def test_fc06_echo_pair_is_request_then_response(self):
        write = append_crc(bytes.fromhex("01 06 00 01 00 09"))
        metrics = SessionMetrics(pending_timeout_seconds=1.0)
        results = process_frames(
            metrics, [(write, "BUS", 100.0), (write, "BUS", 100.040)]
        )
        self.assertEqual(
            [row["type"] for row, _timing in results], ["REQUEST", "RESPONSE"]
        )
        self.assertEqual((metrics.requests, metrics.responses), (1, 1))
        self.assertAlmostEqual(results[1][1]["response_ms"], 40.0, places=6)

    def test_unanswered_fc05_write_times_out(self):
        write = append_crc(bytes.fromhex("02 05 00 10 FF 00"))
        metrics = SessionMetrics(pending_timeout_seconds=1.0)
        results = process_frames(metrics, [(write, "BUS", 5.0)])
        self.assertEqual(results[0][0]["type"], "REQUEST")
        self.assertEqual(metrics.expire_pending(6.5), 1)
```

MBToolkit v3.5/tests/test_regression.py — lines 481-520

```
481
482     def test_different_fc06_writes_are_both_requests(self):
483         first = append_crc(bytes.fromhex("01 06 00 01 00 09"))
484         second = append_crc(bytes.fromhex("01 06 00 02 00 0A"))
485         metrics = SessionMetrics()
486         results = process_frames(
487             metrics, [(first, "BUS", 1.0), (second, "BUS", 1.1)]
488         )
489         self.assertEqual(
490             [row["type"] for row, _timing in results], ["REQUEST", "REQUEST"]
491         )
492
493     def test_dual_rx_echo_must_come_from_opposite_channel(self):
494         write = append_crc(bytes.fromhex("01 06 00 01 00 09"))
495         metrics = SessionMetrics()
496         results = process_frames(metrics, [
497             (write, "A→B", 1.0),
498             (write, "A→B", 1.5),
499             (write, "B→A", 1.52),
500         ])
501         self.assertEqual(
502             [row["type"] for row, _timing in results],
503             ["REQUEST", "REQUEST", "RESPONSE"],
504         )
505
506     def test_split_frame_is_held_until_complete(self):
507         response = append_crc(bytes.fromhex("01 03 08") + bytes(8))
508         self.assertTrue(burst_may_continue(response[:7]))
509         self.assertTrue(burst_may_continue(response[:1]))
510         self.assertFalse(burst_may_continue(response))
511         self.assertFalse(burst_may_continue(bytes((0xFF,) * 6)))
512
513     def test_merged_frames_keep_their_chunk_arrival_times(self):
514         request = append_crc(bytes.fromhex("01 03 00 00 00 01"))
515         response = append_crc(bytes.fromhex("01 03 02 00 2A"))
516         rows = parse_burst(
517             request + response,
518             "12:00:00.000",
519             "BUS",
520             100.0,
```

MBToolkit v3.5/tests/test_regression.py — lines 521-560

```
            chunk_times=[(0, 100.0), (len(request), 100.030)],
        )
        self.assertEqual([row["type"] for row in rows], ["REQUEST", "RESPONSE"])
        self.assertAlmostEqual(rows[1]["_epoch"], 100.030, places=6)
        metrics = SessionMetrics()
        metrics.process(rows[0])
        self.assertAlmostEqual(metrics.process(rows[1])["response_ms"], 30.0, places=6)

    def test_read_bits_details_use_request_qty(self):
        response = append_crc(bytes.fromhex("01 01 02 CD 01"))
        self.assertEqual(
            read_bits_details(response),
            "ByteCount=2  Bits=10110011 10000000",
        )
        self.assertEqual(
            read_bits_details(response, qty=10),
            "ByteCount=2  Bits=10110011 10",
        )


class _ChunkedSerial:
    """Delivers byte chunks at fixed delays, like a USB latency timer."""

    def __init__(self, chunks, stop_event):
        self.started = time.perf_counter()
        self.chunks = list(chunks)  # [(delay_s, bytes)]
        self.stop_event = stop_event

    def _due(self):
        elapsed = time.perf_counter() - self.started
        if self.chunks and elapsed >= self.chunks[0][0]:
            return self.chunks[0][1]
        if not self.chunks and elapsed > 0.3:
            self.stop_event.set()
        return b""

    @property
    def in_waiting(self):
        return len(self._due())

```

MBToolkit v3.5/tests/test_regression.py — lines 561-600

```
    def read(self, size):
        data = self._due()
        if data:
            self.chunks.pop(0)
        return data

    def close(self):
        pass


class _ReaderStub(CaptureMixin):
    def __init__(self):
        self.stop_event = threading.Event()
        self.event_queue = queue.Queue()


class ReaderV31RegressionTests(unittest.TestCase):
    def test_frame_split_by_usb_latency_is_reassembled(self):
        response = append_crc(bytes.fromhex("01 03 08") + bytes(8))
        reader = _ReaderStub()
        serial = _ChunkedSerial(
            [(0.0, response[:7]), (0.016, response[7:])], reader.stop_event
        )
        # 115200 bit/s: Auto t3.5 is 1.75 ms, far shorter than the 16 ms split.
        reader.reader_loop(serial, 1.75, "BUS", 1)
        rows = []
        while not reader.event_queue.empty():
            name, payload = reader.event_queue.get()
            if name == "frame":
                rows.append(payload)
        self.assertEqual([(row["type"], row["crc"]) for row in rows], [("RESPONSE", "OK")])


class _FakeVar:
    def __init__(self, value):
        self.value = value

    def get(self):
        return self.value

```

MBToolkit v3.5/tests/test_regression.py — lines 601-640

```
601
602 class _FakeTree:
603     def __init__(self):
604         self.tags = {}
605
606     def item(self, iid, tags=()):
607         self.tags[iid] = tags
608
609
610 class _TimeoutView(ViewMixin):
611     def __init__(self, type_filter):
612         self.tree = _FakeTree()
613         self.advanced_filter_type_var = _FakeVar(type_filter)
614         self.highlight_anomalies_var = _FakeVar(True)
615         self._successful_request_seqs = set()
616         self._frame_history = deque([{
617             "seq": 5, "type": "REQUEST", "crc": "OK", "timed_out": False,
618             "response_text": "",
619         }])
620         self._tree_iids_by_seq = {5: ["I001", "I002"]}
621         self.full_renders = 0
622
623     def render_filtered_view(self):
624         self.full_renders += 1
625
626     def selected_record(self):
627         return None
628
629
630 class ViewV31RegressionTests(unittest.TestCase):
631     def test_timeout_retags_rows_without_full_rebuild(self):
632         view = _TimeoutView("Todos")
633         self.assertEqual(view.mark_timed_out_records([{"seq": 5}]), 1)
634         self.assertEqual(view.full_renders, 0)
635         self.assertEqual(view.tree.tags["I001"], ("anomaly_timeout",))
636         self.assertEqual(view.tree.tags["I002"], ("anomaly_timeout",))
637
638     def test_timeouts_filter_still_rebuilds_view(self):
639         view = _TimeoutView("Timeouts")
640         view.mark_timed_out_records([{"seq": 5}])
```

MBToolkit v3.5/tests/test_regression.py — lines 641-680

```
641         self.assertEqual(view.full_renders, 1)
642
643     def test_inspector_shows_address_of_each_register(self):
644         frame = append_crc(bytes.fromhex("01 03 04 00 2A 00 2B"))
645         row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
646         row["request_pdu_address"] = 100
647         pt = inspector_values(row, language=LANGUAGE_PORTUGUESE)
648         en = inspector_values(row, language=LANGUAGE_ENGLISH)
649         self.assertIn("Registos (PDU Address=valor): 100=42, 101=43", pt["data"])
650         self.assertIn("Registers (PDU Address=value): 100=42, 101=43", en["data"])
651
652     def test_inspector_shows_address_of_each_bit(self):
653         frame = append_crc(bytes.fromhex("01 02 01 05"))
654         row = parse_burst(frame, "12:00:00.000", "BUS", 1.0)[0]
655         row.update({"request_pdu_address": 8, "request_qty": 3})
656         values = inspector_values(row, language=LANGUAGE_PORTUGUESE)
657         self.assertIn("Bits (PDU Address=valor): 8=1, 9=0, 10=1", values["data"])
658
659
660 class _FakeFinderSerial:
661     def __init__(self, incoming=b""):
662         self.incoming = bytearray(incoming)
663         self.writes = []
664
665     @property
666     def in_waiting(self):
667         return len(self.incoming)
668
669     def read(self, size):
670         data = bytes(self.incoming[:size])
671         del self.incoming[:size]
672         return data
673
674     def write(self, data):
675         self.writes.append(bytes(data))
676         return len(data)
677
678     def reset_input_buffer(self):
679         pass
680
```

MBToolkit v3.5/tests/test_regression.py — lines 681-720

```
681     def reset_output_buffer(self):
682         pass
683 
684     def close(self):
685         pass
686 
687 
688 class _FinderStub(SlaveFinderMixin):
689     current_language = LANGUAGE_PORTUGUESE
690 
691     def __init__(self, fake_serial):
692         self.fake_serial = fake_serial
693         self.event_queue = queue.Queue()
694         self.discovery_stop_event = threading.Event()
695         self.discovery_serial = None
696 
697     def tr(self, text):
698         return translate_text(text, self.current_language)
699 
700     def trf(self, text, **values):
701         return self.tr(text).format(**values)
702 
703     def open_discovery_serial_port(self, serial, port, baud, parity, stopbits):
704         return self.fake_serial
705 
706     def run_worker(self, slaves, attempts):
707         self.discovery_worker(
708             None, "COM9", [(115200, "None", "1")], slaves, 10, False, False,
709             attempts,
710         )
711         events = []
712         while not self.event_queue.empty():
713             events.append(self.event_queue.get())
714         return events
715 
716 
717 class FinderV31RegressionTests(unittest.TestCase):
718     def setUp(self):
719         self._idle_check = mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS
720         mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS = 0.05
```

MBToolkit v3.5/tests/test_regression.py — lines 721-760

```
721
722     def tearDown(self):
723         mb_slave_finder.DISCOVERY_IDLE_CHECK_SECONDS = self._idle_check
724
725     def test_each_silent_slave_gets_all_attempts(self):
726         stub = _FinderStub(_FakeFinderSerial())
727         events = stub.run_worker([1, 2], attempts=3)
728         done = [payload for name, payload in events if name == "discovery_done"][0]
729         self.assertIsNone(done["error"])
730         self.assertEqual(len(stub.fake_serial.writes), 6)
731
732     def test_traffic_before_scan_blocks_transmission(self):
733         stub = _FinderStub(_FakeFinderSerial(b"\x01\x03\x00"))
734         events = stub.run_worker([1], attempts=1)
735         done = [payload for name, payload in events if name == "discovery_done"][0]
736         self.assertIn("Foi detetado tráfego em COM9", done["error"])
737         self.assertEqual(stub.fake_serial.writes, [])
738
739     def test_legacy_default_timeout_migrates(self):
740         with tempfile.TemporaryDirectory() as folder:
741             path = Path(folder) / "settings.json"
742             original = mb_config.get_settings_file
743             mb_config.get_settings_file = lambda: path
744             try:
745                 path.write_text(
746                     json.dumps({"bus_slave_finder": {"min_timeout_ms": 40}}),
747                     encoding="utf-8",
748                 )
749                 finder = mb_config.load_ui_settings()["bus_slave_finder"]
750                 self.assertEqual(finder["min_timeout_ms"], "100")
751                 self.assertEqual(finder["attempts"], "1")
752
753                 path.write_text(
754                     json.dumps({"bus_slave_finder": {
755                         "min_timeout_ms": 40, "attempts": 2,
756                     }}),
757                     encoding="utf-8",
758                 )
759                 finder = mb_config.load_ui_settings()["bus_slave_finder"]
760                 self.assertEqual(finder["min_timeout_ms"], "40")
```

MBToolkit v3.5/tests/test_regression.py — lines 761-787

```
761                 self.assertEqual(finder["attempts"], "2")
762             finally:
763                 mb_config.get_settings_file = original
764 
765 
766 class SourceHygieneV31Tests(unittest.TestCase):
767     def test_no_inline_language_branches(self):
768         package = Path(__file__).resolve().parents[1]
769         for source in package.glob("*.py"):
770             text = source.read_text(encoding="utf-8")
771             self.assertNotIn(
772                 'current_language == "English"', text, source.name
773             )
774 
775     def test_message_templates_translate(self):
776         self.assertEqual(
777             translate_text("Encontrados: {count}", LANGUAGE_ENGLISH).format(count=3),
778             "Found: 3",
779         )
780         self.assertEqual(
781             translate_text("A verificar se o barramento está livre…", LANGUAGE_ENGLISH),
782             "Checking that the bus is idle…",
783         )
784 
785 
786 if __name__ == "__main__":
787     unittest.main(verbosity=2)
```

MBToolkit v3.5/tests/test_master.py

602 source lines.

MBToolkit v3.5/tests/test_master.py — lines 1-40

```python
import json
import os
import queue
import struct
import tempfile
import time
import unittest
from pathlib import Path

import mb_config
import mb_master_ui
import mb_table_ui
from mb_master import (
    BYTE_ORDERS,
    BYTE_ORDERS_64,
    format_value,
    format_window_values,
    raw_text_for_scaled,
    register_groups,
    validate_scale,
    MasterEngine,
    PollDefinition,
    build_read_request,
    build_write_request,
    encode_coil_value,
    encode_register_value,
    find_master_response,
    format_register_values,
)
from mb_master_ui import MasterMixin, MasterWindow, apply_format_to_rows
from mb_protocol import append_crc, crc_ok


class FakeSlaveSerial:
    """In-memory Modbus RTU slave behind a fake serial port.

    Answers FC01/03 reads and FC05/06/15/16 writes for ``slave_id`` from its
    coil/register banks, can echo every request like some RS485 adapters, and
    can hold bytes that are already on the bus before the master transmits.
    """
```

MBToolkit v3.5/tests/test_master.py — lines 41-80

```
41
42      def __init__(self, slave_id=1, *, echo=False, bus_traffic=b"", exception_for=None):
43          self.slave_id = slave_id
44          self.echo = echo
45          self.exception_for = exception_for or {}
46          self.registers = {i: i * 10 for i in range(200)}
47          self.coils = {i: i % 2 for i in range(200)}
48          self.rx = bytearray(bus_traffic)
49          self.requests = []
50
51      @property
52      def in_waiting(self):
53          return len(self.rx)
54
55      def read(self, size):
56          data = bytes(self.rx[:size])
57          del self.rx[:size]
58          return data
59
60      def reset_input_buffer(self):
61          self.rx.clear()
62
63      def reset_output_buffer(self):
64          pass
65
66      def cancel_read(self):
67          pass
68
69      def cancel_write(self):
70          pass
71
72      def close(self):
73          pass
74
75      def write(self, data):
76          data = bytes(data)
77          self.requests.append(data)
78          if self.echo:
79              self.rx.extend(data)
80          if crc_ok(data) and data[0] == self.slave_id:
```

MBToolkit v3.5/tests/test_master.py — lines 81-120

```
            self.rx.extend(self._answer(data))
        elif crc_ok(data) and data[0] == 0:
            self._answer(data)  # broadcast: execute, never answer
        return len(data)

    def _answer(self, req):
        fc = req[1]
        if fc in self.exception_for:
            return append_crc(bytes((req[0], fc | 0x80, self.exception_for[fc])))
        address = (req[2] << 8) | req[3]
        qty = (req[4] << 8) | req[5]
        if fc == 3:
            payload = b"".join(struct.pack(">H", self.registers.get(address + i, 0))
                               for i in range(qty))
            return append_crc(bytes((req[0], 3, len(payload))) + payload)
        if fc == 1:
            data = bytearray((qty + 7) // 8)
            for i in range(qty):
                if self.coils.get(address + i):
                    data[i // 8] |= 1 << (i % 8)
            return append_crc(bytes((req[0], 1, len(data))) + bytes(data))
        if fc == 6:
            self.registers[address] = qty
            return req
        if fc == 5:
            self.coils[address] = 1 if qty == 0xFF00 else 0
            return req
        if fc == 16:
            for i in range(qty):
                self.registers[address + i] = (req[7 + 2 * i] << 8) | req[8 + 2 * i]
            return append_crc(req[:6])
        if fc == 15:
            for i in range(qty):
                self.coils[address + i] = (req[7 + i // 8] >> (i % 8)) & 1
            return append_crc(req[:6])
        return append_crc(bytes((req[0], fc | 0x80, 1)))


def drain(event_queue):
    events = []
```

MBToolkit v3.5/tests/test_master.py — lines 121-160

```
121     while not event_queue.empty():
122         events.append(event_queue.get())
123     return events
124 
125 
126 def make_engine(serial, **overrides):
127     options = dict(baud=115200, parity="None", stopbits="1",
128                    response_timeout_ms=30, delay_ms=0)
129     options.update(overrides)
130     return MasterEngine(serial, queue.Queue(), **options)
131 
132 
133 class MasterFormatTests(unittest.TestCase):
134     def test_32_bit_formats_round_trip_in_every_byte_order(self):
135         for order in BYTE_ORDERS:
136             for kind, text in (("int32", "-123456"), ("uint32", "4000000000"),
137                                ("float32", "123.456")):
138                 fmt = f"{kind}_{order}"
139                 registers = encode_register_value(text, fmt)
140                 self.assertEqual(len(registers), 2)
141                 self.assertEqual(format_register_values(registers, fmt)[0], text)
142 
143     def test_float_byte_orders_match_wire_layout(self):
144         big_endian = struct.pack(">f", 1.5)  # 3F C0 00 00 = A B C D
145         self.assertEqual(encode_register_value("1.5", "float32_ABCD"), [0x3FC0, 0x0000])
146         self.assertEqual(encode_register_value("1.5", "float32_CDAB"), [0x0000, 0x3FC0])
147         self.assertEqual(encode_register_value("1.5", "float32_BADC"), [0xC03F, 0x0000])
148         self.assertEqual(encode_register_value("1.5", "float32_DCBA"), [0x0000, 0xC03F])
149         self.assertEqual(big_endian, bytes.fromhex("3FC00000"))
150 
151     def test_16_bit_formats(self):
152         self.assertEqual(format_register_values([0xFFFE], "int16"), ["-2"])
153         self.assertEqual(format_register_values([0xFFFE], "uint16"), ["65534"])
154         self.assertEqual(format_register_values([0x00AB], "hex16"), ["0x00AB"])
155         self.assertEqual(format_register_values([0x0105], "bin16"), ["00000001 00000101"])
156         self.assertEqual(encode_register_value("-2", "int16"), [0xFFFE])
157         self.assertEqual(encode_register_value("0x1A2B", "hex16"), [0x1A2B])
158         self.assertEqual(encode_register_value("0000 0001 0000 0101", "bin16"), [0x0105])
159 
160     def test_values_outside_the_format_are_rejected(self):
```

MBToolkit v3.5/tests/test_master.py — lines 161-200

```
161         for text, fmt in (("40000", "int16"), ("-1", "uint16"), ("abc", "float32_ABCD"),
162                           ("1e40", "float32_ABCD"), ("", "int16")):
163             with self.assertRaises(ValueError):
164                 encode_register_value(text, fmt)
165         with self.assertRaises(ValueError):
166             encode_coil_value("2")
167 
168     def test_unpaired_register_in_32_bit_format(self):
169         self.assertEqual(format_register_values([1, 2, 3], "uint32_ABCD"), ["65538", "", "—"])
170 
171 
172 class MasterFrameTests(unittest.TestCase):
173     def test_write_frames_match_modbus_specification_examples(self):
174         # Modbus Application Protocol V1.1b3 examples (CRC appended for RTU).
175         self.assertEqual(build_write_request(0x11, 16, 1, [0x000A, 0x0102])[:-2],
176                          bytes.fromhex("11 10 0001 0002 04 000A 0102"))
177         coils = [1, 0, 1, 1, 0, 0, 1, 1, 1, 0]
178         self.assertEqual(build_write_request(0x11, 15, 0x13, coils)[:-2],
179                          bytes.fromhex("11 0F 0013 000A 02 CD 01"))
180         self.assertEqual(build_write_request(0x11, 5, 0xAC, [1])[:-2],
181                          bytes.fromhex("11 05 00AC FF00"))
182         self.assertEqual(build_write_request(0x11, 6, 1, [3])[:-2],
183                          bytes.fromhex("11 06 0001 0003"))
184 
185     def test_invalid_definitions_are_rejected(self):
186         for args in ((0, 3, 0, 1), (1, 7, 0, 1), (1, 3, 0, 126), (1, 1, 0, 2001),
187                      (1, 3, 65535, 2)):
188             with self.assertRaises(ValueError):
189                 build_read_request(*args)
190         with self.assertRaises(ValueError):
191             build_write_request(1, 6, 0, [1, 2])
192 
193     def test_read_response_found_after_adapter_echo(self):
194         request = build_read_request(1, 3, 0, 2)
195         response = append_crc(bytes.fromhex("01 03 04 0001 0002"))
196         found = find_master_response(request + response, request)
197         self.assertEqual(found["kind"], "RESPONSE")
198         self.assertEqual(found["values"], [1, 2])
199 
200     def test_coil_response_is_trimmed_to_quantity(self):
```

MBToolkit v3.5/tests/test_master.py — lines 201-240

```
        request = build_read_request(1, 1, 0, 3)
        found = find_master_response(append_crc(bytes.fromhex("01 01 01 FD")), request)
        self.assertEqual(found["values"], [1, 0, 1])

    def test_exception_and_bad_crc(self):
        request = build_read_request(1, 3, 0, 1)
        exception = append_crc(bytes.fromhex("01 83 02"))
        self.assertEqual(find_master_response(exception, request)["exception_code"], 2)
        damaged = bytearray(append_crc(bytes.fromhex("01 03 02 0001")))
        damaged[-1] ^= 0xFF
        self.assertIsNone(find_master_response(bytes(damaged), request))

    def test_write_response_must_match_request(self):
        request = build_write_request(1, 16, 5, [1, 2])
        self.assertIsNotNone(find_master_response(append_crc(request[:6]), request))
        other = append_crc(bytes.fromhex("01 10 0006 0002"))
        self.assertIsNone(find_master_response(other, request))


class MasterEngineTests(unittest.TestCase):
    def run_engine(self, engine, seconds):
        engine.start()
        time.sleep(seconds)
        engine.stop()
        return drain(engine.event_queue)

    def test_scheduler_serves_writes_then_read_once_then_due_polls(self):
        engine = make_engine(FakeSlaveSerial())
        engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 1000))
        engine.set_poll(PollDefinition(2, 1, 3, 0, 1, 1000))
        engine.read_once(2)
        engine.queue_write(1, build_write_request(1, 6, 0, [5]))
        self.assertEqual(engine.next_job(now=10.0)[0], "write")
        self.assertEqual(engine.next_job(now=10.0)[1:], (engine._polls[2][0], True))
        first = engine.next_job(now=10.0)
        second = engine.next_job(now=10.0)
        self.assertEqual({first[1].window_id, second[1].window_id}, {1, 2})
        self.assertEqual(engine.next_job(now=10.5)[0], "wait")
        self.assertEqual(engine.next_job(now=11.01)[0], "read")

```

MBToolkit v3.5/tests/test_master.py — lines 241-280

```
241     def test_fast_and_slow_windows_share_one_bus(self):
242         serial = FakeSlaveSerial()
243         engine = make_engine(serial)
244         engine.set_poll(PollDefinition(1, 1, 3, 0, 4, 0))
245         engine.set_poll(PollDefinition(2, 1, 1, 0, 8, 10000))
246         events = self.run_engine(engine, 0.4)
247         results = [p for name, p in events if name == "master_result"]
248         fast = [r for r in results if r["window_id"] == 1]
249         slow = [r for r in results if r["window_id"] == 2]
250         self.assertGreater(len(fast), 3)
251         self.assertEqual(len(slow), 1)
252         self.assertEqual(fast[0]["values"], [0, 10, 20, 30])
253         self.assertEqual(slow[0]["values"], [0, 1, 0, 1, 0, 1, 0, 1])
254         self.assertEqual(events[0][0], "master_ready")
255         self.assertEqual(events[-1], ("master_stopped", {"error": None}))
256 
257     def test_write_is_executed_and_reported(self):
258         serial = FakeSlaveSerial(echo=True)
259         engine = make_engine(serial)
260         engine.queue_write(1, build_write_request(1, 16, 10, [0xABCD, 0x1234]), "FC16")
261         results = [p for n, p in self.run_engine(engine, 0.15) if n == "master_result"]
262         self.assertEqual(results[0]["status"], "ok")
263         self.assertEqual((serial.registers[10], serial.registers[11]), (0xABCD, 0x1234))
264 
265     def test_broadcast_write_does_not_wait_for_a_response(self):
266         serial = FakeSlaveSerial()
267         engine = make_engine(serial, response_timeout_ms=5000)
268         engine.queue_write(1, build_write_request(0, 6, 3, [77]))
269         started = time.perf_counter()
270         result = engine.transact(engine._writes.popleft()[1])
271         self.assertLess(time.perf_counter() - started, 1.0)
272         self.assertEqual(result["status"], "broadcast")
273         self.assertEqual(serial.registers[3], 77)
274 
275     def test_timeout_exception_and_invalid_results(self):
276         engine = make_engine(FakeSlaveSerial(slave_id=9))
277         self.assertEqual(engine.transact(build_read_request(1, 3, 0, 1))["status"], "timeout")
278         engine = make_engine(FakeSlaveSerial(exception_for={3: 2}))
279         result = engine.transact(build_read_request(1, 3, 0, 1))
280         self.assertEqual((result["status"], result["exception_code"]), ("exception", 2))
```

MBToolkit v3.5/tests/test_master.py — lines 281-320

```
281
282     def test_traffic_on_the_bus_blocks_the_master(self):
283         serial = FakeSlaveSerial(bus_traffic=b"\x01\x03\x00")
284         engine = make_engine(serial, idle_check_seconds=0.05)
285         engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
286         events = self.run_engine(engine, 0.2)
287         self.assertEqual(serial.requests, [])
288         self.assertEqual(events[-1][0], "master_stopped")
289         self.assertEqual(events[-1][1]["error"][0], "traffic")
290
291
292 class MasterWindowTests(unittest.TestCase):
293     def test_workspace_entry_round_trip_and_validation(self):
294         window = MasterWindow(1, 1, slave=5, fc=3, address=100, qty=4, scan_ms=250, base=1,
295                               fmt="float32_CDAB", names={100: "Temperatura"})
296         data = MasterWindow.validated_kwargs(window.to_json())
297         self.assertEqual(data["names"], {100: "Temperatura"})
298         self.assertEqual((data["slave"], data["fmt"], data["base"]), (5, "float32_CDAB", 1))
299         for bad in ({"slave": 0}, {"fc": 6}, {"qty": 200}, {"scan_ms": -1}, "x"):
300             with self.assertRaises((ValueError, TypeError)):
301                 MasterWindow.validated_kwargs(bad)
302
303     def test_write_dialog_builds_fc16_for_float_values(self):
304         window = MasterWindow(1, 1, slave=2, fc=3, address=0, qty=10, base=1,
305                               fmt="float32_ABCD")
306         request, _ = MasterMixin.master_build_write(None, window, "2", "11", "1.5;2", 16)
307         self.assertEqual(request[:-2], bytes.fromhex("02 10 000A 0004 08 3FC0 0000 4000 0000"))
308         with self.assertRaises(ValueError):
309             MasterMixin.master_build_write(None, window, "2", "11", "1.5", 6)
310
311
312 class MasterWorkspaceFolderTests(unittest.TestCase):
313     def setUp(self):
314         self._appdata = os.environ.get("APPDATA")
315         self._folder = tempfile.TemporaryDirectory()
316         os.environ["APPDATA"] = self._folder.name
317
318     def tearDown(self):
319         if self._appdata is None:
320             os.environ.pop("APPDATA", None)
```

MBToolkit v3.5/tests/test_master.py — lines 321-360

```
        else:
            os.environ["APPDATA"] = self._appdata
        self._folder.cleanup()

    def test_workspaces_live_beside_the_settings(self):
        folder = mb_config.get_workspaces_folder()
        self.assertEqual(folder.name, "Workspaces")
        self.assertEqual(folder.parent, mb_config.get_settings_file().parent)
        self.assertTrue(folder.is_dir())
        master = mb_config.get_master_workspaces_folder()
        self.assertEqual((master.name, master.parent), ("Master", folder))
        self.assertEqual(mb_master_ui.workspace_path("Linha 1").parent, master)

    def test_loose_master_workspaces_move_into_the_master_folder(self):
        root = mb_config.get_workspaces_folder()
        (root / "Linha 1.mbsw").write_text('{"windows": []}', encoding="utf-8")
        (root / "notas.txt").write_text("x", encoding="utf-8")
        self.assertEqual(mb_master_ui.list_workspaces(), ["Linha 1"])
        master = mb_config.get_master_workspaces_folder()
        self.assertTrue((master / "Linha 1.mbsw").is_file())
        self.assertFalse((root / "Linha 1.mbsw").exists())
        self.assertTrue((root / "notas.txt").exists())
        # Only the first time: a file put in Workspaces later stays there.
        (root / "Outro.mbsw").write_text("{}", encoding="utf-8")
        self.assertEqual(mb_master_ui.list_workspaces(), ["Linha 1"])

    def test_saved_workspaces_are_listed_by_name(self):
        for name in ("Subestação B", "armazém", "Linha 1"):
            mb_master_ui.workspace_path(name).write_text("{}", encoding="utf-8")
        (mb_config.get_master_workspaces_folder() / "notas.txt").write_text("x", encoding="utf-8")
        self.assertEqual(mb_master_ui.list_workspaces(),
                         ["armazém", "Linha 1", "Subestação B"])

    def test_workspace_names_must_be_valid_file_names(self):
        self.assertEqual(mb_table_ui.validate_workspace_name("  Linha 1. "), "Linha 1")
        for bad in ("", "   ", "a/b", "x:y", "CON", "com3", "a" * 61, 'q"'):
            with self.assertRaises(ValueError):
                mb_table_ui.validate_workspace_name(bad)

    def test_mbsniffer_settings_and_workspaces_are_migrated_once(self):
```

MBToolkit v3.5/tests/test_master.py — lines 361-400

```
361         legacy = Path(self._folder.name) / "MBSniffer"
362         (legacy / "Workspaces").mkdir(parents=True)
363         (legacy / "settings.json").write_text('{"language": "English"}', encoding="utf-8")
364         (legacy / "Workspaces" / "Linha 1.mbsw").write_text(
365             '{"format": "MBSniffer Master workspace", "windows": []}', encoding="utf-8")
366 
367         settings = mb_config.get_settings_file()
368         self.assertEqual(settings.parent.name, "MBToolkit")
369         self.assertEqual(mb_config.load_ui_settings()["language"], "English")
370         self.assertEqual(mb_master_ui.list_workspaces(), ["Linha 1"])
371         self.assertTrue((legacy / "settings.json").exists())  # old folder kept
372 
373         # Later changes in the old folder are not copied again.
374         (legacy / "Workspaces" / "Outro.mbsw").write_text("{}", encoding="utf-8")
375         mb_config.get_settings_file()
376         self.assertEqual(mb_master_ui.list_workspaces(), ["Linha 1"])
377 
378     def test_legacy_workspace_format_is_accepted(self):
379         self.assertIn("MBSniffer Master workspace", mb_config.MASTER_WORKSPACE_FORMATS)
380         self.assertEqual(mb_config.MASTER_WORKSPACE_FORMAT, "MBToolkit Master workspace")
381 
382     def test_window_title_is_saved_in_the_workspace(self):
383         window = MasterWindow(1, 2, title="  Inversor 3  ")
384         self.assertEqual(window.title, "Inversor 3")
385         self.assertEqual(MasterWindow.validated_kwargs(window.to_json())["title"], "Inversor 3")
386         self.assertEqual(MasterWindow.validated_kwargs({"slave": 1})["title"], "")
387 
388 
389 class MasterPerAddressFormatTests(unittest.TestCase):
390     def test_64_bit_formats_round_trip_in_every_byte_order(self):
391         for order in BYTE_ORDERS_64:
392             for kind, text in (("int64", "-1234567890123"), ("uint64", "18000000000000000000"),
393                                ("float64", "123456.789012345")):
394                 fmt = f"{kind}_{order}"
395                 registers = encode_register_value(text, fmt)
396                 self.assertEqual(len(registers), 4)
397                 self.assertEqual(format_value(registers, fmt), text)
398 
399     def test_double_byte_orders_match_wire_layout(self):
400         # 1.0 = 3FF0 0000 0000 0000 in IEEE 754 big-endian.
```

MBToolkit v3.5/tests/test_master.py — lines 401-440

```
        self.assertEqual(encode_register_value("1", "float64_ABCDEFGH"), [0x3FF0, 0, 0, 0])
        self.assertEqual(encode_register_value("1", "float64_GHEFCDAB"), [0, 0, 0, 0x3FF0])
        self.assertEqual(encode_register_value("1", "float64_BADCFEHG"), [0xF03F, 0, 0, 0])
        self.assertEqual(encode_register_value("1", "float64_HGFEDCBA"), [0, 0, 0, 0xF03F])

    def test_per_address_formats_override_the_window_format(self):
        registers = [0x42F6, 0xE979, 0xFFFF, 0x0001, 0x0002, 7]
        texts = format_window_values(registers, 100, "int16", {100: "float32_ABCD"})
        self.assertEqual(texts, ["123.456", "", "-1", "1", "2", "7"])
        self.assertEqual(register_groups(6, 100, "int16", {102: "int64_ABCDEFGH"}),
                         [(0, 1, "int16"), (1, 1, "int16"), (2, 4, "int64_ABCDEFGH")])

    def test_default_multi_register_values_cut_by_an_override_show_a_dash(self):
        texts = format_window_values([1, 2, 3, 4, 5], 0, "uint32_ABCD", {1: "int16"})
        self.assertEqual(texts, ["—", "2", "196612", "", "—"])

    def test_selection_rules_for_multi_register_formats(self):
        window = MasterWindow(1, 1, address=100, qty=10)
        apply_format_to_rows(window, [2, 3, 4, 5], "float32_CDAB")
        self.assertEqual(window.formats, {102: "float32_CDAB", 104: "float32_CDAB"})
        apply_format_to_rows(window, [4, 5, 6, 7], "float64_ABCDEFGH")
        self.assertEqual(window.formats, {102: "float32_CDAB", 104: "float64_ABCDEFGH"})
        apply_format_to_rows(window, [0, 1], "hex16")
        self.assertEqual(window.formats[100], "hex16")
        self.assertEqual(window.formats[101], "hex16")
        apply_format_to_rows(window, [4], None)
        self.assertNotIn(104, window.formats)
        for rows, fmt in (([1], "float32_ABCD"), ([1, 3], "float32_ABCD"),
                          ([0, 1, 2], "float32_ABCD"), ([0, 1], "int64_ABCDEFGH"),
                          ([8, 9, 10, 11], "int64_ABCDEFGH"), ([], "int16")):
            with self.assertRaises(ValueError):
                apply_format_to_rows(window, rows, fmt)

    def test_formats_are_saved_and_invalid_ones_dropped(self):
        window = MasterWindow(1, 1, address=0, qty=4, formats={0: "float32_ABCD"})
        data = MasterWindow.validated_kwargs(window.to_json())
        self.assertEqual(data["formats"], {0: "float32_ABCD"})
        data = MasterWindow.validated_kwargs(
            {"formats": {"0": "nonsense", "x": "int16", "70000": "int16", "2": "hex16"}})
        self.assertEqual(data["formats"], {2: "hex16"})
```

MBToolkit v3.5/tests/test_master.py — lines 441-480

```
441
442     def test_write_uses_the_format_of_the_clicked_value(self):
443         window = MasterWindow(1, 1, slave=3, address=0, qty=8, fmt="int16")
444         request, _ = MasterMixin.master_build_write(None, window, "3", "4", "1", 16,
445                                                     "float64_ABCDEFGH")
446         self.assertEqual(request[:-2], bytes.fromhex("03 10 0004 0004 08 3FF0 0000 0000 0000"))
447
448
449 class MasterScaleTests(unittest.TestCase):
450     SCALE = validate_scale("2000", "10000", "-180", "180", 1, "°")
451
452     def test_scaled_values_are_shown_in_engineering_units(self):
453         texts = format_window_values([2000, 6000, 10000, 0x00AB], 0, "int16", {3: "hex16"},
454                                      {0: self.SCALE, 1: self.SCALE, 2: self.SCALE,
455                                       3: self.SCALE})
456         self.assertEqual(texts, ["-180.0 °", "0.0 °", "180.0 °", "0x00AB"])
457
458     def test_scale_applies_to_multi_register_values(self):
459         registers = encode_register_value("50", "float32_ABCD")
460         scale = validate_scale(0, 100, 0, 10, 2, "bar")
461         self.assertEqual(format_window_values(registers, 7, "float32_ABCD", None, {7: scale}),
462                          ["5.00 bar", ""])
463
464     def test_written_engineering_values_are_converted_to_raw(self):
465         self.assertEqual(raw_text_for_scaled("90", self.SCALE, "int16"), "8000")
466         self.assertEqual(raw_text_for_scaled("-180", self.SCALE, "uint16"), "2000")
467         window = MasterWindow(1, 1, slave=1, address=0, qty=4,
468                               scales={0: self.SCALE})
469         request, _ = MasterMixin.master_build_write(None, window, "1", "0", "45", 6,
470                                                     "int16", self.SCALE)
471         self.assertEqual(request[:-2], bytes.fromhex("01 06 0000 1B58"))  # 7000
472
473     def test_invalid_scales_are_rejected(self):
474         for args in (("1", "1", "0", "10"), ("a", "1", "0", "1"), ("0", "1", "0", "1", "9"),
475                      ("0", "nan", "0", "1")):
476             with self.assertRaises(ValueError):
477                 validate_scale(*args)
478         with self.assertRaises(ValueError):
479             raw_text_for_scaled("5", validate_scale(0, 10, 3, 3), "int16")
480
```

MBToolkit v3.5/tests/test_master.py — lines 481-520

```
481     def test_scales_are_saved_and_invalid_ones_dropped(self):
482         window = MasterWindow(1, 1, scales={4: self.SCALE})
483         data = MasterWindow.validated_kwargs(window.to_json())
484         self.assertEqual(data["scales"], {4: self.SCALE})
485         data = MasterWindow.validated_kwargs({"scales": {
486             "1": {"raw": [0, 0], "eng": [0, 1]}, "x": self.SCALE, "2": "bad",
487             "3": {"raw": [0, 10], "eng": [0, 1], "decimals": 2, "unit": "A"}}})
488         self.assertEqual(list(data["scales"]), [3])
489 
490 
491 class MasterOpenFromTrafficTests(unittest.TestCase):
492     def definition(self, record):
493         return MasterMixin.master_definition_from_record(None, record)
494 
495     def test_request_and_matched_response_give_the_read_definition(self):
496         request = {"type": "REQUEST", "slave": "5", "fc": "03", "pdu_address": 100, "qty": 4}
497         self.assertEqual(self.definition(request),
498                          {"slave": 5, "fc": 3, "address": 100, "qty": 4})
499         response = {"type": "RESPONSE", "slave": "5", "fc": "04",
500                     "request_pdu_address": 7, "request_qty": 2}
501         self.assertEqual(self.definition(response)["address"], 7)
502         exception = {"type": "EXCEPTION", "slave": "5", "fc": "83",
503                      "request_pdu_address": 1, "request_qty": 1}
504         self.assertEqual(self.definition(exception)["fc"], 3)
505 
506     def test_frames_without_a_read_definition_are_refused(self):
507         for record in (
508             {"type": "REQUEST", "slave": "5", "fc": "06", "pdu_address": 1, "qty": None},
509             {"type": "RESPONSE", "slave": "5", "fc": "03"},
510             {"type": "RAW/UNPARSED", "slave": "", "fc": ""},
511             {"type": "REQUEST", "slave": "0", "fc": "03", "pdu_address": 1, "qty": 1},
512         ):
513             self.assertIsNone(self.definition(record))
514 
515 
516 class FlakySlaveSerial(FakeSlaveSerial):
517     """Fake slave whose port fails (like an unplugged USB adapter)."""
518 
519     def __init__(self, failures, **kwargs):
520         super().__init__(**kwargs)
```

MBToolkit v3.5/tests/test_master.py — lines 521-560

```
        self.failures = failures
        self.closed = False

    def write(self, data):
        if self.failures:
            self.failures -= 1
            raise OSError("device disconnected")
        return super().write(data)

    def close(self):
        self.closed = True


class MasterReconnectTests(unittest.TestCase):
    def test_engine_reopens_the_port_after_a_failure(self):
        broken = FlakySlaveSerial(failures=1)
        fresh = FakeSlaveSerial()
        reopen_calls = []
        engine = make_engine(broken, reconnect_interval_s=0.05,
                             reopen=lambda: reopen_calls.append(1) or fresh)
        engine.set_poll(PollDefinition(1, 1, 3, 0, 2, 0))
        engine.start()
        time.sleep(0.4)
        engine.stop()
        events = drain(engine.event_queue)
        names = [name for name, _payload in events]
        self.assertIn("master_reconnecting", names)
        self.assertIn("master_reconnected", names)
        self.assertTrue(broken.closed)
        self.assertIs(engine.ser, fresh)
        after = names[names.index("master_reconnected"):]
        self.assertIn("master_result", after)
        self.assertEqual(events[-1], ("master_stopped", {"error": None}))

    def test_engine_keeps_retrying_until_stopped(self):
        attempts = []

        def reopen():
            attempts.append(1)
            raise OSError("still unplugged")
```

MBToolkit v3.5/tests/test_master.py — lines 561-600

```
561
562         engine = make_engine(FlakySlaveSerial(failures=5), reconnect_interval_s=0.02,
563                              reopen=reopen)
564         engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
565         engine.start()
566         time.sleep(0.3)
567         engine.stop()
568         events = drain(engine.event_queue)
569         self.assertGreater(len(attempts), 3)
570         self.assertNotIn("master_reconnected", [n for n, _p in events])
571         self.assertEqual(events[-1][0], "master_stopped")
572
573     def test_without_reopen_a_failure_ends_the_session(self):
574         engine = make_engine(FlakySlaveSerial(failures=1))
575         engine.set_poll(PollDefinition(1, 1, 3, 0, 1, 0))
576         engine.start()
577         time.sleep(0.2)
578         engine.stop()
579         stopped = drain(engine.event_queue)[-1]
580         self.assertEqual(stopped[0], "master_stopped")
581         self.assertEqual(stopped[1]["error"][0], "serial")
582
583
584 class MasterSettingsTests(unittest.TestCase):
585     def test_last_workspace_is_validated(self):
586         with tempfile.TemporaryDirectory() as folder:
587             path = Path(folder) / "settings.json"
588             original = mb_config.get_settings_file
589             mb_config.get_settings_file = lambda: path
590             try:
591                 for value, expected in (("Linha 1", "Linha 1"), ("a/b", ""), (5, ""),
592                                         ("x" * 61, ""), ("  Subestação  ", "Subestação")):
593                     path.write_text(json.dumps({"master": {"last_workspace": value}}),
594                                     encoding="utf-8")
595                     self.assertEqual(
596                         mb_config.load_ui_settings()["master"]["last_workspace"], expected)
597             finally:
598                 mb_config.get_settings_file = original
599
600
```

MBToolkit v3.5/tests/test_master.py — lines 601-602

```
601 if __name__ == "__main__":
602     unittest.main(verbosity=2)
```

MBToolkit v3.5/tests/test_slave.py

537 source lines.

MBToolkit v3.5/tests/test_slave.py — lines 1-40

```
import os
import queue
import random
import tempfile
import threading
import time
import unittest

from mb_master import (
    MasterEngine,
    PollDefinition,
    build_read_request,
    build_write_request,
    find_master_response,
)
from mb_protocol import append_crc, crc_ok
from mb_slave import (
    SlaveEngine,
    SlaveStore,
    corrupt_crc,
    fault_active,
    handle_request,
    request_length,
    validate_fault,
    validate_generator,
    ValueGenerator,
)
import mb_config
from mb_i18n import LANGUAGE_ENGLISH, translate_text
from mb_slave_ui import SlaveWindow, fault_parts, slave_definition_from_record, validate_area
from mb_table_ui import list_workspaces, workspace_path


class BusEnd:
    """One end of an in-memory serial link: what one end writes, the other reads.

    With *echo*, every byte written also comes back to the writer, like an
    RS485 adapter that hears its own transmission.
    """

```

MBToolkit v3.5/tests/test_slave.py — lines 41-80

```
    def __init__(self, echo=False):
        self.echo = echo
        self.peer = None
        self.rx = bytearray()
        self.written = []
        self._lock = threading.Lock()

    @staticmethod
    def pair(echo_a=False, echo_b=False):
        a, b = BusEnd(echo_a), BusEnd(echo_b)
        a.peer, b.peer = b, a
        return a, b

    @property
    def in_waiting(self):
        with self._lock:
            return len(self.rx)

    def read(self, size):
        with self._lock:
            data = bytes(self.rx[:size])
            del self.rx[:size]
            return data

    def write(self, data):
        data = bytes(data)
        self.written.append(data)
        if self.echo:
            self.inject(data)
        if self.peer is not None:
            self.peer.inject(data)
        return len(data)

    def inject(self, data):
        with self._lock:
            self.rx.extend(data)

    def reset_input_buffer(self):
        with self._lock:
            self.rx.clear()
```

MBToolkit v3.5/tests/test_slave.py — lines 81-120

```
81
82      def reset_output_buffer(self):
83          pass
84
85      def cancel_read(self):
86          pass
87
88      def cancel_write(self):
89          pass
90
91      def close(self):
92          pass
93
94
95  def make_store():
96      store = SlaveStore()
97      store.set_areas([(1, "holding", 0, 20), (1, "coils", 0, 16), (1, "input", 100, 4),
98                       (1, "discrete", 0, 8), (2, "holding", 0, 10)])
99      store.write(1, "holding", 0, [i * 10 for i in range(20)])
100     store.write(1, "input", 100, [7, 8, 9, 10])
101     store.write(1, "coils", 0, [1, 0, 1, 1])
102     store.write(1, "discrete", 0, [0, 1])
103     return store
104
105
106 def drain(event_queue):
107     events = []
108     while not event_queue.empty():
109         events.append(event_queue.get())
110     return events
111
112
113 def wait_for(predicate, timeout=2.0):
114     deadline = time.monotonic() + timeout
115     while time.monotonic() < deadline:
116         if predicate():
117             return True
118         time.sleep(0.005)
119     return predicate()
120
```

MBToolkit v3.5/tests/test_slave.py — lines 121-160

```
121
122 class SlaveRequestTests(unittest.TestCase):
123     def test_reads_answer_from_the_tables(self):
124         store = make_store()
125         reply = handle_request(store, build_read_request(1, 3, 2, 3))
126         self.assertEqual(reply.status, "ok")
127         found = find_master_response(reply.response, build_read_request(1, 3, 2, 3))
128         self.assertEqual(found["values"], [20, 30, 40])
129         found = find_master_response(handle_request(store, build_read_request(1, 1, 0, 4)).response,
130                                      build_read_request(1, 1, 0, 4))
131         self.assertEqual(found["values"], [1, 0, 1, 1])
132         found = find_master_response(handle_request(store, build_read_request(1, 4, 100, 4)).response,
133                                      build_read_request(1, 4, 100, 4))
134         self.assertEqual(found["values"], [7, 8, 9, 10])
135         found = find_master_response(handle_request(store, build_read_request(1, 2, 0, 2)).response,
136                                      build_read_request(1, 2, 0, 2))
137         self.assertEqual(found["values"], [0, 1])
138
139     def test_writes_change_the_tables_and_answer_like_a_device(self):
140         store = make_store()
141         request = build_write_request(1, 6, 5, [1234])
142         reply = handle_request(store, request)
143         self.assertEqual(reply.response, request)
144         self.assertEqual(reply.written, [(1, "holding", 5, 1)])
145         self.assertEqual(store.read(1, "holding", 5, 1), [1234])
146
147         request = build_write_request(1, 16, 10, [1, 2, 3])
148         reply = handle_request(store, request)
149         self.assertEqual(reply.response, append_crc(request[:6]))
150         self.assertEqual(store.read(1, "holding", 10, 3), [1, 2, 3])
151
152         handle_request(store, build_write_request(1, 5, 1, [1]))
153         handle_request(store, build_write_request(1, 15, 8, [1, 1, 0, 1, 0, 0, 1]))
154         self.assertEqual(store.read(1, "coils", 0, 2), [1, 1])
155         self.assertEqual(store.read(1, "coils", 8, 7), [1, 1, 0, 1, 0, 0, 1])
156
157     def test_exceptions_like_a_real_device(self):
158         store = make_store()
159         # Address outside the defined areas, also when only partly outside.
160         for request in (build_read_request(1, 3, 20, 1), build_read_request(1, 3, 18, 4),
```

MBToolkit v3.5/tests/test_slave.py — lines 161-200

```
                        build_read_request(1, 4, 0, 1), build_write_request(1, 6, 50, [1])):
            reply = handle_request(store, request)
            self.assertEqual((reply.status, reply.exception_code), ("exception", 2))
            self.assertEqual(reply.response, append_crc(bytes((1, request[1] | 0x80, 2))))
        # Unsupported function.
        reply = handle_request(store, append_crc(bytes((1, 0x08, 0, 0, 0x12, 0x34))))
        self.assertEqual(reply.exception_code, 1)
        # Bad quantity and bad FC05 value.
        reply = handle_request(store, append_crc(bytes((1, 3, 0, 0, 0, 0))))
        self.assertEqual(reply.exception_code, 3)
        reply = handle_request(store, append_crc(bytes((1, 5, 0, 1, 0x12, 0x34))))
        self.assertEqual(reply.exception_code, 3)

    def test_other_slave_ids_are_ignored(self):
        reply = handle_request(make_store(), build_read_request(9, 3, 0, 1))
        self.assertEqual(reply.status, "ignored")
        self.assertIsNone(reply.response)

    def test_broadcast_writes_every_covering_slave_without_answering(self):
        store = make_store()
        reply = handle_request(store, build_write_request(0, 6, 3, [77]))
        self.assertEqual(reply.status, "broadcast")
        self.assertIsNone(reply.response)
        self.assertEqual(sorted(reply.written), [(1, "holding", 3, 1), (2, "holding", 3, 1)])
        self.assertEqual(store.read(2, "holding", 3, 1), [77])
        # Slave 2 does not cover address 15: only slave 1 is written.
        reply = handle_request(store, build_write_request(0, 6, 15, [5]))
        self.assertEqual(reply.written, [(1, "holding", 15, 1)])
        # Broadcast reads and errors are never answered.
        self.assertIsNone(handle_request(store, append_crc(bytes((0, 3, 0, 0, 0, 1)))).response)
        self.assertIsNone(handle_request(store, append_crc(bytes((0, 6, 0, 90, 0, 1)))).response)

    def test_request_length(self):
        self.assertIsNone(request_length(b"\x01"))
        self.assertEqual(request_length(build_read_request(1, 3, 0, 1)), 8)
        self.assertEqual(request_length(build_write_request(1, 16, 0, [1, 2])), 13)
        self.assertIsNone(request_length(b"\x01\x10\x00\x00\x00\x02"))
        self.assertIsNone(request_length(b"\x01\x2B\x0E\x01"))

    def test_window_areas_define_the_slave(self):
```

MBToolkit v3.5/tests/test_slave.py — lines 201-240

```
        window = SlaveWindow(1, 1, slave=4, table="input", address=10, qty=5)
        store = SlaveStore()
        store.set_areas([window.area()])
        self.assertEqual(store.slave_ids(), {4})
        self.assertTrue(store.covers(4, "input", 10, 5))
        self.assertFalse(store.covers(4, "input", 10, 6))


class SlaveFaultTests(unittest.TestCase):
    def test_fault_validation(self):
        self.assertFalse(fault_active(validate_fault({})))
        fault = validate_fault({"delay_ms": "250", "probability": "10", "exception": 6})
        self.assertTrue(fault_active(fault))
        self.assertEqual((fault["delay_ms"], fault["probability"], fault["exception"]),
                         (250, 10, 6))
        for bad in ({"delay_ms": -1}, {"delay_ms": 10001}, {"probability": 0},
                    {"probability": 101}, {"exception": 5}, {"delay_ms": "x"}):
            with self.assertRaises(ValueError):
                validate_fault(bad)

    def test_inactive_faults_are_not_stored(self):
        store = make_store()
        store.set_fault(1, validate_fault({"probability": 50}))
        self.assertIsNone(store.fault(1))
        store.set_fault(1, validate_fault({"bad_crc": True}))
        self.assertTrue(store.fault(1)["bad_crc"])
        store.set_fault(1, None)
        self.assertEqual(store.faults(), {})

    def test_corrupt_crc(self):
        frame = build_read_request(1, 3, 0, 1)
        self.assertFalse(crc_ok(corrupt_crc(frame)))


class SlaveEngineTests(unittest.TestCase):
    """The Master engine talking to the Slave engine over an in-memory bus."""

    def setUp(self):
        self.store = make_store()
        self.master_end, self.slave_end = BusEnd.pair()
```

MBToolkit v3.5/tests/test_slave.py — lines 241-280

```
        self.slave_events = queue.Queue()
        self.master_events = queue.Queue()
        self.slave = None
        self.master = None

    def tearDown(self):
        for engine in (self.master, self.slave):
            if engine is not None:
                engine.stop()

    def start_slave(self, **options):
        self.slave = SlaveEngine(self.slave_end, self.slave_events, self.store, baud=115200,
                                 parity="None", stopbits="1", **options)
        self.slave.start()

    def start_master(self, timeout_ms=200):
        self.master = MasterEngine(self.master_end, self.master_events, baud=115200,
                                   parity="None", stopbits="1",
                                   response_timeout_ms=timeout_ms, delay_ms=0)
        self.master.start()

    def master_results(self, count, timeout=3.0):
        results = []
        deadline = time.monotonic() + timeout
        while len(results) < count and time.monotonic() < deadline:
            try:
                event, payload = self.master_events.get(timeout=0.05)
            except queue.Empty:
                continue
            if event == "master_result":
                results.append(payload)
        return results

    def test_master_reads_and_writes_every_function(self):
        self.start_slave()
        self.start_master()
        for window_id, (fc, address, qty) in enumerate(
                ((3, 0, 5), (4, 100, 4), (1, 0, 4), (2, 0, 2)), start=1):
            self.master.set_poll(PollDefinition(window_id, 1, fc, address, qty, 10000))
        values = {r["window_id"]: r["values"] for r in self.master_results(4)}
```

MBToolkit v3.5/tests/test_slave.py — lines 281-320

```
        self.assertEqual(values, {1: [0, 10, 20, 30, 40], 2: [7, 8, 9, 10],
                                  3: [1, 0, 1, 1], 4: [0, 1]})

        for request in (build_write_request(1, 6, 0, [999]), build_write_request(1, 16, 1, [5, 6]),
                        build_write_request(1, 5, 1, [1]),
                        build_write_request(1, 15, 2, [0, 0])):
            self.master.queue_write(1, request, "w")
        statuses = [r["status"] for r in self.master_results(4) if r["kind"] == "write"]
        self.assertEqual(statuses, ["ok"] * 4)
        self.assertEqual(self.store.read(1, "holding", 0, 3), [999, 5, 6])
        self.assertEqual(self.store.read(1, "coils", 0, 4), [1, 1, 0, 0])
        written = [p["written"] for e, p in drain(self.slave_events)
                   if e == "slave_request" and p["written"]]
        self.assertIn([(1, "holding", 1, 2)], written)

    def test_exception_reaches_the_master(self):
        self.start_slave()
        self.start_master()
        self.master.set_poll(PollDefinition(1, 1, 3, 30, 2, 10000))
        result = self.master_results(1)[0]
        self.assertEqual((result["status"], result["exception_code"]), ("exception", 2))

    def test_adapter_echo_and_foreign_traffic_are_ignored(self):
        self.slave_end.echo = True
        self.start_slave()
        # A response of another slave on the bus before the request.
        self.slave_end.inject(append_crc(bytes((9, 3, 2, 0x12, 0x34))))
        self.start_master()
        self.master.set_poll(PollDefinition(1, 1, 3, 2, 2, 20))
        results = self.master_results(5)
        self.assertEqual(len(results), 5)
        self.assertTrue(all(r["status"] == "ok" and r["values"] == [20, 30] for r in results))
        answered = []

        def five_answered():
            answered.extend(p for e, p in drain(self.slave_events)
                            if e == "slave_request" and p["status"] == "ok")
            return len(answered) >= 5

        self.assertTrue(wait_for(five_answered))
```

MBToolkit v3.5/tests/test_slave.py — lines 321-360

```
321
322     def test_split_request_is_assembled(self):
323         self.start_slave()
324         request = build_read_request(1, 3, 0, 2)
325         self.slave_end.inject(request[:3])
326         time.sleep(0.01)
327         self.slave_end.inject(request[3:])
328         self.assertTrue(wait_for(lambda: len(self.slave_end.written) == 1))
329         found = find_master_response(self.slave_end.written[0], request)
330         self.assertEqual(found["values"], [0, 10])
331
332     def test_garbage_before_a_request_is_dropped(self):
333         self.start_slave(silence_ms=20)
334         request = build_read_request(1, 3, 0, 1)
335         self.slave_end.inject(b"\x01\x10\xFF" + request)
336         self.assertTrue(wait_for(lambda: len(self.slave_end.written) == 1))
337         self.assertEqual(find_master_response(self.slave_end.written[0], request)["values"], [0])
338
339     def test_unknown_function_gets_exception_01_after_silence(self):
340         self.start_slave(silence_ms=10)
341         self.slave_end.inject(append_crc(bytes((1, 0x2B, 0x0E, 0x01, 0x00))))
342         self.assertTrue(wait_for(lambda: len(self.slave_end.written) == 1))
343         self.assertEqual(self.slave_end.written[0], append_crc(bytes((1, 0xAB, 1))))
344
345     def test_faults(self):
346         self.start_slave(rng=random.Random(1))
347         self.start_master(timeout_ms=100)
348         self.master.set_poll(PollDefinition(1, 1, 3, 0, 1, 10000))
349
350         self.store.set_fault(1, validate_fault({"no_response": True}))
351         self.master.read_once(1)
352         self.assertEqual(self.master_results(2)[-1]["status"], "timeout")
353
354         self.store.set_fault(1, validate_fault({"exception": 6}))
355         self.master.read_once(1)
356         result = self.master_results(1)[0]
357         self.assertEqual((result["status"], result["exception_code"]), ("exception", 6))
358
359         self.store.set_fault(1, validate_fault({"bad_crc": True}))
360         self.master.read_once(1)
```

MBToolkit v3.5/tests/test_slave.py — lines 361-400

```
361         self.assertEqual(self.master_results(1)[0]["status"], "invalid")
362
363         self.store.set_fault(1, validate_fault({"delay_ms": 300}))
364         self.master.read_once(1)
365         self.assertEqual(self.master_results(1)[0]["status"], "timeout")
366         time.sleep(0.35)  # let the late response pass before the next request
367
368         self.store.set_fault(1, None)
369         self.master.read_once(1)
370         self.assertEqual(self.master_results(1)[0]["status"], "ok")
371
372     def test_fault_probability(self):
373         store = make_store()
374         store.set_fault(1, validate_fault({"no_response": True, "probability": 30}))
375         engine = SlaveEngine(BusEnd(), queue.Queue(), store, baud=115200, parity="None",
376                              stopbits="1", rng=random.Random(7))
377         engine.gap_s = 0
378         for _ in range(400):
379             engine._answer(build_read_request(1, 3, 0, 1))
380         statuses = [p["status"] for _e, p in drain(engine.event_queue)]
381         silent = statuses.count("no_response")
382         self.assertTrue(80 <= silent <= 160, silent)
383         self.assertEqual(silent + statuses.count("ok"), 400)
384
385
386 class SlaveWindowTests(unittest.TestCase):
387     def test_workspace_entry_round_trip_and_validation(self):
388         window = SlaveWindow(1, 1, slave=3, table="coils", address=5, qty=16, base=1,
389                              title=" Saídas ", names={5: "Bomba"})
390         data = SlaveWindow.validated_kwargs(window.to_json())
391         self.assertEqual((data["title"], data["slave"], data["table"], data["address"],
392                           data["qty"], data["base"], data["names"]),
393                          ("Saídas", 3, "coils", 5, 16, 1, {5: "Bomba"}))
394         for bad in ({"slave": 0}, {"slave": 248}, {"table": "x"}, {"qty": 0},
395                     {"qty": 2001}, {"address": 65530, "qty": 10}):
396             with self.assertRaises(ValueError):
397                 SlaveWindow.validated_kwargs(bad)
398         validate_area(1, "input", 65535, 1)
399
400     def test_table_field_shows_the_read_function_code(self):
```

MBToolkit v3.5/tests/test_slave.py — lines 401-440

```
401         from mb_slave import table_choice_label
402         from mb_slave_ui import TABLE_CHOICES, TABLE_KEYS_BY_LABEL
403         self.assertEqual(TABLE_CHOICES, ("01 Coils", "02 Discrete Inputs",
404                                          "03 Holding Registers", "04 Input Registers"))
405         for table in ("coils", "discrete", "holding", "input"):
406             self.assertEqual(TABLE_KEYS_BY_LABEL[table_choice_label(table)], table)
407
408     def test_window_overlap(self):
409         window = SlaveWindow(1, 1, address=10, qty=5)
410         self.assertEqual(window.overlap(0, 12), (0, 2))
411         self.assertEqual(window.overlap(12, 100), (2, 5))
412         self.assertIsNone(window.overlap(15, 3))
413         self.assertIsNone(window.overlap(0, 10))
414
415     def test_fault_description_translates(self):
416         fault = validate_fault({"no_response": True, "delay_ms": 50, "exception": 4})
417         parts = [translate_text(template, LANGUAGE_ENGLISH).format(**values)
418                  for template, values in fault_parts(fault)]
419         self.assertEqual(parts, ["no response", "Exception 04", "delay 50 ms"])
420         self.assertEqual(translate_text("Falhas…", LANGUAGE_ENGLISH), "Faults…")
421
422
423 class SlaveGeneratorTests(unittest.TestCase):
424     def test_validation(self):
425         config = validate_generator({"kind": "counter", "min": "0", "max": "10",
426                                      "step": "2", "period_s": "0,5"})
427         self.assertEqual(config, {"kind": "counter", "period_s": 0.5, "min": 0.0,
428                                   "max": 10.0, "step": 2.0})
429         self.assertEqual(validate_generator({"kind": "toggle", "period_s": 2}, bits=True),
430                          {"kind": "toggle", "period_s": 2.0})
431         for bad, bits in (({"kind": "sine", "min": 5, "max": 5}, False),
432                           ({"kind": "ramp", "period_s": 0.05}, False),
433                           ({"kind": "ramp", "period_s": 4000}, False),
434                           ({"kind": "counter", "step": 0}, False),
435                           ({"kind": "toggle"}, False),
436                           ({"kind": "sine"}, True),
437                           ({"kind": "random", "min": "x"}, False)):
438             with self.assertRaises(ValueError):
439                 validate_generator(bad, bits)
440
```

MBToolkit v3.5/tests/test_slave.py — lines 441-480

```
441     def test_values_over_time(self):
442         counter = ValueGenerator(validate_generator(
443             {"kind": "counter", "min": 0, "max": 3, "step": 1, "period_s": 1}))
444         self.assertEqual([counter.value(t) for t in (0, 1.2, 2.5, 3.1, 4.0)], [0, 1, 2, 3, 0])
445         ramp = ValueGenerator(validate_generator(
446             {"kind": "ramp", "min": 10, "max": 20, "period_s": 10}))
447         self.assertAlmostEqual(ramp.value(5), 15)
448         self.assertAlmostEqual(ramp.value(12.5), 12.5)
449         sine = ValueGenerator(validate_generator(
450             {"kind": "sine", "min": -1, "max": 1, "period_s": 4}))
451         self.assertAlmostEqual(sine.value(1), 1)
452         self.assertAlmostEqual(sine.value(3), -1)
453         toggle = ValueGenerator(validate_generator({"kind": "toggle", "period_s": 1}, True))
454         self.assertEqual([toggle.value(t) for t in (0.5, 1.5, 2.5)], [0, 1, 0])
455         noise = ValueGenerator(validate_generator(
456             {"kind": "random", "min": 5, "max": 6, "period_s": 1}), random.Random(3))
457         first = noise.value(0.1)
458         self.assertTrue(5 <= first <= 6)
459         self.assertEqual(noise.value(0.9), first)  # one value per period
460         bits = ValueGenerator(validate_generator({"kind": "random", "period_s": 1}, True),
461                               random.Random(3))
462         self.assertIn(bits.value(0), (0, 1))
463 
464     def test_generators_are_saved_and_follow_the_window(self):
465         window = SlaveWindow(1, 1, table="holding", address=0, qty=4, generators={
466             2: {"kind": "sine", "min": 0, "max": 10, "period_s": 5}})
467         data = SlaveWindow.validated_kwargs(window.to_json())
468         self.assertEqual(data["generators"][2]["kind"], "sine")
469         bad = window.to_json()
470         bad["generators"]["3"] = {"kind": "toggle"}
471         self.assertEqual(list(SlaveWindow.validated_kwargs(bad)["generators"]), [2])
472         window.qty = 2  # address 2 is no longer in the window
473         window.drop_invalid_generators()
474         self.assertEqual(window.generators, {})
475 
476 
477 class SlaveOpenFromTrafficTests(unittest.TestCase):
478     def test_reads_and_writes_give_the_slave_area(self):
479         request = {"type": "REQUEST", "slave": "5", "fc": "03", "pdu_address": 100, "qty": 4}
480         self.assertEqual(slave_definition_from_record(request),
```

MBToolkit v3.5/tests/test_slave.py — lines 481-520

```
                        {"slave": 5, "table": "holding", "address": 100, "qty": 4})
        response = {"type": "RESPONSE", "slave": "5", "fc": "04",
                    "request_pdu_address": 7, "request_qty": 2}
        self.assertEqual(slave_definition_from_record(response)["table"], "input")
        coil = {"type": "REQUEST", "slave": "2", "fc": "05", "pdu_address": 9, "qty": None}
        self.assertEqual(slave_definition_from_record(coil),
                         {"slave": 2, "table": "coils", "address": 9, "qty": 1})
        echo = {"type": "RESPONSE", "slave": "2", "fc": "10", "pdu_address": 0, "qty": 3}
        self.assertEqual(slave_definition_from_record(echo)["table"], "holding")

    def test_frames_without_an_area_are_refused(self):
        for record in ({"type": "REQUEST", "slave": "0", "fc": "06", "pdu_address": 1},
                       {"type": "REQUEST", "slave": "1", "fc": "08", "pdu_address": 0, "qty": 1},
                       {"type": "RESPONSE", "slave": "1", "fc": "03"},
                       {"type": "RAW", "slave": "", "fc": ""}):
            self.assertIsNone(slave_definition_from_record(record))


class SlaveSettingsTests(unittest.TestCase):
    def setUp(self):
        self._appdata = os.environ.get("APPDATA")
        self._folder = tempfile.TemporaryDirectory()
        os.environ["APPDATA"] = self._folder.name

    def tearDown(self):
        if self._appdata is None:
            os.environ.pop("APPDATA", None)
        else:
            os.environ["APPDATA"] = self._appdata
        self._folder.cleanup()

    def test_slave_workspaces_have_their_own_folder(self):
        folder = mb_config.get_slave_workspaces_folder()
        self.assertEqual(folder.parent, mb_config.get_workspaces_folder())
        workspace_path("Banco", folder).write_text("{}", encoding="utf-8")
        self.assertEqual(list_workspaces(folder), ["Banco"])
        # Not listed with the Master ones.
        self.assertEqual(list_workspaces(mb_config.get_master_workspaces_folder()), [])

    def test_slave_settings_are_validated(self):
```

MBToolkit v3.5/tests/test_slave.py — lines 521-537

```
521         mb_config.save_ui_settings(settings={
522             "slave": {"baud": "19200", "parity": "Even", "stop_bits": "2",
523                       "last_workspace": "Banco"},
524             "last_main_tab": "Slave"})
525         loaded = mb_config.load_ui_settings()
526         self.assertEqual(loaded["slave"], {"baud": "19200", "parity": "Even",
527                                            "stop_bits": "2", "last_workspace": "Banco"})
528         self.assertEqual(loaded["last_main_tab"], "Slave")
529         mb_config.save_ui_settings(settings={
530             "slave": {"baud": "1", "parity": "x", "stop_bits": "3", "last_workspace": "a/b"}})
531         self.assertEqual(mb_config.load_ui_settings()["slave"],
532                          {"baud": "9600", "parity": "None", "stop_bits": "1",
533                           "last_workspace": ""})
534
535
536 if __name__ == "__main__":
537     unittest.main()
```

MBToolkit v3.5/tests/test_tcp.py

270 source lines.

MBToolkit v3.5/tests/test_tcp.py — lines 1-40

```
import os
import queue
import random
import socket
import tempfile
import time
import unittest

import mb_config

from mb_master import build_write_request
from mb_protocol import append_crc, crc_ok
from mb_slave import SlaveStore, validate_fault
from mb_tcp import (
    ClientEngine,
    ClientPollDefinition,
    ServerEngine,
    adu_length,
    adu_to_rtu,
    build_adu,
    parse_adu,
    rtu_to_adu,
    validate_host,
    validate_tcp_port,
)
from mb_tcp_ui import ClientTab, ClientWindow, ServerTab


def make_store():
    store = SlaveStore()
    store.set_areas([(1, "holding", 0, 10), (1, "coils", 0, 8), (2, "input", 100, 4)])
    store.write(1, "holding", 0, [i * 10 for i in range(10)])
    store.write(1, "coils", 0, [1, 0, 1])
    store.write(2, "input", 100, [7, 8, 9, 10])
    return store


def wait_event(event_queue, name, timeout=3.0):
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
```

MBToolkit v3.5/tests/test_tcp.py — lines 41-80

```
        try:
            event, payload = event_queue.get(timeout=0.05)
        except queue.Empty:
            continue
        if event == name:
            return payload
    raise AssertionError(f"no {name} event")


class MbapFramingTests(unittest.TestCase):
    def test_adu_round_trip(self):
        request = build_write_request(17, 16, 0x6B, [0x0102, 0x0304])
        adu = rtu_to_adu(request, 0x1234)
        self.assertEqual(adu[:7], bytes.fromhex("1234 0000 000B 11"))
        self.assertEqual(adu_length(adu), len(adu))
        transaction, protocol, unit, pdu = parse_adu(adu)
        self.assertEqual((transaction, protocol, unit), (0x1234, 0, 17))
        self.assertEqual(adu_to_rtu(unit, pdu), request)
        self.assertTrue(crc_ok(adu_to_rtu(unit, pdu)))

    def test_incomplete_and_impossible_headers(self):
        self.assertIsNone(adu_length(b"\x00\x01\x00\x00\x00"))
        with self.assertRaises(ValueError):
            adu_length(bytes.fromhex("0001 0000 0001"))
        with self.assertRaises(ValueError):
            adu_length(bytes.fromhex("0001 0000 0100"))

    def test_host_and_port_validation(self):
        self.assertEqual(validate_host(" 192.168.1.10 "), "192.168.1.10")
        self.assertEqual(validate_tcp_port("502"), 502)
        for bad in ("", "a b", "x" * 254):
            with self.assertRaises(ValueError):
                validate_host(bad)
        for bad in ("0", "65536", "porta"):
            with self.assertRaises(ValueError):
                validate_tcp_port(bad)


class ServerClientTests(unittest.TestCase):
    """The Client engine talking to the Server engine over 127.0.0.1."""
```

MBToolkit v3.5/tests/test_tcp.py — lines 81-120

```
 81
 82    def setUp(self):
 83        self.store = make_store()
 84        self.server_events = queue.Queue()
 85        self.server = ServerEngine(self.server_events, self.store, host="127.0.0.1", port=0,
 86                                   rng=random.Random(1))
 87        self.server.start()
 88        self.port = wait_event(self.server_events, "server_ready")["port"]
 89        self.client_events = queue.Queue()
 90        self.client = None
 91
 92    def tearDown(self):
 93        if self.client is not None:
 94            self.client.stop()
 95        self.server.stop()
 96
 97    def start_client(self, timeout_ms=300):
 98        self.client = ClientEngine(self.client_events, response_timeout_ms=timeout_ms,
 99                                   delay_ms=0)
100        self.client.start()
101
102    def poll(self, window_id, unit, fc, address, qty, port=None):
103        self.client.set_poll(ClientPollDefinition(window_id, unit, fc, address, qty, 10000,
104                                                  host="127.0.0.1", port=port or self.port))
105
106    def results(self, count, timeout=3.0):
107        found = []
108        deadline = time.monotonic() + timeout
109        while len(found) < count and time.monotonic() < deadline:
110            try:
111                event, payload = self.client_events.get(timeout=0.05)
112            except queue.Empty:
113                continue
114            if event == "client_result":
115                found.append(payload)
116        return found
117
118    def test_reads_writes_and_exceptions(self):
119        self.start_client()
120        self.poll(1, 1, 3, 0, 4)
```

MBToolkit v3.5/tests/test_tcp.py — lines 121-160

```
        self.poll(2, 2, 4, 100, 4)
        self.poll(3, 1, 1, 0, 3)
        values = {r["window_id"]: r["values"] for r in self.results(3)}
        self.assertEqual(values, {1: [0, 10, 20, 30], 2: [7, 8, 9, 10], 3: [1, 0, 1]})

        self.client.queue_write(1, build_write_request(1, 16, 2, [222, 333]), "w")
        self.client.queue_write(3, build_write_request(1, 5, 7, [1]), "w")
        writes = [r for r in self.results(2) if r["kind"] == "write"]
        self.assertEqual([r["status"] for r in writes], ["ok", "ok"])
        self.assertEqual(self.store.read(1, "holding", 2, 2), [222, 333])
        self.assertEqual(self.store.read(1, "coils", 7, 1), [1])

        self.poll(4, 1, 3, 50, 1)       # outside the windows -> Exception 02
        self.poll(5, 9, 3, 0, 1)        # Unit ID not simulated -> Exception 0B
        codes = {r["window_id"]: (r["status"], r["exception_code"]) for r in self.results(2)}
        self.assertEqual(codes, {4: ("exception", 2), 5: ("exception", 0x0B)})

    def test_server_events(self):
        self.start_client()
        self.poll(1, 1, 3, 0, 1)
        self.results(1)
        client = wait_event(self.server_events, "server_client")
        self.assertTrue(client["connected"])
        request = wait_event(self.server_events, "server_request")
        self.assertEqual((request["slave"], request["table"], request["address"], request["qty"],
                          request["status"]), (1, "holding", 0, 1, "ok"))

    def test_faults(self):
        self.start_client(timeout_ms=150)
        self.poll(1, 1, 3, 0, 1)
        self.results(1)
        self.store.set_fault(1, validate_fault({"exception": 6}))
        self.client.read_once(1)
        self.assertEqual(self.results(1)[0]["exception_code"], 6)
        self.store.set_fault(1, validate_fault({"bad_crc": True}))
        self.client.read_once(1)
        self.assertEqual(self.results(1)[0]["status"], "invalid")
        self.store.set_fault(1, validate_fault({"no_response": True}))
        self.client.read_once(1)
        self.assertEqual(self.results(1)[0]["status"], "timeout")
```

MBToolkit v3.5/tests/test_tcp.py — lines 161-200

```
161         self.store.set_fault(1, validate_fault({"delay_ms": 300}))
162         self.client.read_once(1)
163         self.assertEqual(self.results(1)[0]["status"], "timeout")
164         time.sleep(0.35)  # the late answer arrives and is skipped by Transaction ID
165         self.store.set_fault(1, None)
166         self.client.read_once(1)
167         self.assertEqual(self.results(1)[0]["status"], "ok")
168 
169     def test_several_clients_at_once(self):
170         sockets = [socket.create_connection(("127.0.0.1", self.port), timeout=1)
171                    for _ in range(3)]
172         try:
173             for index, sock in enumerate(sockets):
174                 sock.sendall(build_adu(index, 1, bytes.fromhex("03 0000 0002")))
175             for index, sock in enumerate(sockets):
176                 answer = sock.recv(64)
177                 self.assertEqual(parse_adu(answer)[0], index)
178                 self.assertEqual(parse_adu(answer)[3], bytes.fromhex("03 04 0000 000A"))
179         finally:
180             for sock in sockets:
181                 sock.close()
182 
183     def test_unreachable_device_and_reconnection(self):
184         self.start_client(timeout_ms=200)
185         free = socket.socket()
186         free.bind(("127.0.0.1", 0))
187         dead_port = free.getsockname()[1]
188         free.close()
189         self.poll(1, 1, 3, 0, 1, port=dead_port)
190         result = self.results(1)[0]
191         self.assertEqual(result["status"], "no_connection")
192         self.assertTrue(result["error"])
193         # The same window pointed at the live server works again.
194         self.poll(1, 1, 3, 0, 1)
195         self.client.read_once(1)
196         self.assertTrue(any(r["status"] == "ok" for r in self.results(3)))
197 
198     def test_port_in_use_is_reported(self):
199         events = queue.Queue()
200         second = ServerEngine(events, self.store, host="127.0.0.1", port=self.port)
```

MBToolkit v3.5/tests/test_tcp.py — lines 201-240

```
        second.start()
        stopped = wait_event(events, "server_stopped")
        self.assertIn("Error", stopped["error"])

    def test_unit_ids_above_247(self):
        definition = ClientPollDefinition(1, 255, 3, 0, 1, 1000)
        self.assertEqual(definition.request()[0], 255)
        self.assertTrue(crc_ok(append_crc(definition.request()[:-2])))


class ClientWindowTests(unittest.TestCase):
    def test_workspace_entry_round_trip_and_validation(self):
        window = ClientWindow(1, 1, slave=255, fc=4, address=10, qty=5,
                              host="192.168.1.50", port=1502, names={10: "P"})
        data = ClientWindow.validated_kwargs(window.to_json())
        self.assertEqual((data["slave"], data["host"], data["port"], data["fc"], data["names"]),
                         (255, "192.168.1.50", 1502, 4, {10: "P"}))
        definition = ClientWindow(2, 2, **data).definition()
        self.assertEqual((definition.host, definition.port, definition.request()[0]),
                         ("192.168.1.50", 1502, 255))
        for bad in ({"slave": 256}, {"host": ""}, {"port": 0}, {"fc": 5}):
            with self.assertRaises(ValueError):
                ClientWindow.validated_kwargs(dict(window.to_json(), **bad))

    def test_write_uses_the_unit_id(self):
        window = ClientWindow(1, 1, slave=1, fc=3, address=0, qty=4)
        tab = ClientTab(object())  # no widgets needed to build a request
        request, _ = tab.master_build_write(window, "250", "2", "7", 6)
        self.assertEqual(request[:6], bytes((250, 6, 0, 2, 0, 7)))
        with self.assertRaises(ValueError):
            tab.master_build_write(window, "256", "2", "7", 6)

    def test_tabs_keep_their_own_settings_and_folders(self):
        self.assertEqual(ClientTab._master_settings_key, "client")
        self.assertEqual(ServerTab._slave_settings_key, "server")
        self.assertEqual(ClientTab._master_ws_folder().name, "Client")
        self.assertEqual(ServerTab._slave_ws_folder().name, "Server")


class TcpSettingsTests(unittest.TestCase):
```

MBToolkit v3.5/tests/test_tcp.py — lines 241-270

```
    def setUp(self):
        self._appdata = os.environ.get("APPDATA")
        self._folder = tempfile.TemporaryDirectory()
        os.environ["APPDATA"] = self._folder.name

    def tearDown(self):
        if self._appdata is None:
            os.environ.pop("APPDATA", None)
        else:
            os.environ["APPDATA"] = self._appdata
        self._folder.cleanup()

    def test_client_and_server_settings_are_validated(self):
        mb_config.save_ui_settings(settings={
            "client": {"response_timeout_ms": "500", "delay_ms": "5", "last_workspace": "Obra"},
            "server": {"bind": "192.168.1.20", "port": "1502", "last_workspace": "Sim"},
            "last_main_tab": "Server"})
        loaded = mb_config.load_ui_settings()
        self.assertEqual(loaded["client"], {"response_timeout_ms": "500", "delay_ms": "5",
                                            "last_workspace": "Obra"})
        self.assertEqual(loaded["server"], {"bind": "192.168.1.20", "port": "1502",
                                            "last_workspace": "Sim"})
        self.assertEqual(loaded["last_main_tab"], "Server")
        mb_config.save_ui_settings(settings={"server": {"bind": "x", "port": "70000"}})
        self.assertEqual(mb_config.load_ui_settings()["server"],
                         {"bind": "127.0.0.1", "port": "502", "last_workspace": ""})


if __name__ == "__main__":
    unittest.main()
```

MBToolkit.bat

210 source lines.

MBToolkit.bat — lines 1-40

```
@echo off
setlocal EnableExtensions
title MBToolkit 3.5
set "ROOT=%~dp0"
set "APPDIR=%~dp0MBToolkit v3.5"
set "SCRIPT=%APPDIR%\MBToolkit.py"
cd /d "%APPDIR%"
set "PYRUN="
set "PYEXE="
set "PYWEXE="

if not exist "%SCRIPT%" (
    echo.
    echo ERRO: os ficheiros do MBToolkit nao foram encontrados em:
    echo   %SCRIPT%
    echo.
    pause
    exit /b 1
)

rem ============================================================
rem 1. Procurar uma instalacao funcional de Python 3
rem ============================================================
call :DETECT_PYTHON

if defined PYRUN goto PYTHON_OK

echo.
echo Python 3 nao foi encontrado.
echo A tentar instalar automaticamente Python 3.13 com WinGet...
echo.

where winget >nul 2>&1
if errorlevel 1 goto NO_WINGET

winget install --id Python.Python.3.13 --exact --scope user --silent ^
    --accept-package-agreements --accept-source-agreements

if errorlevel 1 (
    echo.
```

MBToolkit.bat — lines 41-80

```
    echo ERRO: A instalacao automatica do Python falhou.
    echo O WinGet pode estar bloqueado pela politica da empresa ou sem acesso a Internet.
    echo.
    pause
    exit /b 1
)

rem O instalador pode alterar o PATH sem atualizar esta janela CMD.
rem Por isso voltamos a procurar tambem nos caminhos standard.
call :DETECT_PYTHON

if not defined PYRUN (
    echo.
    echo ERRO: Python foi instalado, mas nao consegui localizar python.exe.
    echo Fecha esta janela e volta a executar MBToolkit.bat.
    echo.
    pause
    exit /b 1
)

:PYTHON_OK
rem ============================================================
rem 2. Descobrir o python.exe real e o respetivo pythonw.exe
rem ============================================================
for /f "usebackq delims=" %%P in (`%PYRUN% -c "import sys; print(sys.executable)" 2^>nul`) do (
    set "PYEXE=%%P"
)

if not defined PYEXE (
    echo.
    echo ERRO: Nao foi possivel determinar a instalacao do Python.
    echo.
    pause
    exit /b 1
)

for %%P in ("%PYEXE%") do set "PYWEXE=%%~dpPpythonw.exe"

rem ============================================================
rem 3. Verificar tkinter
```

MBToolkit.bat — lines 81-120

```
rem ============================================================
%PYRUN% -c "import tkinter" >nul 2>&1
if not errorlevel 1 goto TK_OK

echo.
echo tkinter nao esta disponivel nesta instalacao do Python.
echo A tentar instalar/reparar Python 3.13 com WinGet...
echo.

where winget >nul 2>&1
if errorlevel 1 goto TK_ERROR

winget install --id Python.Python.3.13 --exact --scope user --silent --force ^
    --accept-package-agreements --accept-source-agreements

call :DETECT_PYTHON
if not defined PYRUN goto TK_ERROR

%PYRUN% -c "import tkinter" >nul 2>&1
if errorlevel 1 goto TK_ERROR

for /f "usebackq delims=" %%P in (`%PYRUN% -c "import sys; print(sys.executable)" 2^>nul`) do (
    set "PYEXE=%%P"
)
for %%P in ("%PYEXE%") do set "PYWEXE=%%~dpPpythonw.exe"

:TK_OK
rem ============================================================
rem 4. Verificar pyserial (obrigatorio)
rem ============================================================
%PYRUN% -c "import serial" >nul 2>&1
if not errorlevel 1 goto PYSERIAL_OK

echo.
echo pyserial nao esta instalado.
echo A instalar pyserial...
echo.

%PYRUN% -m pip --version >nul 2>&1
if errorlevel 1 (
```

MBToolkit.bat — lines 121-160

```
121     %PYRUN% -m ensurepip --upgrade
122     if errorlevel 1 goto DEPENDENCIES_ERROR
123 )
124
125 %PYRUN% -m pip install --user pyserial
126 if errorlevel 1 goto DEPENDENCIES_ERROR
127
128 %PYRUN% -c "import serial" >nul 2>&1
129 if errorlevel 1 goto DEPENDENCIES_ERROR
130
131 :PYSERIAL_OK
132 rem ============================================================
133 rem 5. Dependencias verificadas
134 rem ============================================================
135 :DEPENDENCIES_OK
136 rem ============================================================
137 rem 6. Executar o MBToolkit
138 rem ============================================================
139 if exist "%PYWEXE%" (
140     start "" "%PYWEXE%" "%SCRIPT%"
141     exit /b 0
142 )
143
144 rem Fallback caso pythonw.exe nao exista.
145 start "" %PYRUN% "%SCRIPT%"
146 exit /b 0
147
148
149 :DETECT_PYTHON
150 set "PYRUN="
151
152 rem Python Launcher for Windows
153 py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
154 if not errorlevel 1 (
155     set "PYRUN=py -3"
156     exit /b 0
157 )
158
159 rem Python no PATH
160 python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
```

MBToolkit.bat — lines 161-200

```
161 if not errorlevel 1 (
162     set "PYRUN=python"
163     exit /b 0
164 )
165
166 rem Caminhos standard para Python 3.13 instalado por utilizador/WinGet.
167 if exist "%LocalAppData%\Programs\Python\Python313\python.exe" (
168     set PYRUN="%LocalAppData%\Programs\Python\Python313\python.exe"
169     exit /b 0
170 )
171
172 if exist "%ProgramFiles%\Python313\python.exe" (
173     set PYRUN="%ProgramFiles%\Python313\python.exe"
174     exit /b 0
175 )
176
177 if exist "%ProgramFiles(x86)%\Python313\python.exe" (
178     set PYRUN="%ProgramFiles(x86)%\Python313\python.exe"
179     exit /b 0
180 )
181
182 exit /b 0
183
184
185 :NO_WINGET
186 echo.
187 echo ERRO: Python 3 nao esta instalado e o WinGet nao esta disponivel.
188 echo Nao e possivel fazer a instalacao automatica neste PC.
189 echo Instala Python 3 manualmente e volta a executar MBToolkit.bat.
190 echo.
191 pause
192 exit /b 1
193
194
195 :TK_ERROR
196 echo.
197 echo ERRO: tkinter continua indisponivel.
198 echo E necessario usar uma instalacao normal do Python para Windows com Tcl/Tk.
199 echo.
200 pause
```

MBToolkit.bat — lines 201-210

```
exit /b 1


:DEPENDENCIES_ERROR
echo.
echo ERRO: Nao foi possivel instalar pyserial.
echo Verifica acesso a Internet, proxy/firewall e permissoes do Python.
echo.
pause
exit /b 1
```

MBToolkit v3.5/build_exe.bat

151 source lines.

MBToolkit v3.5/build_exe.bat — lines 1-40

```
@echo off
setlocal EnableExtensions
title Build MBToolkit 3.5

rem Normalizar caminhos sem barra final. Isto evita problemas de parsing
rem do Windows/Python quando um argumento quoted termina em "\".
for %%I in ("%~dp0.") do set "APPDIR=%%~fI"
for %%I in ("%APPDIR%\..") do set "DISTDIR=%%~fI"

set "SCRIPT=%APPDIR%\MBToolkit.py"
set "ICON=%APPDIR%\MBToolkit.ico"
set "WORKDIR=%APPDIR%\build"
set "SPECFILE=%APPDIR%\MBToolkit.spec"
set "PYRUN="

cd /d "%APPDIR%"

if not exist "%SCRIPT%" (
    echo.
    echo ERRO: MBToolkit.py nao foi encontrado.
    echo.
    pause
    exit /b 1
)

rem ============================================================
rem 1. Procurar Python 3
rem ============================================================
py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 (
    set "PYRUN=py -3"
    goto PYTHON_OK
)

python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 (
    set "PYRUN=python"
    goto PYTHON_OK
)

```

MBToolkit v3.5/build_exe.bat — lines 41-80

```
41 if exist "%LocalAppData%\Programs\Python\Python313\python.exe" (
42     set PYRUN="%LocalAppData%\Programs\Python\Python313\python.exe"
43     goto PYTHON_OK
44 )
45
46 echo.
47 echo ERRO: Python 3 nao foi encontrado.
48 echo Executa primeiro MBToolkit.bat para instalar/verificar as dependencias.
49 echo.
50 pause
51 exit /b 1
52
53
54 :PYTHON_OK
55 rem ============================================================
56 rem 2. Instalar/atualizar dependencias de build
57 rem ============================================================
58 echo.
59 echo [1/4] A verificar pyserial e PyInstaller...
60 %PYRUN% -m pip install --upgrade pyserial pyinstaller
61 if errorlevel 1 (
62     echo.
63     echo ERRO ao instalar pyserial/PyInstaller.
64     echo.
65     pause
66     exit /b 1
67 )
68
69 rem ============================================================
70 rem 3. Executar testes de regressao antes do build
71 rem ============================================================
72 echo.
73 echo [2/4] A executar testes de regressao...
74 %PYRUN% -m unittest discover -s tests -p "test_*.py" -v
75 if errorlevel 1 (
76     echo.
77     echo ERRO: os testes de regressao falharam. Build cancelado.
78     echo.
79     pause
80     exit /b 1
```

MBToolkit v3.5/build_exe.bat — lines 81-120

```
)

rem ============================================================
rem 4. Limpar builds anteriores
rem ============================================================
echo.
echo [3/4] A limpar build anterior...
if exist "%WORKDIR%" rmdir /s /q "%WORKDIR%"
if exist "%SPECFILE%" del /q "%SPECFILE%"
if exist "%DISTDIR%\MBToolkit.exe" del /q "%DISTDIR%\MBToolkit.exe"

rem ============================================================
rem 5. Criar EXE
rem ============================================================
echo.
echo [4/4] A criar MBToolkit.exe...

if exist "%ICON%" (
    %PYRUN% -m PyInstaller ^
      --noconfirm ^
      --clean ^
      --onefile ^
      --windowed ^
      --name MBToolkit ^
      --distpath "%DISTDIR%" ^
      --workpath "%WORKDIR%" ^
      --icon "%ICON%" ^
      --add-data "%ICON%;." ^
      "%SCRIPT%"
) else (
    echo AVISO: MBToolkit.ico nao foi encontrado.
    %PYRUN% -m PyInstaller ^
      --noconfirm ^
      --clean ^
      --onefile ^
      --windowed ^
      --name MBToolkit ^
      --distpath "%DISTDIR%" ^
      --workpath "%WORKDIR%" ^
      "%SCRIPT%"
```

MBToolkit v3.5/build_exe.bat — lines 121-151

```
)

if errorlevel 1 (
    echo.
    echo ERRO: o build falhou.
    echo.
    pause
    exit /b 1
)

if not exist "%DISTDIR%\MBToolkit.exe" (
    echo.
    echo ERRO: o PyInstaller terminou sem criar:
    echo   %DISTDIR%\MBToolkit.exe
    echo.
    pause
    exit /b 1
)

echo.
echo ============================================================
echo BUILD CONCLUIDO
echo ============================================================
echo.
echo Executavel criado em:
echo   %DISTDIR%\MBToolkit.exe
echo.
echo O icone MBToolkit.ico foi aplicado ao EXE e incluido
echo como recurso para a janela/barra de tarefas.
echo.
pause
```

MBToolkit v3.5/run_tests.bat

45 source lines.

MBToolkit v3.5/run_tests.bat — lines 1-40

```
@echo off
setlocal EnableExtensions
title MBToolkit regression tests

for %%I in ("%~dp0.") do set "APPDIR=%%~fI"
cd /d "%APPDIR%"
set "PYRUN="

py -3 -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
if not errorlevel 1 set "PYRUN=py -3"

if not defined PYRUN (
    python -c "import sys; assert sys.version_info.major == 3" >nul 2>&1
    if not errorlevel 1 set "PYRUN=python"
)

if not defined PYRUN (
    echo.
    echo ERRO: Python 3 nao foi encontrado.
    echo.
    pause
    exit /b 1
)

echo.
echo A executar testes de regressao do MBToolkit...
echo.
%PYRUN% -m unittest discover -s tests -p "test_*.py" -v
if errorlevel 1 (
    echo.
    echo ============================================================
    echo TESTES FALHARAM - build/release nao deve prosseguir.
    echo ============================================================
    echo.
    pause
    exit /b 1
)

echo.
echo ============================================================
```

MBToolkit v3.5/run_tests.bat — lines 41-45

```
41 echo TODOS OS TESTES PASSARAM
42 echo ============================================================
43 echo.
44 pause
45 exit /b 0
```